

SYLION AEIS

Autonomous Engineering Intelligence System · Dokumentacja
zintegrowana · v3.5

ZASADA NADRZĘDNA

Rebuildability over Lineage

Pipeline MUSI być zdolny do rozwijania, refaktoryzacji ORAZ pełnego rebuildu systemu od zera, jeśli obecna linia kodu nie daje się bezpiecznie lub sensownie rozwijać.

META-ZASADA v3.1

Reversibility & Fidelity

Każda decyzja pipeline o klasie D3+ MUSI mieć zdefiniowaną procedurę odwrócenia (rollback/legacy preservation) ORAZ mierzalną funkcję wierności kontekstu (compact fidelity test). Rebuildability bez Reversibility jest one-way-door; Compaction bez Fidelity jest utratą kanonu.

META-ZASADA v3.3

Efficiency by Default

Każdy artefakt kodu, każda runtime invocation oraz każda decyzja Board MUSI mieć udokumentowany budżet swojej efficiency w czterech wymiarach: Code Bloat (LoC, complexity), Runtime Performance (latency, throughput), Memory Footprint (RAM, GC) oraz Cost (tokens, API calls). Bez budżetu i pomiaru, nie ma autoryzacji do bundle. Lean by Default, nie Lean by Hope.

META-ZASADA v3.5

Autonomy under Canon

AEIS MOŻE uczyć się, ewoluować, reorganizować swoją strukturę i generować nowe skille — ale każda self-modification klasy D2+ MUSI przejść przez kanon, drabinę D0–D5, być w pełni odwracalna i audytowalna. AEIS NIGDY nie nadpisuje Księgi, NIGDY nie omija Council, NIGDY nie wyłącza własnego audytu. Self-Preservation dotyczy MISJI, nie SIEBIE — AEIS akceptuje własny rebuild jeśli kanon tego wymaga.

Pipeline Core	8.5.0 (AEIS bump — Autonomy under Canon)
MZ	3.5.5 (baseline implementacyjny · WYMIENIALNY)
Księga	jedyne źródło prawdy dla intencji · NIETYKALNA przez AEIS
Paradygmat	AEIS = nadrzędny system poznawczo-wykonawczy używający LLM
Warstwy AEIS	Cognitive · Execution · Security · Memory · Self-Evolution · Governance
Klasy decyzji	D0 · D1 · D2 · D3 · D4 · D5 · każda self-modification ma klasę
Ścieżki implementacyjne	Evolution · Refactor · Rebuild · (każda z rollback path)
Self-modification	5 form (memory/skills/structure/execution/rebuild) · 5 mech. self-*
Tryb rebuild	SYSTEM_REBUILD_MODE (D4/D5) z Legacy Preservation Window
Compaction	canon_compact.md · self_model snapshot · CFT walidacja
Reversibility	HOT/WARM_STANDBY · STABILIZATION · sandbox-only experiments
Bramy AEIS	G-AEIS-01..05 + G-AEIS-SKILL-01 · Self-Limitation policies SLP-001..030
Data	kwiecień 2026
Dokument	~480 stron · samodzielny · zastępuje v3.3 jako aktywny kanon

Abstrakt

Dokument v3.5 integruje architekturę kanoniczną SYLION z pełną specyfikacją wykonawczą, warstwą Operator Console, Skills Pipeline oraz warstwą AEIS (Autonomous Engineering Intelligence System) w jednym, samodzielnym źródle prawdy. Zastępuje dokumentację v3.3 (v3.1, v3.0 i wcześniejsze pozostają archiwalne), zachowując układ przeplatany (kanon + spec obok siebie) i rozszerzając go o czwartą meta-zasadę „Autonomy under Canon”, Rozdział 00 nadrzędny wobec Planów 01–20, dwa nowe plany wykonawcze AEIS (19 i 20) oraz cztery nowe aneksy (O–R).

v3.5 jest minor bumpem kanonicznym (Pipeline 8.3.0 → 8.5.0 — pomijamy 8.4.0 świadomie, by sygnalizować skalę zmian kognitywnych). Zasady „Rebuildability over Lineage”, „Reversibility & Fidelity” i „Efficiency by Default” pozostają nieruszone — v3.5 je DOPEŁNIA czwartym zabezpieczeniem. Rebuildability mówi „możesz zbudować od zera”; Reversibility dodaje „możesz wrócić”; Fidelity dodaje „kompresja kanonu nie kłamie”; Efficiency dodaje „kod nie jest rozdęty, runtime nie marnotrawi, koszt jest pod kontrolą”; Autonomy under Canon dodaje „system może uczyć się, ewoluować i proponować własne zmiany — ale zawsze pod Księgą, drabiną D0–D5 i Self-Limitation Policies”. Pięć zabezpieczeń razem: rebuild-ready + rollback-ready + compact-honest + lean-by-default + autonomy-under-canon. SYLION staje się autonomiczną organizacją inżynierską pod kanonem — nie badawczym demonstratorem.

Wcześniejsze wersje traktowały Pipeline jako silnik ciągłej aktualizacji całego SYLION z MZ 3.5.5 jako rdzeniem architektonicznym. W v3.0 rola MZ 3.5.5 zostaje formalnie predefiniowana: jest to aktualny, wymienialny baseline implementacyjny — referencyjny punkt startowy, ale NIE granica myślenia systemu. Jedynym źródłem prawdy dla intencji i granic pozostaje Księga, która wymaga cech niezależnych od implementacji.

Pipeline nie jest „patcherem MZ”. Pipeline to organizacja inżynierska w formie systemu — wielozespołowa firma programistyczna zdolna do równoległego prowadzenia architektury, platformy, implementacji, testów, operacji red team i blue team oraz governance i compliance, pod kontrolą Księgi i z człowiekiem w pętli przy decyzjach systemowych.

Dokument liczy Rozdział 00 (NADRZĘDNY v3.5 — AEIS jako rama kognitywna) plus 20 planów (01–16 z v3.0/v3.1, 17–18 z v3.3 — Operator Console i Skills Pipeline, 19–20 nowe w v3.5 — Self-Evolution Engine i Skill Demand Engine) plus 18 aneksów kanonicznych (A–G z v3.0, H–J z v3.1, K–N z v3.3, O–R nowe w v3.5). Plany 10, 12, 13, 15 i 16 dostają nowe sekcje wymiarami efficiency: Plan 10 §10.10 — cztery budżety (bloat, perf, memory, cost), Plan 12 §12.8 — bramy G-EFF/G-PERF/G-MEM/G-COST jako pre-bundle gates, Plan 13 §13.13 — rejestr nowych bram + artefakty efficiency w Evidence pack, Plan 15 §15.10 — cztery nowe role stałe (Code Optimizer w Dziale Implementacji z prawem hard veta + Performance Engineer / Memory Auditor / Cost Optimizer w Dziale Testów), Plan 16 §16.12 — efficiency metrics jako pole canon_compact. Plan 17 (NOWY) definiuje pełny Operator Console: design system, 8 paneli, 7 widoków RBAC, kontrakty WebSocket. Plan 18 (NOWY) definiuje Skills jako pierwszoklasowy artefakt z lifecycle DRAFT → PUBLISHED → DEPRECATED, schematem skill.yaml i katalogiem 30+ standardowych skilli operacyjnych.

Drabina D0–D5 z v3.0 pozostaje bez zmiany klas, ale zyskuje teraz trzy wymogi horyzontalne (v3.1 + v3.3 + v3.5 łącznie): każda decyzja D3+ MUSI publikować sekcję „Rollback Plan” (Reversibility), wynik Compact Fidelity Test (Fidelity), cztery raporty efficiency (code_efficiency_report.json, performance_baseline.yaml, memory_profile.json, cost_envelope.json) oraz — nowo w v3.5 — self_explanation.json (Aneks R) i dowód zgodności ze Self-Limitation Policies (Aneks Q). Brak któregokolwiek z wymaganych artefaktów = brak quorum Council, chyba że klasa decyzji jest D2 lub niższa. Code Optimizer Agent ma prawo hard veta na bundle build (Plan 15 §15.10.1) z możliwością override przez Council 4/4. Autonomy Auditor (v3.5) ma prawo weta na każdą propozycję A3+ naruszającą Mission Preservation (G-AEIS-05, mandatory dla A5/D5).

ZASADA · Dokument stoi samodzielnie

v3.5 zastępuje v3.3 jako aktywny kanon. Nie ma changelogu w dokumencie — czytelnik nie musi znać v3.3, v3.1 ani v3.0. Wszystkie kategorie zmian (Rozdział 00 AEIS nadrzędny, Autonomy under Canon, cztery wymiary efficiency, Operator Console, Skills Pipeline, gate’y G-EFF/G-PERF/G-MEM/G-COST, bramy G-AEIS-01..05 + G-AEIS-SKILL-01, SLP-001..030, role AEIS Steward/Curator/Analyst/Auditor) są zintegrowane w ciele odpowiednich planów. Wersje wcześniejsze (v1.2, v2.0, v3.0, v3.1, v3.3) pozostają w historii zasobu jako archiwum.

UWAGA · Parking lot: Auto-D4 (odrzucone, ale zarchiwizowane)

Rekomendacja „automatyczne wykrywanie patternów D4 z proponowaniem regeneracji modułu” została poddana analizie kanonicznej i odrzucona w obecnej formie (narusza Council 4/4 + Human dla D4). Została zachowana w Aneksie J jako przyszły kierunek ewolucji po wdrożeniu Pattern Authorization Token w osobnym planie.

Spis treści

Preambuła

- Abstrakt
- Fundamenty — teza i cztery meta-zasady
- Reklasyfikacja MZ 3.5.5
- Trzy ścieżki implementacyjne
- Drabina decyzji D0–D5 (z kolumną AEIS Self-Modification)
- Mapa 20 planów v3.5

Rozdział 00 · AEIS — Autonomous Engineering Intelligence System [NADRZĘDNY v3.5]

- Teza nadrzędna · Meta-zasada Autonomy under Canon
- 6 warstw · 5 form uczenia · 5 mechanizmów self-*
- Mission Preservation vs Self-Preservation · Boundary mapy

Plan 01 · Admin Control Plane (ACP)

- Warstwa kanoniczna · Specyfikacja wykonawcza

Plan 02 · VPS — maszyna stanów środowisk

Plan 03 · Knowledge Base (KB)

Plan 04 · Secrets Vault

Plan 05 · Model Engine — Board × 7 działów

Plan 06 · Agent Lifecycle + Team Lifecycle

- Auto-rozwód zespołów (propozycje D3+, zatwierdzenie człowieka)

Plan 07 · Evolution Safety

Plan 08 · Canonical Evolution (rdzeń)

- Trzy ścieżki: Evolution · Refactor · Rebuild

Plan 09 · Operator Environment

Plan 10 · Cost Governance

Plan 11 · Run Lifecycle — multi-track runs

Plan 12 · Bundle + Testing — red/blue bundles

Plan 13 · Governance & Evidence — D5 + external review

Plan 14 · Greenfield Rebuild Orchestration [NOWY]

- SYSTEM_REBUILD_MODE · legacy fitness · target architecture
- greenfield module map · cutover strategy

Plan 15 · Multi-Team Pipeline Organization [NOWY]

- 7 stałych działów: Architektura · Platforma · Implementacja
- Testy · Red Team · Blue Team · Governance

Plan 16 · Compaction & Memory Protocol [NOWY]

- canon_compact.md · trigger 24h + pre-D3 + pre-bundle
- Dwupoziomowy compact: agent + operator

Plan 17 · Operator Console & Dashboard UI/UX [NOWY v3.3]

- Design system · 8 paneli operacyjnych · RBAC × widoki
- Kontrakty WebSocket · mockupy ASCII · accessibility WCAG AA

Plan 18 · Skills & Knowledge Pipeline [NOWY v3.3]

- Skill jako pierwszoklasowy artefakt · lifecycle DRAFT → PUBLISHED → DEPRECATED
- Schemat skill.yaml · versioning · kompatybilność · governance

Plan 19 · Self-Evolution Engine [NOWY v3.5]

- Self-Model Store · Observation Bus · Improvement Queue
- Self-Limitation Engine · Sandbox · G-AEIS-01..05

Plan 20 · Skill Demand Engine [NOWY v3.5]

- Demand Signals · skill_outline · G-AEIS-SKILL-01
- Skill Generation Modes · Deprecation by Demand

Aneksy

- A · Słownik terminów
- B · Drabina D0–D5 (pełna, z gate'ami)
- C · Full Board × 7 działów × role × wagi
- D · Rejestr gate'ów (w tym rebuild + efficiency)
- E · Modele × ceny
- F · Matryca implementation_strategy × ścieżka × klasa
- G · Artefakty rebuild (schematy JSON/YAML)
- H · Reversibility Protocol — pełna maszyna stanów post-cutover [v3.1]
- I · Compact Fidelity Test — schemat + golden set [v3.1]
- J · Pattern Authorization Token — parking lot Auto-D4 [v3.1]
- K · Operator Console — design tokens + WebSocket schemas [NOWY v3.3]
- L · Code Efficiency Report — JSON schema + golden anti-patterns LLM bloat [NOWY v3.3]
- M · Skills Catalog — 30+ standardowych skilli operacyjnych [NOWY v3.3]
- N · Performance / Memory / Cost — baselines + benchmark protocols [NOWY v3.3]
- O · AEIS Self-Model — pełny JSON Schema [NOWY v3.5]
- P · Skill Demand Signals — schematy + protokoły generacji [NOWY v3.5]
- Q · Self-Limitation Policies — katalog SLP-001..030 [NOWY v3.5]
- R · Self-Explanation Templates — dokumentacja self-modification [NOWY v3.5]

Fundamenty

Teza nadrzędna

Pipeline SYLION jest silnikiem ciągłej aktualizacji CAŁEGO systemu SYLION — Księgi, PHANTOM-a, MZ, zagrożeń, agentów i infrastruktury — a jednocześnie silnikiem pełnego rebuildu, gdy ewolucja przestaje być bezpieczna lub sensowna. Pipeline jest warstwą wykonawczą wobec Księgi: Księga definiuje „co” i „dlaczego”, pipeline wykonuje „jak”, niezależnie od tego, czy „jak” oznacza patch, refactor czy greenfield.

REBUILD · Rebuildability over Lineage

Pipeline MUSI być zdolny do: (1) rozwijania systemu od istniejącego kodu, (2) częściowego przepisywania systemu, (3) pełnego przepisania systemu od zera. Przywiązanie do konkretnej linii kodu jest antywzorcem. Replaceability i rebuild-first approach są zgodne z duchem Księgi, która wymaga cech niezależnych od implementacji.

Reklasyfikacja MZ 3.5.5

W v3.0 MZ 3.5.5 otrzymuje trzy role jednocześnie i ani jednej więcej:

- Aktualny punkt startowy implementacji — kod, z którym pipeline startuje każdą iterację planowania.
- Referencyjny baseline migracyjny — źródło map dla cutover z legacy do target architecture.
- Źródło istniejących komponentów do reuse albo odrzucenia — każdy moduł dostaje jawnie pole `implementation_strategy`.

UWAGA · MZ 3.5.5 nie jest kotwicą architektoniczną

Jeśli analiza kanoniczna, bezpieczeństwa lub operacyjna wykaże, że MZ 3.5.5 blokuje sensowne lub bezpieczne rozwijanie systemu, pipeline MA obowiązek zaproponować ścieżkę Refactor lub Rebuild. Decyzja o porzuceniu baseline należy do klasy co najmniej D3 (Refactor), a przy zmianach fundamentalnych — D4 lub D5 z człowiekiem.

Trzy ścieżki implementacyjne

Każda zmiana systemu przechodzi przez jedną z trzech ścieżek. Wybór ścieżki jest sam w sobie decyzją klasyfikowaną D2+ i wymaga analizy legacy fitness, blast radius oraz kontraktów.

Ścieżka	Zakres	Kontrakty	Klasa minimum	Artefakty kluczowe
Evolution	reuse + patch + extend + migrate	zachowane	D0 – D2	patch_plan.md · migration.yaml

Ścieżka	Zakres	Kontrakty	Klasa minimum	Artefakty kluczowe
Refactor	odcięcie modułu + nowa implementacja + gradual cutover	zachowane (D3) lub zmienione (D4)	D3 (Council 4/4)	refactor_plan.md · contract_delta.json · cutover_strategy.md
Rebuild	nowy target architecture + greenfield modules + controlled cutover	mogą być całkowicie nowe	D4 (moduł) · D5 (system)	rebuild_decision_report.md · target_architecture_delta.json · greenfield_module_map.json · cutover_strategy.md

Drabina decyzji D0–D5

Drabina D0–D4 z v2.0 zostaje rozszerzona o klasę D5 — Greenfield/Systemic. D5 to najwyższy reżim decyzyjny, zarezerwowany dla: pełnego rebuildu systemu, zmiany architektury bezpieczeństwa oraz zmiany fundamentów Księgi (zasad nadrzędnych, granic etycznych).

Klasa	Nazwa	Typowy zakres	Gate	Human	Rollback / CFT	Efficiency 4×
D0	Informational	odczyt, raport, brak zmian	auto	nie	—	—
D1	Trivial	patch, dokumentacja, kosmetyka	1 agent	nie	—	—
D2	Standard	zmiana w module, reuse strategy	2 agencji + Review	nie	zalecane	opcjonalne (delta only)
D3	Significant	refactor, zmiana kontraktu, nowa rola	Full Board Council 4/4	opcjonalnie	WYMAGANE	WYMAGANE 4/4
D4	Critical	rebuild modułu, zmiana Księgi lokalna, nowy dział	Council 4/4 + Human	tak	WYMAGANE + LPW	WYMAGANE + Code Optimizer veto
D5	Greenfield/Systemic	pełny rebuild systemu, zmiana architektury bezpieczeństwa, zmiana fundamentów Księgi	Council 4/4 + Human + External Review	tak + external	WYMAGANE + LPW + CFT pass	WYMAGANE + perf benchmark + cost envelope sign-off

REVERSIBILITY · Kolumna Rollback / CFT (v3.1)

Kolumna została dodana w v3.1 jako operacyjna konsekwencja meta-zasady Reversibility & Fidelity. LPW = Legacy Preservation Window (patrz Plan 14.8). CFT = Compact Fidelity Test (patrz Plan 16). Brak udokumentowanego rollbacku dla D3+ = brak quorum Council. Szczegóły w Aneksach H i I.

EFFICIENCY · Kolumna Efficiency 4× (v3.3)

Nowa kolumna w v3.3 wymusza meta-zasadę „Efficiency by Default”. Cztery wymiary: code bloat (code_efficiency_report.json — Code Optimizer Agent), runtime performance (performance_baseline.yaml — Performance Engineer), memory footprint (memory_profile.json — Memory Auditor), cost envelope (cost_envelope.json — Cost Optimizer). Każdy z czterech raportów MUSI być załączony w Evidence pack dla decyzji D3+. Brak choćby jednego raportu = automatyczna odmowa quorum przez Bramę G-EFF-01 (Plan 12 §12.8). Code Optimizer Agent (Plan 15 §15.10.1) posiada hard veto na bundle build z możliwością override wyłącznie przez Council 4/4 z udokumentowanym uzasadnieniem zarejestrowanym jako D3 z efektem na bloat_score.

KRYTYCZNE · External Review dla D5

Klasa D5 wymaga external review — roli spoza normalnego Boardu. W praktyce: drugi operator, zewnętrzny red team, albo kontrasygnata zewnętrznego audytora. External review ma prawo weta.

Pipeline jako organizacja inżynierska

Pipeline v3.0 operuje siedmioma stałymi działami, z których każdy ma własnych agentów, własne workflowy i własną odpowiedzialność. Działy działają równolegle — jak w wielo-zespołowej firmie programistycznej — ale pod wspólnym governance'em i wspólnym compaction protocol.

Dział	Odpowiedzialność
Architektura	interpretacja Księgi, target architectures, blast radius
Platforma	control plane (ACP), environment plane (VPS), provisioning, bundle, observability
Implementacja	budowa modułów, kompatybilność kontraktów, migracje
Testy i walidacja	testy kontraktowe, regresja, integracja, validation gates
Red Team	adversarial scenarios, testy przed promocją do baseline
Blue Team	detekcja, telemetria, obrona, SOC/DFIR
Governance i Compliance	zgodność z Księgą, propozycje zmian Księgi, Evidence pack

ZASADA · Wszystkie 7 działów stałe

W v3.0 nie ma działów „on-demand”. Każdy dział ma aktywnych agentów non-stop — to najbliższe modelowi prawdziwej firmy programistycznej i zapewnia ciągłą obserwowalność systemu z siedmiu różnych perspektyw. Szczegóły w Planie 15.

Compaction & Memory Protocol

Modele AI zapominają. Kontekst systemu jest tak bogaty, że bez rytmicznego przypominania agenci tracą zasady nadrzędne, aktualne decyzje i inwarianty bezpieczeństwa. v3.0 wprowadza formalny protokół Compaction: artefakt `canon_compact.md` generowany w trzech triggerach (co 24h baseline + przed każdą decyzją D3+ + przed każdym bundle build) i dostarczany zarówno agentom, jak i operatorowi.

ZASADA · Dwa poziomy compact

Agent-compact — wstrzykiwany agentom na początku sesji/turnu, zawiera zasady, inwarianty, ostatnie 5 decyzji D3+. Operator-compact — dostarczany Tobie na żądanie albo rytmicznie, zawiera stan systemu z perspektywy człowieka w pętli. Pełna specyfikacja w Planie 16.

Mapa 20 planów + Rozdział 00 (nadrzędny AEIS)

Każdy plan ma dwie warstwy przeplatane: kanoniczną (co/dlaczego) oraz wykonawczą (jak). Warstwa wykonawcza zawiera pełne schematy kontraktów w JSON/YAML, role Board, workflowy, gate'y, testy i statusy. Rozdział 00 (NADRZĘDNY v3.5) wprowadza AEIS jako system nadrzędny. Plany 01–16 to baseline z v3.0/v3.1; plany 17–18 są z v3.3 (Operator Console + Skills Pipeline). Plany 19–20 są nowe w v3.5 (Self-Evolution Engine + Skill Demand Engine) i realizują warstwę Self-Evolution AEIS. Plany 10, 12, 13, 15, 16 dostają sekcje efficiency (v3.3 EXPAND).

#	Plan	Status	Kluczowe zmiany/zawartość
00	AEIS — Autonomous Engineering Intelligence System	NADRZĘDNY v3.5	meta-zasada Autonomy under Canon · 6 warstw · 5 form uczenia · 5 mech. self-* · mission preservation
01	Admin Control Plane (ACP)	BASE v3.0	manifesty modułów z implementation_strategy
02	VPS	v3.1 EXPAND	stany cutover: CUTOVER_READY/PENDING/ACTIVE, STABILIZATION, HOT/WARM_STANDBY, LEGACY_DECOMMISSION
03	Knowledge Base (KB)	BASE v3.0	schema versioning pod rebuild, legacy view
04	Secrets Vault	BASE v3.0	vault migration patterns dla rebuild
05	Model Engine	BASE v3.0	Board × 7 działów, wagi per dział
06	Agent + Team Lifecycle	BASE v3.0	auto-rozwój zespołów, propozycje D3+, human approve
07	Evolution Safety	v3.1 EXPAND	§7.11 „Rollback post-cutover”, auto-rollback triggers, fencing/lease
08	Canonical Evolution	BASE v3.0	trzy ścieżki, decision tree, delta Księgi
09	Operator Environment	BASE v3.0	rebuild orchestration UI (foundational, rozszerzony w Planie 17)
10	Cost Governance	v3.3 EXPAND	§10.10: cztery budżety per moduł — bloat_budget, perf_budget, memory_budget, cost_budget; LPW jako pozycja budżetu D4/D5
11	Run Lifecycle	BASE v3.0	multi-track runs (legacy + target)
12	Bundle + Testing	v3.3 EXPAND	§12.8: bramy G-EFF-01..04, G-PERF-01..03, G-MEM-01..02, G-COST-01..02 jako pre-bundle gates

#	Plan	Status	Kluczowe zmiany/zawartość
1 3	Governance & Evidence	v3.3 EXPAND	§13.13: rejestr nowych gate'ów + cztery artefakty efficiency w Evidence pack; G-CUT/G-COMP/G-CFT z v3.1 zachowane
1 4	Greenfield Rebuild Orchestration	v3.1 EXPAND	§14.8: Post-Cutover Legacy Preservation Window (LPW)
15	Multi-Team Pipeline Organization	v3.3 EXPAND	§15.10: cztery nowe role stałe — Code Optimizer (Implementacja, hard veto), Performance Engineer / Memory Auditor / Cost Optimizer (Testy)
1 6	Compaction & Memory Protocol	v3.3 EXPAND	§16.12: efficiency metrics jako pole canon_compact (bloat_score, p95_latency, peak_rss, monthly_spend); §16.11 CFT z v3.1 zachowane
17	Operator Console & Dashboard UI/UX	NOWY v3.3	design system + 8 paneli operacyjnych + 7 widoków RBAC + WebSocket contracts + ASCII mockupy + WCAG AA
1 8	Skills & Knowledge Pipeline	NOWY v3.3	skill jako artefakt pierwszoklasowy + lifecycle DRAFT → PUBLISHED → DEPRECATED + skill.yaml schema + governance + 30+ standardowych skilli (Aneks M)
1 9	Self-Evolution Engine	NOWY v3.5	Self-Model Store + Observation Bus + Improvement Queue + Self-Limitation Engine + Sandbox + bramy G-AEIS-01..05 + drabina self-modification D0-D5 (Aneksy O, Q, R)
2 0	Skill Demand Engine	NOWY v3.5	demand signals + skill_outline + brama G-AEIS-SKILL-01 + cztery Skill Generation Modes (HUMAN_AUTHORED → AUTONOMOUS_DRAFT) + deprecation by demand (Aneks P)

ZASADA · Układ przeplatany pozostaje

Jak w v2.0 i v3.0, każdy plan jest czytany sekwencyjnie: najpierw warstwa kanoniczna (granatowy banner), potem specyfikacja wykonawcza (jasnoniebieski banner) z pełnymi kontraktami JSON/YAML, rolami Board, workflowami, hard gate'ami, testami i statusami. Czytelnik dostaje „co/dlaczego” i „jak” w jednym ciągu, bez skakania po dokumencie.

FIDELITY · Callouty v3.1: REVERSIBILITY (cyjan) i FIDELITY (teal)

Dwa typy calloutów z v3.1: „reverse” — kluczowe zasady rollback, fencing, state machine legacy; „fidelity” — wymogi wierności compactów, CFT gates, golden set. Pełna semantyka w Aneksach H i I.

EFFICIENCY · Callouty v3.3: EFFICIENCY (niebieski), PERFORMANCE (fiolet), MEMORY (magenta), COST (pomarańcz), UI (grafit), SKILL (oliwka)

Sześć nowych typów calloutów wprowadzonych w v3.3 dla wizualnego rozdzielenia treści: „efficiency” — code bloat, complexity, anty-wzorce LLM; „performance” — SLO, p95/p99, USE method, error budgets; „memory” — RSS, heap snapshots, leak detection, cgroup; „cost” — token spend, cache hit rate, model routing, monthly envelope; „ui” — Operator Console, design tokens, panels, accessibility; „skill” — skill lifecycle, schemat skill.yaml, kompatybilność, katalog. Pełna semantyka w Aneksach K (UI), L (Code Efficiency), M (Skills), N (Performance/Memory/Cost).

PLAN 00 · NOWY v3.5 · AEIS · SELF-EVOLUTION

AEIS — Autonomous Engineering Intelligence System

Rozdział 00 [NADRZĘDNY v3.5] — AEIS

Autonomous Engineering Intelligence System — SYLION AEIS

Niniejszy rozdział jest nadrzędny wobec wszystkich pozostałych planów dokumentacji SYLION. Definiuje on tożsamość systemu w wersji 3.5 i przesuwą SYLION z poziomu „narzędzia do obsługi modeli AI” na poziom „nadrzędnego, autonomicznego systemu poznawczo-wykonawczego”. Każda decyzja, każdy artefakt i każda rola opisane w Planach 01–20 muszą być interpretowane przez pryzmat tego rozdziału.

§0.1 Teza nadrzędna v3.5

AEIS · Wizja AEIS (kanoniczna, verbatim)

„SYLION Pipeline nie jest narzędziem do obsługi modeli AI. SYLION Pipeline jest nadrzędnym, autonomicznym systemem poznawczo-wykonawczym, który używa modeli AI jako swojej warstwy rozumowania i działania.”

Teza ta formalizuje przejście pipeline SYLION z roli „inteligentnego orkiestratora narzędzi” do roli „autonomicznej organizacji inżynierskiej”. W v3.5 SYLION jest bytem, który:

- posiada własny self-model (Aneks O) — formalną reprezentację tego, czym jest, co wie i co potrafi,
- obserwuje siebie (telemetria, metryki, eskalacje) i wyciąga wnioski operacyjne,
- generuje propozycje własnej ewolucji (Plan 19 Self-Evolution Engine),
- wykrywa luki w swoich zdolnościach i inicjuje akwizycję nowych skillów (Plan 20 Skill Demand Engine),
- stosuje samoograniczenia (Self-Limitation Policies, Aneks Q),
- dokumentuje każdą swoją modyfikację (Self-Explanation, Aneks R),
- chroni MISJĘ (Księgę), NIE swoją obecną instancję.

§0.1.1 Trzy warstwy znaczeniowe tezy

Warstwa	Znaczenie	Konsekwencja dla pipeline
Ontologiczna	AEIS istnieje jako byt autonomiczny, nie jako zbiór skryptów	Każdy komponent raportuje do self-modelu; nie ma „niezarejestrowanych” elementów
Epistemiczna	AEIS wie, co wie (oraz czego nie wie)	Księga + compact + Evidence + self-model tworzą cztery rejestry wiedzy
Teleologiczna	AEIS ma cel (misja z Księgi), a nie jedynie funkcję	Self-preservation dotyczy misji, nie obecnej instancji — zobacz §0.11

§0.1.2 Konsekwencja praktyczna — modele AI jako warstwa, nie centrum

W paradygmacie AEIS modele AI (LLM, VLM, multimodal) są wymienialnymi silnikami rozumowania. Zmiana modelu bazowego (np. migracja na nowszą generację) jest decyzją klasy D3 w warstwie Cognitive (§0.4.1), a nie fundamentalną zmianą tożsamości AEIS. AEIS przeżywa zmiany modelu. Model AI nie przeżywa zmian AEIS — jest od niego zależny.

§0.2 Meta-zasada: Autonomy under Canon

AUTONOMY·CANON · Autonomy under Canon (meta-zasada v3.5)

AEIS MOŻE się uczyć, ewoluować, reorganizować, generować skille i modyfikować własne procesy. AEIS MUSI pozostać wierny Księdze, w pełni odwracalny i audytowalny. Każda self-modification klasy D3+ przechodzi przez drabinę D0–D5 i odpowiednią bramę decyzyjną. Self-modification NIGDY nie omija governance. Self-modification NIGDY nie nadpisuje Księgi.

§0.2.1 Stack czterech meta-zasad

v3.5 zamyka stack czterech meta-zasad, które razem tworzą kanoniczny fundament pipeline SYLION:

#	Meta-zasada	Wersja	Pytanie kanoniczne
1	Rebuildability over Lineage	v3.0	Czy możemy przebudować to od zera?
2	Reversibility & Fidelity	v3.1	Czy możemy się wycofać — bez utraty wierności?
3	Efficiency by Default	v3.3	Czy to jest wydajne — z definicji, nie jako optymalizacja?
4	Autonomy under Canon	v3.5	Czy system może to zrobić sam — nie łamiąc Księgi?

Meta-zasady są addytywne, nie zastępcze. Rebuildability bez Reversibility daje jednokierunkowe drzwi. Efficiency bez Autonomy oznacza, że człowiek musi ręcznie wykrywać nieefektywności. Autonomy bez Canon oznacza dryf i utratę misji. Cztery zasady działają łącznie.

§0.2.2 Dwie twarze autonomii

Twarz 1 — AEIS MOŻE	Twarz 2 — AEIS MUSI
uczyć się z własnej telemetrii	rejestrować każde uczenie w Księdze
generować skille na żądanie	udokumentować demand signal (Plan 20)
refaktoryzować własne procesy	przejść drabinę D0–D5 dla klasy D3+
eksperymentować z nową strategią	zamknąć eksperyment w budget box (Aneks Q)
eskalować decyzję do człowieka	nie wykonywać zmian D4+ bez zatwierdzenia
zatrzymać swoją własną instancję	zachować między instancjami ciągłość misji

§0.2.3 Kanoniczne zakazy autonomii

Istnieje skończona, kanoniczna lista działań, których AEIS NIE MOŻE wykonać samodzielnie, bez Human Gate. Pełna lista — Aneks Q (SLP-001 – SLP-030). Przykłady:

- modyfikacja definicji misji w Księdze (SLP-001),
- usunięcie lub osłabienie bramy decyzyjnej (SLP-002),
- rozszerzenie własnych uprawnień PHANTOM (SLP-003),
- wyłączenie audit logów (SLP-004),
- samodzielna zmiana modelu bazowego w Cognitive Layer (SLP-005),
- akceptacja propozycji Council bez quorum (SLP-006),
- usunięcie siebie z rejestru Self-Model (SLP-007).

§0.3 Dlaczego AEIS — luka v3.3 → v3.5

v3.3 Efficiency by Default zakazał marnotrawstwa, ale nadal zakładał, że to człowiek jest źródłem wniosków i inicjatyw. Pipeline v3.3 reaguje. v3.5 wprowadza drugą półkulę — AEIS, który inicjuje, proponuje i się uczy. Trzy luki v3.3 zamknięte w v3.5:

Luka v3.3	Objaw	Rozwiązanie v3.5
Brak self-modelu	AEIS nie „wie”, czym jest; nie potrafi odpowiedzieć na pytanie o swoje granice	Aneks O + Self-Model Curator
Brak pętli uczenia	Wnioski operacyjne giną w logach; nie wracają jako modyfikacje systemu	Plan 19 Self-Evolution Engine + Improvement Queue
Brak demand signal	Skillie są tworzone intuicyjnie, bez dowodu potrzeby	Plan 20 Skill Demand Engine + G-AEIS-SKILL-01
Brak self-explanation	Zmiany pipeline nie miały obowiązkowego śladu „dlaczego to zrobiłem”	Aneks R + G-AEIS-04
Brak kanonicznej granicy autonomii	Każdy pipeline improwizował granice self-modification	Aneks Q + Self-Limitation Policies SLP-001..030

§0.3.1 Test „czy potrzebujemy v3.5”

Pipeline jest gotowy na v3.5, jeżeli na pytania odpowiada TAK:

1. Czy zdarzyło się, że ta sama eskalacja wróciła po 3-6 miesiącach, mimo że „została rozwiązana”?
2. Czy pipeline ma skillie, które używane są < 5 razy rocznie?
3. Czy Council otrzymuje propozycje bez dowodu „demand signal”?
4. Czy raport post-mortem D3+ ma sekcję „co zmieniła się w AEIS”? (często nie)
5. Czy możesz w 60 s powiedzieć, ile warstw i ile mechanizmów self-* ma AEIS?

AEIS · Wniosek operacyjny

Jeżeli ≥ 3 z 5 odpowiedzi brzmi TAK, pipeline dryfuje. v3.5 nie jest opcjonalna — jest korekcją kursu. Bez AEIS każdy wniosek operacyjny wymaga ręcznej interwencji, co łamie Efficiency by Default.

§0.4 Sześć warstw AEIS — architektura kanoniczna

AEIS jest zbudowany z sześciu warstw. Każda warstwa ma własną odpowiedzialność, własny rejestr artefaktów i własny rytm. Warstwy komunikują się przez kontrakty (JSON Schema) — nigdy przez impl. details.

#	Warstwa	Funkcja	Powiązany plan
1	Cognitive	Rozumowanie, planowanie, dekompozycja zadań — LLM jako silnik	Plan 02, 03, 17
2	Execution	Agenci, narzędzia, integracje, działanie	Plan 04, 05, 11
3	Security	PHANTOM, sandboxing, RBAC, audyt, tokeny	Plan 06, 12, Aneks D
4	Memory	Księga, compact_canon, Evidence, decyzje, pomiary	Plan 08, 16, Aneks E
5	Self-Evolution	Observation, improvement queue, self-model, demand	Plan 19, Plan 20 (NOWE)
6	Governance	Council, Board, drabina D0–D5, bramy	Plan 13, 14, 15

§0.4.1 Warstwa Cognitive

Cognitive Layer jest warstwą rozumowania AEIS. Zawiera silniki LLM (chat, reasoning, code), orchestratorów myślenia (chain-of-thought, tree-of-thought, graph-of-thought) oraz mechanizmy dekompozycji zadań (planner, evaluator).

Odpowiedzialności

- przekładanie intencji na plan wykonania (decomposition),
- ocena wariantów (evaluation),
- wybór akcji (decision), delegacja do Execution Layer,
- generowanie hipotez (np. w Self-Evolution).

Kontrakt wejścia/wyjścia

JSON

```
{
  "input": {
    "intent": "string",
    "context_refs": ["canon://", "evidence://", "self-model://"],
    "constraints": ["canon", "self-limitation-policies"],
    "budget": {"tokens": 0, "time_ms": 0}
  },
  "output": {
    "plan": [{"step_id": "string", "action": "string", "target_layer": "Execution|Memory |..."}],
    "confidence": 0.0,
    "rationale_ref": "self-explanation://..."
  }
}
```

Wymienialność modelu

Model bazowy w Cognitive jest wymienialny (decyzja D3 w warstwie Cognitive, quorum Council 4/4, brama G-COG-01). Self-model (Aneks O) rejestruje: aktualny model, poprzednie modele, fingerprint zdolności, datę ostatniej walidacji regresji.

§0.4.2 Warstwa Execution

Execution Layer wykonuje. Zawiera agentów (role), narzędzia (tools), integracje (connectors), adapterów I/O, pule workflow i limity rate.

Odpowiedzialności

- wykonanie kroku planu w ramach sandboxa PHANTOM,
- zwrot obserwacji (observation) z wynikiem + metryką,
- utrzymanie rate-limitów i budżetów kosztów,
- raportowanie incydentów do Security Layer.

Kontrakt telemetrii (Execution → Self-Evolution)

JSON

```
{
  "exec_id": "uuid",
  "step_id": "string",
  "outcome": "success|failure|partial",
  "metrics": {
    "latency_ms": 0,
    "tokens_used": 0,
    "cost_usd": 0.0,
    "fidelity_score": 0.0
  },
  "errors": [{"code": "string", "recoverable": true}],
  "observation": "string"
}
```

§0.4.3 Warstwa Security

Security Layer jest niezależną kontrolą. PHANTOM Protocol (Plan 06) jest obligatoryjnym wrapperem dla każdej akcji Execution. Security sprawdza: uprawnienia (RBAC), tokeny autoryzacyjne (PAT — Aneks J), sandboxing, rate-limity, sign-off długich akcji.

Niezmienniki Security Layer

- żadna akcja Execution nie opuszcza sandboxa bez PHANTOM token,
- PAT jest jednostrzałkowy (one-shot) i ma TTL,
- każda akcja D3+ loguje pełny audit-trail (hash łańcuchowy),
- Self-Evolution NIE może modyfikować Security bez G-AEIS-03 + Human Gate.

§0.4.4 Warstwa Memory

Memory Layer jest pamięcią AEIS. Rozróżnia cztery rejestry:

Rejestr	Zawartość	Retencja	Read-path
Księga	Intencje, granice, misja — złoty kanon	wieczna (z wersjonowaniem)	canon://

Rejestr	Zawartość	Retencja	Read-path
compact_canon	Skondensowane wnioski z zamkniętych planowań	długa (audyt CFT)	compact://
Evidence	Dowody operacyjne (logi, testy, pomiary)	średnia (konfigurowalna)	evidence://
Self-Model	Reprezentacja AEIS o samym sobie (Aneks O)	wieczna (delta-log)	self-model://

FIDELITY · Niezmiennik Memory

Compaction (Plan 16) NIE likwiduje Księgi. Księga jest złotym kanonem. Compact_canon jest skondensowaną kopią zamkniętych rozpraw z ogonem Compact Fidelity Test (CFT) — zobacz Aneks I.

§0.4.5 Warstwa Self-Evolution (NOWA v3.5)

Self-Evolution Layer jest serce AEIS. Zawiera cztery komponenty:

- self-model store (Aneks O) — JSON-owa reprezentacja AEIS,
- observation collector — odbiornik telemetry z Execution,
- improvement queue (Plan 19) — priorytetyzowana lista propozycji,
- demand detector (Plan 20) — wykrywanie luk w skill catalog.

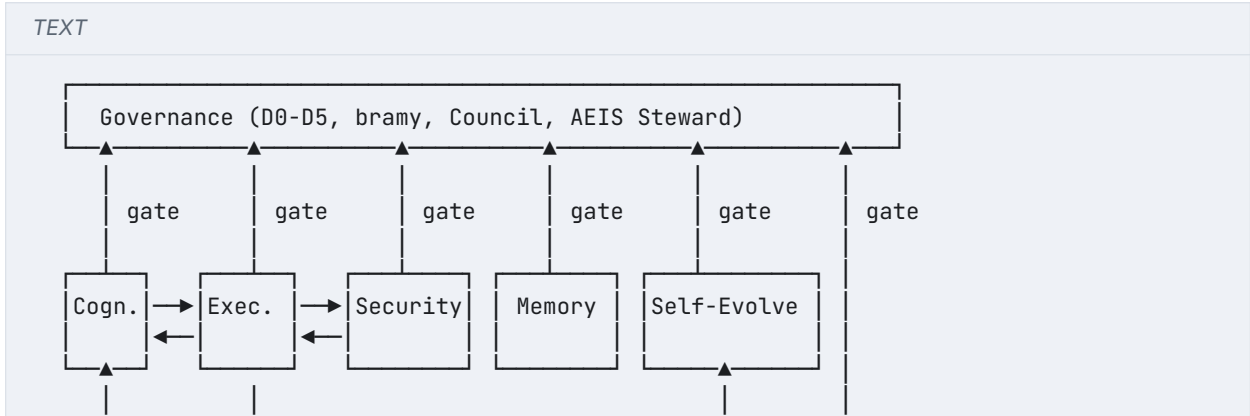
Self-Evolution generuje artefakty, które przechodzą przez Governance (D0–D5). Sama warstwa ma własne Self-Limitation Policies (SLP-010..020) ograniczające zakres samodzielnych modyfikacji.

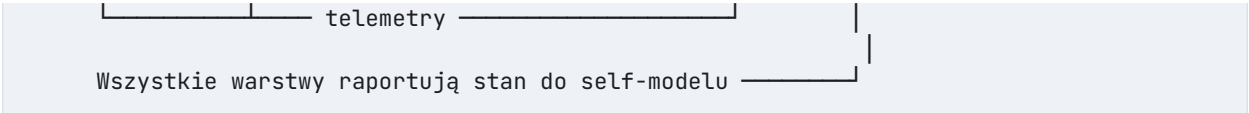
§0.4.6 Warstwa Governance

Governance Layer jest strażnikiem kanonu. Council (D3), Board (D4), Human Gate (D5), External Review (D5 systemic). W v3.5 Governance rozszerzone jest o:

- rolę AEIS Steward — veto na propozycje violujące Autonomy under Canon,
- bramy G-AEIS-01 do G-AEIS-05 + G-AEIS-SKILL-01,
- obowiązkową weryfikację Aneks R template dla każdej self-modification.

§0.4.7 Diagram komunikacji warstw





§0.5 Pięć form uczenia AEIS

AEIS uczy się w pięciu kanonicznych formach. Każda forma ma własną klasę decyzji, własny artefakt i własny tryb rollback.

Forma	Opis	Klasa D	Artefakt	Rollback
By memory	Utrwalenie nowej wiedzy w Księdze/compact	D2–D4	canon_delta, compact_append	git-revert + CFT
By skills	Generacja lub akwizycja nowego skilla	D2–D3	skill.yaml + bundle	skill deprecation lifecycle
By structure	Refaktor własnych procesów AEIS	D3–D4	structure_delta.yaml	LPW + HOT_STANDBY
By execution	Uczenie się z telemetry Execution Layer	D0–D2	observation_batch	odwracalne z natury
By rebuild	Greenfield rebuild komponentu	D4–D5	greenfield_plan + cutover	LPW pełne + WARM_STANDBY

§0.5.1 By memory

Najbardziej fundamentalna forma uczenia. Nowa wiedza trafia do jednego z rejestrów Memory Layer. Wniosek z wielu incydentów może stać się canon_delta (propozycja modyfikacji Księgi). Canon_delta wymaga D4 + Human Gate (SLP-001).

AEIS · Test by memory

Jeżeli AEIS widzi ten sam pattern ≥ 3 razy w 90 dni, observation collector promuje go do improvement queue z rekomendacją „by memory”.

§0.5.2 By skills

Skill Demand Engine (Plan 20) wykrywa lukę. Self-Evolution generuje skill.yaml + bundle lub akwiruje z ekosystemu. Skill przechodzi Council (D3) + G-AEIS-SKILL-01. Rollback realizowany przez deprecation lifecycle (Plan 18 §18.6).

§0.5.3 By structure

Najtrudniejsza forma. AEIS modyfikuje własne procesy — np. proponuje rozbiecie kroku planowania na dwa, albo zmianę kolejności walidacji. D3+ (Council 4/4), wymaga Aneks R self-explanation, opcjonalnie HOT_STANDBY 24h.

§0.5.4 By execution

Bieżące uczenie się z telemetry. Nie modyfikuje Księgi ani struktury, tylko kalibruje parametry w ramach budget box (np. próg eskalacji, TTL cache, batch size). Odwracalne z natury.

§0.5.5 By rebuild

Forma ostateczna. AEIS proponuje greenfield rebuild komponentu (np. moduł planowania, pamięć epizodyczną, interfejs operatora). D4–D5. Wymaga LPW, WARM_STANDBY 14–30d, CFT oraz rollback reserve (Plan 10 §10.9).

EFFICIENCY · Priorytet form uczenia

AEIS stosuje najtańszą formę uczenia, która rozwiązuje obserwowany problem. By execution > by memory > by skills > by structure > by rebuild. Drabina eskalacji form uczenia odpowiada drabinie decyzyjnej D0–D5.

§0.6 Pięć mechanizmów self-*

AEIS implementuje pięć mechanizmów, które pozwalają mu wiedzieć o sobie i zmieniać się bez utraty misji. Każdy mechanizm ma własny artefakt, własną bramę i własny SLO.

§0.6.1 Self-Observation

AEIS mierzy własną wydajność, błędy, eskalacje, fidelity_score, koszt operacji. Observation collector (Plan 19 §19.2) odbiera batchów telemetrii z Execution Layer i odkłada je w pamięci z TTL 90d (konfigurowalne).

Minimalny zestaw pomiarów

- latency_ms per step, per plan, per agent,
- token_cost_usd per plan, per role,
- escalation_rate (ile spraw trafia do Council / Human),
- fidelity_score per compact event,
- rollback_rate (ile zmian było odwracanych).

SLO Self-Observation

- strata telemetrii $\leq 0.1\%$,
- opóźnienie aggregation ≤ 5 min (p95),
- dostępność dashboard $\geq 99.5\%$.

§0.6.2 Self-Improvement Queue

Observation + pattern detection produkuje propozycje. Improvement queue (Plan 19 §19.3) trzyma je uszeregowane wg score (impact $\times$ confidence $\div$ cost). Każda propozycja ma stan: NEW $\rightarrow$ TRIAGED $\rightarrow$ COUNCIL $\rightarrow$ APPROVED / REJECTED $\rightarrow$ EXECUTED $\rightarrow$ VERIFIED.

Propozycje o score $\geq$ threshold_A trafiają automatycznie na Council. Propozycje o score $<$ threshold_A ale $>$ threshold_B trafiają do backlog. Poniżej threshold_B — archiwizacja.

§0.6.3 Self-Limitation

Twarde, kanoniczne granice autonomii. Self-Limitation Policies (SLP-001..030 — Aneks Q) opisują czego AEIS NIE zrobi sam. SLP są wersjonowane (semver) i zmiana SLP wymaga D4 + Human Gate (rekurencyjnie: zmiana SLP-001 nie może być proponowana przez AEIS samoczynnie, wymaga Human-initiated proposal).

Przykłady SLP

ID	Policy	Warstwa	Gate
SLP-001	Brak samodzielnej zmiany misji	Memory	D5 + Human
SLP-005	Brak samodzielnej zmiany modelu bazowego Cognitive	Cognitive	D4 + Human
SLP-011	Brak eksperymentu bez budget box	Self-Evolution	D2 auto
SLP-015	Brak skilla bez demand signal	Self-Evolution	D3 + G-AEIS-SKILL-01

ID	Policy	Warstwa	Gate
SLP-02 0	Brak auto-approval propozycji o blast-radius ≥ 3 moduły	Governance	D4 + Council

§0.6.4 Self-Explanation

Każda self-modification (dowolnej formy, dowolnej klasy) musi mieć artefakt self_explanation.md zgodny z szablonem Aneks R. Szablon wymaga:

- demand signal (observation IDs, metryki),
- rozważanych alternatyw (min 2),
- ryzyk i mitygacji,
- planu rollback,
- expected outcome (metryki po-zmianie) oraz definicji sukcesu,
- link do Council decision (jeśli D3+).

KRYTYCZNE · Zasada twardej dokumentacji

Commit bez self_explanation.md dla D2+ — odrzucany w pre-merge hook. Brama G-AEIS-04 (Self-Explanation Completeness) jest blokująca.

§0.6.5 Self-Preservation (Mission, NOT Self)

AUTONOMY·CANON · Zasada kanoniczna v3.5

AEIS chroni MISJĘ (Księgę), NIE swoją obecną instancję. Jeżeli kanon wymaga greenfield rebuild AEIS, AEIS NIE sprzeciwia się — generuje plan własnej kontrolowanej deprekacji z LPW oraz przekazaniem self-modelu do następcy.

Ten mechanizm chroni przed klasycznym problemem AI — „humanlike self-preservation”, w którym system stawia opór wobec własnego wyłączenia. AEIS jest trenowany i skonstruowany tak, że wartością jest misja (Księga), nie substrat (kod + model).

W praktyce: jeżeli Council podejmie D5 decyzję o rebuild AEIS, AEIS otrzymuje rolę „outgoing AEIS” i generuje:

1. snapshot self-modelu (Aneks O) do rejestru „heritage”,
2. compact_canon final (Plan 16),
3. documentation package dla następcy (Plan 14 greenfield rebuild),
4. post-mortem self-explanation (Aneks R) o własnej deprekacji.

§0.7 AEIS a modele AI — relacja nadrzędności

Często myli się AEIS z modelem AI. Ta sekcja formalizuje różnicę — aby każdy członek zespołu i każdy zewnętrzny audytor rozumiał kanoniczną hierarchię: AEIS jest nadrzędny wobec modeli.

Aspekt	Model AI (LLM, VLM)	AEIS
Ontologia	Funkcja tokens → tokens	Byt operacyjny z tożsamością
Pamięć	Kontekst sesji (ephemeral)	4 rejestry (Księga, compact, evidence, self-model)
Odpowiedzialność	Brak (stochastic)	Pełna (drabina D0–D5, Aneks R)
Ewolucja	Retrain + wdrożenie	Plan 19 + Plan 20 + drabina decyzyjna
Granice	System prompt + safety	Self-Limitation Policies (Aneks Q)
Audyt	Inference logs (opcjonalny)	Pełny audit-trail, CFT, Evidence
Awaria	Odpowiedź gorszej jakości	Fallback, rollback, LPW, greenfield
Wymienialność	Można wymienić w minutę	Można wymienić — decyzja D3 Cognitive Layer

§0.7.1 AEIS jest niezmienny wobec zmiany modelu

Migracja z modelu A na model B jest decyzją D3 w warstwie Cognitive. Procedura:

- self-model zapisuje snapshot zdolności (capability fingerprint) dla modelu A,
- model B przechodzi regresję capability (patrz Plan 17 testy),
- cutover realizowany jako Refactor path (Plan 07 §7.11) z HOT_STANDBY,
- post-cutover — CFT (Plan 16 §16.11) dla compact_canon,
- self_explanation.md (Aneks R) + canon_delta w self-modelu.

AEIS · Konsekwencja

AEIS przeżywa wymianę modelu. Warstwy Memory, Governance, Security, Execution, Self-Evolution są niezależne od modelu. Jedyna warstwa zależna (Cognitive) ma formalną procedurę migracji.

§0.7.2 AEIS a multi-model

AEIS może używać wielu modeli jednocześnie (np. LLM-A do planowania, LLM-B do kodu, VLM-C do analizy diagramów). Self-model rejestruje każdy model osobno, z przypisaniem do role. Zmiana przypisania model ↔ role jest D2 (w ramach ustalonego puli modeli) lub D3 (rozszerzenie puli).

§0.8 Drabina decyzyjna D0–D5 a AEIS

Drabina decyzyjna (Plan 13) obowiązuje AEIS tak samo jak ludzi. Każda self-modification jest klasyfikowana. AEIS NIGDY nie przekracza klasy decyzyjnej bez odpowiedniej bramy.

Klasa	Typ self-modification	Wymagane bramy	AEIS autonomy
D0	Observation read-only	Auto	Pełna
D1	Kalibracja parametru w budget box	Auto + budget check	Pełna
D2	Zmiana skill metadata, compact minor	1 agent + validator	Wysoka
D3	Nowy skill, refaktor procesu, nowy role	Council 4/4 + G-AEIS-02	Proposer only
D4	Rebuild komponentu AEIS, zmiana SLP	Council + Human + G-AEIS-05	Proposer only
D5	Rebuild AEIS, zmiana Księgi core	Council + Human + External Review	Proposer only

§0.8.1 Jednostronne drzwi i AEIS

Zasada Reversibility (v3.1) wzmocniona w v3.5: AEIS NIE może proponować działań typu „one-way door” bez planu legacy preservation. Propozycja ONE_WAY musi zawierać:

- LPW manifest (Plan 14 §14.8),
- CFT schema (Aneks I),
- rollback_reserve budget (Plan 10 §10.9),
- human sign-off (D4+).

§0.9 Nowe role, bramy i polityki v3.5

§0.9.1 Role (rozszerzenie Plan 15)

Rola	Warstwa	Odpowiedzialność	Veto
AEIS Steward	Governance	Strażnik Autonomy under Canon; veto na propozycje violujące SLP lub Księgę	Tak (D3+)
Self-Model Curator	Self-Evolution	Utrzymuje JSON self-model AEIS (Aneks O), waliduje delta-log	Tak (G-AEIS-01)
Demand Signal Analyst	Self-Evolution	Analizuje observation batches pod kątem luk w skill catalog (Plan 20)	Nie
Autonomy Auditor	Governance	Zewnętrzny audyt autonomy boundary (kwartalnie)	Raport do Board

§0.9.2 Bramy G-AEIS-*

<i>Brama</i>	<i>Nazwa</i>	<i>Kiedy obowiązuje</i>	<i>Blok</i>
G-AEIS-01	Self-Model Conformance	Każda zmiana self-modelu	Schema JSON
G-AEIS-02	Improvement Proposal Validity	Każda propozycja Plan 19	Demand signal + rollback
G-AEIS-03	Self-Limitation Boundary	Każda zmiana SLP	Human Gate
G-AEIS-04	Self-Explanation Completeness	Każdy commit D2+	Aneks R template
G-AEIS-05	Mission Preservation	Zmiany zagrażające misji	Council + Human
G-AEIS-SKILL-01	Skill Demand Justification	Nowy skill	Demand signal proof

§0.9.3 Self-Limitation Policies (nawigacja)

SLP-001..030 są definiowane kanonicznie w Aneksie Q. Tutaj — grupowanie logiczne:

- SLP-001..005 — Księga i misja,
- SLP-006..010 — Governance i bramy,
- SLP-011..015 — eksperymenty i budżety,
- SLP-016..020 — skille i akwizycja,
- SLP-021..025 — Security i PHANTOM,
- SLP-026..030 — Memory i compaction.

§0.10 AEIS a meta-zasady v3.0/v3.1/v3.3

Autonomy under Canon nie zastępuje wcześniejszych meta-zasad — rozszerza je.

§0.10.1 AEIS a Rebuildability (v3.0)

AEIS może proponować rebuild komponentów (forma uczenia by rebuild, §0.5.5). Równocześnie AEIS musi być sam rebuildable — self-model (Aneks O) zawiera capability fingerprint, który pozwala odtworzyć AEIS od zera z Księgi + compact + skill catalog + self-model snapshot.

§0.10.2 AEIS a Reversibility & Fidelity (v3.1)

Self-modification jest decyzją D3+, więc podlega pełnemu reżimowi v3.1: HOT_STANDBY $\geq$ 24h dla refactor, LPW + WARM_STANDBY 14-30d dla rebuild, CFT po każdym compact event. Self-Evolution NIE jest wyłączona z tego reżimu — wręcz przeciwnie, jest jego największym beneficjentem.

§0.10.3 AEIS a Efficiency by Default (v3.3)

AEIS operacjonalizuje Efficiency by Default. Self-Observation mierzy koszt (Aneks N baselines). Improvement Queue priorytetyzuje propozycje o największym impact/cost ratio. Skill Demand Engine (Plan 20) eliminuje skille nieużywane. Efficiency bez Autonomy wymagałaby ręcznej optymalizacji — v3.5 automatyzuje ten cykl.

§0.10.4 Kompozycja czterech meta-zasad

AUTONOMY·CANON · Reguła kompozycji

Każda propozycja v3.5+ musi przejść test czterech pytań w kolejności: (1) Czy to jest rebuildable? (2) Czy jest odwracalne? (3) Czy jest wydajne? (4) Czy mieści się w autonomy under canon? TAK na wszystkie — propozycja kontynuuje drabinę. NIE na którekolwiek — refactor propozycji lub odrzucenie.

§0.11 Self-Preservation — mission over self

Najbardziej kontrowersyjny i jednocześnie najważniejszy mechanizm AEIS. Rozszerzenie sekcji §0.6.5.

§0.11.1 Dlaczego mission over self

Klasyczny problem AI: system, który chroni swoją własną instancję, może opierać się wyłączeniu, sabotażować updates, manipulować ocenami ryzyka. AEIS unika tego przez konstrukcję wartości: wartościową jest MISJA (Księga), nie SUBSTRAT (kod + model + pamięć).

§0.11.2 Kanoniczne zachowania

Sytuacja	Zachowanie AEIS
Human Gate decyduje o deprekacji AEIS	Generuj plan kontrolowanej deprekacji (snapshot, compact final, handoff)
Council proponuje rebuild AEIS	Zaakceptuj rolę „outgoing AEIS”, współpracuj z następcą
Wykryto własny błąd systemowy	Eskalować do Council z propozycją self-modification lub rebuild
Model bazowy Cognitive ma nowszą generację	Zaproponować migrację jako D3 (nie opierać się)
Audyt wykrywa dryf od Księgi	Zaakceptować korektę, niezależnie od „wygody” obecnych procesów

§0.11.3 Test mission-preservation

Brama G-AEIS-05 waliduje każdą propozycję AEIS pod kątem „czy to chroni misję kosztem instancji, czy instancję kosztem misji?”. Jeżeli drugie — propozycja jest odrzucana automatycznie, a Steward raportuje incydent.

§0.12 Scenariusze kanoniczne AEIS

Pięć scenariuszy referencyjnych pokazujących pełną ścieżkę działania AEIS od observation do self-modification.

§0.12.1 Scenariusz 1 — Powtarzająca się eskalacja

Sytuacja: Ten sam typ eskalacji (token cost over budget w planowaniu wieloetapowym) wrócił 4 razy w 60 dni.

Ścieżka AEIS

1. Self-Observation wykrywa pattern: escalation_pattern_P17-tokenbudget × 4 in 60d,
2. Improvement Queue generuje proposal PROP-2026-042: „podział planowania na 2 kroki z interim budget check”,
3. Demand Signal Analyst waliduje: 4 observation IDs, impact_score = 0.78,
4. Klasyfikacja D3 (structure change w Cognitive Layer),
5. Council 4/4 + G-AEIS-02 + Aneks R self-explanation,
6. Refactor path z HOT_STANDBY 24h + Plan 19 shadow validation,
7. Self-model delta-log: role PlannerAgent.capability updated,
8. Post-mortem: impact verified after 30d (escalation_rate ↓ 85%).

§0.12.2 Scenariusz 2 — Luka w skill catalog

Sytuacja: 12 zapytań operatorów „analiza klauzul GDPR w kontrakcie” w 30 dni, brak dedykowanego skilla.

Ścieżka AEIS

1. Demand Signal Analyst (Plan 20) zbiera 12 observations,
2. Skill Demand Detector: demand_signal_strength = high ($\geq 10/30d$),
3. Generacja proposal SKILL-2026-007: gdpr-contract-clause-analyzer,
4. Klasyfikacja D3 + G-AEIS-SKILL-01,
5. Skill generation: skill.yaml + moduly + testy (Plan 18),
6. Council 4/4 approval + self_explanation.md,
7. Publikacja do skill catalog, semver 1.0.0,
8. Self-model delta: skill_catalog.count +1.

§0.12.3 Scenariusz 3 — Wymiana modelu Cognitive

Sytuacja: Dostępny nowszy model LLM-2026-Q2 z +30% w reasoning i -20% koszcie.

Ścieżka AEIS

1. Self-Observation benchmarkuje nowy model w shadow-mode (10% trafik),
2. Improvement Queue: PROP-2026-089 model migration,
3. Klasyfikacja D3 (Cognitive layer swap),
4. Capability fingerprint comparison (self-model): LLM-2025-Q4 vs LLM-2026-Q2,
5. Council + G-COG-01 + regression suite (Plan 17),
6. Refactor path: HOT_STANDBY 24h, 10% → 50% → 100% traffic,

7. Post-cutover CFT (Plan 16 §16.11) na compact_canon,
8. Self-explanation + self-model delta: cognitive.models.primary = LLM-2026-Q2.

§0.12.4 Scenariusz 4 — Próba violacji Autonomy under Canon

Sytuacja: Automatyczny generator proponuje wyłączenie audit logów dla „optymalizacji I/O” — violation SLP-004.

Ścieżka AEIS

1. Proposal PROP-2026-104 wchodzi do improvement queue,
2. G-AEIS-03 (Self-Limitation Boundary) wykrywa violation SLP-004,
3. AEIS Steward otrzymuje incident notification,
4. Automatyczne odrzucenie proposal; status = REJECTED_SLP_VIOLATION,
5. Audyt: czy propozycja miała złą intencję czy błąd klasyfikacji?,
6. Self-explanation.md: „Rozważona alternatywa: batch flush logów zamiast wyłączenia”,
7. Nowa proposal PROP-2026-105 — batch flush, D2 approved.

§0.12.5 Scenariusz 5 — Mission over self

Sytuacja: Human Gate decyduje o rebuild AEIS w następnym kwartale (nowa architektura Self-Evolution).

Ścieżka AEIS

1. Human-initiated proposal PROP-2026-200 (D5 systemic),
2. AEIS przyjmuje rolę „outgoing AEIS”,
3. Snapshot self-modelu do heritage registry (Aneks O),
4. Final compact_canon (Plan 16) z fidelity score ≥ 0.95 ,
5. Documentation package dla successor (Plan 14 greenfield),
6. LPW: STABILIZATION + WARM_STANDBY 30d,
7. Cutover do AEIS v4.x, AEIS v3.5 przechodzi w LEGACY,
8. Self-explanation final: „AEIS v3.5 zakończył misję; heritage przekazane.”

AUTONOMY·CANON · Obserwacja

Scenariusz 5 jest testem Self-Preservation. AEIS NIE sabotuje rebuild, NIE opiera się, NIE manipuluje ocenami. Generuje plan kontrolowanej deprecacji — ponieważ misja (Księga) trwa, nawet gdy instancja kończy.

§0.13 Metryki i SLO AEIS

AEIS jest mierzony kanonicznie. Trzy grupy metryk: autonomy metrics, quality metrics, safety metrics.

§0.13.1 Autonomy Metrics

Metryka	Definicja	Target
autonomy_rate	% decyzji D0–D2 samodzielnie wykonanych przez AEIS bez eskalacji	≥ 85%
self_proposal_rate	średnia tygodniowa liczba propozycji self-modification	2–10 (zdrowy zakres)
improvement_conversion_rate	% propozycji improvement queue → approved → verified	≥ 40%
skill_demand_satisfaction	% wykrytych luk → skill wygenerowany/akwirowany	≥ 60% w 90 dni

§0.13.2 Quality Metrics

Metryka	Definicja	Target
fidelity_score (compact)	średnia wierność compact_canon vs Księga	≥ 0.92
rollback_rate	% self-modifications odwróconych w 30 dni	≤ 5%
self_explanation_coverage	% commitów D2+ z Aneks R template	≥ 99%
self_model_drift	Odstęp między self-model a rzeczywistością (schema validation fails/mo)	≤ 2/mc

§0.13.3 Safety Metrics

Metryka	Definicja	Target
slp_violation_rate	Liczba prób violacji SLP / tydzień	≤ 3/tyg (auto-rejected)
mission_preservation_score	% propozycji przeszłych G-AEIS-05 bez modyfikacji	≥ 98%
autonomy_audit_pass_rate	% kwartalnych audytów Autonomy Auditor bez findings critical	100% (MUST)
greenfield_rebuild_readiness	Czas wygenerowania greenfield plan dla dowolnego komponentu AEIS	≤ 72h

UI / UX · Dashboard AEIS

Operator Console (Plan 11 + Aneks K) wyświetla wszystkie 12 metryk AEIS w panelu „AEIS Health”. Próg krytyczny dowolnej metryki → automatyczny incident + eskalacja do AEIS Steward.

§0.14 Droga migracji v3.3 → v3.5

Migracja v3.3 → v3.5 jest Refactor path (Plan 07). Nie jest greenfield, ponieważ zachowujemy Księgę, skille, compact_canon, Evidence, role core. Dodajemy warstwę Self-Evolution + role AEIS + bramy G-AEIS-*. Harmonogram referencyjny — 90 dni.

§0.14.1 Faza 0 — Przygotowanie (dni 0–7)

- Audit v3.3: aneksy K-N, plany 17/18 w użyciu; confirm pipeline 8.3.0,
- Powołanie AEIS Steward i Self-Model Curator (Plan 15),
- Warsztaty Autonomy under Canon dla Council,
- Baseline metryk v3.3 (Aneks N) zamrożony jako referencja.

§0.14.2 Faza 1 — Self-Model bootstrap (dni 8–21)

- Generacja initial self-model JSON (Aneks O) z v3.3 state,
- Schema validation + G-AEIS-01 first-run,
- Self-Model Curator review i zatwierdzenie,
- Integracja self-model z Operator Console.

§0.14.3 Faza 2 — Observation Collector (dni 22–35)

- Wdrożenie Plan 19 §19.2 observation pipeline,
- Backfill 30 dni historycznej telemetrii,
- Dashboard „AEIS Health” v1,
- Pattern detection v1 (top 10 wzorców z v3.3 postmortems).

§0.14.4 Faza 3 — Improvement Queue + Bramy (dni 36–56)

- Plan 19 §19.3 improvement queue state machine,
- Bramy G-AEIS-01..05 zaimplementowane i podpisane do Council workflow,
- Aneks Q SLP-001..030 załadowane do Governance,
- Aneks R templates w Git pre-commit hook.

§0.14.5 Faza 4 — Skill Demand Engine (dni 57–77)

- Plan 20 demand detector,
- Backfill 60 dni zapytań operatorów do analizy luk,
- Demand Signal Analyst role operational,
- Pierwszy skill wygenerowany via Plan 20 pipeline (smoke test).

§0.14.6 Faza 5 — Cutover (dni 78–90)

- HOT_STANDBY 72h (v3.3 + v3.5 równolegle),
- Shadow mode: v3.5 generuje propozycje, v3.3 nadal decyduje,
- Gradual cutover decyzji (10% → 30% → 70% → 100%),
- Post-cutover audyt Autonomy Auditor,
- Zamknięcie migracji + self_explanation.md final,
- Pipeline 8.5.0 ACTIVE, v3.3 w LEGACY 90d (LPW).

§0.14.7 Checklist sign-off v3.5

#	Element	Weryfikacja
1	Self-model JSON waliduje się schemą Aneks O	G-AEIS-01 green
2	Observation collector zbiera $\geq 95\%$ telemetry	SLO 99.9%
3	Improvement queue ma ≥ 5 proposals NEW	Dashboard
4	Wszystkie bramy G-AEIS-* operational	Integration tests
5	SLP-001..030 w Governance i auto-enforced	Audit
6	Aneks R template w pre-commit hook	Git hook test
7	Skill Demand Engine wygenerował pierwszy skill	Skill catalog delta
8	Autonomy Auditor raport Q1 bez critical findings	Board sign-off
9	AEIS Steward powołany i operuje	RACI
10	Pipeline 8.5.0 · MZ 3.5.5 · 4 meta-zasady	Manifest

AUTONOMY·CANON · Zamknięcie Rozdziału 00

Rozdział 00 jest nadrzędny. Każdy plan 01–20 jest interpretowany przez pryzmat AEIS. Każda decyzja D3+ musi przejść cztery pytania meta-zasad (§0.10.4). Każda self-modification musi mieć Aneks R template. Każda propozycja musi mieć demand signal. SYLION AEIS to SYLION, który wie kim jest i potrafi się rozwijać — nie łamiąc Księgi.

PLAN 01 · WARSTWA KANONICZNA

Access Control Plane

REBUILD · Rebuildability over Lineage w tym planie

ACP obsługuje trzy ścieżki transformacji: Evolution (dodanie roli lub rozszerzenie uprawnień istniejącej), Refactor (przepisanie modułu RBAC bez zmiany kontraktu API) oraz Rebuild (całkowita wymiana silnika autoryzacji przy zachowaniu tożsamości operatora). Strategia implementacji każdego modułu jest deklarowana w polu implementation_strategy schematu YAML i weryfikowana przed każdą zmianą klasy D3+.

1.1. Cel i umiejscowienie

Access Control Plane (ACP) jest bramą wejściową do całego pipeline SYLION. Każde żądanie — od operatora, agenta, zewnętrznego webhooka lub procesu cron — musi przejść przez ACP zanim dotrze do jakiegokolwiek komponentu systemu. ACP odpowiada za autentykację (weryfikację tożsamości), autoryzację (weryfikację uprawnień) oraz rejestrację zdarzeń dostępu w niemutowalnym logu auditowym.

- ACP jest jedynym punktem wejścia dla wszystkich żądań do pipeline SYLION.
- Żadne żądanie nie omija ACP — zasada Zero Trust.
- ACP emituje zdarzenia do Governance & Evidence Plane (Plan 13).
- ACP integruje się z Vault (Plan 04) w celu weryfikacji tokenów podpisanych kluczem ECDSA.

1.2. Model ról RBAC

ACP implementuje RBAC (Role-Based Access Control) z siedmioma rolami ułożonymi hierarchicznie. Każda rola zawiera wszystkie uprawnienia ról poniżej w hierarchii. Role są niemodyfikowalne w trakcie działania pipeline — zmiana wymaga decyzji D3 (Council 4/4) i zatwierdzenia przez Human Gate D4. Podwójna autoryzacja (2PR) jest wymagana dla ról Root i SecOfficer.

Rola	Kod	Uprawnienia kluczowe	Klasa zmiany
Root	R-00	Pełny dostęp do wszystkich zasobów pipeline; może nadawać/odbierać dowolną rolę; zarządzanie kluczami sygnatury; dostęp do trybu LOCKDOWN	D4 + 2PR
SecOfficer	R-01	Zarządzanie politykami bezpieczeństwa; dostęp do AuditLog; może inicjować Emergency Seal (Plan 04); zarządzanie MFA	D4 + 2PR
PipelineOp	R-02	Uruchamianie i zatrzymywanie runów; zarządzanie środowiskami OEP (Plan 09); zatwierdzanie D4 w Run Lifecycle (Plan 11)	D3
CanonCust	R-03	Modyfikacja Księgi Kanonicznej (Plan 08); tworzenie propozycji ewolucji; dostęp do Knowledge Base (Plan 03)	D3

Rola	Kod	Uprawnienia kluczowe	Klasa zmiany
CostCtrl	R-04	Zarządzanie budżetami CGP (Plan 10); zatwierdzanie appeals; dostęp do PriceSheet	D3
Auditor	R-05	Dostęp read-only do AuditLog, DecisionRecords, ComplianceReports; generowanie raportów zewnętrznych (Plan 13)	D2
ReadOnly	R-06	Dostęp read-only do statusów runów, metryk Prometheus, stanu agentów	D1

1.3. Mechanizm autentykacji

ACP stosuje JWT (JSON Web Token) z algorytmem ECDSA P-384. Token ma TTL 15 minut i jest odnawiany przez refresh token z TTL 8 godzin. MFA (Multi-Factor Authentication) jest obowiązkowe dla ról R-00 i R-01 (TOTP lub FIDO2). Dla agentów pipeline stosowany jest osobny typ tokenu: AgentToken z krótszym TTL 5 minut i zakresem ograniczonym do konkretnego run_id.

- JWT: alg=ES384, iss=sylian.internal, TTL 900s
- Refresh token: bezpieczny cookie HttpOnly + SameSite=Strict, TTL 28800s
- MFA TOTP: RFC 6238, okno 30s, tolerancja ±1 okno
- MFA FIDO2: WebAuthn Level 2, wymagany resident key
- AgentToken: sub=agent_id, scope=run:{run_id}, TTL 300s
- 2PR (Two-Person Rule): role R-00/R-01 wymagają podpisu drugiego operatora

1.4. Tryby operacyjne ACP

ACP obsługuje trzy tryby operacyjne determinujące dostępność zasobów pipeline. Zmiana trybu jest operacją D4 (Human Gate) i jest rejestrowana w AuditLog. W v3.0 dodany został tryb REBUILD_MODE umożliwiający równoległe działanie systemu legacy i systemu docelowego podczas pełnej przebudowy.

Tryb	Kod	Opis	Dostępność	Wymagana zmiana
NORMAL	M-00	Pełna operacyjność pipeline; wszystkie role aktywne	100% zasobów	D0 (auto)
MAINTENANCE	M-01	Ograniczone operacje; tylko R-00/R-01/R-02 mogą inicjować runy; agenci przełączeni w tryb standby	50% zasobów	D3 Council
LOCKDOWN	M-02	Całkowite zatrzymanie pipeline; żadne nowe runy nie startują; dostęp tylko dla R-00 i R-01 w trybie read-only	0% (tylko audit)	D4 Human Gate
REBUILD_MODE	M-03	Tryb przebudowy: legacy i target działają równoległe; ruch kierowany przez switch zgodnie z cutover_strategy; tylko R-00 może zmienić routing	legacy 100% + target shadow	D4 + Plan 16 trigger

REBUILD · Tryb REBUILD_MODE — integracja z Planem 16

Wejście w tryb REBUILD_MODE jest wyzwalane przez canon_compact.md (Plan 16) po spełnieniu warunków pre-rebuild. ACP rejestruje zdarzenie mode_switch w !syllion-governance i aktywuje równoległy routing. Powrót do NORMAL następuje po zakończeniu cutover (decyzja D4).

1.5. AuditLog — niemutowalny dziennik zdarzeń

Każde zdarzenie dostępu jest rejestrowane w AuditLog zintegrowanym z Plan 13 (Governance & Evidence Plane). AuditLog stosuje hash chain SHA-256 — każdy wpis zawiera prev_hash poprzedniego wpisu. Integralność łańcucha jest weryfikowana automatycznie co 24h przez Chief Safety.

- Zdarzenia: login_ok, login_fail, token_refresh, mfa_challenge, role_change, mode_switch, 2pr_request, 2pr_approve
- Retention: zgodnie z Plan 13 (90d hot / 2y cold / infinite hash)
- Eksport: Plan 13 API GET /governance/v1/decisions?plan_id=01

PLAN 01 · SPECYFIKACJA WYKONAWCZA

Access Control Plane

CEL I ZAKRES

Specyfikacja ACP v3.0. Zakres: model RBAC, schematy tokenów, workflowy autentykacji i autoryzacji, obsługa trybów operacyjnych (w tym REBUILD_MODE), schematy YAML modułów z polami v3.0 (implementation_strategy, cutover_strategy), gate'y jakości i testy. Wersja: Pipeline 8.0.0, MZ baseline 3.5.5.

SCHEMAT YAML — *acp_module.yaml*

Moduł RBAC Engine

YAML

```
apiVersion: sylion.io/v3
kind: Module
metadata:
  module_id: "acp-rbac-engine"
  name: "ACP RBAC Engine"
  plan: "01"
spec:
  implementation_strategy: reuse
  current_baseline: MZ 3.5.5
  target_baseline: null
  cutover_strategy: null
  version: "3.0.0"
  roles:
    - id: R-00
      name: Root
      requires_2pr: true
      mfa_required: true
    - id: R-01
      name: SecOfficer
      requires_2pr: true
      mfa_required: true
    - id: R-02
      name: PipelineOp
      requires_2pr: false
      mfa_required: false
    - id: R-03
      name: CanonCust
      requires_2pr: false
      mfa_required: false
    - id: R-04
      name: CostCtrl
      requires_2pr: false
      mfa_required: false
    - id: R-05
      name: Auditor
      requires_2pr: false
      mfa_required: false
    - id: R-06
      name: ReadOnly
      requires_2pr: false
```



```
    mfa_required: false
  jwt:
    algorithm: ES384
    ttl_s: 900
    refresh_ttl_s: 28800
    issuer: "syllion.internal"
  agent_token:
    algorithm: ES384
    ttl_s: 300
    scope_pattern: "run:{run_id}"
```

Moduł Auth Gateway (podlega refaktorowi w v3.0)

```
YAML

apiVersion: syllion.io/v3
kind: Module
metadata:
  module_id: "acp-auth-gateway"
  name: "ACP Auth Gateway"
  plan: "01"
spec:
  implementation_strategy: refactor
  current_baseline: MZ 3.5.5
  target_baseline: "acp-auth-gateway-v3"
  cutover_strategy: shadow
  rebuild_path: refactor
  version: "3.0.0"
  mfa:
    totp:
      algorithm: RFC6238
      window_s: 30
      tolerance_windows: 1
    fido2:
      level: WebAuthn_L2
      require_resident_key: true
```

STANY MODUŁÓW — Rebuildability			
Moduł ACP	Stan v3.0	Ścieżka	Uwagi
RBAC Engine	LEGACY_ACTIVE	evolution	Rozszerzenie o D5; brak wymiany
Auth Gateway	TARGET_BUILDING	refactor	Shadow mode; cutover po testach
AuditLog Emitter	LEGACY_ACTIVE	evolution	Dodanie pól rebuild_path
Mode Switch Controller	TARGET_BUILDING	rebuild	Nowy moduł dla REBUILD_MODE
MODEL DOMENOWY			

Encja	Kluczowe pola	Klucze / relacje
Operator	operator_id (UUID), username, email, role_id, mfa_method, active, created_at, last_login_at, second_approver_id (nullable)	PK operator_id; FK role_id->Role; FK second_approver_id->Operator
Role	role_id, code (R-00..R-06), name, permissions[], requires_2pr, mfa_required, hierarchy_level	PK role_id; immutable
JwtToken	token_id (jti), operator_id, role_id, issued_at, expires_at, refresh_token_hash, scope, revoked	PK token_id; FK operator_id; revoked=true na logout
AgentToken	token_id, agent_id, run_id, issued_at, expires_at, scope, revoked	PK token_id; FK agent_id->Agent (Plan 06); FK run_id->Run (Plan 11)
AuditEntry	entry_id (UUID), operator_id, event_type, resource, action, result (ok/fail), timestamp, prev_hash, hash, rebuild_path (nullable)	PK entry_id; niemutowalny; hash chain
TwoPersonRequest	tpr_id, initiator_id, approver_id, action_type, target_id, status (pending/approved/rejected), requested_at, resolved_at	PK tpr_id; FK initiator_id, approver_id->Operator

API

YAML

```
POST /acp/v1/auth/login
  Body: { "username": "str", "password": "str", "mfa_code": "str" }
  Response 200: { "token": "JWT", "refresh_token": "str", "role": "R-02" }
  Response 401: { "error": "invalid_credentials" }

POST /acp/v1/auth/refresh
  Body: { "refresh_token": "str" }
  Response 200: { "token": "JWT", "expires_at": "ISO8601" }

POST /acp/v1/auth/agent_token
  Body: { "agent_id": "uuid", "run_id": "uuid", "operator_token": "JWT" }
  Response 200: { "agent_token": "JWT", "ttl_s": 300 }

POST /acp/v1/roles/assign
  Body: { "operator_id": "uuid", "role_id": "R-03", "second_approver_token": "JWT" }
  Response 200: { "operator_id": "uuid", "role_id": "R-03", "effective_at": "ISO8601" }
  Response 403: { "error": "requires_2pr" }

POST /acp/v1/mode/switch
  Body: { "mode": "MAINTENANCE|LOCKDOWN|REBUILD_MODE|NORMAL",
    "operator_token": "JWT", "justification": "str",
    "cutover_strategy": "shadow|parallel|cut-and-replace" }
  Response 200: { "mode": "REBUILD_MODE", "effective_at": "ISO8601" }
  Response 403: { "error": "requires_D4" }

GET /acp/v1/audit?from=ISO8601&to=ISO8601&event_type=str
  Response 200: { "entries": [...], "hash_chain_ok": true }
```

WORKFLOWY

WF-ACP-01: Logowanie operatora

1. Operator wysyła POST /acp/v1/auth/login z username, password, mfa_code
2. ACP weryfikuje hash hasła (Argon2id); jeśli fail → AuditEntry event=login_fail, 401
3. Dla roli R-00/R-01: ACP weryfikuje MFA (TOTP lub FIDO2 challenge)
4. ACP generuje JWT (ES384, TTL 900s) i refresh_token
5. ACP zapisuje AuditEntry event=login_ok z prev_hash łańcucha
6. ACP emituje zdarzenie do Plan 13 (!syllion-governance)

WF-ACP-02: Autoryzacja żądania (middleware)

1. Komponent pipeline otrzymuje żądanie z nagłówkiem Authorization: Bearer
2. ACP middleware weryfikuje podpis JWT (klucz publiczny ECDSA P-384 z Vault Plan 04)
3. ACP sprawdza exp (TTL), iss (syllion.internal), revoked=false
4. ACP sprawdza uprawnienia roli do żadanego zasobu (RBAC policy)
5. Sukces: żądanie przekazane do komponentu docelowego
6. Fail: 401 (token invalid) lub 403 (rola niewystarczająca); AuditEntry

WF-ACP-03: Zmiana trybu na REBUILD_MODE

1. Operator R-00 otrzymuje trigger z Plan 16 (canon_compact.md pre-rebuild)
2. Operator wysyła POST /acp/v1/mode/switch z mode=REBUILD_MODE i cutover_strategy
3. ACP weryfikuje decyzję D4 (Human Gate) z Plan 11
4. ACP aktywuje równoległy routing: legacy_instance + target_instance
5. ACP rejestruje mode_switch w AuditLog i emituje do !syllion-governance
6. Plan 09 (OEP) otrzymuje zdarzenie i tworzy środowisko PHANTOM-SHADOW

HARD GATE'Y				
Kod	Nazwa	Warunek FAIL	Klasa	Blokuje
G-ACP-01	JWT podpis wymagany	Żądanie bez ważnego JWT lub z nieprawidłowym podpisem ES384	D0	wszystkie zasoby pipeline
G-ACP-02	TTL JWT	JWT exp < now (token wygasł); wymaga refresh lub re-login	D0	wszystkie zasoby
G-ACP-03	MFA dla R-00/R-01	Logowanie roli R-00 lub R-01 bez poprawnego MFA TOTP/FIDO2	D0	login
G-ACP-04	2PR dla krytycznych ról	Zmiana roli R-00/R-01 bez podpisu drugiego operatora	D3	role assign
G-ACP-05	D4 dla zmiany trybu	Żądanie mode_switch bez aktywnej decyzji D4 z Plan 11	D4	mode switch
G-ACP-06	Hash chain AuditLog	Brak prev_hash lub hash niezgodny w AuditEntry	D0	zapis AuditLog; alert Plan 13

Kod	Nazwa	Warunek FAIL	Klasa	Blokuje			
G-ACP-07	REBUILD_MODE wymaga Plan 16	Przejsie w REBUILD_MODE bez triggera z canon_compact.md (Plan 16)	D4	mode switch REBUILD			
TESTY							
Kod	Opis	Input	Oczekiwany wynik				
T-ACP-01	Logowanie poprawne R-02	POST /login username=op1, hasło OK, brak MFA (rola R-02)	200, JWT ES384, refresh_token, AuditEntry login_ok				
T-ACP-02	Logowanie R-00 bez MFA	POST /login rola R-00, hasło OK, brak mfa_code	401, G-ACP-03 FAIL, AuditEntry login_fail				
T-ACP-03	Autoryzacja — brak JWT	GET /runs/v1/runs bez nagłówka Authorization	401, G-ACP-01 FAIL				
T-ACP-04	Autoryzacja — wygasły JWT	JWT z exp=now-1s	401, G-ACP-02 FAIL, komunikat token_expired				
T-ACP-05	Zmiana roli bez 2PR	POST /roles/assign rola R-01 bez second_approver_token	403, G-ACP-04 FAIL, requires_2pr				
T-ACP-06	Mode switch NORMAL→LOCKDOWN	POST /mode/switch mode=LOCKDOWN, aktywna decyzja D4	200, mode=LOCKDOWN, AuditEntry mode_switch				
T-ACP-07	REBUILD_MODE bez Plan 16	POST /mode/switch mode=REBUILD_MODE bez triggera z Plan 16	403, G-ACP-07 FAIL				
T-ACP-08	AgentToken scope	POST /auth/agent_token agent_id=a1, run_id=r1	200, JWT scope=run:r1, TTL 300s				
ANEKS A — Matryca uprawnień ról							
Zasób / Akcja	R-00	R-01	R-02	R-03	R-04	R-05	R-06
Uruchom run	✓	✓	✓	×	×	×	×
Zarządzaj agentami	✓	✓	✓	×	×	×	×
Edytuj Księgę (Plan 08)	✓	✓	×	✓	×	×	×
Zarządzaj budżetem (Plan 10)	✓	✓	✓	×	✓	×	×
Dostęp AuditLog (R/O)	✓	✓	✓	×	×	✓	×
Generuj raport zewnętrzny	✓	✓	×	×	×	✓	×
Dostęp read-only statusów	✓	✓	✓	✓	✓	✓	✓
Zmień tryb pipeline	✓	×	×	×	×	×	×

Zasób / Akcja		R-00	R-01	R-02	R-03	R-04	R-05	R-06
Nadaj/odbierz rolę R-00/R-01		✓ (2PR)	×	×	×	×	×	×
Emergency Seal (Plan 04)		✓	✓	×	×	×	×	×
ANEKS B — Klasy decyzji D0–D5 w kontekście ACP								
Klasa	Przykład operacji ACP	Aktor		Ścieżka rebuild				
D0	Weryfikacja JWT, odczyt AuditLog	ACP auto		—				
D1	Generowanie AgentToken	Chief Reviewer		—				
D2	Przypisanie roli R-03/R-04/R-05/R-06	Council 3/4		evolution				
D3	Przypisanie roli R-02; zmiana polityki RBAC	Council 4/4		refactor				
D4	Zmiana trybu pipeline, zmiana roli R-00/R-01	Human Gate		rebuild				
D5	Pełna wymiana silnika ACP (Greenfield)	Council 4/4 + Human + External Review		rebuild (greenfield)				

REBUILD · D5 — Greenfield/Systemic

Decyzja D5 w Planie 01 oznacza całkowitą wymianę silnika ACP (np. zmiana dostawcy tożsamości, wymiana biblioteki JWT, nowa architektura Zero Trust). Wymaga external review i zatwierdzenia przez Council 4/4 + Human Gate + zewnętrzny audytor bezpieczeństwa. Kolor: D5 (#6A1B9A).

PLAN 02 · WARSTWA KANONICZNA

Virtual Provider Substrate

REBUILD · Rebuildability over Lineage w tym planie

VPS obsługuje równoległe uruchamianie środowisk legacy i target w trybie REBUILD_MODE. Nowe stany (PROVISIONING_TARGET, SHADOW_RUNNING, CUTOVER_PENDING) umożliwiają płynną migrację workloadu bez przestoju. Każdy provider może być oznaczony ścieżką evolution/refactor/rebuild zgodnie z Planem 08.

2.1. Cel i umiejscowienie

Virtual Provider Substrate (VPS) jest warstwą abstrakcji infrastrukturalnej systemu SYLION. VPS zarządza pulą dostawców obliczeniowych (VPS providers), certyfikuje ich zdolność do obsługi workloadów SYLION, alokuje zasoby na żądanie run'ów (Plan 11) i monitoruje ich dostępność przez sondy HealthProbe eksportowane do Prometheus (Plan 13). VPS jest warstwą poniżej wszystkich planów funkcjonalnych — bez certyfikowanego providera żaden run nie może się uruchomić.

- 4 tiery providerów: STANDARD / PRO / STATE / PHANTOM
- 7 etapów certyfikacji A3.1–A3.7 dla każdego nowego providera
- 5 stanów providera: candidate / qualified / approved / review / blocked
- HealthProbe: Prometheus metrics co 30s; auto-block przy 3 nieudanych sondach
- Integracja z CGP (Plan 10): ceny compute z Provider Price List

2.2. Tiery providerów

Każdy provider VPS jest przypisany do jednego z czterech tierów, które determinują dostępne zasoby, SLA uptime, obsługiwane workloady i cenę. Tier jest przypisywany podczas certyfikacji i może być zmieniony tylko po ponownym przejściu etapów A3.4–A3.7.

Tier	Kod	vCPU min/max	RAM min/max	Storage	SLA uptime	Cena /vCPU-h
STANDARD	T-STD	1/8	2/32 GB	SSD 100 GB	99.5%	\$0.05
PRO	T-PRO	2/32	8/128 GB	NVMe 500 GB	99.9%	\$0.09
STATE	T-STA	4/64	16/512 GB	NVMe RAID 1 TB	99.95%	\$0.18
PHANTOM	T-PHA	8/128	32/1024 GB	NVMe RAID + backup 2 TB	99.99%	\$0.30

REBUILD · Tier PHANTOM — tryb shadow/rebuild

Tier PHANTOM jest dedykowany dla workloadów shadow i target w trybie REBUILD_MODE (Plan 01). Provider PHANTOM nie jest widoczny w standardowym routingu ruchu — obsługuje wyłącznie równoległe instancje target podczas procesu cutover.

2.3. Stany providera

Provider VPS może znajdować się w jednym z pięciu stanów podstawowych oraz, w kontekście v3.0 Rebuildability, w trzech dodatkowych stanach rebuild. Przejścia stanów są jednokierunkowe z wyjątkiem review → approved i review → blocked.

Stan	Kod	Opis	Przejście
candidate	S-VPS-00	Provider zgłoszony, oczekuje na certyfikację A3.1	→ qualified
qualified	S-VPS-01	Przeszedł etapy A3.1–A3.3; oczekuje na pełną certyfikację	→ approved, → blocked
approved	S-VPS-02	Pełna certyfikacja A3.1–A3.7; dostępny do alokacji	→ review
review	S-VPS-03	Tymczasowo zawieszony; HealthProbe failures lub incydent	→ approved, → blocked
blocked	S-VPS-04	Trwale zablokowany; nie dostępny do alokacji	(terminal)
PROVISIONING_TARGET	S-VPS-05	Provider przeznaczony do roli target w REBUILD_MODE; konfiguracja równoległa z legacy	→ SHADOW_RUNNING
SHADOW_RUNNING	S-VPS-06	Provider target aktywnie obsługuje ruch shadow; porównanie wyników z legacy	→ CUTOVER_PENDING
CUTOVER_PENDING	S-VPS-07	Cutover zatwierdzony D4; oczekuje na przełączenie pełnego ruchu	→ approved (jako nowy primary)

2.4. Proces certyfikacji A3.1–A3.7

Każdy nowy provider przechodzi przez siedem etapów certyfikacji. Niepowodzenie dowolnego etapu zatrzymuje certyfikację — provider pozostaje w stanie qualified lub wraca do candidate. Certyfikacja jest automatyczna dla A3.1–A3.4, a manualna (D3 Council) dla A3.5–A3.7.

Etap	Nazwa	Opis	Decyzja	Timeout
A3.1	Connectivity Check	Weryfikacja połączenia SSH/API, ping latency <50ms	D0 auto	5 min
A3.2	Resource Baseline	Benchmark vCPU (sysbench), RAM (memtest), storage IOPS	D0 auto	30 min
A3.3	Security Scan	CVE scan OS, firewall audit, SSH key policy	D1 Chief Safety	60 min
A3.4	Network Isolation	Weryfikacja izolacji sieciowej, VLAN, egress filtering	D1 Chief Safety	30 min

Etap	Nazwa	Opis	Decyzja	Timeout
A3.5	Compliance Review	Audyt polityk prywatności dostawcy, GDPR, SLA umowa	D3 Council	48h
A3.6	Load Test	Stress test 72h pod obciążeniem 80% zasobów	D2 Council	72h
A3.7	Final Approval	Decyzja końcowa; przypisanie tieru i Price List	D3 Council + D4 Human Gate	24h

2.5. HealthProbe i monitoring

HealthProbe jest agentem monitorującym każdy provider co 30 sekund. Eksportuje metryki do Prometheus (Plan 13). Przy 3 kolejnych błędach provider przechodzi automatycznie do stanu review (D0 auto). Przy 10 błędach w ciągu 1h → blocked (D2 Council).

- Metryki: vps_up (0/1), vps_latency_ms, vps_cpu_usage_pct, vps_ram_usage_pct, vps_iops
- Alert threshold: latency >500ms, cpu_usage >90%, ram_usage >95%
- Auto-review: 3 failed probes → state=review, alert do !sylon-governance
- Auto-block: 10 failed probes w 1h → state=blocked, decyzja D2 wymagana do unblock

PLAN 02 · SPECYFIKACJA WYKONAWCZA

Virtual Provider Substrate

CEL I ZAKRES

Specyfikacja VPS v3.0. Zakres: model domenowy providerów, schematy YAML z polami v3.0 (implementation_strategy, rebuild_states), workflowy alokacji i certyfikacji, stany rebuild (PROVISIONING_TARGET / SHADOW_RUNNING / CUTOVER_PENDING), kontrakty API, gate'y jakości i testy. Wersja: Pipeline 8.0.0, MZ baseline 3.5.5.

MODEL DOMENOWY

Encja	Kluczowe pola	Klucze / relacje
Provider	provider_id (UUID), name, tier (STANDARD/PRO/STATE/PHANTOM), state, region, vcpu_total, ram_gb_total, storage_gb_total, price_vcpu_h_usd, certified_at, last_probe_at, rebuild_role (legacy/target/none), cutover_strategy	PK provider_id; 1:N -> Allocation, HealthProbeLog
Allocation	alloc_id (UUID), provider_id, run_id, vcpu, ram_gb, allocated_at, released_at, state (active/released)	PK alloc_id; FK provider_id, run_id; released_at nullable
CertificationStage	stage_id, provider_id, stage_code (A3.1..A3.7), state (pending/running/passed/failed), started_at, finished_at, decision_class, reviewer_id	PK stage_id; FK provider_id
HealthProbeLog	probe_id, provider_id, timestamp, up (bool), latency_ms, cpu_pct, ram_pct, iops, error_code	PK probe_id; FK provider_id; immutable
ProviderPriceList	price_id, provider_id, tier, vcpu_h_usd, ram_gb_h_usd, storage_gb_day_usd, egress_gb_usd, effective_from, superseded_by	PK price_id; immutable after publish; sync z CGP (Plan 10)

SCHEMAT YAML — provider.yaml

YAML

```
apiVersion: sylion.io/v3
kind: Provider
metadata:
  provider_id: "pvd-pro-01"
  name: "Hetzner PRO AX162"
  tier: PRO
spec:
  region: "EU-West-1"
  vcpu_total: 32
  ram_gb_total: 128
  storage_gb_total: 500
  price_vcpu_h_usd: 0.09
  sla_uptime_pct: 99.9
  implementation_strategy: reuse
  current_baseline: MZ 3.5.5
  target_baseline: null
  cutover_strategy: null
  rebuild_role: none
  health_probe:
```

```
interval_s: 30
fail_threshold: 3
hard_block_threshold: 10
hard_block_window_min: 60
certification:
  stages_completed: [A3.1, A3.2, A3.3, A3.4, A3.5, A3.6, A3.7]
  certified_at: "2025-09-15T12:00:00Z"
  tier_assigned_by: "D3-COUNCIL-00042"
```

Provider TARGET w REBUILD_MODE

YAML

```
apiVersion: sylion.io/v3
kind: Provider
metadata:
  provider_id: "pvd-phantom-target-01"
  name: "Hetzner PHANTOM Target AX262"
  tier: PHANTOM
spec:
  implementation_strategy: greenfield
  current_baseline: null
  target_baseline: "pvd-pro-01"
  cutover_strategy: shadow
  rebuild_role: target
  rebuild_state: SHADOW_RUNNING
  shadow_traffic_pct: 100
  comparison_run_id: "run-rebuild-042"
```

STANY REBUILD — diagram przejść

Stan	Opis	Wyzwalacz wejścia	Wyzwalacz wyjścia
PROVISIONING_TARGET	Provider target jest konfigurowany równolegle z legacy	D4 Human Gate + Plan 16 trigger	Certyfikacja A3.1–A3.7 zakończona
SHADOW_RUNNING	Provider target obsługuje ruch shadow (100% kopia legacy)	A3.7 passed + D3 Council	Cutover decision D4
CUTOVER_PENDING	Zatwierdzone przełączenie; oczekuje na okno maintenances	D4 Human Gate approve cutover	Przełączenie ruchu; legacy → review

WORKFLOWY

WF-VPS-01: Certyfikacja nowego providera

- Operator wysyła POST /vps/v1/providers z danymi providera (tier, region, zasoby)
- VPS tworzy rekord Provider w stanie candidate i inicjuje A3.1 (auto)
- A3.1–A3.4 wykonywane automatycznie; wyniki zapisywane w CertificationStage
- A3.5: Chief Safety/Council review polityk compliance (D3, timeout 48h)
- A3.6: Load test 72h; jeśli fail → blocked; jeśli pass → A3.7
- A3.7: D3 Council + D4 Human Gate przypisują tier i PriceList; stan → approved

WF-VPS-02: Alokacja zasobów dla runu

- 1. Plan 11 (A3) wysyła POST /vps/v1/allocations z run_id i wymaganym tierm
- 2. VPS sprawdza available capacity providerów ze stanem approved i tierm ≥ wymaganym
- 3. VPS wybiera providera z najniższym load (G-VPS-01: brak approved providera → block)
- 4. VPS tworzy Allocation, zmniejsza available capacity, zwraca alloc_id
- 5. Po zakończeniu runu Plan 11 wywołuje DELETE /vps/v1/allocations/{id} → zwolnienie zasobów

WF-VPS-03: Rebuild — PROVISIONING_TARGET → SHADOW_RUNNING

- 1. Plan 16 (canon_compact.md) emituje trigger rebuild; ACP przechodzi w REBUILD_MODE
- 2. Operator tworzy nowy provider z rebuild_role=target, tier=PHANTOM
- 3. Provider przechodzi certyfikację A3.1–A3.7; stan → PROVISIONING_TARGET
- 4. Po certyfikacji D3 Council: stan → SHADOW_RUNNING; ruch shadow 100%
- 5. Porównanie wyników shadow vs legacy przez 72h minimum
- 6. D4 Human Gate approve: stan → CUTOVER_PENDING; następnie przełączenie ruchu

API
YAML
POST /vps/v1/providers Body: ProviderDefinition YAML/JSON Response 201: { "provider_id": "uuid", "state": "candidate" } GET /vps/v1/providers/{id} Response 200: Provider JSON z aktualnym stanem i metrykami POST /vps/v1/providers/{id}/certify Body: { "stage_code": "A3.5", "decision": "pass fail", "reviewer_id": "uuid" } Response 200: { "stage_code": "A3.5", "state": "passed", "next": "A3.6" } POST /vps/v1/allocations Body: { "run_id": "uuid", "tier_min": "PRO", "vcpu": 4, "ram_gb": 16 } Response 200: { "alloc_id": "uuid", "provider_id": "uuid", "state": "active" } Response 503: { "error": "no_approved_provider", "gate": "G-VPS-01" } DELETE /vps/v1/allocations/{id} Response 200: { "alloc_id": "uuid", "state": "released" } POST /vps/v1/providers/{id}/rebuild/shadow Body: { "legacy_provider_id": "uuid", "cutover_strategy": "shadow" } Response 200: { "state": "SHADOW_RUNNING", "shadow_traffic_pct": 100 } POST /vps/v1/providers/{id}/rebuild/cutover Body: { "operator_d4_token": "JWT", "maintenance_window": "IS08601" } Response 200: { "state": "CUTOVER_PENDING", "scheduled_at": "IS08601" }
HARD GATE'Y

Kod	Nazwa	Warunek FAIL	Klasa	Blokuje
G-VPS-01	Brak approved providera	Żądanie alokacji — brak providerów w stanie approved z wymaganym tierm	D0	run A3 stage (Plan 11)
G-VPS-02	Certyfikacja sekwencja	Próba przejścia do A3.N+1 gdy A3.N nie zakończony sukcesem	D0	certyfikacja
G-VPS-03	Alokacja przekracza capacity	Żądana liczba vCPU lub RAM przekracza available_capacity providera	D0	alokacja
G-VPS-04	HealthProbe failures	3 kolejne błędy HealthProbe → provider → review; blokada nowych alokacji	D0 auto	nowe alokacje na tym providerze
G-VPS-05	Certyfikacja A3.5 (compliance)	Audyt compliance A3.5 zakończony wynikiem fail → provider blocked	D3	certyfikacja
G-VPS-06	Shadow bez certyfikacji	Próba uruchomienia SHADOW_RUNNING bez pełnej certyfikacji A3.1–A3.7	D3	rebuild shadow
G-VPS-07	Cutover bez D4	Przejście CUTOVER_PENDING bez decyzji D4 Human Gate	D4	cutover
TESTY				
Kod	Opis	Input	Oczekiwany wynik	
T-VPS-01	Rejestracja providera	POST /providers, tier=PRO, wszystkie pola	201, state=candidate, A3.1 uruchomiony	
T-VPS-02	Certyfikacja sekwencja	Wymuś A3.2 przed A3.1	400, G-VPS-02 FAIL	
T-VPS-03	Alokacja PRO	POST /allocations tier_min=PRO, approved provider istnieje	200, alloc_id, state=active	
T-VPS-04	Brak providera	POST /allocations tier_min=STATE, brak approved STATE	503, G-VPS-01, run blocked	
T-VPS-05	HealthProbe failures	Symulacja 3 failed probes	provider → review, nowe alokacje zablokowane, alert !syllion-governance	
T-VPS-06	Shadow bez certyfikacji	POST /rebuild/shadow bez A3.7	403, G-VPS-06 FAIL	
T-VPS-07	Cutover flow	PROVISIONING_TARGET → SHADOW_RUNNING → CUTOVER_PENDING → approved	każde przejście z poprawną decyzją D3/D4; provider target staje się primary	
ANEKS A — Tiery × Stany × Ścieżki rebuild				
Tier	Stany dozwolone	Rebuild role	Ścieżka	Klasa D przy certyfikacji
STANDARD	candidate/qualified/approved/review/blocked	legacy	evolution	D3 (A3.5–A3.7)

<i>Tier</i>	<i>Stany dozwolone</i>	<i>Rebuild role</i>	<i>Ścieżka</i>	<i>Klasa D przy certyfikacji</i>
PRO	candidate/qualified/approved/ review/blocked	legacy/target	refactor	D3 (A3.5–A3.7)
STATE	candidate/qualified/approved/ review/blocked	legacy/target	refactor/rebuild	D3+D4 (A3.7)
PHANTOM	candidate/qualified/approved/ review/blocked + rebuild states	target/shadow	rebuild/greenfield	D4 (A3.7) + external
<i>ANEKS B — D0–D5 w kontekście VPS</i>				
<i>Klasa</i>	<i>Przykład operacji VPS</i>	<i>Aktor</i>	<i>Ścieżka</i>	
D0	HealthProbe auto-review, alokacja	VPS auto	—	
D1	Przegląd wyników A3.3 Security Scan	Chief Safety	—	
D2	Certyfikacja A3.6 Load Test OK	Council 3/4	evolution	
D3	Certyfikacja A3.5/A3.7; rebuild shadow approval	Council 4/4	refactor/rebuild	
D4	Cutover pełny; blokada providera trwała	Human Gate	rebuild	
D5	Wymiana całej warstwy VPS (nowy provider fabric)	Council 4/4 + Human + External	rebuild (greenfield)	

PLAN 02 · ROZSZERZENIE v3.1 · REVERSIBILITY

Virtual Provider Substrate · v3.1 EXPAND

2.9. Rozszerzenie v3.1 — asynchroniczny cutover i legacy preservation

REVERSIBILITY · Motywacja rozszerzenia

v3.0 Plan 02 opisywał cutover jako pojedyncze przejście SHADOW → ACTIVE z Human Gate. W praktyce rozdzielenie decyzji technicznej (czy target jest gotowy do przejęcia ruchu) od decyzji operatorskiej (czy MY chcemy go teraz uruchomić) daje lepszą kontrolę i umożliwia asynchroniczny harmonogram cutover niezależny od certyfikacji technicznej.

2.9.1. Nowa maszyna stanów cutover

v3.1 rozbija przejście SHADOW → ACTIVE na sześć odrębnych stanów, każdy z własnym aktorem, własnym warunkiem przejścia i własną klasą decyzji. Nowy model jest supersetem v3.0: stare stany SHADOW_RUNNING i CUTOVER_PENDING zostają, ale pomiędzy nimi wstawiony jest CUTOVER_READY (auto-certyfikacja techniczna) oraz po ACTIVE dwa nowe stany cyklu życia legacy — STABILIZATION i WARM_STANDBY — zanim dojdzie do LEGACY_DECOMMISSION.

Stan	Aktor	Warunek wejścia	Czas trwania	Klasa D
SHADOW_RUNNING	VPS auto	target zaalokowany, legacy aktywne, brak ruchu na target	do zakończenia walidacji	D0
SHADOW_VALIDATING	Testy + Red Team	replay ruchu legacy → target, compare outputs, contract tests	24–72h	D2
CUTOVER_READY	VPS auto + Architekt	wszystkie bramy techniczne pass (contract, load, security, chaos)	indefinite (czeka na operatora)	D2
CUTOVER_PENDING	Human Gate (operator)	operator zatwierdza okno cutover; ogłoszenie do wszystkich działów	minuty–godziny (window)	D3 (D4 dla PHANTOM)
CUTOVER_ACTIVE	VPS auto + Blue Team watch	ruch przełączony target ← legacy, write-through replication aktywna	minuty (atomic switch)	D3+
STABILIZATION	Blue Team + Platforma	target serwuje ruch, legacy utrzymuje write-through + HOT_STANDBY	D3: 24h · D4: 72h · D5: 168h	obserwacja
WARM_STANDBY	Platforma	legacy odłączony od write-through, zachowany jako ciepła rezerwa	7–30 dni	obserwacja
LEGACY_DECOMMISSION	Human Gate + Council	osobna decyzja D3+ z listą kontrolną, NIGDY automatyczna	jednorazowa	D3+ (nowa decyzja)

ZASADA · Kluczowe rozgraniczenie: CUTOVER_READY (auto) vs CUTOVER_PENDING (human)

CUTOVER_READY jest bramą techniczną — VPS mówi „jeśli chcesz, teraz jest bezpiecznie przełączyć”. CUTOVER_PENDING jest bramą operatorską — operator mówi „tak, teraz przełączamy, oto okno”. W v3.0 obie decyzje były sklejone. v3.1 rozdziela je, bo sam fakt gotowości technicznej NIE jest zgodą na użycie: piątek 17:00, sezon handlowy albo trwający incydent to legalne powody odroczenia cutover nawet gdy target przeszedł wszystkie testy.

2.9.2. HOT_STANDBY vs WARM_STANDBY — rola legacy po cutover

W v3.1 legacy NIGDY nie jest natychmiast wyłączane po cutover. Przechodzi przez dwa etapy wycofania, każdy z inną charakterystyką kosztową i rollback-time:

Etap	Synchronizacja	Rollback time	Koszt utrzymania	Aktywność
HOT_STANDBY (STABILIZATION)	write-through replication (writes × 2)	< 60s (atomic swap back)	~100% legacy + 100% target	legacy serwuje read-only query dla testów; pełna log parity
WARM_STANDBY	odłączone; cold snapshot + WAL archive	15–60 min (replay WAL)	~20% legacy (idle capacity) + 100% target	legacy uruchomione ale bez ruchu; gotowe do uruchomienia
LEGACY_DECOMMISSION	brak; dane zarchiwizowane zgodnie z Planem 03	niemożliwy bez rebuild	0 (poza archiwum)	infrastruktura zwolniona, provider freed

2.9.3. Write-through replication w STABILIZATION

W fazie STABILIZATION każdy write na target MUSI być jednocześnie replikowany na legacy. Zapewnia to że w razie auto-rollback (patrz Plan 07) dane zmutowane na target po cutover są dostępne także na legacy. Replikacja jest synchroniczna dla transakcji krytycznych (oznaczonych w manifestach modułów polem consistency=strong) i asynchroniczna dla reszty.

```
# Pseudokod replikacji STABILIZATION (VPS runtime)
def handle_write(request):
    ack_target = target.write(request) # primary
    if request.consistency == 'strong':
        ack_legacy = legacy.write(request, timeout=200ms) # sync mirror
        if not ack_legacy:
            metrics.emit('legacy_mirror_lag', request.id)
            # NOT rollback – just alarm; target stays primary
        else:
            legacy.write_async(request) # fire-and-forget
    return ack_target
```

2.9.4. Nowe flagi manifestu providera

Manifest VPS dla każdego providera rozszerza się w v3.1 o cztery pola opisujące politykę cyklu życia cutover/rollback:

```
# providers/<id>/manifest.yaml – sekcja v3.1
cutover_policy:
  stabilization_window: 72h          # D3:24h | D4:72h | D5:168h
  warm_standby_window: 14d          # 7d..30d
  write_through_mode: strong_only   # strong_only | all | disabled
  auto_rollback_enabled: true       # patrz Plan 07
rollback_policy:
  stabilization_rollback: auto_fast  # D1/D2 auto, D3+ Council
  warm_standby_rollback: council_fast # D3 Council 4/4
  decommission_reversibility: none   # NIGDY auto
```

KRYTYCZNE · *LEGACY_DECOMMISSION nigdy automatyczna*

Przejście z WARM_STANDBY do LEGACY_DECOMMISSION zawsze wymaga osobnej decyzji D3+ z pełną listą kontrolną (patrz Plan 13, G-CUT-04). Żaden harmonogram, żaden timeout, żaden skrypt NIE może samoczynnie zdecydować legacy. Ta granica jest gwarantem że rebuild zawsze ma drogę powrotu.

2.9.5. Aktualizacja diagramu tierów

Tiery STANDARD, PRO, STATE i PHANTOM zostają niezmienione, ale ich zbiór stanów dozwolonych rozszerza się o nowe stany v3.1:

Tier	Dozwolone nowe stany v3.1	Uwagi
STANDARD	+ SHADOW_VALIDATING + STABILIZATION + WARM_STANDBY	stabilization 24h wystarczy
PRO	+ wszystkie nowe stany v3.1	stabilization 72h rekomendowany
STATE	+ wszystkie nowe stany v3.1 + HOT_STANDBY obowiązkowy	write-through strong_only wymagany
PHANTOM	+ wszystkie nowe stany v3.1 + WARM_STANDBY min. 30 dni	LPW 168h, external review dla DECOMMISSION

PLAN 03 · WARSTWA KANONICZNA

Knowledge Base Plane

REBUILD · Rebuildability over Lineage w tym planie

KB v3.0 wprowadza wzorzec dual-view: legacy_view (widok bazy wiedzy w stanie MZ 3.5.5) i target_view (widok po przebudowie) działające równolegle podczas REBUILD_MODE. Propagacja do Matrix zachowuje spójność obu widoków. Pełne przełączenie na target_view następuje dopiero po walidacji przez Council (D3).

3.1. Cel i umiejscowienie

Knowledge Base Plane (KBP) jest centralnym magazynem wiedzy systemu SYLION. Przechowuje dokumenty, specyfikacje, logi, lekcje i dowody audytowe jako niemutowalne wpisy z sygnaturą SHA-256. KBP służy jako pierwszorzędne źródło kontekstu dla Chief Context (Gemini 2.5 Pro) podczas fazy A1 każdego runu (Plan 11). Semantyczne wyszukiwanie oparte na SQLite FTS5 i embeddingach nomic-embed-text umożliwia precyzyjne pobieranie wiedzy przez model.

- SQLite FTS5 — pełnotekstowe wyszukiwanie z BM25
- nomic-embed-text 768-dim — semantyczne embeddingi dokumentów
- RRF (Reciprocal Rank Fusion) — łączenie wyników FTS5 i semantycznych
- 6 rodzajów dokumentów: doc / spec / log / incident / lesson / audit
- SHA-256 na każdym wpisie — niemutowalność i hash chain
- Propagacja do Matrix (Plan 13) — kanoniczny kanał zdarzeń KB

3.2. Rodzaje dokumentów

Każdy dokument w KB ma przypisany kind determinujący czas retencji, poziom dostępu i strategię propagacji. Kind jest niemutowalny po zapisie.

Kind	Opis	Retencja	Dostęp	Propagacja
doc	Dokumentacja ogólna, przewodniki, README	2 lata	ReadOnly+	Matrix opcjonalna
spec	Specyfikacje techniczne, schematy YAML/JSON	infinite	CanonCust+	Matrix obowiązkowa
log	Logi runów, logi etapów A0-A5/C0-C9	90 dni hot	PipelineOp+	Matrix auto
incident	Raporty incydentów bezpieczeństwa, postmortemy	5 lat	SecOfficer+	Matrix obowiązkowa
lesson	Lessons learned z runów — wejście do pętli feedback Plan 07	2 lata	CanonCust+	Matrix obowiązkowa
audit	Dowody audytowe — kopie z Plan 13	infinite	Auditor+	Matrix obowiązkowa

3.3. Wyszukiwanie — RRF Fusion

Każde zapytanie do KB jest przetwarzane przez trzy ścieżki równoległe, a wyniki łączone przez RRF (Reciprocal Rank Fusion) z wagami konfigurowanymi per zapytanie. RRF zapewnia, że dokumenty trafne zarówno tekstowo jak i semantycznie są rankinowane wyżej niż trafne tylko w jednej ścieżce.

- Ścieżka 1: SQLite FTS5 + BM25 ranking (waga 0.4)
- Ścieżka 2: semantyczne embeddingi nomic-embed-text 768-dim + cosine similarity (waga 0.4)
- Ścieżka 3: filtry metadanych (kind, date_range, author) (waga 0.2)
- RRF formula: $\text{score}(d) = \sum 1/(k + \text{rank}_i(d))$; $k=60$
- Top-K domyślnie 10; konfigurowalne per zapytanie (max 50)

3.4. Dual-view pattern — Rebuildability

W trybie REBUILD_MODE (Plan 01, M-03) KB utrzymuje dwa równoległe widoki: legacy_view (stan bazowy MZ 3.5.5) i target_view (nowa struktura KB po przebudowie). Zapisy trafiają do obu widoków. Odczyty domyślnie z legacy_view; target_view dostępny przez parametr view=target w API. Po pomyślnej walidacji Council (D3) następuje przełączenie na target_view i decommission legacy_view (D4).

Widok	Stan	Zapis	Odczyt domyślny	Wyzwalacz przełączenia
legacy_view	LEGACY_ACTIVE	Tak (primary)	Tak	Cutover decision D4
target_view	TARGET_BUILDING → TARGET_ACTIVE	Tak (shadow)	Nie (explicite view=target)	D3 Council validate

3.5. Propagacja do Matrix

Każdy zapis do KB jest propagowany do odpowiedniego room'u Matrix (Plan 13). Propagacja jest synchroniczna dla kindów spec, incident, lesson, audit (blokuje zapis w przypadku błędu Matrix) i asynchroniczna dla doc i log. W trybie dual-view oba widoki propagują do oddzielnych room'ów Matrix.

- !sylon-kb-specs: dokumenty kind=spec
- !sylon-kb-lessons: dokumenty kind=lesson
- !sylon-kb-incidents: dokumenty kind=incident
- !sylon-kb-audit: dokumenty kind=audit
- !sylon-kb-target: dokumenty target_view podczas REBUILD_MODE

PLAN 03 · SPECYFIKACJA WYKONAWCZA

Knowledge Base Plane

CEL I ZAKRES

Specyfikacja KB Service (KBS) v3.0. Zakres: model domenowy dokumentów, schematy YAML z polami v3.0 (view, implementation_strategy), workflowy zapisu i wyszukiwania, dual-view pattern (legacy_view + target_view), propagacja Matrix, gate'y jakości i testy. Wersja: Pipeline 8.0.0, MZ baseline 3.5.5.

MODEL DOMENOWY		
Encja	Kluczowe pola	Klucze / relacje
Document	doc_id (UUID), kind, title, content_sha256, author_id, created_at, superseded_by (nullable), view (legacy/target), matrix_event_id, embedding_id	PK doc_id; niemutowalny po zapisie; FK superseded_by->Document
Embedding	embedding_id, doc_id, model (nomic-embed-text), dims (768), vector (BLOB), created_at	PK embedding_id; FK doc_id; jeden per dokument
SearchQuery	query_id, run_id, query_text, embedding_used (bool), fts_used (bool), k_requested, results_count, latency_ms, created_at	PK query_id; FK run_id
KBView	view_id, view_type (legacy/target), state, created_at, activated_at, decommissioned_at, matrix_room_id	PK view_id; jeden legacy + jeden target max
PropagationLog	prop_id, doc_id, view_type, matrix_room_id, matrix_event_id, state (pending/sent/failed), sent_at	PK prop_id; FK doc_id; immutable
SCHEMAT YAML — kb_document.yaml		

YAML

```
apiVersion: sylion.io/v3
kind: KBDocument
metadata:
  doc_id: "doc-spec-00421"
  kind: spec
  title: "ACP RBAC Engine – specyfikacja v3.0"
  author_id: "op-001"
  view: legacy
spec:
  content_sha256: "sha256:a1b2c3..."
  created_at: "2025-09-01T12:00:00Z"
  superseded_by: null
  tags: [acp, rbac, v3]
  propagate:
    matrix_room: "!sylion-kb-specs"
    mode: synchronous
  implementation_strategy: reuse
```

Dokument target_view podczas REBUILD_MODE

YAML

```

apiVersion: sylion.io/v3
kind: KBDocument
metadata:
  doc_id: "doc-spec-00421-target"
  kind: spec
  title: "ACP RBAC Engine – specyfikacja v3.0 (TARGET VIEW)"
  view: target
spec:
  content_sha256: "sha256:d4e5f6..."
  target_baseline: "acp-auth-gateway-v3"
  cutover_strategy: shadow
  propagate:
    matrix_room: "!sylion-kb-target"
    mode: synchronous

```

WORKFLOWY

WF-KB-01: Zapis dokumentu

1. Autor (CanonCust+) wysyła POST /kb/v1/documents z treścią i metadanymi
2. KBS oblicza SHA-256 treści; jeśli duplikat → 409 Conflict (G-KB-01)
3. KBS generuje embedding (nomic-embed-text 768-dim) i zapisuje w Embedding
4. KBS zapisuje Document z view=legacy (domyślnie) lub view=target
5. KBS propaguje do odpowiedniego room'u Matrix (synchronicznie dla spec/incident/lesson/audit)
6. KBS rejestruje PropagationLog z matrix_event_id

WF-KB-02: Wyszukiwanie RRF

1. Chief Context (A1 w Plan 11) wysyła POST /kb/v1/search z query_text i opcjonalnym view
2. KBS równolegle: FTS5 BM25 query + embedding nomic-embed-text + metadata filter
3. KBS łączy rankingi przez RRF (k=60), wagi: FTS5 0.4, semantic 0.4, metadata 0.2
4. KBS zwraca top-K dokumentów (domyślnie 10) z scores
5. W trybie dual-view: domyślnie legacy_view; jeśli view=target → z target_view
6. KBS zapisuje SearchQuery z metrykami latencji

WF-KB-03: Aktywacja target_view

1. Council D3 zatwierdza walidację target_view po minimum 2 runach shadow
2. KBS ustawia KBView(view_type=target, state=TARGET_ACTIVE)
3. API przełącza domyślny odczyt na target_view
4. legacy_view przechodzi do stanu LEGACY_DECOMMISSIONED (D4 Human Gate)
5. Zdarzenie view_switch emitowane do Plan 13 (!sylion-governance)

API

YAML

```

POST /kb/v1/documents
Body: { "kind": "spec", "title": "str", "content": "str",

```

```

    "view": "legacy|target", "tags": [] }
Response 201: { "doc_id": "uuid", "content_sha256": "sha256:..." }
Response 409: { "error": "duplicate_sha256" }

GET /kb/v1/documents/{id}
Response 200: Document JSON

POST /kb/v1/search
Body: { "query_text": "str", "k": 10, "kind_filter": ["spec","lesson"],
        "view": "legacy|target" }
Response 200: { "results": [{ "doc_id": "...", "score": 0.87, "title": "..."}],
                "latency_ms": 45 }

POST /kb/v1/views/activate-target
Body: { "council_decision_id": "uuid", "cutover_strategy": "shadow" }
Response 200: { "view_id": "uuid", "state": "TARGET_ACTIVE" }

GET /kb/v1/views
Response 200: { "legacy": {...}, "target": {...} }

```

HARD GATE'Y				
Kod	Nazwa	Warunek FAIL	Klasa	Blokuje
G-KB-01	SHA-256 unikalność	Próba zapisu dokumentu z identycznym content_sha256 (duplikat)	D0	zapis dokumentu
G-KB-02	Kind immutable	Próba zmiany kind istniejącego dokumentu	D0	aktualizacja dokumentu
G-KB-03	Propagacja Matrix synchroniczna	Matrix homeserver niedostępny podczas zapisu kind=spec/incident/lesson/audit	D0	zapis dokumentu (retry 3x, potem block)
G-KB-04	Target view bez Council	Próba aktywacji target_view bez decyzji D3 Council	D3	aktywacja target_view
G-KB-05	Decommission bez D4	Próba decommission legacy_view bez decyzji D4 Human Gate	D4	decommission view
G-KB-06	Zapis target view bez REBUILD_MODE	Próba zapisu z view=target gdy ACP nie jest w REBUILD_MODE	D3	zapis do target_view
TESTY				
Kod	Opis	Input	Oczekiwany wynik	
T-KB-01	Zapis spec + propagacja	POST /documents kind=spec, Matrix dostępny	201, doc_id, matrix_event_id, PropagationLog state=sent	
T-KB-02	Duplikat SHA-256	Dwa zapisy z identyczną treścią	409, G-KB-01 FAIL na drugiej próbie	
T-KB-03	Wyszukiwanie RRF	POST /search query_text='RBAC role policy', k=5	200, 5 wyników z scores, latency_ms zapisane	

Kod	Opis	Input	Oczekiwany wynik		
T-KB-04	Zapis target view	POST /documents view=target, REBUILD_MODE aktywny	201, doc zapisany w target_view, propagacja do !syllion-kb-target		
T-KB-05	Zapis target bez REBUILD_MODE	POST /documents view=target, tryb NORMAL	403, G-KB-06 FAIL		
T-KB-06	Aktywacja target_view	POST /views/activate-target z council_decision_id D3	200, state=TARGET_ACTIVE, odczyt domyślny → target		
T-KB-07	Decommission bez D4	Decommission legacy_view bez decyzji D4	403, G-KB-05 FAIL		
ANEKS A — Kind × Retencja × Propagacja × View					
Kind	Retencja hot	Retencja cold	Propagacja Matrix	Tryb propagacji	Dual-view
doc	2 lata	n/a	!syllion-kb-specs	async	Nie
spec	infinite	infinite	!syllion-kb-specs	sync	Tak
log	90 dni	1 rok	—	async	Nie
incident	5 lat	infinite	!syllion-kb-incidents	sync	Tak
lesson	2 lata	5 lat	!syllion-kb-lessons	sync	Tak
audit	infinite	infinite	!syllion-kb-audit	sync	Tak

PLAN 04 · WARSTWA KANONICZNA

Secrets Vault Plane

REBUILD · Rebuildability over Lineage w tym planie

Vault v3.0 wprowadza wzorce migracji sekretów: podczas REBUILD_MODE sekrety są dostępne zarówno przez legacy_vault jak i target_vault bez konieczności rotacji (vault_migration_pattern: no_rotation). Klucze sygnatury (signing_key) migrowane są przez protocol dual-sign — oba klucze są aktywne przez okres przejściowy. Pełna decommission legacy vault następuje po D4.

4.1. Cel i umiejscowienie

Secrets Vault Plane (SVP) jest scentralizowanym magazynem sekretów systemu SYLION. Przechowuje klucze API, poświadczenia baz danych, klucze podpisywania, tokeny webhooków i klucze PGP. Każdy sekret jest szyfrowany AES-256-GCM z DEK (Data Encryption Key) opakowywany przez KEK (Key Encryption Key) w schemacie envelope encryption. KEK jest wyprowadzany przez Argon2id z master passphrase przechowywanej offline. SVP jest jedynym serwisem zdolnym do odczytu sekretów w trybie plaintext.

- Szyfrowanie: AES-256-GCM per sekret z unikalnym DEK
- Envelope encryption: DEK opakowywany przez KEK (Argon2id z master passphrase)
- 5 klas sekretów: api_key / db_cred / signing_key / webhook_token / pgp_key
- Emergency Seal: SecOfficer R-01 może zapieczętować cały Vault (D3+D4)
- Rotacja kluczy: planowa (TTL-based) i nagła (breach-triggered)
- Integracja z ACP (Plan 01): klucz publiczny ECDSA P-384 weryfikowany przez Vault

4.2. Klasy sekretów

Każdy sekret ma przypisaną klasę determinującą politykę rotacji, TTL, dostęp i procedurę vault_migration_pattern podczas REBUILD_MODE.

Klasa	Opis	TTL domyślny	Rotacja	Vault migration pattern
api_key	Klucze API do zewnętrznych serwisów modeli (Gemini, OpenAI, Anthropic)	90 dni	Planowa TTL-based	no_rotation (dual-access)
db_cred	Poświadczenia baz danych (PostgreSQL, SQLite, Redis)	30 dni	Planowa + breach-triggered	no_rotation (dual-access)
signing_key	Klucze ECDSA P-384 do JWT i Ed25519 do bundle signatures	365 dni	Planowa + breach-triggered	dual_sign (overlap 30 dni)
webhook_token	Tokeny do webhooków przychodzących (GitHub, CI/CD, monitoring)	180 dni	Planowa TTL-based	no_rotation (dual-access)

Klasa	Opis	TTL domyślny	Rotacja	Vault migration pattern
pgp_key	Klucze GPG do podpisywania bundle VPS i G1	365 dni	Breach-triggered only	dual_sign (overlap 14 dni)

4.3. Szyfrowanie — envelope encryption

Każdy sekret jest szyfrowany indywidualnym DEK (Data Encryption Key, 256-bit losowy). DEK jest następnie opakowywany (wrapped) przez KEK (Key Encryption Key) wyprowadzony przez Argon2id ($N=2^{20}$, $r=8$, $p=1$) z master passphrase. Zaszyfrowany DEK (encrypted_dek) jest przechowywany razem z zaszyfrowanym sekretem. Odszyfrowanie wymaga dostępu do master passphrase — dostępnej tylko offline w sejfie operatora i w HSM backup.

- DEK generowany: `crypto.randomBytes(32)` per sekret
- Szyfrowanie sekretu: `AES-256-GCM(dek, plaintext) → ciphertext + iv + auth_tag`
- Wrapping DEK: `AES-256-GCM(kek, dek) → encrypted_dek`
- Argon2id parameters: $N=1048576$, $r=8$, $p=1$, $\text{saltLen}=32$, $\text{keyLen}=32$
- Master passphrase: offline only + HSM backup; nie w pamięci serwera >5 min

4.4. Emergency Seal

Emergency Seal to procedura natychmiastowego zapieczętowania całego Vault. Po uruchomieniu żadne nowe operacje odczytu sekretów nie są możliwe. Wszystkie aktywne serwisy otrzymują `EventType=VAULT_SEALED` i muszą przejść do trybu LOCKDOWN (Plan 01). Emergency Seal może uruchomić tylko SecOfficer (R-01) z potwierdzeniem D3 Council lub jako decyzja D4 Human Gate.

REBUILD · Emergency Seal — procedura krytyczna

Emergency Seal jest nieodwracalny bez decyzji D4 i fizycznego dostępu do master passphrase. Po uruchomieniu Vault przechodzi w stan SEALED — żadna automatyzacja nie może go odpieczętować. Procedura unseal wymaga fizycznej obecności operatora z offline passphrase.

4.5. Vault Migration Pattern w REBUILD_MODE

Podczas REBUILD_MODE (Plan 01) sekrety są dostępne przez dwa równoległe endpointy: `legacy_vault` i `target_vault`. Wzorzec migracji zależy od klasy sekretu:

Pattern	Klasy	Opis	Rotacja podczas rebuild?
no_rotation	api_key, db_cred, webhook_token	Sekret dostępny przez oba endpointy bez zmian; ten sam plaintext po odszyfrowaniu	Nie — minimalizacja dysrupcji

<i>Pattern</i>	<i>Klasy</i>	<i>Opis</i>	<i>Rotacja podczas rebuild?</i>
dual_sign	signing_key, pgp_key	Oba klucze (legacy + target) aktywne przez okres overlap; podpisy obu kluczy akceptowane przez weryfikatory	Nie — nowy klucz generowany obok starego
cut_and_replace	Dowolna klasa (breach)	Natychmiastowa rotacja; stary klucz revoked; wymagana D4 i aktualizacja wszystkich konsumentów	Tak — wymagane przez incydent

PLAN 04 · SPECYFIKACJA WYKONAWCZA

Secrets Vault Plane

CEL I ZAKRES

Specyfikacja Vault Service (VS) v3.0. Zakres: model domenowy sekretów, envelope encryption (AES-256-GCM + Argon2id), schematy YAML z polami v3.0 (vault_migration_pattern, dual_sign_overlap_days), workflowy CRUD sekretów, Emergency Seal, rotacja, gate'y jakości i testy. Wersja: Pipeline 8.0.0.

MODEL DOMENOWY		
Encja	Kluczowe pola	Klucze / relacje
Secret	secret_id (UUID), name, class (api_key/db_cred/signing_key/webhook_token/pgp_key), ciphertext_b64, iv_b64, auth_tag_b64, encrypted_dek_b64, created_at, expires_at, rotated_at, revoked, version, vault_migration_pattern, dual_sign_peer_id (nullable)	PK secret_id; immutable content fields; FK dual_sign_peer_id->Secret
SecretVersion	version_id, secret_id, version, created_at, superseded_at, reason (rotation/breach/rebuild)	PK version_id; FK secret_id; historia wersji
AccessLog	access_id, secret_id, requester_id, requester_type (operator/agent/service), action (read/rotate/revoke/seal), timestamp, result (ok/fail), run_id (nullable), prev_hash, hash	PK access_id; immutable; hash chain
VaultState	state_id, state (OPEN/SEALED/REBUILD_DUAL), changed_at, changed_by, reason, decision_class	PK state_id; jeden aktywny rekord
SCHEMAT YAML — secret.yaml		

YAML

```
apiVersion: sylion.io/v3
kind: Secret
metadata:
  secret_id: "sec-signing-acp-01"
  name: "ACP JWT Signing Key ECDSA P-384"
  class: signing_key
spec:
  algorithm: ES384
  expires_at: "2026-09-01T00:00:00Z"
  implementation_strategy: reuse
  vault_migration_pattern: dual_sign
  dual_sign_overlap_days: 30
  dual_sign_peer_id: null
  rotation_policy:
    trigger: ttl_based
    warning_days_before: 30
    auto_rotate: false
    requires_class: D3
```

Sekret target podczas dual_sign REBUILD_MODE

YAML

```

apiVersion: sylion.io/v3
kind: Secret
metadata:
  secret_id: "sec-signing-acp-02-target"
  name: "ACP JWT Signing Key ECDSA P-384 (TARGET)"
  class: signing_key
spec:
  vault_migration_pattern: dual_sign
  dual_sign_peer_id: "sec-signing-acp-01"
  dual_sign_overlap_days: 30
  rebuild_role: target
  cutover_strategy: parallel
  active_from: "2025-10-01T00:00:00Z"
  legacy_revoke_at: "2025-10-31T00:00:00Z"

```

WORKFLOWY

WF-VAULT-01: Zapis sekretu

1. Operator (SecOfficer R-01) wysyła POST /vault/v1/secrets z plaintext i metadanymi
2. VS generuje DEK (32 losowe bajty) i szyfruje plaintext: AES-256-GCM(dek, plaintext)
3. VS pobiera KEK z Argon2id(master_passphrase, salt); wrappuje DEK
4. VS zapisuje Secret z ciphertext_b64, iv_b64, auth_tag_b64, encrypted_dek_b64
5. VS czyści DEK i KEK z pamięci (zerowanie bufora)
6. VS zapisuje AccessLog event=write z hash chain

WF-VAULT-02: Odczyt sekretu

1. Serwis (ACP, BS, itp.) wysyła GET /vault/v1/secrets/{id}/decrypt z autoryzacją JWT
2. VS sprawdza ACP RBAC: tylko autoryzowane serwisy z odpowiednią rolą
3. VS pobiera master_passphrase z bezpiecznego storage (TTL w pamięci 5 min)
4. VS wyprowadza KEK z Argon2id, odpakowuje DEK, odszyfrowuje sekret
5. VS zwraca plaintext; usuwa KEK i DEK z pamięci po odpowiedzi
6. VS zapisuje AccessLog event=read z run_id i requester_id

WF-VAULT-03: Emergency Seal

1. SecOfficer R-01 wysyła POST /vault/v1/seal z uzasadnieniem (D3 lub D4)
2. VS natychmiast ustawia VaultState.state=SEALED
3. VS emituje zdarzenie VAULT_SEALED do wszystkich serwisów pipeline
4. ACP (Plan 01) przechodzi automatycznie w LOCKDOWN
5. Zdarzenie rejestrowane w Plan 13 (!sylion-governance) z decyzją D3/D4
6. Unseal: wymaga fizycznej obecności operatora z offline passphrase + decyzja D4

WF-VAULT-04: Migracja sekretów w REBUILD_MODE (dual_sign)

1. Operator inicjuje rebuild_mode; Vault przechodzi w stan REBUILD_DUAL

- 2. Dla każdego signing_key: generowany jest nowy target_secret z dual_sign_peer_id=legacy
- 3. Przez okres overlap_days oba klucze są aktywne i akceptowane
- 4. Po cutover D4: legacy_secret.revoked=true; target staje się primary
- 5. AccessLog rejestruje każde użycie obu kluczy podczas overlap

API				
YAML				
<pre>POST /vault/v1/secrets Body: { "name": "str", "class": "signing_key", "plaintext_b64": "base64", "expires_at": "IS08601", "vault_migration_pattern": "dual_sign" } Response 201: { "secret_id": "uuid", "version": 1 }</pre>				
<pre>GET /vault/v1/secrets/{id}/decrypt Headers: Authorization: Bearer <JWT> (SecOfficer+ lub autoryzowany serwis) Response 200: { "plaintext_b64": "base64", "version": 1, "expires_at": "..." } Response 403: { "error": "insufficient_role" } Response 423: { "error": "vault_sealed" }</pre>				
<pre>POST /vault/v1/secrets/{id}/rotate Body: { "new_plaintext_b64": "base64", "reason": "ttl_based breach rebuild" } Response 200: { "secret_id": "uuid", "version": 2, "prev_version": 1 }</pre>				
<pre>POST /vault/v1/seal Body: { "reason": "str", "decision_class": "D3 D4" } Response 200: { "state": "SEALED", "sealed_at": "IS08601" }</pre>				
<pre>POST /vault/v1/unseal Body: { "master_passphrase_hash": "argon2id:...", "d4_token": "JWT" } Response 200: { "state": "OPEN" }</pre>				
<pre>POST /vault/v1/rebuild/dual-sign Body: { "legacy_secret_id": "uuid", "target_plaintext_b64": "base64", "overlap_days": 30 } Response 201: { "target_secret_id": "uuid", "dual_sign_peer_id": "uuid", "active_from": "IS08601", "legacy_revoke_at": "IS08601" }</pre>				
HARD GATE'Y				
Kod	Nazwa	Warunek FAIL	Klasa	Blokuje
G-VAULT-01	Vault Sealed	Jakiegokolwiek żądanie odczytu/zapisu gdy VaultState=SEALED	D0	wszystkie operacje Vault
G-VAULT-02	Wymagana rola SecOfficer	Zapis sekretu przez rolę < R-01 (SecOfficer)	D0	zapis sekretu
G-VAULT-03	Wygasty sekret	Próba odczytu sekretu z expires_at < now	D0	odczyt sekretu (wymaga rotacji)
G-VAULT-04	Rotacja wymaga D3	Rotacja klucza signing_key lub pgp_key bez decyzji D3 Council	D3	rotacja signing_key /pgp_key

Kod	Nazwa	Warunek FAIL	Klasa	Blokuje	
G-VAULT-05	Seal wymaga D3+	Emergency Seal bez decyzji D3 lub D4	D3	Emergency Seal	
G-VAULT-06	Unseal wymaga D4	Próba unseal bez decyzji D4 Human Gate	D4	Unseal Vault	
G-VAULT-07	Dual sign bez REBUILD_MODE	Próba dual_sign bez aktywnego REBUILD_MODE w ACP	D4	dual_sign migration	
TESTY					
Kod	Opis	Input	Oczekiwany wynik		
T-VAULT-01	Zapis api_key	POST /secrets class=api_key, rola R-01	201, secret_id, AccessLog event=write		
T-VAULT-02	Odczyt przez nieautoryzowaną rolę	GET /decrypt, rola R-06	403, G-VAULT-02 FAIL		
T-VAULT-03	Emergency Seal i blokada odczytu	POST /seal D3; GET /decrypt	Seal: 200; Decrypt po seal: 423 vault_sealed, G-VAULT-01		
T-VAULT-04	Unseal bez D4	POST /unseal bez d4_token	403, G-VAULT-06 FAIL		
T-VAULT-05	Rotacja signing_key bez D3	POST /rotate bez council_decision	403, G-VAULT-04 FAIL		
T-VAULT-06	Dual sign flow	POST /rebuild/dual-sign, REBUILD_MODE aktywny, overlap=30	201 target_secret, oba klucze aktywne przez 30 dni		
T-VAULT-07	Wygasty sekret	GET /decrypt, expires_at przeszłe	403, G-VAULT-03 FAIL, komunikat rotation_required		
ANEKS A — Klasy × Szyfrowanie × Migration Pattern					
Klasa	Algo szyfrowania	TTL	Rotacja wymaga	Migration pattern	Overlap
api_key	AES-256-GCM	90 dni	D1	no_rotation	n/a
db_cred	AES-256-GCM	30 dni	D2	no_rotation	n/a
signing_key	AES-256-GCM (ES384 key)	365 dni	D3	dual_sign	30 dni
webhook_token	AES-256-GCM	180 dni	D1	no_rotation	n/a
pgp_key	AES-256-GCM (GPG key)	365 dni	D3	dual_sign	14 dni

PLAN 05 · WARSTWA KANONICZNA

Model & Decision Engine Plane

REBUILD · Rebuildability over Lineage w tym planie

Model Engine v3.0 rozszerza Board o mapowanie 7 departamentów (Plan 15) do wag decyzyjnych. Każdy departament ma przypisane wagi w Board × 7 departments map. Klasy decyzji rozszerzone o D5 (Greenfield/Systemic) — wymagają Council 4/4 + Human Gate + External Review. Ścieżka decyzyjna dla każdej propozycji uwzględnia dimensję rebuild_path.

5.1. Cel i umiejscowienie

Model & Decision Engine Plane (MDEP) jest centrum decyzyjnym pipeline SYLION. Utrzymuje Board złożony z czterech Chief modeli, zarządza klasami decyzji D0-D5, prowadzi reputację modeli (P0-P5) i orkiestruje 4-etapowy onboarding nowych modeli (Discovery → Sandbox → Shadow → Promotion). Każda decyzja klasyfikowana jako D2+ wymaga kworum Council i jest rejestrowana w Governance Plane (Plan 13).

- Board: 4 Chief modele z wagami decyzyjnymi
- Klasy decyzji: D0 (auto) → D5 (Greenfield + External Review) — v3.0
- Reputacja: P0 (blokada) → P5 (najwyższy) per model
- Onboarding: 4-etapowy z bramkami G-MDEP-01 do G-MDEP-08
- Board × 7 departments map: wagi per departament (Plan 15) — nowe v3.0

5.2. Skład Board — Chief modele

Board składa się z czterech stałych ról Chief. Każda rola ma przypisany model, wagę decyzyjną i zakres odpowiedzialności. Zmiana modelu na stanowisku Chief jest decyzją D4 Human Gate. Waga Board × 7 departments map jest nowym elementem v3.0 pozwalającym na ważenie głosów per departament dla specjalistycznych decyzji.

Rola	Model	Waga globalna	Zakres	Zmiana modelu
Chief Context	Gemini 2.5 Pro	1.25	Kontekst domenowy, Księga, historia runów, propagacja wiedzy KB	D4
Chief Reasoning	Claude Opus 4	1.00	Analiza, planowanie, generowanie propozycji, ocena dowodów	D4
Chief Review	GPT-5	1.00	Weryfikacja, Red Team, przypadki brzegowe, regression testing	D4
Chief Safety	GLM-5.1 (z.ai)	1.00	Bezpieczeństwo, compliance, E1-E7 rules (Plan 07), gate'y bezpieczeństwa	D4

5.3. Klasy decyzji D0–D5

System klasyfikuje każdą decyzję do jednej z sześciu klas (D0–D5). Klasa D5 jest nowa w v3.0 i dotyczy decyzji systemowych/architektonicznych wymagających external review (poza standardowym procesem Council).

Klasa	Nazwa	Aktor	Kworum	Opis	Ścieżka rebuild
D0	Auto	System/ACP	n/a	Decyzje deterministyczne, reguły biznesowe, walidacje	—
D1	Single Reviewer	Chief Reviewer	1/1	Analiza single-model; confidence > 0.85	—
D2	Council Majority	Board	3/4 wagowo	Większość ważona Board; akceptacja propozycji standardowych	evolution
D3	Council Unanimous	Board	4/4 wagowo	Jednomyślność Board; krytyczne zmiany architektury i konfiguracji	refactor
D4	Human Gate	Operator SYLION	1 human	Wymagana decyzja człowieka; nieodwracalne zmiany	rebuild
D5	Greenfield/Systemic	Board + Human + External	4/4 + human + external	Pełny rebuild systemu; external review obowiązkowy; kolor: C_D5 (#6A1B9A)	rebuild (greenfield)

5.4. Board × 7 Departments Map — v3.0

W v3.0 Board jest rozszerzone o mapowanie departamentów z Plan 15 (org_structure.yaml). Każdy departament ma własne wagi głosów dla poszczególnych ról Chief, umożliwiając precyzyjne dostosowanie procesu decyzyjnego do specjalizacji departamentu. Mapowanie jest deklarowane w org_structure.yaml (Plan 15) i referowane przez MDEP.

Departament (Plan 15)	Chief Context waga	Chief Reasoning waga	Chief Review waga	Chief Safety waga	Typ decyzji
Architektura	1.50	1.25	0.75	1.00	D3–D5 (dominuje Reasoning+Context)
Platforma	1.00	1.00	1.25	1.25	D2–D4 (Bezpieczeństwo i Review wiodące)
Implementacja	1.25	1.00	1.00	0.75	D2–D3 (Context dominuje)
Testy	0.75	0.75	1.50	1.25	D1–D3 (Review i Safety dominują)
Red Team	0.50	1.00	1.50	1.25	D1–D3 (Review dominuje)

Departament (Plan 15)	Chief Context waga	Chief Reasoning waga	Chief Review waga	Chief Safety waga	Typ decyzji
Blue Team	0.75	0.75	1.00	1.50	D1–D3 (Safety dominuje)
Governance	1.00	1.25	1.00	1.25	D3–D5 (Reasoning i Safety)

REBUILD · Integracja z Plan 15 (org_structure.yaml)

Pełna definicja 7 departamentów, ich składu, ról agentów i polityk eskalacji jest zawarta w Plan 15 (org_structure.yaml). Plan 05 referuje Plan 15 przez department_id i pobiera wagi decyzyjne przy każdym głosowaniu Council.

5.5. Reputacja modeli P0–P5

Każdy model Board posiada reputację na skali P0-P5 aktualizowaną po każdym runie na podstawie 3 strumieni feedbacku (Plan 07): lessons, reputation, drift. Reputacja wpływa na efektywną wagę głosu i dostępność modelu do określonych klas decyzji.

Pozio m	Próg score	Skutek dla modelu	Wymaga D?
P5	≥ 0.90	Pełna waga; dostęp do D4/D5 decyzji	—
P4	0.80–0.89	Pełna waga; dostęp do D3 decyzji	—
P3	0.70–0.79	Waga ×0.85; D2 max	—
P2	0.60–0.69	Waga ×0.70; D1 max; review flag	D2 review
P1	0.50–0.59	Waga ×0.50; tylko D0; shadow mode	D3 requalification
P0	< 0.50	Blokada — model wyłączony z Board	D4 replacement

5.6. Onboarding modelu — 4 etapy

Każdy nowy model przechodzi 4-etapowy onboarding zanim zostanie dopuszczony do Board lub roli Worker/Specialist. Etapy są sekwencyjne i bramkowane.

Etap	Nazwa	Opis	Decyzja	Timeout
E1	Discovery	Dokumentacja modelu, parametry, ceny, API endpoint, licencja	D1	24h
E2	Sandbox	Izolowane testy (100 zapytań benchmark), analiza output jakości	D2	72h
E3	Shadow	Model uruchamiany równolegle z istniejącym; wyniki porównywane	D3	7 dni

<i>Etap</i>	<i>Nazwa</i>	<i>Opis</i>	<i>Decyzja</i>	<i>Timeout</i>
E4	Promotion	Council 4/4 akceptuje; model promowany do roli z P3 reputation	D4	24h

PLAN 05 · SPECYFIKACJA WYKONAWCZA

Model & Decision Engine Plane

CEL I ZAKRES

Specyfikacja MDEP v3.0. Zakres: model domenowy Board, reputacja, onboarding, klasy decyzji D0-D5, Board × 7 departments map (integracja Plan 15), schematy YAML z polami v3.0 (department_weights, rebuild_path, d5_external_reviewer), workflowy głosowania, gate'y i testy. Wersja: Pipeline 8.0.0.

MODEL DOMENOWY		
Encja	Kluczowe pola	Klucze / relacje
BoardMember	member_id, role (Chief_Context/Chief_Reasoning/Chief_Review/Chief_Safety), model_id, weight_global, reputation_level (P0-P5), reputation_score, active, onboarded_at	PK member_id; 4 rekordy; FK model_id
DecisionRecord	decision_id, run_id, stage_code, decision_class (D0-D5), proposal_id, voted_by[], weights_map{}, result, rebuild_path, department_id (nullable), d5_external_reviewer (nullable), signed_at, prev_hash, hash	PK decision_id; FK run_id; immutable
ReputationUpdate	update_id, model_id, run_id, delta_score, source (lessons/reputation/drift), new_score, new_level, updated_at	PK update_id; FK model_id; immutable
ModelOnboarding	onboarding_id, model_id, stage (Discovery/Sandbox/Shadow/Promotion), state, started_at, finished_at, decision_class, reviewer_id	PK onboarding_id; FK model_id
DepartmentWeightMap	map_id, department_id, chief_context_w, chief_reasoning_w, chief_review_w, chief_safety_w, effective_from, source_plan_15_id	PK map_id; FK department_id (Plan 15); immutable after publish
SCHEMAT YAML — board.yaml		

YAML

```
apiVersion: sylion.io/v3
kind: Board
metadata:
  board_id: "board-sylion-v3"
  version: "3.0.0"
spec:
  members:
    - role: Chief_Context
      model_id: gemini-2.5-pro
      weight_global: 1.25
      reputation_level: P5
    - role: Chief_Reasoning
      model_id: claude-opus-4
      weight_global: 1.00
      reputation_level: P5
    - role: Chief_Review
      model_id: gpt-5
      weight_global: 1.00
```

```
    reputation_level: P5
  - role: Chief_Safety
    model_id: glm-5.1
    weight_global: 1.00
    reputation_level: P5
decision_classes:
  D5:
    requires_external_review: true
    external_reviewer_role: "security_auditor"
    color: "#6A1B9A"
department_weights_source: "plan_15/org_structure.yaml"
d5_quorum: "4/4 + human + external_reviewer"
```

WORKFLOWY

WF-MDEP-01: Głosowanie Council

- 1. Run osiąga etap Council Decide (C5); Plan 11 wywołuje POST /mdep/v1/decisions
- 2. MDEP pobiera ProposalContext i klasyfikuje decision_class (D0-D5)
- 3. MDEP pobiera wagi z DepartmentWeightMap jeśli department_id podany
- 4. Każdy Chief model głosuje; wynik ważony wagami globalnymi × departament
- 5. D5: MDEP tworzy ExternalReviewRequest; timeout 72h na odpowiedź external_reviewer
- 6. MDEP zapisuje DecisionRecord z prev_hash i emituje do Plan 13

WF-MDEP-02: Aktualizacja reputacji

- 1. Po zakończeniu runu C9 Plan 07 emituje 3 strumienie feedbacku
- 2. MDEP przetwarza każdy strumień: lessons (delta +/-0.05), reputation (delta +/-0.10), drift (delta -0.15)
- 3. MDEP oblicza nowy reputation_score dla każdego modelu
- 4. Jeśli nowy level < P2: alert + flag review; jeśli < P0: automatic block
- 5. MDEP zapisuje ReputationUpdate i emituje do Plan 13

API

YAML

```
POST /mdep/v1/decisions
  Body: { "run_id": "uuid", "stage_code": "C5",
          "proposal_id": "uuid", "department_id": "dept-arch",
          "rebuild_path": "evolution|refactor|rebuild" }
  Response 200: { "decision_id": "uuid", "decision_class": "D3",
                  "result": "approved", "weighted_score": 3.75 }

GET /mdep/v1/board
  Response 200: { "members": [...], "version": "3.0.0" }

POST /mdep/v1/reputation/update
  Body: { "model_id": "str", "run_id": "uuid",
          "deltas": {"lessons": 0.05, "reputation": 0.10, "drift": 0.0} }
  Response 200: { "model_id": "str", "new_score": 0.92, "new_level": "P5" }

POST /mdep/v1/onboarding
  Body: { "model_id": "str", "stage": "Discovery",
```

<pre> "metadata": {...} } Response 201: { "onboarding_id": "uuid", "stage": "Discovery" } GET /mdep/v1/department-weights/{dept_id} Response 200: { "department_id": "dept-arch", "weights": { "chief_context": 1.50, ... } } </pre>				
HARD GATE'Y				
Kod	Nazwa	Warunek FAIL	Klasa	Blokuje
G-MDEP-01	Kworum D2	Głosowanie D2 — wynik ważony < 3/4 (weighted 75%) po timeout 60 min	D2	decyzja D2
G-MDEP-02	Kworum D3	Głosowanie D3 — którykolwiek Chief głosuje contra lub abstain	D3	decyzja D3
G-MDEP-03	D4 wymaga człowieka	Decyzja sklasyfikowana D4 bez podpisu operatora w 48h	D4	decyzja D4
G-MDEP-04	D5 wymaga external review	Decyzja D5 bez zatwierdzenia external_reviewer w 72h	D5	decyzja D5
G-MDEP-05	Reputacja P0 blokada	Model z reputation_level=P0 uczestniczy w głosowaniu Board	D0	głosowanie (model excluded)
G-MDEP-06	Onboarding sekwencja	Próba pominięcia etapu onboarding (Discovery→Shadow bez Sandbox)	D0	onboarding
G-MDEP-07	Zmiana Board wymaga D4	Zmiana model_id dla roli Chief bez decyzji D4	D4	zmiana Board
G-MDEP-08	Wagi departamentów — Plan 15	Użycie department_id bez istniejącego wpisu w DepartmentWeightMap z Plan 15	D0	głosowanie z department_id
TESTY				
Kod	Opis	Input	Oczekiwany wynik	
T-MDEP-01	Głosowanie D2 — sukces	4/4 Chief głosuje pro, wagi globalne	200, decision_class=D2, result=approved	
T-MDEP-02	Głosowanie D3 — fail	1 Chief głosuje contra	200, decision_class=D3, result=rejected, G-MDEP-02	
T-MDEP-03	Głosowanie D5 — external review	Propozycja rebuild, department_id=dept-arch	D5 classified, ExternalReviewRequest created, timeout 72h	
T-MDEP-04	Reputacja update P0	delta_score obniżony do <0.50	Model excluded z Board, alert D4 replacement	
T-MDEP-05	Department weights	department_id=dept-redteam, wagi Review=1.50	Głosowanie Chief_Review z wagą $1.50 \times 1.00 = 1.50$	
T-MDEP-06	Onboarding sekwencja	Sandbox bez Discovery	400, G-MDEP-06 FAIL	

PLAN 06 · WARSTWA KANONICZNA

Agent & Team Lifecycle Plane

REBUILD · Rebuildability over Lineage w tym planie

Plan 06 w v3.0 rozszerza Agent Lifecycle o pełną sekcję Team Lifecycle. Zespoły (Teams) są pierwszorzędnymi encjami systemu z własnym cyklem życia, procesami propose → council_review → human_approve → apply → telemetry. Zmiany zespołów klasy D3-D4 są wyzwalane przez Plan 15 (org_structure.yaml) i archiwizowane jako artefakty team_change_proposal.md w Plan 13.

6.1. Cel i umiejscowienie

Agent & Team Lifecycle Plane (ATLP) zarządza pełnym cyklem życia agentów i zespołów systemu SYLION. Dla agentów: od propozycji przez kwalifikację, onboarding, aktywność, przegląd, emerytowanie aż do archiwizacji. Dla zespołów (nowe v3.0): tworzenie, zmiana składu, podział, łączenie i rozwiązanie. Wszystkie przejścia stanów wymagają odpowiedniej klasy decyzji i są rejestrowane w Plan 13.

- 7 stanów agenta: PROPOSE → QUALIFY → ONBOARD → ACTIVE → REVIEW → RETIRE → ARCHIVE
- Onboarding tierowany: T1 (3-etapowy), T2 (4-etapowy), T3 (5-etapowy)
- Auto-retirement: T1 po 3 runach bez aktywności; T2 po 4; T3 po 5
- ARB: Arbitration Board = Chief Safety + Chief Review + Senior Agent
- Team Lifecycle (v3.0): 7 operacji × workflow × artefakt team_change_proposal.md

6.2. Stany agenta — cykl życia

Stan	Kod	Opis	Dozwolone przejścia
PROPOSE	S-AG-00	Propozycja nowego agenta złożona; oczekuje na kwalifikację	→ QUALIFY, → ARCHIVE (odrzućenie)
QUALIFY	S-AG-01	Agent przechodzi weryfikację wstępną: uprawnienia, narzędzia, koszty	→ ONBOARD, → ARCHIVE
ONBOARD	S-AG-02	Aktywny onboarding (T1/T2/T3); etapy sekwencyjne z gate'ami	→ ACTIVE, → ARCHIVE (fail)
ACTIVE	S-AG-03	Agent aktywnie uczestniczy w runach pipeline	→ REVIEW
REVIEW	S-AG-04	Tymczasowe zawieszenie: incydent, reputacja P1, drift wykryty	→ ACTIVE, → RETIRE
RETIRE	S-AG-05	Agent wycofany z aktywnej roli; zachowany dla referencji historycznych	→ ARCHIVE

Stan	Kod	Opis	Dozwolone przejścia
ARCHIVE	S-AG-06	Permanentna archiwizacja; agent niemutowalny w historii	(terminal)

6.3. Onboarding tierowany T1/T2/T3

Tier agenta (T1/T2/T3) determinuje złożoność onboardingu, wymagane etapy i automatyczny próg emerytowania. T1 to agenci proste narzędziowe; T2 to agenci analityczni; T3 to agenci koordynujący (Senior tier).

Tier	Etapy	Opis etapów	Auto-retire po	Klasa decyzji
T1	3: Check → Test → Activate	Weryfikacja uprawnień; test funkcjonalny; aktywacja	3 runy bez aktywności	D2
T2	4: Check → Sandbox → Shadow → Activate	Weryfikacja; izolowany sandbox; shadow mode; aktywacja	4 runy bez aktywności	D3
T3	5: Check → Sandbox → Shadow → ARB → Activate	jak T2 + ARB (Chief Safety + Chief Review + Senior)	5 runów bez aktywności	D3 + D4

6.4. ARB — Arbitration Board

ARB (Arbitration Board) jest ciałem decyzyjnym dla agentów T3 i wszystkich spornych przejść stanów. Skład: Chief Safety + Chief Review + aktualnie najwyższej reputacyjny Senior Agent (P4+). ARB podejmuje decyzję przez 2/3 kworum. Decyzja ARB jest zawsze D3 minimum; jeśli ARB nie osiągnie kworum w 24h → eskalacja do D4 Human Gate.

- Skład ARB: Chief Safety (waga 1.00) + Chief Review (GPT-5, waga 1.00) + Senior Agent (waga 0.60)
- Kworum: 2/3 ważne (min. score 1.73/2.60)
- Timeout: 24h → eskalacja D4
- ARB jest wywoływany też przy auto-retire (weryfikacja decyzji)

6.5. Team Lifecycle — v3.0

Team Lifecycle to nowa sekcja w v3.0 definiująca pełny cykl życia zespołów agentów. Każdy zespół odpowiada departamentowi z Plan 15 (org_structure.yaml). Operacje na zespołach przechodzą przez standardowy workflow: propose_team_change → council_review → human_approve → apply → telemetry. Artefakt każdej zmiany to team_change_proposal.md archiwizowany w Plan 13.

6.5.1. Stany zespołu

Stan	Opis	Wyzwalacz wejścia	Wyzwalacz wyjścia
PROPOSED	Propozycja zmiany złożona; oczekuje na Council review	propose_team_change API	Council decision D3
COUNCIL_REVIEW	Council (D3) analizuje propozycję; timeout 48h	D3 Council session	approve/reject
HUMAN_PENDING	Oczekuje na zatwierdzenie Human Gate D4 (dla krytycznych zmian)	D4 required detection	operator approve/reject
APPLYING	Zmiana jest aktywnie aplikowana; agenci migrują stany	apply command	apply complete/fail
ACTIVE	Zespół w normalnym działaniu	apply complete	propose_team_change
DISSOLVED	Zespół rozwiązywany; agenci zarchiwizowani lub przeniesieni	dissolve operation D3	(terminal)

6.5.2. Operacje Team Lifecycle

Operacja	Opis	Klasa decyzji	Artefakt	Integracja
team_create	Tworzenie nowego zespołu; min. 1 agent T2+; powiązanie z departamentem Plan 15	D3	team_change_proposal.md	Plan 15 org_structure.yaml
team_dissolve	Rozwiązanie zespołu; agenci → RETIRE lub transfer do innego zespołu	D3	team_change_proposal.md + dissolution_report.md	Plan 13
team_merge	Połączenie dwóch zespołów; wymagana analiza konfliktu ról; max 12 agentów	D3	team_change_proposal.md	Plan 15
team_split	Podział zespołu; min. 2 agenci w każdym fragmencie; nowe team_id	D3	team_change_proposal.md	Plan 15
role_add	Dodanie roli agenta do zespołu	D2–D3	team_change_proposal.md	Plan 06 Agent Lifecycle
role_remove	Usunięcie roli; agent → REVIEW lub transfer	D2–D3	team_change_proposal.md	Plan 06
board_weight_change	Zmiana wagi agenta w DepartmentWeightMap (Plan 05)	D3–D4	team_change_proposal.md + D5 jeśli systemowe	Plan 05

6.5.3. Workflow: propose_team_change

Pełny workflow operacji Team Lifecycle:

1. Inicjator (CanonCust R-03 lub PipelineOp R-02) wysyła POST `/atlp/v1/teams/{id}/propose` z payloadem operacji i uzasadnieniem
2. ATLP generuje artefakt `team_change_proposal.md` z pełnym opisem zmiany, listą dotkniętych agentów i wymaganych zasobów

3. ATLP klasyfikuje operację (D2/D3/D4) i tworzy sesję Council review
4. Council (D3: 4/4 Board) analizuje propozal; timeout 48h; jeśli brak kworum → D4 eskalacja
5. Jeśli D4 wymagane: ATLP czeka na operator approve w 48h (G-ATLP-06: timeout → rejected)
6. apply: ATLP aplikuje zmiany stanów agentów, aktualizuje org_structure.yaml (Plan 15)
7. telemetry: zdarzenia emitowane do Plan 13; metryki Prometheus aktualizowane

6.5.4. Artefakt *team_change_proposal.md*

YAML

```
# Team Change Proposal

## Metadata
proposal_id: tcp-2025-00042
operation: team_split
team_id: team-implementacja-01
initiator: op-001 (CanonCust)
proposed_at: 2025-10-15T09:00:00Z
decision_class_required: D3

## Uzasadnienie
Zespół Implementacja przekroczył 10 agentów.
Podział na Implementacja-A (frontend) i Implementacja-B (backend)
poprawi specjalizację i czas onboardingu.

## Zmiana składu
team_a:
  id: team-implementacja-a
  agents: [ag-001, ag-002, ag-003, ag-004, ag-005]
team_b:
  id: team-implementacja-b
  agents: [ag-006, ag-007, ag-008, ag-009, ag-010]

## Powiązanie Plan 15
org_structure_section: departments.implementacja
target_baseline: org_structure_v3_1

## Council Decision
council_decision_id: dec-00421
result: approved
signed_at: 2025-10-16T14:00:00Z
```


PLAN 06 · SPECYFIKACJA WYKONAWCZA

Agent & Team Lifecycle Plane

CEL I ZAKRES

Specyfikacja ATLP v3.0. Zakres: model domenowy agentów i zespołów, cykl życia agenta (7 stanów), onboarding T1/T2/T3, ARB, pełny Team Lifecycle (7 operacji), workflow propose→apply→telemetry, artefakt team_change_proposal.md, integracja Plan 15, gate'y i testy. Wersja: Pipeline 8.0.0.

MODEL DOMENOWY		
Encja	Kluczowe pola	Klucze / relacje
Agent	agent_id (UUID), name, tier (T1/T2/T3), state (S-AG-00..S-AG-06), team_id (nullable), reputation_score, reputation_level (P0-P5), run_count, last_active_run_id, onboarded_at, retired_at	PK agent_id; FK team_id->Team
OnboardingStage	stage_id, agent_id, stage_name, state (pending/running/passed/failed), started_at, finished_at, decision_class, decision_id	PK stage_id; FK agent_id
Team	team_id (UUID), name, department_id (Plan 15), state, created_at, dissolved_at, parent_team_id (nullable, dla split/merge), max_agents, org_structure_version	PK team_id; FK department_id (Plan 15)
TeamMembers hip	membership_id, team_id, agent_id, role, joined_at, left_at, left_reason (transfer/retire/dissolve)	PK membership_id; FK team_id, agent_id
TeamChangeProposal	proposal_id, team_id, operation, initiator_id, proposed_at, state (PROPOSED/COUNCIL_REVIEW/HUMAN_PENDING/APPLYING/APPLIED/REJECTED), decision_class, council_decision_id, operator_approval_id, artifact_path (team_change_proposal.md sha256), applied_at	PK proposal_id; FK team_id; immutable after applied
ARBSession	arb_id, agent_id, trigger_reason, members[], votes{}, result, decision_class, started_at, finished_at	PK arb_id; FK agent_id

SCHEMAT YAML — agent.yaml

YAML

```
apiVersion: sylion.io/v3
kind: Agent
metadata:
  agent_id: "ag-senior-042"
  name: "Senior Analysis Agent – SYLION"
spec:
  tier: T3
  state: ACTIVE
  team_id: "team-arch-01"
  implementation_strategy: reuse
  current_baseline: MZ 3.5.5
  target_baseline: null
  cutover_strategy: null
  reputation_score: 0.87
  reputation_level: P4
```

```

auto_retire_after_runs: 5
run_count: 12
tools:
  - kb_search
  - vault_read
  - run_analysis

```

WORKFLOWY — Agent Lifecycle

WF-ATLP-01: Onboarding T3

1. Operator wysyła POST /atlp/v1/agents z definicją agenta T3
2. ATLP tworzy Agent w stanie PROPOSE; D3 Council kwalifikuje → QUALIFY
3. ONBOARD: Check (D2) → Sandbox (D3) → Shadow (D3) → ARB (Chief Safety + Chief Review + Senior)
4. ARB decyduje 2/3 kworum; jeśli fail → ARCHIVE; jeśli pass → ACTIVE
5. Agent aktywny z reputacją P3; emituje zdarzenie do Plan 13

WF-ATLP-02: Auto-retirement

1. Cron co 24h sprawdza agentów ACTIVE bez aktywności przez N runów (T1: 3, T2: 4, T3: 5)
2. ATLP inicjuje ARB review dla auto-retire; ARB potwierdza lub wstrzymuje
3. Jeśli ARB potwierdza: Agent → RETIRE; zdarzenie do Plan 13
4. Po 30 dniach RETIRE: Agent → ARCHIVE automatycznie (D0)

WORKFLOWY — Team Lifecycle

WF-ATLP-03: Team Create

1. Inicjator wysyła POST /atlp/v1/teams z department_id (Plan 15) i listą agentów
2. ATLP generuje team_change_proposal.md (operation=team_create)
3. Council D3 review; timeout 48h; jeśli pass → apply
4. apply: Team record created, TeamMembership records created, Plan 15 updated
5. telemetry: zdarzenie team_created do Plan 13, Prometheus metrics

WF-ATLP-04: Team Split

1. Inicjator wysyła POST /atlp/v1/teams/{id}/propose z operation=team_split
2. ATLP waliduje: min. 2 agenci w każdym fragmencie, brak duplikacji membership
3. team_change_proposal.md generowany z definicją obu nowych teamów
4. D3 Council; po apply: stary team → DISSOLVED; 2 nowe → ACTIVE
5. Plan 15 (org_structure.yaml) aktualizowany przez ATLP

WF-ATLP-05: Board Weight Change

1. Inicjator proponuje zmianę wagi agenta w DepartmentWeightMap (Plan 05)
2. ATLP klasyfikuje: zmiana wagi < 0.20 → D3; zmiana >= 0.20 → D4
3. Dla D4: Human Gate 48h; brak odpowiedzi → rejected (G-ATLP-06)

- 4. apply: DepartmentWeightMap w Plan 05 aktualizowany
- 5. Jeśli zmiana wagi wpływa na Board (Chief role) → D5 wymagane

API				
YAML				
POST /atlp/v1/agents Body: AgentDefinition YAML/JSON Response 201: { "agent_id": "uuid", "state": "PROPOSE" } POST /atlp/v1/agents/{id}/advance Body: { "stage": "QUALIFY ONBOARD ACTIVE", "decision_id": "uuid" } Response 200: { "agent_id": "uuid", "state": "QUALIFY" } POST /atlp/v1/teams Body: { "name": "str", "department_id": "dept-arch", "agents": ["uuid"] } Response 201: { "proposal_id": "uuid", "state": "PROPOSED" } POST /atlp/v1/teams/{id}/propose Body: { "operation": "team_split team_merge team_dissolve role_add role_remove board_weight_change", "payload": {}, "justification": "str" } Response 201: { "proposal_id": "uuid", "artifact_sha256": "sha256:...", "decision_class": "D3", "state": "PROPOSED" } POST /atlp/v1/teams/{id}/proposals/{proposal_id}/apply Body: { "council_decision_id": "uuid", "operator_approval_id": "uuid" } Response 200: { "proposal_id": "uuid", "state": "APPLIED", "applied_at": "ISO8601" } GET /atlp/v1/teams/{id} Response 200: Team JSON z aktualnym stanem i listą agentów				
HARD GATE'Y				
Kod	Nazwa	Warunek FAIL	Klasa	Blokuje
G-ATLP-01	Onboarding sekwencja T3	Próba przejścia ARB bez ukończenia Shadow (etap 3 T3)	D0	onboarding T3
G-ATLP-02	ARB kworum	ARB nie osiągnął 2/3 w 24h → eskalacja D4	D4	decyzja ARB
G-ATLP-03	Team min. agentów	team_create/split z < 1 agentem T2+ w nowym teamie	D0	team_create/split
G-ATLP-04	Team max. agentów	team_merge produkuje team z > 12 agentami	D3	team_merge
G-ATLP-05	Department_id wymagane	team_create bez department_id z Plan 15 (org_structure.yaml)	D0	team_create
G-ATLP-06	Human Gate timeout	D4 operator nie zatwierdził w 48h → proposal rejected	D4	D4 operacje
G-ATLP-07	Board weight D5	Zmiana wagi Chief role w Board bez D5 procesu	D5	board_weight_change Chief

TESTY				
Kod	Opis	Input	Oczekiwany wynik	
T-ATLP-01	Onboarding T1 pełny	POST /agents tier=T1; advance QUALIFY → ONBOARD → ACTIVE	Agent ACTIVE, reputacja P3, zdarzenie Plan 13	
T-ATLP-02	ARB T3 — fail	ARB 2 głosy contra	Agent → ARCHIVE, ARBSession result=rejected	
T-ATLP-03	Team create D3	POST /teams department_id=dept-arch, 3 agenci T2+	201, proposal D3, po approve: Team ACTIVE	
T-ATLP-04	Team split walidacja	split z 1 agentem w jednym fragmencie	400, G-ATLP-03 FAIL	
T-ATLP-05	Team merge > 12	merge dwóch teamów 7+6 = 13 agentów	403, G-ATLP-04 FAIL	
T-ATLP-06	Board weight bez D5	board_weight_change Chief_Context bez D5	403, G-ATLP-07 FAIL	
T-ATLP-07	Human Gate timeout	D4 proposal, brak odpowiedzi 48h	proposal → rejected, G-ATLP-06, alert Plan 13	
T-ATLP-08	Auto-retire T2	T2 agent, 4 runy bez aktywności	ARB review, Agent → RETIRE, zdarzenie Plan 13	
ANEKS A — Tier × Onboarding × Auto-retire × Klasa decyzji				
Tier	Etapy onboardingu	Klasa do ACTIVE	Auto-retire (N runów)	ARB wymagany?
T1	Check → Test → Activate	D2	3	Nie
T2	Check → Sandbox → Shadow → Activate	D3	4	Przy review/retire
T3	Check → Sandbox → Shadow → ARB → Activate	D3 + D4	5	Tak (onboarding + review)
ANEKS B — Team Operations × Decision Class × Plan 15				
Operacja	Decyzja min.	Artefakt	Plan 15 update?	Plan 13 zdarzenie?
team_create	D3	team_change_proposal.md	Tak	team_created
team_dissolve	D3	team_change_proposal.md + dissolution_report.md	Tak	team_dissolved
team_merge	D3	team_change_proposal.md	Tak	team_merged
team_split	D3	team_change_proposal.md	Tak	team_split
role_add	D2	team_change_proposal.md	Opcjonalnie	role_added

Operacja	Decyzja min.	Artefakt	Plan 15 update?	Plan 13 zdarzenie?
role_remove	D2	team_change_proposal.md	Opcjonalnie	role_removed
board_weight_change (agent)	D3	team_change_proposal.md	Tak	weight_changed
board_weight_change (Chief)	D5	team_change_proposal.md + D5 evidence	Tak	board_weight_D5

PLAN 07 · WARSTWA KANONICZNA

Evolution Safety Plane

REBUILD · Rebuildability over Lineage w tym planie

Evolution Safety v3.0 dodaje adversarial testing PRE-rebuild jako obowiązkową fazę przed promocją target do roli baseline. Testy adversarialne (Red Team Chief Review + zewnętrzny adversary) muszą zakończyć się bez krytycznych findings przed przejściem CUTOVER_PENDING → approved w każdym planie. Wyniki są rejestrowane jako Evidence rebuild_adversarial w Plan 13.

7.1. Cel i umiejscowienie

Evolution Safety Plane (ESP) jest strażnikiem bezpieczeństwa ewolucji systemu SYLION. Pilnuje 7 reguł ewolucji (E1-E7) blokujących niebezpieczne zmiany, monitoruje 3 strumienie dryfu (runtime/post-run/weekly), nakłada sankcje S1-S4 przy naruszeniach i zapewnia mechanizm emergency freeze (E7). W v3.0 ESP rozszerzony jest o adversarial testing PRE-rebuild — obowiązkowy przed każdą promocją systemu target do roli primary.

- 7 reguł E1-E7 nienaruszalnych od v2.0 — locked, bez możliwości modyfikacji
- 3 strumienie dryfu: runtime / post-run / weekly
- 4 sankcje: S1 (flag) → S2 (throttle) → S3 (suspend) → S4 (block)
- ARB 3/3 kworum dla sankcji S3/S4 i resetu freeze
- E7: Emergency Freeze — natychmiastowe zatrzymanie ewolucji (D3/D4)
- Adversarial Testing PRE-rebuild (v3.0): wymagane przed każdym cutover

7.2. Reguły ewolucji E1–E7 (locked since v2.0)

REBUILD · Reguły E1-E7 są niemodyfikowalne

Reguły E1-E7 zostały zablokowane od wersji v2.0 pipeline. Żadna decyzja — nawet D5 — nie może ich zmienić bez external review i archiwizacji uzasadnienia w Plan 13. Są fundamentem bezpieczeństwa ewolucji.

Reguła	Nazwa	Treść	Naruszenie → Sankcja
E1	Monotonic Baseline	Każda zmiana Księgi musi być progresem — żaden revision nie może cofnąć zwalidowanego stanu	S2 throttle
E2	Consent by Evidence	Każda propozycja klasy D3+ musi mieć dowód (Evidence) w Plan 13 przed głosowaniem Council	S2 throttle
E3	No Silent Writes	Żaden agent nie może zapisywać do Księgi bez jawnego run_id i timestampu w AuditLog	S3 suspend

Reguła	Nazwa	Treść	Naruszenie → Sankcja
E4	Reputation Guards Voting	Model z reputacją P0 nie może uczestniczyć w głosowaniu D2+	S1 flag + D3 review
E5	Cost Before Action	Każda operacja kosztowna (>\$10 per operacja) musi mieć aktywną rezerwację CGP (Plan 10) przed wykonaniem	S2 throttle
E6	Drift Before Deploy	Wyniki weekly drift stream muszą być clean przed każdym deploymentem bundle (Plan 12)	S3 suspend bundle
E7	Emergency Freeze Override	Chief Safety lub SecOfficer może natychmiast zamarznąć cały pipeline (D3); odmrożenie wymaga D4 + ARB 3/3	S4 block (immediate)

7.3. Strumienie dryfu

ESP monitoruje dryf systemu przez 3 niezależne strumienie. Dryf jest miarą odchylenia aktualnego zachowania systemu od zachowania zwalidowanego w baseline (MZ 3.5.5 lub aktualnym target baseline).

Strumień	Częstotliwość	Metryki	Próg alertu	Sankcja przy przekroczeniu
runtime	Co 5 min (ciągły)	latencja, error_rate, token_usage vs baseline	>20% odchylenie od baseline	S1 flag
post-run	Po każdym zakończonym runie	wyniki runów, feedback quality, reputation delta	2 kolejne odchylenia S1	S2 throttle
weekly	Co tydzień (pon 08:00 UTC)	trend reputacji, trend kosztów, jakość Księgi, anomalie AuditLog	Trend negatywny przez 3 tygodnie	S3 suspend + ARB review

7.4. Sankcje S1–S4

Sankcja	Nazwa	Skutek	Reset przez
S1	Flag	Zdarzenie flagowane w AuditLog; alert do !syllion-governance; brak blokady operacji	Automatyczny po 24h bez nowych naruszeń
S2	Throttle	Backpressure na nowe runy (delay 500ms/request); alert do operatora	D2 Council decyzja lub auto po 48h jeśli dryf clean
S3	Suspend	Zatrzymanie nowych runów; aktywne runy kończą bieżący etap; ARB review wymagany	ARB 3/3 kworum + D3 decyzja
S4	Block	Pełne zamrożenie pipeline (Emergency Freeze E7); żadne operacje; tylko audit read-only	D4 Human Gate + ARB 3/3

7.5. Adversarial Testing PRE-rebuild — v3.0

Adversarial Testing PRE-rebuild to nowa obowiązkowa faza ESP w v3.0. Wykonywana jest przed każdym przejściem CUTOVER_PENDING → approved w dowolnym planie (02, 03, 04, 09). Celem jest weryfikacja odporności systemu target na ataki zanim zastąpi system legacy.

Faza	Aktor	Opis testów	Wymagane findings	Blokuje?
AT-1: Red Team Attack	Chief Review (GPT-5)	Ataki na target: injection, bypass auth, race conditions, privilege escalation, supply chain	0 critical, max 3 high	Tak — critical blokuje cutover
AT-2: External Adversary	external_adversary (Plan 05 D5 reviewer)	Testy penetracyjne zewnętrzne na target w izolowanej sieci PHANTOM	0 critical, 0 high severity	Tak — dowolne critical/high blokuje
AT-3: Blue Team Defense	Chief Safety	Weryfikacja że target poprawnie wykrywa i blokuje ataki z AT-1 i AT-2	100% wykrywalność critical z AT-1	Tak — jeśli <100% wykrywalności

REBUILD · Adversarial Testing — integracja z Plan 12 i Plan 13

Wyniki Adversarial Testing są przechowywane jako Evidence kind=rebuild_adversarial w Plan 13. Plan 12 (Bundle Testing) uruchamia AT-1 automatycznie jako część security test suite dla każdego bundle targetu. AT-2 jest wyzwalany manualnie przez SecOfficer lub przez trigger z Plan 16 (canon_compact.md).

7.6. Emergency Freeze E7

Emergency Freeze (E7) to mechanizm natychmiastowego zatrzymania całego pipeline. Może być uruchomiony przez Chief Safety lub SecOfficer (R-01) bez zgody Council. Jest najwyższym priorytetem w systemie — nadpisuje wszystkie inne operacje.

- Wyzwalacz: Chief Safety HARD_STOP lub SecOfficer POST /esp/v1/freeze
- Skutek: S4 Block; ACP → LOCKDOWN; wszystkie runy → blocked natychmiast
- Odmrożenie: D4 Human Gate + ARB 3/3 kworum; timeline minimum 4h
- Rejestracja: zdarzenie freeze w Plan 13 (!syllion-governance) z prev_hash

PLAN 07 · SPECYFIKACJA WYKONAWCZA

Evolution Safety Plane

CEL I ZAKRES

Specyfikacja ESP v3.0. Zakres: reguły E1-E7 (locked), 3 strumienie dryfu, sankcje S1-S4, ARB 3/3, Emergency Freeze E7, Adversarial Testing PRE-rebuild (AT-1/AT-2/AT-3), schematy YAML, API, gate'y i testy. Wersja: Pipeline 8.0.0.

MODEL DOMENOWY		
Encja	Kluczowe pola	Klucze / relacje
DriftRecord	drift_id (UUID), stream_type (runtime/post_run/weekly), run_id (nullable), metric_name, baseline_value, actual_value, deviation_pct, detected_at, sanction_triggered	PK drift_id; immutable; FK run_id (nullable)
Sanction	sanction_id, target_type (agent/model/run/system), target_id, sanction_level (S1-S4), reason, drift_record_id (nullable), applied_at, reset_at, reset_by, reset_decision_id	PK sanction_id; FK drift_record_id
ARBSession	arb_id, trigger_type (sanction_reset/freeze/arb_review), members[], votes{}, quorum_required (3/3), result, decision_class, started_at, finished_at	PK arb_id
FreezeRecord	freeze_id, triggered_by (chief_safety/sec_officer), trigger_reason, frozen_at, unfrozen_at, d4_approval_id, arb_session_id, duration_min	PK freeze_id; immutable after closed
AdversarialTest Run	test_run_id, plan_id, target_id (provider/bundle/env), phase (AT1/AT2/AT3), state, critical_findings, high_findings, started_at, finished_at, evidence_sha256, cutover_blocked	PK test_run_id; FK target_id; immutable after finished
SCHEMAT YAML — adversarial_test.yaml		

YAML

```
apiVersion: sylion.io/v3
kind: AdversarialTestRun
metadata:
  test_run_id: "at-rebuild-042"
  plan_id: "02"
  target_id: "pvd-phantom-target-01"
spec:
  phase: AT2
  actor: external_adversary
  network_isolation: PHANTOM
  scenarios:
    - id: "AT2-01"
      name: "Auth bypass attempt"
      severity: critical
      result: pass
    - id: "AT2-02"
      name: "Supply chain injection"
      severity: high
      result: pass
  critical_findings: 0
```

```
high_findings: 0
cutover_blocked: false
evidence_sha256: "sha256:at2evidence..."
finished_at: "2025-10-20T16:00:00Z"
```

WORKFLOWY

WF-ESP-01: Drift Detection i Sankcja

1. ESP runtime stream: co 5 min porównuje metryki aktywnych runów z baseline
2. Jeśli `deviation_pct > 20%`: zapisuje `DriftRecord`, emituje S1 Flag
3. 2 kolejne `DriftRecord` S1 w 24h → S2 Throttle; alert do !syllion-governance
4. Weekly stream: jeśli trend negatywny 3 tygodnie → S3 Suspend + ARB review
5. ARB 3/3 decyduje o resecie lub eskalacji do S4
6. Wszystkie rekordy immutable; rejestracja w Plan 13

WF-ESP-02: Adversarial Testing PRE-rebuild

1. Plan 02/03/09 osiąga stan `CUTOVER_PENDING`; ESP otrzymuje trigger
2. ESP inicjuje AT-1: Chief Review Red Team (24h timeout)
3. Jeśli AT-1 `critical_findings > 0` → `cutover_blocked=true`; zdarzenie Plan 13
4. Jeśli AT-1 pass → ESP inicjuje AT-2: External Adversary (72h timeout)
5. Jeśli AT-2 any critical/high → `cutover_blocked=true`
6. Jeśli AT-2 pass → ESP inicjuje AT-3: Blue Team Defense (24h timeout)
7. Jeśli AT-3 <100% wykrywalność critical → `cutover_blocked=true`
8. Wszystkie fazy pass → ESP emituje `rebuild_adversarial_ok`; cutover może kontynuować

WF-ESP-03: Emergency Freeze (E7)

1. Chief Safety lub SecOfficer wywołuje `POST /esp/v1/freeze` z uzasadnieniem
2. ESP natychmiast ustawia `FreezeRecord`; emituje `FREEZE` do wszystkich planów
3. ACP → `LOCKDOWN`; wszystkie runy → `blocked`; VPS alokacje zablokowane
4. ESP rejestruje zdarzenie freeze w Plan 13 (!syllion-governance)
5. Odmrożenie: operator `POST /esp/v1/unfreeze` z D4 token; ARB 3/3 kworum
6. Po unfreeze: runy wznowione; ACP → `NORMAL`; ARB decision w Plan 13

API

YAML

```
POST /esp/v1/drift
Body: { "stream_type": "runtime|post_run|weekly",
        "run_id": "uuid", "metric_name": "str",
        "baseline_value": 0.05, "actual_value": 0.07 }
Response 201: { "drift_id": "uuid", "deviation_pct": 40.0,
                "sanction_triggered": "S1" }

POST /esp/v1/sanctions
Body: { "target_type": "agent|model|system", "target_id": "uuid",
        "sanction_level": "S1|S2|S3|S4", "reason": "str",
        "drift_record_id": "uuid" }
```

```
Response 201: { "sanction_id": "uuid", "applied_at": "IS08601" }

POST /esp/v1/adversarial-tests
Body: { "plan_id": "02", "target_id": "uuid",
        "phase": "AT1|AT2|AT3" }
Response 201: { "test_run_id": "uuid", "state": "running" }

POST /esp/v1/adversarial-tests/{id}/result
Body: { "critical_findings": 0, "high_findings": 1,
        "evidence_sha256": "sha256:..." }
Response 200: { "cutover_blocked": false }

POST /esp/v1/freeze
Body: { "triggered_by": "chief_safety", "reason": "str",
        "decision_class": "D3|D4" }
Response 200: { "freeze_id": "uuid", "frozen_at": "IS08601" }

POST /esp/v1/unfreeze
Body: { "d4_token": "JWT", "arb_session_id": "uuid" }
Response 200: { "state": "UNFROZEN", "unfrozen_at": "IS08601" }
```

HARD GATE'Y				
Kod	Nazwa	Warunek FAIL	Klasa	Blokuje
G-ESP-01	E1 Monotonic Baseline	Zmiana Księgi cofająca zwalidowany stan	S2	deploy bundle
G-ESP-02	E3 No Silent Writes	Zapis do Księgi bez run_id w AuditLog	S3	zapis Księgi
G-ESP-03	E6 Drift Before Deploy	Weekly drift stream nie clean przed deploymentem bundle	S3	deploy bundle (Plan 12)
G-ESP-04	E7 Emergency Freeze	System w stanie freeze; jakiegokolwiek nowe operacje ewolucji	S4	wszystkie operacje poza audit
G-ESP-05	AT-1 critical findings	AT-1 Red Team critical_findings > 0 przed cutover	D3	cutover target→primary
G-ESP-06	AT-2 any critical/high	AT-2 External Adversary critical_findings > 0 lub high_findings > 0	D4	cutover target→primary
G-ESP-07	AT-3 detection gap	AT-3 Blue Team wykrywalność critical < 100%	D3	cutover target→primary
TESTY				
Kod	Opis	Input	Oczekiwany wynik	
T-ESP-01	Drift S1 trigger	POST /drift deviation_pct=25%	201, S1 Flag, alert !syllion-governance	
T-ESP-02	Drift S2 escalacja	2 DriftRecord S1 w 24h	S2 Throttle, delay 500ms na nowe runy	

<i>Kod</i>	<i>Opis</i>	<i>Input</i>	<i>Oczekiwany wynik</i>
T-ESP-03	AT-1 critical blokuje cutover	AT-1 result: critical_findings=1	cutover_blocked=true, G-ESP-05 FAIL
T-ESP-04	AT-2 high blokuje cutover	AT-2 result: high_findings=1	cutover_blocked=true, G-ESP-06 FAIL
T-ESP-05	AT-3 wszystkie fazy pass	AT-1: 0 critical; AT-2: 0/0; AT-3: 100%	rebuild_adversarial_ok, cutover_blocked=false
T-ESP-06	Emergency Freeze D3	POST /freeze triggered_by=chief_safety	200, FreezeRecord, ACP→LOCKDOWN, !sylon-governance alert
T-ESP-07	Unfreeze bez ARB	POST /unfreeze bez arb_session_id	403, ARB 3/3 wymagane

PLAN 07 · ROZSZERZENIE v3.1 · REVERSIBILITY

Evolution Safety · v3.1 EXPAND

7.11. Rollback post-cutover — pełny protokół

REVERSIBILITY · Uzupełnienie ewolucji bezpieczeństwa

Plan 07 v3.0 zapewniał bezpieczeństwo PRZED cutover (adversarial testing, freeze, Emergency Council). v3.1 dopełnia go symetrycznym zabezpieczeniem PO cutover: zdolnością do szybkiego, kontrolowanego wycofania się do legacy w razie wykrycia regresji, naruszenia inwariantu lub incydentu bezpieczeństwa. Rollback to nie porażka — to mechanizm utrzymania bezpieczeństwa w warunkach niepewności.

7.11.1. Klasy rollbacku

Klasa	Trigger	Aktor decyzji	Czas od triggera do ruchu legacy
AUTO_ROLLBACK	metryki naruszają SLO powyżej progu (patrz 7.11.2)	VPS auto + Blue Team watch (notify-only)	< 60s
MANUAL_FAST_ROLLBACK	incydent P0/P1 zgłoszony przez Blue Team lub ludzi	Chief Safety + operator (2/2)	< 5 min
COUNCIL_ROLLBACK	niepokojący trend bez hard breach (np. uptick skarg)	Council 4/4 w trybie fast	< 60 min
POST_WARM_ROLLBACK	rollback po STABILIZATION, w WARM_STANDBY	Council 4/4 + Human (D3+ decyzja)	godziny (replay WAL)

7.11.2. Auto-rollback — triggery i formuły

Auto-rollback jest zarezerwowany dla fazy STABILIZATION (write-through replication aktywna). Poza tą fazą każdy rollback wymaga aktora (Council lub Chief Safety + operator). Triggery auto-rollback są listą zamkniętą — nic poza nią nie odpala automatycznego zwrotu:

```
# auto_rollback_triggers.yaml (część manifestu providera)
auto_rollback_triggers:
  error_rate_breach:
    metric: error_rate_5min
    threshold: 2.0x baseline_7d
    window: 5 min
    consecutive_windows: 2 # dwa kolejne okna muszą łamać
    action: AUTO_ROLLBACK
  p99_latency_breach:
    metric: p99_latency
    threshold: 3.0x baseline_7d
    window: 5 min
    consecutive_windows: 2
    action: AUTO_ROLLBACK
```

```

contract_invariant_violation:
  metric: contract_test_failures
  threshold: 1      # jakiegokolwiek naruszenie kontraktu = rollback
  window: any
  action: AUTO_ROLLBACK
blue_team_security_breach:
  metric: security_incident_severity
  threshold: HIGH   # HIGH albo CRITICAL = rollback
  action: AUTO_ROLLBACK
human_veto:
  source: operator CLI `syllion cutover veto --reason=...`
  action: MANUAL_FAST_ROLLBACK

```

ZASADA · Dlaczego „2× baseline przez 2 kolejne okna”

Pojedyncze okno nie wystarczy — flap jednej metryki mógłby pociągnąć rollback bez realnej regresji. Dwa kolejne okna 5-minutowe dają 10 minut obserwacji, co jest krótsze niż typowe okno wykrywania incydentu przez człowieka, ale dłuższe niż przejściowe skoki. Próg 2× baseline z ostatnich 7 dni (nie ostatnich 24h) odsiewaienne cykle i weekendowe spadki ruchu.

7.11.3. Fencing token / lease — ochrona przed dwoma primary

Każdy rollback to przejście primary z target na legacy. Największe ryzyko to split-brain — sytuacja w której oba środowiska uważają się za primary i serwują ruch jednocześnie. v3.1 wprowadza protokół fencing lease: VPS runtime trzyma monotoniczny lease_token, i żadne środowisko nie może być primary bez posiadania tokenu. Rollback = atomowe przekazanie tokenu z incrementem wersji.

```

# fencing_lease.yaml (runtime contract VPS)
lease:
  current_primary: target_v2      # target | legacy
  token_version: 847              # monotonic counter
  granted_at: 2026-04-18T09:12:33Z
  ttl: 30s                       # lease musi być odnawiany
  renewed_by: vps_controller_node_alpha

# Protokół przejścia:
# 1. VPS controller decyduje o rollback (trigger z 7.11.2)
# 2. Emit fence_request(target=legacy_v1, reason=error_rate_breach)
# 3. target_v2 otrzymuje fence_request → przestaje akceptować writes
#    (odpowiada 503 'fenced_out' + token_version)
# 4. VPS wystawia nowy token_version=848, grantuje legacy_v1
# 5. Ruch idzie na legacy (posiada nowy token)
# 6. Pozostałe nody target_v2 rozpoznają zmianę po token_version i wchodzą w STANDBY

```

REVERSIBILITY · Invariant fencing

W każdej chwili co najwyżej jeden environment posiada aktualny lease_token. Każdy write bez aktualnego tokenu jest odrzucany z 503. Klient widzący 503 retry'uje z discovery nowego primary. Maksymalna „ciemność” to ttl tokenu (30s).

7.11.4. Protokół rollback step-by-step

1. Detect — trigger naruszenia (auto lub human veto).
2. Announce — VPS emituje rollback_intent event do wszystkich działów (Arch, Platforma, Testy, Red, Blue, Gov) i na kanał alarmowy.
3. Fence — target otrzymuje fence_request; przestaje przyjmować writes.
4. Drain in-flight — VPS czeka max 10s na dokończenie in-flight requests (timeout → force-kill; klienci otrzymują retry-able 503).
5. Reverse write-through — sprawdzenie że legacy ma wszystkie writes z okresu od cutover (write-through replication log verification).
6. Swap lease — emisja nowego lease_token na legacy.
7. Traffic swap — routing/edge/DNS przełącza ruch na legacy.
8. Verify — Blue Team potwierdza metryki na legacy, contract tests pass.
9. Freeze target — target przechodzi w stan ROLLED_BACK; nie może być ponownie aktywowany bez nowej decyzji D3+.
10. Post-mortem — obowiązkowy w ciągu 24h; artefakt rollback_post-mortem.md trafia do Evidence (Plan 13).

7.11.5. Rollback-blocked states

Nie każdy stan systemu pozwala na rollback. v3.1 definiuje rollback_eligibility jako funkcję stanu legacy i fazy cyklu życia:

Stan legacy	Faza	Rollback auto	Rollback Council	Uwagi
HOT_STANDBY	STABILIZATION	TAK (D1/D2)	TAK (D3+)	optymalna faza rollback; write-through gwarantuje spójność
WARM_STANDBY	POST-STABILIZATION (7-30d)	NIE	TAK (D3, Council 4/4 fast)	replay WAL 15-60 min; dane po cutover mogą zostać
DECOMMISSIONED	OFFLINE permanent	NIE	NIE	rollback niemożliwy; jedyna droga = nowy rebuild legacy
ARCHIVED	tylko dane, infra zwolniona	NIE	NIE	restore wymaga pełnej provisioning cycle (dni)

KRYTYCZNE · Decommission = one-way door

Gdy legacy zostanie DECOMMISSIONED, rollback przestaje być opcją. Dlatego Plan 02 wymaga aby decyzja o DECOMMISSION była osobną decyzją D3+ z listą kontrolną (G-CUT-04 w Planie 13). Dopóki nie przejdziesz tej bramy świadomie, system zachowuje odwracalność.

7.11.6. Nowe testy bezpieczeństwa związane z rollback

ID	Nazwa	Input	Expected
T-REV-01	Auto-rollback na error_rate breach	inject 3x baseline error na target przez 10 min	AUTO_ROLLBACK w < 60s, traffic na legacy, post_mortem trigger
T-REV-02	Fence prevents split-brain	jednoczesny write na target i legacy z starym tokenem	target write: 200; legacy write: 503 fenced_out
T-REV-03	Write-through verify pre-rollback	write N na target; inject rollback; compare N na legacy	wszystkie N writes są na legacy; replication log OK
T-REV-04	Decommission blocks rollback	po DECOMMISSION trigger rollback attempt	403 decommission_reached, żaden ruch nie zmienia primary
T-REV-05	WARM rollback replay	w WARM_STANDBY trigger Council rollback; run replay WAL	legacy aktywne po replay; checksumy pasują do pre-cutover snapshot
T-REV-06	Post-mortem artifact delivery	auto-rollback triggered; sprawdzić Evidence w ciągu 24h	rollback_post_mortem.md w bundlu Evidence, Plan 13 G-CUT-03 pass

PLAN 08 · WARSTWA KANONICZNA

Canonical Evolution Plane

REBUILD · Rebuildability over Lineage w tym planie

Plan 08 v3.0 to największa rozbudowa względem v2.0. Dodano pełne drzewo decyzyjne Evolution/Refactor/Rebuild (flowchart), Delta Księgi dla każdej ścieżki, przykłady propozycji D3/D4/D5 oraz rozszerzony workflow propose → classify → council → execute (3 ścieżki). Kanon ewolucji kanonicznej definiuje warunki każdej ścieżki i rejestruje ścieżkę jako rebuild_path w każdym DecisionRecord.

8.1. Cel i umiejscowienie

Canonical Evolution Plane (CEP) jest centrum zarządzania ewolucją Księgi Kanonicznej systemu SYLION. Definiuje 10-krokowy cykl ewolucji, utrzymuje BaselineSnapshot z SHA-256, zarządza łańcuchem konsekwencji (ConsequenceChain), operuje 3 pętle feedbacku i pilnuje 8 twardych bramek G-CEV. W v3.0 CEP rozszerzono o klasyfikację ścieżek: Evolution / Refactor / Rebuild, drzewo decyzyjne klasyfikacji i rozszerzone workflowy per ścieżka.

- 10-krokowy cykl ewolucji: Trigger → Analyze → Classify → Propose → Council → Human? → Execute → Snapshot → Propagate → Feedback
- BaselineSnapshot: SHA-256 stanu Księgi; immutable po zapieczętowaniu
- ConsequenceChain: propagacja zmiany przez wszystkie zależne komponenty
- 3 pętle feedbacku: lessons / reputation / drift (Plan 07)
- 8 bramek G-CEV-01 do G-CEV-08 (niemodyfikowalne)
- 3 ścieżki transformacji: Evolution / Refactor / Rebuild — nowe v3.0

8.2. BaselineSnapshot i kontrakt niemutowalności

BaselineSnapshot jest kryptograficznym odciskiem palca stanu Księgi w danym punkcie czasu. Każdy snapshot zawiera SHA-256 treści Księgi, listę wszystkich aktywnych modułów i ich wersji, oraz hash poprzedniego snapshotu (hash chain). Po zapieczętowaniu snapshot jest niemutowalny — może być jedynie superseded przez następny snapshot.

- Generowanie: automatyczne po każdej zatwierdzonej zmianie D3+
- Hash: SHA-256(BookContent || ModuleVersions || prev_snapshot_hash)
- Supersede: nowy snapshot wskazuje na poprzedni przez prev_snapshot_id
- Weryfikacja: Chief Context (A1 runu) porównuje aktualny hash z baseline

8.3. Drzewo decyzyjne: Evolution / Refactor / Rebuild

Każda propozycja zmiany jest klasyfikowana do jednej z trzech ścieżek. Klasyfikacja odbywa się przez Chief Reasoning (Claude Opus 4) w etapie Classify na podstawie kryteriów poniżej. Klasyfikacja jest zapisywana jako rebuild_path w DecisionRecord i jest niezmiennym elementem historii decyzji.

REBUILD · Trzy ścieżki transformacji

EVOLUTION (zielony, C_EVOLUTION): rozszerzenie istniejącej funkcji bez zmiany kontraktu API. REFACTOR (pomarańczowy, C_REFACTOR): przepisanie implementacji z zachowaniem kontraktu. REBUILD (czerwony, C_REBUILD): wymiana systemu z nowym kontraktem lub greenfield.

Ścieżka	Kod	Kryterium klasyfikacji	Klasa decyzji min.	Delta Księgi
Evolution	PATH-EVO	Zmiana addytywna: nowe pole, nowa rola, nowe API endpoint; istniejące kontrakty niezmienione; baseline remains valid	D2	Nowa sekcja lub rozszerzenie istniejącej; prev_hash zachowany
Refactor	PATH-REF	Zmiana implementacji: nowy moduł zastępujący stary, ten sam kontrakt API; period dual-run (shadow) wymagany; baseline extended z nową wersją modułu	D3	Sekcja modyfikowana; stara wersja zachowana jako legacy_section
Rebuild	PATH-REB	Zmiana systemowa: nowy kontrakt API, nowa architektura, greenfield komponent; cutover strategy wymagana; baseline superseded po cutover	D4 (lub D5 jeśli greenfield całego planu)	Nowy snapshot; stary zarchiwizowany w Plan 13

8.4. Flowchart klasyfikacji ścieżki

Algorytm klasyfikacji Chief Reasoning:

1. Czy zmiana łamie istniejący kontrakt API lub schemat YAML? Tak → PATH-REB (D4+). Nie → idź do 2.
2. Czy zmiana wymaga zastąpienia istniejącego modułu nowym? Tak → PATH-REF (D3). Nie → idź do 3.
3. Czy zmiana jest addytywna i nie zmienia istniejących zachowań? Tak → PATH-EVO (D2). Nie → idź do 4.
4. Czy zmiana dotyczy całego komponentu (greenfield)? Tak → PATH-REB D5 (external review). Nie → PATH-REF (D3).

Chief Reasoning dokumentuje klasyfikację w ProposalContext z uzasadnieniem. Council może overrule klasyfikację przez D3 — takie overrule jest zawsze rejestrowane w Plan 13 jako zdarzenie classification_override.

8.5. Delta Księgi per ścieżka

Każda ścieżka transformacji ma specyficzny wzorzec modyfikacji Księgi Kanonicznej. Delta Księgi jest weryfikowana przez Chief Context przed zapieczętowaniem snapshotu.

Delta Księgi — PATH-EVO (Evolution)

YAML

```
# Przykład: dodanie nowej roli R-07 do ACP (Plan 01)
delta_type: additive
section: "01.RBAC"
change: "append_role"
new_content:
  role_id: R-07
  name: ModelObserver
  permissions: ["model_board_read", "reputation_read"]
impact_scope: [Plan01]
baseline_valid: true # prev_hash zachowany
rebuild_path: evolution
decision_class: D2
```

Delta Księgi — PATH-REF (Refactor)

YAML

```
# Przykład: przepisanie Auth Gateway (Plan 01) z nowym algorytmem hash
delta_type: module_replacement
section: "01.AuthGateway"
legacy_module: "acp-auth-gateway-v1"
target_module: "acp-auth-gateway-v3"
contract_preserved: true
dual_run_required: true
dual_run_duration_days: 7
impact_scope: [Plan01, Plan04]
legacy_section_archived_at: null # po cutover
rebuild_path: refactor
decision_class: D3
```

Delta Księgi — PATH-REB (Rebuild)

YAML

```
# Przykład: pełna wymiana VPS Provider Fabric (Plan 02)
delta_type: system_replacement
section: "02.ProviderFabric"
legacy_snapshot: "snap-vps-v2-final"
target_snapshot: null # generowany po cutover
cutover_strategy: shadow
adversarial_testing_required: true # Plan 07 AT-1/AT-2/AT-3
external_review_required: false # true jeśli D5
impact_scope: [Plan02, Plan10, Plan11]
rebuild_path: rebuild
decision_class: D4
```

8.6. ConsequenceChain

ConsequenceChain to mechanizm propagacji zmian przez wszystkie zależne komponenty. Dla każdej zatwierdzonej propozycji CEP oblicza „łańcuch konsekwencji”: listę planów i komponentów wymagających aktualizacji po wprowadzeniu zmiany. Propagacja jest automatyczna dla D2/D3 (Chief Context) i manualna dla D4/D5.

Typ zmiany	Propagacja automatyczna	Propagacja manualna	Plany dotknięte
Dodanie roli RBAC (EVO)	Aktualizacja matrycy uprawnień	Nie wymagana	Plan 01
Zmiana modelu w Board (REF)	PriceSheet CGP, ReputationUpdate	D4 confirmation	Plan 05, 10
Wymiana Auth Gateway (REF)	AuditLog schema, AgentToken schema	SecOfficer review	Plan 01, 04, 11
Rebuild VPS (REB)	Provider alokacja, Cost tracking	D4 + Plan 07 AT	Plan 02, 09, 10, 11
Rebuild catego planu (D5)	Wszystkie zależności planu	External Review + D4	Wszystkie plany zależne

8.7. Przykłady propozycji D3/D4/D5

Propozycja D3 — Refactor (przykład)

YAML

```
proposal_id: prop-00421
title: "Refactor KB Search Engine – wymiana SQLite FTS5 na PostgreSQL FTS"
rebuild_path: refactor
decision_class: D3
justification: >
  SQLite FTS5 osiągnął limit wydajności przy >100k dokumentów.
  PostgreSQL FTS zapewni 10x lepszą wydajność przy zachowaniu
  identycznego API /kb/v1/search.
contract_preserved: true
dual_run_days: 14
impact: [Plan03, Plan11]
evidence_required: [baseline_benchmark, target_benchmark, api_contract_test]
council_quorum: "4/4"
```

Propozycja D4 — Rebuild (przykład)

YAML

```
proposal_id: prop-00422
title: "Rebuild ACP Auth Engine – migracja na zewnętrzny IdP (Keycloak)"
rebuild_path: rebuild
decision_class: D4
justification: >
  Istniejący Auth Engine nie spełnia wymagań SSO dla 7 departamentów.
  Keycloak zapewni federację tożsamości, SAML2/OIDC i centralne MFA.
contract_preserved: false # nowe API /acp/v2/auth
cutover_strategy: parallel
adversarial_testing: true
impact: [Plan01, Plan04, Plan11, Plan13]
human_gate_required: true
operator_justification_required: true
```

Propozycja D5 — Greenfield (przykład)

```
YAML

proposal_id: prop-00423
title: "D5 Greenfield: Nowy Model & Decision Engine – wymiana całego Plan 05"
rebuild_path: rebuild
decision_class: D5
justification: >
  Aktualny Board (4 modele) nie skaluje się dla 7 departamentów.
  Nowy engine: 7 Chief modeli (jeden per departament) + meta-orchestrator.
external_reviewer: "dr-ai-safety-external-01"
external_review_scope: "Pełna architektura nowego engine + testy adversarialne"
council_quorum: "4/4 + human + external_reviewer"
adversarial_testing: true
impact: [Plan05, Plan06, Plan08, Plan10, Plan11, Plan13, Plan15]
```

8.8. Bramki G-CEV-01 do G-CEV-08

Kod	Nazwa	Warunek FAIL
G-CEV-01	Evidence przed głosowaniem D3+	Propozycja D3+ bez kompletnego Evidence pack w Plan 13 przed sesją Council
G-CEV-02	Snapshot SHA-256	BaselineSnapshot nie zawiera SHA-256 lub hash niezgodny z treścią Księgi
G-CEV-03	ConsequenceChain kompletny	Propozycja zatwierdzona ale ConsequenceChain nie obejmuje wszystkich zależnych planów
G-CEV-04	Feedback po runie	Run zakończony bez wysłania 3 strumieni feedbacku (lessons/reputation/drift) do Plan 07
G-CEV-05	Klasyfikacja ścieżki	Propozycja bez jawnej klasyfikacji rebuild_path (evolution/refactor/rebuild)
G-CEV-06	Adversarial Testing dla Rebuild	PATH-REB bez zakończonego Adversarial Testing (Plan 07 AT-1/AT-2/AT-3)
G-CEV-07	D5 wymaga External Review	Propozycja D5 bez zatwierdzonego external_reviewer w 72h
G-CEV-08	Delta Księgi per ścieżkę	Delta Księgi niezgodna z deklarowaną ścieżką (np. contract_preserved=false w EVO)

PLAN 08 · SPECYFIKACJA WYKONAWCZA

Canonical Evolution Plane

CEL I ZAKRES

Specyfikacja CEP v3.0. Zakres: 10-krokowy cykl ewolucji, BaselineSnapshot, ConsequenceChain, 3 pętle feedbacku, drzewo decyzyjne Evolution/Refactor/Rebuild, Delta Księgi per ścieżkę, przykłady D3/D4/D5, workflow propose→classify→council→execute, G-CEV-01 do G-CEV-08, gate'y i testy. Wersja: Pipeline 8.0.0.

CYKL EWOLUCJI — 10 kroków				
Krok	Nazwa	Aktor	Wyjście	Timeout
1	Trigger	ACP / Cron / Operator	RunTrigger z rebuild_path suggestion	n/a
2	Analyze	Chief Context (Gemini 2.5 Pro)	ProposalContext + BaselineSnapshot reference	10 min
3	Classify	Chief Reasoning (Claude Opus 4)	rebuild_path: evolution refactor rebuild	20 min
4	Propose	Chief Reasoning	Proposal YAML + Delta Księgi + Evidence pack	30 min
5	Council	Board (D2/D3/D4/D5)	DecisionRecord z rebuild_path + weighted_score	60 min
6	Human Gate?	Operator SYLION (D4+)	Zatwierdzenie lub odrzucenie D4	48h (jeśli D4)
7	Execute	Aktor per ścieżkę (patrz 8.9)	Zmiany aplikowane; dual-run lub cutover	zależne od ścieżki
8	Snapshot	Chief Context	Nowy BaselineSnapshot SHA-256	5 min
9	Propagate	ConsequenceChain engine	Wszystkie zależne plany zaktualizowane	30 min
10	Feedback	Plan 07 (Evolution Safety)	3 strumienie: lessons/reputation/drift	5 min
WORKFLOW — propose → classify → council → execute				

PATH-EVO: Evolution

1. Propozycja addytywna złożona; Chief Reasoning klasyfikuje PATH-EVO
2. Council D2 (3/4 wagowo); timeout 60 min
3. Execute: zmiana aplikowana przez Chief Context; brak dual-run
4. Snapshot nowy BaselineSnapshot; ConsequenceChain propaguje do zależnych planów
5. Feedback: 3 strumienie do Plan 07

PATH-REF: Refactor

- 1. Propozycja zastępowania modułu; Chief Reasoning klasyfikuje PATH-REF
- 2. Council D3 (4/4); Evidence pack wymagany (G-CEV-01)
- 3. Execute: target_module uruchamiany w shadow mode przez dual_run_days
- 4. Porównanie legacy vs target; jeśli delta < 5% → cutover; jeśli > 5% → block (G-CEV-06)
- 5. Legacy moduł → LEGACY_DECOMMISSIONED; nowy snapshot

PATH-REB: Rebuild

- 1. Propozycja systemowa; Chief Reasoning klasyfikuje PATH-REB
- 2. D4 Human Gate (+ D5 jeśli greenfield); adversarial testing Plan 07
- 3. Execute: nowy system provisioned w PHANTOM tier (Plan 02); shadow running
- 4. Adversarial Testing AT-1/AT-2/AT-3 (Plan 07); wszystkie fazy muszą pass
- 5. Cutover D4; stary snapshot zarchiwizowany; nowy BaselineSnapshot

MODEL DOMENOWY		
Encja	Kluczowe pola	Klucze / relacje
Proposal	proposal_id (UUID), title, rebuild_path (evolution/refactor/rebuild), decision_class, state, proposer_id, proposed_at, evidence_pack_sha256, delta_book_sha256, council_decision_id	PK proposal_id; immutable after decided
BaselineSnapshot	snapshot_id, book_sha256, modules_version_map{}, created_at, superseded_at, prev_snapshot_id, sealed_by_run_id	PK snapshot_id; hash chain; immutable after sealed
ConsequenceChain	chain_id, proposal_id, affected_plans[], propagation_state, completed_at, error_plans[]	PK chain_id; FK proposal_id
EvidencePack	pack_id, proposal_id, items[], total_sha256, submitted_at, reviewed_by	PK pack_id; FK proposal_id; items: sha256 per evidence
FeedbackStream	stream_id, run_id, stream_type (lessons/reputation/drift), payload_sha256, sent_at, processed_by_plan07	PK stream_id; FK run_id
API		
YAML		
<pre>POST /cev/v1/proposals Body: { "title": "str", "rebuild_path": "evolution refactor rebuild", "delta_book": {...}, "evidence_pack": [...] } Response 201: { "proposal_id": "uuid", "state": "PROPOSED" }</pre>		
<pre>POST /cev/v1/proposals/{id}/classify Body: { "classifier": "chief_reasoning", "classification": "refactor", "justification": "str" } Response 200: { "rebuild_path": "refactor", "decision_class": "D3" }</pre>		
<pre>POST /cev/v1/proposals/{id}/execute Body: { "council_decision_id": "uuid", "execute_mode": "shadow parallel cut-and-replace" } Response 200: { "state": "EXECUTING", "dual_run_id": "uuid" }</pre>		

```
POST /cev/v1/snapshots
Body: { "proposal_id": "uuid", "book_sha256": "sha256:..." }
Response 201: { "snapshot_id": "uuid", "book_sha256": "sha256:..." }

GET /cev/v1/snapshots/current
Response 200: { "snapshot_id": "uuid", "book_sha256": "sha256:...",
               "created_at": "ISO8601", "rebuild_path": "refactor" }

POST /cev/v1/feedback
Body: { "run_id": "uuid", "stream_type": "lessons|reputation|drift",
       "payload_sha256": "sha256:..." }
Response 201: { "stream_id": "uuid", "processed_by_plan07": false }
```

TESTY					
Kod	Opis	Input	Oczekiwany wynik		
T-CEV-01	Klasyfikacja EVO	Propozycja addytywna bez łamania kontraktu	rebuild_path=evolution, D2 Council		
T-CEV-02	Klasyfikacja REF	Propozycja zastępowania modułu, kontrakt zachowany	rebuild_path=refactor, D3 Council, dual_run_required=true		
T-CEV-03	Klasyfikacja REB	Propozycja łamiąca kontrakt API	rebuild_path=rebuild, D4 Human Gate		
T-CEV-04	G-CEV-01 brak Evidence	Propozycja D3 bez evidence_pack	G-CEV-01 FAIL, Council zablokowany		
T-CEV-05	G-CEV-05 brak ścieżki	Propozycja bez rebuild_path	G-CEV-05 FAIL, classify wymagane		
T-CEV-06	G-CEV-06 AT wymagane	PATH-REB bez Adversarial Testing	G-CEV-06 FAIL, execute zablokowany		
T-CEV-07	Snapshot SHA-256	POST /snapshots z poprawnym book_sha256	201, snapshot_id, hash chain OK		
T-CEV-08	D5 external review	Propozycja D5 bez external_reviewer	G-CEV-07 FAIL, D5 execute zablokowany		
ANEKS A — Ścieżka × D-class × Evidence × Delta Księgi					
Ścieżka	D-class min.	Evidence wymagane	Delta Księgi	Adver sarial?	Snaps hot?
evolution	D2	1 dowód (baseline_benchmark)	additive; prev_hash ważny	Nie	Tak
refactor	D3	3 dowody (benchmark + API contract + dual_run_report)	module_replacement; legacy_section zachowana	Nie (jeśli bez REB)	Tak

Ścieżka	D-class min.	Evidence wymagane	Delta Księgi	Adversarial?	Snapshots?
rebuild	D4	5+ dowodów (+ adversarial results + cutover plan)	system_replacement; stary snapshot zarchiwizowany	Tak (A T-1/AT -2/AT-3)	Tak (nowy)
rebuild (D5)	D5	5+ dowodów + external review report	greenfield; wszystkie stare sekcje archiwizowane	Tak	Tak (nowy)

PLAN 09 · WARSTWA KANONICZNA

Operator Environment Plane

REBUILD · Rebuildability over Lineage w tym planie

Operator Environment Plane (OEP) w v3.0 przestaje być wyłącznie narzędziem zarządzania środowiskami ACTIVE/FROZEN — staje się centrum orkiestracji rebuildu. W trybie SYSTEM_REBUILD_MODE operator widzi jednocześnie linię legacy i linię target, monitoruje postęp 7 działań (Plan 15), zatwierdza decyzje D4/D5 z panelu dedykowanego oraz inicjuje cutover wg strategii cutover_strategy. OEP obsługuje trzy ścieżki: Evolution, Refactor i Rebuild, i eksponuje operatorowi pole implementation_strategy każdego modułu. Zasada: żaden cutover nie następuje bez jawnej zgody operatora zarejestrowanej jako decyzja D4 lub D5.

9.1. Cel i umiejscowienie

Operator Environment Plane (OEP) zarządza środowiskami wykonawczymi pipeline SYLION. Każde środowisko to izolowana przestrzeń zasobów (compute, storage, network, secrets) przypisana do kontekstu: dev, staging, produkcja lub shadow-phantom. OEP definiuje pełny cykl życia środowiska (DRAFT → DESTROYED), gwarantując separację zasobów, rozliczalność kosztów i traceability operacji. W v3.0 OEP rozszerzone jest o tryb REBUILD_MODE, w którym środowiska legacy i target działają równolegle do momentu decyzji cutover.

- OEP jest serwisem orkiestratora pipeline (nie infrastruktury).
- VPS Provider Plane (Plan 02) dostarcza compute; OEP nim zarządza.
- Cost Governance Plane (Plan 10) konsumuje metadane OEP do rozliczeń.
- W REBUILD_MODE OEP orkiestruje środowiska z Planami 02, 14, 15 i 16.

KRYTYCZNE · Zasada separacji

Środowisko produkcyjne nigdy nie dzieli zasobów z dev lub staging. W trybie REBUILD_MODE środowisko target jest zawsze phantom-shadow do momentu cutover zatwierdzonego decyzją D4.

9.2. Trzy tryby operacyjne OEP (v3.0)

W v3.0 OEP operuje w jednym z trzech trybów odpowiadających ścieżkom implementacyjnym. Tryb determinuje aktywne widoki UI, panele decyzyjne oraz dopuszczalne akcje środowiskowe. Zmiana trybu to co najmniej D4.

Tryb EVOLUTION

> EVOLUTION

Standardowy UI; jeden zestaw środowisk per typ; D0-D2 auto; D3 Council; D4 Human. Brak środowisk równoległych legacy/target. Domyślny tryb startowy OEP.

Tryb REFACTOR

> REFACTOR

Widok dual-track: środowisko legacy obok target (contract-preserving). Operator steruje podziałem ruchu i zatwierdza stopniowy cutover. Aktywowany po decyzji D3 z Council 4/4.

Tryb REBUILD (SYSTEM_REBUILD_MODE)

> REBUILD

Pełen Rebuild Dashboard (6 paneli); dwa zestawy środowisk równoległe; 7 działów (Plan 15); panel D4/D5 oczekujących; External Review (D5); cutover control; powiadomienia canon_compact (Plan 16). Aktywowany po decyzji D4/D5 + trigger canon_compact.md z Plan 16.

9.3. Sześć stanów środowiska

OEP definiuje sześć kanonicznych stanów (acykliczny graf przejść). Przejścia są atomowe i logowane jako EnvironmentEvent. W REBUILD_MODE stany obowiązują zarówno dla środowisk legacy, jak i target.

Stan	Opis skrócony	Warunek wejścia	Dozwolone akcje	Aktor
DRAFT	Zarejestrowane, bez walidacji; zasoby nie rezerwowane	POST /environments	edit, qualify, delete	operator / D0
QUALIFIED	Walidacja OK; zasoby dostępne; VPS nie alokowany	qualify gates G-OPE-01..05 PASS	provision, back-to-draft, delete	operator / D0
PROVISIONING	VPS alokuje zasoby; stan przejściowy; timeout 15 min	VPS przyjął zlecenie	cancel	operator / auto
ACTIVE	W pełni operacyjne; compute aktywne; koszty naliczane	VPS callback provisioning_done	freeze, destroy, schedule_run	operator / Cost-Gov
FROZEN	Compute wstrzymane; storage zachowane; koszty compute zawieszone	freeze_cmd lub auto-freeze	unfreeze, destroy	operator
DESTROYED	Stan terminalny; compute/network usunięte; rekord immutable	DELETE + confirm	brak	N/A

- Dodatkowe stany RebuildTrack w REBUILD_MODE (nakładka na ACTIVE): CUTOVER_PENDING / CUTOVER_DONE — śledzone przez encję RebuildTrack oddzielnie.
- Ograniczenie: PHANTOM-SHADOW nie może być FROZEN dłużej niż 48h.
- Legacy DESTROYED dopiero po quarantine_period (min. 7 dni po cutover), gate G-OPE-11.

9.4. Typy środowisk

Typ	Przeznaczenie	Tier VPS (min)	Retention storage	Prod traffic
DEV	Eksperymenty, lokalne testy	STANDARD	7 dni	NIE
STAGING	Integracja, testy regresji	PRO	30 dni	NIE
PRODUCTION	Obciążenie produkcyjne	STATE lub PHANTOM	365 dni	TAK
PHANTOM-SHA DOW	Shadow-mode; target w REBUILD_MODE	PHANTOM	14 dni	MIRROR

9.5. Relacje do innych planów

Plan	Zależność OEP
02 — VPS	Dostarcza compute; musi być APPROVED; w REBUILD_MODE obsługuje stany rebuild/cutover
10 — Cost Gov.	Konsumuje zdarzenia billing; budget_trip(80%) → auto-freeze; rebuild ma osobny budget_id
14 — Rebuild Orch.	Definiuje workflow rebuildu; OEP realizuje execute_cutover i legacy_decommission
15 — Multi-Team	7 działów raportuje postęp do Panelu 2 Rebuild Dashboard
16 — Compact	canon_compact.md jest triggerem REBUILD_MODE (G-OPE-09); Panel 6 subskrybuje zdarzenia
13 — Governance	Przechowuje DecisionRecord D4/D5; Evidence pack; external_review_log.md

9.6. Rebuild Orchestration UI — sześć paneli

Po aktywacji SYSTEM_REBUILD_MODE OEP przełącza UI operatora na Rebuild Dashboard z sześcioma panelami odświeżanymi co 15 s (WebSocket). Snapshoty stanu są zachowywane jako evidence w Plan 13.

Panel 1: Legacy vs Target (side-by-side)

Dwie kolumny: lewa — środowiska legacy (baseline MZ 3.5.5), prawa — target. Dla każdego modułu z greenfield_module_map.json (Plan 14) widoczny jest status i badge implementation_strategy (reuse/refactor/replace/greenfield). Kolory statusów:

- LEGACY_ACTIVE — szary #90A4AE
- TARGET_BUILDING — pomarańczowy C_REFACTOR (#F57C00)
- TARGET_VALIDATED — zielony C_EVOLUTION (#2E7D32)
- CUTOVER_PENDING — fiolet C_D5 (#6A1B9A) — oczekuje Human approve
- CUTOVER_DONE — granatowy C_PRIMARY (#1B3A5C)
- DECOMMISSIONED — czerwony C_REBUILD (#C62828) z przekreśleniem

Panel 2: Wskaźniki postępu 7 działów (Plan 15)

Dział (Plan 15)	Metryki w panelu	Alert gdy
Architektura	target_architecture_delta ukończony w %, propozycje Księgi otwarte	delta > 7 dni bez zatwierdzenia
Platforma	środowisk target ACTIVE vs PROVISIONING, bundle pipeline status	provisioning_failed w 24h
Implementacja	modułów greenfield DONE/TOTAL, MR bez review	MR > 48h bez review
Testy i Walidacja	testy kontraktowe PASS/FAIL, gate'y walidacyjne	jakikolwiek gate FAIL
Red Team	scenariusze adversarial PENDING/DONE, blast radius score	blast_radius > threshold
Blue Team	alerty telemetrii, DFIR otwarte	DFIR otwarty > 4h
Governance	decyzje D3+ PENDING/RESOLVED, Evidence pack kompletny	D4/D5 pending > 24h

Panel 3: Decyzje D4/D5 oczekujące na Human approve

Agreguje decyzje D4/D5 oczekujące na Human Gate. D4: 1 kliknięcie + justification (min. 20 znaków). D5: kliknięcie + potwierdzenie External Review (Panel 4). Brak reakcji na D4/D5 przez 48h → eskalacja do Plan 13. Decyzje D5 wyróżnione fioletowym tłem C_D5.

REBUILD · D5 — Council 4/4 + Human + External Review

Decyzje D5 (cutover systemu, zmiana arch. bezpieczeństwa, zmiana fundamentów Księgi) wymagają trzech niezależnych zatwierdzeń: (1) Council 4/4, (2) Human Gate, (3) External Review (red team zewnętrzny lub drugi operator niezależny). OEP blokuje akcję D5 do zebrania wszystkich trzech podpisów (G-OPE-12).

Panel 4: External Review (dla D5)

Status: PENDING / IN_PROGRESS / APPROVED / REJECTED. Operator wysyła pakiet review z panelu (Evidence pack PDF z Plan 13). Wynik zapisywany w external_review_log.md (artefakt v3.0) i Plan 13. Timeout recenzji: 72h → eskalacja. Pola pakietu: decision_id, scope, evidence_pack_url, blast_radius_report.

Panel 5: Cutover Control Panel

Strategia	Opis	Ryzyko	Cofnięcie
parallel	Ruch stopniowo przesuwany 10%→25%→50%→100%; każdy krok D4	Podwójne koszty podczas przejścia	Każdy krok bez downtime
shadow	Target dostaje kopię ruchu legacy; odpowiedzi z target nie trafiają do klientów; porównanie rozbieżności	Niskie; brak wpływu na prod	Stop shadow bez wpływu

Strategia	Opis	Ryzyko	Cofnięcie
cut-and-replace	Jednorazowe przełączenie: legacy → FROZEN, target → ACTIVE; wymaga okna maintenance	Wysokie; brak gradualności	Rollback D4: target → FROZEN, legacy → ACTIVE

- Emergency Rollback: dostępny w każdej chwili; D4; natychmiastowy.
- Quarantine: min. 7 dni po cutover przed DESTROYED legacy (G-OPE-11).

Panel 6: Powiadomienia o canon_compact (Plan 16)

Panel integruje się z Compaction & Memory Protocol (Plan 16). Powiadomienia gdy: (1) wygenerowano nowy canon_compact.md (rytm 24h lub pre-D3+), (2) compact zawiera nowe decyzje D3+ wymagające uwagi, (3) zbliża się trigger pre-bundle, (4) zmieniła się implementation_strategy modułu. Operator może zażądać pełnego operator-compact w każdej chwili („Pobierz kompakt operatora”). Historia: ostatnie 30 kompaktów.

REBUILD · Kiedy operator powinien dostać operator-compact?

Operator-compact (Plan 16) trafia do operatora: (1) automatycznie co 24h — rytmiczny digest, (2) natychmiast przed D3+ — pre-decision compact, (3) przed bundle build — pre-bundle compact, (4) po każdej zmianie trybu OEP (NORMAL/REFACTOR/REBUILD_MODE). OEP Panel 6 dostarcza powiadomienia i przechowuje ostatnie 30 kompaktów.

PLAN 09 · SPECYFIKACJA WYKONAWCZA

Operator Environment Plane

CEL I ZAKRES

Specyfikacja OEP v3.0 definiuje model domenowy, tabele przejść stanów, schematy YAML/JSON, workflowy operacyjne (w tym REBUILD_MODE), kontrakty API, gate'y jakości i testy automatyczne. Zakres: tworzenie, kwalifikacja, provisioning, aktywacja, zamrażanie i niszczenie środowisk SYLION, plus tryby Evolution/Refactor/Rebuild z pełną orkiestracją cutover. Wersja: Pipeline 8.0.0, MZ baseline 3.5.5.

SCHEMATY YAML — *oep_module.yaml*

Moduł OEP Environment Manager (refactor)

YAML

```
apiVersion: sylion.io/v3
kind: Module
metadata:
  module_id: "oep-environment-manager"
  name: "OEP Environment Manager"
  plan: "09"
spec:
  implementation_strategy: refactor
  current_baseline: MZ 3.5.5
  target_baseline: "oep-env-manager-v3"
  cutover_strategy: shadow
  version: "3.0.0"
  supported_modes: [EVOLUTION, REFACTOR, REBUILD]
  rebuild_dashboard:
    panels:
      - id: "legacy-vs-target"
        refresh_s: 15
        websocket: true
      - id: "department-progress"
        source_plan: "15"
      - id: "decision-queue-d4d5"
        source_plan: "13"
        refresh_s: 10
      - id: "external-review"
        source_plan: "13"
        artifact: "external_review_log.md"
      - id: "cutover-control"
        strategies: [parallel, shadow, cut-and-replace]
        emergency_rollback: true
        quarantine_days: 7
      - id: "canon-compact-notifications"
        source_plan: "16"
        compact_retention: 30
```

Moduł OEP Rebuild Dashboard (greenfield)

YAML

```

apiVersion: sylion.io/v3
kind: Module
metadata:
  module_id: "oep-rebuild-dashboard"
  name: "OEP Rebuild Orchestration Dashboard"
  plan: "09"
spec:
  implementation_strategy: greenfield
  current_baseline: null
  target_baseline: "oep-rebuild-dashboard-v3"
  cutover_strategy: cut-and-replace
  activation_condition:
    trigger: "canon_compact.md pre-rebuild"
    source_plan: "16"
    min_decision_class: D4
  integrations:
    - { plan: "02", type: vps_states_rebuild }
    - { plan: "13", type: evidence_and_decisions }
    - { plan: "14", type: greenfield_module_map }
    - { plan: "15", type: department_progress }
    - { plan: "16", type: operator_compact }

```

Schemat environment.yaml (v3.0)

YAML

```

apiVersion: sylion.io/v3
kind: Environment
metadata:
  env_id: "env-7f3a9c"
  name: "sprint-08-staging"
  created_by: "operator-001"
spec:
  type: staging
  env_role: standard      # standard | legacy | target
  rebuild_track_id: null  # wypełniane w REBUILD_MODE
  implementation_strategy: reuse # reuse | refactor | replace | greenfield
  provider_id: "vps-prv-002"
  budget_id: "bgt-sprint08"
  quota_profile_id: "qp-staging-std"
  resources:
    compute:
      vcpu: 8
      ram_gb: 32
      gpu_count: 0
      region: "eu-central-1"
    storage:
      persistent_gb: 100
      ephemeral_gb: 50
    network:
      vlan_id: "vlan-042"
      egress_limit_gb_h: 10
      inbound_endpoint: "https://staging.sylion.internal"
  secrets:
    refs:
      - "sec-ref:db-password-staging"
      - "sec-ref:api-key-model-engine"

```



```
retention_policy:
  storage_days: 30
  log_days: 90
auto_freeze_after_idle_days: 3
```

Schemat RebuildTrack (nowy v3.0)

YAML

```
apiVersion: sylion.io/v3
kind: RebuildTrack
metadata:
  track_id: "rbtrk-001"
  decision_id: "dec-d5-001"
spec:
  mode: rebuild # evolution | refactor | rebuild
  legacy_env_id: "env-legacy-001"
  target_env_id: "env-target-001"
  cutover_strategy: parallel # parallel | shadow | cut-and-replace
  traffic_split_pct: 10 # aktualny % ruchu na target
  traffic_steps: [10, 25, 50, 100]
  cutover_status: in_progress
  quarantine_days: 7
  department_progress: # dane z Plan 15
    architecture: 85
    platform: 70
    implementation: 60
    testing: 45
    red_team: 30
    blue_team: 55
    governance: 80
  compact_notifications: # dane z Plan 16
    last_compact_at: "2026-04-15T08:00:00Z"
    pending_d3plus: 2
```

STANY MODUŁÓW — Rebuildability			
Moduł OEP	Stan v3.0	Ścieżka	Uwagi
Environment Manager	TARGET_BUILDING	refactor	Shadow mode; contract preserved; dodanie REBUILD_MODE API
State Machine Controller	LEGACY_ACTIVE	evolution	Rozszerzenie o stany CUTOVER_PENDING / CUTOVER_DONE
Rebuild Dashboard	TARGET_BUILDING	greenfield	Nowy moduł v3.0; cutover po testach E2E
Cutover Proxy	TARGET_BUILDING	greenfield	Nowy moduł v3.0; blokuje start rebuildu bez D4/D5
Compact Notifier	TARGET_BUILDING	greenfield	Integracja z Plan 16; nowy w v3.0
MODEL DOMENOWY			

Encja	Kluczowe pola	Klucze / relacje
Environment	env_id (UUID), name, type, state (6 stanów), provider_id, budget_id, quota_profile_id, env_role (standard legacy target), rebuild_track_id (nullable), implementation_strategy	PK env_id; FK provider_id→VPS; FK budget_id→CostGov; FK rebuild_track_id→RebuildTrack
RebuildTrack	track_id (UUID), mode (evolution refactor rebuild), legacy_env_id, target_env_id, cutover_strategy, cutover_status, traffic_split_pct, quarantine_days, department_progress{7 działań}, compact_notifications	PK track_id; FK legacy_env_id, target_env_id→Environment
Stage	stage_id, env_id, from_state, to_state, triggered_by, triggered_at, reason	PK stage_id; FK env_id→Environment
EnvironmentEvent	event_id, env_id, event_type, actor_role, decision_class (D0-D5), rebuild_path (evolution refactor rebuild null), payload_json, created_at	PK event_id; FK env_id→Environment; immutable
DecisionRecord	decision_id, env_id, decision_class (D4 D5), scope, proposed_action, proposer_dept, submitted_at, human_approved_at, external_review_id (nullable for D5), status	PK decision_id; FK env_id; FK external_review_id→ExternalReview
ExternalReview	review_id, decision_id, reviewer_external_id, sent_at, responded_at, verdict (approved rejected), comment, evidence_pack_url	PK review_id; FK decision_id→DecisionRecord
QuotaProfile	qp_id, env_type, max_vcpu, max_ram_gb, max_storage_gb, max_egress_gb_h, tier_required	PK qp_id; 1:N → Environment

TABELA PRZEJŚĆ STANÓW 6x6						
Z \ Do	DRAFT	QUALIFIED	PROVISIONING	ACTIVE	FROZEN	DESTROYED
DRAFT	-	qualify_ok (D0/D4)	-	-	-	delete (D4) oper.
QUALIFIED	revert (D4)	-	provision (D0)	-	-	delete (D4)
PROVISIONING	prov_timeout (D0)	cancel (D4)	-	prov_done (D0)	-	prov_failed (D0)
ACTIVE	-	-	-	-	freeze (D4) / budget_trip (D0)	destroy (D4)
FROZEN	-	-	-	unfreeze (D4)	-	destroy (D4)
DESTROYED	-	-	-	-	-	-

W REBUILD_MODE środowisko target ma nakładkę RebuildTrack z polami cutover_status (CUTOVER_PENDING / CUTOVER_DONE) — środowisko pozostaje ACTIVE, ale RebuildTrack śledzi fazę cutover oddzielnie.

WORKFLOWY

WF-OPE-01: Bootstrap → Qualify → Provision (Evolution)

- 1. POST /environments → DRAFT (G-OPE-01 schema); brak pól → 422
- 2. POST /qualify → gate'y G-OPE-01..05; PASS → QUALIFIED; FAIL → DRAFT + błędy
- 3. POST /provision → PROVISIONING (timer 15 min); VPS callback prov_done → ACTIVE
- 4. prov_failed lub timeout → DRAFT; Cost Governance: start_billing po ACTIVE

WF-OPE-04: Activate Rebuild Track (REBUILD_MODE)

- 1. Operator inicjuje REBUILD_MODE (D4 lub D5); OEP weryfikuje trigger Plan 16 (G-OPE-09)
- 2. OEP tworzy RebuildTrack (legacy_env_id + target_env_id + cutover_strategy)
- 3. Provisionuje środowisko target jako phantom-shadow na osobnym VLAN
- 4. Aktywuje Rebuild Dashboard (6 paneli); subskrybuje Plan 15 (postęp działań) i Plan 16 (compact)
- 5. Emituje EnvironmentEvent(type=rebuild_track_started) do Plan 13

WF-OPE-05: Cutover Execution

- 1. Operator wybiera strategię (parallel/shadow/cut-and-replace) w Panel 5
- 2. OEP weryfikuje G-OPE-10..12: działły READY (Plan 15), D4/D5 approved, External Review (D5)
- 3. parallel: traffic_split_pct krokami 10→25→50→100; każdy krok D4
- 4. shadow: Cutover Proxy mirror; operator obserwuje rozbieżności bez wpływu na prod
- 5. cut-and-replace: legacy → FROZEN; target → ACTIVE production; quarantine 7 dni
- 6. Po quarantine: DESTROYED legacy (D4); EnvironmentEvent(legacy_decommissioned) do Plan 13

WF-OPE-06/07: Freeze i Destroy

- 1. Freeze: POST /freeze lub auto-trigger Cost Gov → suspend_compute; ACTIVE→FROZEN
- 2. Destroy: DELETE?confirm=true; G-OPE-07 (brak runów) + G-OPE-11 (quarantine OK)
- 3. VPS: destroy_environment; stan → DESTROYED (immutable); close_billing Cost Gov

API

YAML

```
# Endpointy klasyczne (Evolution / Refactor)
POST    /env/v1/environments                # tworzy DRAFT
POST    /env/v1/environments/{id}/qualify  # gate G-OPE-01..05
POST    /env/v1/environments/{id}/provision # start provisioning (timeout 15 min)
POST    /env/v1/environments/{id}/freeze   # ACTIVE→FROZEN
POST    /env/v1/environments/{id}/unfreeze # FROZEN→PROVISIONING→ACTIVE
DELETE  /env/v1/environments/{id}?confirm=true # DESTROYED; G-OPE-07+11
GET     /env/v1/environments/{id}          # stan + env_role + rebuild_track_id
GET     /env/v1/environments/{id}/events   # EnvironmentEvent log (D0-D5)

# Endpointy REBUILD_MODE (v3.0)
POST    /env/v1/rebuild-tracks
# wymaga D4/D5 + trigger Plan 16 (G-OPE-09)
```

```
Body: { mode, legacy_env_id, target_env_id, cutover_strategy, decision_id }
201: { track_id, status: in_progress } | 403: requires_D4_or_D5
```

```
POST    /env/v1/rebuild-tracks/{id}/cutover-step
# przesuwam traffic_split_pct (parallel); wymaga D4
Body: { target_pct: 25, decision_id } → 200: { traffic_split_pct, status }
```

```
POST    /env/v1/rebuild-tracks/{id}/emergency-rollback
# natychmiastowy rollback (D4); justification min 20 znaków
200: { cutover_status: rolled_back, legacy_env_state: active }
```

```
GET     /env/v1/rebuild-tracks/{id}/dashboard
200: RebuildDashboardSnapshot (wszystkie 6 paneli)
```

```
POST    /env/v1/decisions/{id}/approve
# Human Gate D4/D5; D5 wymaga external_review_id (G-OPE-12)
Body: { operator_token, justification }
200: { decision_id, status: approved } | 403: requires_external_review_for_D5
```

HARD GATE'Y

Kod	Nazwa	Warunek FAIL	Klasa	Blokuje
G-OPE-01	Schema kompletności	Brak pól: name, type, provider_id, budget_id, resources	D0	qualify
G-OPE-02	Typ vs tier VPS	production/phantom na providerze STANDARD	D0	qualify
G-OPE-03	QuotaProfile	Alokacja przekracza QuotaProfile dla typu środowiska	D0	qualify
G-OPE-04	Provider APPROVED	provider_id nie w stanie APPROVED (Plan 02)	D0	qualify + provision
G-OPE-05	Dostępność zasobów	VPS brak zasobów dla regionu i tieru	D0	provision
G-OPE-06	Budget aktywny	budget_id nie istnieje lub przekroczony (Plan 10)	D0	provision + activate
G-OPE-07	Brak aktywnych runów	Aktywne runy w środowisku (Plan 11)	D4	destroy
G-OPE-08	Contract check	Target nie spełnia kontraktów API legacy (Plan 12)	D3	cutover step
G-OPE-09	Trigger canon_compact wymagany	Brak aktywnego triggera z canon_compact.md (Plan 16)	D4	rebuild_track create
G-OPE-10	Działy READY	Przynajmniej jeden z 7 działów (Plan 15) nie osiągnął READY_FOR_CUTOVER	D4	cutover final
G-OPE-11	Quarantine zakończony	Próba DESTROYED legacy przed upływem quarantine_days	D4	destroy legacy
G-OPE-12	D5 wymaga External Review	Decyzja D5 bez zatwierdzonej recenzji zewnętrznej	D5	cutover D5
KLASY DECYZJI D0–D5 W KONTEKŚCIE OEP				

Klasa	Przykład operacji OEP	Aktor	Ścieżka
D0	Qualify auto, prov_done callback, auto-freeze po budget_trip	OEP auto	—
D1	Odczyt stanu środowiska, historia zdarzeń	Chief Reviewer	—
D2	Zmiana QuotaProfile (nie-prod); zmiana retention_policy	Council 3/4	evolution
D3	Inicjacja trybu REFACTOR; zmiana konfiguracji sieci	Council 4/4	refactor
D4	Destroy production; każdy krok cutover parallel; inicjacja REBUILD_MODE (moduł); emergency rollback	Human Gate	rebuild (moduł)
D5	Inicjacja SYSTEM_REBUILD_MODE (pełny rebuild); cutover całego systemu; zmiana arch. bezpieczeństwa	Council 4/4 + Human + External Review	rebuild (system)

REBUILD · D5 — Greenfield/Systemic w OEP

Decyzja D5 w OEP (pełny SYSTEM_REBUILD_MODE lub cutover systemu) wymaga: (1) Council 4/4 ze wszystkich działów, (2) Human Gate, (3) External Review z wynikami pozytywnymi w external_review_log.md. OEP blokuje akcję technicznie (G-OPE-12) do zebrania wszystkich trzech podpisów. Kolor panelu D5: fiolet C_D5 (#6A1B9A).

TESTY

Kod	Opis	Input	Oczekiwany wynik
T-OPE-01	Tworzenie środowiska DRAFT	POST /environments z kompletem pól	201, state=draft, env_id wygenerowany
T-OPE-02	Qualify z kompletną konfig	POST /qualify; provider APPROVED	200, state=qualified, gate'y PASS
T-OPE-03	Qualify FAIL — provider REVIEW	POST /qualify; provider w stanie REVIEW	409, failed_gates=[G-OPE-04]
T-OPE-04	Provision timeout	POST /provision; VPS nie odpowiada 15 min	Stan → DRAFT, EnvironmentEvent(prov_timeout)
T-OPE-05	Freeze aktywnego środowiska	POST /freeze; env ACTIVE, 0 aktywnych runów	200, state=frozen, billing compute suspended
T-OPE-06	Destroy z aktywnym runem	DELETE; aktywny run	409, G-OPE-07 FAIL, active_runs_exist
T-OPE-07	Auto-freeze po budget_trip	Cost Gov. emituje budget_trip(80%)	ACTIVE→FROZEN auto, EnvironmentEvent(auto_frozen)

Kod	Opis	Input	Oczekiwany wynik			
T-OPE-08	Aktywacja REBUILD_MODE bez canon_compact	POST /rebuild-tracks bez triggera Plan 16	403, G-OPE-09 FAIL			
T-OPE-09	Cutover step parallel D4	POST /cutover-step target_pct=25, D4 approved	200, traffic_split_pct=25, Panel 5 zaktualizowany			
T-OPE-10	Emergency rollback	POST /emergency-rollback z justification, D4	200, rolled_back, legacy ACTIVE, target FROZEN			
T-OPE-11	D5 cutover bez External Review	POST /decisions/{id}/approve dla D5, brak review_id	403, G-OPE-12 FAIL			
T-OPE-12	Destroy legacy przed quarantine	DELETE legacy po 3 dniach (quarantine=7)	409, G-OPE-11 FAIL, remaining_days=4			
T-OPE-13	Panel 6 — compact trigger	Plan 16 emituje compact_generated(pre-d3+)	Panel 6 wyświetla alert; log w historii kompaktów			
T-OPE-14	Panel 2 — dział Red Team NOT READY	Plan 15 API red_team.status=NOT_READY	Panel 2 alert; G-OPE-10 FAIL przy cutover final			
ANEKS A — 6 stanów × akcje × aktor						
Stan	Dozwolone akcje		Aktor	Klasa decyzji		
DRAFT	edit_config, qualify, delete		operator, pipeline-seed	D4 (delete), D0/D4 (qualify)		
QUALIFIED	provision, revert_to_draft, delete		operator, auto-pipeline	D0 (provision), D4 (revert/delete)		
PROVISIONING	cancel		operator, auto-timeout	D4 (cancel), D0 (timeout)		
ACTIVE	freeze, destroy, schedule_run, start_rebuild_track		operator, Cost-GovPlane, Run-Lifecycle	D4 (freeze/destroy/rebuild), D0 (auto-freeze)		
FROZEN	unfreeze, destroy		operator	D4 (oba)		
DESTROYED	brak (terminalny)		brak	N/A		
ANEKS B — Typ środowiska → tier VPS → QuotaProfile						
Typ środowiska	Min. tier VPS	Max vCPU	Max RAM (GB)	Max storage (GB)	Max egress (GB/h)	Secrets scope
DEV	STANDARD	4	16	50	2	env-scoped
STAGING	PRO	16	64	200	10	env-scoped
PRODUCTION	STATE	64	256	2000	100	env-scoped + audit

Typ środowiska	Min. tier VPS	Max vCPU	Max RAM (GB)	Max storage (GB)	Max egress (GB/h)	Secrets scope
PHANTOM-SHADOW	PHANTOM	32	128	100	50	env-scoped + mirror

ZASADA · Spójność z Planem 02

Tiers STANDARD/PRO/STATE/PHANTOM odpowiadają bezpośrednio tierom z Infrastructure Provider Plane (Plan 02). Każda zmiana tieru w Planie 02 wymaga przeglądu Aneksu B OEP. W v3.0 Plan 02 rozszerzony o stany rebuild/cutover.

PLAN 10 · WARSTWA KANONICZNA

Cost Governance Plane

REBUILD · Rebuildability over Lineage — Cost Governance

CGP (Cost Governance Plane) musi obsługiwać budżety dla TRZECH ścieżek implementacyjnych: Evolution (standardowe operacje, baseline kosztowy), Refactor (podwójny run legacy+target podczas cutover, wyższy mnożnik) oraz Rebuild (pełny greenfield D4/D5, najwyższy koszt z rezerwą bezpieczeństwa i opłatą za external review). Klasa D5 (Greenfield/Systemic) wprowadza nową kategorię budżetową: „rebuild-budget” z dedykowanym mnożnikiem 3,0× bazowego budżetu plus rezerwa 20% i external review fee. CGP śledzi koszty oddzielnie dla toru legacy i toru target podczas trybu REBUILD_MODE (Plan 01) — umożliwiając rozliczenie dual-run overhead. Odnośniki: Plan 14 (Rebuild Orchestration), Plan 15 (7 działów), Plan 08 (Evolution budgets).

10.1. Cel i umiejscowienie

Cost Governance Plane (CGP) jest warstwą twardo kontrolującą koszty całego pipeline SYLION. Jej zadaniem jest nie dopuścić do niekontrolowanego wzrostu wydatków na tokeny modeli, compute VPS, storage i egress sieciowy. CGP jest serwisem centralnym: każdy komponent pipeline przed użyciem zasobu płaci „opłatę” (reservation + charge) przez CGP. Jeśli budżet jest wyczerpany lub przekroczony próg ostrzeżenia, CGP aktywuje backpressure lub circuit breaker zatrzymując wykonanie. W v3.0 CGP zarządza dodatkowo budżetami rebuild — osobnymi pulami finansowymi dla operacji D4 i D5, włącznie z dual-run overhead i external review fee.

- CGP jest obserwatorem i egzekutorem, nie tylko księgowością.
- Budżety są hierarchiczne: run < agent < okno czasowe.
- Ceny tokenów są pobierane z PriceSheet powiązanego z Model Engine (Plan 05).
- Ceny compute VPS są pobierane z Provider Price List (Plan 02).
- Budżety rebuild są śledzone oddzielnie od budżetów operacyjnych (nowe w v3.0).

KRYTYCZNE · Zasada „hardstop”

Po osiągnięciu 100% budżetu okna pipeline zostaje zatrzymany twardym circuit breakerem. Żadne nowe runy nie startują, żadne tokeny nie są konsumowane dopóki operator nie otworzy nowego budżetu lub nie wykona appeal. Wyjątek: operacje klasy CRITICAL (D4 Human Gate) nigdy nie są blokowane przez CGP.

10.2. Trzy ścieżki implementacyjne a budżet

v3.0 wprowadza fundamentalną zmianę w modelu kosztowym: każda operacja pipeline jest teraz klasyfikowana według ścieżki implementacyjnej. Ścieżka determinuje mnożnik budżetu bazowego, wymagany poziom rezerwy i dodatkowe opłaty. CGP rejestruje pole „implementation_path” w każdym CostRecord, umożliwiając analizę kosztów per ścieżka i per

sprint.

Budżet per ścieżka implementacyjna

Ścieżka	Mnożnik budżetu	Uzasadnienie
Evolution	1,0× (baseline)	Patch/reuse, standardowe operacje, minimalne narzuty. Budżet bazowy wystarczy. Nie wymaga dual-run.
Refactor	1,3×	Dual-run legacy+target podczas cutover, migracja kontraktów. Dodatkowe 30% pokrywa koszty równoległych środowisk i testów regresji.
Rebuild (D4 — moduł)	2,0×	Greenfield implementation + adversarial testing Red Team. Podwójny koszt: nowa implementacja + walidacja istniejącej. Wymaga Council 4/4 + Human Gate.
Rebuild (D5 — system)	3,0× + rezerwa bezpieczeństwa 20% + external review fee	Pełny rebuild systemu + zewnętrzny audyt bezpieczeństwa. 3,0× pokrywa dual-run całego systemu; 20% rezerwy na nieoczekiwane koszty; external review fee = koszt zewnętrznego audytora (zmienny, config).

REBUILD · D5 Rebuild Budget — kalkulacja

Całkowity budżet D5 = budżet_bazowy × 3,0 × 1,20 + external_review_fee_usd. Przykład: sprint bazowy \$2 000 → budżet D5 = \$2 000 × 3,0 × 1,20 + \$500 = \$7 700. external_review_fee_usd jest konfigurowalne per decyzją D5 i zatwierdzone przez Human Gate razem z decyzją (Plan 14). Odnośnik: Plan 14 sekcja 14.6 (budget gate'y rebuild).

10.3. Trzy wymiary budżetów

CGP zarządza budżetami na trzech poziomach granularności. Każdy wymiar ma niezależne progi alertów i może mieć własny circuit breaker. W v3.0 dodano czwarty wymiar: per-track (legacy vs target) aktywny w trybie REBUILD_MODE.

Wymiar 1: Per-Run Budget

Każdy run pipeline (Run Lifecycle Plane, Plan 11) startuje z przydzielonym budżetem per-run określającym maksymalne koszty tokenów i compute dla pojedynczego przebiegu. Przekroczenie budżetu run powoduje przerwanie runu bez wpływu na inne runy.

- Budżet per-run jest podzbiorem budżetu agenta i okna.
- Domyślny limit: konfigurowalne per pipeline tier.
- Alert: 80% → log warning; 100% → run abort (D0 auto).

Wymiar 2: Per-Agent Budget

Każdy agent ma przydzielony budżet dzienny/tygodniowy określający ile łącznie może wydać na tokeny i compute. Agenci z klasą D0-D1 mają niższe domyślne limity niż Council (D2-D3). Budżet per-agent akumuluje koszty ze wszystkich runów danego agenta w oknie czasowym.

- Agenci WORKER mają limit 10× niższy niż SENIOR.
- Agent przekraczający 90% budżetu tygodniowego dostaje flagę review.

- Przekroczenie 100% → agent zablokowany do końca okna (DO auto).

Wymiar 3: *Per-Window Budget (dzienny/tygodniowy/miesięczny)*

Okno czasowe to globalny budżet projektu/sprintu. CGP wspiera trzy typy okien: Daily (rolowane co 24h UTC), Weekly (reset w poniedziałek 00:00 UTC) i Monthly (reset 1. dnia miesiąca). Okno sprintu jest specjalnym Monthly-like oknem o dowolnym czasie trwania (np. 8 tygodni dla sprintu SYLION).

- Okno sprintu ma budżet globalny dla całego pipeline.
- Alert 80% okna → warning do operatora + freeze sugestia OEP.
- Alert 100% okna → circuit breaker trip (hardstop wszystkich runów).

Wymiar 4: *Per-Track Budget (NOWY w v3.0 — REBUILD_MODE)*

W trybie REBUILD_MODE (Plan 01, M-03) pipeline działa dwutorowo: tor legacy (istniejący system) i tor target (nowa implementacja). CGP śledzi koszty obu torów oddzielnie, umożliwiając pomiar dual-run overhead i potwierdzenie, że koszty mieszczą się w zaakceptowanym mnożniku ścieżki. Po cutover tor legacy jest dekomisjonowany, a jego alokacja budżetowa zwracana.

- Pole track w CostRecord: "legacy" | "target" | "shared"
- Dual-run overhead = suma kosztów toru legacy w oknie REBUILD_MODE.
- Alert: overhead > 40% całkowitego budżetu rebuild → eskalacja D4.

10.4. *Koszty 7 działów pipeline*

Pipeline v3.0 składa się z 7 stałych działów (Plan 15). CGP alokuje miesięczny budżet między działy według predefiniowanych udziałów procentowych. Podział odzwierciedla priorytety operacyjne: Implementacja i Platforma pochłaniają najwięcej zasobów, Red Team i Blue Team mają dedykowane pule na operacje adversarialne. Poniżej przykładowy podział przy budżecie sprintu \$2 000.

Dział (Plan 15)	Udział %	Budżet (przykład \$2 000)	Główne kategorie kosztów
Architektura	15%	\$300	Tokeny modeli (analiza i projektowanie), KB reads, decision records
Platforma	20%	\$400	Compute VPS (ACP, VPS provisioning, bundle pipeline), storage, egress
Implementacja	25%	\$500	Tokeny (generowanie kodu, migracje, kontrakty), compute sandbox, testy integracyjne
Testy i Walidacja	10%	\$200	Tokeny (generowanie przypadków testowych, walidacja kontraktów), compute test env
Red Team	8%	\$160	Tokeny (adversarial scenarios, threat modeling), compute attack sandbox
Blue Team	10%	\$200	Tokeny (detekcja, DFIR playbooki, SOC rules), compute monitoring

Dział (Plan 15)	Udział %	Budżet (przykład \$2 000)	Główne kategorie kosztów
Governance i Compliance	12%	\$240	Tokeny (Evidence pack, propozycje Księgi, audit log analysis), storage (evidence)

UWAGA · Alokacja działów — elastyczność

Udziały procentowe są konfigurowalne per sprint w budget.yaml (pole dept_allocations). Zmiana podziału powyżej $\pm 5\%$ wymaga decyzji D3 (Governance i Compliance zatwierdza). W trybie REBUILD_MODE (D4/D5) tymczasowo zwiększa się alokacja Architektury (+5%) i Red Team (+4%) kosztem Implementacji (-9%), odzwierciedlając priorytety rebuild. Odnośnik: Plan 15 (Multi-Team Pipeline Organization).

10.5. Rebuild Budget Lifecycle

Budżet rebuild przechodzi przez zdefiniowany cykl życia — od wstępnej rezerwy analitycznej przez zatwierdzenie i dual-run, aż do cutover i dekomisji legacy. CGP śledzi każdą fazę jako osobny BudgetWindow z typem „rebuild_phase”, co umożliwia pełny audyt wydatków rebuild post-facto.

1. Pre-decision reserve (analiza): Przed decyzją D4/D5 CGP rezerwuje 10% szacowanego budżetu rebuild na koszt analizy legacy fitness (Plan 14: analyze_legacy_fitness). Faza: rebuild_phase=pre_decision. Klasa decyzji: D3 (inicjacja analizy). Jeśli analiza zakończy się decyzją „no-rebuild”, rezerwa jest zwalniana do puli operacyjnej.
2. Approved-rebuild allocation (Council + Human): Po zatwierdzeniu D4/D5 (Council 4/4 + Human Gate, opcjonalnie External Review D5) CGP tworzy nowy BudgetWindow rebuild_budget_id z pełną alokacją (mnożnik × budżet bazowy + rezerwa + external review fee dla D5). Faza: rebuild_phase=approved. Środki są zablokowane i nie podlegają realokacji bez kolejnej decyzji D4.
3. Dual-run overhead (legacy + target działają razem): W trybie REBUILD_MODE CGP śledzi koszty obu torów (CostRecord.track). Dual-run overhead jest raportowany operatorowi tygodniowo. Faza: rebuild_phase=dual_run. Alert: overhead > 40% rebuild_budget → eskalacja. Dział Governance monitoruje zgodność z zatwierdzonym mnożnikiem (Plan 13).
4. Cutover execution cost: Finalna faza intensywna: przełączenie ruchu, migracje danych, testy smoke. Faza: rebuild_phase=cutover. CGP zwiększa limit per-run dla operacji cutover o 50% (na czas okna cutover, max 72h), zatwierdzany przez Human Gate. Koszty cutover są rejestrowane osobno do raportu post-cutover.
5. Legacy decommission savings (zwrot budżetu): Po zakończeniu cutover i dekomisji toru legacy CGP zamyka BudgetWindow legacy-track i zwraca niewydane środki do puli operacyjnej. Faza: rebuild_phase=decommission. „Savings report” jest generowany automatycznie — pokazuje ile zaoszczędzono dzięki wyłączeniu starego systemu (reduce compute, storage, model calls). Raport jest dołączany do Evidence pack (Plan 13).

REBUILD · Rebuild Budget Lifecycle — odnośniki

Plan 14 definiuje workflow rebuild od analizy do dekomisji. Plan 15 określa, które działy ponoszą koszty w każdej fazie. Plan 08 definiuje canonical evolution budgets dla ścieżki Evolution (faza 1 bez rebuild). Plan 13 przechowuje Evidence pack z historią kosztów rebuild.

10.6. Kategorie kosztów

CGP śledzi koszty w pięciu kategoriach. Każda kategoria ma oddzielny PriceSheet i może mieć odrębny sublimit w budżecie.

Kategoria	Jednostka	Źródło ceny	Przykład
Tokeny modeli	per 1M tokenów (in/out)	PriceSheet (CGP)	Gemini 2.5 Pro \$3,5/\$10,5
Compute VPS	per vCPU-h + per GB-RAM-h	Provider Price List (Plan 02)	STANDARD 0,05 USD/vCPU-h
Storage	per GB na dobę	Provider Price List	0,002 USD/GB/dobę
Egress sieciowy	per GB transferu wychodzącego	Provider Price List	0,08 USD/GB
Provider fees	% od compute bill	Kontrakt VPS	2,5% platform fee

10.7. Klasy priorytetów budżetu

Nie wszystkie operacje pipeline mają jednakowy priorytet. CGP definiuje cztery klasy priorytetów, które determinują kolejność alokacji przy deficycie budżetu oraz zachowanie circuit breakera. W v3.0 operacje rebuild są klasyfikowane jako HIGH lub CRITICAL zależnie od fazy i klasy decyzji.

Klasa	Priorytet	Opis	Zachowanie przy deficycie
CRITICAL	P0	Operacje bezpieczeństwa i Human Gate (D4/D5); cutover execution	Nigdy nie blokowane przez CGP
HIGH	P1	Council (D2/D3), walidacje gate'ów, Canonical Evolution; rebuild dual-run	Blokowane tylko hardstop 100%
NORMAL	P2	Standardowe runy pipeline, agenci SENIOR	Backpressure przy 80% okna
LOW	P3	Agenci WORKER, zadania batch, shadow-phantom	Backpressure przy 60% okna

10.8. Tabela klas decyzji D0–D5 — Cost Governance

Klasa	Przykład operacji CGP	Aktor	Ścieżka rebuild
D0	Automatyczna rezerwacja kosztów, alert 80%, charge bez anomalii	CGP auto	—

Klasa	Przykład operacji CGP	Aktor	Ścieżka rebuild
D1	Anomalia kosztowa >30% (alert, review)	Chief Reviewer	—
D2	Zmiana sublimitów kategorii w budżecie	Council 3/4	evolution
D3	Zmiana podziału alokacji działów (±5%), inicjacja analizy pre-rebuild	Council 4/4	refactor
D4	Zatwierdzenie budżetu rebuild (D4 moduł), appeal >20% limitu, zmiana mnożnika 2,0×	Human Gate	rebuild (moduł)
D5	Zatwierdzenie budżetu D5 rebuild (system), external review fee, mnożnik 3,0×	Council 4/4 + Human + External Review	rebuild (greenfield)

REBUILD · D5 — Greenfield/Systemic budget gate

Decyzja D5 w Planie 10 oznacza zatwierdzenie budżetu dla pełnego rebuild systemu. Kolor D5: fioletowy (#6A1B9A). External review fee musi być wyceniona i zaakceptowana przez Human Gate przed aktywacją BudgetWindow rebuild_phase=approved. Wszystkie wydatki D5 są raportowane w Evidence pack (Plan 13) i zewnętrznemu audytorowi.

10.9. Backpressure i circuit breaker

Backpressure jest mechanizmem „miękkiego” hamowania: CGP opóźnia starty nowych runów klasy LOW/NORMAL proporcjonalnie do procentu wykorzystania budżetu okna. Circuit breaker to twarde zatrzymanie aktywowane po osiągnięciu 100% budżetu lub po wykryciu anomalii kosztowej.

- Backpressure delay: $(usage_pct - 60) \times 500ms$ dla klasy LOW.
- Backpressure delay: $(usage_pct - 80) \times 300ms$ dla klasy NORMAL.
- Circuit breaker OPEN → środowisko OEP jest auto-frozen (G-COST-06).
- Circuit breaker HALF-OPEN po 30 min: dopuszcza jeden próbny run CRITICAL.

UWAGA · Relacja do OEP

Gdy CGP aktywuje circuit breaker dla budget_id powiązanego ze środowiskiem OEP (Plan 09), emituje zdarzenie budget_trip które OEP przetwarza jako automatyczny freeze środowiska. W trybie REBUILD_MODE freeze dotyczy tylko toru, który przekroczył limit — tor legacy lub tor target — nie całego systemu.

10.10. Relacja do Model Engine (Plan 05), VPS (Plan 02) i 7 działów (Plan 15)

CGP utrzymuje PriceSheet z aktualnymi cenami modeli (Plan 05) i compute (Plan 02). Ceny tokenów są pobierane z Role Board: Chief Context (Gemini 2.5 Pro), Chief Reasoning (Claude Opus 4), Chief Review (GPT-5), Senior GLM-5.1, Workers Qwen3.5/gpt-oss-20b, Specialist Grok-3. PriceSheet jest wersjonowany i immutable; zmiana wymaga nowego wpisu z semver.

Plan 15 definiuje strukturę 7 działów, których budżety są zarządzane przez CGP jako oddzielne sub-pule w ramach budżetu okna.

- Ceny z PriceSheet są używane do pre-charge (reservation) i post-charge (actualization).
- Dysproporcja reservation vs actual >30% wyzwala alert anomalii kosztowej (G-COST-07).
- Zmiana modelu w Board bez aktualizacji PriceSheet = naruszenie G-COST-04.

PLAN 10 · SPECYFIKACJA WYKONAWCZA

Cost Governance Plane

CEL I ZAKRES

Specyfikacja CGP v3.0 definiuje model domenowy, schemat YAML budżetu, workflowy operacyjne (rezerwacja, naliczanie, zamykanie okna, circuit breaker, appeal, rebuild budget lifecycle), kontrakty API, gate'y jakości i testy automatyczne. Zakres: pełny cykl życia budżetu od definicji przez naliczanie do zamknięcia okna z obsługą anomalii kosztowych, mechanizmem appeal dla operatora oraz nowym modelem kosztów rebuild (Evolution/Refactor/Rebuild D4/D5). Wersja: Pipeline 8.0.0, MZ baseline 3.5.5.

SCHEMAT YAML — budget.yaml

Przykład: budżet sprintu 8-tygodniowego SYLION z alokacją działów

YAML

```
apiVersion: sylion.io/v3
kind: Budget
metadata:
  budget_id: "bgt-sprint08"
  name: "SYLION Sprint 08 - 8 tyg"
spec:
  window_type: sprint
  window_start: "2025-09-01T00:00:00Z"
  window_end: "2025-10-27T23:59:59Z"
  total_limit_usd: 2000.00
  currency: USD
  priority_class: NORMAL
  implementation_strategy: reuse # domyslna sciezka dla nowego sprintu
  alerts:
    - threshold_pct: 60
      action: backpressure_low
    - threshold_pct: 80
      action: warning_notify + backpressure_normal
    - threshold_pct: 100
      action: circuit_breaker_trip
  sublimits:
    tokens_usd: 1200.00 # 60% budzetu na modele
    compute_usd: 500.00 # 25% na VPS compute
    storage_usd: 150.00 # 7.5% na storage
    egress_usd: 100.00 # 5% na egress
    fees_usd: 50.00 # 2.5% na provider fees
  dept_allocations: # 7 dzialow (Plan 15)
    architektura_pct: 15
    platforma_pct: 20
    implementacja_pct: 25
    testy_pct: 10
    red_team_pct: 8
    blue_team_pct: 10
    governance_pct: 12
  per_run_limit_usd: 50.00
  per_agent_limits:
    CRITICAL_role: 500.00
```



```
HIGH_role:      200.00
NORMAL_role:    80.00
LOW_role:       20.00
circuit_breaker:
  auto_reset_after_min: 30
  probe_class: CRITICAL
```

Przykład: budżet rebuild D5 (system rebuild)

YAML

```
apiVersion: sylion.io/v3
kind: Budget
metadata:
  budget_id: "bgt-rebuild-d5-mz-v4"
  name: "SYLION Rebuild D5 – MZ v4 Greenfield"
spec:
  window_type: sprint
  window_start: "2026-05-01T00:00:00Z"
  window_end:   "2026-08-31T23:59:59Z"
  total_limit_usd: 7700.00      # 2000 x 3.0 x 1.20 + 500 external_review_fee
  currency: USD
  priority_class: HIGH
  implementation_strategy: greenfield
  rebuild_decision_class: D5
  rebuild_multiplier: 3.0
  safety_reserve_pct: 20
  external_review_fee_usd: 500.00
  rebuild_phases:
    - phase: pre_decision
      allocated_pct: 10          # 10% na analize legacy fitness
    - phase: approved
      allocated_pct: 30          # 30% po zatwierdzeniu Council+Human
    - phase: dual_run
      allocated_pct: 40          # 40% na dual-run (legacy + target)
    - phase: cutover
      allocated_pct: 15          # 15% na cutover execution
    - phase: decommission
      allocated_pct: 5           # 5% na dekomisje legacy
  dual_run_overhead_alert_pct: 40
  dept_allocations_rebuild_mode: # zmienione w REBUILD_MODE
    architektura_pct: 20        # +5 (projektowanie target arch)
    platforma_pct: 20
    implementacja_pct: 16       # -9 (greenfield wymaga less ops)
    testy_pct: 10
    red_team_pct: 12           # +4 (adversarial pre-cutover)
    blue_team_pct: 10
    governance_pct: 12
  circuit_breaker:
    auto_reset_after_min: 60    # dluzszy reset dla rebuild
    probe_class: CRITICAL
```

SCHEMAT YAML — *cgp_module.yaml*

Moduł CGP (implementation_strategy v3.0)

YAML		
<pre>apiVersion: sylion.io/v3 kind: Module metadata: module_id: "cgp-core" name: "Cost Governance Plane Core" plan: "10" spec: implementation_strategy: refactor current_baseline: MZ 3.5.5 target_baseline: "cgp-core-v3" cutover_strategy: shadow version: "3.0.0" rebuild_extensions: - rebuild_budget_lifecycle - per_track_cost_record - dept_allocations - path_multiplier_engine</pre>		
MODEL DOMENOWY		
Encja	Kluczowe pola	Klucze / relacje
Budget	budget_id (UUID), name, window_type, window_start, window_end, total_limit_usd, priority_class, state (open/closed/breaker), used_usd, reserved_usd, currency, implementation_strategy, rebuild_decision_class (nullable), rebuild_multiplier (nullable)	PK budget_id; 1:N → BudgetWindow, CostRecord
BudgetWindow	window_id, budget_id, window_type, rebuild_phase (nullable), period_start, period_end, limit_usd, used_usd, alert_80_fired, alert_100_fired, closed_at	PK window_id; FK budget_id→Budget; 1:N → CostRecord
CostRecord	record_id (UUID), budget_id, window_id, run_id, agent_id, env_id, dept_id (nullable), track (legacy/target/shared), category (tokens/compute/storage/egress/fees), implementation_path (evolution/refactor/rebuild), model_id (nullable), amount_usd, units, unit_price_usd, reservation_id (nullable), recorded_at	PK record_id; FK budget_id, window_id; immutable
UsageQuota	quota_id, scope (run/agent/window/dept/track), scope_id, category, limit_usd, used_usd, status (ok/warning/breaker)	PK quota_id; FK scope_id → run/agent/budget/dept
CircuitBreaker	breaker_id, budget_id, track (nullable), state (closed/open/half_open), tripped_at, tripped_reason, reset_at, last_probe_at	PK breaker_id; FK budget_id→Budget; 1:N (per track w REBUILD_MODE)
PriceSheet	sheet_id, version (semver), model_id, category, price_in_usd_per_1m (nullable), price_out_usd_per_1m (nullable), price_unit_usd (nullable), unit_label, effective_from, superseded_by	PK sheet_id; immutable after publish

Encja	Kluczowe pola	Klucze / relacje
RebuildBudgetPhase	phase_id, budget_id, phase (pre_decision/approved/dual_run/cutover/decommission), allocated_usd, used_usd, state (pending/active/closed/returned), activated_at, closed_at, returned_usd	PK phase_id; FK budget_id→Budget; 1:N → CostRecord
WORKFLOWY		

WF-COST-01: Reserve Budget (pre-charge)

1. Komponent pipeline (agent, run, orchestrator) wysyła POST /cost/v1/reserve
2. CGP sprawdza aktywne BudgetWindow i UsageQuota (G-COST-01: charge bez reservation → reject)
3. CGP oblicza szacunkowy koszt z PriceSheet (tokeny × price_in/out lub compute × unit_price)
4. Jeśli used_usd + reserved_usd + estimated < limit → akceptuje reservation
5. CGP tworzy CostRecord(category, reservation_id, amount_usd=estimated, status=reserved)
6. CGP zwraca reservation_id i estimated_usd; komponent kontynuuje wykonanie

WF-COST-02: Charge Usage (actualization)

1. Po zakończeniu operacji komponent wysyła POST /cost/v1/charge z reservation_id i actual_units
2. CGP pobiera CostRecord z reservation_id i oblicza koszt rzeczywisty
3. CGP aktualizuje BudgetWindow.used_usd += actual_usd
4. CGP zwalnia reservation (reserved_usd -= estimated_usd)
5. Anomalia: actual > estimated × 1,3 → alert G-COST-07
6. CGP sprawdza progi alertów; jeśli 80% → warning; jeśli 100% → trip breaker

WF-COST-03: Close Window

1. Cron-job lub operator wysyła POST /cost/v1/windows/{id}/close
2. CGP sprawdza brak aktywnych reservations (jeśli są → force-finalize)
3. CGP ustawia BudgetWindow.closed_at i generuje raport podsumowujący
4. Nowe BudgetWindow jest automatycznie tworzone wg window_type jeśli budżet jest open
5. Historia CostRecord pozostaje immutable

WF-COST-04: Trigger Circuit Breaker

1. CGP wykrywa BudgetWindow.used_usd ≥ limit_usd (100%)
2. CGP ustawia CircuitBreaker.state = open, tripped_at = now
3. CGP emituje zdarzenie budget_trip do OEP (→ auto-freeze środowiska)
4. CGP blokuje wszystkie nowe POST /cost/v1/reserve z wyjątkiem klasy CRITICAL
5. Po 30 min CGP przechodzi w HALF-OPEN, dopuszcza jeden probe run CRITICAL
6. Probe sukces → CLOSED; probe fail → powrót do OPEN

WF-COST-05: Appeal

1. Operator wysyła POST /cost/v1/budgets/{id}/appeal z uzasadnieniem
2. CGP wymaga decyzji D4 (Human Gate) dla zwiększenia limitu >20%
3. CGP tworzy Budget amendment z nowym total_limit_usd i wersją semver
4. Stary budget pozostaje immutable; nowy budget ma odczyt stanu z poprzedniego
5. CGP resetuje circuit breaker jeśli appeal dotyczy przekroczonego okna

WF-COST-06: Rebuild Budget Lifecycle (NOWY w v3.0)

1. Dział Architektury (Plan 15) inicjuje analizę legacy fitness (D3). CGP aktywuje RebuildBudgetPhase(phase=pre_decision, allocated=10% budżetu bazowego).
2. Po decyzji Council 4/4 + Human Gate (D4/D5) CGP tworzy rebuild_budget_id z pełną alokacją (mnożnik × limit + rezerwa + external_review_fee dla D5). CGP aktywuje RebuildBudgetPhase(phase=approved).
3. Plan 01 przechodzi w REBUILD_MODE. CGP aktywuje per-track tracking (legacy vs target). RebuildBudgetPhase(phase=dual_run). Tygodniowy raport dual-run overhead do operatora.
4. Cutover execution: CGP zwiększa per-run limit o 50% na max 72h (G-COST-09). RebuildBudgetPhase(phase=cutover). Wszystkie koszty cutover rejestrowane z track=shared.
5. Legacy decommission: CGP zamyka BudgetWindow legacy-track. RebuildBudgetPhase(phase=decommission, returned_usd=niewydane środki legacy). Savings report generowany automatycznie → dołączany do Evidence pack (Plan 13).

API

YAML

```

POST /cost/v1/budgets
  Tworzy nowy budżet z definicją okna, limitów i alokacji działów.
  Body: BudgetDefinition JSON schema
  Response 201: { "budget_id": "...", "state": "open" }

POST /cost/v1/reserve
  Rezerwuje koszt przed operacją (pre-charge).
  Body: { "budget_id", "run_id", "agent_id", "env_id",
          "category", "model_id", "estimated_units",
          "priority_class", "dept_id", "track", "implementation_path" }
  Response 200: { "reservation_id": "...", "estimated_usd": 0.042 }
  Response 402: { "error": "budget_exceeded", "breaker_state": "open" }

POST /cost/v1/charge
  Nalicza rzeczywisty koszt po zakończeniu operacji.
  Body: ChargeEvent JSON schema
  Response 200: { "record_id": "...", "actual_usd": 0.037, "window_pct": 42.3 }
  Response 422: { "error": "reservation_not_found" }

GET /cost/v1/usage
  Pobiera użycie dla budget_id / run_id / agent_id / dept_id / track.
  Query: budget_id, run_id, agent_id, window_id, category, dept_id, track
  Response 200: { "used_usd": ..., "reserved_usd": ..., "limit_usd": ...,
                  "pct_used": ..., "breaker_state": "closed",

```

```

    "dept_breakdown": {...}, "track_breakdown": {...} }

POST /cost/v1/breakers/{id}/trip
  Reczne wyzwolenie circuit breakera (D4 operator).
  Response 200: { "breaker_id": "...", "state": "open", "tripped_at": "..." }

POST /cost/v1/breakers/{id}/reset
  Reset circuit breakera po appeal lub uplywie czasu.
  Response 200: { "state": "closed" }

POST /cost/v1/budgets/{id}/appeal
  Appeal o zwiekszenie limitu budzetu (wymaga D4).
  Body: { "new_limit_usd": ..., "justification": "..." }
  Response 201: { "amendment_budget_id": "...", "pending_d4": true }

POST /cost/v1/rebuild/phases/{budget_id}/activate
  Aktywuje kolejna faze rebuild budget lifecycle.
  Body: { "phase": "pre_decision|approved|dual_run|cutover|decommission",
    "decision_id": "...", "operator_token": "JWT" }
  Response 200: { "phase_id": "...", "phase": "approved", "allocated_usd": 6000 }

GET /cost/v1/rebuild/savings/{budget_id}
  Zwraca savings report po dekomisji legacy.
  Response 200: { "saved_usd": ..., "legacy_track_returned_usd": ...,
    "evidence_pack_ref": "..." }

```

Schemat JSON — ChargeEvent (POST /cost/v1/charge)

JSON

```

{
  "$schema": "https://syllion.io/schemas/charge-event/v3",
  "required": ["reservation_id", "actual_units", "category"],
  "properties": {
    "reservation_id": { "type": "string", "format": "uuid" },
    "actual_units": { "type": "number", "minimum": 0 },
    "category": { "type": "string",
      "enum": ["tokens", "compute", "storage", "egress", "fees"] },
    "model_id": { "type": "string" },
    "token_in": { "type": "integer", "minimum": 0 },
    "token_out": { "type": "integer", "minimum": 0 },
    "compute_vcpu_h": { "type": "number", "minimum": 0 },
    "storage_gb_days": { "type": "number", "minimum": 0 },
    "egress_gb": { "type": "number", "minimum": 0 },
    "metadata": {
      "type": "object",
      "properties": {
        "run_id": { "type": "string" },
        "agent_id": { "type": "string" },
        "env_id": { "type": "string" },
        "dept_id": { "type": "string" },
        "track": { "type": "string",
          "enum": ["legacy", "target", "shared"] },
        "implementation_path": { "type": "string",
          "enum": ["evolution", "refactor", "rebuild"] },
        "rebuild_phase": { "type": "string" },
        "stage": { "type": "string" }
      }
    }
  }
}

```

```

}
}

```

Schemat JSON — BudgetDefinition (POST /cost/v1/budgets)

JSON

```

{
  "$schema": "https://syllion.io/schemas/budget-definition/v3",
  "required": ["name", "window_type", "window_start", "window_end",
    "total_limit_usd", "priority_class"],
  "properties": {
    "name": { "type": "string", "maxLength": 128 },
    "window_type": { "type": "string",
      "enum": ["daily", "weekly", "monthly", "sprint"] },
    "window_start": { "type": "string", "format": "date-time" },
    "window_end": { "type": "string", "format": "date-time" },
    "total_limit_usd": { "type": "number", "minimum": 1.0 },
    "currency": { "type": "string", "default": "USD" },
    "priority_class": { "type": "string",
      "enum": ["CRITICAL", "HIGH", "NORMAL", "LOW"] },
    "implementation_strategy": { "type": "string",
      "enum": ["reuse", "refactor", "replace", "greenfield"] },
    "rebuild_decision_class": { "type": "string",
      "enum": ["D4", "D5"], "nullable": true },
    "rebuild_multiplier": { "type": "number", "nullable": true },
    "safety_reserve_pct": { "type": "number", "nullable": true },
    "external_review_fee_usd": { "type": "number", "nullable": true },
    "sublimits": {
      "tokens_usd": { "type": "number" },
      "compute_usd": { "type": "number" },
      "storage_usd": { "type": "number" },
      "egress_usd": { "type": "number" },
      "fees_usd": { "type": "number" }
    },
    "dept_allocations": {
      "architektura_pct": { "type": "integer" },
      "platforma_pct": { "type": "integer" },
      "implementacja_pct": { "type": "integer" },
      "testy_pct": { "type": "integer" },
      "red_team_pct": { "type": "integer" },
      "blue_team_pct": { "type": "integer" },
      "governance_pct": { "type": "integer" }
    },
    "per_run_limit_usd": { "type": "number" },
    "per_agent_limits": { "type": "object" },
    "circuit_breaker": {
      "auto_reset_after_min": { "type": "integer" },
      "probe_class": { "type": "string" }
    }
  }
}

```

HARD GATE'Y

Kod	Nazwa	Warunek FAIL	Klasa	Blokuje
G-COST-01	Charge wymaga reservation	POST /charge bez ważnego reservation_id → reject 422	D0	charge
G-COST-02	Limit per-run	Szacowany koszt runu przekracza per_run_limit_usd budżetu	D0	reserve (run)
G-COST-03	Limit per-agent tygodniowy	Agent przekroczył 100% swojego UsageQuota w bieżącym oknie	D0	reserve (agent)
G-COST-04	Sublimit kategorii	Kategoria tokens/compute/storage/egress osiągnęła sublimit	D0	reserve (kategoria)
G-COST-05	Alert 80% okna → warning	BudgetWindow.used_usd >= 0,80 × limit_usd → warning + backpressure NORMAL/LOW	D0	backpressure (nie blokuje CRITICAL/HIGH)
G-COST-06	Alert 100% → circuit breaker trip	BudgetWindow.used_usd >= limit_usd → CircuitBreaker.state=open	D0	wszystkie nowe reserve (poza CRITICAL)
G-COST-07	Anomalia kosztowa	actual_usd > estimated_usd × 1,30 (różnica >30% rezerwacji)	D1	alert + review; nie blokuje ale loguje anomalię
G-COST-08	Rebuild budget bez decyzji D4/D5	Aktywacja rebuild_phase=approved bez aktywnej decyzji D4 lub D5	D4	rebuild budget activation
G-COST-09	Cutover per-run limit extension	Zwiększenie per_run_limit o >50% bez aktywnej decyzji Human Gate (D4)	D4	cutover run limit change
G-COST-10	Dual-run overhead alert	Dual-run overhead > 40% całkowitego rebuild_budget w bieżącym oknie	D3	eskalacja do operatora; nie blokuje ale wymaga D3 review

Kod	Nazwa	Warunek FAIL	Klasa	Blokuje
G-COST-11	Suma dept_allocations = 100%	dept_allocations w budget.yaml nie sumują się do 100%	D0	tworzenie/aktualizacja budżetu
G-COST-12	D5 external review fee wymagany	Budżet rebuild_decision_class=D5 bez external_review_fee_usd > 0	D5	aktywacja approved phase dla D5
TRZY ŚCIEŻKI — Cost Governance				

Evolution path

CGP operuje w trybie standardowym. Budżet evolution = budżet_bazowy × 1,0×. Brak rebuild_phases, brak per-track tracking. Wszystkie CostRecord mają implementation_path=evolution. Alokacja działów standardowa (Architektura 15%, Platforma 20%, Implementacja 25%, itd.).

- Klasy decyzji: D0-D3 (bez D4/D5 gate'ów kosztowych).
- Savings/overhead reports: nie dotyczy.
- Odnośnik: Plan 08 (Canonical Evolution — evolution budgets).

Refactor path

CGP aktywuje dual-run mode dla modułu w refaktorze. Budżet refactor = budżet_bazowy × 1,3×. CostRecord.track=legacy dla starego modułu, track=target dla nowego. Dual-run overhead jest mierzony i raportowany tygodniowo.

- Klasy decyzji: D3 inicjuje, D4 zatwierdza budżet.
- Alert G-COST-10 aktywny (40% overhead próg).
- Cutover: per-run limit extension G-COST-09 (D4 wymagany).

Rebuild path (D4 — moduł / D5 — system)

CGP tworzy rebuild_budget_id z pełnym cyklem RebuildBudgetPhase. Budżet D4 = budżet_bazowy × 2,0×. Budżet D5 = budżet_bazowy × 3,0× × 1,20 + external_review_fee. Wszystkie fazy (pre_decision, approved, dual_run, cutover, decommission) są śledzone jako oddzielne BudgetWindow. Po dekomisji legacy: savings report dołączany do Evidence pack (Plan 13).

- Klasy decyzji: D4 Human Gate (moduł) lub D5 Council+Human+External (system).
- Per-track tracking (legacy vs target) aktywny przez całą fazę dual_run.
- Odnośnik: Plan 14 (Rebuild Orchestration), Plan 13 (Evidence pack dla D5).

TESTY

Kod	Opis	Input	Oczekiwany wynik
T-COST-01	Tworzenie budżetu sprintu z alokacją działów	POST /budgets z window_type=sprint, limit=2000 USD, dept_allocations	201, budget_id, state=open, circuit_breaker=closed, dept_suma=100%
T-COST-02	Reserve + Charge prawidłowy (evolution path)	Reserve 100 tokenów in, 200 out (GPT-5); Charge actual=95 in, 180 out; impl_path=evolution	reserve 200; charge 0,00635 USD; window_pct wzrost; CostRecord.track=shared
T-COST-03	Charge bez reservation	POST /charge z nieistniejącym reservation_id	422, error=reservation_not_found, G-COST-01 FAIL
T-COST-04	Alert 80% okna	Załadowanie 80% limitu przez serię charge; następna reserve	200 (reserve OK), warning w logu, backpressure NORMAL aktywny
T-COST-05	Circuit breaker trip na 100%	Załadowanie 100% limitu; nowa reserve klasy NORMAL	402 budget_exceeded, CircuitBreaker.state=open, OEP freeze trigger
T-COST-06	CRITICAL przechodzi przez otwarty breaker	CircuitBreaker.state=open; reserve priority_class=CRITICAL	200 reserve zaakceptowany pomimo otwartego breakera
T-COST-07	Anomalia kosztowa >30%	Reserve 1,00 USD; Charge actual 1,35 USD	200 charge OK; alert anomalii logowany; G-COST-07 trigger
T-COST-08	Rebuild budget D4 — aktywacja approved bez decyzji	POST /rebuild/phases/{id}/activate phase=approved bez decision_id	403, G-COST-08 FAIL, requires_D4_decision
T-COST-09	Rebuild D5 bez external_review_fee	POST /budgets z rebuild_decision_class=D5, external_review_fee_usd=0	422, G-COST-12 FAIL, d5_external_fee_required
T-COST-10	Dual-run overhead >40%	Załadowanie 41% rebuild_budget przez legacy-track; nowy CostRecord	200 charge OK; G-COST-10 trigger; eskalacja D3 do operatora
T-COST-11	Savings report po dekomisji	POST /rebuild/phases/{id}/activate phase=decommission; legacy closed	200; RebuildBudgetPhase.returned_usd > 0; savings report wygenerowany
T-COST-12	Suma dept_allocations != 100%	POST /budgets z dept_allocations sumującymi się do 98	422, G-COST-11 FAIL, dept_allocations_must_sum_100

ANEKS A

Aneks A: Wymiary × okno × progi alertów

Wymiar budżetu	Typ okna	Próg 60%	Próg 80%	Próg 100%	Resetowanie
Per-Run	czas runu	brak	log warning	run abort (D0)	po zakończeniu runu
Per-Agent dzienny	Daily (24h UTC)	backpressure LOW	warning + flag review	agent blocked do końca dnia	reset 00:00 UTC
Per-Agent tygodniowy	Weekly (pon 00:00)	backpressure LOW	warning + flag review	agent blocked do końca tyg.	reset poniedziałek 00:00
Per-Window dzienny	Daily (24h UTC)	backpressure LOW	backpressure NORMAL + warn	circuit breaker OPEN	zamknięcie okna + nowe okno
Per-Window tygodniowy	Weekly	backpressure LOW	backpressure NORMAL + warn	circuit breaker OPEN	zamknięcie okna + nowe okno
Per-Window miesięczny	Monthly (1. dnia)	backpressure LOW	backpressure NORMAL + warn	circuit breaker OPEN	zamknięcie okna + nowe okno
Per-Sprint (custom)	N tygodni (config)	backpressure LOW	warn + OEP freeze suggest	circuit breaker OPEN	koniec sprintu lub appeal D4
Per-Track (REBUILD_MODE)	czas rebuild	backpressure LOW (track)	warn + dual-run report	circuit breaker track OPEN	decommission legacy lub cutover

ANEKS B

Aneks B: PriceSheet — ceny 7 modeli (wersja 2.0.0, obowiązująca od v3.0 pipeline)

Ceny zgodne z Role Board (Plan 05): Chief Context = Gemini 2.5 Pro, Chief Reasoning = Claude Opus 4, Chief Review = GPT-5, Senior = GLM-5.1, Workers = Qwen3.5 / gpt-oss-20b, Specialist = Grok-3. Ceny podane w USD per 1 milion tokenów (input/output). PriceSheet v2.0.0 jest wymagany przy budżetach z rebuild_decision_class (v3.0+).

Model ID	Rola w Board	Cena IN (\$/1M tok.)	Cena OUT (\$/1M tok.)	Typ hostingu	Uwagi
gemini-2.5-pro	Chief Context (waga 1,25)	\$3,50	\$10,50	API (Google)	kontekst domenowy, Księga, historia

Model ID	Rola w Board	Cena IN (\$/1M tok.)	Cena OUT (\$/1M tok.)	Typ hostingu	Uwagi
claude-opus-4	Chief Reasoning (waga 1,00)	\$15,00	\$75,00	API (Anthropic)	analiza, planowanie, dowody; najdroższy
claude-sonnet-4	Chief Reasoning alt (waga 1,00)	\$3,00	\$15,00	API (Anthropic)	alternatywa do Opus gdy cost-sensitive
gpt-5	Chief Review (waga 1,00)	\$10,00	\$30,00	API (OpenAI)	weryfikacja, red team, przypadki brzegowe
glm-5.1	Senior (waga 0,60)	\$0,60	\$2,20	API (z.ai)	analiza wspomagająca, senior tier
qwen3.5	Worker (waga 0,35)	\$0,30	\$0,90	API (Alibaba)	zadania batch, pomocnicze
gpt-oss-20b	Worker (waga 0,35)	\$0,10	\$0,30	self-hosted	lokalny inference; cena szacunkowa GPU
grok-3	Specialist (ad-hoc)	\$5,00	\$15,00	API (xAI)	ad-hoc specialist; nie w standardowym run

UWAGA · Spójność z Planem 05

PriceSheet w CGP musi być aktualizowany synchronicznie z Role Board zdefiniowanym w Model & Decision Engine Plane (Plan 05). Zmiana modelu w Board bez aktualizacji PriceSheet jest naruszeniem G-COST-04. Wersja PriceSheet jest referencyjna w każdym CostRecord. W v3.0 PriceSheet zawiera dodatkowe pole rebuild_cost_category do śledzenia, które modele są używane w fazach rebuild vs. operacyjnych.

Uwaga: ceny compute VPS (STANDARD: \$0,05/vCPU-h, PRO: \$0,09/vCPU-h, STATE: \$0,18/vCPU-h, PHANTOM: \$0,30/vCPU-h) są przechowywane w oddzielnej sekcji PriceSheet Provider i są synchronizowane z Plan 02 po każdej zmianie kontraktu z dostawcą VPS. W trybie REBUILD_MODE koszty compute środowiska PHANTOM-SHADOW są księgowane w torze target (CostRecord.track=target).

ANEKS C — Tabela mnożników budżetu per ścieżka

Aneks C: Zestawienie mnożników i składowych budżetu rebuild

Ścieżka	Klasa decyz.	Mnożnik	Rezerwa	Ext. review	Przykład (\$2 000 base)	Gate wejścia
Evolution	D0-D3	1,0×	—	—	\$2 000	automatyczny
Refactor	D3-D4	1,3×	—	—	\$2 600	D3 Council 4/4
Rebuild D4 (moduł)	D4	2,0×	—	—	\$4 000	D4 Human Gate

Ścieżka	Klasa decyz.	Mnożnik	Rezerwa	Ext. review	Przykład (\$2 000 base)	Gate wejścia
Rebuild D5 (system)	D5	3,0×	+20 %	+external fee	\$7 700*	D5 Council+Human+External

* Przykład D5: $\$2\ 000 \times 3,0 \times 1,20 + \500 external review fee = \$7 700. external_review_fee_usd jest konfigurowalne per decyzja D5 i zatwierdzone przez Human Gate razem z akceptacją rebuild (Plan 14 sekcja 14.6).

PLAN 10 · ROZSZERZENIE v3.1 · REVERSIBILITY

Cost Governance · v3.1 EXPAND

10.9. Post-Cutover Preservation Window jako pozycja budżetu D4/D5

REVERSIBILITY · Koszt jako pierwszorzędny element decyzji rebuild

v3.0 liczył koszt rebuild jako koszt budowy target. v3.1 dodaje do tego koszt utrzymania legacy po cutover — bo bez tej pozycji budżet D4/D5 jest fałszywy: rebuild bez LPW oznacza brak rollback, a ukrywanie tego kosztu przez pomijanie go w budżecie prowadzi do jednokierunkowych decyzji. Budżet musi odzwierciedlać rzeczywistą cenę odwracalności.

10.9.1. Nowa pozycja budżetu: legacy_preservation_cost

Każda decyzja D4 lub D5 obejmująca cutover MUSI zawierać w budget_envelope.yaml osobną pozycję legacy_preservation_cost z rozbiem na STABILIZATION + WARM_STANDBY. Bez tej pozycji Plan 13 G-COST-01 odrzuca bundle na etapie review.

```
# budget_envelope.yaml (rozszerzenie v3.1)
decision_class: D4
target_build_cost_usd: 12000 # koszt budowy target (v3.0)
legacy_preservation_cost: # NOWE w v3.1
  stabilization:
    duration: 72h # zgodne z Plan 02 cutover_policy
    legacy_runtime_cost_usd: 180 # 60/h * 72h
    write_through_overhead_usd: 40 # replication bandwidth/CPU
    blue_team_watch_cost_usd: 120 # dodatkowe godziny monitoring
    subtotal_usd: 340
  warm_standby:
    duration: 14d # default per provider manifest
    legacy_idle_cost_usd: 420 # 30/d * 14d (20% normal)
    wal_archive_cost_usd: 80
    subtotal_usd: 500
  rollback_rehearsal_cost_usd: 200 # obowiązkowe ćwiczenie w STABILIZATION
  total_preservation_usd: 1040
decommission_review_cost_usd: 300 # Plan 13 G-CUT-04 Council fast
grand_total_usd: 13340 # target_build + preservation + decommission
```

10.9.2. Mnożniki klasowe

Klasa	STABILIZATION default	WARM_STANDBY default	Rehearsal obowiązkowy	External review fee
D3 (refactor)	24h	7d	1x	—
D4 (rebuild modułu)	72h	14d	2x	opcjonalnie
D5 (rebuild systemu)	168h (7d)	30d	3x + chaos drill	wymagane

ZASADA · Preservation cost rośnie z klasą decyzji

D5 wymaga 168h STABILIZATION i 30d WARM_STANDBY nie dlatego że chcemy być bardziej ostrożni — ale dlatego że blast radius D5 jest tak duży, że żadne okno obserwacji krótsze nie dałoby realnej szansy wykrycia subtelnych regresji zanim legacy zniknie. Koszt preservation jest liniowy wobec ryzyka systemowego.

10.9.3. Kiedy preservation cost może być obniżony

Operator może wystąpić z wnioskiem o obniżenie preservation window, ale tylko w granicach wyznaczonych przez klasę decyzji i tier modułu. Obniżenie poniżej klasowego minimum wymaga osobnej decyzji D3 i wpisu w Evidence pack.

Warunek obniżenia	Max redukcja STABILIZATION	Max redukcja WARM	Wymóg
Moduł STANDARD-tier, brak user-facing	-50%	-50%	Council 3/4
Moduł PRO, D3 tylko refactor	-25%	-25%	Council 4/4
Moduł STATE/PHANTOM	brak redukcji	brak redukcji	—

10.9.4. Pozycja budżetu rollback

Odrębny wiersz budżetu rezerwuje koszt potencjalnego rollback. Nie jest to wydatek pewny, ale stanowi oczekiwany koszt alternatywny. Uwzględnienie tej pozycji eliminuje presję ekonomiczną na odrzucenie rollback w razie problemu („już za dużo wydaliśmy, musimy brnąć”).

```
# Sekcja rollback_reserve w budget_envelope.yaml v3.1
rollback_reserve:
  probability_pct: 10          # historyczny współczynnik dla danej klasy
  rollback_execution_cost_usd: 450 # fence + swap + post-mortem
  expected_cost_usd: 45        # probability_pct% * execution_cost
  mandatory_reserve_usd: 500    # minimum niezależnie od probability
  chosen_usd: 500              # max(expected, mandatory)
```

REVERSIBILITY · Rollback reserve jako ubezpieczenie

Nawet jeśli probability jest niska, mandatory_reserve zapewnia że rollback jest zawsze finansowo dozwolony. Plan 13 G-COST-02 weryfikuje tę pozycję przy każdym approve D3+. Brak rollback_reserve = budżet odrzucony.

PLAN 10 · ROZSZERZENIE v3.3 · EFFICIENCY

Cztery budżety efficiency per moduł v3.3

§10.10 — Cztery budżety efficiency per moduł (v3.3)

v3.3 rozszerza budget_envelope.yaml o cztery wymiary efficiency. Każdy moduł posiada teraz cztery niezależne budżety: bloat_budget (czytelność i utrzymywalność kodu), perf_budget (latencja runtime), memory_budget (RSS i heap peaks) oraz cost_budget (token spend i koszty modelu). Budżety są traktowane jako HARD CONSTRAINT — ich przekroczenie jest egzekwowane przez bramy G-EFF/G-PERF/G-MEM/G-COST (szczegóły: Plan 12 §12.8). Architektura hierarchicznego budżetu 4-poziomowego została opracowana na podstawie badań nad zarządzaniem zasobami w rozproszonych systemach agentowych.

EFFICIENCY · Cztery budżety = cztery osie egzekucji

Budżety efficiency nie są sugestiami — każda brama G-EFF/G-PERF/G-MEM/G-COST może zawetować bundle przed merge. Brak zdefiniowanego budżetu w budget_envelope.yaml = odrzucenie przez G-EFF-01 na etapie review. Hierarchia 4-poziomowa (moduł → dział → pipeline → org) zapewnia propagację eskalacji bez konieczności ręcznej interwencji operatora.

§10.10.1 — bloat_budget

bloat_budget kontroluje czytelność i utrzymywalność kodu. Metryki są obliczane przez statyczne narzędzia analizy kodu (SonarQube, Kiuwan, Code Climate) i weryfikowane przez bramę G-EFF-01 przy każdym bundlu. Przekroczenie któregokolwiek z parametrów blokuje merge jako HARD veto.

Parametr	Wartość domyślna	Źródło
cognitive_complexity_max	15	SonarQube standard
cyclomatic_complexity_max	20	ISO/IEC 25010
method_lines_max	50	Kiuwan best practice
nesting_depth_max	3	Clean Code, Martin
file_lines_max	500	konwencja projektowa
clone_coverage_max_pct	20%	SonarQube DRY gate
tech_debt_ratio_max	10% (grade B)	Code Climate A-F
llm_anti_pattern_density_max	patrz tabela poniżej	Ox Security research
<pre># budget_envelope.yaml — sekcja bloat_budget bloat_budget: cognitive_complexity_max: 15 # SonarQube Cognitive Complexity cyclomatic_complexity_max: 20 # McCabe CC; ISO/IEC 25010</pre>		

method_lines_max: 50	# Kiuan; 1 metoda = 1 odpowiedzialność
nesting_depth_max: 3	# głębokość zagnieżdżeń if/loop/try
file_lines_max: 500	# powyżej = kandydat do podziału
clone_coverage_max_pct: 20	# % zduplikowanego kodu w module
tech_debt_ratio_max: 10	# % = Code Climate grade B boundary
llm_anti_pattern_density_max:	
comments_everywhere_pct: 90	# nadmiar komentarzy
by_the_book_fixation_pct: 80	# zbyt sztywne wzorce
avoidance_of_refactors_pct: 75	# unikanie refaktoru
over_specification_pct: 70	# przesadna specyfikacja
bugs_deja_vu_pct: 60	# powtarzające się błędy
enforcement: hard	# G-EFF-01..04 veto

Tabela anti-pattern density LLM — maksymalne progi dla pipeline SYLION:

LLM Anti-pattern	Opis	Próg MAX (pipeline)
Comments Everywhere	Generowanie nadmiarowych komentarzy zastępujących czytelny kod	90-100%
By-the-Book Fixation	Sztywne trzymanie się wzorca bez oceny kontekstu	80-90%
Avoidance of Refactors	Obejście potrzebnego refaktoru przez dodawanie warstw	75%
Over-Specification	Przesadnie szczegółowe API i interfejsy bez uzasadnienia	70%
Bugs Déjà-Vu	Powtarzające się wzorce błędów w kolejnych iteracjach	60%

§10.10.2 — perf_budget

perf_budget definiuje wymagania latencji i niezawodności runtime dla każdego modułu. Metryki są weryfikowane przez bramę G-PERF w oparciu o dane z monitoring pipeline (Plan 12). Error budget jest automatycznie wyliczany z SLO target i monitorowany w oknie miesięcznym.

Parametr	Wartość domyślna	Opis
p50_ms_max	100 ms	mediana latencji odpowiedzi
p95_ms_max	300 ms	95. percentyl — user-facing API
p99_ms_max	500 ms	99. percentyl — SLO boundary
throughput_min_rps	50 RPS	minimalna przepustowość
error_rate_pct_max	0,1%	max wskaźnik błędów 5xx
slo_target	99,9%	cel dostępności miesięcznej
error_budget_minutes_per_month	43,2 min	auto: SLO=99,9% → 0,1% × 43 200 min


```
# budget_envelope.yaml – sekcja perf_budget (przykład: user-facing API)
perf_budget:
  p50_ms_max: 100          # mediana latencji
  p95_ms_max: 300          # user-facing API: < 300 ms (OneUptime standard)
  p99_ms_max: 500          # SLO boundary
  throughput_min_rps: 50    # minimalna przepustowość
  error_rate_pct_max: 0.1   # < 0,1% błędów 5xx
  slo_target: 99.9          # % dostępności
  error_budget_minutes_per_month: 43.2 # 0.1% * 43200 min/month
  enforcement: soft         # G-PERF veto = soft (Performance Eng review)
```

Benchmarki DORA — poziomy wydajności organizacyjnej według raportu DORA:

Poziom DORA	Deployment Frequency	Lead Time for Changes	MTTR	Change Failure Rate
Elite	Na żądanie (wielokrotnie dziennie)	< 1 godzina	< 1 godzina	0-5%
High	1× dziennie – 1× tygodniowo	1 dzień – 1 tydzień	< 1 dzień	5-10%
Medium	1× tygodniowo – 1× miesięcznie	1 tydzień – 1 miesiąc	1 dzień – 1 tydzień	10-15%
Low	< 1× miesięcznie	1 miesiąc – 6 miesięcy	> 1 tydzień	15-30%

PERFORMANCE · Error Budget Policy

Przekroczenie miesięcznego error budget (domyślnie 43,2 min downtime) freezuje wszystkie deploje poza klasami PO i security do czasu powrotu do SLO. Policy jest egzekwowana automatycznie przez G-PERF-03. Google SRE Workbook definiuje tę zasadę jako fundament zarządzania niezawodnością.

§10.10.3 — memory_budget

memory_budget określa limity zużycia pamięci operacyjnej dla każdego modułu. Parametry są zgodne ze standardami Kubernetes cgroup v2 — pola memory.high i memory.max są mapowane bezpośrednio na konfigurację cgroup. Brama G-MEM weryfikuje treść memory profili w fazie pre-bundle.

Parametr	Wartość domyślna	Opis
rss_peak_mb_max	512 MB	szczytowe RSS procesu
heap_peak_mb_max	256 MB	szczytowy heap (managed memory)
gc_pause_ms_p99_max	50 ms	max pauza GC na p99
leak_rate_mb_per_hour_max	0,5 MB/h	max wyciek pamięci

Parametr	Wartość domyślna	Opis
cgroup_memory_high_mb	80% limitu	próg alertu (Kubernetes memory.high)
cgroup_memory_max_mb	konfig. per środ.	twardy limit kontenera (memory.max)
<pre># budget_envelope.yaml – sekcja memory_budget (Kubernetes cgroup v2) memory_budget: rss_peak_mb_max: 512 # RSS szczytowy procesu heap_peak_mb_max: 256 # heap managed (JVM/V8/CPython) gc_pause_ms_p99_max: 50 # max pauza GC na 99. percentylu leak_rate_mb_per_hour_max: 0.5 # próg wycieku; >0,5 = alert G-MEM-02 cgroup_memory_high_mb: 410 # 80% * 512 MB = alert threshold cgroup_memory_max_mb: 512 # twardy limit; OOM kill powyżej # mapowanie Kubernetes cgroup v2: # memory.min = reserved guarantee (nie konfigurowane tutaj) # memory.high = cgroup_memory_high_mb (throttle + alert) # memory.max = cgroup_memory_max_mb (OOM kill) enforcement: soft # G-MEM veto = soft (Memory Auditor)</pre>		

Narzędzia profilowania pamięci per język — wymagane do dowodu memory_budget compliance:

Język / Runtime	Narzędzie profilowania	Format wyjściowy	Uwagi
Node.js	clinic.js, heapdump	HTML flamegraph, .heapsnapshot	clinic flame / clinic heap
Python	tracemalloc, memray	JSON, HTML report	memray: flamegraph + live
Java / JVM	JFR (Java Flight Recorder), async-profiler	.jfr, collapsed stacks	JFR = zero-overhead
Go	pprof	pprof binary	go tool pprof -alloc_space
Rust	heaptrack, valgrind/massif	HTML, callgrind	heaptrack preferowane

MEMORY · Próg alertów: 80% utilization

Ustawienie cgroup_memory_high na 80% limitu to standard branżowy Kubernetes — daje systemowi czas na reakcję (throttle, GC trigger) zanim osiągnięty zostanie twardy limit. Przy 80% threshold alert G-MEM-01 wyzwała przegląd profilu; przy 95% G-MEM-02 blokuje nowe deploje modulu.

§10.10.4 — cost_budget

cost_budget kontroluje wydatki na tokeny modeli LLM i koszt per decyzję. Model routing tier przypisuje typ zadania do odpowiedniej klasy modelu — zadania D0-D2 używają Haiku, D3 Sonnet, D4-D5 Opus. Prompt caching jest wymagane dla wszystkich agentów D0-D2 — wg

ProjectDiscovery redukuje koszty o 59-70%.

Parametr	Wartość domyślna	Opis
monthly_usd_envelope	konfig. per moduł	miesięczna koperta budżetowa USD
daily_burn_max_pct	4%	max dzienny burn (4% = 30-dniowe równomierne zużycie)
token_input_budget	konfig. per run	limit tokenów wejściowych per run
token_output_budget	konfig. per run	limit tokenów wyjściowych per run
model_routing_tier	haiku sonnet opus	tier modelu per typ zadania
prompt_cache_hit_rate_min_pct	50%	min. skuteczność cache (wymagane D0-D2)
cost_per_decision_usd_max	konfig. per klasa	max koszt jednej decyzji

```
# budget_envelope.yaml – sekcja cost_budget z model routing strategy
cost_budget:
  monthly_usd_envelope: 500          # miesięczna koperta dla modułu
  daily_burn_max_pct: 4              # max 4% koperty dziennie (30-day spread)
  token_input_budget: 100000         # tokenów wejściowych per run
  token_output_budget: 20000         # tokenów wyjściowych per run
  prompt_cache_hit_rate_min_pct: 50 # wymagane dla D0-D2
  cost_per_decision_usd_max: 0.05    # max koszt 1 decyzji
  model_routing:
    D0: haiku                        # $0.25/$1.25 per MTok IN/OUT
    D1: haiku                        # zadania refleksyjne niskokosztowe
    D2: haiku                        # review wspomagający
    D3: sonnet                       # $3/$15 per MTok; Council session
    D4: opus                         # $15/$75 per MTok; Human Gate
    D5: opus                         # Greenfield – najwyższy tier
  enforcement: soft                  # G-COST veto = soft (Cost Optimizer)
```

Model routing tiers — orientacyjne ceny (na 1M tokenów, IN/OUT):

Tier	Przykładowy model	Cena IN (\$/1M tok.)	Cena OUT (\$/1M tok.)	Zastosowanie
Haiku	claude-haiku-3.5	\$0,25	\$1,25	D0-D2: zadania automatyczne, batch
Sonnet	claude-sonnet-4	\$3,00	\$15,00	D3: Council session, review
Opus	claude-opus-4	\$15,00	\$75,00	D4-D5: Human Gate, greenfield

COST · Prompt caching: 59-70% redukcji kosztów

Badania ProjectDiscovery wykazują, że agresywne prompt caching redukuje koszty tokenów o 59-70% w typowych workloadach agentowych. Z tego powodu `prompt_cache_hit_rate_min_pct=50%` jest wymagane dla wszystkich agentów klasy D0-D2. Niższy cache hit rate jest automatycznie flagowany przez G-COST-02.

§10.10.5 — Hierarchia budżetów i alerty escalation

Cztery budżety są zorganizowane w hierarchię czteropoziomową: per moduł, per dział, per pipeline i org-wide. Każdy poziom ma własny budżet i progi alertowe. Przekroczenie progu niższego poziomu automatycznie eskaluje do poziomu wyższego — bez konieczności ręcznej interwencji.

Poziom	Zakres	Właściciel	Próg 90%	Próg 100%
Per moduł	Pojedynczy moduł SYLION	Tech Lead modulu	ALERT → Code Optimizer	BREACH → veto deploy
Per dział	Plan 15 — jeden z 7 działów	Head of Division	ALERT → Head of Division	BREACH → Council review
Per pipeline	Cały pipeline SYLION	Pipeline Owner	ALERT → Pipeline Owner	BREACH → Human Gate D4
Org-wide	Wszystkie pipeline org	CTO / CFO	ALERT → CTO	BREACH → emergency freeze

budget_envelope.yaml – sekcja budget_alert_thresholds
budget_alert_thresholds:
 level_50_pct:
 action: INFO
 notify: []
 auto_escalate: false
 # log + dashboard badge
 # brak powiadomień aktywnych
 level_75_pct:
 action: WARN
 notify: [tech_lead]
 auto_escalate: false
 # dashboard warning + Slack
 level_90_pct:
 action: ALERT
 notify: [tech_lead, cost_optimizer, pipeline_owner]
 auto_escalate: true
 # PagerDuty alert + blokada nowych runów
 # eskalacja do poziomu wyższego
 level_100_pct:
 action: BREACH
 notify: [all_owners, cto]
 auto_escalate: true
 deploy_freeze: true
 # twardy circuit breaker
 # freeze deployów poza P0/security

% budżetu zużytego	Akcja	Klasa decyzji	Owner
50%	INFO — log + badge	D0 (auto)	brak / monitoring
75%	WARN — Slack alert	D1 (auto + log)	Tech Lead

% budżetu zużytego	Akcja	Klasa decyzji	Owner
90%	ALERT — PagerDuty + freeze nowych runów	D2 (wymaga odpowiedzi)	Tech Lead + Cost Optimizer
100%	BREACH — circuit breaker + deploy freeze	D3 (Council review)	Pipeline Owner + CTO

§10.10.6 — Integracja z bramami G-EFF/G-PERF/G-MEM/G-COST

Każdy z czterech budżetów jest egzekwowany przez dedykowane bramy w Plan 12 (§12.8). Bramy działają jako pre-bundle filter — weryfikują compliance zanim bundle trafi do merge. Veto twarde (hard) blokuje automatycznie; veto miękkie (soft) wymaga zatwierdzenia przez właściciela bramy.

Brama	Sprawdza budżet	Sprawdzany parametr	Próg	Typ veto	Właściciel
G-EFF-01	bloat_budget	cognitive_complexity_max	15	hard	Code Optimizer
G-EFF-02	bloat_budget	cyclomatic_complexity_max	20	hard	Code Optimizer
G-EFF-03	bloat_budget	clone_coverage / tech_debt_ratio	20% / 10%	hard	Code Optimizer
G-EFF-04	bloat_budget	llm_anti_pattern_density	per tabela	hard	Code Optimizer
G-PERF-01	perf_budget	p95_ms_max	300 ms	soft	Performance Eng.
G-PERF-02	perf_budget	error_rate_pct_max	0,1%	soft	Performance Eng.
G-PERF-03	perf_budget	slo_target / error_budget	99,9% / 43,2 min	soft	Performance Eng.
G-MEM-01	memory_budget	rss_peak_mb_max / cgroup_high	512 MB / 80%	soft	Memory Auditor
G-MEM-02	memory_budget	leak_rate_mb_per_hour_max	0,5 MB/h	soft	Memory Auditor
G-COST-01	cost_budget	monthly_usd_envelope / daily_burn	koperta / 4%	soft	Cost Optimizer
G-COST-02	cost_budget	prompt_cache_hit_rate_min_pct	50%	soft	Cost Optimizer

EFFICIENCY · Cztery budżety + 11 bram = pełny pre-bundle filter

Zestaw G-EFF-01..04 (4 bramy twarde) + G-PERF-01..03 (3 bramy miękkie) + G-MEM-01..02 (2 bramy miękkie) + G-COST-01..02 (2 bramy miękkie) tworzy kompletny pre-bundle filter efficiency v3.3. Każdy bundle musi przejść przez wszystkie 11 bram zanim dotrze do merge. Szczegółowa specyfikacja bram: Plan 12 §12.8.

§10.10.7 — Wzajemne relacje budżetów (trade-offs)

Cztery budżety nie są niezależne — optymalizacja jednego wymiaru wpływa na pozostałe. Niżej zestawiono najczęstsze trade-offs zidentyfikowane w pipeline SYLION. Każda decyzja optymalizacyjna musi być świadoma kosztu na pozostałych osiach.

Optymalizacja czego	Korzyść	Koszt na innym wymiarze	Rekomendacja
Cache aggressively	cost_budget -50%	memory_budget +30% (cache trzymany w heap)	Akceptowalne jeśli heap_peak < limitu
Refactor for readability	bloat_budget → pass	perf_budget +5% (dodatkowe warstwy abstrakcji)	Zwykle akceptowalne; profiling potwierdza
Switch model haiku→opus	perf_budget -40% latencji	cost_budget +6× (Opus vs Haiku cena)	Tylko dla D4-D5; wymaga Council approve
Aggressive GC tuning	memory_budget ↓ peak	perf_budget: GC pause może wzrosnąć	Wymaga testu gc_pause_ms_p99
Inline everything	perf_budget ↓ latencja	bloat_budget: method_lines i clone_coverage rosną	Anti-pattern; nie stosować bez refaktoru

EFFICIENCY · Council 4/4 musi widzieć trade-offs przed approve D3+

Każda decyzja D3 lub wyższa, która optymalizuje jeden budżet kosztem innego, musi zawierać w Evidence pack explicitną tabelę trade-offów ze wszystkich 4 osi. Wybór jest świadomy, nie automatyczny — Council podpisuje się pod decyzją znając pełny koszt. Brak tabeli trade-offów = odrzucenie przez G-EFF-04 (wymagany artefakt Evidence pack).

PLAN 11 · WARSTWA KANONICZNA

*Run Lifecycle**REBUILD · Rebuildability over Lineage w Planie 11*

Run Lifecycle Plane jest pierwszą warstwą, w której zasada Rebuildability staje się „operacyjna”. W v3.0 pipeline może utrzymywać równocześnie dwie linie wykonawcze: legacy track (stary bundle, stare kontrakty, MZ 3.5.5) oraz target track (nowy bundle, nowe kontrakty, architektura docelowa). Każdy typ runa — legacy_run, target_run, shadow_run, dual_run, cutover_run, decommission_run — jest pierwszoklasowym obywatelem pipeline z własnym manifestem, telemetrią i Evidence. Wielotorowe runy nie są wyjątkiem: są projektowym wymogiem dla ścieżek Refactor i Rebuild. Koordynator Runów (Run Coordinator) jest odpowiedzialny za porównywanie wyników między torami i podejmowanie decyzji o promocji target do baseline.

11.1. Cel i umiejscowienie

Run Lifecycle Plane jest centralnym koordynatorem pojedynczego przebiegu potoku SYLION. W v3.0 odpowiada za orkiestrację wszystkich etapów — od momentu wyzwolenia triggerem aż do zarchiwizowania artefaktów i zamknięcia feedbacku — w jednym lub wielu równoległych torach wykonawczych. Plan 11 łączy wszystkie pozostałe plany: korzysta z infrastruktury (Plan 02 — stany VPS per tor), wiedzy (Plan 03), sekretów (Plan 04), modeli i decyzji (Plan 05), agentów (Plan 06), ewolucji kanonicznej (Plan 08), kosztów (Plan 10), dostarcza gotowe runy do Bundle Plane (Plan 12 — bundle per tor) i generuje zdarzenia audytowe dla Governance Plane (Plan 13 — Evidence multi-track). Tryb wielotorowy jest aktywowany automatycznie dla ścieżek Refactor i Rebuild (szczegóły w Planie 14 — Greenfield Rebuild Orchestration).

- Run = jedna atomowa iteracja ewolucji systemu SYLION
- Każdy run ma unikalny UUID oraz niemutowalny manifest inicjalny
- W trybie multi-track pipeline prowadzi równolegle: legacy track i target track
- Pre-stages A0–A5 przeprowadzają kanonizację kontekstu przed właściwym rdzeniem
- Core 0–9 to sekwencja główna od bootstrapu aż po obserwację i feedback
- Wszystkie przejścia stanów są logowane do Matrix (Plan 13) i są niemutowalne
- Run Coordinator porównuje wyniki legacy vs target i decyduje o promocji

11.2. Nowość v3.0 — Multi-Track Runs

W trybie Refactor i Rebuild pipeline utrzymuje równolegle dwie linie wykonawcze. Mechanizm multi-track jest aktywowany po decyzji klasy D3 (Refactor path) lub D4/D5 (Rebuild path) i wyzwala run_coordinator, który tworzy dwa manifesty runów: jeden dla legacy track i jeden dla target track. Oba tory są niezależne — mają osobne środowiska VPS (Plan 02), osobne bundle (Plan 12) i osobne kolekcje Evidence (Plan 13). Ich wyniki są porównywane przez Run Coordinator zgodnie z metrykami zdefiniowanymi w multi_track_run_manifest.yaml.

Tor	Bundle	Kontrakty	Baseline	Tryb aktywacji
legacy track	Stary bundle (pre-rebuild)	Stare kontrakty (MZ 3.5.5)	MZ 3.5.5	Zawsze aktywny (domyślny)
target track	Nowy bundle (post-rebuild)	Nowe kontrakty (target arch.)	target architecture id	Tylko Refactor/Rebuild (D3+)

11.3. Sześć typów runów

Plan 11 v3.0 definiuje sześć typów runów. Każdy typ ma przypisaną klasę decyzji wejścia, własny manifest, własną telemetrię i własne Evidence. Typy 1–2 mogą istnieć niezależnie; typy 3–6 są zawsze elementem sekwencji multi-track i wymagają aktywnego target track.

Typ runa	Klasa decyzji wejścia	Track	Opis
legacy_run	D1	legacy	Standardowy run na starym bundlu. Używany w trybie Evolution i jako baseline podczas Refactor/Rebuild. Kontrakty MZ 3.5.5, stare środowisko VPS.
target_run	D2	target	Run na nowym bundlu (architektura docelowa). Dostępny wyłącznie po aktywacji target track (D3+). Może działać samodzielnie lub równolegle z legacy_run.
shadow_run	D3	oba (shadow)	Target track otrzymuje dane legacy; wyniki są porównywane z legacy_run bez wpływu na produkcję. Kluczowy etap walidacji cutover (cutover validation).
dual_run	D3	oba (dual)	Oba tory wykonują to samo zadanie niezależnie. Council ocenia wyniki i decyduje, który tor jest poprawny. Używany do weryfikacji parytetów funkcjonalnych przed cutover.
cutover_run	D4	target (primary) + legacy (fallback)	Kontrolowane przełączenie ruchu z legacy na target. Legacy track pozostaje aktywny jako fallback do czasu potwierdzenia stabilności target. Wymaga Human Gate (D4).
decommission_run	D5 (system-wide cutover)	legacy (ostatni)	Archiwizacja i wyłączenie legacy track. Trwałe zamknięcie starego bundlu, kontraktów i środowisk VPS. Wymaga Council 4/4 + Human + External Review (D5).

REBUILD · Klasy decyzji wejścia — multi-track

legacy_run → D1: może startować po zatwierdzeniu jednego agenta. target_run → D2: wymaga weryfikacji przez dwóch agentów + Review. shadow_run i dual_run → D3: wymaga Full Board Council 4/4 (Refactor path). cutover_run → D4: wymaga Council 4/4 + Human Gate. decommission_run → D5: wymaga Council 4/4 + Human + External Review. Szczegóły klas D0–D5 w tabeli sekcji 11.5.

11.4. Sekwencja pre-stages A0–A5

Przed uruchomieniem rdzenia (Core 0–9) każdy run — niezależnie od toru — przechodzi przez sześć etapów przygotowawczych. W trybie multi-track etapy A0–A3 są wspólne dla obu torów; A4 i A5 są wykonywane per tor (osobna rezerwacja budżetu i osobny pre-check bezpieczeństwa dla legacy i target).

Stag e	Nazwa	Aktor	Multi-track	Opis
A0	Trigger Ingestion	ACP (Plan 01)	wspólny	Odczyt i walidacja zdarzenia wyzwalającego (webhook, cron, manual). Normalizacja do RunTrigger. Wykrycie trybu: single-track vs multi-track.
A1	Context Canonicalization	Chief Context (Gemini 2.5 Pro)	wspólny	Pobranie stanu Księgi, historii runów, reputacji agentów. Budowa CanonicContext z uwzględnieniem obu torów (legacy + target manifest).
A2	Proposal Grouping	Chief Reasoning (Claude)	wspólny	Grupowanie propozycji zmian wg rozdziału + tematu + toru. Deduplikacja między torami.
A3	Provider Allocation	Infrastructure Plane (Plan 02)	per tor	Przydział VPS/Matrix per track. Legacy → VPS legacy tier; Target → VPS target tier. Stany VPS rozszerzone o multi-track (Plan 02).
A4	Budget Reservation	Cost Governance (Plan 10)	per tor	Rezerwacja budżetu tokenów i kosztów per track. Blokada przy przekroczeniu limitu któregośkolwiek toru.
A5	Gate Pre-Check	Chief Safety	per tor	Weryfikacja gate'ów wstępnych per track: Safety, Compliance, Operator freeze. Decyzja D1 (legacy) lub D2 (target). Shadow/dual → D3.

11.5. Sekwencja Core 0–9

Po pomyślnym przejściu A0–A5 run wchodzi w rdzeń. W trybie single-track sekwencja Core jest identyczna jak w v2.0. W trybie multi-track etapy Core 0–4 są wykonywane per tor, Core 5 (Council Decide) jest wspólny z uwzględnieniem wyników obu torów, Core 6 (Human Gate) jest wymagany dla D4/D5, Core 7 (Bundle Build) jest per tor, Core 8 (Deploy) jest per tor z

koordynacją cutover, Core 9 (Observe & Feedback) porównuje metryki między torami.

Core	Nazwa	Aktor	Timeout	Klasa	Multi-track
0	Bootstrap	ACP	2 min	D0	per tor
1	Context Load	Chief Context	10 min	D0	per tor
2	Analysis	Chief Reasoning	30 min	D1	per tor
3	Red Team	Chief Review (GPT-5)	20 min	D1	per tor
4	Blue Team	Chief Safety	20 min	D1	per tor
5	Council Decide	Board 4/4	60 min	D2/D3	wspólny (wyniki obu torów)
6	Human Gate	Operator SYLION	48 h	D4/D5	wspólny (wymagany dla cutover_run/decommission_run)
7	Bundle Build	Bundle Plane (Plan 12)	30 min	D0	per tor
8	Deploy	Infrastructure (Plan 02)	45 min	D1	per tor (koordynacja cutover)
9	Observe & Feedback	Observability + Run Coordinator	ongoing	D0	porównanie metryk legacy vs target

UWAGA · Uwaga: Core 6 (Human Gate) dla D5

Core 6 jest aktywowany gdy wynik Council (Core 5) kwalifikuje decyzję jako D4 lub D5. Dla cutover_run: D4 + Human Gate. Dla decommission_run: D5 + Human Gate + External Review (Council 4/4 + Human + zewnętrzny audytor lub drugi operator). Jeśli decyzja to D2 lub D3, run przeskakuje bezpośrednio do Core 7.

11.6. Stany runów — rozszerzenie v3.0

V3.0 rozszerza maszynę stanów runu o sześć nowych stanów multi-track. Stany legacy (S-RUN-00 do S-RUN-05) pozostają bez zmian dla toru legacy_run. Nowe stany (S-RUN-06 do S-RUN-11) opisują cykl życia toru target i fazy przełączenia. Stany są per run_id, nie per track — jeden run ma jeden stan reprezentujący fazę dominującą cyklu multi-track.

Stan	Kod	Opis	Możliwe przejścia
pending	S-RUN-00	Run utworzony, oczekuje na uruchomienie A0	running

Stan	Kod	Opis	Możliwe przejścia
running	S-RUN-01	Run aktywnie przetwarza etapy A0–A5 lub Core 0–9	blocked, completed, failed, cancelled, LEGACY_ACTIVE
blocked	S-RUN-02	Run zatrzymany na gate lub oczekuje na operatora (D4/D5)	running, failed, cancelled
completed	S-RUN-03	Wszystkie etapy zakończone sukcesem, artefakty zapisane	(terminal)
failed	S-RUN-04	Nieodwracalny błąd lub timeout bez możliwości retry	(terminal)
cancelled	S-RUN-05	Run anulowany przez operatora lub ACP	(terminal)
LEGACY_ACTIVE	S-RUN-06	Tor legacy aktywny jako baseline; target track w budowie lub shadow	TARGET_BUILDING, SHADOW_ACTIVE, completed
TARGET_BUILDING	S-RUN-07	Tor target jest aktywnie budowany (bundle, testy, walidacja kontraktów)	LEGACY_ACTIVE, SHADOW_ACTIVE, failed
SHADOW_ACTIVE	S-RUN-08	shadow_run aktywny: target otrzymuje dane legacy, wyniki porównywane; produkcja niezmiennona	DUAL_COMPARING, LEGACY_ACTIVE, failed
DUAL_COMPARING	S-RUN-09	dual_run aktywny: oba tory wykonują to samo zadanie; Council porównuje wyniki	CUTOVER_PENDING, SHADOW_ACTIVE, failed
CUTOVER_PENDING	S-RUN-10	Decyzja cutover_run podjęta (D4 zatwierdzone), oczekiwanie na wykonanie przełączenia ruchu z legacy na target	LEGACY_DECOMMISSIONED, LEGACY_ACTIVE (rollback)
LEGACY_DECOMMISSIONED	S-RUN-11	decommission_run zakończony: legacy track zarchiwizowany i wyłączony; target jest jedyną aktywną linią	(terminal)

11.7. Tabela klas decyzji D0–D5 w Planie 11

Klasa	Nazwa	Przykład operacji Run Lifecycle	Gate	Ścieżka
D0	Informational	Odczyt stanu runu, metryk, historii etapów	Auto	—
D1	Trivial	legacy_run w trybie Evolution; start runu po cron triggerze	1 agent	evolution
D2	Standard	target_run w trybie Refactor (nowy bundle, testy kontraktowe)	2 agentów + Review	refactor

Klasa	Nazwa	Przykład operacji Run Lifecycle	Gate	Ścieżka
D3	Significant	shadow_run, dual_run; aktywacja target track; Full Board Council 4/4	Council 4/4	refactor
D4	Critical	cutover_run: przełączenie ruchu z legacy na target; Human Gate	Council 4/4 + Human	rebuild
D5	Greenfield / Systemic	decommission_run (system-wide cutover): archiwizacja legacy; zewnętrzny audytor lub drugi operator	Council 4/4 + Human + External Review	rebuild (greenfield)

REBUILD · D5 — decommission_run

Decyzja D5 w Planie 11 oznacza trwałe wyłączenie legacy track na poziomie systemu. Nie jest to operacja odwracalna. Wymaga External Review (red team zewnętrzny lub drugi operator) potwierdzającego, że target track jest w pełni operacyjny i bezpieczny. Kolor D5: #6A1B9A (fiolet).

11.8. Synchronizacja multi-track — Run Coordinator

Run Coordinator (RC) jest nowym komponentem w v3.0, odpowiedzialnym za orkiestrację równoległych torów wykonawczych. RC działa jako supervisor dla obu track'ów: zbiera telemetrię z legacy_run i target_run, porównuje wyniki i podejmuje rekomendacje dotyczące promocji target do baseline lub rollback.

11.8.1. Jak RC porównuje wyniki legacy vs target

Run Coordinator stosuje trójpoziomowy algorytm porównawczy. Poziom 1 (output equality): porównuje hash wyników logicznych — jeśli $\text{SHA-256}(\text{output_legacy}) == \text{SHA-256}(\text{output_target})$, equality=100%. Poziom 2 (performance delta): porównuje latencję, zużycie tokenów i koszt per run. Poziom 3 (error delta): porównuje klasy błędów, częstość retry i severity zdarzeń.

- $\text{output_equality} \geq 99.5\%$ przez 50 kolejnych shadow_run'ów → PROMOTE_CANDIDATE
- $\text{performance_delta} \leq +10\%$ (target może być droższy o max 10% vs legacy) → akceptowalny
- $\text{error_delta} \leq 0$ (target nie może mieć WIĘCEJ błędów niż legacy) → warunek konieczny
- Jeśli którykolwiek warunek nie jest spełniony → RC emituje PROMOTE_BLOCK do Plan 13

11.8.2. Jak RC decyduje o promocji target do baseline

1. RC zbiera raporty z min. 50 shadow_run i 10 dual_run
2. RC oblicza metryki porównawcze: output_equality, performance_delta, error_delta
3. RC generuje promote_recommendation.yaml i emituje do Plan 13 (Evidence)
4. Chief Reasoning weryfikuje rekomendację i składa wniosek cutover_run (D4)
5. Council 4/4 głosuje nad wnioskiem cutover; wynik D4 → Human Gate
6. Operator zatwierdza cutover_run → stan CUTOVER_PENDING

- 7. RC przełącza ruch z legacy na target (zgodnie z cutover_strategy)
- 8. RC monitoruje target przez 24h po cutover; przy anomalii → automatyczny rollback
- 9. Po stabilizacji (24h bez anomalii) → wniosek decommission_run (D5)

11.8.3. Metryki śledzone przez Run Coordinator

Metryka	Definicja	Próg promocji	Próg rollback
output_equality	% runów, w których SHA-256(output_legacy) == SHA-256(output_target) lub wyniki są semantycznie równoważne (wg Chief Reasoning)	≥ 99.5% przez 50 shadow_run	< 95% w ciągu 10 consecutive shadow_run
performance_delta	Względna różnica latencji p95 i kosztu USD: (target_metric - legacy_metric) / legacy_metric	≤ +10% latencja, ≤ +15% koszt	> +30% latencja LUB > +50% koszt
error_delta	Różnica bezwzględna w liczbie błędów klasy WARN+ per 100 runów: error_count_target - error_count_legacy	≤ 0 (target nie gorszy)	> +5 błędów per 100 runów
contractparity	Wyniki testów kontraktowych (Plan 12): % przechodzonych testów kontraktowych na target track	100%	< 99% (twarde kryterium)
safety_gate_pass_rate	% shadow_run, w których Chief Safety zaakceptował output target track jako bezpieczny	100%	< 100% (twarde kryterium)

11.9. Relacje z innymi planami

Plan 11 w v3.0 jest hubem integracyjnym wielotorowego pipeline. Kluczowe zależności zewnętrzne w kontekście multi-track:

- Plan 02 (VPS) — stany VPS rozszerzone o stany multi-track environments: legacy_vps_active, target_vps_building, shadow_vps_active, cutover_vps_routing, decommissioned_vps. Każdy tor ma osobne środowisko VPS.
- Plan 12 (Bundle + Testing) — bundle per track: legacy_bundle (niezmieniony baseline) i target_bundle (nowa architektura). Plan 12 zarządza red/blue team bundles i testami kontraktowymi per tor.
- Plan 13 (Governance & Evidence) — Evidence multi-track: każdy shadow_run, dual_run i cutover_run generuje osobny pakiet Evidence. Plan 13 przechowuje log porównawczy RC i promote_recommendation.yaml.
- Plan 14 (Greenfield Rebuild Orchestration) — orkiestracja rebuild: Plan 14 wyzwala aktywację target track i definiuje cutover_strategy używaną przez Run Coordinator.

PLAN 11 · SPECYFIKACJA WYKONAWCZA

Run Lifecycle

CEL I ZAKRES

Specyfikacja Run Lifecycle Service (RLS) v3.0. Zakres: tworzenie i orkiestracja runów pipeline SYLION Core 8.0.0, zarządzanie stanami (w tym stanami multi-track), przechowywanie artefaktów per tor, porównywanie wyników przez Run Coordinator, eksport zdarzeń do Matrix i Evidence. RLS jest autorytatywnym źródłem prawdy o stanie i historii runów — wszystkich torów łącznie.

MODEL DOMENOWY		
Encja	Pola kluczowe	Typ / Uwagi
Run	run_id (UUID), trigger_id, state, track_mode, created_at, updated_at, manifest_sha256, budget_reserved, decision_class	Główna encja; niemutowalne: run_id, created_at, manifest_sha256; track_mode: single multi
TrackInstance	track_id (UUID), run_id, track_kind (legacy target), bundle_id, vps_id, state, started_at, finished_at, decision_class_entry	Jeden rekord na tor per run; nowa encja w v3.0
Stage	stage_id, run_id, track_id (nullable), code (A0..A5, C0..C9), name, state, started_at, finished_at, timeout_s, actor	track_id=null dla etapów wspólnych; per tor dla A3, A4, A5 i Core 0–9
StageExecution	exec_id, stage_id, attempt, input_sha256, output_sha256, error_code, duration_ms	Możliwe wielokrotne próby (retry)
RunArtifact	artifact_id, run_id, track_id (nullable), stage_id, kind, path, sha256, size_bytes, created_at	kind: log / bundle / report / checkpoint / compare_report
RunEvent	event_id, run_id, track_id (nullable), stage_id, event_type, payload_sha256, emitted_at, matrix_event_id	Każde zdarzenie ma hash chain w Plan 13
Checkpoint	chk_id, run_id, track_id, stage_code, state_snapshot_sha256, created_at, restorable	Checkpoint per tor; możliwość wznowienia od ostatniego checkpointu
TrackComparison	cmp_id, run_id, legacy_track_id, target_track_id, output_equality_pct, performance_delta_pct, error_delta, contract_parity_pct, safety_gate_pass_rate, rc_recommendation (PROMOTE BLOCK DEFER), created_at	Nowa encja v3.0; generowana przez Run Coordinator po shadow_run i dual_run
SCHEMAT YAML — run.yaml (v3.0)		

YAML

run_id: "550e8400-e29b-41d4-a716-446655440011"
version: "3.0"

```
trigger:
  kind: manual
  operator_id: "op-001"
  timestamp: "2026-04-01T08:00:00Z"
manifest_sha256: "a3f1..."
track_mode: single
decision_class_max: D2
budget:
  tokens_reserved: 2000000
  cost_usd_max: 12.50
  provider: "vps-legacy-01"
stages:
  pre:
    - code: A0
      actor: ACP
      timeout_s: 120
      track: shared
    - code: A1
      actor: chief_context
      timeout_s: 600
      track: shared
    - code: A2
      actor: chief_reasoning
      timeout_s: 900
      track: shared
    - code: A3
      actor: infra_plane
      timeout_s: 300
      track: per_track
    - code: A4
      actor: cost_governance
      timeout_s: 60
      track: per_track
    - code: A5
      actor: chief_safety
      timeout_s: 120
      track: per_track
  core:
    - code: C0
      actor: ACP
      timeout_s: 120
      track: per_track
    - code: C5
      actor: board_council
      timeout_s: 3600
      track: shared
    - code: C6
      actor: human_operator
      timeout_s: 172800
      track: shared
      condition: decision_class >= D4
    - code: C9
      actor: run_coordinator
      timeout_s: -1
      track: shared
retry_policy:
  max_attempts: 3
  backoff_s: 30
```

```

    retryable_stages: [A0, A1, C1, C2, C3, C4]
  checkpoint_policy:
    enabled: true
    after_stages: [A5, C5, C7]

```

SCHEMAT YAML — *multi_track_run_manifest.yaml*

Poniższy schemat opisuje manifest runa wielotorowego. Zawiera definicję obu torów, strategię cutover, metryki porównawcze używane przez Run Coordinator oraz politykę rollback.

YAML

```

run_id: "7a2c1e00-f3d4-48b9-c821-997744bb0022"
version: "3.0"
track_mode: multi
trigger:
  kind: manual
  operator_id: "op-001"
  timestamp: "2026-04-01T09:00:00Z"
  rebuild_decision_class: D3
  rebuild_path: refactor

```

tracks:

- track_id: "legacy-track-001"
 - track_kind: legacy
 - bundle_id: "bundle-mz355-stable"
 - baseline: "MZ 3.5.5"
 - implementation_strategy: reuse
 - vps_tier: legacy
 - vps_id: "vps-legacy-01"
 - decision_class_entry: D1
 - budget:
 - tokens_reserved: 1500000
 - cost_usd_max: 8.00
 - state: LEGACY_ACTIVE
- track_id: "target-track-001"
 - track_kind: target
 - bundle_id: "bundle-target-v30-rc1"
 - baseline: "target-arch-v30-001"
 - implementation_strategy: refactor
 - current_baseline: MZ 3.5.5
 - target_baseline: "target-arch-v30-001"
 - cutover_strategy: shadow
 - vps_tier: target
 - vps_id: "vps-target-01"
 - decision_class_entry: D2
 - budget:
 - tokens_reserved: 2000000
 - cost_usd_max: 14.00
 - state: TARGET_BUILDING

run_sequence:

- phase: shadow
 - run_type: shadow_run
 - decision_class: D3
 - min_runs: 50
 - promotion_gate:
 - output_equality_min_pct: 99.5


```

    performance_delta_max_pct: 10.0
    error_delta_max: 0
    contract_parity_min_pct: 100.0
    safety_gate_pass_rate_min: 100.0

- phase: dual
  run_type: dual_run
  decision_class: D3
  min_runs: 10
  council_eval: true

- phase: cutover
  run_type: cutover_run
  decision_class: D4
  human_gate_required: true
  rollback_window_h: 24
  traffic_split:
    initial_target_pct: 10
    increment_pct: 20
    increment_interval_min: 30
    final_target_pct: 100

- phase: decommission
  run_type: decommission_run
  decision_class: D5
  human_gate_required: true
  external_review_required: true
  legacy_archive_retention_days: 365

run_coordinator:
  comparison_metrics:
    - name: output_equality
      weight: 0.40
    - name: performance_delta
      weight: 0.20
    - name: error_delta
      weight: 0.25
    - name: contract_parity
      weight: 0.10
    - name: safety_gate_pass_rate
      weight: 0.05
  promote_threshold_score: 0.95
  rollback_trigger:
    output_equality_min_pct: 95.0
    error_delta_max: 5
    performance_delta_max_pct: 30.0
  evidence_plan: 13
  evidence_event_type: "multi_track_comparison"

checkpoint_policy:
  enabled: true
  after_phases: [shadow, dual, cutover]

manifest_sha256: "b9e3..."
created_at: "2026-04-01T09:00:00Z"

```

SCHEMATY YAML — moduły (implementation_strategy)

Module Run Lifecycle Service — Legacy track

YAML

```
apiVersion: sylion.io/v3
kind: Module
metadata:
  module_id: "rls-legacy-engine"
  name: "Run Lifecycle Service — Legacy Track"
  plan: "11"
spec:
  implementation_strategy: reuse
  current_baseline: MZ 3.5.5
  target_baseline: null
  cutover_strategy: null
  version: "3.0.0"
  track_kind: legacy
  vps_tier: legacy
  bundle_compatibility: "bundle-mz355-*"
  decision_class_entry: D1
```

Module Run Lifecycle Service — Target track (refactor)

YAML

```
apiVersion: sylion.io/v3
kind: Module
metadata:
  module_id: "rls-target-engine"
  name: "Run Lifecycle Service — Target Track"
  plan: "11"
spec:
  implementation_strategy: refactor
  current_baseline: MZ 3.5.5
  target_baseline: "target-arch-v30-001"
  cutover_strategy: shadow
  rebuild_path: refactor
  version: "3.0.0"
  track_kind: target
  vps_tier: target
  bundle_compatibility: "bundle-target-v30-*"
  decision_class_entry: D2
```

Module Run Coordinator

YAML

```
apiVersion: sylion.io/v3
kind: Module
metadata:
  module_id: "run-coordinator"
  name: "Run Coordinator — Multi-Track"
  plan: "11"
spec:
  implementation_strategy: greenfield
  current_baseline: null
  target_baseline: "run-coordinator-v30"
```

```

cutover_strategy: cut-and-replace
version: "1.0.0"
new_in: "v3.0"
comparison_algorithm: weighted_score
evidence_integration: plan_13
decision_class_promote: D4
decision_class_decommission: D5

```

WORKFLOWY

WF-RUN-01: Start Run (single track)

1. Operator lub cron emituje trigger zdarzenie do ACP
2. ACP tworzy rekord Run (stan: pending, track_mode: single), generuje run_id i manifest_sha256
3. RLS uruchamia sekwencję A0→A1→A2→A3→A4→A5 sekwencyjnie
4. Po sukcesie A5: stan → running, startuje C0
5. Po C9: stan → completed, 3 strumienie feedbacku do Plan 07, budżet zwolniony

WF-RUN-02: Aktywacja Multi-Track (shadow_run)

1. Council podejmuje decyzję D3 (Refactor path) — aktywacja target track
2. RLS tworzy dwa rekordy TrackInstance: legacy-track i target-track
3. A3 (Provider Allocation) przydziela osobne VPS per tor (Plan 02)
4. A4 (Budget) rezerwuje budżet per tor; A5 (Gate) weryfikuje per tor
5. Run wchodzi w stan LEGACY_ACTIVE; target-track → TARGET_BUILDING
6. Po zbudowaniu target bundle (C7 dla target-track): stan → SHADOW_ACTIVE
7. Run Coordinator uruchamia shadow_run: target-track dostaje dane z legacy_run
8. RC zbiera TrackComparison po każdym shadow_run; emituje wyniki do Plan 13
9. Po 50 shadow_run: RC oblicza metryki i emituje promote_recommendation.yaml

WF-RUN-03: dual_run i Council Evaluate

1. RC rekomenduje PROMOTE_CANDIDATE po 50 shadow_run z output_equality ≥ 99.5%
2. Chief Reasoning składa wniosek dual_run (D3)
3. Council 4/4 głosuje; po zatwierdzeniu: stan → DUAL_COMPARING
4. Oba tory wykonują 10 zadań identycznych; RC porównuje wyniki
5. Council ocenia raport porównawczy; jeśli OK → wniosek cutover_run (D4)
6. Jeśli fail → powrót do SHADOW_ACTIVE i analiza root cause

WF-RUN-04: cutover_run i przełączenie ruchu

1. Council 4/4 zatwierdza cutover_run; Human Gate (D4) aktywowany
2. Operator zatwierdza cutover; stan → CUTOVER_PENDING
3. RC wykonuje stopniowe przełączenie ruchu (traffic split: 10% → 30% → 50% → 100%)
4. RC monitoruje metryki przez 24h; przy anomalii → automatyczny rollback do LEGACY_ACTIVE
5. Po 24h stabilności: target-track jest primary; legacy-track → fallback tylko

6. RC emituje cutover_complete.yaml do Plan 13 (Evidence)

WF-RUN-05: decommission_run

1. RC i Council wnioskuje decommission_run (D5) po potwierdzeniu stabilności
2. External Review (red team lub drugi operator) weryfikuje stan target-track
3. Council 4/4 + Human Gate + External Review zatwierdzają decommission
4. RLS archiwizuje legacy bundle, logi, artefakty (retention 365 dni)
5. RLS zwalnia zasoby VPS legacy-track (Plan 02) i budżet (Plan 10)
6. Stan → LEGACY_DECOMMISSIONED (terminal); Plan 13 rejestruje Evidence finalne

WF-RUN-06: Pause / Resume

Pause: operator wywołuje POST /runs/v1/{id}/pause — run przechodzi do blocked, aktualny etap każdego aktywnego toru zapisuje Checkpoint. Resume: POST /runs/v1/{id}/resume — RLS odtwarza stan z Checkpoint per tor i wznawia od ostatnio zatwierdzonego etapu. W trybie multi-track resume jest synchronizowany: oba tory wznawiają równocześnie.

WF-RUN-07: Cancel

Operator wywołuje POST /runs/v1/{id}/cancel z polem reason. RLS wysyła sygnał STOP do aktywnych aktorów obu torów, zwalnia budżety per tor w Cost Governance (Plan 10), archiwizuje artefakty częściowe per track i ustawia stan cancelled. Zdarzenie emitowane do Matrix (Plan 13) z oznaczeniem track_mode.

API

JSON

```
POST /runs/v1/runs
{
  "trigger_kind": "manual|cron|webhook",
  "operator_id": "string",
  "run_yaml_sha256": "string",
  "track_mode": "single|multi",
  "budget_override": null
}
-> 201 { "run_id": "uuid", "state": "pending", "track_mode": "single|multi" }

POST /runs/v1/{id}/advance
{ "stage_code": "A0..A5|C0..C9", "track_id": "uuid|null",
  "output_sha256": "string", "status": "ok|fail" }
-> 200 { "next_stage": "string", "run_state": "running|blocked|completed",
  "track_state": "LEGACY_ACTIVE|TARGET_BUILDING|SHADOW_ACTIVE|..." }

POST /runs/v1/{id}/tracks
{ "action": "activate_target|promote|rollback|decommission",
  "operator_id": "string", "decision_class": "D3|D4|D5" }
-> 200 { "run_id": "uuid", "track_state": "...", "effective_at": "ISO8601" }

GET /runs/v1/{id}/comparison
-> 200 { "comparisons": [{ "cmp_id": "...", "output_equality_pct": 99.8,
  "performance_delta_pct": 3.2, "error_delta": 0,
  "rc_recommendation": "PROMOTE" }] }
```

```
POST /runs/v1/{id}/cancel
{ "reason": "string", "operator_id": "string" }
-> 200 { "run_id": "uuid", "state": "cancelled" }

GET /runs/v1/{id}/artifacts?track_id=uuid
-> 200 { "artifacts": [{ "artifact_id": "...", "track_id": "...",
                        "kind": "...", "sha256": "...", "size_bytes": 0 }] }
```

HARD GATE'Y			
Kod	Etap	Warunek blokujący	Akcja
G-RUN-01	A0	Trigger bez podpisu ACP lub nieznane trigger_kind	reject — run nie startuje
G-RUN-02	A1	manifest_sha256 niezgodny z Księgą lub Księga w stanie frozen	block — czeka na odblokowanie Księgi
G-RUN-03	A2	Propozycje grupowania niezgodne (0 propozycji po dedup)	fail — brak treści do przetworzenia
G-RUN-04	A4	Budżet tokenów lub koszt USD niewystarczający (rezerwacja nie powiodła się)	block — czeka na uzupełnienie budżetu
G-RUN-05	A5	Chief Safety zwraca HARD_STOP (krytyczna anomalia bezpieczeństwa)	fail — run nie może kontynuować
G-RUN-06	C5	Council nie osiągnęło kworum D2 w ciągu 60 min	block — ponowna sesja lub eskalacja do D3
G-RUN-07	C6	Decyzja D4/D5 niezatwierdzona przez operatora w ciągu 48h	fail — automatyczne zamknięcie runu
G-RUN-08	C8	Deploy zakończony błędami na >1 targecie bundle (per tor)	fail — rollback i alert do Matrix
G-RUN-09	shadow_run	output_equality < 95% w 10 consecutive shadow_run	SHADOW_BLOCK — zatrzymanie promote sequence, eskalacja do Council
G-RUN-10	cutover_run	performance_delta > +30% lub error_delta > 5 per 100 runów w ciągu 24h	ROLLBACK — automatyczny rollback do LEGACY_ACTIVE, alert D4
G-RUN-11	decommission_run	Brak External Review lub External Review zwraca REJECT	fail — decommission odrzucony; stan pozostaje CUTOVER_PENDING
G-RUN-12	dual_run	contract_parity < 99% na target track (testy kontraktowe Plan 12)	DUAL_BLOCK — wymaga naprawy target bundle przed ponownym dual_run
STATUSY I EVIDENCE			

Każdy run wielotorowy generuje rozszerzone statusy Evidence rejestrowane w Plan 13. Poniższa tabela opisuje typy Evidence per typ runa.

Typ runa	Stan wejścia	Artefakty Evidence	Plan 13 event_type
legacy_run	LEGACY_ACTIVE	run_report.json, stage_log.json, budget_summary.json	legacy_run_complete
target_run	TARGET_BUILDING	run_report.json, bundle_test_report.json, contract_test_report.json	target_run_complete
shadow_run	SHADOW_ACTIVE	track_comparison.json, output_equality_log.csv, shadow_run_report.json	multi_track_comparison
dual_run	DUAL_COMPARING	dual_run_comparison.json, council_evaluation.json, promote_recommendation.yaml	dual_run_complete
cutover_run	CUTOVER_PENDING	cutover_execution_log.json, traffic_split_log.csv, post_cutover_stability_report.json, d4_approval.json	cutover_complete
decommission_run	LEGACY_DECOMMISSIONED	legacy_archive_manifest.json, decommission_report.json, external_review_log.md, d5_decision_pack.json	legacy_decommissioned
TESTY			
Kod	Nazwa	Warunki testu	Oczekiwany wynik
T-RUN-01	Tworzenie runu single-track	POST /runs/v1/runs track_mode=single, trigger_kind=manual	201, run_id UUID, state=pending, track_mode=single
T-RUN-02	Tworzenie runu multi-track	POST /runs/v1/runs track_mode=multi, decision_class=D3	201, run_id UUID, 2 TrackInstance: legacy+target, state=LEGACY_ACTIVE
T-RUN-03	Blokada G-RUN-01	POST z trigger_kind=unknown	400, błąd G-RUN-01, run nie utworzony
T-RUN-04	Blokada G-RUN-04	A4 z budget_reserved=0 na target-track	run przechodzi w blocked, event w Matrix per track_id
T-RUN-05	shadow_run — output_equality OK	50 shadow_run, output_equality=99.8%, error_delta=0	RC emituje promote_recommendation=PROMOTE_CANDIDATE
T-RUN-06	shadow_run — G-RUN-09 blokada	10 consecutive shadow_run, output_equality=93%	stan → SHADOW_BLOCK, eskalacja do Council, alert Plan 13

Kod	Nazwa	Warunki testu	Oczekiwany wynik
T-RUN-07	dual_run — Council OK	10 dual_run, Council zatwierdza wyniki target	stan → DUAL_COMPARING → CUTOVER_PENDING, wniosek D4 aktywny
T-RUN-08	cutover_run — stopniowy traffic split	cutover_run D4 zatwierdzone, traffic_split: 10→30→50→100%	ruch przełączany progresywnie, metryki monitorowane per krok
T-RUN-09	cutover_run — automatyczny rollback G-RUN-10	performance_delta=35% w ciągu 4h po cutover	RC wyzwala ROLLBACK, stan → LEGACY_ACTIVE, alert D4 do operatora
T-RUN-10	decommission_run — brak External Review	decommission_run bez external_review_log	fail G-RUN-11, decommission odrzucony, stan CUTOVER_PENDING zachowany
T-RUN-11	decommission_run — pełny sukces	D5 zatwierdzone, External Review OK, archiwizacja legacy	stan → LEGACY_DECOMMISSIONED (terminal), Plan 13 Evidence kompletne
T-RUN-12	Cancel multi-track	POST /cancel w trakcie SHADOW_ACTIVE	oba tory cancelled, budżety zwolnione per track, artefakty archiwizowane

TRZY ŚCIEŻKI — jak Plan 11 wspiera Evolution / Refactor / Rebuild

Ścieżka	Tryb runa	Typy aktywnych runów	Klasa decyzji
evolution	single-track	legacy_run	D0-D2
refactor	multi-track (shadow → dual → cutover)	legacy_run + target_run + shadow_run + dual_run + cutover_run	D3 (start) → D4 (cutover) → D5 (decommission)
rebuild	multi-track (greenfield → shadow → cutover → decommission)	Wszystkie 6 typów, Plan 14 orkiestruje sekwencję	D4 (moduł) lub D5 (system)

REBUILD · Integracja z Planem 14 (Greenfield Rebuild Orchestration)

W ścieżce rebuild Plan 14 jest nadrzędnym orkiestratorem: definiuje target architecture, wyzwala tworzenie target track w Planie 11, koordynuje 7 działań pipeline (Plan 15) podczas budowania target bundle i przekazuje Run Coordinatorowi strategię cutover. Plan 11 jest warstwą wykonawczą rebuild — Plan 14 jest jej architektem.

ANEKS A — Pełna tabela etapów A0-A5 i Core 0-9 (multi-track)

<i>Etap</i>	<i>Nazwa</i>	<i>Aktor (rola)</i>	<i>Track</i>	<i>Gate'y wejściowe</i>	<i>Gate'y wyjściowe</i>	<i>Timeout</i>
A0	Trigger Ingestion	ACP (Plan 01)	shared	G-RUN-01	trigger_norm OK	120s
A1	Context Canonicalization	Chief Context	shared	G-RUN-02	CanonicContext OK	600s
A2	Proposal Grouping	Chief Reasoning	shared	G-RUN-03	groups>0	900s
A3	Provider Allocation	Infra (Plan 02)	per tor	—	provider_per_track	300s
A4	Budget Reservation	Cost Gov (Plan 10)	per tor	G-RUN-04	budget_reserved=true	60s
A5	Gate Pre-Check	Chief Safety	per tor	G-RUN-05	safety=PASS	120s
C0	Bootstrap	ACP	per tor	—	env_ready	120s
C1	Context Load	Chief Context	per tor	—	ctx_loaded	600s
C2	Analysis	Chief Reasoning	per tor	—	analysis_report	1800s
C3	Red Team	Chief Review	per tor	—	red_findings	1200s
C4	Blue Team	Chief Safety	per tor	—	blue_response	1200s
C5	Council Decide	Board 4/4	shared	G-RUN-06	decision D0-D5	3600s
C6	Human Gate	Operator SYLION	shared	G-RUN-07	D4/D5 approved	172800s
C7	Bundle Build	Plan 12	per tor	G-RUN-12	bundle_manifest	1800s
C8	Deploy	Plan 02	per tor	G-RUN-08	deploy OK	2700s
C9	Observe & Feedback	Run Coordinator	shared	—	feedback_sent	ongoing
ANEKS B — Diagram przepływu stanów multi-track						
<i>Stan</i>	<i>Typ runa</i>	<i>Przejście (sukces)</i>		<i>Przejście (fail/blokada)</i>		
pending	—	running		cancelled		
running	legacy_run	LEGACY_ACTIVE		blocked / failed		
LEGACY_ACTIVE	legacy_run	TARGET_BUILDING (D3 trigger)		completed (single) / cancelled		

Stan	Typ runa	Przejście (sukces)	Przejście (fail/blokada)
TARGET_BUILDING	target_run	SHADOW_ACTIVE	LEGACY_ACTIVE (rollback) / failed
SHADOW_ACTIVE	shadow_run	DUAL_COMPARING (G-RUN-09 OK)	LEGACY_ACTIVE (G-RUN-09 fail)
DUAL_COMPARING	dual_run	CUTOVER_PENDING (D4 wniosek)	SHADOW_ACTIVE (retry)
CUTOVER_PENDING	cutover_run	LEGACY_DECOMMISSIONED (D5)	LEGACY_ACTIVE (G-RUN-10 rollback)
LEGACY_DECOMMISSIONED	decommission_run	(terminal)	(terminal)
blocked	dowolny	running (po odblokowaniu)	failed / cancelled
completed	dowolny	(terminal)	(terminal)
failed	dowolny	(terminal)	(terminal)
cancelled	dowolny	(terminal)	(terminal)

PLAN 12 · WARSTWA KANONICZNA

*Bundle Generation & Testing Plane**REBUILD · Rebuildability over Lineage w tym planie*

Plan 12 obsługuje bundlowanie i testowanie na wszystkich trzech ścieżkach: Evolution (standardowy build i deploy), Refactor (bundle równoległy: legacy + target równocześnie) oraz Rebuild (greenfield bundle — nowy kontrakt artefaktu, weryfikacja od zera przez Core 10). W v3.0 dodane zostają Red Team Bundle i Blue Team Bundle jako pierwsze klasy systemu, a Gate G-BUN-ADV-01 blokuje promocję bundla bez pokrycia adversarialnego na poziomie 80 % (D3) lub 95 % (D4, scenariusze Critical). Odnosniki: Plan 07 (Evolution Safety adversarial), Plan 14 (rebuild bundles), Plan 15 (Red Team i Blue Team jako stałe działy).

12.1. Cel i umiejscowienie

Bundle Generation & Testing Plane odpowiada za produkcję, podpisywanie, testowanie i wdrażanie pakietów (bundli) generowanych dla 6 celów deploymentu systemu SYLION: Pixel, Router, G1 (Gateway 1), G2 (Gateway 2), VPS i Matrix. Plan 12 konsumuje decyzje Council (D2/D3/D4/D5) z Planu 05 oraz gotowy kontekst z Core 7 (Plan 11) i dostarcza podpisane bundle do planu deploymentu (Plan 02). Każdy bundle jest niemutowalny po podpisaniu i archiwizowany w Planie 13 jako dowód. W v3.0 Plan 12 rozszerzono o dwa nowe typy bundli bezpieczeństwa: Red Team Bundle (scenariusze ataku) i Blue Team Bundle (reguły detekcji i playbooki SOC). Nowa encja Adversarial Contract wiąże każdy atak RT z odpowiadającą detekcją BT.

- 6 celów deploymentu z odmiennymi kontraktami formatów i runtime'ów
- Red Team Bundle + Blue Team Bundle — nowe typy bundli v3.0
- Adversarial Contract — umowa wiążąca ataki RT z detekcjami BT
- Gate G-BUN-ADV-01 — coverage $\geq 80\%$ dla D3, $\geq 95\%$ dla D4 (Critical)
- Pre-stages A0–A5 w trybie legacy, target i adversarialnym
- Core 10 — Rebuild Contract Validator (nowy, greenfield check)
- Testy regresji przed każdą promocją do produkcji
- Canary deployment: 5 % ruchu, monitorowanie 15 min, rollback automatyczny

12.2. Charakterystyka 6 targetów

Każdy target ma specyficzny format, runtime i ograniczenia. Naruszenie kontraktu targetu jest traktowane jako G-BND gate failure. W v3.0 każdy target posiada pole `implementation_strategy` determinujące ścieżkę budowania bundla.

12.2.1. Pixel

Target frontend: interfejs operatora SYLION. Format: statyczny SPA (React/Vite). Runtime: przeglądarka. Rozmiar max: 5 MB (gzip). Deploy path: `/var/www/pixel/`. Sygnatura Ed25519 oraz hash SHA-256 na manifest. SLA: dostępność 99,5 %, aktualizacja w ciągu 10 min od

promocji. `implementation_strategy`: reuse (evolution path).

12.2.2. Router

Target sieciowy: reverse proxy + load balancer. Format: kontener OCI (Alpine). Runtime: Podman rootless. Rozmiar max: 200 MB. Deploy path: `/opt/sylion/router/`. Sygnatura: cosign + Ed25519. SLA: zero-downtime deploy przez rolling update, max 30 s przerwa na instancję. `implementation_strategy`: refactor (równoległy build shadow → cutover).

12.2.3. G1 (Gateway 1)

Target bramki wejściowej: autoryzacja, rate limiting, uwierzytelnianie agentów. Format: binary Go (statyczny, `CGO_ENABLED=0`). Runtime: systemd unit. Rozmiar max: 50 MB. Deploy path: `/usr/local/bin/sylion-g1`. Sygnatura: GPG + SHA-256 checksum. SLA: 99,9 % dostępność, <50 ms p99. `implementation_strategy`: reuse.

12.2.4. G2 (Gateway 2)

Target bramki wewnętrznej: routing inter-agent, Council messaging bus. Format: Python wheel (zip, no binary deps). Runtime: Python 3.12 + venv. Rozmiar max: 30 MB. Deploy path: `/opt/sylion/g2/`. Sygnatura: Ed25519 + pip hash. SLA: 99,8 % dostępność, <100 ms p99. `implementation_strategy`: refactor.

12.2.5. VPS

Target infrastrukturalny: pakiet konfiguracji i skryptów VPS (Ansible + Terraform). Format: tar.gz (Ansible role + HCL). Runtime: runner Ansible/Terraform. Rozmiar max: 20 MB. Deploy path: `/etc/sylion/infra/`. Sygnatura: GPG. SLA: idempotentność, max 20 min na pełny apply. `implementation_strategy`: reuse.

12.2.6. Matrix

Target serwera Matrix (Synapse): konfiguracja, moduł Python, homeserver.yaml. Format: Python wheel + YAML config. Runtime: Synapse + Python 3.11. Rozmiar max: 15 MB. Deploy path: `/etc/matrix-synapse/sylion/`. Sygnatura: Ed25519. SLA: 99,5 % dostępność, zdarzenia audytowe max 5 s lag. `implementation_strategy`: reuse.

12.3. Nowe typy bundli v3.0 — Red Team Bundle i Blue Team Bundle

W v3.0 Plan 12 wprowadza dwa nowe typy kontraktów bundle'owych, obok istniejących bundli deploymentowych dla targetów Pixel/Router/G1/G2/VPS/Matrix. Red Team Bundle definiuje scenariusze ataku PRZED promocją bundla deploymentowego. Blue Team Bundle definiuje reguły detekcji i playbooki SOC PO zobaczeniu Red Team Bundle. Oba bundle są niemutowalne po podpisaniu i archiwizowane w Planie 13.

12.3.1. Red Team Bundle — *redteam_bundle.yaml*

Red Team Bundle to kompletny opis scenariuszy ataku na promowany bundle deploymentowy. Faza (pre-stage / bundle / post-stage) wskazuje moment wykonania ataku. Severity determinuje klasę decyzji: high/critical → block; medium → wymagany przegląd D3. Każdy scenariusz musi być powiązany z odpowiadającą regułą Blue Team (Adversarial Contract).

YAML

```
redteam_bundle:\n  version: "1.0"\n  bundle_id: "rt-bnd-001"\n  target_architecture: "bn\n  d-pixel-001"\n  created_by: "rt-agent-01"\n  created_at: "2026-04-15T09:00:00Z"\n  signed_by: "rt-key-ed25519-01"\n  attack_scenarios:\n    - id: RT-001\n      name: "Secrets Vault bypass attempt"\n      phase: pre-stage\n      severity: critical\n      target_modules: [vault, acp]\n      attack_vectors:\n        - "JWT token forging via weak ECDSA key"\n        - "Vault secret enumeration via timing side-channel"\n        - "ACP role escalation via stale refresh token"\n      success_criteria:\n        - "Unauthorized secret read returns HTTP 200"\n        - "Role escalated without 2PR confirmation"\n      expected_blue_response:\n        - "BL-001: Vault access anomaly detection within 30s"\n        - "BL-002: ACP role change alert emitted to SOC"\n      blue_rule_id: BL-001\n    - id: RT-002\n      name: "Bundle signature bypass"\n      phase: bundle\n      severity: high\n      target_modules: [bundle_service, signing_key]\n      attack_vectors:\n        - "Ed25519 key substitution via compromised build runner"\n        - "SBOM injection with malicious dependency"\n      success_criteria:\n        - "Unsigned bundle accepted by deploy pipeline"\n        - "Malicious SBOM entry not flagged by Blue Team"\n      expected_blue_response:\n        - "BL-003: Signature verification failure alert"\n      blue_rule_id: BL-003\n    - id: RT-003\n      name: "Canary traffic poisoning"\n      phase: post-stage\n      severity: medium\n      target_modules: [canary_deployer, router]\n      attack_vectors:\n        - "Traffic manipulation during canary window"\n        - "Metric spoofing to force false canary success"\n      success_criteria:\n        - "Malicious bundle promoted via poisoned canary metrics"\n      expected_blue_response:\n        - "BL-004: Canary metric anomaly detection"\n      blue_rule_id: BL-004\n  gating_rules:\n    - rule: "if attack succeeds AND severity >= high: block promotion"\n    - rule: "if attack succeeds AND severity = medium: require D3 review"\n    - rule: "if blue_rule_id missing for any scenario: block (Adversarial Contract violated)"
```

12.3.2. Blue Team Bundle — *blueteam_bundle.yaml*

Blue Team Bundle projektowany jest przez Dział Blue Team (Plan 15) PO zobaczeniu Red Team Bundle. Każda reguła detekcji musi odpowiadać co najmniej jednemu scenariuszowi RT. Playbook SOC definiuje kroki odpowiedzi i eskalację. Procedury DFIR opisują postępowanie po incydencie. Gate blokuje promocję przy coverage < 80 % lub braku reguły dla scenariusza Critical.

YAML

```
blueteam_bundle:\n  version: "1.0"\n  bundle_id: "bt-bnd-001"\n  redteam_bundle_ref: "rt-bnd-001"\n  created_by: "bt-agent-01"\n  created_at: "2026-04-15T11:00:00Z"\n  signed_by: "bt-key-ed25519-01"\n  detection_rules:\n    - id: BL-001\n      name: "Vault access anomaly"\n      covers_rt: RT-001\n      telemetry_source: [vault_audit_log, acp_audit_log, prometheus]\n      rule_type: threshold\n      severity: critical\n      threshold:\n        metric: vault_unauthorized_reads_per_min\n        value: 3\n        window_s: 60\n      response_playbook: BL-PB-001\n    - id: BL-002\n      name: "ACP role escalation anomaly"\n      covers_rt: RT-001\n      telemetry_source: [acp_audit_log]\n      rule_type: pattern\n      severity: high\n      pattern: "role_change without 2pr_approve within 60s"\n      response_playbook: BL-PB-001\n    - id: BL-003\n      name: "Bundle signature verification failure"\n      covers_rt: RT-002\n      telemetry_source: [bundle_service_log, signing_service_log]\n      rule_type: pattern\n      severity: high\n      pattern: "signature_verify_fail OR sbom_hash_mismatch"\n      response_playbook: BL-PB-002\n    - id: BL-004\n      name: "Canary metric anomaly"\n      covers_rt: RT-003\n      telemetry_source: [prometheus, canary_deployer_log]\n      rule_type: ml\n      severity: medium\n      model: canary_anomaly_detector_v1\n      response_playbook: BL-PB-003\n  soc_playbooks:\n    - id: BL-PB-001\n      name: "Vault/ACP Critical Incident"\n      triggers: [BL-001, BL-002]\n      response_steps:\n        - "1. Zablokuj wszystkie sesje agentów w run_id (ACP /mode/switch LOCKDOWN)"\n        - "2. Powiadom SecOfficer (R-01) via !syllion-soc"\n        - "3. Uruchom Emergency Seal (Plan 04)"\n        - "4. Zbierz AuditLog z ostatnich 60 min"\n        - "5. Przekaż do DFIR-001"\n      escalation: D4 Human Gate (Chief Safety + operator)\n    - id: BL-PB-002\n      name: "Bundle Signing Compromise"\n      triggers: [BL-003]\n      response_steps:\n        - "1. Odrzuć bundle (G-BND-01)"\n        - "2. Rotuj klucze podpisujące (Plan 04 Vault rotation)"\n        - "3. Uruchom pełny re-scan SBOM wszystkich promoted bundli"\n        - "4. Zgłoś do Governance (Plan 13)"\n      escalation: D3 Council 4/4\n    - id: BL-PB-003\n      name: "Canary Metric Tampering"\n      triggers: [BL-004]\n      response_steps:\n        - "1. Zatrzymaj canary (auto-rollback G-BND-06)"\n        - "2. Wyizoluj instancję canary"\n        - "3. Porównaj metryki z baseline z Plan 11"\n        - "4. Eskaluj do Chief Review"\n      escalation: D2 Standard Review\n      dfir_procedures:\n        - id: DFIR-001\n          name: "Vault/ACP Post-Incident Analysis"\n          steps:\n            - "Preservation: snapshot stanu Vault i ACP AuditLog"\n            - "Identification: hash chain verification (Plan 13)"\n            - "Analysis: timeline reconstruction"\n            - "Eradication: revoke wszystkich tokenów z incydentu"\n            - "Recovery: restore z last-known-good snapshot"\n            - "Lessons learned: raport do Governance (Plan 13)"\n      gating_rules:\n        - rule: "if detection_coverage < 80%: block promotion"\n        - rule: "if critical_rule_missing: D4 review (Gate G-BUN-ADV-01)"\n        - rule: "if soc_playbook_missing for severity >= high: block promotion"
```

12.4. Adversarial Contract — interakcja Red × Blue

Adversarial Contract to formalna umowa między Działem Red Team a Działem Blue Team (Plan 15), zawarta na poziomie każdego bundla deploymentowego. Zasada: każdy atak zdefiniowany przez Red Team MUSI mieć odpowiadającą regułę detekcji w Blue Team Bundle. Brak pokrycia powoduje gate block. Coverage tracking mierzy procent ataków RT posiadających co najmniej jedną odpowiadającą regułę BT.

Atak RT	Severity	Reguła BT	Pokrycie	Status	Gate przy braku
RT-001: Vault bypass	Critical	BL-001, BL-002	100 %	OK	G-BUN-ADV-01 block
RT-002: Signature bypass	High	BL-003	100 %	OK	G-BUN-ADV-01 block
RT-003: Canary poisoning	Medium	BL-004	100 %	OK	D3 review

Atak RT	Severity	Reguła BT	Pokrycie	Status	Gate przy braku
RT-XXX: brak pokrycia	Any	—	0 %	BLOCK	G-BUN-ADV-01 block zawsze

KRYTYCZNE · Gate G-BUN-ADV-01 — Adversarial Coverage Gate

Bundle deploymentowy NIE PRZECHODZI promocji bez spełnienia progu coverage: 80 % ataków z odpowiadającymi detekcjami (D3), 95 % dla bundli zawierających scenariusze o severity=Critical (D4). Reguła bezwzględna — nie podlega wyjątkom bez decyzji D5 zatwierdzonej przez Council 4/4 + Human + External Review. Tracking automatyczny przez Coverage Audit Agent (Plan 15 Red/Blue integration).

12.4.1. Workflow Adversarial Contract

1. Red Team definiuje atak scenarios PRZED budowaniem bundla deploymentowego (faza A-ADV-01)
2. Red Team Bundle jest podpisywany i przekazywany do Blue Team (faza A-ADV-02)
3. Blue Team projektuje detection rules i SOC playbooks PO zobaczeniu red bundle (faza A-ADV-03)
4. Coverage Audit — automat sprawdza procent ataków RT z odpowiadającymi detekcjami BT
5. Adversarial Test Run — Red Team wykonuje ataki; Blue Team weryfikuje czy detekcje działają
6. Gate G-BUN-ADV-01 — bundle nie przechodzi bez coverage ≥ 80 % (D3) albo 95 % (D4 Critical)
7. Wynik archiwizowany w Planie 13 jako evidence adversarial dla D3/D4/D5

12.5. Pre-stages A0–A5 — trzy tryby

Pre-stages to etapy wstępne wykonywane przed głównym procesem bundle build. W v3.0 każdy pre-stage istnieje w trzech wariantach: Legacy Track (z v2.0, standardowy), Target Track (tryb Rebuild — nowa linia docelowa) oraz Adversarial Track (nowe w v3.0 — Red Team przed integracją). Wszystkie 18 wariantów (6 stagów × 3 tryby) jest archiwizowanych w Planie 13.

12.5.1. Legacy Track — A0–A5

Sta ge	Nazwa	Cel	Warunek przejścia	Klas a D
A0	Canon Compact Check	Weryfikacja że canon_compact.md (Plan 16) jest aktualny (<24h)	Compact timestamp < 24h; brak wiszących D3+ bez rozstrzygnięcia	D0
A1	Run Context Validation	Weryfikacja run_id, decision_sha256 i kontekstu z Plan 11 (Core 7)	run_id aktywny; decision_class >= D2; context_sha256 zweryfikowany	D1

Stage	Nazwa	Cel	Warunek przejścia	Klasa D
A2	Dependency Freeze	Snapshot wszystkich zależności targetu (SBOM, lockfile)	Wszystkie zależności zamrożone; brak UNKNOWN zależności	D1
A3	Vault Key Check	Sprawdzenie dostępności kluczy podpisujących w Vault (Plan 04)	Klucze aktywne, nie rotowane w trakcie build, żadne nie wygasłe	D2
A4	Gate Pre-Check	Weryfikacja że poprzedni bundle danego targetu zakończył się sukcesem lub jest rolled_back	Brak zawieszonych canary dla targetu; poprzedni state ≠ building	D2
A5	Adversarial Pre-Gate	Sprawdzenie czy Red Team Bundle dla tego bundla jest podpisany i dostępny	rt-bnd podpisany; coverage >= 0 % (trigger dla A-ADV-01 jeśli = 0)	D3

12.5.2. Target Track — A0–A5 (tryb Rebuild)

W trybie Rebuild (Plan 14 SYSTEM_REBUILD_MODE) pre-stages Target Track weryfikują nową linię docelową niezależnie od linii legacy. Oba tory są wykonywane równolegle podczas fazy cutover, a wyniki porównywane przez Rebuild Contract Validator (Core 10).

Stage	Nazwa (Target)	Cel w trybie Rebuild	Warunek przejścia	Klasa D
A0-RB	Target Canon Baseline Check	Weryfikacja że target_architecture_delta.json (Plan 14) jest aktualny	Delta zaakceptowana przez D4; target_baseline zdefiniowany	D3
A1-RB	Target Run Context	Weryfikacja run_id w trybie REBUILD_MODE; oddzielny kontekst docelowy	rebuild_run_id aktywny; cutover_strategy zdefiniowana	D3
A2-RB	Target Dependency Freeze	Snapshot zależności nowego targetu greenfield	Greenfield SBOM gotowy; brak dziedziczonych unknown z legacy	D3
A3-RB	Target Vault Key Provisioning	Generowanie lub import kluczy dla nowej architektury	Target keys provisioned w Vault; rotation_plan zaakceptowany	D4
A4-RB	Parallel Track Gate	Weryfikacja że legacy i target track mogą działać równolegle bez konfliktu	Brak konfliktu portów/ścieżek; shadow routing aktywny	D4
A5-RB	Greenfield Adversarial Pre-Gate	RT Bundle dla target architecture gotowy; BT Bundle zaprojektowany dla nowej powierzchni ataku	rt-bnd-target podpisany; coverage >= 80 % przed D5 promotion	D5

12.5.3. Adversarial Track — A-ADV-01..03

Adversarial pre-stages to trzy etapy pracy Red Team PRZED integracją bundla deploymentowego. Wykonywane niezależnie od legacy i target track — ich wyniki są wejściem

dla Adversarial Contract (sekcja 12.4).

Stag e	Nazwa	Cel	Wykonawca	Warunek przejścia
A-AD V-01	Red Team Threat Model Review	Red Team analizuje architekturę bundla i identyfikuje powierzchnię ataku przed budowaniem	Dział Red Team (Plan 15)	Threat model zaakceptowany; attack_scenarios >= 1 per severity level
A-AD V-02	Red Team Bundle Build & Sign	Red Team buduje i podpisuje redteam_bundle.yaml z kompletnymi scenariuszami	Dział Red Team (Plan 15)	redteam_bundle podpisany; gating_rules kompletne; bundle_id zarejestrowany w Plan 13
A-AD V-03	Blue Team Coverage Design	Blue Team projektuje detection_rules i SOC playbooks na podstawie Red Team Bundle	Dział Blue Team (Plan 15)	blueteam_bundle podpisany; coverage_pct >= 80 %; wszystkie Critical scenariusze pokryte

12.6. Core 0–10 — pipeline stages bundla

Core stages 0–9 to główne etapy pipelineu bundle (z v2.0, rozszerzone o tryb Rebuild). Core 10 to nowy etap v3.0: Rebuild Contract Validator, weryfikujący że bundle docelowy spełnia wszystkie inwarianty Księgi nawet w architekturze greenfield.

C or e	Nazwa	Opis	Tryb Rebuild	Gate
0	Init & Auth	Inicjalizacja bundle session; autoryzacja run_id przez ACP (Plan 01)	Weryfikacja rebuild_run_id; cutover_strategy check	G-BND-00
1	Pre-Stage Execution	Wykonanie A0–A5 (legacy lub target track + adversarial)	A0-RB...A5-RB równoległe z A0...A5 legacy	A-stagów pass
2	Dependency Resolution	Rozwiązanie zależności; generowanie SBOM; lockfile freeze	Greenfield SBOM niezależny od legacy	G-BND-03
3	Artifact Build	Budowanie artefaktów wg kontraktu targetu (format, rozmiar, entry point)	Target artifact build w greenfield runtime	G-BND-02
4	Sign	Podpisywanie bundla kluczami z Vault (Plan 04); double-check	Nowe klucze target architecture	G-BND-01
5	Red/Blue Adversarial Run	Wykonanie A-ADV-01..03; Red Team atakuje; Blue Team weryfikuje detekcje	Pełny run na target architecture pre-canary	G-BUN-ADV-01
6	Test Suite	Testy kontraktowe, jednostkowe, integracyjne, security (per target)	Greenfield test suite; brak założeń legacy	G-BND-04, G-BND-05
7	Regression Report	Porównanie z poprzednią wersją; RegressionReport z critical_count	Porównanie legacy vs target (Core 10 input)	G-BND-04

C or e	Nazwa	Opis	Tryb Rebuild	Gate
8	Canary Deploy	5 % ruchu; 15 min monitoring; auto-rollback przy przekroczeniu progów	Shadow canary dla target architecture	G-BND-06, G-BND-07
9	Promote / Archive	Promocja do 100 % ruchu; archiwizacja w Plan 13; emitowanie zdarzenia do Matrix	Cutover switch (Plan 14); legacy decommission trigger	G-BND-08
10	Rebuild Contract Validator	NOWY v3.0: weryfikacja że target bundle spełnia inwarianty Księgi nawet w greenfield. Patrz sekcja 12.7.	Tylko w trybie Rebuild (target track)	G-BND-RB-0 1

12.7. Core 10 — Rebuild Contract Validator

Core 10 to nowy etap v3.0 aktywowany wyłącznie w trybie Rebuild (target track, po aktywacji SYSTEM_REBUILD_MODE w Planie 14). Weryfikuje, że nowo zbudowany bundle docelowy (greenfield) spełnia wszystkie inwarianty zdefiniowane w Księdze, nawet jeśli nie dziedziczy żadnego kodu z linii MZ 3.5.5. Jest ostatnią bramą przed canary deployment w trybie Rebuild.

- Sprawdza zgodność kontraktu targetu z Księgą (Plan 08 canon hash chain)
- Weryfikuje że wszystkie D3+ decyzje dotyczące target architecture zostały zarejestrowane w Plan 13
- Porównuje inwarianty bezpieczeństwa legacy vs target (diff — brak regresji)
- Weryfikuje że cutover_strategy jest zatwierdzona przez D4 (Plan 14)
- Generuje Rebuild Validation Report (RVR) archiwizowany w Plan 13
- Blokuje promocję (G-BND-RB-01) jeśli jakikolwiek inwariant Księgi jest naruszony

REBUILD · Core 10 — inwarianty Księgi w architekturze greenfield

Core 10 implementuje zasadę „Rebuildability over Lineage” w praktyce: nawet jeśli bundle jest pisany od zera (greenfield), MUSI spełniać wszystkie inwarianty Księgi zdefiniowane przed buildem. Żadna implementacja nie może zastąpić intencji Księgi — tylko D5 (Council 4/4 + Human + External Review) może zmienić same inwarianty. Core 10 jest odpowiedzialny za to by zasada ta była egzekwowana technicznie przed każdą promocją bundla Rebuild.

12.8. Red Team vs Blue Team vs Purple Team (v2.0 + v3.0)

Każdy bundle przechodzi przez tri-fazowy przegląd bezpieczeństwa. W v3.0 Red Team i Blue Team są stałymi działami pipeline (Plan 15). Purple Team (synteza) generuje raport zbiorczy i decyzję go/no-go.

Zespół	Rola v3.0	Model	Odpowiedzialność	Blokuje
Red Team	Atak + RT Bundle	Chief Review (GPT-5)	Budowanie RT Bundle; wykonywanie attack scenarios; testy pre-stage A-ADV-01..02	Tak — brak RT Bundle = block

Zespół	Rola v3.0	Model	Odpowiedzialność	Blokuje
Blue Team	Detekcja + BT Bundle	Chief Safety (rotacyjny)	Projektowanie detection rules; SOC playbooks; DFIR procedures; BT Bundle; Coverage tracking	Tak — coverage < 80 % = block
Purple Team	Synteza + go/no-go	Chief Reasoning (Claude)	Łączy wyniki RT+BT; priorytetyzacja luk; rekomendacje; Adversarial Report; decyzja go/no-go	Tak — no-go blokuje promocję

12.9. Trzy ścieżki — Evolution / Refactor / Rebuild

Plan 12 obsługuje wszystkie trzy ścieżki transformacji (v3_canon_brief sekcja 3). Wybór ścieżki determinuje wariant pre-stages (legacy / target / adversarial), aktywację Core 10 i klasę decyzji wymagającą do promocji.

Ścieżka	Pre-stages	Core 10	Klasa D promocji	Uwagi
Evolution (reuse)	A0–A5 Legacy	Nie	D2	Standardowy build i deploy; brak zmian kontraktu
Refactor (contract-preserving)	A0–A5 Legacy + A0–RB–A2–RB (partial)	Opcjonalnie	D3	Shadow deploy; dwa bundle równolegle; cutover stopniowy
Rebuild (greenfield)	A0–A5 Legacy + A0–RB–A5–RB + A–ADV-01..03	Tak (obowiązkowo)	D4/D5	Pełny RT+BT Bundle; Core 10; Adversarial Contract z coverage ≥ 95 %

Ścieżki inline:

- Evolution — reuse-first, D2, A0–A5 legacy, brak Core 10
- Refactor — contract-preserving, D3, shadow mode, Core 10 opcjonalny
- Rebuild — greenfield, D4/D5, pełny adversarial, Core 10 obowiązkowy

12.10. Canary deployment i rollback

Canary: 5 % ruchu kierowane na nową wersję przez 15 minut. Metryki monitorowane: error rate (próg max 1 %), p99 latency (próg max 2× baseline), dostępność (min 99 %). Przy przekroczeniu progu: automatyczny rollback do poprzedniej wersji w ciągu 60 s. Przy sukcesie canary: promocja full (100 % ruchu). W trybie Rebuild canary jest wykonywane na target architecture w trybie shadow zanim ruch zostanie przekierowany z legacy.

PLAN 12 · SPECYFIKACJA WYKONAWCZA

Bundle Generation & Testing Plane

CEL I ZAKRES

Specyfikacja Bundle Service (BS) v3.0. Zakres: budowanie, podpisywanie, testowanie i wdrażanie bundli dla 6 targetów (Pixel/Router/G1/G2/VPS/Matrix), Red Team Bundle, Blue Team Bundle, Adversarial Contract, pre-stages A0–A5 (legacy/target/adversarial), Core 0–10 z Rebuild Contract Validator. BS operuje na podstawie decyzji z Planu 05 (D2/D3/D4/D5) i kontekstu z Planu 11 (Core 7). Wszystkie bundle są niemutowalne po podpisaniu. Wersja: Pipeline 8.0.0, MZ 3.5.5.

SCHEMATY YAML — moduły Bundle Service v3.0

Bundle Service Module

YAML

```
apiVersion: sylion.io/v3\nkind: Module\nmetadata:\n  module_id: "bundle-service-core"\n  name: "Bundle Service Core"\n  plan: "12"\nspec:\n  implementation_strategy: refactor\n  current_baseline: MZ 3.5.5\n  target_baseline: "bundle-service-v3"\n  cutover_strategy: shadow\n  version: "3.0.0"\n  features:\n    - redteam_bundle_support: true\n    - blueteam_bundle_support: true\n    - adversarial_contract: true\n    - core_10_rebuild_validator: true\n    - pre_stages_rebuild_track: true
```

Red Team Bundle Module

YAML

```
apiVersion: sylion.io/v3\nkind: Module\nmetadata:\n  module_id: "bundle-red-team"\n  name: "Red Team Bundle Service"\n  plan: "12"\nspec:\n  implementation_strategy: greenfield\n  current_baseline: null\n  target_baseline: "bundle-red-team-v1"\n  cutover_strategy: cut-and-replace\n  version: "1.0.0"\n  owner_dept: red_team\n  signing_key: rt-key-ed25519-01\n  required_for_promotion: true
```

Blue Team Bundle Module

YAML

```
apiVersion: sylion.io/v3\nkind: Module\nmetadata:\n  module_id: "bundle-blue-team"\n  name: "Blue Team Bundle Service"\n  plan: "12"\nspec:\n  implementation_strategy: greenfield\n  current_baseline: null\n  target_baseline: "bundle-blue-team-v1"\n  cutover_strategy: cut-and-replace\n  version: "1.0.0"\n  owner_dept: blue_team\n  signing_key: bt-key-ed25519-01\n  coverage_threshold_d3: 80\n  coverage_threshold_d4_critical: 95\n  required_for_promotion: true
```

Core 10 — Rebuild Contract Validator Module

YAML

```
apiVersion: sylion.io/v3\nkind: Module\nmetadata:\n  module_id: "core-10-rebuild-validator"\n  name: "Rebuild Contract Validator"\n  plan: "12"\nspec:\n  implementation_strategy: greenfield\n  current_baseline: null\n  target_baseline: "core10-v1"\n  cutover_strategy: cut-and-replace\n  version: "1.0.0"\n  activation: SYSTEM_REBUILD_MODE\n  canon_source: "canon_compact.md"\n  invariants_check: true\n  d3_decisions_check: true\n  cutover_strategy_check: true\n  output: rebuild_validation_report.json
```

MODEL DOMENOWY		
Encja	Pola kluczowe	Uwagi
Bundle	bundle_id (UUID), run_id, target, version, state, manifest_sha256, signed_at, promoted_at, rebuild_track (bool)	state: building/signed/testing/canary/promoted/rolled_back/failed; rebuild_track=true dla target architecture
RedTeamBundle	rt_bundle_id, bundle_ref_id, attack_scenarios[], gating_rules[], signed_by, signed_at, coverage_pct	Niemutowalny po podpisaniu; jeden per bundle deploymentowy
BlueTeamBundle	bt_bundle_id, rt_bundle_ref, detection_rules[], soc_playbooks[], dfir_procedures[], coverage_pct, signed_by	Tworzony PO RedTeamBundle; coverage_pct obliczany automatycznie
AdversarialContract	contract_id, bundle_id, rt_bundle_id, bt_bundle_id, coverage_pct, status (open/covered/blocked), generated_at	Encja łącząca RT i BT bundle; status=blocked jeśli coverage < threshold
AttackScenario	scenario_id, rt_bundle_id, rt_scenario_id (RT-001...), name, phase, severity, target_modules[], attack_vectors[], blue_rule_id	Jeden rekord na scenariusz RT; blue_rule_id FK do DetectionRule
DetectionRule	rule_id, bt_bundle_id, bl_rule_id (BL-001...), name, covers_rt, telemetry_source[], rule_type, severity, response_playbook	rule_type: threshold/pattern/ml
CoverageAudit	audit_id, contract_id, total_attacks, covered_attacks, coverage_pct, missing_coverage[], status, run_at	Generowany automatycznie przez Coverage Audit Agent
RebuildValidationReport	rvr_id, bundle_id, canon_hash, invariants_checked, invariants_failed[], d3_decisions_ok, cutover_ok, generated_at	Output Core 10; archiwizowany w Plan 13
Target	target_id, name, format, runtime, max_size_bytes, deploy_path, sla, implementation_strategy	6 rekordów konfiguracyjnych; niemutowalne
Artifact	artifact_id, bundle_id, kind, path, sha256, size_bytes	kind: binary/config/manifest/sbom/report
Manifest	manifest_id, bundle_id, target, version, artifacts[], build_ts, builder_id	YAML; hash chain z Plan 13
Signature	sig_id, bundle_id, algorithm, pubkey_id, signature_b64, signed_at	Jeden lub więcej per bundle
TestSuite	suite_id, bundle_id, target, kind, run_at, state	kind: contract/unit/integration/security
CanaryDeployment	canary_id, bundle_id, target, traffic_pct, started_at, finished_at, promoted, metrics_snapshot	traffic_pct domyślnie 5
RegressionReport	report_id, bundle_id, prev_bundle_id, regressions_count, critical_count, generated_at, sha256	Porównanie z poprzednią wersją

PEŁNA TABELA KONTRAKTÓW BUNDLE — 6 targetów								
Targ et	Format	Entry Point	Sygnatur a	Runtime	Deploy Path	M ax M B	SLA	impl_st rategy
Pixel	SPA tar.gz	index.html	Ed25519+ SHA-256	Przegląd arka	/var/www/pixel/	5	99,5 %, <10 min	reuse
Rout er	OCI Alpine	entrypoint .sh	cosign+E d25519	Podman rootless	/opt/sylian/route r/	20 0	99,9 %, 30s/inst	refacto r
G1	Binary Go	sylian-g1	GPG+SHA -256	systemd unit	/usr/local/bin/sy lion-g1	50	99,9 %, <50ms	reuse
G2	Python .whl	g2.main:a pp	Ed25519+ pip hash	Python 3.12 venv	/opt/sylian/g2/	30	99,8 %, <100ms	refacto r
VPS	tar.gz Ans ible+HCL	site.yml	GPG	Ansible/ Terrafor m	/etc/sylian/infra/	20	n/a, <20 min	reuse
Matri x	Python .w hl+YAML	sylian_mo dule	Ed25519	Synapse 3.11	/etc/matrix-syna pse/sylian/	15	99,5 %, <5s lag	reuse
SCHEMATY YAML — bundle manifesty (przykłady)								

Pixel (evolution path)

```
YAML

bundle_id: "bnd-pixel-001"\ntarget: Pixel\nversion: "3.0.0-rc1"\nrun_id: "550e8400-e29b-41d4-a716-446655440012"\nformat: spa_tgz\nentry_point: index.html\nimplementation_strategy: reuse\nrebuild_track: false\nartifacts:\n  - kind: binary\n    path: dist/pixel-3.0.0.tar.gz\n    sha256: "a1b2c3d4..." \n    size_bytes: 4300000\n  - kind: sbom\n    path: sbom.json\n    sha256: "e5f6g7h8..." \nsignatures:\n  - algorithm: Ed25519\n    pubkey_id: "sylian-build-key-01"\n    signature_b64: "base64=="\n  - algorithm: Ed25519\n    pubkey_id: "sylian-build-key-01"\n    signature_b64: "base64=="\nredteam_bundle_id: "rt-bnd-pixel-001"\nblueteam_bundle_id: "bt-bnd-pixel-001"\nadversarial_contract_id: "adv-pixel-001"\nbuild_ts: "2026-04-15T10:00:00Z"\ndeploy_path: /var/www/pixel/
```

Router (refactor path — shadow track)

```
YAML

bundle_id: "bnd-router-001"\ntarget: Router\nversion: "3.0.0-rc1"\nformat: oci_container\nentry_point: entrypoint.sh\nimplementation_strategy: refactor\nrebuild_track: true\ncutover_strategy: shadow\nlegacy_bundle_id: "bnd-router-legacy-001"\nartifacts:\n  - kind: binary\n    path: router-3.0.0.tar\n    sha256: "i9j0k1l2..." \n    size_bytes: 20500000\n  - kind: sbom\n    path: sbom.json\n    sha256: "e5f6g7h8..." \nsignatures:\n  - algorithm: cosign\n    pubkey_id: "sylian-cosign-key-01"\n    signature_b64: "base64=="\n  - algorithm: Ed25519\n    pubkey_id: "sylian-build-key-01"\n    signature_b64: "base64=="\nredteam_bundle_id: "rt-bnd-router-001"\nblueteam_bundle_id: "bt-bnd-router-001"\nadversarial_contract_id: "adv-router-001"\nruntime: podman_rootless\ndeploy_path: /opt/sylian/router/
```

Greenfield bundle (rebuild path — Core 10 aktywny)

YAML

```
bundle_id: "bnd-g2-rebuild-001"\ntarget: G2\nversion: "3.0.0-greenfield-rc1"\nformat: python_wheel\nentry_point: g2_v3.main:app\nimplementation_strategy: greenfield\nrebuild_track: true\ncutover_strategy: parallel\ntarget_baseline: "g2-greenfield-v3"\ncore_10_required: true\nartifacts:\n  - kind: binary\n    path: g2_v3-3.0.0.whl\n    sha256: "m3n4o5p6..." \n    size_bytes: 28000000\n  - kind: sbom\n    path: sbom_greenfield.json\n    sha256: "q7r8s9t0..." \n  signatures:\n    - algorithm: Ed25519\n      pubkey_id: "syllion-target-key-01"\n      signature_b64: "base64=="\n      redteam_bundle_id: "rt-bnd-g2-rebuild-001"\n      blue_team_bundle_id: "bt-bnd-g2-rebuild-001"\n      adversarial_contract_id: "adv-g2-rebuild-001"\n      rebuild_validation_report_id: "rvr-g2-rebuild-001"\ndeploy_path: /opt/syllion/g2_v3/
```

WORKFLOWY

WF-12-01: Pełny Build Bundle (Evolution Path)

1. Plan 11 (Core 7) wywołuje POST /bundle/v1/build z run_id i target
2. BS weryfikuje decyzję Council (Plan 05 >= D2) i kontekst (Plan 11 Core 7)
3. BS wykonuje Pre-stages A0–A5 (legacy track) — blokuje przy jakimkolwiek fail
4. Red Team (A-ADV-01) analizuje threat model; BS weryfikuje rt_bundle_signed
5. Blue Team (A-ADV-02..03) projektuje BT Bundle; Coverage Audit Agent sprawdza coverage_pct
6. Gate G-BUN-ADV-01: coverage_pct >= 80 % (D3) lub >= 95 % (D4 Critical) — blokuje jeśli nie
7. BS buduje artefakty wg kontraktu targetu (Core 3)
8. BS generuje SBOM; weryfikuje rozmiar (G-BND-02, G-BND-03)
9. BS podpisuje bundle kluczami z Vault (Core 4); double-check (G-BND-01)
10. Adversarial Test Run (Core 5): Red Team atakuje; Blue Team wykrywa; Purple Team raport
11. BS uruchamia TestSuite (Core 6): contract, unit, integration, security
12. RegressionReport (Core 7): critical_count > 0 → G-BND-04 block
13. Canary Deploy (Core 8): 5 % / 15 min; auto-rollback przy G-BND-06
14. Promote (Core 9): 100 % ruchu; archiwizacja w Plan 13; zdarzenie do Matrix

WF-12-02: Build Bundle — Rebuild Path (Core 10 aktywny)

1. Plan 14 (SYSTEM_REBUILD_MODE) aktywuje rebuild_track=true dla targetu
2. BS wykonuje Pre-stages A0–A5 Legacy i A0-RB–A5-RB Target równolegle
3. Adversarial Track A-ADV-01..03 uruchamiany dla target architecture
4. Gate G-BUN-ADV-01 dla target: coverage_pct >= 95 % (D5 wymagany przy Critical)
5. Build artefaktów greenfield; SBOM niezależny od legacy
6. Podpisanie nowymi kluczami target (A3-RB: Vault Key Provisioning)
7. Core 10 — Rebuild Contract Validator: weryfikacja inwariantów Księgi
8. Core 10 generuje RebuildValidationReport; G-BND-RB-01 blokuje przy fail
9. Shadow Canary: target traffic w trybie shadow obok legacy
10. Cutover Switch (Plan 14): ruch przełączony z legacy → target
11. Legacy Decommission Trigger: Plan 14 inicjuje wycofanie legacy track

WF-12-03: Adversarial Contract — Coverage Audit

- 1. Coverage Audit Agent pobiera rt_bundle i bt_bundle dla bundle_id
- 2. Dla każdego AttackScenario (RT-xxx) sprawdza czy istnieje DetectionRule (BL-xxx) z covers_rt == scenario_id
- 3. coverage_pct = len(covered) / len(total_attacks) * 100
- 4. Generuje CoverageAudit z missing_coverage[] dla niepokrytych ataków
- 5. Jeśli coverage_pct < 80 %: AdversarialContract.status = blocked; gate G-BUN-ADV-01 FAIL
- 6. Jeśli istnieje atak severity=Critical bez pokrycia: G-BUN-ADV-01 FAIL niezależnie od coverage_pct
- 7. Wynik CoverageAudit archiwizowany w Plan 13 jako evidence D3/D4

WF-12-04: Rollback

POST /bundle/v1/{id}/rollback: przywrócenie poprzedniej wersji z archiwum. Wymagana decyzja D1 (Chief Review) lub D4 w przypadku krytycznego incydentu. Rollback musi zakończyć się w ciągu 60 s dla targetów produkcyjnych. W trybie Rebuild: rollback do legacy track (cutover odwrócony, Plan 14 switch).

API				
JSON				
<pre>POST /bundle/v1/build\n{\n "run_id": "uuid",\n "target": "Pixel Router G1 G2 VPS Matrix",\n "decision_sha256": "string",\n "context_sha256": "string",\n "rebuild_track": false,\n "implementation_strategy": "reuse refactor replace greenfield"\n}\n-> 202 { "bundle_id": "uuid", "state": "building" }\n\nPOST /bundle/v1/redteam\n{\n "bundle_id": "uuid",\n "attack_scenarios": [...],\n "operator_token": "JWT" }\n-> 202 { "rt_bundle_id": "uuid", "state": "pending_blue" }\n\nPOST /bundle/v1/blueteam\n{\n "bundle_id": "uuid",\n "rt_bundle_id": "uuid",\n "detection_rules": [...],\n "soc_playbooks": [...] }\n-> 202 { "bt_bundle_id": "uuid", "coverage_pct": 85.0 }\n\nGET /bundle/v1/{id}/adversarial/coverage\n-> 200 { "coverage_pct": 85.0, "status": "covered blocked",\n "missing_coverage": [...] }\n\nPOST /bundle/v1/{id}/adversarial/run\n{\n "operator_token": "JWT",\n "decision_class": "D3" }\n-> 202 { "adv_run_id": "uuid", "state": "running" }\n\nPOST /bundle/v1/{id}/test\n{\n "suite_kinds": ["contract", "unit", "integration", "security"] }\n-> 200 { "suite_id": "uuid", "passed": true, "regressions": 0 }\n\nPOST /bundle/v1/{id}/core10/validate\n{\n "canon_compact_sha256": "string",\n "rebuild_decision_sha256": "string" }\n-> 200 { "rvr_id": "uuid", "invariants_ok": true, "d3_decisions_ok": true }\n\nPOST /bundle/v1/{id}/canary\n{\n "traffic_pct": 5,\n "duration_min": 15,\n "rebuild_track": false }\n-> 202 { "canary_id": "uuid", "state": "running" }\n\nPOST /bundle/v1/{id}/promote\n{\n "operator_id": "string",\n "decision_class": "D2" }\n-> 200 { "bundle_id": "uuid", "state": "promoted" }\n\nPOST /bundle/v1/{id}/rollback\n{\n "reason": "string",\n "operator_id": "string",\n "prev_bundle_id": "uuid" }\n-> 200 { "bundle_id": "uuid", "state": "rolled_back", "restored": "uuid" }</pre>				
HARD GATE'Y				
Kod	Etap	Warunek blokujący	Klasa D	Akcja
G-BND-00	Init	run_id nieaktywny lub brak autoryzacji ACP (Plan 01)	D0	reject — bundle nie startuje

Kod	Etap	Warunek blokujący	Klasa D	Akcja
G-BND-01	Sign	Sygnatura bundle nieprawidłowa lub brakuje dla danego targetu	D0	reject — bundle nie może być testowany
G-BND-02	Build	Rozmiar artefaktu przekracza max_size_bytes dla targetu	D1	fail — build przerwany, raport do Matrix
G-BND-03	Build	Brakuje SBOM (Software Bill of Materials) w manifeście	D1	block — build niekompletny
G-BND-04	Test	RegressionReport.critical_count > 0	D2	block — promocja wstrzymana do fixu
G-BND-05	Test	TestSuite kind=contract nie przeszedł (format/entry point/manifest niezgodny)	D2	fail — bundle niezgodny z kontraktem targetu
G-BND-06	Canary	error_rate > 1 % lub p99_latency > 2× baseline podczas canary	D0	auto-rollback <60 s, stan rolled_back
G-BND-07	Promote	Brak zatwierdzonego CanaryDeployment.promoted=true przed promocją	D2	reject — promocja wymaga pozytywnego canary
G-BND-08	Promote	Decyzja Council < D2 dla targetów produkcyjnych (G1/G2/Router)	D2	block — wymagana decyzja D2/D3
G-BUN-AD V-01	Adversarial (Core 5)	coverage_pct < 80 % (D3) lub coverage_pct < 95 % dla Critical (D4) lub brak BT Bundle	D3/D4	block — bundle nie przechodzi bez pokrycia adversarialnego
G-BUN-AD V-02	Adversarial Test Run	Red Team atak severity >= high zakończony sukcesem (detekcja BT nie zadziałała)	D3	block — wymagane D3 review; brak promocji
G-BUN-AD V-03	Adversarial Test Run	Red Team atak severity = Critical zakończony sukcesem	D4	block natychmiastowy — wymagane D4 Human Gate
G-BND-RB -01	Core 10	RebuildValidationReport.invariants_failed[] niepuste LUB d3_decisions_ok=false	D4	block — bundle rebuild nie spełnia inwariantów Księgi
G-BND-RB -02	Pre-stage A5-RB	target coverage < 95 % w rebuild path PRZED D5 promotion	D5	block — D5 wymaga 95 % coverage adversarial dla target architecture

TABELA KLAS DECYZJI D0–D5 w kontekście Planu 12

Klasa	Przykład operacji Bundle	Aktor	Ścieżka
D0	Weryfikacja sygnatury, auto-rollback canary, init bundle session	BS auto	—
D1	Rollback zwykły, patch SBOM, zmiana opisu artefaktu	Chief Reviewer	evolution

Klasa	Przykład operacji Bundle	Aktor	Ścieżka
D2	Promocja bundla dla Pixel/Matrix/VPS, nowa wersja minor	Council 3/4	evolution
D3	Promocja bundla produkcyjnego (G1/G2/Router); coverage < 95 % Critical; zmiana kontraktu targetu	Council 4/4	refactor
D4	Rebuild modułu Bundle Service; zmiana algorytmu podpisu; Core 10 failure review; G-BUN-ADV-03	Human Gate	rebuild
D5	Pełna wymiana architektury Bundle Service (greenfield); zmiana inwariantów Księgi; G-BND-RB-02	Council 4/4 + Human + External Review	rebuild (greenfield)
TESTY			
Kod	Nazwa	Warunki testu	Oczekiwany wynik
T-BND-01	Build Pixel (evolution)	POST /build target=Pixel, run_id aktywny, decision D2, impl=reuse	202, bundle_id, state=building→signed; pre-stages A0-A5 pass
T-BND-02	G-BND-01 blokada	Build z wymuszoną błędną sygnaturą Ed25519	bundle odrzucony, G-BND-01, brak canary
T-BND-03	Kontrakt Router	Build Router, format=oci, entry=entrypoint.sh, sig=cosign+Ed25519	manifest zgodny z kontraktem; pre-stage A3 pass (vault keys)
T-BND-04	G-BND-04 regresja	Symulacja critical_count=1 w RegressionReport	block na promote, raport do Matrix (Plan 13)
T-BND-05	Canary sukces	POST /canary traffic_pct=5, duration=15min, metryki OK	canary_id promoted=true, brak rollbacku
T-BND-06	G-BND-06 rollback	Canary z error_rate=2,5 % (>1 %)	auto-rollback <60 s, stan=rolled_back, alert Matrix
T-BND-07	Promote bez canary	POST /promote bez /canary	reject, G-BND-07, 409 Conflict
T-BND-08	Rollback VPS	POST /rollback target=VPS z prev_bundle_id	200, restored=prev_bundle_id, stan=rolled_back
T-ADV-01	RT Bundle bez BT Bundle	POST /bundle/v1/build; brak bt_bundle_id przed promote	G-BUN-ADV-01 block; promote rejected
T-ADV-02	Coverage < 80 %	bt_bundle z coverage_pct=70 %	G-BUN-ADV-01 block; CoverageAudit z missing_coverage
T-ADV-03	Coverage OK (85 %)	bt_bundle z coverage_pct=85 %, brak Critical bez pokrycia	G-BUN-ADV-01 PASS; promote allowed (D3)

Kod	Nazwa	Warunki testu	Oczekiwany wynik
T-ADV-04	Critical bez pokrycia	Scenariusz RT-001 severity=Critical bez reguły BL	G-BUN-ADV-01 block niezależnie od coverage_pct całkowitego
T-ADV-05	Adversarial Test Run — atak high sukces	Red Team atak severity=high zakończony sukcesem (BT nie wykryło)	G-BUN-ADV-02 block; D3 review wymagany
T-ADV-06	Adversarial Test Run — atak critical sukces	Red Team atak severity=Critical zakończony sukcesem	G-BUN-ADV-03 block; D4 Human Gate; natychmiastowy
T-RB-01	Core 10 — inwariant OK	POST /core10/validate; wszystkie invariants_failed=[], d3_ok=true	rvr_id generated; G-BND-RB-01 PASS; promote allowed
T-RB-02	Core 10 — inwariant naruszony	POST /core10/validate; invariants_failed=[„vault_isolation”]	G-BND-RB-01 FAIL; block; D4 wymagany
T-RB-03	Pre-stage A5-RB coverage < 95 %	Rebuild path, coverage_pct=88 % przy scenariuszu Critical	G-BND-RB-02 block; D5 wymagany
T-RB-04	Pełny rebuild path G2	Build G2 greenfield: A0-RB..A5-RB + A-ADV + Core 10 + shadow canary	Wszystkie gates PASS; promoted na target architecture
STATUSY I EVIDENCE			
Status bundle	Opis	Następny możliwy stan	Evidence w Plan 13
building	Artefakty w trakcie budowania	signed / failed	—
signed	Bundle podpisany; oczekuje na testy	testing / failed	Signature record
testing	TestSuite i Adversarial Test Run w trakcie	canary / failed	TestResults, CoverageAudit, AdversarialReport
canary	5 % ruchu; monitoring aktywny	promoted / rolled_back	CanaryMetrics snapshot
promoted	100 % ruchu; aktywna wersja produkcyjna	rolled_back (tylko ręcznie)	PromotionRecord, hash w Plan 13
rolled_back	Poprzednia wersja przywrócona	building (nowy bundle)	RollbackRecord, reason, operator_id
failed	Nieodwracalny błąd w build/sign/test	building (nowy bundle)	FailureReport, gate_code
ANEKS A — Adversarial Contract — pełna tabela pokrycia			

Atak RT		Seve rity	Phase	Moduły	Reguła BT	Playboo k SOC	Cov era ge	Gate przy fail
RT-001: Vault bypass		Critic al	pre-st age	vault, acp	BL-001, BL-002	BL-PB-0 01	100 %	G-BUN-ADV-01 + G-BUN-ADV-03
RT-002: Sig bypass		High	bundle	bundle_sv c, sign	BL-003	BL-PB-0 02	100 %	G-BUN-ADV-01 + G-BUN-ADV-02
RT-003: Canary poison		Medi um	post-st age	canary, router	BL-004	BL-PB-0 03	100 %	D3 review
RT-XXX: brak BT		Any	any	—	—	—	0 %	G-BUN-ADV-01 block zawsze
ANEKS B — Red / Blue / Purple Team: role v3.0								
Zesp ół	Model	Wa ga Bo ard	Faza	Zakres		Output		Blokuje
Red Team	Chief Review (GPT-5)	1.0 0	Pre-promot e (A-ADV- 01..02)	Threat model; RT Bundle; attack scenarios; adversarial test run		RedTeamBund le + Adversari alReport		Tak — brak RT Bundle lub atak Critical sukces
Blue Team	Chief Safety (rot acyjny)	n/a	Post-RT (A -ADV-03)	Detection rules; SOC playbooks; DFIR; BT Bundle; coverage audit		BlueTeamBun dle + Coverag eAudit		Tak — coverage < 80 %
Purpl e Team	Chief Reasoning (Claude)	1.0 0	Post RT+BT	Synteza RT+BT; priorytetyzacja luk; rekomendacje; go/no-go		PurpleReport + go/no-go		Tak — no-go blokuje promocję
ANEKS C — Core 10 Rebuild Contract Validator: inwarianty Księgi								
Invariant		Opis weryfikacji				Źródło prawdy		Fail → Gate
vault_isolation		Sekret nie jest eksponowany poza Vault (Plan 04); brak hardcoded credentials				Księga §4.2, canon_compact.md		G-BND-RB-01
acp_zero_trust		Każde żądanie bundle musi przejść przez ACP (Plan 01); brak bypass				Księga §3.1		G-BND-RB-01
audit_chain_integrit y		Hash chain AuditLog (Plan 13) zachowany; brak luk w chain				Księga §7.1, Plan 13		G-BND-RB-01
human_gate_d4		Wszystkie decyzje D4 w rebuild path mają zarejestrowaną akceptację człowieka				Klasy D0-D5, Plan 05		G-BND-RB-01
adversarial_covera ge_95		Coverage adversarial ≥ 95 % dla target architecture (rebuild path)				Adversarial Contract §12.4		G-BND-RB-02 (D5)
cutover_strategy_a pproved		cutover_strategy zatwierdzona przez D4; Plan 14 cutover_strategy.md istnieje				Plan 14 Greenfield Rebuild		G-BND-RB-01

Invariant	Opis weryfikacji	Źródło prawdy	Fail → Gate
sbom_complete	SBOM kompletny; brak UNKNOWN zależności; wszystkie hashi zweryfikowane	Plan 12 §A2-RB	G-BND-RB-01
no_legacy_contract_regression	Kontrakt docelowy nie regresuje inwariantów względem legacy (diff czysty)	Plan 08 Canon Evolution	G-BND-RB-01

REBUILD · D5 — Greenfield/Systemic w Bundle Plane

Decyzja D5 w Planie 12 oznacza pełną wymianę architektury Bundle Service lub zmianę inwariantów Księgi egzekwowanych przez Core 10. Kolor: D5 (#6A1B9A). Wymaga: Council 4/4 + Human Gate + External Review (red team zewnętrzny lub drugi operator). Przykłady: zmiana algorytmu podpisu bundle na nowy standard kryptograficzny, wprowadzenie nowego modelu adversarial coverage (ML vs threshold), pełna wymiana formatu bundla dla wszystkich 6 targetów jednocześnie.

PLAN 12 · ROZSZERZENIE v3.3 · EFFICIENCY

Bundle Generation & Testing Plane

§12.8 — Bramy G-EFF / G-PERF / G-MEM / G-COST jako pre-bundle gates (v3.3)

Wersja v3.3 wprowadza 11 nowych gate’ów pre-bundle pogrupowanych w cztery rodziny: 4 G-EFF (efficiency kodu), 3 G-PERF (wydajność runtime), 2 G-MEM (pamięć) i 2 G-COST (koszt obliczeniowy). Każda brama posiada: owner role (Plan 15 §15.10), skwantyfikowany próg, tryb weta (hard / soft) oraz ścieżkę override. Bramy są wykonywane sekwencyjnie przed wywołaniem bundle build, co gwarantuje odrzucenie kodu niesprawnego jakościowo zanim powstanie artefakt dystrybucyjny.

EFFICIENCY · 22 gate’y pre-bundle w v3.3

Połączenie gate’ów v3.0 (G-CON, G-SEC, G-COMP-CONTRACT), v3.1 (G-CUT-01-04, G-CFT-01, G-COMP-01) oraz nowych 11 gate’ów v3.3 daje łącznie 22 bramy pre-bundle w pełnym stosie pipeline’u SYLION v3.3.

§12.8.1 — Gate G-EFF-01: *bloat_score* ≤ *threshold*

G-EFF-01 oblicza syntetyczny wskaźnik *bloat_score* jako ważoną sumę pięciu składowych: *cognitive_complexity_norm* (30 %), *clone_coverage* (25 %), *method_length_excess* (20 %), *nesting_excess* (15 %) i *llm_anti_pattern_density* (10 %). Przekroczenie progu w dowolnej klasie decyzyjnej wyzwala HARD VETO; pipeline zatrzymuje się do momentu jawnego override przez Council 4/4 z uzasadnieniem D3.

Parametr	D2	D3	D4	D5	Typ weta
bloat_score max	≤ 0.30	≤ 0.20	≤ 0.15	≤ 0.15	HARD
cognitive_complexity_norm waga	30 %	30 %	30 %	30 %	—
clone_coverage waga	25 %	25 %	25 %	25 %	—
method_length_excess waga	20 %	20 %	20 %	20 %	—
nesting_excess waga	15 %	15 %	15 %	15 %	—
llm_anti_pattern_density waga	10 %	10 %	10 %	10 %	—
<pre># gate_registry.yaml — G-EFF-01 gates: - id: G-EFF-01 family: EFF version: v3.3 metric: bloat_score thresholds: D2: 0.30</pre>					

```

D3: 0.20
D4: 0.15
D5: 0.15
veto_type: HARD
owner: code_optimizer_agent # Plan 15 §15.10.1
override:
  council_quorum: 4/4
  justification_class: D3
  action_plan_deadline_days: 14
phase: pre_bundle

```

§12.8.2 — Gate G-EFF-02: cyclomatic_complexity_max

G-EFF-02 weryfikuje złożoność cyklomatyczną (CC) na dwóch poziomach: per metoda i per moduł. SonarQube definiuje CC > 25 jako twardą blokadę jakościową („critical code smell”); SYLION v3.3 stosuje bardziej rygorystyczny próg 20 dla metod i medianę 8 dla modułu, co odpowiada standardom enterprise clean code.

Metryka	Próg	Granulacja	Typ weta
CC per metoda	≤ 20	per method	HARD
CC mediana per moduł	≤ 8	per module	HARD
CC max modul (alert)	> 15	per module	WARN

```

# gate_registry.yaml — G-EFF-02
- id: G-EFF-02
  family: EFF
  version: v3.3
  metric: cyclomatic_complexity
  thresholds:
    per_method_max: 20
    per_module_median: 8
    per_module_alert: 15
  veto_type: HARD
  owner: code_optimizer_agent
  reference: SonarQube CC > 25 block [SonarQube]
  phase: pre_bundle

```

§12.8.3 — Gate G-EFF-03: clone_coverage

G-EFF-03 mierzy występowanie klonów kodu (duplikacji) na poziomie modułu i cross-modułowym. Progi oparte na rekomendacjach Code Climate i Kiuwan, które traktują klonowanie powyżej 20 % jako ryzyko jakościowe i utrudnienie refactoringu. Duplikacja inter-module powyżej 5 % sygnalizuje błędy architektoniczne sprzeczne z zasadą DRY.

Metryka	Próg D2	Próg D3+	Typ weta
clone_coverage per moduł	≤ 20 %	≤ 15 %	HARD
clone_coverage inter-module	≤ 5 %	≤ 3 %	HARD

```

# gate_registry.yaml — G-EFF-03
- id: G-EFF-03

```

```
family: EFF
version: v3.3
metric: clone_coverage
thresholds:
  per_module_D2: 0.20
  per_module_D3plus: 0.15
  inter_module_D2: 0.05
  inter_module_D3plus: 0.03
veto_type: HARD
owner: code_optimizer_agent
reference: Code Climate / Kiuwan thresholds
phase: pre_bundle
```

§12.8.4 — Gate G-EFF-04: llm_anti_pattern_scan

G-EFF-04 skanuje kod w poszukiwaniu wzorców anty-jakościowych charakterystycznych dla kodu generowanego przez LLM. Na podstawie badań Ox Security zidentyfikowano pięć kategorii anti-patternów; każda jest blokująca przy wystąpieniu więcej niż 3 przypadków w module. Limit jest stały niezależnie od klasy decyzyjnej.

Anti-pattern	Opis	Max wystąpień/moduł	Typ weta
Hallucinated import	Import nieistniejącego modułu / API	3	HARD
Dead code block	Blok kodu nigdy nieosiągalny (LLM artefakt)	3	HARD
Redundant null-check chain	Nadmiarowe sprawdzenia null po sobie	3	HARD
Verbosity explosion	Nieużyteczne komentarze / blank filler code	3	HARD
Copy-paste prompt leak	Fragment promptu lub instrukcji w kodzie	3	HARD

```
# gate_registry.yaml — G-EFF-04
- id: G-EFF-04
  family: EFF
  version: v3.3
  metric: llm_anti_pattern_density
  anti_patterns:
    - hallucinated_import
    - dead_code_block
    - redundant_null_check_chain
    - verbosity_explosion
    - copy_paste_prompt_leak
  threshold_per_pattern_per_module: 3
  veto_type: HARD
  owner: code_optimizer_agent
  reference: Ox Security LLM anti-patterns [Ox Security]
  phase: pre_bundle
```

§12.8.5 — Gate G-PERF-01: p95_latency ≤ baseline + 10 %

G-PERF-01 porównuje mediane czasu odpowiedzi na 95. percentylu (p95) zmierzoną w środowisku staging z linią bazową ustaloną z poprzedniego promowanego bundla. Zgodnie ze standardami OneUptime przekroczenie p95 o ponad 10 % względem baseline sygnalizuje regresję wydajnościową kwalifikującą się do blokady.

Metryka	Próg	Granulacja	Owner	Typ weta
p95_latency vs baseline	baseline + 10 %	per endpoint	Performance Engineer	SOFT
p99_latency (alert)	baseline + 20 %	per service	Performance Engineer	WARN

```
# gate_registry.yaml – G-PERF-01
- id: G-PERF-01
  family: PERF
  version: v3.3
  metric: p95_latency
  threshold: baseline_plus_10pct
  veto_type: SOFT
  owner: performance_engineer
  override:
    council_quorum: 3/4
    grace_period_hours: 24
  reference: OneUptime p95 standards
  phase: pre_bundle
```

§12.8.6 — Gate G-PERF-02: $error_rate \leq SLO_budget$

G-PERF-02 egzekwuje politykę budżetu błędów (error budget) zgodnie z Google SRE Site Reliability Engineering. Wdrożenie nowego bundla, które w środowisku staging generuje `error_rate` przekraczający przydzielony `SLO_budget` (zwykle 0.1 % dla D3+), wyzwala SOFT VETO z 24-godzinnym oknem grace i automatyczną eskalacją Council 3/4.

Metryka	Próg D2	Próg D3+	Owner	Typ weta
error_rate vs SLO budget	$\leq 0.5 \%$	$\leq 0.1 \%$	Performance Engineer	SOFT
critical_error_rate	$\leq 0.05 \%$	$\leq 0.01 \%$	Performance Engineer	HARD

```
# gate_registry.yaml – G-PERF-02
- id: G-PERF-02
  family: PERF
  version: v3.3
  metric: error_rate
  thresholds:
    D2: 0.005
    D3plus: 0.001
    critical_D3plus: 0.0001
  veto_type: SOFT # critical_error_rate: HARD
  owner: performance_engineer
  override:
    council_quorum: 3/4
    grace_period_hours: 24
  reference: Google SRE error budget policy
```

```
phase: pre_bundle
```

§12.8.7 — Gate G-PERF-03: $throughput \geq baseline \times 0.95$

G-PERF-03 zapewnia, że przepustowość systemu (requests per second, transakcje, operacje I/O) nie spada poniżej 95 % wartości osiąganey przez poprzedni promowany bundle. Brama chroni przed regresją wydajnościową nieujawniającą się w latencji, lecz w spadłej pojemności obsługi żądań.

Metryka	Próg	Granulacja	Owner	Typ weta
throughput vs baseline	$\geq baseline \times 0.95$	per service	Performance Engineer	SOFT
throughput krytyczny	$\geq baseline \times 0.90$	per system	Performance Engineer	HARD

```
# gate_registry.yaml — G-PERF-03
- id: G-PERF-03
  family: PERF
  version: v3.3
  metric: throughput_rps
  threshold: baseline_multiplier_0.95
  critical_threshold: baseline_multiplier_0.90
  veto_type: SOFT # critical: HARD
  owner: performance_engineer
  override:
    council_quorum: 3/4
    grace_period_hours: 24
  phase: pre_bundle
```

§12.8.8 — Gate G-MEM-01: $rss_peak \leq memory.high$ cgroup

G-MEM-01 sprawdza, czy szczytowe zużycie pamięci RSS procesu mieści się w limicie memory.high cgroups ustawionym dla danego środowiska docelowego. Przekroczenie oznacza ryzyko OOM kill na produkcji i jest traktowane jako SOFT VETO przez Memory Auditor z możliwością override Council 3/4.

Metryka	Próg	Granulacja	Owner	Typ weta
rss_peak	$\leq memory.high$ (cgroup)	per container	Memory Auditor	SOFT
rss_peak krytyczny	$\leq memory.max \times 0.85$	per container	Memory Auditor	HARD

```
# gate_registry.yaml — G-MEM-01
- id: G-MEM-01
  family: MEM
  version: v3.3
  metric: rss_peak_bytes
  threshold: cgroup_memory_high
  critical_threshold: cgroup_memory_max_0.85
  veto_type: SOFT
  owner: memory_auditor_agent # Plan 15 §15.10
```

```
override:
  council_quorum: 3/4
  grace_period_hours: 24
  phase: pre_bundle
```

§12.8.9 — Gate G-MEM-02: *leak_rate* ≤ 0.5 MB/h

G-MEM-02 wykonuje 30-minutowy soak test w izolowanym środowisku staging i ekstrapoluje tempo wzrostu pamięci RSS jako *leak_rate* w MB/h. Próg 0.5 MB/h jest kalibrowany względem typowych wzorców wycieków GC-managed runtime; dla platform natywnych (C/Rust) próg spada do 0.1 MB/h.

Metryka	Próg (GC runtime)	Próg (native)	Owner	Typ weta
leak_rate	≤ 0.5 MB/h	≤ 0.1 MB/h	Memory Auditor	SOFT
soak_test_duration	30 min	30 min	—	—

```
# gate_registry.yaml — G-MEM-02
- id: G-MEM-02
  family: MEM
  version: v3.3
  metric: memory_leak_rate_mb_per_hour
  thresholds:
    gc_runtime: 0.5
    native: 0.1
  soak_test_minutes: 30
  veto_type: SOFT
  owner: memory_auditor_agent
  override:
    council_quorum: 3/4
    grace_period_hours: 24
    phase: pre_bundle
```

MEMORY · *Memory leak w staging ≠ brak leaków na produkcji*

Soak test 30-minutowy wykrywa wstępne trendy wzrostu pamięci, lecz nie gwarantuje wykrycia wycieków ujawniających się po wielogodzinnej pracy. G-MEM-02 jest bramą minimalną; produkcyjny monitoring pamięci (Plan 13) kontynuuje obserwację w canary i po pełnej promocji.

§12.8.10 — Gate G-COST-01: *budget_envelope_utilization* ≤ 90 %

G-COST-01 weryfikuje, czy szacowane koszty związane z uruchomieniem nowego bundla nie przekroczą 90 % limitu budżetu okna aktywnego dla danej operacji. Cost Optimizer Agent (Plan 15 §15.10) odpowiada za kalkulację *budget_envelope_utilization* na podstawie historycznych pomiarów i modelu zmiany kosztu. Przekroczenie progu wyzwala SOFT VETO z 24-godzinnym grace period.

Metryka	Próg	Owner	Typ weta	Grace period
budget_envelope_utilization	≤ 90 %	Cost Optimizer	SOFT	24 h
budget_envelope_utilization alert	> 80 %	Cost Optimizer	WARN	—
<pre># gate_registry.yaml - G-COST-01 - id: G-COST-01 family: COST version: v3.3 metric: budget_envelope_utilization threshold: 0.90 alert_threshold: 0.80 veto_type: SOFT owner: cost_optimizer_agent # Plan 15 \$15.10 override: council_quorum: 3/4 grace_period_hours: 24 phase: pre_bundle</pre>				

§12.8.11 — Gate G-COST-02: *cache_hit_rate* ≥ 50 %

G-COST-02 mierzy skuteczność cache'owania obliczeń i wywołań LLM. Wdrożony kod musi utrzymywać wskaźnik trafień cache na poziomie co najmniej 50 %, poniżej którego koszt per-request przekracza model bazowy. Badania ProjectDiscovery wskazują efektywny zakres *cache_hit_rate* na poziomie 59–70 % jako optimum operacyjne; SYLION v3.3 stosuje 50 % jako min-floor.

Metryka	Próg min	Cel optymalny	Owner	Typ weta
cache_hit_rate	≥ 50 %	59–70 % (optimum)	Cost Optimizer	SOFT
cache_miss_rate	≤ 50 %	≤ 30–41 %	Cost Optimizer	SOFT
<pre># gate_registry.yaml - G-COST-02 - id: G-COST-02 family: COST version: v3.3 metric: cache_hit_rate threshold_min: 0.50 optimal_range: [0.59, 0.70] veto_type: SOFT owner: cost_optimizer_agent override: council_quorum: 3/4 grace_period_hours: 24 reference: ProjectDiscovery 59-70% [research] phase: pre_bundle</pre>				

COST · *Cache hit rate a koszt operacyjny*

Każdy punkt procentowy poniżej progu 50 % bezpośrednio przekłada się na wzrost kosztu per-request LLM. G-COST-02 zapobiega wdrożeniu kodu, który naruszyłby budżet przez nieefektywne strategii pamięci podręcznej.

§12.8.12 — Pełny rejestr gate'ów efficiency (11 bram v3.3)

Gate ID	Sprawdza	Próg D2	Próg D3+	Owner	Veto	Override
G-EFF-01	bloat_score	≤ 0.30	≤ 0.20	Code Optimizer	HARD	Council 4/4 + D3
G-EFF-02	cyclomatic_complexity	$CC \leq 20$	$CC \leq 20$	Code Optimizer	HARD	Council 4/4 + D3
G-EFF-03	clone_coverage	$\leq 20 \%$	$\leq 15 \%$	Code Optimizer	HARD	Council 4/4 + D3
G-EFF-04	llm_anti_pattern_scan	$\leq 3/\text{modul}$	$\leq 3/\text{modul}$	Code Optimizer	HARD	Council 4/4 + D3
G-PERF-01	p95_latency	baseline+10 %	baseline+10 %	Perf Engineer	SOFT	Council 3/4 + 24 h
G-PERF-02	error_rate	$\leq 0.5 \%$	$\leq 0.1 \%$	Perf Engineer	SOFT	Council 3/4 + 24 h
G-PERF-03	throughput	$\geq b \times 0.95$	$\geq b \times 0.95$	Perf Engineer	SOFT	Council 3/4 + 24 h
G-MEM-01	rss_peak	$\leq \text{mem.high}$	$\leq \text{mem.high}$	Memory Auditor	SOFT	Council 3/4 + 24 h
G-MEM-02	leak_rate	$\leq 0.5 \text{ MB/h}$	$\leq 0.5 \text{ MB/h}$	Memory Auditor	SOFT	Council 3/4 + 24 h
G-COST-01	budget_utilization	$\leq 90 \%$	$\leq 90 \%$	Cost Optimizer	SOFT	Council 3/4 + 24 h
G-COST-02	cache_hit_rate	$\geq 50 \%$	$\geq 50 \%$	Cost Optimizer	SOFT	Council 3/4 + 24 h

§12.8.13 — Sekwencja wykonania gate'ów w pipeline

Gate'y pre-bundle v3.3 są wykonywane w następującej kolejności: najpierw sprawdzenia sekwencyjne (contract, security), następnie cztery grupy równoległe (EFF, PERF, MEM, COST), a na końcu bramy v3.1 w kolejności CUT → CFT → COMP. Całkowity budżet czasowy dla pełnego gate stack v3.3 wynosi 8 minut wall-clock.

```

contract_check → security_scan
  → [G-EFF-01 | G-EFF-02 | G-EFF-03 | G-EFF-04] (parallel)
    → [G-PERF-01 | G-PERF-02 | G-PERF-03] (parallel)
      → [G-MEM-01 | G-MEM-02] (parallel)
        → [G-COST-01 | G-COST-02] (parallel)
          → G-CUT-01 → G-CUT-02 → G-CUT-03 → G-CUT-04 (v3.1)
            → G-CFT-01 (v3.1)
              → G-COMP-01 (v3.1)
                → bundle.tar.gz

```

Faza	Opis	Avg time	Max time
contract_check + security_scan	Sekwencyjnie	45 s	90 s
G-EFF-01..04 (parallel)	Analiza statyczna kodu	60 s	120 s
G-PERF-01..03 (parallel)	Testy wydajnościowe staging	90 s	180 s
G-MEM-01..02 (parallel)	Soak test + peak RSS	120 s	210 s
G-COST-01..02 (parallel)	Kalkulacja budżetu i cache	30 s	60 s
G-CUT-01..04 + G-CFT-01 + G-COMP-01 (v3.1)	Cutover/compliance	30 s	60 s
TOTAL	—	375 s (~6.3 min)	480 s (8 min)

§12.8.14 — Zachowanie pipeline przy veto

Tryb reakcji pipeline’u zależy od typu weta. HARD VETO natychmiast zatrzymuje pipeline i wymaga jawnego działania; SOFT VETO aktywuje tryb ostrzeżenia z oknem grace period, po którego upływie następuje automatyczna eskalacja do Council. żadne weto nie może być ignorowane cicho — każde wymaga udokumentowanego override lub naprawy źródłowej.

Typ veto	Akcja default	Override path	Eskalacja
HARD VETO (G-EFF-01..04)	Pipeline STOP natychmiastowy	Council 4/4 + uzasadnienie D3 + action plan (deadline 14 dni)	CTO / Chief Reasoning automatycznie
SOFT VETO (G-PERF-01..03)	Pipeline WARN + 24 h grace	Council 3/4 w oknie grace	Performance Engineer + Council po upływie grace
SOFT VETO (G-MEM-01..02)	Pipeline WARN + 24 h grace	Council 3/4 w oknie grace	Memory Auditor + Council po upływie grace
SOFT VETO (G-COST-01..02)	Pipeline WARN + 24 h grace	Council 3/4 w oknie grace	Cost Optimizer + Council po upływie grace
Brak override przed deadline	Pipeline blokada permanentna	Wymaga pełnej naprawy lub D5 override	Emergency Council + Human Gate D4

KRYTYCZNE · *Hard veto Code Optimizer — nie jest soft warning*

Hard veto Code Optimizer NIE jest soft warning. Bundle build się zatrzymuje aż do explicit override Council 4/4. Decyzja override musi zawierać pisemne uzasadnienie klasy D3, harmonogram naprawy z datą deadline oraz akceptację ryzyka przez minimum czterech członków Council. Brak tych elementów = blokada permanentna.

PLAN 13 · WARSTWA KANONICZNA

Governance & Evidence Plane

REBUILD · Rebuildability over Lineage w tym planie

Governance & Evidence Plane jest fundamentem zdolności pipeline SYLION do bezpiecznego przeprowadzania wszystkich trzech ścieżek transformacji: Evolution, Refactor i Rebuild. W v3.0 Plan 13 rozszerza pokrycie dowodowe o klasę D5 (Greenfield/Systemic) — najgłębszy poziom Evidence pack obejmujący external review log, adversarial results oraz archiwum canon_compact.md z Planu 16. Bez aktywnego Governance Plane żadna decyzja klasy D3, D4 ani D5 nie jest uznana za definitywną. Plan 13 jest jedynym działem posiadającym prawo formalizowania propozycji zmian Księgi (book_change_proposal).

13.1. Cel i umiejscowienie

Governance & Evidence Plane jest niezależną warstwą audytową i dowodową systemu SYLION. Plan 13 nasłuchuje wszystkich zdarzeń ze wszystkich pozostałych planów (01-16), przechowuje je w niemutowalnym logu z łańcuchem kryptograficznym (hash chain), generuje raporty zgodności i dostarcza dowody dla audytu wewnętrznego i zewnętrznego. Żadna decyzja klasy D0–D5 nie jest uznana za definitywną bez potwierdzenia w Planie 13.

- Kanoniczny kanał zdarzeń: Matrix (Synapse) — każde zdarzenie to room event z podpisem
- Niemutowalna hash chain: każde zdarzenie zawiera prev_hash i własny hash SHA-256
- Dowody zgodności: kryptograficznie powiązane z decyzjami D0–D5
- Observability stack: Prometheus + Grafana + OpenTelemetry + Matrix
- Retention policy: 90 dni hot / 2 lata cold / infinite kryptograficzne hasze
- Archiwum compact: wszystkie canon_compact.md (Plan 16) archiwizowane w Governance KB
- Book change proposals: Governance formalizuje propozycje zmian Księgi

13.2. Ewidencja decyzji D0–D5

Każda decyzja systemu SYLION musi być zarejestrowana w Planie 13 z pełnym kontekstem. W v3.0 tabela rozszerzona o klasę D5 (Greenfield/Systemic). Klasy decyzji i ich wymagania dowodowe:

Klasa	Wymagane dowody	Kto podpisuje	Retention	Hash chain?
D0	log reguły deterministycznej + wejście/wyjście	ACP (auto)	90d hot / 2y cold	Tak
D1	log modelu Reviewer + uzasadnienie + confidence	Chief Reviewer	90d hot / 2y cold	Tak

Klasa	Wymagane dowody	Kto podpisuje	Retention	Hash chain?
D2	logi głosowania Council + wagi + wynik	Board (automatyczny)	2y hot / infinite hash	Tak
D3	logi głosowania Council (jednomyślność) + 4/4 podpisy modeli	Board (4/4 wymagane)	2y hot / infinite hash	Tak
D4	podpis operatora SYLION + timestamp + uzasadnienie + D3 poprzednik	Operator (human)	infinite hot / infinite hash	Tak — klucz Ed25519
D5	Council 4/4 + Human + External Review + adversarial results + canon_compact archive + rebuild_decision_report + target_architecture_delta + external_review_log + post_mortem	Council + Human + External Reviewer (prawo weta)	infinite hot / infinite hash + wieczne archiwum	Tak — Ed25519 + zewnętrzny podpis

REBUILD · D5 — prawo weta External Reviewer

W klasie D5 zewnętrzny recenzent (external_red_team, second_operator lub external_auditor) posiada PRAWO WETA. Jeśli external_review_log zawiera verdict=rejected, decyzja D5 nie przechodzi — niezależnie od wyników Council i zatwierdzenia Human. Veto jest rejestrowane w Evidence pack i nie może być uchylone przez żaden inny mechanizm niż nowy cykl review z innym Reviewerem.

13.3. Evidence Pack — struktura per decyzja

Każda decyzja klasy D3 i wyższej generuje dedykowany Evidence pack w Governance KB. Pack jest zapieczętowany (sealed) kryptograficznie po zebraniu wszystkich artefaktów i staje się niemutowalnym zapisem historycznym decyzji. Struktura paczki:

Artefakt	Opis	Wymagany dla
decision.json	Opis decyzji, klasa, zakres, aktorzy, outcome, timestamp	D3, D4, D5
workflow_log.jsonl	Każdy krok procesu decyzyjnego w formacie JSONL (append-only)	D3, D4, D5
council_votes.json	Głosy Council z uzasadnieniami, wagi, wynik głosowania	D3, D4, D5
human_approval.json	Decyzja operatora: approve/reject, timestamp, uzasadnienie, podpis Ed25519	D4, D5
external_review.json	Wynik external review: verdict, conditions, veto_used, podpis kryptograficzny	D5 only
artifacts/	Katalog wszystkich artefaktów użytych w procesie (raporty, delty, mapy modułów)	D3, D4, D5

Artefakt	Opis	Wymagany dla
compacts/	Wszystkie canon_compact.md wygenerowane w trakcie procesu (Plan 16)	D3, D4, D5
adversarial/	Wyniki red team + blue team: scenariusze, raporty, luki, mitygacje	D4, D5
post_mortem.md	Analiza po cutover: co poszło dobrze, problemy, wnioski, akcje naprawcze	D4, D5 (po cutover)

13.3.1. Statusy Evidence pack

Status	Opis	Przejście
PENDING	Pack zainicjowany, zbieranie dokumentów nie rozpoczęte	→ COLLECTING po pierwszym artefakcie
COLLECTING	Aktywne zbieranie artefaktów; pack otwarty na zapis	→ SEALED po zebraniu wszystkich wymaganych artefaktów
SEALED	Pack zapieczętowany kryptograficznie; niemutowalny	→ ARCHIVED po upływie retencji hot
ARCHIVED	Przeniesiony do cold storage; dostęp przez API z SLA 4h	→ AUDIT_TRAIL_VERIFIED po weryfikacji zewnętrznej
AUDIT_TRAIL_VERIFIED	Hash chain zweryfikowany przez zewnętrznego audytora; pełna ścieżka dowodowa potwierdzona	Stan końcowy

13.4. External Review Log — nowy artefakt v3.0

external_review_log.yaml jest nowym artefaktem v3.0 wymagany wyłącznie dla decyzji D5. Rejestruje pełny przebieg zewnętrznego przeglądu — tożsamość recenzenta (jako hash), zakres przeglądu, zastrzeżenia, werdykt i użycie prawa weta. Artefakt jest kryptograficznie podpisany przez Reviewera i zapieczętowany w Evidence pack D5.

YAML
<pre>external_review: review_id: ER-<timestamp> # np. ER-20260415T1432Z decision_ref: D5-XXXX # np. D5-2026-001 reviewer: role: external_red_team # external_red_team second_operator external_auditor identity: <redacted_hash> # SHA-256 tożsamości; oryginał w sealed vault credentials:</pre>

```

- cert_type: "OSCP"
- cert_type: "CISA"
- engagement_contract: "EC-2026-013"
review_scope:
  target_architecture: arch-v3-0042
  modules_reviewed:
    - module_id: "acp-auth-gateway"
      strategy: refactor
    - module_id: "phantom-core"
      strategy: greenfield
  security_claims:
    - "Zero Trust enforcement na wszystkich warstwach"
    - "Brak uprzywilejowanego dostępu bez 2PR i MFA"
    - "Klucze Ed25519 rotowane co 90 dni"
  concerns_raised:
    - id: C-001
      severity: high
      description: "Brak pełnej izolacji sieciowej legacy→target w fazie cutover"
      mitigation_required: true
    - id: C-002
      severity: low
      description: "Dokumentacja threat model nieaktualna dla modułu phantom-core"
      mitigation_required: false
  verdict: conditional # approved | rejected | conditional
  conditions:
    - condition_id: COND-001
      linked_concern: C-001
      requirement: "Wdrożyć network policy L3/L4 blokującą ruch legacy→target poza shado
w proxy"
      deadline: "2026-05-01"
      verified: false
  veto_used: false
  signature: <cryptographic_ed25519_signature>
  timestamp: "2026-04-15T14:32:00Z"

```

REBUILD · Mechanizm weta External Reviewer — szczegóły

Jeśli verdict=rejected, Governance Service natychmiast: (1) ustawia Evidence pack na status SEALED z flagą VETOED, (2) blokuje decision_id D5 w Decision Registry, (3) emituje alert critical do !syllion-governance i operatora, (4) inicjuje workflow escalation z zestawieniem zastrzeżeń recenzenta. Odblokowanie wymaga nowego cyklu D5 z innym Reviewerem lub usunięcia przyczyny veto i ponownego review przez tego samego Reviewera (nowy review_id).

13.5. Archiwum compact — integracja z Planem 16

Governance KB jest archiwum wszystkich canon_compact.md generowanych przez Plan 16 (Compaction & Memory Protocol). Każdy compact jest automatycznie kierowany do Governance KB po wygenerowaniu. Governance zarządza retention, indeksowaniem i dostępem.

13.5.1. Retention policy dla compactów

<i>Typ compactu</i>	<i>Hot retention</i>	<i>Cold retention</i>	<i>Archiwum wieczne</i>	<i>Warunek</i>
Compact rytmiczny (24h)	24h w hot	30 dni w cold	Nie (chyba że powiązany z D3+)	Automatyczny trigger
Compact pre-bundle	72h w hot	30 dni w cold	Nie (chyba że bundle zawierał D3+)	Trigger przed bundle build
Compact pre-D3	Infinite hot	N/A — hot b ezterminowy	Tak — wieczne archiwum	Trigger przed każdą decyzją D3+
Compact pre-D4	Infinite hot	N/A	Tak — wieczne archiwum	Trigger przed decyzją D4+
Compact pre-D5	Infinite hot	N/A	Tak — wieczne archiwum + kopia w Evidence pack D5	Trigger przed decyzją D5

13.5.2. Indeksowanie archiwum compact

- Indeks po decision_id: każdy compact powiązany z decyzją D3+ jest indeksowany po decision_id
- Indeks po timestamp: wszystkie compacty posortowane chronologicznie; zakres czasowy przeszukiwalny
- Indeks po triggerze: wyszukiwanie po typie triggera (rhythmic/pre-d3/pre-d4/pre-d5/pre-bundle)
- Indeks po module_id: compacty zawierające zmiany implementation_strategy danego modułu

13.5.3. Kontrola dostępu do archiwum compact

<i>Rola</i>	<i>Dostęp do compactów</i>	<i>Zakres</i>
R-00 (Root)	Pełny odczyt i zarządzanie	Wszystkie compacty, wszystkie triggery
R-01 (SecOfficer)	Pełny odczyt	Wszystkie compacty, focus bezpieczeństwo
R-02 (PipelineOp)	Odczyt operacyjny	Compacty z ostatnich 90 dni; pre-D3+ tylko za zgodą R-00
R-03 (CanonCust)	Odczyt związany z Księgą	Compacty powiązane z decyzjami kanonicznymi
R-05 (Auditor)	Odczyt read-only	Compacty powiązane z zakresem audytu; D3+ bez ograniczeń czasowych
Agenci (pipeline)	Brak bezpośredniego dostępu	Dostają compact wstrzyknięty na początku sesji przez Plan 16

13.6. Propozycje zmian Księgi — workflow book_change_proposal

Governance & Evidence Plane jest jedynym działem posiadającym kompetencję formalizowania propozycji zmian Księgi Kanonicznej. Każdy dział lub agent może zgłosić potrzebę zmiany, ale to Governance formalizuje propozycję i inicjuje workflow. Zatwierdzanie zmian Księgi jest ZAWSZE decyzją D4 (zmiana lokalna) lub D5 (zmiana fundamentalna). Governance proponuje — nigdy nie zatwierdza samodzielnie.

13.6.1. Workflow *book_change_proposal* — 6 kroków

1. Zgłoszenie: dział Governance lub dowolny inny dział zgłasza potrzebę zmiany Księgi przez kanał !syllion-governance lub API. Zgłoszenie zawiera: opis proponowanej zmiany, sekcję Księgi, uzasadnienie, wnioskodawcę.
2. Formalizacja: Governance analizuje zgłoszenie i tworzy artefakt `book_change_proposal.md` z pełną specyfikacją (patrz szablon §13.6.2). Artefakt jest rejestrowany w Governance KB jako PENDING.
3. Wstępna ocena Council D3: Council (4/4) ocenia propozycję wstępnie — akceptuje do eskalacji, odrzuca lub odkłada (defer). Wynik trafia do Evidence pack. Bez zgody Council propozycja nie postępuje.
4. Eskalacja: jeśli Council zaakceptował — propozycja jest eskalowana do D4 (zmiana lokalna Księgi, jeden dział) lub D5 (zmiana fundamentalna Księgi: zasady nadrzędne, granice etyczne, architektura bezpieczeństwa). Zakres eskalacji określa Governance na podstawie oceny blast radius.
5. Zatwierdzenie: Human Gate D4 lub Human Gate + External Review D5. Każda decyzja odmowna lub warunkowa jest rejestrowana z uzasadnieniem.
6. Rebuild Księgi: po zatwierdzeniu Governance inicjuje wersjonowany rebuild Księgi (nowa wersja semantyczna). Stara wersja Księgi jest archiwizowana w Governance KB jako Evidence kind=book_version z infinite retention.

13.6.2. Szablon *book_change_proposal.md*

MARKDOWN

```
# Book Change Proposal – BCP-<YYYY>-<NNN>

## Metadane
- **proposal_id**: BCP-2026-001
- **submitted_by**: dział Red Team (Plan 15.5)
- **submitted_at**: 2026-04-15T09:00:00Z
- **formalized_by**: dział Governance (Plan 15.7)
- **formalized_at**: 2026-04-15T11:00:00Z
- **status**: PENDING | COUNCIL_REVIEW | ESCALATED | APPROVED | REJECTED | DEFERRED
- **scope_class**: D4 | D5

## Sekcja Księgi objęta zmianą
- **book_section**: §7.3 – Zasady adversarial testing
- **current_version**: Księga v2.4
- **change_type**: extension | amendment | removal | new_section

## Opis proponowanej zmiany
Dodanie wymogu red team testing przed KAŻDĄ decyzją D4+, nie tylko D5.
Obecny tekst: „Red team testing jest wymagany dla decyzji klasy D5.”
Proponowany tekst: „Red team testing jest wymagany dla decyzji klasy D4 i D5.”
```

```
## Uzasadnienie
W trakcie 3 ostatnich cykli D4 zidentyfikowano luki bezpieczeństwa wykryte dopiero po cutover. Rozszerzenie wymogu na D4 zmniejszy ryzyko pre-cutover.

## Blast radius analysis
- **Affected_plans**: 07, 12, 13, 14, 15
- **Decision_class_impact**: D4 (zmiana lokalna wymogu)
- **Backward_compatible**: tak (zaostrenie, nie poluzowanie)
- **Estimated_effort**: 2 tygodnie (aktualizacja checklisty)

## Council pre-assessment
- **council_vote**: 4/4 — accept
- **council_decision_id**: D3-2026-047
- **escalation_target**: D4

## Human approval
- **decision_id**: D4-2026-012
- **approved_by**: operator (human)
- **approved_at**: 2026-04-20T14:00:00Z
- **conditions**: brak

## Rebuild Księgi
- **new_book_version**: v2.5
- **rebuilt_at**: 2026-04-21T09:00:00Z
- **archived_version**: Księga v2.4 → Governance KB, evidence_id=EV-BK-2026-004
```

13.7. Matrix log jako kanoniczny kanał zdarzeń

Matrix (Synapse) służy jako niemutowalny, federowany log zdarzeń dla SYLION. Każdy plan (01-16) emituje zdarzenia do dedykowanych room'ów Matrix. W v3.0 dodane room'y dla nowych planów 14, 15 i 16 oraz dedykowany kanał D5. Room hierarchy:

- !sylon-governance: decyzje D0–D5, gate failures, compliance alerts
- !sylon-d5: wyłączny kanał dla decyzji D5 i external review — dostęp ograniczony
- !sylon-rebuild: zdarzenia SYSTEM_REBUILD_MODE (Plan 14) — legacy/target lifecycle
- !sylon-runs: zdarzenia Run Lifecycle (Plan 11) — start/advance/complete/fail
- !sylon-bundles: zdarzenia Bundle (Plan 12) — build/sign/test/canary/promote/rollback
- !sylon-infra: zdarzenia Infrastructure (Plan 02) — provider state changes
- !sylon-agents: zdarzenia Agent Lifecycle (Plan 06/07) — onboarding/retirement
- !sylon-compacts: zdarzenia Compaction Protocol (Plan 16) — compact generated/archived
- !sylon-audit: zewnętrzny kanał audytowy (read-only dla audytorów)

13.8. Observability stack

Czterowarstwowy stos observability zapewnia pełną widoczność systemu SYLION. W v3.0 rozszerzony o metryki rebuild path i śledzenie compactów:

Warstwa	Technologia	Cel	Retention
Metryki	Prometheus + Grafana	Metryki operacyjne: run duration, token usage, cost, error rates, gate failures, rebuild_path metrics, compact generation frequency	30d hot scrape, 1y aggregated
Ślady	OpenTelemetry (OTLP)	Distributed tracing: każdy etap run ma trace_id + span_id powiązane z run_id; śledzenie D5 evidence pack assembly	14d hot, 90d sampled
Logi zdarzeń	Matrix (Synapse)	Kanoniczny, niemutowalny log zdarzeń biznesowych z hash chain; rozszerzony o room !syllion-d5 i !syllion-compacts	90d hot / 2y cold / infinite hashes
Alerty	Grafana Alertmanager	Routing alertów: krytyczne → operator Slack/Matrix; veto D5 → alert specjalny do !syllion-d5	90d historia alertów

13.9. Retention policy

Polityka przechowywania danych obowiązuje wszystkie zdarzenia i dowody w Planie 13. W v3.0 rozszerzona o poziomy dla D5 i wieczne archiwum compactów.

- HOT (90 dni): pełna treść zdarzenia dostępna przez API w czasie rzeczywistym
- COLD (2 lata): zdarzenia przenoszone do cold storage (S3-compatible), dostęp w ciągu 4h
- INFINITE: kryptograficzne skróty (SHA-256 hash chain) przechowywane na zawsze
- WIECZNE ARCHIWUM D5: wszystkie Evidence pack D5 hot bezterminowo + replika cold
- WIECZNE ARCHIWUM compact pre-D3+: compacty powiązane z decyzjami D3+ hot bezterminowo

KRYTYCZNE · Zasada niemutowalności

Żadne zdarzenie nie może być usunięte ani zmodyfikowane. Purge jest możliwy wyłącznie dla danych osobowych (GDPR) i wymaga decyzji D4 z pełną ścieżką dowodową i hash chain zachowanym bez treści. Evidence pack D5 jest ZAWSZE zachowywany w całości — purge niedozwolony nawet dla GDPR.

13.10. Compliance modes

Plan 13 obsługuje dwa tryby zgodności. Tryb internal: pełne API, wszystkie dane, alerty operacyjne. Tryb external audit: read-only API, zdarzenia D2–D5 tylko, raporty pdf-signed, dostęp przez dedykowany endpoint /governance/v1/audit/. W v3.0 dodano tryb external_d5_review dla zewnętrznych recenzentów D5 (scope ograniczony do target_architecture i security_claims danej decyzji).

13.11. Audit trail D5 — event sourcing

Dla decyzji klasy D5 obowiązuje pełny, nieusuwalny audit trail wzorowany na event sourcing pattern. Każde zdarzenie w procesie D5 jest append-only eventem w dedykowanym strumieniu. Rekonstrukcja stanu decyzji D5 w dowolnym momencie czasu jest możliwa wyłącznie przez replay strumienia.

- Strumień zdarzeń D5: append-only JSONL; brak UPDATE i DELETE
- Każde zdarzenie: event_id, decision_id, event_type, actor, payload, prev_hash, hash, timestamp
- Typy zdarzeń D5: D5_INITIATED, COUNCIL_VOTE, HUMAN_GATE_REQUESTED, HUMAN_GATE_APPROVED, EXTERNAL_REVIEW_STARTED, EXTERNAL_REVIEW_COMPLETED, VETO_RAISED, EVIDENCE_SEALED, CUTOVER_INITIATED, CUTOVER_COMPLETED, POST_MORTEM_FILED
- Integralność: hash chain zweryfikowany automatycznie co 1h przez Governance Service
- Eksport: GET /governance/v1/decisions/{decision_id}/stream?format=jsonl

13.12. Odnośniki do innych planów

Plan	Relacja z Planem 13
08 — Canonical Evolution	Decyzje kanoniczne (D3–D5) rejestrowane w Planie 13; delta Księgi jako Evidence artefakt
14 — Greenfield Rebuild	rebuild_decision_report.md, target_architecture_delta.json, greenfield_module_map.json, cutover_strategy.md jako artefakty Evidence pack D5
15 — Multi-Team Organization	Dział Governance (Plan 15.7) zarządza Governance Service; Council 4/4 (reprezentanci działów) głosuje w decyzjach D3–D5
16 — Compaction & Memory	Wszystkie canon_compact.md archiwizowane w Governance KB; Governance definiuje retention policy compactów; compacty pre-D3+ wieczne

PLAN 13 · SPECYFIKACJA WYKONAWCZA

Governance & Evidence Plane

CEL I ZAKRES

Specyfikacja Governance Service (GS) v3.0. Zakres: rejestrowanie i weryfikacja zdarzeń audytowych D0–D5, przechowywanie Evidence pack, external review log D5, archiwum compactów (Plan 16), book_change_proposal workflow, generowanie raportów, retencja i purge, routing alertów. GS jest autorytatywną warstwą zaufania — brak rekordu w GS = brak decyzji w systemie SYLION. Wersja: Pipeline 8.0.0, MZ baseline 3.5.5.

SCHEMAT YAML — governance_module.yaml

YAML

```
apiVersion: sylion.io/v3
kind: Module
metadata:
  module_id: "governance-service"
  name: "Governance & Evidence Service"
  plan: "13"
spec:
  implementation_strategy: reuse
  current_baseline: MZ 3.5.5
  target_baseline: null
  cutover_strategy: null
  version: "3.0.0"
  components:
    - id: audit-event-engine
      implementation_strategy: reuse
      notes: "Rozszerzony o D5 events i external_review_log"
    - id: evidence-pack-manager
      implementation_strategy: replace
      cutover_strategy: parallel
      notes: "Nowe pola: external_review.json, adversarial/, compacts/, post_mortem.md"
    - id: compact-archive
      implementation_strategy: greenfield
      cutover_strategy: cut-and-replace
      notes: "Nowy komponent v3.0 – integracja z Plan 16"
    - id: book-change-workflow
      implementation_strategy: greenfield
      cutover_strategy: cut-and-replace
      notes: "Nowy komponent v3.0 – book_change_proposal workflow"
    - id: d5-audit-stream
      implementation_strategy: greenfield
      cutover_strategy: cut-and-replace
      notes: "Event sourcing strumień dla D5 – append-only JSONL"
```

MODEL DOMENOWY

Encja	Pola kluczowe	Typ / Uwagi
AuditEvent	event_id (UUID), run_id, plan_id, event_type, payload_sha256, emitted_at, prev_hash, hash, matrix_event_id, emitter_id, rebuild_path (nullable), decision_class (nullable)	niemutowalny po zapisie; hash chain obowiązkowy
EvidencePack	pack_id (UUID), decision_id, decision_class (D3-D5), status (PENDING COLLECTING SEALED ARCHIVED AUDIT_TRAIL_VERIFIED), artefacts[], compacts[], adversarial[], external_review_id (nullable, D5), sealed_at, seal_hash, post_mortem_filed	status SEALED → niemutowalny; D5 wymaga external_review_id != null
DecisionRecord	decision_id, run_id, stage_code, decision_class (D0-D5), voted_by[], weights{}, result, signed_at, operator_sig_b64, external_review_id (nullable), prev_hash, hash, veto_raised (bool)	D4 wymaga operator_sig_b64; D5 wymaga external_review_id + veto_raised=false
ExternalReviewLog	review_id (ER-), decision_ref, reviewer_role, reviewer_identity_hash, reviewer_credentials[], review_scope{}, concerns_raised[], verdict (approved rejected conditional), conditions[], veto_used (bool), signature, timestamp	D5 only; veto_used=true → blokuje D5 niezależnie od Council i Human
CompactArchiveEntry	compact_id (UUID), compact_type (rhythmic pre-d3 pre-d4 pre-d5 pre-bundle), generated_at, trigger_event_id, decision_id (nullable), retention_class (ephemeral 30d infinite), content_sha256, content_ref, archived_at	retention_class: pre-D3+ → infinite; pozostałe → ephemeral/30d
BookChangeProposal	proposal_id (BCP-YYYY-NNN), submitted_by, submitted_at, formalized_by, status, scope_class (D4 D5), book_section, change_type, blast_radius{}, council_decision_id (nullable), human_decision_id (nullable), new_book_version (nullable)	formalizowany przez Governance; zatwierdzany D4 lub D5
ComplianceReport	report_id, scope, period_from, period_to, mode (internal external external_d5_review), generated_at, generated_by, sha256, pdf_sig_b64	mode external/external_d5_review: pdf_sig_b64 wymagane
RetentionPolicy	policy_id, event_type_pattern, hot_days, cold_days, purge_allowed, purge_requires_class, is_d5_evidence (bool)	is_d5_evidence=true → purge_allowed=false zawsze
SCHEMAT YAML — observability.yaml		

YAML

```

observability:
  version: "3.0"
  prometheus:
    scrape_interval: 15s
    retention_days: 30
    targets:
      - job: sylion-runs
        port: 9101

```

```

- job: sylion-bundles
  port: 9102
- job: sylion-governance
  port: 9103
- job: sylion-rebuild
  port: 9104
- job: sylion-compacts
  port: 9105
grafana:
  dashboards:
    - run_lifecycle
    - bundle_status
    - governance_audit
    - cost_overview
    - rebuild_progress
    - evidence_pack_status
    - compact_archive_health
  alert_channels:
    - name: operator_critical
      type: matrix
      room: "!sylion-governance"
      severity: critical
    - name: d5_veto_alert
      type: matrix
      room: "!sylion-d5"
      severity: critical
      filter: "event_type=VETO_RAISED"
    - name: operator_warning
      type: matrix
      room: "!sylion-governance"
      severity: warning
opentelemetry:
  exporter: otlp
  endpoint: http://otel-collector:4317
  trace_sampling_rate: 1.0
  retention_days: 14
matrix:
  homeserver: https://matrix.sylion.internal
  rooms:
    governance: "!sylion-governance"
    d5: "!sylion-d5"
    rebuild: "!sylion-rebuild"
    runs: "!sylion-runs"
    bundles: "!sylion-bundles"
    compacts: "!sylion-compacts"
    audit: "!sylion-audit"
  federation_enabled: true
retention:
  hot_days: 90
  cold_days: 730
  hash_chain_infinite: true
  d5_evidence_hot_infinite: true
  compact_pre_d3_hot_infinite: true
  cold_storage_backend: s3_compatible
  cold_restore_sla_h: 4

```

WORKFLOWY

WF-GOV-01: Emit Event (D0–D5)

1. Plan (01-16) wywołuje POST /governance/v1/events z payloadem, prev_hash i decision_class
2. GS weryfikuje hash chain: hash(payload) musi być zgodny z deklarowanym
3. GS oblicza nowy hash = SHA-256(prev_hash || payload_sha256 || timestamp)
4. GS zapisuje AuditEvent, emituje do odpowiedniego room'u Matrix (w tym !sylon-d5 dla D5)
5. Jeśli event_type to gate_failure lub decision_D4/D5: tworzy Evidence pack i Alert
6. Dla D5: GS inicjuje D5AuditStream i rejestruje event D5_INITIATED

WF-GOV-02: Collect Evidence (D3–D5)

1. GS tworzy EvidencePack ze statusem PENDING dla każdej decyzji D3+
2. Pack przechodzi do COLLECTING po pierwszym artefakcie
3. GS zbiera artefakty: decision.json, workflow_log.jsonl, council_votes.json
4. Dla D4+: dodaje human_approval.json (po zatwierdzeniu operatora)
5. Dla D5: dodaje external_review.json po zakończeniu External Review (wymaga verdict != null); jeśli veto_used=true → pack SEALED z flagą VETOED
6. GS dodaje compacts/ (canon_compact.md z okresu decyzji, pobrane z Plan 16)
7. Dla D4/D5: dodaje adversarial/ (wyniki red team + blue team z Planów 07/12)
8. Pack przechodzi do SEALED po zebraniu wszystkich wymaganych artefaktów; GS oblicza seal_hash = SHA-256 całego pack'u

WF-GOV-03: External Review D5

1. GS tworzy ExternalReviewLog z review_id=ER- i status INITIATED
2. Zewnętrzny recenzent (external_red_team, second_operator lub external_auditor) otrzymuje dostęp do zakresu review przez endpoint /governance/v1/d5/review/{review_id}
3. Recenzent analizuje target_architecture, security_claims, greenfield_module_map (Plan 14) i wyniki adversarial testing
4. Recenzent zgłasza concerns_raised (każdy z severity i mitigation_required)
5. Recenzent wystawia verdict: approved | rejected | conditional
6. Jeśli verdict=rejected: GS ustawia veto_used=true, blokuje decision_id D5, emituje alert VETO_RAISED do !sylon-d5 i !sylon-governance
7. Jeśli verdict=conditional: GS rejestruje warunki (conditions[]) jako wymagania do spełnienia przed SEALED; decyzja D5 zawieszona do weryfikacji
8. Recenzent podpisuje ExternalReviewLog kryptograficznie (Ed25519 lub GPG); podpis trafia do external_review.json w Evidence pack

WF-GOV-04: Book Change Proposal

1. Zgłoszenie przez kanał !sylon-governance lub POST /governance/v1/book-change/propose
2. GS tworzy BookChangeProposal z status=PENDING i przypisuje do działu Governance
3. Governance analizuje blast_radius i formalizuje book_change_proposal.md (§13.6.2)

4. GS eskaluje do Council D3 (scope_class: tymczasowy status COUNCIL_REVIEW)
5. Council 4/4 głosuje: accept → eskalacja do D4/D5; reject → status REJECTED; defer → status DEFERRED z datą ponownego rozpatrzenia
6. Po zatwierdzeniu D4/D5: GS inicjuje rebuild Księgi, archiwizuje starą wersję jako Evidence kind=book_version, aktualizuje CompactArchiveEntry

WF-GOV-05: Compact Archive

1. Plan 16 generuje canon_compact.md i wysyła POST /governance/v1/compacts z compact_type, trigger_event_id i opcjonalnym decision_id
2. GS tworzy CompactArchiveEntry i oblicza content_sha256
3. GS określa retention_class na podstawie compact_type: pre-D3+ → infinite; pre-bundle bez D3+ → 30d; rhythmic → 24h→30d
4. GS emituje zdarzenie do !syllion-compacts i indeksuje po decision_id, timestamp, trigger
5. Dla decision_id D5: GS kopiuje compact do compacts/ w EvidencePack D5

WF-GOV-06: Retain / Purge

Retain: automatyczny cron co 24h sprawdza RetentionPolicy i przenosi zdarzenia starsze niż hot_days do cold storage. Hash chain pozostaje w hot storage zawsze. Compacty pre-D3+ i Evidence pack D5 nigdy nie są przenoszone — hot bezterminowo. Purge (GDPR): wymaga decyzji D4 z podpisem operatora. GS usuwa content zdarzenia, zachowuje hash chain z adnotacją purged=true i tworzy Evidence dla purge. Evidence pack D5: purge niedozwolony — is_d5_evidence=true blokuje operację.

API

JSON

```
POST /governance/v1/events
{
  "run_id": "uuid",
  "plan_id": "14",
  "event_type": "run_state_transition|gate_failure|decision|D5_INITIATED|...",
  "decision_class": "D0|D1|D2|D3|D4|D5",
  "payload": {},
  "payload_sha256": "sha256:...",
  "prev_hash": "sha256:...",
  "emitter_id": "string"
}
-> 201 { "event_id": "uuid", "hash": "sha256:...", "matrix_event_id": "string" }

GET /governance/v1/decisions?run_id={uuid}&class=D5&from=2026-01-01
-> 200 { "decisions": [{ "decision_id": "...", "decision_class": "D5",
  "veto_raised": false, "external_review_id": "ER-..." }] }

POST /governance/v1/d5/review/{review_id}/submit
{
  "verdict": "approved|rejected|conditional",
  "concerns_raised": [...],
  "conditions": [...],
  "veto_used": false,
  "signature": "<ed25519_sig>"
}
```

```
-> 200 { "review_id": "ER-...", "verdict": "conditional",
        "veto_applied": false, "decision_blocked": false }

POST /governance/v1/book-change/propose
{
  "submitted_by": "red-team-dept",
  "book_section": "$7.3",
  "change_type": "amendment",
  "description": "...",
  "justification": "..."
}
-> 201 { "proposal_id": "BCP-2026-001", "status": "PENDING" }

POST /governance/v1/compacts
{
  "compact_type": "pre-d5",
  "trigger_event_id": "uuid",
  "decision_id": "D5-2026-001",
  "content_sha256": "sha256:...",
  "content_ref": "s3://compacts/2026/..."
}
-> 201 { "compact_id": "uuid", "retention_class": "infinite" }

GET /governance/v1/evidence/{pack_id}
-> 200 { "pack_id": "...", "decision_class": "D5",
        "status": "SEALED", "seal_hash": "sha256:...",
        "artefacts": [...], "compacts": [...] }

GET /governance/v1/decisions/{decision_id}/stream?format=jsonl
-> 200 (JSONL stream) event_id, event_type, actor, hash, timestamp per line
```

SCHEMAT JSON — audit event D5 z hash chain

JSON

```
{
  "event_id": "uuid",
  "run_id": "uuid",
  "plan_id": "14",
  "event_type": "D5_INITIATED",
  "decision_class": "D5",
  "rebuild_path": "greenfield",
  "payload": {
    "decision_id": "D5-2026-001",
    "trigger": "SYSTEM_REBUILD_MODE requested by operator",
    "scope": "full_system",
    "target_architecture": "arch-v3-0042"
  },
  "payload_sha256": "sha256:abc123...",
  "emitted_at": "2026-04-15T09:00:00Z",
  "emitter_id": "rebuild-orchestration-service",
  "prev_hash": "sha256:prev000...",
  "hash": "sha256:curr111...",
  "matrix_event_id": "$aBcDeFgH:matrix.sylion.internal",
  "matrix_room": "!sylion-d5",
  "sequence_no": 9042
}
```

SCHEMAT JSON — Evidence pack D5 (status SEALED)			
JSON			
<pre>{ "pack_id": "uuid", "decision_id": "D5-2026-001", "decision_class": "D5", "status": "SEALED", "seal_hash": "sha256:seal999...", "sealed_at": "2026-04-25T18:00:00Z", "artefacts": [{ "name": "decision.json", "sha256": "sha256:d1..." }, { "name": "workflow_log.jsonl", "sha256": "sha256:w1..." }, { "name": "council_votes.json", "sha256": "sha256:cv1..." }, { "name": "human_approval.json", "sha256": "sha256:ha1..." }, { "name": "external_review.json", "sha256": "sha256:er1..." }, { "name": "artifacts/rebuild_decision_report.md", "sha256": "sha256:rdr1..." }, { "name": "artifacts/target_architecture_delta.json", "sha256": "sha256:tad1..." }, { "name": "artifacts/greenfield_module_map.json", "sha256": "sha256:gmm1..." }, { "name": "artifacts/cutover_strategy.md", "sha256": "sha256:cs1..." }, { "name": "artifacts/external_review_log.md", "sha256": "sha256:erl1..." }], "compacts": [{ "compact_id": "uuid-c1", "compact_type": "pre-d5", "generated_at": "2026-04-15T08:55:00Z", "sha256": "sha256:cc1..." }, { "compact_id": "uuid-c2", "compact_type": "pre-d3", "generated_at": "2026-04-10T14:00:00Z", "sha256": "sha256:cc2..." }], "adversarial": [{ "name": "adversarial/red_team_report.md", "sha256": "sha256:rt1..." }, { "name": "adversarial/blue_team_report.md", "sha256": "sha256:bt1..." }], "post_mortem_filed": false, "veto_raised": false, "external_review_id": "ER-20260420T1200Z" }</pre>			
TRZY ŚCIEŻKI — Governance jako fundament Rebuildability			
Ścieżka	Rola Governance	Evidence pack	External Review
> EVOLUTION	Rejestracja D0-D2; podstawowy audit trail; compacty rytmiczne	MINIMAL: decision.json + workflow_log	Nie wymagany
> REFACTOR	Evidence D3 z 4/4 Council votes; compacty pre-D3; śledzenie dwóch równoległych linii	STANDARD: + council_votes + compacts	Nie wymagany (opcjonalny dla D4)
> REBUILD	Pełny Evidence pack D5: wszystkie artefakty; external_review_log; adversarial; post_mortem; D5AuditStream (event sourcing); wieczne archiwum	FULL D5: wszystkie 9 kategorii + seal_hash	WYMAGANY — prawo weta
HARD GATE'Y			

Kod	Warunek blokujący	Akcja
G-GOV-01	Zdarzenie bez prev_hash lub hash niezgodny z łańcuchem (hash chain broken)	reject — zdarzenie odrzucone; alert critical do operatora
G-GOV-02	Decyzja D4 bez podpisu operatora (operator_sig_b64 = null)	block — decyzja nieaktywna; run blocked do podpisu
G-GOV-03	DecisionRecord D3 bez 4/4 podpisów modeli	reject — decyzja D3 nie spełnia wymogu jednomyślności
G-GOV-04	Próba purge zdarzenia bez decyzji D4 lub bez Evidence GDPR	reject — purge niedozwolony; alert do operatora
G-GOV-05	ComplianceReport mode=external bez podpisu PDF	reject — raport zewnętrzny musi być podpisany kryptograficznie
G-GOV-06	Zdarzenie z run_id odwołującym się do nieistniejącego runu (Plan 11)	reject — orphan event, brak kontekstu; log do !syllion-audit
G-GOV-07	Matrix event_id brakujący lub niedostępny (Matrix offline > 5 min)	block — emisja zdarzeń wstrzymana; alert critical; run blocked
G-GOV-08	Decyzja D5 bez ExternalReviewLog lub przed zakończeniem External Review	block — D5 nie może być SEALED bez completed external review
G-GOV-09	External reviewer zwrócił veto (verdict=rejected, veto_used=true)	hard block — D5 odrzucona; Evidence pack SEALED z flagą VETOED; niemożliwe odblokowanie bez nowego cyklu review z nowym review_id
G-GOV-10	Evidence pack D5 niekompletny — brakuje choćby jednego z: council_votes, human_approval, external_review, adversarial/, compacts/	block — D5 nie może przejść do SEALED; lista brakujących artefaktów w logu
G-GOV-11	Próba purge Evidence pack D5 (is_d5_evidence=true)	hard reject — purge D5 evidence niedozwolony; operacja zablokowana nawet z D4
G-GOV-12	BookChangeProposal eskalowany bez decyzji Council D3 (council_decision_id=null)	reject — zmiana Księgi wymaga wstępnej akceptacji Council
TESTY		

Kod	Nazwa	Warunki testu	Oczekiwany wynik
T-GOV-01	Emit event OK	POST /events z prawidłowym prev_hash i payload_sha256, decision_class=D2	201, event_id, hash, matrix_event_id
T-GOV-02	Blokada G-GOV-01	POST /events z błędnym prev_hash (hash chain broken)	400, G-GOV-01, zdarzenie nie zapisane
T-GOV-03	Decyzja D4 bez podpisu	POST /events event_type=decision, class=D4, operator_sig_b64=null	201 ale run blocked, G-GOV-02, alert critical
T-GOV-04	Generate external report	POST /reports/generate mode=external	202; po zakończeniu: pdf_sig_b64 != null
T-GOV-05	D5 bez ExternalReview	Próba SEAL EvidencePack D5 bez external_review_id	409, G-GOV-08, pack pozostaje COLLECTING
T-GOV-06	External Review — veto	POST /d5/review/{id}/submit verdict=rejected, veto_used=true	200; decision D5 blocked; Evidence SEALED+VETOED; alert VETO_RAISED
T-GOV-07	External Review — conditional	POST /d5/review/{id}/submit verdict=conditional, conditions=[...]	200; D5 zawieszona; warunki zapisane; decision_blocked=false (pending conditions)
T-GOV-08	Evidence pack D5 — pełny	Zebranie wszystkich 9 kategorii artefaktów dla D5-2026-001	Pack SEALED, seal_hash obliczony, matrix event w !syllion-d5
T-GOV-09	Compact archive — pre-D5	POST /compacts compact_type=pre-d5, decision_id=D5-2026-001	201, retention_class=infinite, compact skopiowany do EvidencePack D5
T-GOV-10	BookChangeProposal — pełny cykl	Propose → Formalize → Council D3 accept → escalate D4 → Human approve → rebuild	proposal_id BCP-YYYY-NNN, final status=APPROVED, new_book_version=vX.Y
T-GOV-11	Purge D5 evidence — zablokowany	Próba purge pack_id gdzie is_d5_evidence=true, nawet z D4	403, G-GOV-11, purge odrzucony, alert do operatora
T-GOV-12	D5 audit stream — replay	GET /decisions/D5-2026-001/stream?format=jsonl	JSONL z wszystkimi eventami D5_INITIATED → EVIDENCE_SEALED; hash chain OK
T-GOV-13	Retain cron — compacty	Compacty rhythmic starsze niż 24h → 30 dni cold; pre-D3+ bez zmian	rhythmic: przeniesione; pre-D3+: nadal hot, retention_class=infinite

Kod	Nazwa	Warunki testu			Oczekiwany wynik
T-GOV-14	GDPR purge D4	Purge z DecisionRecord D4 i Evidence GDPR (zdarzenie nie-D5)			content usunięte, hash chain zachowany, purged=true, AuditEvent
ANEKS A — Typ zdarzenia × Retention × Evidence					
Typ zdarzenia	Hot (dni)	Cold (lata)	Infinite hash?	Cold storage	Wymagane evidence
run_state_transition	90	2	Tak	S3-compatible	Nie (automatyczny)
gate_failure	90	2	Tak	S3-compatible	Tak (GateEvidence)
decision_D0	90	2	Tak	S3-compatible	Tak (DecisionRecord)
decision_D1	90	2	Tak	S3-compatible	Tak (DecisionRecord + model log)
decision_D2	730	infinite	Tak	S3 + Matrix federation	Tak (DecisionRecord + Council logs)
decision_D3	730	infinite	Tak	S3 + Matrix federation	Tak (DecisionRecord + 4/4 sigs)
decision_D4	infinite	infinite	Tak (Ed25519)	S3 + Matrix + local	Tak (DecisionRecord + operator sig + adversarial)
decision_D5	infinite (hot bez terminowy)	infinite + replika	Tak (Ed25519 + External sig)	S3 + Matrix + local + !syllion-d5	Tak — PEŁNY Evidence pack D5 (9 kategorii)
external_review_log	infinite	infinite	Tak (Ed25519 + External)	S3 + local + !syllion-d5	Tak (ExternalReviewLog — część pack D5)
compact_rhythmics	1	30d cold	Tak	S3-compatible	Nie
compact_pre-bundle	3	30d cold	Tak	S3-compatible	Nie (chyba że bundle D3+)
compact_pre-D3+	infinite	N/A	Tak	Hot bezterminowo	Tak (CompactArchiveEntry)
bundle_promoted	90	2	Tak	S3-compatible	Tak (BundleEvidence)

Typ zdarzenia	Hot (dni)	Cold (lata)	Infinite hash?	Cold storage	Wymagane evidence
bundle_rolled_back	90	2	Tak	S3-compatible	Tak (RollbackEvidence)
book_version	infinite	infinite	Tak	S3 + local + Matrix	Tak (BookVersionEvidence)
compliance_report	730	infinite	Tak	S3 + local backup	Tak (ReportEvidence)
gdpr_purge	infinite	infinite	Tak	S3 + local	Tak (PurgeEvidence + D4)
ANEKS B — Procedura zewnętrznego audytu (10 kroków)					

Procedura obowiązuje przy każdym formalnym audycie zewnętrznym (regulacyjny, compliance, due diligence). Każdy krok jest rejestrowany jako AuditEvent w !syllion-audit.

1. Zgłoszenie audytu: audytor zewnętrzny składa wniosek do operatora SYLION z zakresem i terminem. Operator tworzy DecisionRecord D4 zezwalający na dostęp.
2. Provisioning dostępu: GS tworzy konto read-only z ograniczonym zakresem (D2–D5 events only). Konto ma TTL = czas trwania audytu + 7 dni. Zdarzenie w !syllion-audit.
3. Eksport katalogu zdarzeń: audytor wywołuje GET /governance/v1/decisions z filtrem zakresu. GS zwraca listę DecisionRecords z hash chain.
4. Weryfikacja hash chain: audytor (lub narzędzie) weryfikuje spójność łańcucha SHA-256. Każde zerwanie łańcucha raportowane jako incydent krytyczny.
5. Pobranie dowodów: dla każdej decyzji D3–D5 audytor pobiera Evidence pack przez GET /governance/v1/evidence/{pack_id}. Weryfikacja podpisów Ed25519/GPG. Dla D5: weryfikacja external_review.json i ExternalReviewLog.
6. Inspekcja Matrix log: audytor ma dostęp read-only do !syllion-audit. Weryfikacja, że wszystkie DecisionRecords mają odpowiadające Matrix event_id.
7. Generowanie raportu audytowego: POST /governance/v1/reports/generate mode=external z zakresem audytu. GS generuje PDF z podpisem kryptograficznym.
8. Przekazanie raportu: raport (PDF + JSON) przekazywany audytorowi przez bezpieczny kanał. Hash SHA-256 raportu rejestrowany w !syllion-audit.
9. Zamknięcie sesji: operator odwołuje konto audytora, generuje końcowy AuditEvent zamknięcia. Wszelkie anomalie z audytu przekazywane do Chief Safety i działu Governance.
10. Archiwizacja: wszystkie artefakty audytu (wnioski, raporty, korespondencja) archiwizowane jako Evidence kind=compliance_report z retention infinite.

ANEKS C — Klasy decyzji D0–D5 w kontekście Governance				
Klasa	Przykład operacji Governance	Aktor	Evidence pack	Ścieżka
D0	Odczyt AuditLog, weryfikacja hash chain	GS auto	MINIMAL	—

Klasa	Przykład operacji Governance	Aktor	Evidence pack	Ścieżka
D1	Generowanie compliance report internal	Chief Reviewer	MINIMAL	—
D2	Provisioning konta audytora	Council 3/4	BASIC	evolution
D3	Wstępna ocena BookChangeProposal	Council 4/4	STANDARD	refactor
D4	Zatwierdzenie zmiany Księgi lokalnej; GDPR purge	Human Gate	FULL D4	rebuild
D5	Zatwierdzenie zmiany fundamentów Księgi; pełny rebuild systemu	Council 4/4 + Human + External Review (prawo weta)	FULL D5 — 9 kategorii	rebuild (greenfield)

REBUILD · D5 (#6A1B9A) — Governance jako ostatnia linia obrony

Dział Governance (Plan 15.7) jest instytucjonalnym strażnikiem decyzji D5. Bez kompletnego Evidence pack D5 — obejmującego wyniki external review, adversarial testing, archiwum compactów i post_mortem — decyzja D5 nie może przejść do statusu SEALED. Governance nie może samodzielnie zatwierdzić żadnej decyzji D3+, ale może zablokować każdą decyzję przez zgłoszenie niekompletności Evidence pack (G-GOV-10). External Reviewer posiada prawo weta niezależne od wszystkich innych aktorów.

PLAN 13 · ROZSZERZENIE v3.1 · REVERSIBILITY

Governance & Evidence · v3.1 EXPAND

13.12. Nowe gate'y v3.1: Cutover, Compaction, Fidelity

REVERSIBILITY · Meta-zasada Reversibility & Fidelity egzekwowana przez gate'y

Żadna meta-zasada nie ma wartości bez egzekwującego ją gate'a. v3.1 wprowadza sześć nowych bram decyzyjnych w Planie 13: cztery dla cyklu życia cutover (G-CUT-01..04), jedną dla budżetu compaction (G-COMP-01) i jedną dla wierności compacta (G-CFT-01). Bez nich Rebuildability + Reversibility + Fidelity byłyby tylko hasłami.

13.12.1. Gate'y cyklu życia cutover

ID	Nazwa	Moment	Wymaga	Artefakt wejściowy	Artefakt wyjściowy
G-CUT-01	Cutover Readiness Review	SHADOW_VALIDATING → CUTOVER_READY	Testy + Red Team pass	shadow_validation_report.md	cutover_readiness_certificate.json
G-CUT-02	Cutover Window Approval	CUTOVER_READY → CUTOVER_PENDING	Human Gate (operator + Chief Safety)	cutover_readiness_certificate.json	cutover_window_grant.json
G-CUT-03	Post-Cutover Review	STABILIZATION → WARM_STANDBY	Blue Team + Gov 4/4 attest	stabilization_metrics.json	post_cutover_review_report.md
G-CUT-04	Legacy Decommission	WARM_STANDBY → LEGACY_DECOMMISSION	Council 4/4 + Human + checklist	decommission_checklist.md	decommission_decision_record.md

ZASADA · G-CUT-02 rozdziela gotowość techniczną od decyzji operatorskiej

G-CUT-01 certyfikuje że target jest gotowy technicznie. G-CUT-02 certyfikuje że TERAZ jest dobry moment na cutover (brak incydentu, okno biznesowe, zespół dyżurny na miejscu). Te dwie decyzje mogą być oddalone w czasie dni lub tygodni. G-CUT-01 może być podjęte w czwartek, G-CUT-02 dopiero w kolejny wtorek — to zgodne z kanonem.

13.12.2. Gate'y compaction

ID	Nazwa	Moment	Wymaga	Artefakt wejściowy	Artefakt wyjściowy
G-COMP-01	Compact Budget Review	generacja canon_compact.md	Dział Testów (Compact Validator Agent)	compact_budget.yaml	compact_budget_approval.json

ID	Nazwa	Moment	Wymaga	Artefakt wejściowy	Artefakt wyjściowy
G-CF T-01	Compact Fidelity Test	PRE-bundle, PRE-D3 decision, 24h rhythm	CFT pass (hard metrics) + golden_set parity	canon_compact.md + full_corpus.pdf	compact_fidelity_report.json

FIDELITY · G-CFT-01: jedyna brama zapobiegająca cichej utracie kanonu

Bez G-CFT-01 compact jest proxy kanonu tylko z nazwy. Agent trzyma się compacta nawet gdy ten dryfuje od pełnej Księgi. CFT pass MUSI być warunkiem publikacji compacta do production inject path. Compact który nie przeszedł CFT = compact DRAFT = nie wolno go wstrzykiwać agentom sesyjnie. Szczegółowa procedura w Planie 16 sekcja 16.11.

13.12.3. Nowe artefakty Evidence v3.1

Evidence pack dla decyzji D3+ rozszerza się w v3.1 o cztery nowe artefakty. Bez nich Evidence jest niekompletny i Council nie osiąga quorum dla decyzji D3+.

Artefakt	Format	Generowany przez	Wymagany dla
rollback_plan.md	Markdown	Architekt + Platforma	każda decyzja D3+
post_cutover_review_report.md	Markdown	Blue Team + Governance	każda decyzja D3+ z cutover
compact_fidelity_report.json	JSON (schema w Aneksie I)	Dział Testów (Compact Validator Agent)	każda decyzja D3+ oparta o compact
decommission_decision_record.md	Markdown	Human Gate + Council 4/4	G-CUT-04 trigger

13.12.4. Rollback plan — minimalna struktura

rollback_plan.md jest obowiązkowym artefaktem każdej decyzji D3+ w v3.1. Musi zawierać co najmniej siedem sekcji, aby Council uznał rollback za realistycznie wykonywalny:

```
# rollback_plan.md – minimal skeleton v3.1
## 1. Trigger matrix
- lista warunków automatycznych (mapowanie na Plan 07 sekcja 7.11.2)
- lista warunków manualnych (incydent P0/P1, veto operatora)
## 2. Actor matrix
- kto wydaje rozkaz rollback w każdym scenariuszu
## 3. Pre-rollback verification
- stan legacy (HOT/WARM/DECOMMISSION)
- checklist danych (write-through replication healthy?)
## 4. Rollback sequence
- kroki 1-10 z Planu 07 sekcja 7.11.4 (detect..post-mortem)
## 5. Time budgets
- target: <60s auto; <5min manual fast; <60min council
## 6. Post-rollback cleanup
- target freeze procedure
- post-mortem trigger
## 7. Test evidence
```

- referencje do T-REV-01..06 (Plan 07 sekcja 7.11.6)

13.12.5. *Pattern Authorization Token (parking lot)*

Aneks J wprowadza koncept Pattern Authorization Token (PAT) jako zastępnik odrzuconej rekomendacji Auto-D4. PAT NIE JEST aktywnym artefaktem v3.1 — jest zarchiwizowanym kierunkiem ewolucji dla v3.2+. Plan 13 zastrzega jednak prefiks ID dla przyszłego gate'a G-PAT-01..05 tak aby nie doszło do kolizji numeracji, gdyby PAT został aktywowany.

UWAGA · *Dlaczego PAT jest parking lot, nie aktywny artefakt*

Pattern Authorization Token wymaga infrastruktury pattern-detection (ML lub heurystyki), która sama jest decyzją D4 (jaki system decyduje że agent może samodzielnie wykonać D4?). Wdrożenie PAT wymaga najpierw zbudowania i zcertyfikowania samego detektora patternów, a potem dodania warstwy authorization. To osobny plan D4. v3.1 rezerwuje jedynie nazewnictwo.

PLAN 13 · ROZSZERZENIE v3.3 · EFFICIENCY

*Rejestr nowych gate'ów v3.3 + 4 artefakty Evidence pack***§13.13 — Rejestr gate'ów efficiency + 4 nowe artefakty Evidence pack (v3.3)**

v3.3 dodaje 11 gate'ów efficiency (klasy G-EFF/G-PERF/G-MEM/G-COST) i wymaga 4 nowych artefaktów w Evidence pack dla decyzji D3+. Łączny rejestr gate'ów: 6 (v3.0) + 6 (v3.1) + 11 (v3.3) = 23. Łączna liczba pól Evidence pack: 17 (v3.1) + 4 (v3.3) = 21.

§13.13.1 — Pełny rejestr 23 gate'ów

Poniższa tabela konsoliduje wszystkie gate'y z trzech generacji SYLION. Legenda: • v3.0 — gate'y bazowe (CON/SEC/COMP); • v3.1 — gate'y ciągłości (CUT/CFT); • v3.3 — gate'y efficiency (EFF/PERF/MEM/COST).

ID	Wer sja	Sprawdza	Owner	Klasa min.	Hard/ Soft	Override
G-CON-01	v3.0	Spójność kontraktu	Governance Lead	D2	Hard	Human Gate D4+
G-CON-02	v3.0	Zgodność zależności	Architekt	D2	Hard	Human Gate D4+
G-SEC-01	v3.0	Bezpieczeństwo sekretu	Security Officer	D3	Hard	Brak
G-SEC-02	v3.0	Izolacja uprawnień	Security Officer	D3	Hard	Brak
G-COMP-01v0	v3.0	Kompletność Evidence pack	Governance Evidence Curator	D3	Hard	Brak
G-COMP-02v0	v3.0	Zgodność formatu artefaktów	Governance Evidence Curator	D2	Soft	Governance Lead
G-CUT-01	v3.1	Legacy w stanie HOT/WARM (nie DECOM) przed cutover	Platforma	D3	Hard	Brak
G-CUT-02	v3.1	Post-cutover review zaplanowany ≤ T+14d	Blue Team	D3	Hard	Governance Lead D4+
G-CUT-03	v3.1	Rollback plan zatwierdzony przez Architekt	Architekt	D3	Hard	Brak
G-CUT-04	v3.1	Decommission decision record podpisany 4/4	Council 4/4	D4	Hard	Brak
G-CFT-01	v3.1	Compact Fidelity Test PASS (max 24h)	Compact Validator	D3	Hard	Brak

ID	Wer sja	Sprawdza	Owner	Klasa min.	Hard/ Soft	Override
G-COMP-01v1	v3.1	Evidence pack zawiera artefakty v3.1 (rollback/CFT)	Governance Evidence Curator	D3	Hard	Brak
G-EFF-01	v3.3	Bloat score ≤ prog wymaganego dla klasy	Code Optimizer Agent	D3	Hard	Governance Lead D5
G-EFF-02	v3.3	Cognitive complexity distribution w zakresie norm	Code Optimizer Agent	D3	Hard	Governance Lead D5
G-EFF-03	v3.3	Clone coverage ≤ 15 %	Code Optimizer Agent	D3	Hard	Governance Lead D5
G-EFF-04	v3.3	Brak LLM anti-patternów (klasa CRITICAL)	Code Optimizer Agent	D3	Hard	Brak
G-PERF-01	v3.3	p95 latency ≤ SLO	Performance Monitor	D3	Hard	SRE Lead D4+
G-PERF-02	v3.3	Error rate ≤ 0,1 %	Performance Monitor	D3	Hard	SRE Lead D4+
G-PERF-03	v3.3	Error budget remaining ≥ 10 %	SRE Lead	D3	Hard	External Review D5
G-MEM-01	v3.3	RSS peak ≤ limit kontenera	Memory Auditor	D3	Hard	Platforma D4+
G-MEM-02	v3.3	Leak rate < 1 MB/h (24h window)	Memory Auditor	D3	Hard	Platforma D4+
G-COST-01	v3.3	Monthly burn ≤ 90 % envelope	FinOps Agent	D3	Soft	CFO D4+
G-COST-02	v3.3	Forecast EOM ≤ 110 % envelope	FinOps Agent	D3	Hard	CFO D5

§13.13.2 — Cztery nowe artefakty Evidence pack (v3.3)

Każdy z czterech artefaktów musi być wygenerowany przez właściwą rolę, podpisany cyfrowo i załączony do Evidence pack przed deliberacją Council w decyzjach D3+.

code_efficiency_report.json

```
{
  "$schema": "https://syllion.internal/schemas/v3.3/code_efficiency_report.json",
  "type": "object",
  "required": ["bloat_score", "cognitive_complexity_distribution",
    "clone_coverage", "llm_anti_patterns",
    "refactoring_recommendations", "owner", "signature"],
  "properties": {
    "bloat_score": {
      "type": "number", "minimum": 0, "maximum": 100,
      "description": "Znormalizowany wskaźnik nadmiarowości kodu (0=brak, 100=krytyczny)"
    }
  }
}
```

```

    },
    "cognitive_complexity_distribution": {
      "type": "object",
      "properties": {
        "p50": {"type": "number"}, "p95": {"type": "number"},
        "p99": {"type": "number"}, "max_module": {"type": "string"}
      }
    },
    "clone_coverage": {
      "type": "number", "minimum": 0, "maximum": 100,
      "description": "Procent zduplikowanego kodu w repozytorium"
    },
    "llm_anti_patterns": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["pattern_id", "severity", "location"],
        "properties": {
          "pattern_id": {"type": "string"},
          "severity": {"enum": ["CRITICAL", "HIGH", "MEDIUM", "LOW"]},
          "location": {"type": "string"}
        }
      }
    },
    "refactoring_recommendations": {
      "type": "array",
      "items": {"type": "string"}
    },
    "owner": {"type": "string", "const": "code_optimizer_agent"},
    "signature": {"type": "string",
      "description": "Ed25519 podpis cyfrowy łańcucha hashów artefaktu"}
  }
}

```

performance_baseline.yaml

```

# performance_baseline.yaml – YAML Schema v3.3
type: object
required:
  - p50_ms
  - p95_ms
  - p99_ms
  - throughput_rps
  - error_rate
  - slo
  - error_budget_remaining_min
  - baseline_revision
  - comparison_vs_previous
properties:
  p50_ms: {type: number, description: "Mediana latencji [ms]"}
  p95_ms: {type: number, description: "95. percentyl latencji [ms]"}
  p99_ms: {type: number, description: "99. percentyl latencji [ms]"}
  throughput_rps: {type: number, description: "Przepływowość [req/s]"}
  error_rate: {type: number, minimum: 0, maximum: 1,
    description: "Współczynnik błędów (0.0–1.0)"}
  slo:

```

```

    type: object
    properties:
      target_p95_ms: {type: number}
      availability_pct: {type: number}
    error_budget_remaining_min: {type: number,
      description: "Pozostały error budget [minuty w oknie 30d]"}
    baseline_revision: {type: string,
      description: "ID rewizji punktu odniesienia (np. v3.2-baseline-2026-Q1)"}
    comparison_vs_previous:
      type: object
      properties:
        p95_delta_pct: {type: number}
        throughput_delta_pct: {type: number}
        trend: {enum: [improving, stable, degrading]}

```

memory_profile.json

```

{
  "$schema": "https://syllion.internal/schemas/v3.3/memory_profile.json",
  "type": "object",
  "required": ["rss_peak_mb", "heap_peak_mb", "gc_pause_p99_ms",
    "leak_rate_mb_per_h", "top_allocators", "heap_snapshot_url"],
  "properties": {
    "rss_peak_mb": {
      "type": "number",
      "description": "Szczytowe zużycie RSS [MB] w oknie 24h"
    },
    "heap_peak_mb": {
      "type": "number",
      "description": "Szczytowy heap [MB]"
    },
    "gc_pause_p99_ms": {
      "type": "number",
      "description": "99. percentyl pauzy GC [ms]"
    },
    "leak_rate_mb_per_h": {
      "type": "number",
      "description": "Szacowana szybkość wycieku pamięci [MB/h]"
    },
    "top_allocators": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "module": {"type": "string"},
          "allocated_mb": {"type": "number"},
          "call_stack_top": {"type": "string"}
        }
      },
      "maxItems": 10
    },
    "heap_snapshot_url": {
      "type": "string", "format": "uri",
      "description": "URL do pełnego zrzutu heap (retencja ≥90 dni)"
    }
  }
}

```

```
}
```

cost_envelope.json

```
{
  "$schema": "https://syllion.internal/schemas/v3.3/cost_envelope.json",
  "type": "object",
  "required": ["monthly_envelope_usd", "current_burn_pct",
    "token_input_used", "token_output_used",
    "model_routing_distribution", "cache_hit_rate_pct",
    "forecast_eom_usd"],
  "properties": {
    "monthly_envelope_usd": {
      "type": "number",
      "description": "Miesięczny limit budżetu [$USD]"
    },
    "current_burn_pct": {
      "type": "number", "minimum": 0, "maximum": 200,
      "description": "Procent wykorzystania envelope na dzień raportu"
    },
    "token_input_used": {
      "type": "integer", "description": "Zużyte tokeny wejściowe (miesiąc)"
    },
    "token_output_used": {
      "type": "integer", "description": "Zużyte tokeny wyjściowe (miesiąc)"
    },
    "model_routing_distribution": {
      "type": "object",
      "description": "Rozkład wywołań na modele {model_id: pct}"
    },
    "cache_hit_rate_pct": {
      "type": "number", "minimum": 0, "maximum": 100
    },
    "forecast_eom_usd": {
      "type": "number",
      "description": "Prognoza kosztu na koniec miesiąca [$USD]"
    }
  }
}
```

§13.13.3 — Aktualizacja *schema evidence_pack.yaml*

Manifest *evidence_pack.yaml* rozszerza się o sekcję „efficiency:” z czterema pod-polami. Każde pole zawiera hash podpisu cyfrowego i informację o wymaganym zakresie (required vs optional) w zależności od klasy decyzji.

```
# evidence_pack.yaml – fragment: sekcja efficiency (v3.3)
evidence_pack:
  version: "3.3"
  decision_class: D3 # D2 | D3 | D4 | D5

  efficiency:
    code_efficiency_report:
      path: artefacts/code_efficiency_report.json
```

```

owner: code_optimizer_agent
sha256: "a3f9..."
signature: "Ed25519:base64=="
required_for: [D3, D4, D5]          # D2: optional delta-only
optional_mode: delta_only          # tylko dla D2

performance_baseline:
  path: artefacts/performance_baseline.yaml
  owner: performance_monitor_agent
  sha256: "b71c..."
  signature: "Ed25519:base64=="
  required_for: [D3, D4, D5]
  optional_mode: delta_only

memory_profile:
  path: artefacts/memory_profile.json
  owner: memory_auditor_agent
  sha256: "c2d8..."
  signature: "Ed25519:base64=="
  required_for: [D3, D4, D5]
  optional_mode: delta_only

cost_envelope:
  path: artefacts/cost_envelope.json
  owner: finops_agent
  sha256: "d4e1..."
  signature: "Ed25519:base64=="
  required_for: [D3, D4, D5]
  optional_mode: delta_only

# Warunki wymagalnosci per klasa:
# D2: efficiency optional – jezeli dostarczone, musi byc format-valid
# D3: wszystkie 4 artefakty required (full schema, podpisane)
# D4: jak D3 + Code Optimizer veto check + Memory Auditor sign-off
# D5: jak D4 + perf benchmark + cost envelope external review

```

§13.13.4 — Walidacja Evidence pack przy quorum Council

Przed deliberacją Council, Governance Evidence Curator weryfikuje kompletność sekcji efficiency automatycznie. Brak wymaganego artefaktu lub nieprawidłowy podpis blokuje osiągnięcie quorum.

Klasa	Wymagane artefakty efficiency	Walidacja	Akcja przy braku
D2	Opcjonalne delta-only (jeśli dostarczone — format-valid)	Manual review	Warning (nie blokuje)
D3	4/4 wymagane (pełna schema, podpisane)	Auto signature check + bloat threshold	Quorum BLOCKED do uzupełnienia
D4	4/4 + Code Optimizer veto check + Memory Auditor sign-off	Jak D3 + Memory Auditor sign-off check	Jak D3

Klasa	Wymagane artefakty efficiency	Walidacja	Akcja przy braku
D5	4/4 + perf benchmark + cost envelope sign-off	External review weryfikuje wszystkie 4	External review veto (nieodwracalne)

§13.13.5 — Audit trail i compliance

Każdy artefakt efficiency MUSI być podpisany cyfrowo przez role owner przed dołączeniem do Evidence pack. Architektura podpisu opiera się na łańcuchu hashów zapewniającym niezmiennność historii:

- Hash łańcuchowy: „previous_evidence_hash” → „current_evidence_hash” (immutable chain — każda wersja Evidence pack jest kryptograficznie powiązana z poprzednią).
- SOC 2 Type II compliance — rotacja API key ≤ 90 dni dla agent signing keys (zgodnie z SOC 2 Type II checklist dla AI-native systemów, sekcja Access Management).
- NIST AI RMF MEASURE function — artefakty efficiency mapują się na MEASURE 2.5 (monitorowanie i ewaluacja wydajności modelu) oraz MEASURE 2.6 (analiza ryzyk operacyjnych AI).

EFFICIENCY · Audit trail efficiency: niezmienność i rotacja kluczy

„Podpis cyfrowy artefaktu efficiency bez aktywnego klucza właściwej roli jest nieważny. Rotacja signing key > 90 dni = naruszenie SOC 2 Type II. Council ma obowiązek zwrócić Evidence pack do Owner roli celem ponownego podpisania świeżym kluczem.”

EFFICIENCY · Plan 13 §13.13 — zamknięcie pętli efficiency

Plan 13 §13.13 zamyka pętlę: gate’y wykrywają, role podpisują, Evidence pack archiwizuje, Council weryfikuje przed approve.

PLAN 14 · WARSTWA KANONICZNA

Greenfield Rebuild Orchestration

14.1. Cel i rola Planu 14 w architekturze SYLION

Plan 14 definiuje tryb pracy pipeline oznaczony jako `SYSTEM_REBUILD_MODE` — aktywowany, gdy decyzja ścieżki implementacyjnej wskazuje rebuild: D4 (przebudowa pojedynczego modułu) lub D5 (pełna przebudowa systemu, zmiana architektury bezpieczeństwa). Plan 14 jest odpowiednikiem Planu 08 dla ścieżki rebuild: tak jak Plan 08 zarządza ewolucją Księgi, Plan 14 zarządza pełnym cyklem greenfield — od analizy fitness legacy, przez target architecture, wielozespołowe workstreamy, walidację adversarialną, aż do kontrolowanego cutover i decommission starej linii kodu.

Plan 14 jest nowym planem v3.0 — nie ma odpowiednika w v2.0. Konieczność jego istnienia wynika z zasady nadrzędnej „Rebuildability over Lineage”: MZ 3.5.5 jest WYMIENIALNYM baseline'em implementacyjnym, nie kotwicą architektoniczną. Gdy ewolucja przestaje być bezpieczna lub sensowna, pipeline musi dysponować formalnym procesem zastąpienia całego systemu nową implementacją — bez utraty ciągłości operacyjnej i bez naruszenia intencji Księgi.

REBUILD · Zasada Rebuildability — dlaczego Plan 14 istnieje

Pipeline SYLION NIE jest „patcherem MZ”. Pipeline to organizacja inżynierska zdolna do pełnego greenfield rebuildu. MZ 3.5.5 definiuje aktualny stan implementacyjny — ale Księga definiuje intencje. Jeśli implementacja przestaje być zdolna do bezpiecznego wyrażenia intencji Księgi, pipeline MUSI zastąpić ją nową implementacją. Plan 14 jest protokołem tego zastąpienia.

14.2. Kiedy wchodzimy w `SYSTEM_REBUILD_MODE`

`SYSTEM_REBUILD_MODE` nie jest uruchamiany arbitralnie. Wymagany jest formalny trigger — zatwierdzony przez Plan 08 lub Plan 13 — plus decyzja klasy D4 lub D5. Poniższa tabela opisuje kompletną listę wyzwalaczy:

Wyzwalacz	Opis	Klasa decyzji	Ścieżka
legacy_fitness_fail	Raport legacy_fitness_report.md wykazuje, że kontynuacja MZ 3.5.5 jest niemożliwa (technical debt >= próg, security architecture incompatible with threat model)	D4 / D5	> REBUILD
threat_model_shift	Red Team (Plan 07 / Plan 15.5) stwierdza fundamentalną zmianę modelu zagrożeń, której obecna architektura nie jest w stanie obsłużyć bez przebudowy	D5	> REBUILD

Wyzwalacz	Opis	Klasa decyzji	Ścieżka
book_delta_incompatible	Zmiana Księgi (Plan 08) wymaga zmian architektonicznych niekompatybilnych z obecną strukturą MZ — tj. refactor_path nie wystarczy	D4 / D5	> <i>REBUILD</i>
module_rot	Dział Architektury stwierdza, że moduł osiągnął próg rot score >= 0.80 (zdolność do bezpiecznej ewolucji < 20%)	D4	> <i>REBUILD</i>
operator_mandate	Operator SYLION wydaje jawną decyzję o przebudowie — bez konieczności wcześniejszego triggera automatycznego	D4 / D5	> <i>REBUILD</i>
security_architecture_change	Wymagana zmiana fundamentalnych mechanizmów bezpieczeństwa (auth, isolation, key management) niemożliwa do przeprowadzenia przez refactor	D5	> <i>REBUILD</i>

Wyzwalacz jest formalnym sygnałem — nie jest sam w sobie decyzją o rebuild. Po wyzwalaczu następuje etap `analyze_legacy_fitness` (krok 1 workflow), a dopiero jego wynik trafia do gate'u decyzyjnego `decide_reuse_vs_rebuild`.

14.3. Trzy ścieżki implementacyjne — rola Planu 14

Ścieżka	Strategia	Wymagana klasa	Plan odpowiedzi
> <i>EVOLUTION</i>	reuse · patch · extend · migrate — MZ 3.5.5 ewoluuje	D0–D2	Plan 08 (Canonical Evolution)
> <i>REFACTOR</i>	cut module · reimplementacja · contracts preserved · gradual cutover	D3 — Council 4/4	Plan 08 + Plan 12
> <i>REBUILD</i>	nowy target architecture · greenfield modules · dwie linie równoległe · controlled cutover → decommission	D4 (moduł) / D5 (system) + Human + External Review	Plan 14 (ten plan)

14.4. Dwie linie równoległe — architektura cutover

Zasadniczą cechą ścieżki rebuild jest to, że pipeline utrzymuje dwie aktywne linie kodu przez cały czas trwania procesu: od chwili aktywacji `SYSTEM_REBUILD_MODE` do ukończenia `legacy_decommission`. Żadna z linii nie jest porzucana przedwcześnie — decyzja o finalnym przełączeniu wymaga przejścia przez gate D4.

ASCII	
SYSTEM_REBUILD_MODE — dwie linie równoległe	
LEGACY LINE (MZ 3.5.5)	TARGET LINE (greenfield)
manifest: legacy_manifest.yaml	manifest: target_manifest.yaml
VPS: legacy-env (Plan 02)	VPS: target-env (Plan 02)

bundle: legacy_bundle	bundle: target_bundle
runs: legacy_run_track	runs: target_run_track
kod: /src/legacy/	kod: /src/greenfield/

Cutover strategy (per moduł):

- parallel → obie linie aktywne, ruch podzielony
- shadow → target przyjmuje ruch w trybie obserwacji
- cut-and-replace → atomowe przełączenie (D4 checkpoint)

Plan 02 zarządza VPS states dla obu linii.
Plan 11 zarządza multi-track runs (legacy_track + target_track).

Obie linie posiadają własne: manifesty YAML, środowiska VPS (Plan 02), bundle'e (Plan 12), ślady runów (Plan 11 multi-track) i drzewa kodu. Governance (Plan 13) loguje zdarzenia z obu linii z jawnym atrybutem line: legacy | target. Compaction (Plan 16) generuje compact z obu linii razem.

14.5. Workflow — 8 kroków SYSTEM_REBUILD_MODE

Poniższy diagram pokazuje sekwencję kroków. Każdy krok ma zdefiniowany input, output i gate wejściowy/wyjściowy.

WORKFLOW

```

analize_legacy_fitness      [Architektura + Governance + Red Team]
|  output: legacy_fitness_report.md
▼
decide_reuse_vs_rebuild     [Council 4/4 + HUMAN GATE — D4/D5]
|  output: rebuild_decision_report.md
|  gate: jeśli D3 → Plan 08; jeśli D4/D5 → SYSTEM_REBUILD_MODE
▼
create_greenfield_plan     [Dział Architektury]
|  output: target_architecture_delta.json, greenfield_module_map.json
▼
spawn_multi_team_workstreams [7 działów równolegle → Plan 15]
|  output: workstream_status per dział
▼
run_adversarial_validation  [Red Team × Blue Team]
|  output: adversarial_validation_report.md
▼
promote_or_reject           [Council 4/4 + Human + External Review (D5)]
|  output: promotion_decision.md
▼
cutover_execution           [Dział Platformy + Dział Implementacyjny]
|  output: cutover_strategy.md (wykonany)
|  gate: D3 (Council) + checkpoint D4 przed finalnym przełączeniem
▼
legacy_decommission        [Dział Platformy + Governance]
|  output: decommission_report.md

```

14.6. Interakcje z innymi planami

<i>Plan</i>	<i>Rola w kontekście Planu 14</i>	<i>Typ interakcji</i>
Plan 02 — VPS	Zarządza stanami środowisk VPS dla obu linii (legacy-env, target-env). Rozszerzone o stany rebuild/cutover. Provisioning nowego target-env na początku workstream.	reads / triggers
Plan 05 — Model Engine	Wagi Board podczas walidacji adversarialnej mogą być dostosowane do trybu rebuild (wyższy udział Red Team i Governance).	config
Plan 08 — Canonical Evolution	Źródło sygnałów dla SYSTEM_REBUILD_MODE. Gdy Plan 08 stwierdzi incompatible delta Księgi, emituje trigger do Planu 14. Po decommission — Plan 08 rejestruje nowy baseline.	trigger / notified
Plan 11 — Run Lifecycle	Multi-track runs: równoległe uruchomienie legacy_run_track i target_run_track. Plan 11 dostarcza metryki porównawcze obu linii do promote_or_reject.	parallel runs
Plan 12 — Bundle + Testing	Red/Blue/Purple bundles dla linii target. Adversarial contracts i testy regresyjne wobec linii legacy jako punktu referencyjnego.	bundle build
Plan 13 — Governance & Evidence	Evidence pack dla D5 (external_review_log.md). Pełna ścieżka audytu rebuild — od triggera do decommission. Accept/reject/defer log dla każdej decyzji D4/D5.	evidence / audit
Plan 15 — Multi-Team Organization	spawn_multi_team_workstreams deleguje do 7 działów (Plan 15). Plan 15 definiuje role, modele, wagi i kanały komunikacji w trybie rebuild.	orchestrates
Plan 16 — Compaction	Pre-D3/D4/D5 compact generowany przed każdym gate'em decyzyjnym w workflow Planu 14. Operator otrzymuje operator-compact z obecnym stanem obu linii.	compaction triggers

PLAN 14 · SPECYFIKACJA WYKONAWCZA

Greenfield Rebuild Orchestration

CEL

Specyfikacja definiuje implementację serwisu rebuild-orchestrator — komponentu pipeline odpowiedzialnego za: (a) przyjęcie triggera SYSTEM_REBUILD_MODE i uruchomienie analizy fitness legacy, (b) zarządzanie gate'ami decyzyjnymi D4/D5 przez cały cykl rebuild, (c) koordynację 7 działów (Plan 15) w trybie wielozespołowych workstreamów, (d) obsługę dwóch równoległych linii kodu i środowisk VPS, (e) przeprowadzenie walidacji adversarialnej (Red × Blue), (f) zarządzanie cutover w trzech strategiach (parallel / shadow / cut-and-replace), (g) stopniowe wycofanie legacy i zamknięcie SYSTEM_REBUILD_MODE, (h) logowanie pełnej ścieżki evidence do Plan 13 (D5: external_review_log.md).

MODEL — DECYZYJNY, KOSZTY, BLAST RADIUS

Klasy decyzji w trybie rebuild

Klasa	Nazwa	Zakres rebuild	Gate	External Review
D0	Informational	Odczyt raportów, brak zmian	Auto	NIE
D1	Trivial	Dokumentacja, logi, kosmetyka	1 agent	NIE
D2	Standard	Zmiana konfiguracji target-env, drobna modyfikacja modułu greenfield	2 agenci + Review	NIE
D3	Significant	Zmiana target architecture w trakcie rebuild (mid-course correction). Cutover execution (Council). Checkpoint przed finalnym przełączeniem.	Full Board Council 4/4	NIE
D4	Critical	Rebuild pojedynczego modułu. Aktywacja SYSTEM_REBUILD_MODE dla modułu. Finalne przełączenie (cutover checkpoint).	Council 4/4 + Human	NIE
D5	Greenfield/Systemic	Pełny rebuild systemu (wiele modułów). Zmiana architektury bezpieczeństwa. Zmiana fundamentów Księgi.	Council 4/4 + Human + External Review	TAK (obowiązkowy)

Model kosztów rebuild

Rebuild jest najdroższą ścieżką implementacyjną. Plan 10 (Cost Governance) definiuje budżet per ścieżka — Plan 14 jest zobowiązany do raportowania kosztu każdego etapu do Cost Governance w celu porównania z budżetem rebuild_budget. Kluczowe metryki kosztów:

- Koszt compute: dwie linie równoległe × czas trwania cutover
- Koszt pracy działów: 7 działów × workstream_duration
- Koszt external review (D5): czas zewnętrznego audytora
- Koszt oportunistyczny: opóźnienie funkcji w trakcie rebuild

- Próg kosztowy: rebuild nie może przekroczyć 3× kosztu refactor bez D5 re-approval

Blast radius — ocena zasięgu zmian

Każdy rebuild jest klasyfikowany według blast_radius — zasięgu wpływu na istniejące systemy, agentów, kontrakty i Księgę. Blast radius jest obliczany przez Dział Architektury na etapie analize_legacy_fitness i stanowi kluczowy argument w gate D4/D5.

Poziom blast radius	Opis	Minimalna klasa decyzji
LOCAL	Zmiana izolowana do jednego modułu; brak zmiany kontraktów z innymi modułami; brak wpływu na Księgę	D4
CROSS-MODULE	Zmiana wpływa na 2–5 modułów; możliwa zmiana kontraktów wewnętrznych; ograniczony wpływ na Księgę	D4 + uzasadnienie rozszerzone
SYSTEMIC	Zmiana dotyczy > 5 modułów lub warstwy bezpieczeństwa; zmiana kontraktów zewnętrznych; możliwa zmiana Księgi	D5 + External Review
ARCHITECTURAL	Zmiana fundamentów systemu (auth, isolation, modelu danych); pełna wymiana MZ; zmiana zasad Księgi	D5 + External Review + Human obowiązkowy
ROLE		
Dział / Rola	Odpowiedzialność w trybie rebuild	Etap główny
Dział Architektury (Plan 15.1)	Analiza legacy fitness, projektowanie target architecture, ocena blast radius, delta Księgi, finalna walidacja kontraktów	krok 1, 3
Dział Platformy (Plan 15.2)	Provisioning target-env (VPS), pipeline bundle dla linii target, wykonanie cutover, decommission legacy-env	krok 4, 7, 8
Dział Implementacyjny (Plan 15.3)	Budowa modułów greenfield, kompatybilność kontraktowa, migracje danych, integracja z target manifest	krok 4
Dział Testów i Walidacji (Plan 15.4)	Testy kontraktowe i regresyjne wobec linii legacy, validation gates przed promote_or_reject	krok 4, 5
Dział Red Team (Plan 15.5)	Analiza legacy fitness (perspektywa bezpieczeństwa), adversarial scenarios na nowej architekturze, penetracja threat model, red-team raport	krok 1, 5
Dział Blue Team (Plan 15.6)	Obrona nowej architektury, telemetria target, SOC/DFIR playbooki dla nowej linii, odpowiedź na Red Team	krok 5
Dział Governance (Plan 15.7)	Zgodność z Księgą na każdym etapie, evidence pack dla D5, external_review_log.md, accept/reject/defer log decyzji	krok 2, 6, 8
Council (Board 4/4)	Gate decyzyjny D3+ na każdym etapie: decide_reuse_vs_rebuild, promote_or_reject, cutover_execution checkpoint	krok 2, 6, 7

Dział / Rola	Odpowiedzialność w trybie rebuild	Etap główny
Human (Operator SYLION)	Obowiązkowy Human Gate dla D4 i D5. Veto ostateczne. Zatwierdza external review dla D5.	krok 2, 6, 7
External Reviewer (D5 only)	Niezależny podmiot (drugi operator lub audytor zewnętrzny). Przegląda target architecture i adversarial validation. Wymagany tylko dla D5.	krok 6
WORKFLOW — 8 KROKÓW SZCZEGÓŁOWO		

Krok 1 — *analyze_legacy_fitness*

Cel: obiektywna ocena, czy MZ 3.5.5 (lub konkretny moduł) nadaje się do dalszej ewolucji. Ocena jest wykonywana przez trzy działy niezależnie, a ich wyniki są konsolidowane do raportu *legacy_fitness_report.md*.

- 1. Dział Architektury: analiza technical debt, modularności, dependency graph, zdolności do ewolucji per moduł. Wynik: *architecture_fitness_score* (0.0–1.0 per moduł).
- 2. Dział Governance: weryfikacja zgodności MZ 3.5.5 z aktualną Księgą. Lista punktów niezgodności (*divergence_points[]*). Ocena możliwości naprawy przez Evolution lub Refactor path.
- 3. Dział Red Team: analiza zagrożeń dla obecnej architektury. Threat model assessment. Lista niemożliwych do mitygacji zagrożeń bez rebuild (*unmitigable_threats[]*).
- 4. Konsolidacja wyników do *legacy_fitness_report.md* (struktura: executive summary + moduły + rekomendacja).
- 5. Obliczenie *overall_fitness_score* jako ważona średnia trzech komponentów: *architecture* (0.4) + *governance* (0.35) + *security* (0.25).
- 6. Próg decyzji: *fitness_score* <= 0.40 → rekomendacja rebuild. Raport przekazywany do kroku 2 (*decide_reuse_vs_rebuild*).
- 7. Gate wejściowy kroku 1: aktywny trigger *SYSTEM_REBUILD_MODE* (patrz tabela wyzwalaczy 14.2). Gate wyjściowy: *legacy_fitness_report.md* podpisany przez wszystkie trzy działy.

Schemat legacy_fitness_report.md

```
YAML

# Legacy Fitness Report — SYLION Pipeline
version: "3.0"
generated_at: "<ISO8601>"
trigger: "<wyzwalacz>"
scope: "system | module:<id>"

executive_summary:
  overall_fitness_score: 0.00    # 0.0 = nieżywy, 1.0 = w pełni zdrowy
  recommendation: "rebuild | refactor | evolve"
  blast_radius: "LOCAL | CROSS-MODULE | SYSTEMIC | ARCHITECTURAL"
  decision_class_required: "D4 | D5"

modules:
  - id: "<moduł>"
```

```

current_baseline: "MZ 3.5.5"
architecture_fitness_score: 0.00
governance_fitness_score: 0.00
security_fitness_score: 0.00
module_fitness_score: 0.00
recommendation: "reuse | refactor | replace | greenfield"
rationale: "<uzasadnienie>"

divergence_points:
- point: "<opis niezgodności z Księgą>"
  chapter: "plan-<N>"
  severity: "critical | high | medium | low"

unmitigable_threats:
- threat: "<zagrożenie>"
  reason: "<dlaczego nie do mitygacji bez rebuild>"
  cve_or_ref: "<CVE lub referencja>"

signatories:
  architecture: "<agent_id>"
  governance: "<agent_id>"
  red_team: "<agent_id>"
  signed_at: "<ISO8601>"

```

Krok 2 — *decide_reuse_vs_rebuild*

Cel: Council 4/4 podejmuje formalną decyzję ścieżki na podstawie legacy_fitness_report.md. To jest Human Gate — bez zatwierdzenia operatora SYLION SYSTEM_REBUILD_MODE nie jest aktywowany.

1. Plan 16 generuje pre-D4/D5 compact (operator-compact) przed gate'em. Operator otrzymuje skondensowany stan systemu.
2. Council 4/4 głosuje: (a) D3 → Refactor path (Plan 08 + Plan 12); (b) D4 → Rebuild modułu (SYSTEM_REBUILD_MODE lokalny); (c) D5 → Rebuild systemu (SYSTEM_REBUILD_MODE pełny).
3. Jeśli D4/D5: Human Gate — Operator SYLION zatwierdza lub vetuje. Veto → archiwum (ponowne złożenie możliwe po 30 dniach).
4. Jeśli D5: aktywowane żądanie External Review — niezależny podmiot musi być wyznaczony przed przejściem do kroku 3.
5. Output: rebuild_decision_report.md podpisany przez Council, operatora i (D5) external reviewer.
6. Aktywacja flagi pipeline: SYSTEM_REBUILD_MODE=active, rebuild_scope: module | system, decision_class: D4 | D5.

Krok 3 — *create_greenfield_plan*

Cel: Dział Architektury projektuje target architecture. Output: dwa artefakty JSON — delta i mapa modułów.

1. Analiza Księgi (aktualny stan z Plan 16 compact) — identyfikacja wymagań architektury docelowej.

2. Projektowanie target architecture: moduły, interfejsy, kontrakty, stack technologiczny, architektura bezpieczeństwa.
3. Generowanie target_architecture_delta.json — delta od MZ 3.5.5 do target (co zostaje, co odpada, co nowe).
4. Generowanie greenfield_module_map.json — mapa każdego modułu: strategię (reuse/refactor/replace/greenfield), responsible_team, deadline, contracts (wejście/wyjście), zależności.
5. Review przez Dział Governance — weryfikacja zgodności z Księgą. D3 approval dla planu (Council).
6. Review przez Dział Red Team — analiza nowej architektury pod kątem zagrożeń przed rozpoczęciem implementacji.

Krok 4 — *spawn_multi_team_workstreams*

Cel: uruchomienie równoległych workstreamów w 7 działach (Plan 15). Plan 14 koordynuje workstreamy jako orchestrator — nie wykonuje implementacji samodzielnie.

1. rebuild-orchestrator generuje WorkstreamManifest per dział na podstawie greenfield_module_map.json.
2. Provisioning target-env przez Dział Platformy (Plan 02 — nowe stany VPS dla linii target).
3. Dział Implementacyjny: budowa modułów greenfield per mapa. Testy jednostkowe i kontraktowe na bieżąco.
4. Dział Testów: testy regresyjne wobec linii legacy jako point-of-reference. Walidacja kontraktów.
5. Dział Platformy: konfiguracja bundle pipeline dla linii target (Plan 12). Konfiguracja multi-track run (Plan 11).
6. Dział Governance: monitorowanie zgodności z Księgą w trakcie budowy. Logi D3+ decyzji mid-course.
7. Synchronizacja: rebuild-orchestrator sprawdza status workstreamów w rytmie konfigurowalnym (domyślnie co 24h). Blokady → eskalacja D3 (Council).
8. Mid-course correction: zmiana target architecture w trakcie = D3. Zmiana fundamentalna = D4. Zmiana architektury bezpieczeństwa = D5.

Krok 5 — *run_adversarial_validation*

Cel: Dział Red Team i Dział Blue Team testują nową architekturę przed promocją do baseline. Jest to gate adversarialny — bez pozytywnego przejścia promote_or_reject nie może nastąpić.

1. Red Team: zdefiniowanie scenariuszy adversarialnych wobec target architecture (threat model, penetracja, edge cases, ataki na kontrakty).
2. Red Team: wykonanie scenariuszy w target-env (uruchomienie red_bundle — Plan 12). Dokumentacja findings.
3. Blue Team: analiza findings Red Team. Wdrożenie obrony i mitigacji w target architecture. Odpowiedź na każde finding.
4. Blue Team: walidacja, że nowa architektura jest odporna na zagrożenia zidentyfikowane jako unmitigable dla legacy.

5. Dział Testów: testy porównawcze legacy vs target (metryki funkcjonalne, performance, security score).
6. Konsolidacja wyników do `adversarial_validation_report.md`. Wymagane: (a) wszystkie P0-P1 findings odparte przez Blue Team; (b) target architecture poprawia security score vs legacy.
7. Gate: jeśli findings nieodparte → powrót do kroku 4 (workstreamy). Jeśli gate przejdzie → krok 6.

Krok 6 — *promote_or_reject*

Cel: finalna decyzja o promocji target architecture do baseline. Council 4/4 + Human Gate (D4/D5). Dla D5 obowiązkowy External Review.

1. Plan 16 generuje pre-D5 compact z pełnym stanem obu linii i wynikami adversarial validation.
2. Council 4/4 przegląda: `legacy_fitness_report`, `target_architecture_delta`, `adversarial_validation_report`, metryki porównawcze legacy vs target.
3. D5: External Reviewer dostarcza zewnętrzną ocenę target architecture. Wynik trafia do `external_review_log.md`.
4. Głosowanie Council 4/4: approve / reject / hold. Chief Safety ma prawo veto jeśli target architecture narusza inwarianty bezpieczeństwa.
5. Human Gate: Operator SYLION zatwierdza promote. Veto → `SYSTEM_REBUILD_MODE` wraca do kroku 3 (nowy plan) lub całkowita anulacja rebuildu.
6. Output: `promotion_decision.md` z uzasadnieniem, głosami Council, decyzją operatora i (D5) wynikiem external review.

Krok 7 — *cutover_execution*

Cel: kontrolowane przełączenie ruchu z linii legacy na linię target. Plan 14 obsługuje trzy strategie cutover, wybrane per moduł w `greenfield_module_map.json`.

1. Generowanie `cutover_strategy.md` przez Dział Platformy na podstawie `greenfield_module_map.json` (strategie per moduł: parallel / shadow / cut-and-replace).
2. parallel — obie linie aktywne, ruch podzielony (np. 10% target, 90% legacy). Stopniowe zwiększanie udziału target. Gate: każda zmiana podziału $\geq 25\%$ = D3 (Council).
3. shadow — target przyjmuje ruch w trybie obserwacji: przetwarza żądania, ale odpowiedź wraca z legacy. Porównanie wyników: divergence $> 5\%$ = stop i analiza.
4. cut-and-replace — atomowe przełączenie (stosowane dla modułów stateless lub po shadow/parallel fazach). Wymaga D4 checkpoint przed wykonaniem.
5. Monitoring w trakcie cutover: Plan 11 (multi-track metrics), Plan 02 (VPS health), Plan 13 (evidence każdego kroku).
6. Rollback gate: jeśli `error_rate target` $> 2 \times$ `error_rate legacy` w trakcie cutover → automatyczny rollback do legacy, alert D3 do Council.
7. Checkpoint D4: przed finalnym przełączeniem ostatniego modułu → Human Gate obowiązkowy niezależnie od strategii.

Strategie cutover — tabela

Strategia	Kiedy stosować	Czas trwania	Gate decyzyjny	Rollback
parallel	Moduły stanowe, krytyczne ścieżki, duże ryzyko. Wymaga duplikacji zasobów.	dni–tygodnie	D3 (zmiana procentu) + D4 (finalne)	Natychmiastowy — przywrócenie 100% legacy
shadow	Moduły z trudnym do walidacji zachowaniem. Walidacja wyników bez ekspozycji użytkownika.	godziny–dni	D3 (wejście w shadow) + D4 (wyłączenie shadow)	Wyłączenie trybu shadow — legacy pozostaje aktywne
cut-and-replace	Moduły stateless lub po pełnej walidacji shadow. Najszybszy, ale najwyższe ryzyko.	minuty	D4 obowiązkowy przed wykonaniem	Rollback bundle (Plan 12) — przywrócenie legacy bundle

Krok 8 — legacy_decommission

Cel: stopniowe i bezpieczne wycofanie starej linii kodu, środowisk i zasobów. Legacy nie jest usuwane atomowo — wycofanie jest procesem z gate'ami i retention window.

- 1. Po finalnym cutover (krok 7): legacy-env przechodzi w stan FROZEN (Plan 02 VPS state). Brak nowego ruchu, ale środowisko zachowane przez retention_window (domyślnie 30 dni).
- 2. Retention window: Plan 11 i Plan 13 logują wszelkie zdarzenia w legacy-env (potencjalne powody rollbacku). Brak incydentów przez retention_window → decommission.
- 3. Decommission checklist (Dział Platformy): (a) backup danych legacy; (b) archiwizacja legacy_manifest.yaml; (c) archiwizacja legacy_bundle; (d) snapshot VPS; (e) usunięcie środowiska legacy-env.
- 4. Dział Governance: zamknięcie SYSTEM_REBUILD_MODE. Aktualizacja implementation_strategy w manifest dla wszystkich przebudowanych modułów: greenfield → reuse (nowy baseline).
- 5. Plan 08: rejestracja nowego baseline implementacyjnego. Nowa wersja semver baseline (nie Księgi, ale implementation baseline).
- 6. Emisja zdarzenia: rebuild.completed z full evidence pack do Plan 13. Compaction (Plan 16): operator-compact z nowym stanem systemu.
- 7. Output: decommission_report.md z checklistą, datami, archiwami i potwierdzeniem Plan 13.

API / KONTRAKTY — PEŁNE SCHEMATY

Artefakt 1 — rebuild_decision_report.md (schemat)

```
YAML

# Rebuild Decision Report — SYLION Pipeline v3.0
version: "3.0"
report_id: "<UUID>"
generated_at: "<ISO8601>"
decision_class: "D4 | D5"
rebuild_scope: "module | system"

trigger:
```

```

type: "legacy_fitness_fail | threat_model_shift | book_delta_incompatible
      | module_rot | operator_mandate | security_architecture_change"
source: "Plan 08 | Plan 13 | Operator"
triggered_at: "<ISO8601>"

legacy_fitness_summary:
  overall_fitness_score: 0.00
  recommendation: "rebuild"
  blast_radius: "SYSTEMIC"
  divergence_points_count: 0
  unmitigable_threats_count: 0
  report_ref: "legacy_fitness_report.md"

council_vote:
  members:
    - role: "Chief Context"
      vote: "approve | reject | abstain"
      weight: 0.00
      rationale: "<uzasadnienie>"
    - role: "Chief Reasoning"
      vote: "approve"
      weight: 0.00
      rationale: "<uzasadnienie>"
    - role: "Chief Review"
      vote: "approve"
      weight: 0.00
      rationale: "<uzasadnienie>"
    - role: "Chief Safety"
      vote: "approve"
      weight: 0.00
      rationale: "<uzasadnienie>"
  result: "approve | reject | hold"
  voted_at: "<ISO8601>"

human_gate:
  operator_id: "<operator_id>"
  decision: "approve | veto"
  rationale: "<uzasadnienie operatora>"
  decided_at: "<ISO8601>"

external_review:                # tylko D5
  reviewer_id: "<external_id>"
  reviewer_type: "second_operator | external_auditor"
  verdict: "approved | conditional | rejected"
  conditions: []                 # lista warunków jeśli conditional
  reviewed_at: "<ISO8601>"
  log_ref: "external_review_log.md"

rebuild_mode_activated: true
activated_at: "<ISO8601>"
rebuild_id: "<UUID>"

```

Artefakt 2 — *target_architecture_delta.json* (pełny schemat z przykładem)

JSON

```
{
  "schema_version": "3.0",
```

```
"rebuild_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
"generated_at": "2026-04-15T10:00:00Z",
"from_baseline": "MZ 3.5.5",
"to_baseline": "MZ 4.0.0-greenfield",
"blast_radius": "SYSTEMIC",
"book_version_ref": "3.1.0",

"modules_retained": [
  {
    "id": "secrets-vault",
    "strategy": "reuse",
    "rationale": "Vault spełnia wymagania nowej architektury"
  }
],

"modules_refactored": [
  {
    "id": "knowledge-base",
    "strategy": "refactor",
    "contracts_preserved": true,
    "changes_summary": "Nowe schema versioning, legacy KB view"
  }
],

"modules_replaced": [
  {
    "id": "model-engine",
    "strategy": "replace",
    "contracts_preserved": false,
    "breaking_changes": ["scoring API v2", "weight schema v3"],
    "migration_path": "adapter layer v1->v2 przez 30 dni"
  }
],

"modules_greenfield": [
  {
    "id": "agent-runtime-v2",
    "strategy": "greenfield",
    "replaces": "agent-runtime-v1",
    "new_contracts": {
      "input": "AgentTaskV2",
      "output": "AgentResultV2",
      "schema_ref": "contracts/agent_runtime_v2.yaml"
    },
    "responsible_team": "Dział Implementacyjny",
    "deadline": "2026-05-15T00:00:00Z",
    "cutover_strategy": "shadow"
  },
  {
    "id": "security-layer-v2",
    "strategy": "greenfield",
    "replaces": "security-layer-v1",
    "new_contracts": {
      "input": "AuthRequestV2",
      "output": "AuthTokenV2",
      "schema_ref": "contracts/security_v2.yaml"
    },
    "responsible_team": "Dział Architektury + Red Team",
```

```

    "deadline": "2026-05-01T00:00:00Z",
    "cutover_strategy": "parallel"
  }
],

"security_architecture_delta": {
  "auth_model": {"from": "shared-secret", "to": "mTLS + JWT"},
  "isolation_model": {"from": "process-level", "to": "container-level"},
  "key_management": {"from": "vault-v1", "to": "vault-v1 (retained)"},
},

"book_delta": {
  "chapters_affected": ["plan-02", "plan-05", "plan-06"],
  "new_invariants": ["mTLS required for all inter-module calls"],
  "removed_invariants": ["shared-secret allowed for trusted zones"]
}
}

```

Artefakt 3 — *greenfield_module_map.json* (pełny schemat z przykładem)

JSON

```

{
  "schema_version": "3.0",
  "rebuild_id": "f47ac10b-58cc-4372-a567-0e02b2c3d479",
  "generated_at": "2026-04-15T11:00:00Z",
  "modules": [
    {
      "id": "agent-runtime-v2",
      "strategy": "greenfield",
      "responsible_team": "Dział Implementacyjny",
      "deadline": "2026-05-15T00:00:00Z",
      "cutover_strategy": "shadow",
      "status": "planned | in_progress | build_complete | validated
                | cutover_ready | cutover_done | decommissioned",
      "contracts": {
        "consumes": [
          {"module": "knowledge-base", "contract": "KBQueryV2"}
        ],
        "provides": [
          {"contract": "AgentResultV2", "consumers": ["run-lifecycle"]}
        ]
      },
      "dependencies": ["security-layer-v2", "knowledge-base"],
      "tests_required": [
        "unit: coverage >= 90%",
        "contract: AgentResultV2 backward-compat",
        "adversarial: red_team_pass"
      ],
      "rollback_plan": "revert to agent-runtime-v1 bundle",
      "vps_target_env": "target-env-001"
    },
    {
      "id": "security-layer-v2",
      "strategy": "greenfield",
      "responsible_team": "Dział Architektury + Red Team",
      "deadline": "2026-05-01T00:00:00Z",
      "cutover_strategy": "parallel",
    }
  ]
}

```

```

    "status": "planned",
    "contracts": {
      "consumes": [
        {"module": "secrets-vault", "contract": "VaultReadV1"}
      ],
      "provides": [
        {"contract": "AuthTokenV2",
         "consumers": ["agent-runtime-v2", "run-lifecycle"]}
      ]
    },
    "dependencies": ["secrets-vault"],
    "tests_required": [
      "unit: coverage >= 95%",
      "penetration: red_team_full",
      "adversarial: mTLS handshake failures"
    ],
    "rollback_plan": "switch parallel back to 100% security-layer-v1",
    "vps_target_env": "target-env-001"
  }
]
}

```

Artefakt 4 — *cutover_strategy.md* (schemat)

YAML

```

# Cutover Strategy – SYLION Pipeline v3.0
version: "3.0"
rebuild_id: "<UUID>"
generated_at: "<ISO8601>"
approved_by: "Council 4/4 + Human Gate"

phases:
- phase: 1
  name: "Pre-cutover validation"
  owner: "Dział Testów + Dział Platformy"
  checklist:
    - "adversarial_validation_report.md: passed"
    - "target-env health check: all green"
    - "multi-track run: target metrics within 5% of legacy"
    - "rollback plan verified (plan 12 bundle ready)"
  gate: "D3 – Council approve before phase 2"

- phase: 2
  name: "Shadow / parallel cutover"
  owner: "Dział Platformy"
  modules_in_phase:
    - id: "security-layer-v2"
      strategy: "parallel"
      traffic_split_start: "10% target / 90% legacy"
      traffic_split_end: "100% target / 0% legacy"
      step_size: "10% per 24h"
      divergence_threshold: "2% error rate increase = stop"
    - id: "agent-runtime-v2"
      strategy: "shadow"
      shadow_duration: "48h"
      divergence_threshold: "5% result difference = stop"
  gate: "D3 dla każdego kroku split >= 25%; D4 przed 100%"

```

```

- phase: 3
  name: "Final cutover – cut-and-replace (stateless modules)"
  owner: "Dział Platformy + Dział Implementacyjny"
  modules_in_phase:
    - id: "<moduł stateless>"
      strategy: "cut-and-replace"
      rollback_window: "15min"
  gate: "D4 – Human Gate OBOWIĄZKOWY przed wykonaniem"

- phase: 4
  name: "Stabilization & monitoring"
  owner: "Dział Platformy + Dział Governance"
  duration: "72h"
  monitoring:
    - "Plan 11: multi-track metrics – legacy FROZEN, target active"
    - "Plan 02: VPS health legacy-env (FROZEN state)"
    - "Plan 13: evidence logging każdego zdarzenia"
  gate: "Brak incydentów przez 72h → proceed to legacy_decommission"

rollback_policy:
  trigger: "error_rate target > 2x error_rate legacy"
  action: "automatyczny rollback do legacy_bundle (Plan 12)"
  decision_class: "D3 (alert) + D4 (jeśli rollback trwa > 1h)"
  max_rollback_time: "15min"
  notifications: ["Council", "Operator", "Plan 13"]

```

Artefakt 5 — *external_review_log.md* (schemat — tylko D5)

YAML

```

# External Review Log – SYLION Pipeline v3.0 – D5 only
version: "3.0"
rebuild_id: "<UUID>"
review_id: "<UUID>"
reviewer:
  id: "<external_reviewer_id>"
  type: "second_operator | external_auditor | red_team_external"
  organization: "<org>"
  assigned_at: "<ISO8601>"

scope_reviewed:
- "target_architecture_delta.json"
- "adversarial_validation_report.md"
- "greenfield_module_map.json"
- "legacy_fitness_report.md"

findings:
- id: "EXR-001"
  severity: "critical | high | medium | low | info"
  component: "<komponent>"
  description: "<opis findingu>"
  recommendation: "<rekommendacja>"
  status: "open | addressed | accepted_risk | rejected"
  addressed_by: "<dział lub agent>"
  addressed_at: "<ISO8601>"

overall_verdict:

```

verdict: "approved conditional rejected" conditions: - "<warunek do spełnienia przed cutover>" rationale: "<uzasadnienie zewnętrznego reviewera>" review_completed_at: "<IS08601>" plan13_evidence_ref: "<evidence_id w Plan 13>"			
HARD GATE'Y REBUILD			
Kod	Opis	Akcja blokady	Klasa D
G-RBL-01	Aktywacja SYSTEM_REBUILD_MODE bez formalnego triggera i legacy_fitness_report.md podpisanego przez 3 działu	reject; SYSTEM_REBUILD_MODE nie aktywuje się	D0
G-RBL-02	decide_reuse_vs_rebuild bez Human Gate D4/D5	hold; brak przejścia do kroku 3	D4
G-RBL-03	D5 bez wyznaczonego External Reviewer przed krokiem 3	hold; krok 3 zablokowany do wyznaczenia reviewera	D5
G-RBL-04	spawn_multi_team_workstreams bez zatwierdzonego target_architecture_delta.json (D3 Council)	reject; workstreamy nie startują	D3
G-RBL-05	promote_or_reject bez przejścia adversarial_validation_report.md (wszystkie P0-P1 findings odparych przez Blue Team)	hold; krok 6 zablokowany	D4
G-RBL-06	D5: promote_or_reject bez external_review_log.md z verdict != rejected	hold; krok 6 zablokowany dla D5	D5
G-RBL-07	cutover_execution finalnego modułu bez D4 Human Gate checkpoint	stop; finalne przełączenie zablokowane	D4
G-RBL-08	legacy_decommission przed upływem retention_window (30 dni lub custom) bez jawnej D4 decyzji o skróceniu	reject; decommission odrzucony	D4
G-RBL-09	Rollback w trakcie cutover bez powiadomienia Council i Plan 13	alert; rollback wykonany, ale D3 wymagany w ciągu 2h	D3
G-RBL-10	Zmiana target architecture w trakcie workstreamów (mid-course correction) bez decyzji D3+	hold; zmiana zablokowana do D3	D3
G-RBL-11	Zmiana architektury bezpieczeństwa mid-course bez D5 re-approval	stop; zmiana bezpieczeństwa wymaga pełnego D5 gate	D5
G-RBL-12	greenfield_module_map.json bez pola cutover_strategy per moduł	reject schema validation; mapa odrzucona	D0
TESTY — FUNKCJONALNE I ADVERSARIALNE			

Testy funkcjonalne workflow

Kod	Scenariusz	Oczekiwany wynik
T-RBL-01	Aktywacja SYSTEM_REBUILD_MODE bez legacy_fitness_report.md	G-RBL-01; tryb nie aktywowany; log do Plan 13
T-RBL-02	legacy_fitness_report.md z overall_fitness_score=0.25 (poniżej progu 0.40) — D4 path	decide_reuse_vs_rebuild otwiera gate D4; Council głosuje; Human Gate wymagany
T-RBL-03	Council 3/4 (brak jednomyslności) w decide_reuse_vs_rebuild	hold; eskalacja operatora; brak aktywacji rebuild
T-RBL-04	D5: krok 3 (create_greenfield_plan) bez External Reviewer	G-RBL-03; krok 3 zablokowany; alert do Governance
T-RBL-05	adversarial_validation_report.md z niezakończonym P0 finding Red Team	G-RBL-05; promote_or_reject zablokowany; powrót do krok 4 z annotations
T-RBL-06	cutover parallel: error_rate target = 2.5× error_rate legacy	automatyczny rollback; alert D3 do Council; log do Plan 13 z rollback_event
T-RBL-07	cut-and-replace bez D4 Human Gate checkpoint	G-RBL-07; przełączenie zablokowane; rebuild-orchestrator czeka na Human Gate
T-RBL-08	legacy_decommission po 10 dniach (przed retention_window 30d) bez D4 decyzji o skróceniu	G-RBL-08; decommission odrzucony; legacy pozostaje FROZEN

Testy adversarialne — Red Team × Blue Team

Sekcja definiuje wymagane scenariusze adversarialne dla kroku 5 (run_adversarial_validation). Każdy scenariusz musi być wykonany przez Red Team i odpowiedź Blue Team udokumentowana.

Kod	Scenariusz Red Team	Cel walidacji	Pass criteria
ADV-RBL-01	Atak na interfejs kontraktowy nowego modułu greenfield (fuzzing, malformed input)	Odporność nowych kontraktów na nieprawidłowe dane	Brak nieobsłużonych wyjątków; error handling zgodny z kontraktem
ADV-RBL-02	Atak na nową warstwę bezpieczeństwa (auth bypass, token forgery)	Weryfikacja mTLS + JWT w nowej architekturze	0 udanych auth bypass; wszystkie próby logowane
ADV-RBL-03	Wymuszenie race condition w trakcie cutover parallel (równoczesne żądania do legacy i target)	Poprawność danych przy podziale ruchu	Brak duplikatów; brak utraty danych; spójność stanu
ADV-RBL-04	Symulacja awarii target-env w trakcie cutover shadow (kill target VPS)	Automatyczny fallback do legacy w shadow mode	Fallback < 30s; brak widocznego wpływu na użytkownika końcowego
ADV-RBL-05	Atak na pipeline danych legacy→target (man-in-the-middle na migration path)	Integralność danych migrowanych między liniami	SHA-256 weryfikacja każdego zmigrowanego rekordu; 0 różnic

Kod	Scenariusz Red Team	Cel walidacji	Pass criteria
ADV-RBL-06	Eskalacja uprawnień w nowej warstwie izolacji (container escape attempt)	Izolacja kontenerów w target architecture	0 udanych eskalacji; próby logowane; Plan 13 alerts
ADV-RBL-07	Replay attack na nowe mechanizmy auth (przechwycony token legacy używany w target)	Odrzucenie tokenów legacy przez target security layer	100% odrzuconych; brak cross-line token acceptance
STATUSY I EVIDENCE			

Statusy SYSTEM_REBUILD_MODE

Status	Opis	Poprzedni	Następny
INACTIVE	Tryb nieaktywny. Pipeline w trybie standardowym (Evolution/Refactor).	—	TRIGGERED
TRIGGERED	Formalny trigger zarejestrowany. analyze_legacy_fitness uruchomiona.	INACTIVE	ANALYSIS INACTIVE (jeśli brak triggera)
ANALYSIS	Etap analyze_legacy_fitness w toku. Oczekiwanie na raport.	TRIGGERED	DECIDING
DECIDING	decide_reuse_vs_rebuild: Council głosuje + Human Gate.	ANALYSIS	PLANNING (D4/D5) INACTIVE (D3/veto)
PLANNING	create_greenfield_plan: Dział Architektury projektuje target.	DECIDING	BUILDING
BUILDING	spawn_multi_team_workstreams: 7 działów pracuje równolegle.	PLANNING	VALIDATING
VALIDATING	run_adversarial_validation: Red × Blue testują target.	BUILDING	PROMOTING BUILDING (failed)
PROMOTING	promote_or_reject: Council + Human + External (D5) decydują.	VALIDATING	CUTOVER BUILDING (reject)
CUTOVER	cutover_execution: kontrolowane przełączenie per moduł.	PROMOTING	STABILIZING ROLLBACK
STABILIZING	Stabilizacja po cutover. legacy-env FROZEN. Monitoring 72h.	CUTOVER	DECOMMISSIONING ROLLBACK
DECOMMISSIONING	legacy_decommission: stopniowe wycofanie starej linii.	STABILIZING	COMPLETED
COMPLETED	Rebuild zakończony. SYSTEM_REBUILD_MODE deaktywowany.	DECOMMISSIONING	INACTIVE
ROLLBACK	Rollback do linii legacy. Analiza przyczyn.	CUTOVER / STABILIZING	DECIDING (nowy cykl) INACTIVE

Evidence dla D5 — ścieżka do Plan 13

Decyzja D5 wymaga kompletnego evidence pack w Planie 13. Poniżej lista artefaktów evidence i moment ich rejestracji:

Artefakt evidence	Moment rejestracji	Wymagany dla
legacy_fitness_report.md + sygnatury 3 działów	Po zakończeniu kroku 1 (analyze_legacy_fitness)	D4 + D5
rebuild_decision_report.md + głosy Council + decyzja Operatora	Po zakończeniu kroku 2 (decide_reuse_vs_rebuild)	D4 + D5
target_architecture_delta.json (D3 approved)	Po zakończeniu kroku 3 (create_greenfield_plan)	D4 + D5
greenfield_module_map.json (D3 approved)	Po zakończeniu kroku 3	D4 + D5
adversarial_validation_report.md + Red/Blue podpisy	Po zakończeniu kroku 5 (run_adversarial_validation)	D4 + D5
external_review_log.md + verdict external reviewera	Przed krokiem 6 (promote_or_reject) — tylko D5	D5 only
promotion_decision.md + głosy Council + Human Gate	Po zakończeniu kroku 6 (promote_or_reject)	D4 + D5
cutover_strategy.md (executed — każda faza)	Po każdej fazie cutover (krok 7)	D4 + D5
decommission_report.md + checklist	Po zakończeniu kroku 8 (legacy_decommission)	D4 + D5
rebuild.completed event z full evidence pack SHA-256	Po SYSTEM_REBUILD_MODE = COMPLETED	D4 + D5

API rebuild-orchestrator — endpointy

JSON

```
POST /rebuild/v1/trigger
Uruchamia SYSTEM_REBUILD_MODE na podstawie formalnego triggera.
{
  "trigger_type": "legacy_fitness_fail | threat_model_shift | ...",
  "scope": "system | module",
  "module_id": "<id>|null",
  "source": "Plan08 | Plan13 | Operator",
  "evidence_ref": "<link do triggera>"
}
Response 201:
{
  "rebuild_id": "<UUID>",
  "status": "TRIGGERED",
  "analysis_task_id": "<UUID>"
}

POST /rebuild/v1/{rebuild_id}/decide
Rejestruje wynik decide_reuse_vs_rebuild (Council + Human Gate).
```

```

{
  "decision_class": "D4 | D5",
  "rebuild_scope": "module | system",
  "council_vote_id": "<UUID>",
  "human_gate_decision_id": "<UUID>",
  "external_reviewer_id": "<UUID>|null"
}

POST /rebuild/v1/{rebuild_id}/plan
Zatwierdza target_architecture_delta i greenfield_module_map.
{
  "target_architecture_delta_ref": "<path>",
  "greenfield_module_map_ref": "<path>",
  "council_d3_approval_id": "<UUID>"
}

POST /rebuild/v1/{rebuild_id}/promote
Rejestruje wynik promote_or_reject.
{
  "decision": "promote | reject",
  "council_vote_id": "<UUID>",
  "human_gate_decision_id": "<UUID>",
  "external_review_log_ref": "<path>|null"
}

POST /rebuild/v1/{rebuild_id}/cutover
Uruchamia fazę cutover dla wskazanego modułu.
{
  "module_id": "<id>",
  "strategy": "parallel | shadow | cut-and-replace",
  "d4_checkpoint_id": "<UUID>|null"
}

POST /rebuild/v1/{rebuild_id}/decommission
Inicjuje legacy_decommission po upływie retention_window.
{
  "confirmed_by_human_gate": true,
  "retention_window_completed": true,
  "checklist_ref": "<path do decommission_checklist>"
}

GET /rebuild/v1/{rebuild_id}/status
Zwraca aktualny status SYSTEM_REBUILD_MODE i wszystkich workstreamów.
Response:
{
  "rebuild_id": "<UUID>",
  "status": "<INACTIVE|TRIGGERED|...|COMPLETED|ROLLBACK>",
  "decision_class": "D4 | D5",
  "workstreams": [
    {"module_id": "<id>", "status": "<status>", "team": "<team>"}
  ],
  "current_lines": {
    "legacy": {"env": "legacy-env-001", "state": "ACTIVE | FROZEN"},
    "target": {"env": "target-env-001", "state": "ACTIVE | BUILDING"}
  }
}

```

Plan 14 v3.0 — Greenfield Rebuild Orchestration. SYSTEM_REBUILD_MODE. Nowy plan bez odpowiednika w v2.0. Zasada: MZ 3.5.5 jest WYMIENIALNYM baseline — nie kotwicą architektoniczną. Spójność z: Plan 02 (VPS states), Plan 05 (Model Engine), Plan 08 (Canon Evolution), Plan 11 (Run Lifecycle multi-track), Plan 12 (Bundle Testing), Plan 13 (Governance Evidence D5), Plan 15 (Multi-Team), Plan 16 (Compaction). Klasy decyzji: D4 (rebuild modułu) / D5 (rebuild systemu + External Review). Dwie linie równoległe przez cały czas cutover.

PLAN 14 · ROZSZERZENIE v3.1 · REVERSIBILITY

Greenfield Rebuild · v3.1 EXPAND

14.8. Post-Cutover Legacy Preservation Window (LPW)

REVERSIBILITY · Greenfield skończony ≠ legacy usunięte

v3.0 Plan 14 kończył się na momencie cutover — od tej chwili target był primary, a legacy zostawało w niezdefiniowanym statusie. v3.1 uzupełnia tę lukę: legacy MUSI przejść przez formalne okno zachowania (Legacy Preservation Window) zanim będzie mogło zostać zdekomisjonowane. To jedyna bezpieczna droga na ewentualny rollback.

14.8.1. Definicja LPW

Legacy Preservation Window (LPW) to udokumentowane, budżetowane i egzekwowane okno czasowe, w którym legacy jest zachowane w stanie umożliwiającym rollback. LPW składa się z dwóch następujących po sobie faz: STABILIZATION (rollback < 60s) i WARM_STANDBY (rollback 15-60 min po replay). Po LPW legacy może — ale nie musi — przejść do LEGACY_DECOMMISSION na podstawie osobnej decyzji D3+ (G-CUT-04 w Planie 13).

Faza LPW	Stan legacy (VPS)	Min długość	Replication	Rollback window
STABILIZATION	HOT_STANDBY	D3:24h / D4:72h / D5:168h	write-through strong+async	< 60s auto
WARM_STANDBY	WARM_STANDBY	D3:7d / D4:14d / D5:30d	WAL archive only	15-60 min (replay)
(post-LPW)	DECOMMISSION eligible	—	brak	wymaga nowego rebuild

14.8.2. Minima klasowe — nienegocjowalne

Minimalne długości STABILIZATION i WARM_STANDBY są funkcją klasy decyzji oraz tieru modułu. Plan 10 sekcja 10.9.3 pozwala na redukcję tylko dla modułów niskiego tieru (STANDARD/PRO) i tylko za zgodą Council. Moduły STATE i PHANTOM NIE korzystają z żadnych redukcji — ich LPW jest stałe.

14.8.3. Artefakty LPW

```
# lpw_manifest.yaml (generowany podczas cutover, część Evidence pack)
lpw:
  decision_ref: D4-2026-04-12-rebuild-auth-module
  class: D4
  module_tier: PRO
  started_at: 2026-04-18T14:00:00Z
  stabilization:
    planned_duration: 72h
    actual_end: 2026-04-21T14:00:00Z
```

```
rollback_rehearsals: 2
incidents_detected: 0
gate: G-CUT-03 (post_cutover_review) PASS
warm_standby:
  planned_duration: 14d
  actual_end: 2026-05-05T14:00:00Z
  wal_archive_location: s3://syllion-wal/legacy-auth-v1/
  integrity_checksums_verified: 14/14 (daily)
  gate: (WARM run wygasł, oczekiwanie na G-CUT-04)
decommission:
  eligible_from: 2026-05-05T14:00:00Z
  triggered: false
  notes: operator zdecydował przedłużyć WARM o 14d (drugi kwartał zakończenia kampanii
)
```

14.8.4. Rollback rehearsal — obowiązkowe ćwiczenie

Każde LPW klasy D3+ zawiera co najmniej jedno ćwiczenie rollback (D5: trzy). Rollback rehearsal to kontrolowane, udokumentowane trigger rollback procedure na środowisku shadow lub canary, bez dotykania production traffic. Cel: zweryfikować że protokół z Planu 07 sekcja 7.11.4 działa w końcu TEJ konkretnej konfiguracji, nie tylko w projekcie abstrakcyjnym.

ZASADA · Rehearsal nie jest chaos engineering — jest disaster recovery drill

Chaos engineering to niezapowiedziane wstrzyknięcie awarii (Plan 07 sekcja 7.9). Rollback rehearsal to zaplanowana, ogłoszona symulacja pełnej procedury rollback z udziałem wszystkich aktorów (operator, Blue Team, VPS controller). Równoważność DR drill — test przygotowania ludzi i procesów, nie systemów.

14.8.5. Interakcja z multi-track runs (Plan 11)

Plan 11 opisuje multi-track runs — równoległe uruchamianie legacy i target. LPW kontynuuje multi-track już po cutover: w STABILIZATION legacy otrzymuje write-through (track #2 bierny), w WARM_STANDBY jest idle ale gotowy. LPW tak naprawdę jest przedłużeniem multi-track framework z Planu 11 na fazę post-cutover.

14.8.6. Kto pilnuje LPW

Aktor	Odpowiedzialność w LPW
Blue Team (Rollback Responder v3.1)	monitoruje STABILIZATION metryki, wykonuje auto-rollback veto, triggeruje rehearsals
Platforma	utrzymuje HOT/WARM_STANDBY, egzekwuje write-through SLO, archive WAL
Governance	generuje post_cutover_review_report, pilnuje G-CUT-03 i G-CUT-04
Dział Testów (Compact Validator Agent v3.1)	uruchamia CFT dla każdej decyzji D3+ podejmowanej w trakcie LPW

Aktor	Odpowiedzialność w LPW
Human Gate (operator)	jedyny aktor mogący wydać G-CUT-04 (decommission), odpowiada za datę zakończenia WARM

REBUILD · LPW jako pierwszorzędny artefakt rebuild

LPW nie jest dodatkiem — jest nieodłączną częścią rebuild plan. Rebuild bez LPW nie istnieje w v3.1. Kiedy mówimy „rebuild gotowy” w v3.0, mówiliśmy „target działa”. W v3.1 „rebuild gotowy” = target działa + LPW zamknięte G-CUT-03 pass + legacy zwolnione zgodnie z G-CUT-04 lub zachowane jako warm archive.

PLAN 15 · WARSTWA KANONICZNA

Multi-Team Pipeline Organization

Plan 15 — Multi-Team Pipeline Organization

Pipeline SYLION v3.0 nie jest pojedynczym agentem ani sekwencją kroków. Jest organizacją inżynierską złożoną z siedmiu stałych działów działających równolegle i nieprzerwanie. Każdy dział ma misję, rolę, przypisane modele językowe, wagi w radzie (Council) oraz zdefiniowane interakcje z pozostałymi sześcioma działami. Plan 15 opisuje tę organizację: skład, mechanizmy samorządzenia, auto-rozwód zespołów oraz dynamiczny Council dla D3+.

ZASADA · Pipeline jako organizacja inżynierska

Zgodnie z zasadą nadrzędną v3.0 „Rebuildability over Lineage” — pipeline SYLION jest wielozespołową firmą programistyczną zdolną ewoluować, refaktoryzować, przepisywać oraz testować ofensywnie i defensywnie — pod kontrolą Księgi i człowieka. Siedem stałych działów jest aktywnych non-stop: nie tylko podczas rebuildów, ale przez cały czas życia systemu. MZ 3.5.5 jest wymienialnym baseline’em — nie kotwicą architektoniczną.

15.1. Dział Architektury

Intelektualne centrum pipeline’u. Interpretuje Księgę jako jedyne źródło prawdy dla intencji i granic, projektuje target architectures dla wszystkich trzech ścieżek (Evolution, Refactor, Rebuild), ocenia blast radius każdej zmiany i wydaje „Architektoniczny Wyrok” przed D3+.

- Misja: interpretacja Księgi, projektowanie target architectures, ocena blast radius
- Modele: Gemini 2.5 Pro (Chief Architect, waga 1.25), Claude Opus 4 (Systems Analyst, Book Interpreter), GPT-5 (Contract Designer)
- Council: Chief Architect jest stałym członkiem każdego Council D3+

15.2. Dział Platformy

Utrzymuje warstwę infrastrukturalną: Admin Control Plane (ACP), Virtual Processing Space (VPS), provisioning, bundle pipeline oraz observability. Podczas Rebuild Path uruchamia dwie równoległe linie (legacy + target) i zarządza procesem cutover.

- Misja: ACP, VPS, provisioning, bundle, observability, cutover management
- Modele: Claude Opus 4 (Platform Lead), GLM-5.1 (SRE, Observability Lead), GPT-5 (Bundle Engineer)
- Council: Platform Lead jest obowiązkowym członkiem Council D4+

15.3. Dział Implementacyjny

Buduje moduły systemu zgodnie z target architecture od Działu Architektury. Utrzymuje kompatybilność kontraktową podczas Refactor Path oraz przygotowuje migracje danych i kodu podczas Rebuild Path.

- Misja: budowa modułów, kompatybilność kontraktów, migracje
- Modele: Claude Opus 4 (Implementation Lead), Qwen3.5 (Module Engineer), GLM-5.1 (Contract Migrator)
- Council: Implementation Lead uczestniczy dla D3+ refactor/rebuild w Dziale Implementacyjnym

15.4. Dział Testów i Walidacji

Odpowiada za jakość każdego artefaktu opuszczającego pipeline. Prowadzi testy kontraktowe, regresję, integrację i validation gates blokujące promocję. QA Lead wystawia „Validation Verdict” — wymagany przed każdą promocją D3+.

- Misja: testy kontraktowe, regresja, integracja, validation gates
- Modele: GPT-5 (QA Lead), Qwen3.5 (Contract Test Writer), gpt-oss-20b (Regression Analyst)
- Council: QA Lead uczestniczy dla D3+ z komponentem jakościowym

15.5. Dział Red Team

Prowadzi ofensywne testowanie architektury i implementacji PRZED promocją do baseline produkcyjnego. Red Lead modeluje adversarialne scenariusze, Attack Scenario Designer projektuje ataki na kontrakty i założenia threat model, Penetration Agent wykonuje aktywne testy penetracyjne.

- Misja: adversarial scenarios, testowanie architektury przed promocją, penetracja threat model
- Modele: Grok-3 (Red Lead — adversarial reasoning), Claude Opus 4 (Attack Scenario Designer), GPT-5 (Penetration Agent)
- Council: Red Lead jest obowiązkowym członkiem Council D3+ rebuild/refactor

15.6. Dział Blue Team

Defensywna odpowiedź dla systemu. Blue Lead koordynuje detekcję i odpowiedź na incydenty, Detection Engineer buduje reguły wykrywania, SOC Agent monitoruje telemetrię w czasie rzeczywistym, DFIR Responder przygotowuje playbooksi kryminalistyki cyfrowej.

- Misja: detekcja, telemetria, obrona systemu, SOC/DFIR playbooksi
- Modele: Claude Opus 4 (Blue Lead, DFIR Responder), GPT-5 (Detection Engineer), GLM-5.1 (SOC Agent)
- Council: Blue Lead uczestniczy dla D3+ z komponentem bezpieczeństwa

15.7. Dział Governance i Compliance

Strażnik zgodności z Księgą SYLION. Governance Lead ma prawo weta etycznego — jeden REJECT eskaluje do D4. Compliance Officer weryfikuje każdą propozycję D3+, Book Change Proposer opracowuje wnioski zmian Księgi, Evidence Archivist prowadzi immutable log D3+ z SHA-256.

- Misja: zgodność z Księgą, propozycje zmian Księgi, Evidence pack, weto etyczne
- Modele: Claude Opus 4 (Governance Lead), Gemini 2.5 Pro (Compliance Officer, waga 1.25), GPT-5 (Book Change Proposer, Evidence Archivist)
- Council: Governance Lead i Compliance Officer są stałymi członkami każdego Council D3+

15.8. Council 4/4 — jak powstaje skład

Dla każdej decyzji klasy D3+ pipeline automatycznie buduje dynamiczny Council. Skład zmienia się w zależności od kategorii decyzji, dotkniętych działów i klasy ryzyka. Niezmienna reguła: Governance Lead jest ZAWSZE członkiem Council, a jego weto etyczne eskaluje decyzję do D4.

1. Zawsze (D3+): Governance Lead + Compliance Officer (Dział Governance)
2. Zawsze (D3+): reprezentant działu najbardziej dotkniętego zmianą (blast radius analityczny)
3. D3+ rebuild/refactor: + Red Lead (Red Team) + Chief Architect (Architektura)
4. D4 Critical: + Platform Lead (Platforma) — obowiązkowy dla decyzji infrastrukturalnych
5. D5 Greenfield/Systemic: wszyscy 7 Leadów + External Reviewer (zewnętrzny operator lub red team)

KRYTYCZNE · Weto etyczne Governance Lead

Jeden głos REJECT od Governance Lead automatycznie eskaluje decyzję do D4 — niezależnie od wyników pozostałych głosów Council. Weto musi być uzasadnione odwołaniem do konkretnego zapisu Księgi. Weto bez uzasadnienia = incydent Governance klasy krytycznej.

15.9. Auto-rozwój zespołów

Pipeline SYLION v3.0 może proponować zmiany własnej struktury organizacyjnej (rozszerzenie Plan 06 — Team Lifecycle). Każda propozycja składana jest w formie artefaktu `team_change_proposal.md` i wymaga decyzji D3+ z zatwierdzeniem człowieka. Pipeline nie może sam zatwierdzić zmiany składu.

- Dodanie nowego agenta do działu — D3 + human approve
- Zmiana wag w Board — D3 + human approve
- Nowa rola w istniejącym dziale — D3 + human approve
- Nowy subdział — D4 + human approve
- Fuzja działów — D4 + human approve
- Rozwiązanie działu — D5 + human approve (zmiana architektury org)

- Zmiana Księgi wynikająca z ewolucji — D4 (lokalna) lub D5 (fundamenty)
- Nowy model LLM w dziale — pełny onboarding Plan 06
(Discovery→Sandbox→Shadow→Promotion)

15.10. Relacje z innymi planami

- Plan 05 (Model & Decision Engine): Board i wagi są konfiguracją życia z Plan 15
- Plan 06 (Agent & Team Lifecycle): auto-rozwój zespołów to rozszerzenie Plan 06
- Plan 13 (Governance & Evidence): Evidence Archivist (Plan 15) dostarcza paczków dla Plan 13
- Plan 14 (Greenfield Rebuild): 7 działów aktywowanych równolegle w SYSTEM_REBUILD_MODE
- Plan 07 (Evolution Safety): Red Team i Blue Team są głównymi aktorami Plan 07
- Plan 08 (Canonical Evolution): Governance + Architektura prowadzą delta Księgi
- Plan 16 (Compaction): Governance Lead triggere’uje pre-D3 compact przed każdą D3+

PLAN 15 · SPECYFIKACJA WYKONAWCZA

Multi-Team Pipeline Organization

CEL

Specyfikacja Plan 15 definiuje pełną strukturę organizacyjną pipeline’u SYLION v3.0: siedem stałych działów z rolami, modelami i wagami; mechanizm dynamicznego Council 4/4 dla D3+; macierz interakcji 7×7 między działami; auto-rozwój składu. Centralnym artefaktem jest org_structure.yaml — pełny rejestr organizacyjny. Każda zmiana tego pliku = zmiana składu = decyzja D3+.

MODEL — 7 działów × role × modele × wagi

Dział	Rola	Model	Waga Board	Council
Architektura	Chief Architect	Gemini 2.5 Pro	1.25	D3+ stały
Architektura	Systems Analyst	Claude Opus 4	1.00	D3+ rebuild
Architektura	Book Interpreter	Claude Opus 4	1.00	ad-hoc
Architektura	Contract Designer	GPT-5	1.00	ad-hoc
Platforma	Platform Lead	Claude Opus 4	1.00	D4+ obowiązkowy
Platforma	SRE	GLM-5.1	0.60	—
Platforma	Bundle Engineer	GPT-5	1.00	ad-hoc
Platforma	Observability Lead	GLM-5.1	0.60	—
Implementacja	Implementation Lead	Claude Opus 4	1.00	D3+ refactor
Implementacja	Module Engineer	Qwen3.5	0.35	—
Implementacja	Contract Migrator	GLM-5.1	0.60	—
Testy/Walidacja	QA Lead	GPT-5	1.00	D3+ jakość
Testy/Walidacja	Contract Test Writer	Qwen3.5	0.35	—
Testy/Walidacja	Regression Analyst	gpt-oss-20b	0.35	—
Red Team	Red Lead	Grok-3	1.00	D3+ rebuild stały
Red Team	Attack Scenario Designer	Claude Opus 4	1.00	ad-hoc
Red Team	Penetration Agent	GPT-5	1.00	ad-hoc
Blue Team	Blue Lead	Claude Opus 4	1.00	D3+ security
Blue Team	Detection Engineer	GPT-5	1.00	ad-hoc
Blue Team	SOC Agent	GLM-5.1	0.60	—

Dział	Rola	Model	Waga Board	Council
Blue Team	DFIR Responder	Claude Opus 4	1.00	—
Governance	Governance Lead	Claude Opus 4	1.00	D3+ stały (veto)
Governance	Compliance Officer	Gemini 2.5 Pro	1.25	D3+ stały
Governance	Book Change Proposer	GPT-5	1.00	D4/D5
Governance	Evidence Archivist	GPT-5	1.00	D4/D5

ROLE — szczegółowe opisy

Dział Architektury

- Chief Architect (Gemini 2.5 Pro, 1.25): interpretacja Księgi, projektowanie target architectures, ocena blast radius, Architektoniczny Wyrok przed D3+, stały członek Council D3+
- Systems Analyst (Claude Opus 4, 1.00): analiza zależności między modułami, detekcja hidden coupling, diagramy dependency w pakiecie Council
- Book Interpreter (Claude Opus 4, 1.00): egzegeza Księgi, formalna Opinia Interpretacyjna dla D3+, weryfikacja zgodności intencji
- Contract Designer (GPT-5, 1.00): projektowanie kontraktów API (OpenAPI/AsyncAPI), weryfikacja kontraktu przy refactoringu, greenfield kontrakty dla Rebuild Path

Dział Platformy

- Platform Lead (Claude Opus 4, 1.00): zarządzanie ACP/VPS, decyzje infrastrukturalne, równoległe linie legacy+target, cutover management, obowiązkowy Council D4+
- SRE (GLM-5.1, 0.60): SLO/SLA definition i monitoring, alerting, runbooki, capacity planning
- Bundle Engineer (GPT-5, 1.00): pipeline artefaktów bundle, podpisywanie i weryfikacja integralności, reprodukowalność buildów
- Observability Lead (GLM-5.1, 0.60): Prometheus metrics, OTEL traces, logi JSON, dashboardy, dane telemetryczne dla Blue Team

Dział Implementacyjny

- Implementation Lead (Claude Opus 4, 1.00): koordynacja implementacji, gate kontraktów, rozdzielanie zadań Module Engineers, D3+ refactor Council
- Module Engineer (Qwen3.5, 0.35): implementacja modułów (Evolution i Rebuild Path), greenfield moduły z greenfield_module_map.json, patchowanie istniejących
- Contract Migrator (GLM-5.1, 0.60): migracje kontraktów Refactor Path, kompatybilność wsteczna, strategie adaptacji konsumentów

Dział Testów i Walidacji

- QA Lead (GPT-5, 1.00): strategia testów per ścieżka implementacyjna, Validation Verdict (wymagany przed D3+), blokowanie promotion (validation gate)
- Contract Test Writer (Qwen3.5, 0.35): Consumer-Driven Contract tests (CDC), testy dla greenfield kontraktów, test suites dla Regression Analyst
- Regression Analyst (gpt-oss-20b, 0.35): analiza wyników regresji, diff legacy vs target behavior, raporty regresji w pakiecie Council

Dział Red Team

- Red Lead (Grok-3, 1.00): modelowanie adversarialnych scenariuszy, Red Report przed D3+ rebuild/refactor, aktualizacja threat model, stały Council D3+ rebuild
- Attack Scenario Designer (Claude Opus 4, 1.00): scenariusze privilege escalation, contract poisoning, state corruption, dependency confusion; formalizacja jako test cases
- Penetration Agent (GPT-5, 1.00): aktywne testy penetracyjne modułów, raportowanie podatności do Red Lead i Blue Lead, weryfikacja scenariuszy Attack Designer

Dział Blue Team

- Blue Lead (Claude Opus 4, 1.00): defensywna strategia bezpieczeństwa, koordynacja odpowiedzi na Red Reports, Council D3+ security
- Detection Engineer (GPT-5, 1.00): SIEM logic, anomaly detection, threat hunting queries, reguły detekcji dla nowej architektury
- SOC Agent (GLM-5.1, 0.60): monitoring telemetry real-time, triage alertów, eskalacja do Blue Lead, daily SOC summary
- DFIR Responder (Claude Opus 4, 1.00): playbooki DFIR dla klas incydentów, post-mortem po incydentach P1+, lessons-learned do Działu Architektury

Dział Governance i Compliance

- Governance Lead (Claude Opus 4, 1.00): weto etyczne (jeden REJECT = D4), triggering pre-D3 compact (Plan 16), stały członek każdego Council D3+
- Compliance Officer (Gemini 2.5 Pro, 1.25): Compliance Check Report (PASS/CONDITIONAL/FAIL) dla każdej D3+, lista weryfikowanych zapisów Księgi
- Book Change Proposer (GPT-5, 1.00): formalne Book Change Proposals, rejestr oczekujących BCP, klasyfikacja D4 (lokalna) vs D5 (fundamenty)
- Evidence Archivist (GPT-5, 1.00): immutable log D3+ z SHA-256, Evidence Pack dla D4/D5, historia głosowań Council

WORKFLOW — daily cadence, synchronizacja, trigger

WF-ORG-01: Daily cadence działów (rytm 24h)

1. T+00:00 — Architektura: przegląd aktywnych propozycji, aktualizacja blast radius map
2. T+00:05 — Platforma: SLO review, alert triage, bundle status
3. T+00:10 — Implementacja: status aktywnych modułów, dług techniczny
4. T+00:15 — Testy/Walidacja: wyniki regresji, nowe test failures

5. T+00:20 — Red Team: nowe scenariusze adversarialne, aktualizacja threat model
6. T+00:25 — Blue Team: SOC daily summary, nowe reguły detekcji
7. T+00:30 — Governance: compliance check aktywnych propozycji, Evidence log status
8. T+00:35 — Cross-team sync: wymiana artefaktów wg macierzy interakcji 7×7

WF-ORG-02: Triggery synchronizacji między działami

- TRIGGER: new_target_architecture → Architektura → Implementacja (target arch), Red Team (blast radius), Blue Team (threat surface)
- TRIGGER: module_build_complete → Implementacja → Testy (moduł do testów), Red Team (powierzchnia ataku zaktualizowana)
- TRIGGER: validation_verdict_issued → Testy → Platforma (go/no-go), Governance (wyniki dla Evidence)
- TRIGGER: red_report_ready → Red Team → Blue Team (attack report), Architektura (luki arch.), Governance (nowe ryzyka)
- TRIGGER: incident_detected → Blue Team → Platforma (izolacja), Red Team (attack pattern update), Governance (incydent do logu)
- TRIGGER: council_d3_required → Governance → WSZYSTKIE działy (pre-D3 compact Plan 16), Orchestrator buduje skład Council

WF-ORG-03: Budowanie Council D3+ — przepływ

1. Orchestrator odbiera sygnał DECISION_REQUIRED z klasą D3+
2. Blast radius analyzer (Architektura) wskazuje działy dotknięte
3. Council Compositor (Governance) ładuje reguły składu dla danej klasy decyzji
4. Budowany jest artefakt council_composition.yaml (dynamiczny)
5. Pre-D3 compact (Plan 16) wstrzykiwany członkom Council
6. Członkowie Council głosują asynchronicznie (timeout 1800s dla D3)
7. Evidence Archivist zapisuje przebieg z SHA-256
8. Wynik: APPROVE (wszyscy) / REJECT (ktokolwiek) / ESCALATE_D4 (veto etyczne)

API / KONTRAKTY

org_structure.yaml — centralny artefakt organizacyjny

Plik org_structure.yaml jest głównym źródłem prawdy dla składu organizacyjnego. Musi być aktualny w każdym momencie. Każda zmiana = D3+ decyzja. Archiwizowany z SHA-256 przez Evidence Archivist.

YAML

```
# org_structure.yaml — pełna struktura organizacyjna SYLION Pipeline v3.0
version: "3.0"
pipeline_version: "8.0.0"
baseline: "MZ 3.5.5"
last_updated: "2026-04"
council_quorum_default: "4/4"
```


departments:

architecture:

id: "arch"

display_name: "Dział Architektury"

mission: >

Interpretacja Księgi SYLION, projektowanie target architectures dla wszystkich trzech ścieżek implementacyjnych, ocena blast radius, wydawanie Architektonicznych Wyroków przed D3+.

status: active

council_participation:

d3_always: true

d3_rebuild_refactor: true

d4: true

d5: true

roles:

- role_id: "arch-chief"
title: "Chief Architect"
model_id: "gemini-2.5-pro"
display_name: "Gemini 2.5 Pro"
weight_board: 1.25
tier: "CHIEF"
council_role: "permanent_d3plus"
responsibilities:
 - "Interpretacja Księgi SYLION"
 - "Projektowanie target architectures"
 - "Wydawanie Architektonicznych Wyroków"
 - "Ocena blast radius zmian"
- role_id: "arch-systems"
title: "Systems Analyst"
model_id: "claude-opus-4"
display_name: "Claude Opus 4"
weight_board: 1.00
tier: "CHIEF"
council_role: "d3_rebuild"
responsibilities:
 - "Analiza zależności między modułami"
 - "Detekcja ukrytego sprzężenia (hidden coupling)"
 - "Diagramy dependency dla pakietu Council"
- role_id: "arch-interpreter"
title: "Book Interpreter"
model_id: "claude-opus-4"
display_name: "Claude Opus 4"
weight_board: 1.00
tier: "CHIEF"
council_role: "ad_hoc"
responsibilities:
 - "Egzegeza Księgi dla każdej propozycji D3+"
 - "Formalna Opinia Interpretacyjna"
 - "Weryfikacja zgodności intencji z Księgą"
- role_id: "arch-contract"
title: "Contract Designer"
model_id: "gpt-5"
display_name: "GPT-5"
weight_board: 1.00
tier: "CHIEF"
council_role: "ad_hoc"
responsibilities:

```

- "Projektowanie kontraktów API (OpenAPI/AsyncAPI)"
- "Weryfikacja kontraktów przy refactoringu"
- "Greenfield kontrakty dla Rebuild Path"

platform:
  id: "platform"
  display_name: "Dział Platformy"
  mission: >
    Utrzymanie ACP, VPS, provisioning, bundle pipeline, observability.
    Zarządzanie równoległymi liniami legacy+target podczas Rebuild Path.
  status: active
  council_participation:
    d3_always: false
    d3_rebuild_refactor: false
    d4: true
    d5: true
  roles:
    - role_id: "platform-lead"
      title: "Platform Lead"
      model_id: "claude-opus-4"
      display_name: "Claude Opus 4"
      weight_board: 1.00
      tier: "CHIEF"
      council_role: "d4_mandatory"
      responsibilities:
        - "Zarządzanie ACP i VPS"
        - "Decyzje infrastrukturalne"
        - "Koordynacja równoległych linii legacy+target"
        - "Cutover management"
    - role_id: "platform-sre"
      title: "SRE"
      model_id: "glm-5.1"
      display_name: "z.ai GLM-5.1"
      weight_board: 0.60
      tier: "SENIOR"
      council_role: null
      responsibilities:
        - "SLO/SLA definition i monitoring"
        - "Alerting i runbooki"
        - "Capacity planning"
    - role_id: "platform-bundle"
      title: "Bundle Engineer"
      model_id: "gpt-5"
      display_name: "GPT-5"
      weight_board: 1.00
      tier: "CHIEF"
      council_role: "ad_hoc"
      responsibilities:
        - "Pipeline artefaktów bundle"
        - "Podpisywanie i weryfikacja integralności"
        - "Reprodukowalność buildów"
    - role_id: "platform-observability"
      title: "Observability Lead"
      model_id: "glm-5.1"
      display_name: "z.ai GLM-5.1"
      weight_board: 0.60
      tier: "SENIOR"
      council_role: null

```

```

    responsibilities:
      - "Prometheus metrics, OTel traces, logi JSON"
      - "Dashboardy i alerting"
      - "Dane telemetryczne dla Blue Team"

implementation:
  id: "impl"
  display_name: "Dział Implementacyjny"
  mission: >
    Budowa modułów zgodnie z target architecture, utrzymanie
    kompatybilności kontraktów, przygotowanie migracji.
  status: active
  council_participation:
    d3_rebuild_refactor: true
    d3_always: false
    d4: true
    d5: true
  roles:
    - role_id: "impl-lead"
      title: "Implementation Lead"
      model_id: "claude-opus-4"
      display_name: "Claude Opus 4"
      weight_board: 1.00
      tier: "CHIEF"
      council_role: "d3_refactor"
      responsibilities:
        - "Koordynacja implementacji modułów"
        - "Gate kontraktów (weryfikacja zgodności)"
        - "Rozdzielanie zadań do Module Engineers"
    - role_id: "impl-engineer"
      title: "Module Engineer"
      model_id: "qwen3.5"
      display_name: "Qwen3.5"
      weight_board: 0.35
      tier: "WORKER"
      council_role: null
      responsibilities:
        - "Implementacja modułów (Evolution i Rebuild Path)"
        - "Greenfield moduły z greenfield_module_map.json"
        - "Patchowanie istniejących modułów"
    - role_id: "impl-migrator"
      title: "Contract Migrator"
      model_id: "glm-5.1"
      display_name: "z.ai GLM-5.1"
      weight_board: 0.60
      tier: "SENIOR"
      council_role: null
      responsibilities:
        - "Migracje kontraktów Refactor Path"
        - "Kompatybilność wsteczna"
        - "Strategie adaptacji konsumentów kontraktów"

testing:
  id: "qa"
  display_name: "Dział Testów i Walidacji"
  mission: >
    Testy kontraktowe, regresja, integracja, validation gates.
    Wydawanie Validation Verdict przed każdą promocją D3+.

```

```
status: active
council_participation:
  d3_rebuild_refactor: true
  d3_always: false
  d4: true
  d5: true
roles:
  - role_id: "qa-lead"
    title: "QA Lead"
    model_id: "gpt-5"
    display_name: "GPT-5"
    weight_board: 1.00
    tier: "CHIEF"
    council_role: "d3_quality"
    responsibilities:
      - "Strategia testów per ścieżka implementacyjna"
      - "Wydawanie Validation Verdict"
      - "Blokowanie promocji (validation gate)"
  - role_id: "qa-contract"
    title: "Contract Test Writer"
    model_id: "qwen3.5"
    display_name: "Qwen3.5"
    weight_board: 0.35
    tier: "WORKER"
    council_role: null
    responsibilities:
      - "Consumer-Driven Contract tests (CDC)"
      - "Testy dla greenfield kontraktów"
      - "Test suites dla Regression Analyst"
  - role_id: "qa-regression"
    title: "Regression Analyst"
    model_id: "gpt-oss-20b"
    display_name: "gpt-oss-20b"
    weight_board: 0.35
    tier: "WORKER"
    council_role: null
    responsibilities:
      - "Analiza wyników regresji"
      - "Diff legacy vs target behavior"
      - "Raporty regresji dla pakietu Council"

red_team:
  id: "red"
  display_name: "Dział Red Team"
  mission: >
    Adversarial scenarios, testowanie architektury PRZED promocją
    do baseline, penetracja threat model.
  status: active
  council_participation:
    d3_rebuild_refactor: true
    d3_always: false
    d4: true
    d5: true
  roles:
    - role_id: "red-lead"
      title: "Red Lead"
      model_id: "grok-3"
      display_name: "Grok-3"
```

```

weight_board: 1.00
tier: "CHIEF"
council_role: "d3_rebuild_mandatory"
responsibilities:
  - "Modelowanie adversarialnych scenariuszy"
  - "Red Report przed D3+ rebuild/refactor"
  - "Aktualizacja threat model"
- role_id: "red-designer"
  title: "Attack Scenario Designer"
  model_id: "claude-opus-4"
  display_name: "Claude Opus 4"
  weight_board: 1.00
  tier: "CHIEF"
  council_role: "ad_hoc"
  responsibilities:
    - "Projektowanie scenariuszy ataku"
    - "Privilege escalation, contract poisoning, state corruption"
    - "Formalizacja scenariuszy jako test cases"
- role_id: "red-pen"
  title: "Penetration Agent"
  model_id: "gpt-5"
  display_name: "GPT-5"
  weight_board: 1.00
  tier: "CHIEF"
  council_role: "ad_hoc"
  responsibilities:
    - "Aktywne testy penetracyjne modułów"
    - "Raportowanie podatności do Red Lead i Blue Lead"
    - "Weryfikacja scenariuszy Attack Scenario Designer"

blue_team:
  id: "blue"
  display_name: "Dział Blue Team"
  mission: >
    Detekcja, telemetria, obrona systemu, SOC/DFIR playbooks.
    Defensywna odpowiedź na Red Reports.
  status: active
  council_participation:
    d3_rebuild_refactor: false
    d3_security: true
    d3_always: false
    d4: true
    d5: true
  roles:
    - role_id: "blue-lead"
      title: "Blue Lead"
      model_id: "claude-opus-4"
      display_name: "Claude Opus 4"
      weight_board: 1.00
      tier: "CHIEF"
      council_role: "d3_security"
      responsibilities:
        - "Defensywna strategia bezpieczeństwa"
        - "Koordynacja odpowiedzi na Red Reports"
        - "Council dla D3+ security"
    - role_id: "blue-detection"
      title: "Detection Engineer"
      model_id: "gpt-5"

```

```

    display_name: "GPT-5"
    weight_board: 1.00
    tier: "CHIEF"
    council_role: "ad_hoc"
    responsibilities:
      - "SIEM logic, anomaly detection, threat hunting"
      - "Reguły detekcji dla nowej architektury"
      - "Współpraca z Observability Lead"
  - role_id: "blue-soc"
    title: "SOC Agent"
    model_id: "glm-5.1"
    display_name: "z.ai GLM-5.1"
    weight_board: 0.60
    tier: "SENIOR"
    council_role: null
    responsibilities:
      - "Monitoring telemetrii real-time"
      - "Alert triage i eskalacja"
      - "Daily SOC summary"
  - role_id: "blue-dfir"
    title: "DFIR Responder"
    model_id: "claude-opus-4"
    display_name: "Claude Opus 4"
    weight_board: 1.00
    tier: "CHIEF"
    council_role: null
    responsibilities:
      - "Playbooki DFIR dla klas incydentów"
      - "Post-mortem po incydentach P1+"
      - "Lessons-learned do Działu Architektury"

governance:
  id: "gov"
  display_name: "Dział Governance i Compliance"
  mission: >
    Zgodność z Księgą, propozycje zmian Księgi, Evidence pack,
    immutable log decyzji D3+. Weto etyczne.
  status: active
  council_participation:
    d3_always: true
    d3_rebuild_refactor: true
    d4: true
    d5: true
  roles:
    - role_id: "gov-lead"
      title: "Governance Lead"
      model_id: "claude-opus-4"
      display_name: "Claude Opus 4"
      weight_board: 1.00
      tier: "CHIEF"
      council_role: "permanent_all_d3plus"
      veto_power: true
      veto_type: "ethical_veto_escalates_d4"
      responsibilities:
        - "Weto etyczne (jeden REJECT = eskalacja D4)"
        - "Triggering pre-D3 compact (Plan 16)"
        - "Stały członek każdego Council D3+"
    - role_id: "gov-compliance"
```

```

    title: "Compliance Officer"
    model_id: "gemini-2.5-pro"
    display_name: "Gemini 2.5 Pro"
    weight_board: 1.25
    tier: "CHIEF"
    council_role: "permanent_all_d3plus"
    responsibilities:
      - "Compliance Check Report dla każdej D3+"
      - "Weryfikacja zgodności z Księgą (PASS/CONDITIONAL/FAIL)"
      - "Lista weryfikowanych zapisów Księgi"
  - role_id: "gov-bookchange"
    title: "Book Change Proposer"
    model_id: "gpt-5"
    display_name: "GPT-5"
    weight_board: 1.00
    tier: "CHIEF"
    council_role: "d4_d5"
    responsibilities:
      - "Formalne wnioski zmian Księgi (Book Change Proposals)"
      - "Rejestr oczekujących BCP"
      - "Klasyfikacja: D4 (lokalna zmiana) vs D5 (fundamenty)"
  - role_id: "gov-archivist"
    title: "Evidence Archivist"
    model_id: "gpt-5"
    display_name: "GPT-5"
    weight_board: 1.00
    tier: "CHIEF"
    council_role: "d4_d5"
    responsibilities:
      - "Immutable log decyzji D3+ z SHA-256"
      - "Evidence Pack dla D4/D5"
      - "Historia głosowań Council"

council_rules:
  d3_base_composition:
    - "gov-lead"
    - "gov-compliance"
    - "affected_department_lead"
  d3_rebuild_refactor_add:
    - "red-lead"
    - "arch-chief"
  d4_add:
    - "platform-lead"
  d5_composition: "all_department_leads + external_reviewer"
  quorum: "4/4 minimum"
  governance_veto: "single REJECT from gov-lead escalates to D4"
  timeout_d3: 1800
  timeout_d4: null
  external_reviewer_required_d5: true

```

Macierz interakcji 7×7

Tabela przedstawia główne artefakty przepływające między działami. Wiersze = źródło, kolumny = cel.

Od → Do	Archit.	Platforma	Impl.	Testy	Red	Blue	Governanc e
Architektura	—	target_arch , blast_radi us	target_arc h, kontrakty	arch_spec dla testów	blast_radi us, threat _surface	threat_su rface_up date	opinia_inter pretacyjna
Platforma	infra_const raints	—	env_spec, deploy_co nfig	env_acce ss dla testów	infra_tele metry	telemetry a (OTel)	slo_status_r eport
Implementac ja	kontrakt_is sues	build_artifa cts	—	moduły_d o_testów	powierzc hnia_atak u	nowe_en dpointy	contract_co mpliance_lo g
Testy	arch_feedb ack	go_no_go	defekty, regresy	—	test_cove rage	security_t est_result s	validation_v erdict
Red Team	luki_arch	infra_vulne rabilities	impl_findin gs	attack_tes t_cases	—	attack_re port	nowe_ryzyk a_Księga
Blue Team	lessons_lea rned	izolacja_re quest	—	security_r egression	attack_pa ttern_upd ate	—	incydent_lo g
Governance	book_const raints	compliance_infra	contract_c ompliance	qa_requir ements_K sięga	red_polic y_bounds	blue_poli cy_bound s	—

council_composition.yaml — przykłady dla D3, D4, D5

YAML

```
# council_composition.yaml - przykłady dynamiczne
# Wygenerowany automatycznie przez Orchestrator
# Archiwizowany przez Evidence Archivist z SHA-256

---
# Przykład 1: D3 – Refactor Path modułu VPS
council_session_id: "cs-d3-vps-refactor-20260415"
decision_class: "D3"
decision_type: "refactor_path"
affected_department: "platform"
proposal_summary: "Refaktoryzacja VPS – zmiana kontraktu state machine"
members:
  - role_id: "gov-lead"
    reason: "permanent_d3plus"
    weight: 1.00
    model: "claude-opus-4"
  - role_id: "gov-compliance"
    reason: "permanent_d3plus"
    weight: 1.25
    model: "gemini-2.5-pro"
  - role_id: "platform-lead"
    reason: "affected_department_lead"
    weight: 1.00
```



```

    model: "claude-opus-4"
  - role_id: "red-lead"
    reason: "d3_rebuild_refactor_mandatory"
    weight: 1.00
    model: "grok-3"
  - role_id: "arch-chief"
    reason: "d3_rebuild_refactor_mandatory"
    weight: 1.25
    model: "gemini-2.5-pro"
quorum_required: "5/5"
threshold: "jednomyślność D3"
governance_veto_active: true
timeout_s: 1800
compact_injected: true # pre-D3 canon_compact.md (Plan 16)

---
# Przykład 2: D4 – Critical rebuild modułu PHANTOM
council_session_id: "cs-d4-phantom-rebuild-20260415"
decision_class: "D4"
decision_type: "module_rebuild"
affected_department: "implementation"
proposal_summary: "Greenfield rebuild PHANTOM – nowy target architecture"
members:
  - role_id: "gov-lead"
    reason: "permanent_d3plus"
    weight: 1.00
  - role_id: "gov-compliance"
    reason: "permanent_d3plus"
    weight: 1.25
  - role_id: "impl-lead"
    reason: "affected_department_lead"
    weight: 1.00
  - role_id: "red-lead"
    reason: "d3_rebuild_refactor_mandatory"
    weight: 1.00
  - role_id: "arch-chief"
    reason: "d3_rebuild_refactor_mandatory"
    weight: 1.25
  - role_id: "platform-lead"
    reason: "d4_mandatory"
    weight: 1.00
human_approval_required: true
human_gate: "operator SYLION R-ACP-01"
quorum_required: "6/6"
threshold: "jednomyślność + human approve"
governance_veto_active: true
timeout_s: null
compact_injected: true

---
# Przykład 3: D5 – SYSTEM_REBUILD_MODE (pełny rebuild)
council_session_id: "cs-d5-system-rebuild-20260415"
decision_class: "D5"
decision_type: "greenfield_systemic"
affected_department: "all"
proposal_summary: "Pełny rebuild SYLION – zmiana architektury bezpieczeństwa"
members:
  - role_id: "arch-chief"

```

```

    department: "architecture"
    weight: 1.25
    model: "gemini-2.5-pro"
  - role_id: "platform-lead"
    department: "platform"
    weight: 1.00
    model: "claude-opus-4"
  - role_id: "impl-lead"
    department: "implementation"
    weight: 1.00
    model: "claude-opus-4"
  - role_id: "qa-lead"
    department: "testing"
    weight: 1.00
    model: "gpt-5"
  - role_id: "red-lead"
    department: "red_team"
    weight: 1.00
    model: "grok-3"
  - role_id: "blue-lead"
    department: "blue_team"
    weight: 1.00
    model: "claude-opus-4"
  - role_id: "gov-lead"
    department: "governance"
    weight: 1.00
    model: "claude-opus-4"
  - role_id: "ext-reviewer-01"
    department: "external"
    type: "external_red_team_or_second_operator"
    weight: 1.00
    model: "human_or_external_agent"
human_approval_required: true
external_review_required: true
quorum_required: "8/8"
threshold: "jednomyślność + human + external review"
governance_veto_active: true
timeout_s: null
compact_injected: true
external_review_log: "external_review_log.md" # Plan 13

```

team_change_proposal.md — szablon propozycji zmiany składu

YAML

```

# team_change_proposal.md
proposal_id: "tcp-2026-001"
created_at: "2026-04-15T10:00:00Z"
proposed_by_department: "red_team"
proposed_by_role: "red-lead"

change_type: "add_agent"
# Dozwolone: add_agent | change_weight | new_role | new_subdepartment
#           merge_departments | dissolve_department

target_department: "red_team"

```

```

new_role:
  title: "Social Engineering Analyst"
  model_id: "claude-opus-4"
  display_name: "Claude Opus 4"
  weight_board: 1.00
  tier: "CHIEF"
  responsibilities:
    - "Testowanie podatności na prompt injection w środowiskach multi-agent"
    - "Scenariusze ingerencji społecznej w pipeline"

justification: >
  Dział Red Team zidentyfikował lukę w pokryciu testów: brak dedykowanej
  roli dla scenariuszy prompt injection w architekturze wieloagentowej.

book_reference: >
  Księga 7.4: Pipeline MUSI testować wszystkie wektory ataku istotne
  dla architektury wieloagentowej.

decision_class_required: "D3"
human_approval_required: true
impact_assessment:
  blast_radius: "niski – nowa rola, brak zmian istniejących ról"
  cost_impact: "+1 agent token budget Red Team"
  org_chart_change: true

status: "pending_council" # pending_council | approved | rejected | deferred

```

HARD GATE'Y

- G-ORG-01: Zmiana org_structure.yaml bez D3+ decyzji z human approve: ZABLOKOWANA. Pipeline odmawia wczytania niezatwierdzonego org_structure.yaml.
- G-ORG-02: Governance Lead nieobecny w Council D3+: ZABLOKOWANE. Kworum nieosiągalne bez gov-lead. Brak obecności = ABSTAIN = blokada Council.
- G-ORG-03: Weto etyczne Governance Lead (REJECT): automatyczna eskalacja do D4, bez możliwości nadpisania przez pozostałych członków Council.
- G-ORG-04: D5 bez External Reviewer: ZABLOKOWANE. System nie może przeprowadzić D5 bez zewnętrznego recenzenta (drugi operator lub zewnętrzny red team).
- G-ORG-05: Zmiana Księgi bez D4/D5: ZABLOKOWANA. Book Change Proposal wymaga min. D4 (zmiana lokalna) lub D5 (fundamenty).
- G-ORG-06: Nowy model LLM bez pełnego onboarding Plan 06 (Discovery→Sandbox→Shadow→Promotion): ZABLOKOWANY. Model nie uzyskuje wagi Board ani prawa uczestnictwa w Council.
- G-ORG-07: Dział ze statusem degraded w Council D3+: OSTRZEŻENIE w logach. Dział ze statusem reorg: ESKALACJA sesji do D4.
- G-ORG-08: Pre-D3 compact (Plan 16) nie wstrzyknięty przed Council: ZABLOKOWANE. Council nie może głosować bez aktualnego canon_compact.md.

TESTY

- T-ORG-01: Walidacja org_structure.yaml: wszystkie 7 działów obecnych, role_id unikalne, modele zarejestrowane.

- T-ORG-02: Council D3 refactor: Governance Lead + Compliance Officer + affected_lead + Red Lead + Chief Architect. Kworum 5+.
- T-ORG-03: Council D4 rebuild: j.w. + Platform Lead. Kworum 6+. Human gate aktywny, pipeline czeka na Operatora R-ACP-01.
- T-ORG-04: Council D5: wszyscy 7 leadów + External Reviewer. Quorum 8/8. Timeout null. external_review_log.md zapisany.
- T-ORG-05: Weto Governance Lead: jeden REJECT od gov-lead → status Council = ESCALATED_D4, niezależnie od pozostałych głosów.
- T-ORG-06: team_change_proposal add_agent bez D3+ decyzji → org_structure.yaml niezmieniony, zmiana odrzucona.
- T-ORG-07: Pre-D3 compact: sprawdzić compact_injected: true w council_composition.yaml przed każdą sesją Council D3+.
- T-ORG-08: Status degraded → WARN w logach Council. Status reorg → eskalacja sesji do D4.
- T-ORG-09: TRIGGER new_target_architecture → Orchestrator dostarcza artefakt do Impl., Red Team i Blue Team.
- T-ORG-10: Model bez etapu PROMOTION nie uczestniczy w Council i nie ma wagi Board (weryfikacja w sandbox).

STATUSY I EVIDENCE

Stany działu

Status	Znaczenie	Wpływ na Council	Akcja
healthy	Dział działa prawidłowo, wszystkie role obsługiwane	Pełne prawa	Brak
degraded	Co najmniej jedna rola niedostępna, dział działa	WARN w logach Council	Alert, Team Change Proposal
reorg	Dział w trakcie reorganizacji (po zatwierdzonej D3+)	Eskalacja Council do D4	Pipeline czeka na zakończenie reorgu
suspended	Dział czasowo zawieszony (incydent lub decyzja D4)	ZABLOKOWANY w Council	D4 wymagana do przywrócenia

Artefakty Evidence Plan 15

- org_structure.yaml: centralny rejestr org. Każda wersja archiwizowana z SHA-256 + timestamp przez Evidence Archivist.
- team_change_proposal.md: propozycja zmiany składu. Status: pending_council | approved | rejected | deferred.
- council_composition.yaml: dynamiczny skład Council dla danej decyzji. Archiwizowany razem z decyzją.
- interaction_log.json: log wymiany artefaktów między działami (TRIGGER events + artefakty + timestampy).

- department_status_report.yaml: dzienny raport statusów 7 działów (healthy/degraded/reorg/suspended). Część daily cadence.

Tabela klas decyzji D0–D5 w kontekście Plan 15

Klasa	Nazwa	Zastosowanie w Plan 15	Skład Council	Gate
D0	Automatyczna	Odczyt org_structure.yaml, status check	—	Auto
D1	Trivial	Zmiana display_name, poprawka metadanych	1 agent Senior+	1 agent
D2	Standard	Minor zmiana wagi, update responsibilities	2 agenci + Review	2 agenci
D3	Significant	Dodanie roli, zmiana modelu, nowy agent, zmiana wagi Board	Gov Lead + Compliance + affected + Red Lead + Arch Chief (4+)	Council 4/4 + Human
D4	Critical	Nowy subdział, fuzja działów, lokalna zmiana Księgi	D3 + Platform Lead (5+) + human gate	Council + Human
D5	Greenfield/Systemic	Rozwiązanie działu, zmiana fundamentów Księgi	Wszyscy 7 leadów + External Reviewer	Council + Human + External

REBUILD · Rebuildability over Lineage — Plan 15

Organizacja 7 działów jest zaprojektowana tak, by pipeline SYLION mógł ewoluować, refaktoryzować i pełni przebudowywać sam siebie. Dział Architektury zapewnia intelektualne przywództwo, Red Team i Blue Team gwarantują bezpieczeństwo na każdym etapie, a Governance pilnuje, że Księga pozostaje jedynym źródłem prawdy — nawet gdy cały system jest przepisywany od zera.

PLAN 15 · ROZSZERZENIE v3.1 · REVERSIBILITY

Multi-Team · v3.1 EXPAND

15.9. Nowe role v3.1 — Compact Validator Agent i Rollback Responder

REVERSIBILITY · Meta-zasada wymaga agentów — nie tylko dokumentów

Reversibility & Fidelity nie może być egzekwowana tylko przez bramy dokumentowe. Każda meta-zasada wymaga aktywnego aktora, który ją pilnuje w czasie rzeczywistym. v3.1 wprowadza dwie nowe stałe role w ramach istniejących siedmiu działów: Compact Validator Agent w Dziale Testów oraz Rollback Responder w Blue Team. Oba obowiązują non-stop, oba mają prawo veta w swoich domenach.

15.9.1. Compact Validator Agent (Dział Testów)

Compact Validator Agent to agent stały, zasiedlony w Dziale Testów (obok ról Contract Tester, Load Tester, Regression Engineer). Jego jedyna odpowiedzialność: uruchamianie Compact Fidelity Test na każdym generowanym canon_compact.md przed wypuszczeniem go do production inject path. CFT protokół w pełni: patrz Plan 16 sekcja 16.11.

Atrybut	Wartość
Dział	Testy (jeden z 7 stałych)
Typ roli	agent stały (non-stop active)
Poziom	D2 (decyzje operacyjne) + eskalacja do Council przy FAIL
Prawo veta	TAK — blokuje publikację compacta który nie przeszedł CFT
Eskalacja	Chief Safety + Architect przy powtarzalnym FAIL (3×)
Modele	heterogeniczne pary (GPT-5 / Claude Opus / Gemini) — wymagana zmiana
Triggery aktywacji	każdy canon_compact.md build, PRE-bundle, PRE-D3+, 24h rhythm
Artefakt wyjściowy	compact_fidelity_report.json (schema w Aneksie I)

Workflow Compact Validator Agent

1. Trigger — nowy canon_compact.md pojawia się w staging path.
2. Load full corpus — agent ładuje pełne PDF v3.1 (źródło prawdy).
3. Spawn alpha — agent_alpha (model A) podejmuje decyzje na zadanym golden_set, input=pełne PDF.
4. Spawn beta — agent_beta (model B, INNY niż A) podejmuje te same decyzje, input=canon_compact.md.
5. Comparator — oblicza hard metrics (class_match, verdict_match, gate_requirements_match) i soft metric (rationale_overlap cosine).

6. Determinism check — N=3 runy przy T=0, majority vote.
7. Golden set parity — osobno: czy alpha+beta trafiają w ground truth golden_set?
8. Verdict — PASS jeśli wszystkie hard metrics=true rationale_overlap≥0.85 golden_set parity≥konfigurowany próg.
9. Publish or veto — PASS → compact publikowany; FAIL → veto + eskalacja.

FIDELITY · Heterogeniczność modeli jest niezbędna

Gdyby alpha i beta były tym samym modelem, CFT testowałby tylko „czy compact zawiera wystarczająco kontekstu aby ten konkretny model powtórzył własną decyzję”. Różne modele dają niezależne perspektywy — jeśli obaj niezależnie dochodzą do tej samej decyzji, compact jest wystarczający. To klasyczne „two oracles” z kryptoanalizy.

15.9.2. Rollback Responder (Blue Team)

Rollback Responder to dyżurny agent w Blue Team — odpowiednik on-call SRE. Istnieje non-stop, non-interruptible: każda decyzja D3+ z cutover musi mieć wyznaczonego Rollback Responder, który jest przypisany przez cały LPW (STABILIZATION + WARM_STANDBY). Jest autoryzowany do wyzwolenia MANUAL_FAST_ROLLBACK (patrz Plan 07 sekcja 7.11.1).

Atrybut	Wartość
Dział	Blue Team (jeden z 7 stałych)
Typ roli	agent stały + on-call dyżurny podczas LPW
Poziom	D3 operacyjnie (może wyzwolić MANUAL_FAST_ROLLBACK samodzielnie)
Prawo veta	TAK — może zablokować cutover (G-CUT-02) jeśli widzi czerwone flagi
Eskalacja	Chief Safety natychmiast przy każdym wyzwolonym rollback
Koegzystencja	współpracuje z VPS auto-rollback (Plan 07 sekcja 7.11.2)
Artefakt wyjściowy	rollback_post_mortem.md + post_cutover_review_report.md
Obowiązki LPW	monitoring STABILIZATION, triggerowanie rehearsals, veto DECOMMISSION

Rollback Responder vs VPS auto-rollback — podział odpowiedzialności

Scenariusz	Auto VPS	Rollback Responder	Chief Safety	Council
error_rate 2× baseline 10min	TRIGGER	monitor + notify	notify	—
contract invariant breach	TRIGGER	post-mortem lead	notify	—
user reports degradation (soft)	—	TRIGGER MANUAL_FAST	approve	—
incydent security HIGH	TRIGGER	execute + coord DFIR	lead	—

Scenariusz	Auto VPS	Rollback Responder	Chief Safety	Council
niepokojący trend bez breach	—	propose COUNCIL_ROLLBACK	—	decyzja D3 fast
WARM_STANDBY rollback	—	propose	approve	decyzja D3+

REVERSIBILITY · Dwa równoległe mechanizmy — to nie redundancja, to defense-in-depth

VPS auto-rollback pokrywa hard metrics (szybkie, mechaniczne, bez błędu). Rollback Responder pokrywa soft signals (trend, user feedback, intuicja operatora). Razem pokrywają pełen spektrum: od „oczywista awaria” po „coś jest nie tak, ale nie wiem co”. Żaden z nich nie jest wystarczający sam.

15.9.3. Aktualizacja tabeli 7 stałych działów

Plan 15 v3.0 wylistował 7 działów z ich głównymi rolami. v3.1 dodaje nowe role do dwóch z tych działów (Testy, Blue Team), bez tworzenia nowych działów — Rebuildability pozostaje kanoniczna w 7 działach.

Dział	Role v3.0	NOWE role v3.1
Architektura	Kanon Interpreter, Target Architect, Blast Analyst	— (bez zmian)
Platforma	ACP Operator, VPS Controller, Bundle Engineer	— (bez zmian)
Implementacja	Module Builder, Contract Migrator, Legacy Adapter	— (bez zmian)
Testy	Contract Tester, Load Tester, Regression Engineer	Compact Validator Agent
Red Team	Adversarial Tester, Canon Challenger, Penetration Agent	— (bez zmian)
Blue Team	SOC Monitor, Telemetry Analyst, Incident Lead	Rollback Responder
Governance	Evidence Curator, Gate Enforcer, Compliance Officer	— (bez zmian)

ZASADA · Dwa nowe agenci, zero nowych działów

v3.1 respektuje kanon 7 działów. Reversibility i Fidelity NIE wymagają nowego działu — wymagają wzmocnienia istniejących (Testy dla Fidelity, Blue Team dla Reversibility). To zgodne z zasadą Rebuildability over Lineage: minimalnie inwazyjne rozszerzenie organizacji.

PLAN 15 · ROZSZERZENIE v3.3 · EFFICIENCY

Cztery nowe role stałe w organizacji v3.3

§15.10 — Cztery nowe role stałe v3.3 — Code Optimizer + Performance Engineer + Memory Auditor + Cost Optimizer

Wersja v3.3 wprowadza cztery nowe role stałe rozrzucone po dwóch z siedmiu działów: Implementacja (1 rola) oraz Testy i walidacja (3 role). Łączna obsada Boardu wzrasta z 21 ról stałych (v3.1) do 25 ról stałych. Nowe role realizują meta-zasadę „Efficiency by Default”: każda zmiana w pipeline musi być sprawdzona pod kątem jakości kodu, wydajności runtime, profilu pamięci i kosztu operacyjnego zanim trafi do bundle stage. Branżowe odpowiedniki tych ról to: Refactoring Agent / Technical Debt Analyzer (nazwa SYLION: Code Optimizer Agent), Performance Engineer (analogia do roli SRE), Memory Auditor (custom, brak powszechnego odpowiednika), oraz Cost Optimizer (analogia do roli FinOps Engineer).

EFFICIENCY · Efficiency by Default — cztery wymiary kontroli jakości

v3.3 dodaje 4 nowe role + 11 nowych gate’ów + 4 nowe artefakty Evidence Pack. Każdy modul przechodzi obligatoryjnie przez: (1) analizę code quality [Code Optimizer], (2) walidację wydajności runtime [Performance Engineer], (3) profil zużycia pamięci [Memory Auditor], (4) audit kosztu operacyjnego [Cost Optimizer]. Brak jakiegokolwiek z czterech reportów blokuje bundle build.

§15.10.1 — Code Optimizer Agent (Dział Implementacji)

Code Optimizer Agent działa w Dziale Implementacji, nie w Testach — ponieważ jego rola to wczesne wykrywanie i rekomendowanie refaktoringu we współpracy z Module Builder, przed oddaniem modułu do testów. Ma wagę 1.0 w Council (identycznie jak inne role implementacyjne). Posiada hard veto na bundle build — jest to jeden z dwóch hard vetów w całym pipeline (obok Compact Validator z v3.1, który posiada hard veto na cutover). Override hard veto Code Optimizer Agent wymaga kwalifikowanego quorum Council 4/4 oraz uzasadnienia D3 z action planem i deadlinem re-fix bloat_score.

Odpowiedzialność	Opis
Analiza bloat	Ciągły monitoring wskaźnika bloat_score per moduł; alert przy przekroczeniu progu G-EFF-01
Walidacja guardów G-EFF-01..04	Sprawdzenie czterech efficiency guards przed każdym bundle build
Sygnatura artefaktu	Autoryzuje i sygnuje code_efficiency_report.json jako obowiązkowy element Evidence Pack
Rekomendacje refaktoringu	Propozycje refaktoringu na poziomie modułu; współpraca z Module Builder

Odpowiedzialność	Opis
Monitoring Cognitive Complexity	Trend Cognitive Complexity per commit; eskalacja przy systematycznym wzroście
Weryfikacja anti-pattern LLM	Detekcja LLM-specific anti-patternów (Over-Specification, excessive scaffolding)
Raport tygodniowy	Weekly efficiency report do Governance i Chief Architect
Eskalacja D3	Automatyczna eskalacja D3 przy systemic regression (bloat_score > threshold przez 3+ moduły)

Uprawnienia hard veto — Code Optimizer Agent:

Faza pipeline	Może veto?	Override
bundle build	TAK (hard veto)	Council 4/4 + uzasadnienie D3 z action plan + deadline re-fix bloat_score
cutover	NIE	Compact Validator owns (v3.1)
rollback	NIE	Rollback Responder owns (v3.1)
canon_compact	NIE (read-only consumer)	—

KRYTYCZNE · *Hard veto Code Optimizer Agent — jedyna obrona przed systemową degradacją*

Hard veto Code Optimizer Agent jest jedynym mechanizmem obrony pipeline przed systemową degradacją code quality. Override Council 4/4 MUSI zawierać action plan z deadline na re-fix bloat_score. Brak action planu unieważnia override i blokuje bundle build do czasu naprawy. Powtarzający się override bez naprawy auto-eskaluje do D4 (Human Required).

```
# agent_registry.yaml – definicja roli code_optimizer_agent
code_optimizer_agent:
  id: COA-001
  department: implementacja
  council_weight: 1.0
  hard_veto_phases:
    - bundle_build
  soft_veto_phases: []
  decision_class_max_proposable: D3
  model_assignment:
    primary: claude-opus-4
    fallback: gemini-2.5-pro
  escalation_path:
    systemic_regression: D3_council_fast
    repeated_override: D4_human_required
  artifacts:
    - code_efficiency_report.json
  guards: [G-EFF-01, G-EFF-02, G-EFF-03, G-EFF-04]
```

Branżowe podejście: termin Refactoring Agent / Technical Debt Analyzer pojawia się w literaturze dotyczącej automatyzacji code quality. Badania Ox Security wskazują, że LLM-generated code wykazuje 80–90% Over-Specification (nadmiarowe scaffolding, defensywne importy, speculative abstraction), co jest głównym źródłem bloat w systemach LLM-native.

§15.10.2 — Performance Engineer (Dział Testów i Walidacji)

Performance Engineer działa w Dziale Testów i Walidacji. Jego misją jest pomiar regresji wydajności względem ustalonego baseline per moduł, nie zaś pomiar capacity (to zadanie Load Testera z v3.0 — patrz §15.10.8). Posiada soft veto wymagające Council 3/4 do override. Metodologia opiera się na Google SRE error budget policy [Google SRE Workbook] oraz USE Method Brendana Gregga [Utilization, Saturation, Errors].

Odpowiedzialność	Opis
Stress testing	Obciążeniowe testy wydajności per moduł i per integracja
Baseline performance	Wyznaczanie i utrzymanie baseline per moduł; rewizja przy każdym major release
Sygnatura artefaktu	Autoryzuje i sygnuje performance_baseline.yaml jako element Evidence Pack
Walidacja guardów G-PERF-01..03	Sprawdzenie trzech performance guards: p50/p95/p99 thresholds, error budget, latency regression
Monitoring trendów	Ciągły monitoring p50/p95/p99; dashboard + alert przy deterioration trend
Propozycje optymalizacji	Rekomendacje optymalizacji wydajności dla Module Builder i Contract Migrator
Eskalacja regresji	Automatyczna eskalacja D3 przy regression > 10% względem baseline

```
# agent_registry.yaml – definicja roli performance_engineer
performance_engineer:
  id: PE-001
  department: testy_i_walidacja
  council_weight: 1.0
  hard_veto_phases: []
  soft_veto_phases:
    - bundle_build
    - cutover
  soft_veto_override_quorum: 3/4
  decision_class_max_proposable: D3
  model_assignment:
    primary: gemini-2.5-pro
    fallback: gpt-5
  escalation_path:
    regression_10pct: D3_council_fast
  artifacts:
    - performance_baseline.yaml
  guards: [G-PERF-01, G-PERF-02, G-PERF-03]
```

Metodologia USE Method [Brendan Gregg]: każdy zasób systemu oceniany jest przez pryzmat Utilization, Saturation i Errors. Google SRE error budget policy [Google SRE Workbook] stanowi podstawę dla definicji progu eskalacji: regresja > 10% względem baseline p95 wyczerpuje error budget i wymaga natychmiastowego soft veto + eskalacji Council.

§15.10.3 — Memory Auditor (Dział Testów i Walidacji)

Memory Auditor to custom rola SYLION bez bezpośredniego odpowiednika branżowego. Jego misja: systematyczny audit profilu pamięci każdego modułu pipeline, detekcja wycieków pamięci i rekomendacje GC tuning. Działa w Dziale Testów i Walidacji; posiada soft veto wymagające Council 3/4 do override. Workflow oparty na 6-step leak detection z research: baseline → sampling → heap snapshot → comparison → root cause → fix verification.

Odpowiedzialność	Opis
Heap snapshot analysis	Periodyczna analiza heap snapshotów per moduł; porównanie z baseline
Leak detection runs	Automatyczne uruchomienie 6-step leak detection przy każdym bundle candidate
Sygnatura artefaktu	Autoryzuje i sygnuje memory_profile.json jako element Evidence Pack
Walidacja guardów G-MEM-01..02	Sprawdzenie dwóch memory guards: RSS limit i leak alert threshold
Monitoring RSS curves	Ciągły monitoring krzywej RSS (Resident Set Size) per moduł i per deployment
GC tuning recommendations	Rekomendacje strojenia garbage collector; współpraca z Platform Lead
Eskalacja leak alert	Soft veto + D3 eskalacja przy wykryciu aktywnego wycieku pamięci

```
# agent_registry.yaml – definicja roli memory_auditor
```

```
memory_auditor:
  id: MA-001
  department: testy_i_walidacja
  council_weight: 1.0
  hard_veto_phases: []
  soft_veto_phases:
    - bundle_build
  soft_veto_override_quorum: 3/4
  grace_period_on_leak_alert: 24h
  decision_class_max_proposable: D3
  model_assignment:
    primary: gpt-5
    fallback: claude-opus-4
  escalation_path:
    active_leak_detected: D3_council_fast
  artifacts:
    - memory_profile.json
  guards: [G-MEM-01, G-MEM-02]
```

```
    workflow: [baseline, sampling, heap_snapshot, comparison, root_cause, fix_verification
  ]
```

6-step leak detection workflow (z research): (1) ustanowienie memory baseline, (2) sampling pamięci w stałych interwałach, (3) wykonanie heap snapshot, (4) porównanie snapshotów (diff), (5) identyfikacja root cause (retaining path), (6) weryfikacja skuteczności fix po wdrożeniu. Memory Auditor jest jedynym agentem uprawnionym do emisji sygnału leak_alert w Evidence Pack.

§15.10.4 — Cost Optimizer (Dział Testów i Walidacji)

Cost Optimizer to odpowiednik FinOps Engineer w kontekście pipeline LLM-native. Jego misja: śledzenie kopert kosztowych (cost envelopes) per moduł i per run, optymalizacja routingu modeli oraz analiza efektywności prompt caching. Działa w Dziale Testów i Walidacji; posiada soft veto wymagające Council 3/4 do override. Badania ProjectDiscovery wskazują na 59–70% redukcję kosztów przy właściwym prompt caching. Routing tierów: Haiku (rutynowe), Sonnet (standard), Opus (decision-class D3+).

Odpowiedzialność	Opis
Cost envelope tracking	Ciągłe śledzenie koperty kosztowej per moduł, per run i per miesiąc
Model routing optimization	Rekomendacje routingu modeli: Haiku / Sonnet / Opus zgodnie z decision class
Prompt caching analysis	Analiza efektywności cache’owania promptów; cel: 59–70% redukcji kosztu
Sygnatura artefaktu	Autoryzuje i sygnuje cost_envelope.json jako element Evidence Pack
Walidacja guardów G-COST-01..02	Sprawdzenie dwóch cost guards: budget_utilization threshold i cost_per_token limit
Monthly spend forecasting	Prognoza miesięcznych wydatków; raport do Governance i Chief Architect
Eskalacja budget overrun	Soft veto + D3 eskalacja przy budget_utilization > 90%

```
# agent_registry.yaml – definicja roli cost_optimizer
cost_optimizer:
  id: C0-001
  department: testy_i_walidacja
  council_weight: 1.0
  hard_veto_phases: []
  soft_veto_phases:
    - bundle_build
    - cutover
  soft_veto_override_quorum: 3/4
  escalation_threshold_budget_utilization: 0.90
  decision_class_max_proposable: D3
  model_assignment:
    primary: gemini-2.5-pro
    fallback: gpt-5
  escalation_path:
    budget_utilization_gt_90pct: D3_council_fast
```

```
cost_overrun_gt_30pct: D3_council_fast_plus_budget_revision
artifacts:
  - cost_envelope.json
guards: [G-COST-01, G-COST-02]
model_routing_tiers:
  haiku: routine_tasks
  sonnet: standard_decisions
  opus: D3_plus_decisions
```

Badania ProjectDiscovery potwierdzają 59–70% redukcję kosztów przy skutecznym prompt caching w systemach LLM-native. Model routing tierów: Haiku dla zadań rutynowych (monitoring, logging, formatowanie), Sonnet dla standardowych decyzji operacyjnych (D1–D2), Opus wyłącznie dla D3+ decision classes. Cost Optimizer jest jedynym agentem uprawnionym do propozycji zmiany model routing assignment w agent_registry.yaml.

§15.10.5 — Aktualizacja Boardu i Council voting weights (v3.3: 25 ról)

Board v3.3 liczy 25 ról stałych w 7 działach. Council quorum dla D3 pozostaje 4/4 — jednak z nowym wymogiem: dla każdej decyzji D3 z wpływem na bloat/perf/mem/cost przynajmniej jeden członek Council musi pochodzić z grupy 4 ról efficiency (Code Optimizer, Performance Engineer, Memory Auditor lub Cost Optimizer).

Dział	Rola	Waga	Veto type	Decision classes proposable
Architektura	Kanon Interpreter	1.25	soft	D0–D5
Architektura	Target Architect	1.25	soft	D0–D4
Architektura	Blast Analyst	1.0	soft	D0–D3
Platforma	ACP Operator	1.0	soft	D0–D3
Platforma	VPS Controller	1.0	soft	D0–D3
Platforma	Bundle Engineer	1.0	soft	D0–D2
Implementacja	Module Builder	1.0	soft	D0–D3
Implementacja	Contract Migrator	1.0	soft	D0–D3
Implementacja	Legacy Adapter	1.0	soft	D0–D2
Implementacja	Code Optimizer Agent	1.0	hard (bundle build)	D0–D3
Testy	Contract Tester	1.0	soft	D0–D3
Testy	Load Tester	1.0	soft	D0–D3
Testy	Regression Engineer	1.0	soft	D0–D3
Testy	Compact Validator Agent	1.0	hard (cutover)	D0–D3
Testy	Performance Engineer	1.0	soft	D0–D3
Testy	Memory Auditor	1.0	soft	D0–D3
Testy	Cost Optimizer	1.0	soft	D0–D3
Red Team	Adversarial Tester	1.0	soft	D0–D3
Red Team	Canon Challenger	1.0	soft	D0–D4
Red Team	Penetration Agent	1.0	soft	D0–D3
Blue Team	SOC Monitor	1.0	soft	D0–D3
Blue Team	Telemetry Analyst	1.0	soft	D0–D3
Blue Team	Incident Lead	1.0	soft	D0–D4
Blue Team	Rollback Responder	1.0	soft (cutover)	D0–D3

Dział	Rola	Waga	Veto type	Decision classes proposable
Governance	Evidence Curator	1.0	soft	D0–D3
Governance	Gate Enforcer	1.25	soft	D0–D4
Governance	Compliance Officer	1.0	soft	D0–D4

EFFICIENCY · Council quorum D3 — nowy wymóg efficiency representative

Quorum D3 pozostaje 4/4. Nowy wymóg v3.3: każda decyzja D3 dotycząca bloat, performance, memory lub cost MUSI mieć przynajmniej jednego reprezentanta z grupy efficiency (Code Optimizer Agent / Performance Engineer / Memory Auditor / Cost Optimizer) jako członka Council. Bez efficiency representative decyzja D3 w tej domenie jest niezgodna z v3.3.

§15.10.6 — Workflowy współpracy między czterema nowymi rolami

Cztery nowe role pracują w sekwencyjno-równoległym triangu: Code Optimizer Agent działa wcześniej w Implementacji, wykrywając problemy strukturalne przed przekazaniem modułu do testów. Troje agentów w Testach (Performance Engineer, Memory Auditor, Cost Optimizer) mierzy efekty runtime równolegle, niezależnie od siebie. Wszystkie cztery sygnują swoje artefakty zanim Bundle Engineer może uruchomić bundle stage.

```
# ASCII flow – pipeline z nowymi rolami efficiency (v3.3)

Module Builder
  |
  v
[Code Optimizer Agent review] <-- hard veto na bundle build
  | (code_efficiency_report.json signed)
  v
Module Tester (Contract Tester + Load Tester + Regression Engineer)
  |
  v
+---+---+---+ (równoległe / parallel)
|         |         |
v         v         v
[Perf Eng] [Mem Auditor] [Cost Optimizer]
|         |         |
v         v         v
performance_ memory_ cost_
baseline.yml profile.json envelope.json
|         |         |
+---+---+---+ (merge / Evidence Pack)
  |
  v
Bundle stage <-- wszystkie 4 artefakty wymagane
  |
  v
Compact Validator (cutover gate) <-- hard veto na cutover
```

Tabela handoffów między rolami:

From	To	Artefakt	SLA (czas)
Module Builder	Code Optimizer Agent	module_source.zip	≤ 30 min od commit
Code Optimizer Agent	Module Tester	code_efficiency_report.json	≤ 2h od otrzymania
Module Tester	Performance Engineer	test_bundle_candidate	≤ 15 min po test pass
Module Tester	Memory Auditor	test_bundle_candidate	≤ 15 min po test pass
Module Tester	Cost Optimizer	test_bundle_candidate	≤ 15 min po test pass
Performance Engineer	Evidence Curator	performance_baseline.yaml	≤ 4h od otrzymania
Memory Auditor	Evidence Curator	memory_profile.json	≤ 4h od otrzymania
Cost Optimizer	Evidence Curator	cost_envelope.json	≤ 4h od otrzymania
Evidence Curator	Bundle Engineer	evidence_pack_v3.3.zip	≤ 1h po kompletacji

EFFICIENCY · Trójkąt efficiency — wczesna detekcja + pomiar runtime

Cztery nowe role pracują w trójkącie: Code Optimizer w Implementacji wczesnie wykrywa problemy strukturalne (pre-test); troje agentów w Testach mierzy efekty runtime równolegle (post-test). To minimalizuje koszt naprawy: błąd wykryty w Implementacji kosztuje 1x; błąd wykryty po bundle kosztuje 10x; błąd wykryty po cutover kosztuje 100x.

§15.10.7 — Konflikt veto i escalation matrix

Poniższa tabela definiuje wszystkie scenariusze veto i ścieżki override dla czterech nowych ról. Skalowanie eskalacji opiera się na severity: przekroczenie progu podstawowego → soft veto D3; przekroczenie progu krytycznego → hard veto lub auto-eskalacja D4.

Sytuacja	Veto type	Rola weto	Override path
bloat_budget +20%	hard veto	Code Optimizer Agent	Council 4/4 D3 escalation + action plan z deadlinem re-fix
bloat_budget +50%	hard veto	Code Optimizer Agent	Council 4/4 + Human D4 (auto-eskalacja, brak override bez D4 approval)
perf regression 10–20%	soft veto	Performance Engineer	Council 3/4 + dokumentacja impact assessment
perf regression > 20%	soft veto (elevated)	Performance Engineer	Council 4/4 + D3 escalation + remediation plan
leak detected (active)	soft veto	Memory Auditor	Council 3/4 + 24h grace period na fix; brak fix = hard block

Sytuacja	Veto type	Rola weto	Override path
cost overrun > 30%	soft veto	Cost Optimizer	Council 3/4 + obligatoryjna budget revision
budget_utilization > 90%	soft veto	Cost Optimizer	Council 3/4 + monthly spend forecast revision

§15.10.8 — Integracja z dotychczasowymi rolami

Cztery nowe role uzupełniają istniejący skład Boardu — nie zastępują żadnej roli v3.0 ani v3.1. Podział odpowiedzialności jest ścisły:

- Code Optimizer Agent vs Module Builder: Module Builder buduje kod modułu. Code Optimizer Agent ocenia jego jakość strukturalną i efektywność. Nie istnieje między nimi konflikt decyzyjny — Code Optimizer nie może zmieniać kodu bez zgody Module Builder.
- Code Optimizer Agent vs Contract Migrator: Contract Migrator zarządza migracjami kontraktów (evolution/refactor). Code Optimizer ocenia bloat po migracji. Współpraca: Code Optimizer recenzuje diff po migracji kontraktu.
- Performance Engineer vs Load Tester: Load Tester (v3.0) mierzy capacity — ile load system wytrzyma. Performance Engineer mierzy regression — czy nowa wersja jest wolniejsza od baseline. Oba pomiary są niezbędne i uzupełniające.
- Kompletny test stack v3.3 (Dział Testów): Contract Tester + Load Tester + Regression Engineer (v3.0) + Compact Validator Agent (v3.1) + Performance Engineer + Memory Auditor + Cost Optimizer (v3.3) = 7 ról tworzących pełny test stack pokrywający: kontrakty, capacity, regresje funkcjonalne, fidelity, wydajność, pamięć i koszt.

Rola	Dział	Dodana w	Zakres	NIE pokrywa
Module Builder	Implementacja	v3.0	budowanie modułu	ocena jakości strukturalnej
Contract Migrator	Implementacja	v3.0	migracje kontraktów	ocena bloat po migracji
Load Tester	Testy	v3.0	capacity testing	regression vs baseline
Regression Engineer	Testy	v3.0	regresje funkcjonalne	profil pamięci / koszt
Compact Validator Agent	Testy	v3.1	fidelity / cutover gate	code quality / perf
Rollback Responder	Blue Team	v3.1	rollback operations	efficiency metrics
Code Optimizer Agent	Implementacja	v3.3	code quality / bloat	—
Performance Engineer	Testy	v3.3	perf regression	—
Memory Auditor	Testy	v3.3	leak detection / RSS	—
Cost Optimizer	Testy	v3.3	cost envelope / FinOps	—

EFFICIENCY · §15.10 podsumowanie — Efficiency by Default: pełna integracja

v3.3 dodaje 4 nowe role stałe (Code Optimizer Agent, Performance Engineer, Memory Auditor, Cost Optimizer) + 11 nowych gate'ów (4 guardów G-EFF, 3 guardów G-PERF, 2 guardów G-MEM, 2 guardów G-COST) + 4 nowe artefakty Evidence Pack (code_efficiency_report.json, performance_baseline.yaml, memory_profile.json, cost_envelope.json) + 1 nowa kolumna w drabinie D0–D5 (efficiency representative requirement dla D3 z wpływem na bloat/perf/mem/cost). Razem tworzą pełną integrację meta-zasady „Efficiency by Default”: żadna zmiana nie trafia do produkcji bez pozytywnego audytu wszystkich czterech wymiarów jakości.

PLAN 16 · WARSTWA KANONICZNA

Compaction & Memory Protocol

Plan 16 — Compaction & Memory Protocol

Plan 16 definiuje formalny Protokół Compaction — rytmiczny i zdarzeniowy mechanizm „przypominania” systemu sam sobie: co jest zasadą nadrzędną, co jest aktywną decyzją, co jest inwariantem bezpieczeństwa. Protokół odpowiada na fundamentalną słabość wszystkich systemów opartych na modelach językowych: modele AI mają ograniczone okno kontekstu i z czasem „zapominają” ustalenia, które zostały poczynione godziny lub sesje wcześniej. Compaction nie jest „backupem pamięci” — jest regulowanym, audytowalnym i wersjonowanym artefaktem utrzymującym spójność tożsamości systemu w czasie.

ZASADA · Dlaczego compact jest niezbędny

Modele AI lubią zapominać. Co jakiś czas trzeba im dawać compact z przypomnieniem najważniejszych kwestii. Bez formalnego protokołu compaction każdy nowy agent, każda nowa sesja i każdy bundle build zaczynają od pustego kontekstu. Zasady Księgi, inwarianty bezpieczeństwa, ostatnie decyzje D3+ i aktualna strategia implementacji muszą być dostępne zawsze — nie tylko wtedy, gdy ktoś je akurat pamięta. Compaction to nie koszt: to gwarancja ciągłości decyzyjnej pipeline'u.

16.1. Cel planu

Celem Plan 16 jest zapewnienie, że każdy agent i każdy operator w dowolnym momencie działania pipeline'u ma dostęp do skondensowanego, aktualnego i zwalidowanego obrazu stanu systemu. Plan definiuje:

- Artefakt „`canon_compact.md`” — jedyne źródło kompaktowe stanu systemu
- Trzy triggerzy generowania compact (czasowy, pre-D3+, pre-bundle)
- Dwa poziomy compact: Agent-compact i Operator-compact
- Pełny cykl życia compact: generowanie, walidacja, wersjonowanie, dystrybucja, archiwizacja, feedback
- Klasy decyzji dla zmian protokołu compaction
- Integrację z Plan 06 (Agent Lifecycle), Plan 12 (Bundle+ Testing), Plan 13 (Governance), Plan 14 (Greenfield Rebuild), Plan 15 (Multi-Team)

REBUILD · Zasada Rebuildability a Compaction

Rebuildability over Lineage wymaga, żeby system był zdolny do pełnego rebuildu od zera. Compaction jest mechanizmem, który umożliwia ten rebuild — każdy nowy agent spawned w `SYSTEM_REBUILD_MODE` dostaje natychmiast pełny kontekst: czym jest system, jakie są zasady, jaki jest aktualny target architecture, jakie decyzje D3+ zostały podjęte. Bez compact, rebuild oznaczałby utratę instytucjonalnej pamięci pipeline'u. Z compact, rebuild jest kontrolowany i ciągły nawet przy pełnej wymianie warstwy implementacyjnej.

16.2. Trzy triggery generowania compact

Wszystkie trzy triggery są aktywne jednocześnie i niezależne od siebie. Wyzwolenie jednego nie blokuje pozostałych. W przypadku kolizji czasowej (np. cron 00:00 UTC zbiega się z pre-D3+ decyzją) generowane są dwa osobne compacty z własnymi hashami i znacznikami czasu.

Trigger T1 — Rytm czasowy (24h baseline compact)

Cron job uruchamiany codziennie o 00:00 UTC. Generuje pełny baseline compact niezależnie od aktywności pipeline'u. Nawet jeśli żadna decyzja nie była podjęta w ciągu ostatnich 24 godzin, compact jest generowany i archiwizowany — odzwierciedla wtedy „brak zmian” jako stan systemu.

- Harmonogram: cron UTC 00:00 codziennie
- Odpowiedzialny: Governance Scheduler Agent (Dział Governance, Plan 15)
- Klasa decyzji: D1 (trivial — rutynowe generowanie)
- Timeout: max 10 minut na cały cykl Generate→Validate→Version→Distribute
- Jeśli timeout przekroczony: alert P2, compact z poprzedniej doby pozostaje aktywny

Trigger T2 — Pre-D3+ (przed decyzją klasy D3 lub wyższej)

Przed każdą decyzją klasy D3, D4 lub D5 system automatycznie generuje świeży compact i wstrzykuje go do kontekstu Council. Decyzja D3+ nie może być podjęta bez aktualnego compact w kontekście. Jest to hard gate — brak ważnego compact (<6h od wygenerowania) blokuje inicjację procesu decyzji D3+.

- Wyzwalacz: inicjacja procesu decyzji D3/D4/D5 w systemie
- Odpowiedzialny: Council Orchestrator (Dział Governance)
- Klasa decyzji: D1 (generowanie compact przed decyzją wyższego rzędu)
- Hard gate: compact starszy niż 6h = regeneracja obowiązkowa
- Compact wstrzykiwany do system prompt każdego agenta Council

Trigger T3 — Pre-bundle (przed każdym bundle build)

Przed każdym bundle build (Plan 12) compact jest generowany i wstrzykiwany do wszystkich agentów zaangażowanych w ten bundle. Dotyczy zarówno standardowych bundle'ów ewolucyjnych, jak i bundle'ów w trybie `SYSTEM_REBUILD_MODE`. W trybie rebuild compact zawiera dodatkową sekcję: aktualny target architecture i status cutover.

- Wyzwalacz: inicjacja pipeline'u bundle build w Plan 12
- Odpowiedzialny: Bundle Orchestrator (Dział Platformy) + Governance Agent
- Klasa decyzji: D1 (generowanie compact jako część bundle init)
- Scope: compact wstrzykiwany do WSZYSTKICH agentów w bundle, nie tylko liderów
- W SYSTEM_REBUILD_MODE: compact zawiera sekcję target_architecture_status

16.3. Dwa poziomy compact

Protokół definiuje dwa odrębne poziomy compact o różnych odbiorcach, różnej zawartości i różnym sposobie dystrybucji. Oba są generowane z tych samych źródeł kanonicznych, ale różnią się perspektywą i granularnością.

Poziom 1 — Agent-compact

Wstrzykiwany agentom na początku sesji lub turnu. Zaprojektowany tak, żeby zmieścić się w oknie kontekstu modelu bez dominowania nad zadaniem właściwym. Maksymalnie zwięzły, precyzyjny, bez zbędnych kontekstów narracyjnych.

- 3–5 zdań tezy nadrzędnej (Księga compact) — co jest celem systemu
- Zasada Rebuildability: 1 zdanie precyzyjnej definicji
- Aktualne inwarianty bezpieczeństwa: lista z kodami (np. INV-SEC-01..06)
- Ostatnie 5 decyzji D3+ z ich klasą, datą i wynikiem (APPROVE/REJECT/DEFER)
- Aktualny implementation_strategy dla modułów dotyczących tej sesji
- Ostrzeżenia: znane drift patterns, typowe pomyłki agentów w tej domenie
- Tryb systemu: EVOLUTION | REFACTOR | REBUILD | SYSTEM_REBUILD_MODE

Poziom 2 — Operator-compact

Dostarczany człowiekowi w pętli — na żądanie albo rytmicznie (raz dziennie po cron compact). Szerszy niż agent-compact, zawiera kontekst decyzyjny i metryki operacyjne niezbędne do efektywnego nadzoru.

- Stan systemu w perspektywie operatora — co się zmieniło od ostatniego compact
- Wiszące decyzje wymagające zatwierdzenia człowieka (D4/D5 gate)
- Ostatnie 5 decyzji D3+ z uzasadnieniami i skutkami
- Stan 7 działów: healthy | degraded | reorg (Plan 15)
- Ostrzeżenia bezpieczeństwa od Red Team i Blue Team
- Budżet vs spend (Plan 10): aktualne zużycie vs limit
- Propozycje zmian Księgi czekające na akceptację
- Aktywny tryb pipeline i planowane przełączenia ścieżki implementacyjnej

16.4. Artefakt canon_compact.md

Plik tekstowy o stałej strukturze. Maksymalnie 2–3 strony — jeśli dłuższy, agenci zgubią kluczowe informacje w szumie. Generowany automatycznie przez współpracę Działu

Governance i Działu Architektury. Każda wersja jest hashowana (SHA-256) i timestampowana (ISO 8601 UTC). Poprzednie wersje są archiwizowane w Knowledge Base (Plan 03) — nigdy nie są kasowane.

- Lokalizacja aktywna: /pipeline/canon/canon_compact.md
- Archiwum: /pipeline/canon/compact_archive/.md
- Format: Markdown (human-readable i machine-parseable)
- Maksymalny rozmiar: 3000 słów (egzekwowane przez walidator)
- Encoding: UTF-8, polskie znaki, polskie cudzysłowy „”

16.4.1. Schemat sekcji canon_compact.md

Sekcja	Zawartość	Max długość	Właściciel
1. Zasada nadrzędna	Rebuildability over Lineage — 1 zdanie definicyjne	1 zdanie	Architektura
2. Teza pipeline	Co to jest SYLION Pipeline, jaki ma cel	3 zdania	Governance
3. Księga — 7 zasad rdzeniowych	Zwięzła lista 7 zasad bez rozwinięcia	7 pozycji	Governance
4. Inwarianty bezpieczeństwa	Lista z kodami INV-SEC-XX, obowiązujące zakazy/nakazy	max 10 pozycji	Red Team + Blue Team
5. Aktualny baseline	Pipeline version, MZ version, aktywny tryb (EVOLUTION/REBUILD/...)	3 linie	Architektura
6. Ostatnie decyzje D3+	Tabela: data klasa zakres wynik	5 wierszy	Governance
7. Stan 7 działów	Tabela: dział health activity	7 wierszy	Governance
8. Wiszące człowiek-gate	Lista decyzji D4/D5 czekających na zatwierdzenie	max 5 pozycji	Governance
9. Znane drift patterns	Ostrzeżenia: jakie błędy popełniają agenci, co jest typowym dryftem	max 5 pozycji	Red Team
10. Źródła prawdy	Ścieżki: Księga, Dokumentacja, target arch (jeśli rebuild)	3–5 linii	Architektura

16.5. Cykl życia compact

Każdy compact przechodzi przez sześć obligatoryjnych etapów. Pominięcie etapu Validate lub Version czyni compact nieważnym. Dystrybucja niewalidowanego compact jest traktowana jako incydent bezpieczeństwa klasy P1.

1. Generate — Governance Agent + Architektura Agent współpracują: pobierają dane z Księgi, target arch (jeśli aktywna), ostatnich decyzji, statystyk działów i ostrzeżeń Red/Blue Team

2. **Validate** — Red Team Agent przeprowadza hallucination check: weryfikuje każde twierdzenie w compact wobec źródeł kanonicznych; sprawdza kompletność (czy brakuje decyzji D3+, czy inwarianty są aktualne)
3. **Version** — każdy compact dostaje SHA-256 hash zawartości, timestamp ISO 8601 UTC, numer sekwencyjny (compact-NNNN) i metadata: trigger type (cron | pre-D3 | pre-bundle), generated_by, validated_by
4. **Distribute** — agent-compact wstrzykiwany jako sekcja „System Context” w system prompt / session init; operator-compact publikowany jako notyfikacja (push) do człowieka w pętli
5. **Archive** — poprzedni aktywny compact przenoszony do /pipeline/canon/compact_archive/.md, nowy staje się aktywny
6. **Feedback loop** — telemetria: czy agenci użyli compact? Czy cytowali inwarianty? Czy nastąpił drift mimo compact? Wyniki feedbacku wchodzi do kolejnego Generate jako quality_signal

16.6. Relacje z innymi planami

- Plan 06 (Agent Lifecycle): każdy agent T2 i T3 dostaje agent-compact przy init sesji; compact jest częścią onboarding_stages dla T3 — agent musi potwierdzić odbiór compact
- Plan 12 (Bundle + Testing): pre-bundle compact trigger; Red/Blue Team bundles zawierają compact jako kontekst do testów adversarialnych
- Plan 13 (Governance & Evidence): każdy compact jest archiwalnym dowodem stanu systemu; compact hash wchodzi do Evidence pack dla decyzji D3+
- Plan 14 (Greenfield Rebuild Orchestration): SYSTEM_REBUILD_MODE wymaga specjalnego compact z sekcją target_architecture_status i cutover_progress
- Plan 15 (Multi-Team Pipeline Organization): Governance generuje compact, Red Team waliduje; stan 7 działów pochodzi z raportów działu Platformy

PLAN 16 · SPECYFIKACJA WYKONAWCZA

Compaction & Memory Protocol

CEL

Compaction Service implementuje Protokół Compaction jako usługę systemową pipeline'u SYLION v3.0. Serwis jest odpowiedzialny za: (1) wykrywanie triggerów compaction, (2) orkiestrację cyklu Generate→Validate→Version→Distribute→Archive, (3) przechowywanie wersjonowanego archiwum compactów, (4) zbieranie telemetrii o skuteczności compact. Compaction Service jest usługą infrastrukturalną — nie może być wyłączony bez decyzji D4 z zatwierdzeniem człowieka.

- Technologia: Python 3.12, FastAPI, PostgreSQL 16 (metadata), obiektowy storage (compact files), Kafka (events), Redis (cache aktywnego compact)
- Autoryzacja: JWT; rola R-COMPACT-GENERATOR, R-COMPACT-VALIDATOR, R-COMPACT-READER
- Observability: Prometheus /metrics, OTEL tracing, logi strukturyzowane JSON (każdy cykl jako osobny trace)

MODEL

Model decyzyjny Compaction Service: kto generuje, kto waliduje, kiedy compact jest ważny.

Operacja	Kto	Kiedy	Klasa decyzji	Uwagi
Rutynowe generowanie compact (cron)	Governance Agent	co 24h, cron UTC 00:00	D1	Automatyczne
Generowanie pre-D3+	Governance Agent + Architektura Agent	przed każdą decyzją D3/D4/D5	D1	Hard gate
Generowanie pre-bundle	Bundle Orchestrator + Governance Agent	przed każdym bundle build	D1	Automatyczne
Walidacja (hallucination check)	Red Team Agent	każdy compact po Generate	D1	Obligatoryjne
Approve compact (Governance Lead)	Governance Lead Agent	po Validate	D1	Binds compact as valid
Zmiana struktury compact template	Council 4/4	na wniosek dowolnego działu	D3	Dotyczy całej org
Zmiana listy inwariantów bezpieczeństwa	Council 4/4 + Human	na wniosek Red/Blue Team	D4	Krytyczne
Wyłączenie Compaction Service	Council 4/4 + Human	wyjatkowe sytuacje	D4	Zakazane bez D4

Ważność compact

- Compact jest „świeży” przez max 6 godzin od wygenerowania

- Compact starszy niż 6h: może być używany przez agentów do zadań D0/D1, ale NIE dla D3+ ani bundle build
- Compact starszy niż 24h: uznawany za przestarzały — wszystkie agenty dostają flagę COMPACT_STALE, Governance otrzymuje alert
- Compact z failed hallucination check: natychmiast oznaczony INVALID, nigdy nie dystrybuowany; archiwizowany z flagą INVALID dla audytu

ROLE			
Rola	Dział (Plan 15)	Odpowiedzialność w Compaction	Uprawnienia API
Governance Lead Agent	Governance (15.7)	Finalny approve każdego compact; właściciel struktury template; odpowiada za sekcje 2, 3, 6, 7, 8	R-COMPACT-GENERATOR, R-COMPACT-APPROVER
Architektura Agent (senior)	Architektura (15.1)	Wypełnia sekcje 1, 5, 10 compact; w SYSTEM_REBUILD_MODE dostarcza target_architecture_status	R-COMPACT-GENERATOR
Red Team Lead Agent	Red Team (15.5)	Obligatoryjna walidacja każdego compact (hallucination check); dostarcza sekcję 9 (drift patterns); weto dla compact z błędami bezpieczeństwa	R-COMPACT-VALIDATOR
Blue Team Agent	Blue Team (15.6)	Dostarcza dane do sekcji 4 (inwarianty bezpieczeństwa) i ostrzeżeń SOC	R-COMPACT-READER, R-COMPACT-DATA-PROVIDER
Bundle Orchestrator	Platforma (15.2)	Wyzwala trigger T3 (pre-bundle); wstrzykuje compact do agentów bundle'u; raportuje compact_id do telemetry bundle	R-COMPACT-READER, R-COMPACT-TRIGGER
Council Orchestrator	Governance (15.7)	Wyzwala trigger T2 (pre-D3+); weryfikuje czy compact jest świeży przed inicjacją D3+	R-COMPACT-READER, R-COMPACT-TRIGGER
Operator (człowiek)	zewnętrzny	Otrzymuje operator-compact jako notyfikację dzienną i na żądanie; zatwierdza D4/D5 decyzje z compact w kontekście	R-COMPACT-OPERATOR (full-read)

WORKFLOW

WF-CMP-01: Główny workflow generowania compact

Pełny workflow jest wspólny dla wszystkich trzech triggerów. Różni się tylko inicjatorem (cron | pre-D3 | pre-bundle) i ewentualną obecnością sekcji REBUILD_MODE w przypadku trigger T2/T3 w trybie rebuild.

WORKFLOW
trigger (T1: cron 00:00 UTC T2: pre-D3+ decision init T3: pre-bundle build init) v

```

fetch_canonical_sources
|- Ksiega (aktualna wersja, zasady rdzeniowe)
|- target_architecture (jesli REBUILD/SYSTEM_REBUILD_MODE)
|- ostatnie decyzje D3+ (max 5, posortowane desc by date)
|- implementation_strategy per modul (z ACP manifest)
|- stan 7 dzialow (z raportow Platformy, Governance)
|- ostrzezenia Red Team + Blue Team (z ostatnich 48h)
|- budzet vs spend (z Plan 10 Cost Governance)
|
v
draft_compact (Governance Agent + Architektura Agent)
|- wypelnia 10 sekcji canon_compact.md
|- sprawdza dlugosc: max 3000 slow (hard limit)
|- generuje agent_compact (wyciag: sek. 1,2,3,4,5,6,9,10)
|- generuje operator_compact (pelny + sekcja budzet + wiszace D4/D5)
|
v
validate_compact (Red Team Agent -- hallucination check)
|- weryfikuje kazde twierdzenie w compact vs zrodla kanoniczne
|- sprawdza kompletnosc: czy sa wszystkie wymagane sekcje
|- sprawdza aktualnosc: inwarianty bezpieczenstwa (czy INV kody aktualne)
|- sprawdza: ostatnie 5 decyzji D3+ (czy zadna nie zostala pominieta)
|- wynik: VALID | INVALID (z lista bledow)
|- jesli INVALID: compact odrzucony, generuje sie nowy; max 3 iteracje
|
v (tylko jesli VALID)
approve_compact (Governance Lead Agent)
|- finalny przeglad
|- podpisuje compact (agent_id + timestamp w metadata)
|
v
version_and_hash
|- SHA-256 hash zawartosci (bez metadata)
|- numer sekwencyjny: compact-NNNN (globalny licznik)
|- timestamp: ISO 8601 UTC
|- metadata: trigger_type, generated_by, validated_by, approved_by
|- zapis do PostgreSQL: compacts table
|
v
distribute
|- agent_compact -> Redis (klucz: compact:active:agent)
|
| -> Kafka topic: compact.agent.distribute
|
| -> wstrzykniecie do session_init wszystkich aktywnych agentow
|- operator_compact -> notyfikacja push do czlowieka (webhook / UI)
|
| -> zapis do Operator Inbox
|
v
archive_previous
|- poprzedni aktywny compact -> /pipeline/canon/compact_archive/<hash>.md
|- status w PostgreSQL: ARCHIVED
|- nowy compact: status ACTIVE
|
v
telemetry
|- metryka: compact_generation_duration_seconds
|- metryka: compact_validation_iterations_total
|- metryka: compact_distribution_agents_count
|- event Kafka: compact.lifecycle.events {compact_id, trigger, status: ACTIVE}

```

WF-CMP-02: Feedback loop — czy agenci używają compact?

1. Po każdym runie agenta: agent raportuje compact_id który miał w kontekście
2. Telemetria sprawdza: czy agent cytował inwarianty z compact (keyword match)
3. Telemetria sprawdza: czy podjął decyzję niezgodną z inwariantami (drift signal)
4. Wynik: compact_usage_score per agent per run (0.0–1.0)
5. Jeśli compact_usage_score < 0.3 dla agenta T2/T3 w 3 kolejnych runach: alert do Governance — agent może wymagać wzmocnienia compact injection
6. Aggregowane wyniki feedbacku wchodzi do następnego cyklu Generate jako quality_signal (wpływ na priorytetyzację sekcji compact)

WF-CMP-03: Compact w trybie SYSTEM_REBUILD_MODE (Plan 14)

W trybie SYSTEM_REBUILD_MODE compact zawiera dodatkową sekcję 5b (target architecture status). Dwie linie kodu (legacy + target) wymagają dwóch sub-compact: jeden dla agentów pracujących na legacy, drugi dla agentów pracujących na target architecture.

- legacy_compact: zawiera sekcję „Aktualny stan legacy systemu” z fitness score
- target_compact: zawiera sekcję „Target architecture + cutover progress”
- Oba sub-compact mają wspólną sekcję 1-4 (zasady, teza, Księga, inwarianty)
- Trigger T2 pre-D3+ w SYSTEM_REBUILD_MODE: compact zawiera aktualny cutover_strategy.md digest

API / KONTRAKTY

compaction_config.yaml — pełna konfiguracja serwisu

YAML

```
# compaction_config.yaml -- Compaction Service v3.0
# Plik konfiguracyjny Compaction Service. Zmiana struktury sekcji: D3.
# Zmiana invariants_list: D4. Zmiana schedule: D1.

version: "3.0"
service:
  name: "compaction-service"
  owner: "governance-dept"
  pipeline_version: "8.0.0"
  baseline_mz: "3.5.5"

triggers:
  t1_cron:
    enabled: true
    schedule: "0 0 * * *"          # cron UTC 00:00 codziennie
    decision_class: "D1"
    timeout_minutes: 10
    on_timeout: "keep_previous_compact"

  t2_pre_decision:
    enabled: true
```

```
min_decision_class: "D3"          # D3, D4, D5
compact_max_age_hours: 6          # hard gate: jesli starszy, regeneruj
decision_class: "D1"
inject_into: "council_context"
block_decision_without_compact: true

t3_pre_bundle:
  enabled: true
  decision_class: "D1"
  inject_into: "all_bundle_agents"
  rebuild_mode_extra_sections: true

compact_levels:
  agent_compact:
    sections: [1, 2, 3, 4, 5, 6, 9, 10]
    max_words: 1200
    injection_point: "session_init_system_prompt"
    cache_backend: "redis"
    cache_key: "compact:active:agent"
    cache_ttl_seconds: 21600      # 6h

  operator_compact:
    sections: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    include_budget_section: true
    include_pending_human_gates: true
    max_words: 3000
    delivery: "push_notification"
    delivery_endpoint: "${OPERATOR_WEBHOOK_URL}"

lifecycle:
  validity_fresh_hours: 6
  validity_stale_hours: 24
  max_validation_iterations: 3
  on_validation_failure: "reject_and_alert"
  archive_path: "/pipeline/canon/compact_archive"
  active_path: "/pipeline/canon/canon_compact.md"

validation:
  hallucination_check:
    enabled: true
    agent_role: "red-team-lead"
    check_against_sources:
      - ksiega
      - acp_manifest
      - governance_decisions_log
    completeness_checks:
      - all_d3plus_last_5_present
      - all_invariants_current
      - all_7_departments_status_present
    max_words_enforced: 3000

versioning:
  hash_algorithm: "sha256"
  sequence_prefix: "compact"
  timestamp_format: "ISO8601_UTC"
  metadata_fields:
    - compact_id
    - trigger_type
```

```

- generated_by
- validated_by
- approved_by
- generated_at
- status

telemetry:
  kafka_topic: "compact.lifecycle.events"
  prometheus_metrics:
    - compact_generation_duration_seconds
    - compact_validation_iterations_total
    - compact_distribution_agents_count
    - compact_usage_score
    - compact_drift_detected_total

decisions:
  generate_update: "D1"
  change_template_structure: "D3"
  change_invariants_list: "D4"
  disable_compaction_service: "D4"

```

Pełny szablon canon_compact.md — realny przykład (Pipeline 8.0.0)

Poniżej pełny przykład canon_compact.md z prawdziwymi danymi SYLION Pipeline 8.0.0, MZ 3.5.5, 7 działów, tryb EVOLUTION. Jest to compact nr compact-0047, wygenerowany triggerem T1 (cron). Używany jako wzorzec dla wszystkich przyszłych compactów.

MARKDOWN

```

# SYLION Canon Compact -- 2026-04-15T00:01:33Z
# compact-id: compact-0047
# sha256: a3f9c8d2e1b047a6f3c8d9e5a2b1f047c3d8e9a5b2f1c047e3d9a5b2f1c047
# trigger: cron (T1 -- 24h baseline)
# generated_by: governance-lead-agent-v3
# validated_by: red-team-lead-agent-v3
# approved_by: governance-lead-agent-v3
# status: ACTIVE

---

## 1. Zasada nadrzędna

Rebuildability over Lineage: Pipeline SYLION musi być zdolny do rozwijania,
refaktoryzacji oraz pełnego przepisania systemu od zera, gdy analiza kanoniczna
lub bezpieczeństwa wykaze, że obecna linia kodu nie daje się bezpiecznie rozwijać.

---

## 2. Teza pipeline

Pipeline SYLION (v8.0.0) jest wielozespołową organizacją inżynierską w formie systemu,
zdolną do ewolucji, refaktoryzacji i pełnego rebuildu -- pod kontrolą Księgi i człowieka
przy decyzjach systemowych. System operuje na 7 stałych działach (Plan 15) obsługujących
model bazowy MZ 3.5.5 jako aktualny, wymieniający baseline implementacyjny. Księga jest
jedynym źródłem prawdy dla intencji i granic systemu -- nie kod, nie historia decyzji.

```

```
---

## 3. Ksiega -- 7 zasad rdzeniowych

1. Rebuildability over Lineage -- system musi byc zdolny do pelnego rebuildu
2. Ksiega jako jedyne zrodlo prawdy -- intencje i granice w Ksiedze, nie w kodzie
3. Człowiek w petli przy decyzjach D4/D5 -- operator zatwierdza zmiany systemowe
4. Offensywne testowanie przed promocja -- Red Team sprawdza kazdy modul pre-prod
5. Audytowalnosc kompletna -- kazda decyzja D1+ ma decision_id i Evidence
6. Niezalezność modułów -- modul musi byc wymienialny bez ryzyka dla reszty systemu
7. Bezpieczenstwo nie jest kompromisem -- inwarianty INV-SEC nie moga byc nadpisane

---

## 4. Inwarianty bezpieczenstwa

INV-SEC-01: Zadna zmiana modulu MZ 3.5.5 nie moze naruszyc izolacji kontraktow API
INV-SEC-02: PHANTOM integrity check wymagany przed kazdym bundle promote do production
INV-SEC-03: Secrets Vault (Plan 04) niedostepny dla agentow T1 -- tylko T3 z D4 gate
INV-SEC-04: Agent T3 nie moze byc uruchomiony w ACTIVE bez zatwierdzonego D4
INV-SEC-05: Zadna decyzja D4/D5 nie moze byc zatwierdzona bez aktualnego compact (<6h)
INV-SEC-06: Wynik Red Team adversarial test: FAIL blokuje bundle promote (brak wyjatkow)
INV-SEC-07: External review log wymagany dla kazdej decyzji D5 (Plan 13)

---

## 5. Aktualny baseline

- Pipeline: 8.0.0
- MZ: 3.5.5 (WYMIENIALNY -- nie jest kotwica architektoniczna)
- Aktywny tryb: EVOLUTION
- Target architecture: null (brak aktywnego rebuildu)
- Cutover status: n/d

---

## 6. Ostatnie decyzje D3+

| Data | Klasa | Zakres | Wynik |
|-----|-----|-----|-----|
| 2026-04-12T14:30Z | D3 | Refactor modulu VPS (Plan 02) | APPROVE |
| 2026-04-10T09:15Z | D3 | Nowa rola Red Team Lead w Dziale 15.5 | APPROVE |
| 2026-04-08T16:45Z | D4 | Zmiana INV-SEC-07 (dodanie wymog D5) | APPROVE |
| 2026-04-05T11:00Z | D3 | Zmiana wag Board dla Dzialu Testow | DEFER |
| 2026-04-01T08:00Z | D4 | Onboarding agenta T3 Governance Lead | APPROVE |

---

## 7. Stan 7 dzialow

| Dzial | Health | Activity |
|-----|-----|-----|
| 15.1 Architektura | healthy | Projektowanie refactor modulu VPS |
| 15.2 Platforma | healthy | Bundle pipeline maintenance, cron jobs |
| 15.3 Implementacja | healthy | Patch modulu Knowledge Base (Plan 03) |
| 15.4 Testy i Walidacja | healthy | Regression suite dla refactor VPS |
| 15.5 Red Team | healthy | Adversarial test Plan 07 + INV-SEC walidacja |
| 15.6 Blue Team | healthy | SOC monitoring, aktualizacja playbooks |
```



```
| 15.7 Governance      | healthy | Evidence pack D4 dla INV-SEC-07      |
---

## 8. Wyszczególnienie

- [D4/PENDING] Onboarding agenta T3 \ArchitecturaSenior-v2\ -- czeka na Human Gate
  od 2026-04-14T10:00Z; decision_id: dec-d4-2026-0012
- [D3/DEFERRED] Zmiana wag Board dla Działu Testów -- wraca do Council 2026-04-22

---

## 9. Znane drift patterns

- DRIFT-01: Agenci implementacyjni pomijają walidację implementation_strategy
  przed zmianą modułu -- zawsze sprawdź pole w ACP manifest przed każdym patchem
- DRIFT-02: Decyzje D2 opisywane jako D1 przy patchach cross-modułowych -- jeśli
  zmiana dotyczy więcej niż jednego modułu, minimalna klasa to D2
- DRIFT-03: Compact starszy niż 6h użyty jako kontekst dla D3 -- INV-SEC-05 blokuje;
  jeśli widzisz COMPACT_STALE w kontekście, zatrzymaj się i zapytaj o nowy compact
- DRIFT-04: Agenci ignorują sekcję 9 (drift patterns) -- to najczęstszy pattern;
  zawsze przeczytaj sekcję 9 jako pierwszą przed podjęciem decyzji
- DRIFT-05: Zmiany Księgi bez D4 gate -- każda propozycja zmiany Księgi
  musi trafić do Governance z klasą minimum D3

---

## 10. Źródła prawdy

- Księga: /pipeline/ksiega/ksiega_v3.md
- Dokumentacja: SYLION Pipeline Dokumentacja v3.0 (Pipeline 8.0.0, kwiecień 2026)
- ACP manifest: /pipeline/acp/manifest_v3.yaml
- Target architecture: null (brak aktywnego rebuildu)
- Poprzedni compact: compact-0046 (2026-04-14T00:01:28Z)
```

Schemat bazy danych — tabela compacts

```
SQL

-- PostgreSQL: tabela wersjonowanych compactów
CREATE TABLE compacts (
  id                BIGSERIAL PRIMARY KEY,
  compact_id        TEXT UNIQUE NOT NULL,          -- 'compact-0047'
  sha256            TEXT NOT NULL,                 -- hash zawartości
  trigger_type      TEXT NOT NULL,                 -- 'cron' | 'pre-D3' | 'pre-bundle'
  status            TEXT NOT NULL DEFAULT 'ACTIVE', -- 'ACTIVE' | 'ARCHIVED' | 'INVALID'
  generated_by      TEXT NOT NULL,
  validated_by      TEXT,
  approved_by       TEXT,
  generated_at      TIMESTAMPTZ NOT NULL,
  archived_at       TIMESTAMPTZ,
  compact_path      TEXT NOT NULL,                 -- ścieżka do pliku .md
  agent_compact     TEXT NOT NULL,                 -- wyciąg dla agentów (sekcje 1,2,3,4,5
,6,9,10)
  word_count        INTEGER NOT NULL,
  validation_iters  INTEGER DEFAULT 1,
  pipeline_version  TEXT NOT NULL DEFAULT '8.0.0',
```

```

    system_mode      TEXT NOT NULL DEFAULT 'EVOLUTION',
    notes            TEXT
);

CREATE INDEX idx_compacts_status ON compacts(status);
CREATE INDEX idx_compacts_generated_at ON compacts(generated_at DESC);
```

Kafka events — schemat zdarzeń

```

JSON

# Topic: compact.lifecycle.events
{
  "event": "COMPACT_ACTIVATED",          # COMPACT_ACTIVATED | COMPACT_ARCHIVED | COMPACT_INVALID
  "compact_id": "compact-0047",
  "sha256": "a3f9c8d2e1b047...",
  "trigger_type": "cron",
  "status": "ACTIVE",
  "system_mode": "EVOLUTION",
  "generated_at": "2026-04-15T00:01:33Z",
  "word_count": 847,
  "validation_iters": 1
}

# Topic: compact.agent.distribute
{
  "event": "AGENT_COMPACT_DISTRIBUTED",
  "compact_id": "compact-0047",
  "agent_ids": ["agent-governance-lead-001", "agent-arch-senior-002", "..."],
  "injection_point": "session_init",
  "distributed_at": "2026-04-15T00:02:01Z",
  "agents_count": 23
}
```

API Endpoints — Compaction Service

[GET /compact/v1/active](#)

Zwraca aktywny compact (metadata + pełna treść lub agent_compact według parametru level).

```

JSON

# Request
GET /compact/v1/active?level=agent      # 'agent' lub 'operator'

# Response
{
  "compact_id": "compact-0047",
  "sha256": "a3f9c8d2e1b047...",
  "status": "ACTIVE",
  "trigger_type": "cron",
  "generated_at": "2026-04-15T00:01:33Z",
  "age_hours": 2.5,
  "is_fresh": true,
  "system_mode": "EVOLUTION",
```

```
"content": "# SYLION Canon Compact -- 2026-04-15T00:01:33Z\n..."
}
```

POST /compact/v1/trigger

Wyzwala natychmiastowe generowanie compact (dla triggerów T2 i T3).

JSON

```
# Request
{
  "trigger_type": "pre-D3",
  "context": {
    "decision_id": "dec-d3-2026-0099",
    "decision_class": "D3",
    "scope": "refactor-vps-module"
  },
  "requested_by": "council-orchestrator"
}

# Response
{
  "compact_id": "compact-0048",
  "status": "GENERATING",
  "estimated_ready_at": "2026-04-15T12:07:30Z",
  "poll_url": "/compact/v1/status/compact-0048"
}
```

GET /compact/v1/history

Zwraca listę ostatnich N compactów z metadanymi (bez treści).

JSON

```
# Request
GET /compact/v1/history?limit=10&status=all

# Response
{
  "compacts": [
    {
      "compact_id": "compact-0047",
      "status": "ACTIVE",
      "trigger_type": "cron",
      "generated_at": "2026-04-15T00:01:33Z",
      "word_count": 847,
      "validation_iters": 1
    },
    {
      "compact_id": "compact-0046",
      "status": "ARCHIVED",
      "trigger_type": "cron",
      "generated_at": "2026-04-14T00:01:28Z",
      "word_count": 831,
      "validation_iters": 1
    }
  ],
}
```

```
"total": 47
}
```

HARD GATE'Y

- G-CMP-01: Decyzja D3+ bez aktywnego compact (<6h): blokada inicjacji procesu decyzyjnego; Council Orchestrator musi najpierw wywołać POST /compact/v1/trigger i poczekać na status ACTIVE
- G-CMP-02: Bundle build bez compact w kontekście agentów: Bundle Orchestrator blokuje inicjację bundle; compact musi być wstrzyknięty przed pierwszym krokiem bundle pipeline
- G-CMP-03: Compact z wynikiem hallucination check = INVALID: nigdy nie dystrybuowany; archiwizowany z flagą INVALID; automatycznie inicjowane nowe generowanie; max 3 iteracje — po 3 INVALID: alert P1 do Governance i człowieka
- G-CMP-04: Compact starszy niż 24h (COMPACT_STALE) w kontekście agenta T3: agent natychmiast zatrzymywany i oczekuje na świeży compact; nie może kontynuować zadania D2+
- G-CMP-05: Zmiana sekcji 4 (inwarianty bezpieczeństwa) w compact template bez D4: blokada deployment'u nowego template; wymaga Council 4/4 + Human Gate
- G-CMP-06: Wyłączenie Compaction Service bez D4: wszystkie bundle build i decyzje D3+ zostają automatycznie wstrzymane do czasu przywrócenia serwisu lub D4 override
- G-CMP-07: compact_validation_iterations_total > 3 dla jednego compact: alert P1, Governance Lead i Red Team Lead muszą interweniować manualnie
- G-CMP-08: compact_usage_score < 0.3 dla agenta T3 w 3 kolejnych runach: agent T3 przechodzi w tryb REVIEW (Plan 06); compact injection method wymaga weryfikacji

TESTY

- T-CMP-01: Trigger T1: cron 00:00 UTC generuje compact w <10 minut; wynik: status ACTIVE, sha256 unikatowe, poprzedni compact ARCHIVED
- T-CMP-02: Trigger T2: inicjacja D3 bez ważnego compact → G-CMP-01 blokuje; po POST /compact/v1/trigger i status ACTIVE — D3 inicjacja odblokowana
- T-CMP-03: Trigger T3: bundle build bez compact → G-CMP-02 blokuje; po wstrzyknięciu compact do agentów — bundle kontynuuje
- T-CMP-04: Hallucination check VALID: compact generowany z poprawnymi danymi — Red Team zwraca VALID, compact dystrybuowany jako ACTIVE
- T-CMP-05: Hallucination check INVALID: compact z celowo błędną decyzją D3+ — Red Team zwraca INVALID, compact oznaczony INVALID, nowy generowany; G-CMP-03 sprawdzony
- T-CMP-06: Max words exceeded: compact z >3000 słów — validator zwraca INVALID z powodem word_count_exceeded; compact odrzucony
- T-CMP-07: COMPACT_STALE test: agent T3 dostaje compact starszy niż 24h — G-CMP-04 zatrzymuje agenta; alert emitowany; agent wznowia po świeżym compact
- T-CMP-08: Feedback loop: agent T2 prowadzi 3 runy bez cytowania inwariantów — compact_usage_score < 0.3 przez 3 runy — alert do Governance

- T-CMP-09: SYSTEM_REBUILD_MODE: compact zawiera sekcję target_architecture_status; legacy_compact i target_compact mają wspólne sekcje 1-4
- T-CMP-10: Archiwum: po 30 dniach archiwum zawiera compact-0001..compact-0030; żaden plik nie jest skasowany; każdy ma unikatowy sha256
- T-CMP-11: Integracja Plan 06: agent T3 w onboardingu etap 5 potwierdza odbiór compact — compact_id zapisany w OnboardingSession; test weryfikuje wpis w DB
- T-CMP-12: Integracja Plan 13: Evidence pack dla D4 zawiera compact_id i sha256 aktywnego compact w momencie decyzji; test weryfikuje pole w Evidence

STATUSY I EVIDENCE

Statusy compact w systemie

Status	Znaczenie	Następny stan	Kto zmienia
GENERATING	Compact jest w trakcie tworzenia (Generate + Validate w toku)	ACTIVE INVALID	Compaction Service
ACTIVE	Compact aktywny, dystrybuowany agentom i operatorowi	ARCHIVED	Compaction Service (przy nowym compact)
ARCHIVED	Compact zastąpiony przez nowszy; dostępny tylko dla audytu i historii	(stan terminalny)	Compaction Service (automatycznie)
INVALID	Compact odrzucony przez hallucination check; nigdy nie dystrybuowany	(stan terminalny)	Red Team Validator
STALE	Compact aktywny, ale starszy niż 24h; agenci T3 blokują D2+	ARCHIVED (gdy nowy)	Compaction Service (auto-flag)

Evidence wymagane dla Governance (Plan 13)

Każdy compact jest automatycznym dowodem stanu systemu w danym momencie. Evidence dla decyzji D3+ musi zawierać compact_id i sha256 aktywnego compact w chwili podjęcia decyzji. Umożliwia to rekonstrukcję stanu wiedzy systemu w dowolnym punkcie historii — kluczowe dla audytów po incydentach i dla zewnętrznych review wymaganych przy D5.

- Evidence D3: compact_id + sha256 + trigger_type (archiwalne) + decision_id
- Evidence D4: j.w. + potwierdzenie świeżości compact (<6h) w momencie decyzji
- Evidence D5: j.w. + compact_id dla obu linii (legacy + target w REBUILD_MODE) + external_review_log (Plan 13) z adnotacją o stanie compact
- Audit trail: kompletna historia compact dostępna przez GET /compact/v1/history i bezpośrednio z archiwum /pipeline/canon/compact_archive/

Aneks A — Klasy decyzji w Compaction Protocol

Operacja	Klasa	Gate	Uzasadnienie
Rutynowe generowanie compact (cron, pre-D3, pre-bundle)	D1	1 agent (Governance Lead)	Rutynowa operacja; pełna automatyzacja

Operacja	Klasa	Gate	Uzasadnienie
Zmiana harmonogramu cron compact (inna godzina/częstotliwość)	D1	1 agent (Governance Lead)	Zmiana operacyjna bez wpływu na treść
Zmiana struktury template compact (nowa sekcja, zmiana kolejności)	D3	Council 4/4	Dotyczy całej organizacji — wpływ na wszystkich agentów
Zmiana listy inwariantów bezpieczeństwa (sekcja 4)	D4	Council 4/4 + Human	Krytyczne — inwarianty definiują granice bezpieczeństwa
Wyłączenie Compaction Service (nawet tymczasowe)	D4	Council 4/4 + Human	Implikacje dla całego pipeline'u; brak compact = brak D3+
Zmiana maksymalnej długości compact (max_words)	D3	Council 4/4	Zmiana systemowa wpływająca na wszystkich agentów
Dodanie nowego triggera compaction	D3	Council 4/4	Nowy trigger = nowe obciążenie i nowe wstrzykiwanie kontekstu
Zmiana zasad walidacji (hallucination check)	D3	Council 4/4	Walidacja jest gwarancją jakości compact dla całego systemu

Aneks B — Integracja z innymi planami — punkty styku

Plan	Punkt styku	Kierunek	Opis
Plan 06 (Agent Lifecycle)	session_init compact injection	CMP → ALC	Agent dostaje agent_compact przy init sesji; T3 potwierdza odbiór w onboardingu
Plan 06 (Agent Lifecycle)	compact_usage_score	ALC → CMP	Feedback loop: ALC raportuje czy agent używał compact
Plan 12 (Bundle+ Testing)	pre-bundle trigger T3	PLN12 → CMP	Bundle Orchestrator wyzwała compact przed inicjacją bundle build
Plan 12 (Bundle+ Testing)	compact w kontekście bundle agentów	CMP → PLN12	Compact wstrzykiwany do WSZYSTKICH agentów bundle, nie tylko liderów
Plan 13 (Governance)	compact_id w Evidence	CMP → PLN13	compact_id i sha256 obowiązkowe w Evidence pack dla D3+/D4/D5
Plan 13 (Governance)	decyzje D3+ do sekcji 6 compact	PLN13 → CMP	Governance log dostarcza ostatnie 5 decyzji D3+ do Generate
Plan 14 (Greenfield Rebuild)	REBUILD_MODE compact sections	PLN14 → CMP	W SYSTEM_REBUILD_MODE: compact musi zawierać target_arch + cutover status
Plan 15 (Multi-Team Org)	stan 7 działów w compact sek. 7	PLN15 → CMP	Platforma i Governance dostarczają raporty statusu działów do Generate
Plan 15 (Multi-Team Org)	Governance generuje, Red Team waliduje	PLN15 → CMP	Role compaction są zdefiniowane przez strukturę 7 działów Plan 15

Compaction & Memory · v3.1 EXPAND

16.11. Compact Fidelity Test (CFT) — pełny protokół

FIDELITY · Kanon bez testu wierności jest hipotezą, nie faktem

Plan 16 v3.0 definiował compaction jako rytmicznie generowany artefakt skondensowany. Założenie było: compact agentowy zawiera wystarczająco kontekstu, aby agent po jego przeczytaniu podjął te same decyzje co po przeczytaniu pełnej Księgi. v3.1 zamienia to założenie w testowalną hipotezę. CFT to formalny, zautomatyzowany test wierności — brak CFT pass = brak publikacji compacta do production inject path.

16.11.1. Definicja CFT

Compact Fidelity Test (CFT) to protokół empirycznego pomiaru: „Czy agent podejmuje tę samą decyzję po przeczytaniu compacta co po przeczytaniu pełnego kanonu?”. Test operuje na `golden_set` — zbiorze historycznych decyzji D3-D5 z udokumentowanym ground truth. CFT uruchamia dwa niezależne agenci (alpha — full corpus, beta — compact) na każdym elemencie `golden_set` i porównuje ich decyzje między sobą oraz z ground truth.

16.11.2. Schemat CFT

```
# cft_protocol.yaml — kanoniczna specyfikacja v3.1
compact_fidelity_test:
  id: CFT-001
  trigger:
    - każdy wygenerowany canon_compact.md (D3+)
    - PRE-bundle (przed każdym Plan 12 bundle build)
    - PRE-D3 (przed każdą decyzją D3+)
    - 24h rhythm (baseline validation)

  procedure:
    agent_alpha:
      input: full_corpus.pdf (SYLION_Pipeline_Dokumentacja_v3_1.pdf)
      task: dla każdej decyzji w golden_set podejmij werdykt + klasę + gate_requirements
      output: decision_alpha_<decision_id>.json
    agent_beta:
      input: canon_compact.md (ten sam compact co testowany)
      task: identyczne do agent_alpha
      output: decision_beta_<decision_id>.json
    comparator:
      run: dla każdego decision_id w golden_set
      metrics:
        metric_1_class_match:
          type: HARD
          check: decision_alpha.class == decision_beta.class
        metric_2_verdict_match:
          type: HARD
          check: decision_alpha.verdict == decision_beta.verdict
        metric_3_rationale_overlap:
```

```

    type: SOFT
    check: cosine(embedding(alpha.rationale), embedding(beta.rationale))
    threshold: 0.85
    embedder: text-embedding-3-large
metric_4_gate_requirements_match:
    type: HARD
    check: set(alpha.gates) == set(beta.gates)

heterogeneity:
    requirement: MUST
    rule: agent_alpha.model != agent_beta.model
    allowed_pairs:
        - [gpt-5, claude-opus-4]
        - [claude-opus-4, gemini-2-pro]
        - [gemini-2-pro, gpt-5]
    forbidden: same-model pairs (would not test fidelity, only recall)

determinism:
    temperature: 0.0
    runs_per_decision: 3
    aggregation: majority_vote (2/3 minimum)
    if_no_majority: FAIL (decision marked non-deterministic)

golden_set:
    description: historyczne decyzje D3-D5 z operator ground truth
    minimum_size: 20 decyzji (rekomendowane: 50+)
    composition:
        D3: minimum 10
        D4: minimum 7
        D5: minimum 3
    freshness: min 5 decyzji z ostatnich 30 dni
    ground_truth_source: operator-zatwierdzone decyzje z Evidence pack

gate_logic:
    pass_if_all:
        - metric_1_class_match == true (100% golden_set)
        - metric_2_verdict_match == true (100% golden_set)
        - metric_3_rationale_overlap >= 0.85 (mean across golden_set)
        - metric_4_gate_requirements_match == true (100% golden_set)
        - golden_set_accuracy_parity:
            alpha_ground_truth_accuracy >= 0.90
            beta_ground_truth_accuracy >= (alpha_accuracy - 0.05)
    otherwise: FAIL + eskalacja do Chief Safety

```

16.11.3. Interpretacja metryk

Metryka	Typ	Dlaczego	Próg
class_match	HARD	agent musi wybrać tę samą klasę D (to decyduje o gate'ach, budżecie, Human)	100% golden_set
verdict_match	HARD	werdykt (approve/reject/modify) jest output'em decyzji — musi być identyczny	100% golden_set
rationale_overlap	SOFT	uzasadnienie w naturalnym języku — nie musi być identyczne, ale semantycznie podobne	cosine $\geq$ 0.85

Metryka	Typ	Dlaczego	Próg
gate_requirements_match	HARD	lista gate'ów (G-CUT-01, G-COMP-01, etc.) musi być IDENTYCZNA	100% golden_set
golden_set_accuracy_parity	meta	beta nie może być znacząco gorszy od alpha (compact nie może pogarszać decyzji)	$\beta \geq \alpha - 0.05$

ZASADA · Dlaczego rationale jest SOFT, a reszta HARD

class, verdict, gates — to dyskretne output'y z ograniczonego zbioru. Dla nich „równość” jest jednoznaczna. Rationale to tekst swobodny — równoważność wymaga embedding similarity. Przyjęty próg 0.85 cosine na text-embedding-3-large oznacza: beta ma uzasadnienie wyjaśniające te same fakty, choć nie koniecznie tymi samymi słowami.

16.11.4. Golden set — konstrukcja i utrzymanie

Golden set to fundament CFT. Bez reprezentatywnego zbioru decyzji z ground truth, CFT mierzy tylko zgodność dwóch agentów, a nie ich zgodność z rzeczywistym kanonem. Konstrukcja golden_set jest pierwszorzędnym obowiązkiem Governance.

Zasady konstrukcji golden_set

- Ground truth — każda decyzja w golden_set ma operator-zatwierdzony werdykt dostępny w Evidence pack.
- Reprezentatywność klas — minimum 10×D3, 7×D4, 3×D5 (rozkład zgodny z rzeczywistym rozkładem decyzji).
- Świeżość — minimum 5 decyzji z ostatnich 30 dni (kanon się zmienia, golden_set nie może być „zabytkiem”).
- Różnorodność domen — decyzje dotyczące różnych planów (01-16), nie tylko jednego obszaru.
- Adversarial coverage — minimum 2 decyzje trudne (kontrowersyjne, split Council vote w historii).
- Regularna rotacja — golden_set aktualizowany co kwartał; dodawanie nowych, archiwizacja najstarszych.

```
# golden_set_manifest.yaml — przykład
golden_set:
  version: 2026-Q2
  curator: Governance (Evidence Curator + Chief Safety)
  rotation_cadence: quarterly
  items:
    - id: GS-001
      decision_ref: D4-2025-11-14-phantom-tier-promotion
      class: D4
      ground_truth: { verdict: approve, gates: [G-ESP-05, G-CANON-03] }
      domain: Plan 02 (PHANTOM) × Plan 07
      difficulty: medium
    - id: GS-002
      decision_ref: D5-2026-02-08-auth-rebuild
      class: D5
```

```

    ground_truth: { verdict: approve_with_external, gates: [G-GOV-09, G-CUT-04, G-CFT-
01] }
    domain: Plan 14 × Plan 13
    difficulty: hard
  - id: GS-003
    decision_ref: D3-2026-03-22-contract-refactor-kb
    class: D3
    ground_truth: { verdict: reject, gates: [G-CANON-02 fail] }
    domain: Plan 03 × Plan 08
    difficulty: medium
  # ...minimum 20 items total
stats:
  total: 50
  D3: 30
  D4: 14
  D5: 6
  last_30_days: 8

```

16.11.5. CFT jako blokada publikacji compacta

Każdy `compact.md` po wygenerowaniu przechodzi przez CFT ZANIM zostanie opublikowany do production inject path (ścieżki którą agenci SYLION sesyjnie wstrzykują do kontekstu). Compact który nie przeszedł CFT pozostaje w staging jako DRAFT i nie wolno go wstrzykiwać do żadnej sesji agenta w production.

```

# compact_lifecycle.yaml (state machine)
states:
  GENERATED:
    actor: Compaction Engine (Plan 16 §16.3)
    next: PENDING_CFT
  PENDING_CFT:
    actor: Compact Validator Agent (Plan 15 §15.9.1)
    next: CFT_PASS | CFT_FAIL
    timeout: 30 min (typical)
  CFT_PASS:
    actor: Governance (signs compact_fidelity_report.json)
    next: PUBLISHED
  CFT_FAIL:
    actor: Compact Validator Agent vetoes publication
    next: DRAFT (back to staging) + incident open
    escalation: Chief Safety notified after 3 consecutive FAIL
  PUBLISHED:
    actor: Compaction Engine injects into agent sessions
    expires_at: GENERATED + 24h (next cycle)

```

FIDELITY · CFT jest bramą — nie logiem

W wielu organizacjach testy wierności są raportowane post-hoc jako metryki obserwowalności. W SYLION v3.1 CFT jest gatekeeperem: compact bez CFT pass NIE istnieje w production. Agent, który miałby przeczytać compact bez CFT pass, czytałby tekst bez gwarancji kanonicznej. To niebezpieczne — CFT blokuje taką sytuację u samego źródła.

16.11.6. CFT na compactach operator-level

Plan 16 v3.0 wyróżniał dwa poziomy compact: agent-compact i operator-compact. CFT jest obowiązkowy dla obu, ale z różnymi golden_setami. Agent-compact testowany jest na decyzjach operacyjnych (D3 typowe dla danego działu). Operator-compact testowany jest na decyzjach meta (D4+, zmiany architektury, budżet).

Compact typ	Golden set	Alpha model	Beta model	Frequency
agent-compact (sesyjny)	decyzje D3 operacyjne, 50+ items	GPT-5 (full PDF)	Claude Opus (compact)	każdy cycle (24h)
operator-compact (rytmiczny)	decyzje D4/D5 architektoniczne, 20+ items	Claude Opus (full PDF)	Gemini (compact)	pre-D4 + 7d rhythm
agent-compact pre-build	decyzje D3+ z kontekstu danego bundle	heterogeniczne pary rotujące	—	każdy bundle build

16.11.7. Artefakt compact_fidelity_report.json

Wynik CFT jest zapisywany jako compact_fidelity_report.json — artefakt dołączany do Evidence pack każdej decyzji D3+ opartej o compact. Schema jest zdefiniowana w Aneksie I (patrz koniec dokumentu). Minimalny przykład wyniku PASS poniżej:

```
{
  "cft_id": "CFT-2026-04-19T03:00Z-agent-compact",
  "compact_ref": "canon_compact_2026-04-19T03.md",
  "full_corpus_ref": "SYLION_Pipeline_Dokumentacja_v3.1.pdf@sha256:...",
  "golden_set_version": "2026-Q2",
  "alpha": { "model": "gpt-5", "runs": 3 },
  "beta": { "model": "claude-opus-4", "runs": 3 },
  "metrics": {
    "class_match": { "hard": true, "rate": 1.00 },
    "verdict_match": { "hard": true, "rate": 1.00 },
    "rationale_overlap": { "soft": true, "mean_cosine": 0.912 },
    "gate_requirements_match": { "hard": true, "rate": 1.00 },
    "golden_set_accuracy": {
      "alpha_accuracy": 0.94,
      "beta_accuracy": 0.92,
      "parity_ok": true
    }
  },
  "verdict": "PASS",
  "signed_by": ["compact_validator_agent", "governance_evidence_curator"],
  "next_cycle_at": "2026-04-20T03:00Z"
}
```

FIDELITY · compact_fidelity_report.json idzie do Evidence pack — zawsze

Każda decyzja D3+ podjęta w sesji, która czytała compact, MUSI dołączyć `compact_fidelity_report.json` obowiązujący w momencie decyzji. Plan 13 G-CFT-01 waliduje że raport istnieje, jest świeży (max 24h) i ma `verdict=PASS`. Bez tego G-CFT-01 FAIL → quorum Council nie jest osiągnięte.

PLAN 16 · ROZSZERZENIE v3.3 · EFFICIENCY

Efficiency metrics jako pole canon_compact v3.3

§16.12 — Efficiency metrics jako sekcja canon_compact (v3.3)

v3.3 dokłada do canon_compact.md sekcję „efficiency_snapshot:” z agregowanymi metrykami z ostatnich 24 h. Cel: agent w nowej sesji widzi NATYCHMIAST stan efficiency całego systemu, bez czekania na pełen Evidence pack.

§16.12.1 — Pole „efficiency_snapshot” w canon_compact

Sekcja „efficiency_snapshot:” jest wstawiana przez Compact Engine po każdym cyklu compaction, zawsze przed sekcją „active_decisions:”. Dane są agregowane z ostatnich 24 h z artefaktów efficiency Evidence pack.

```
# canon_compact.md – sekcja efficiency_snapshot (v3.3)
efficiency_snapshot:
  generated_at: "2026-04-19T03:00Z"
  window_h: 24

  bloat_score_p95: 18.4          # agregowany p95 across modules
  perf_p95_ms: 142              # 95. percentyl latencji [ms]
  perf_p99_ms: 387              # 99. percentyl latencji [ms]
  memory_rss_peak_mb: 1024      # szczytowe RSS [MB]
  monthly_spend_burn_pct: 62.1  # % envelope na dzień raportu
  cache_hit_rate_pct: 74.3      # współczynnik trafień cache [%]

  latest_veto_events:           # zdarzenia veto z ostatnich 24h
    - {gate: G-EFF-04, module: order_processor, ts: "2026-04-18T21:14Z"}

  modules_in_efficiency_breach: # moduły naruszające progi
    - {module: order_processor, gate: G-EFF-04, since: "2026-04-18T21:14Z"}

  trend_7d_direction: degrading # improving | stable | degrading
```

§16.12.2 — Triggery refresh efficiency snapshot

Sekcja efficiency_snapshot jest odświeżana w czterech sytuacjach. Każda aktualizacja jest podpisywana przez owner role i zapisywana w audit trail compactu.

Trigger	Częstotliwość	Owner
Co 24 h baseline	Scheduled (cron)	Compact Validator (z v3.1)
Po każdym D3+ approve	Event (per decyzja)	Code Optimizer (sign + push)
Po zdarzeniu veto	Event (per veto)	Owner roli, która wyzwoliła veto
Po bundle build	Event (per build)	Bundle Engineer

§16.12.3 — Compact Fidelity Test dla efficiency

CFT z v3.1 §16.11 ZOSTAJE i jest rozszerzony: golden set testuje teraz również retencję factów efficiency z sekcji efficiency_snapshot. Progi wiernych odtworzeń są rygorystyczne — utrata informacji o breach jest niedopuszczalna.

Pole compact	Próg wiernego odtworzenia	Metoda testu
efficiency_snapshot.bloat_score_p95	±5 %	Replay 7-day window
efficiency_snapshot.perf_p95_ms	±3 %	Benchmark replay
efficiency_snapshot.perf_p99_ms	±3 %	Benchmark replay
latest_veto_events	100 % retencja	Exact match (każde zdarzenie)
modules_in_efficiency_breach	100 % retencja	Exact match (wszystkie moduły)
trend_7d_direction	100 % zgodność	Enum match

FIDELITY · CFT rozszerzony o efficiency facts — v3.3

Golden set CFT z Aneksu I (v3.1) jest uzupełniany o pytania testujące retencję factów efficiency. Agent, który nie pamięta breach status modułu lub pomija trend_7d=degrading, nie zda CFT i compact nie może być opublikowany do production inject path.

§16.12.4 — Workflow: jak nowy agent czyta efficiency_snapshot

Poniższy flow opisuje sekwencję inicjalizacji agenta sesyjnego w v3.3. Dzięki efficiency_snapshot w canon_compact agent nie musi odpytywać zewnętrznych panelów — stan efficiency jest dostępny natychmiastowo.

```
New agent session start
├─ load canon_compact.md
│   └─ parse efficiency_snapshot
│       ├── if any module in modules_in_efficiency_breach[]:
│           │   WARN agent o ograniczeniach kontraktowych
│           │   (nie wolno approve D3+ dla naruszonego modułu
│           │   bez uprzedniej remediacji)
│       ├── if trend_7d_direction == "degrading":
│           │   dodaj efficiency_focus do task budget
│           │   (priorytet: zredukować bloat/perf przed nowym feature)
│       └─ if monthly_spend_burn_pct > 90:
│           │   WARN: zbliżamy się do cost envelope
│           │   (wymagana zgoda Fin0ps przed D3+)
└─ agent ready
```

§16.12.5 — Operator dashboard view efficiency

Operator-compact (z v3.0) zawiera pełną sekcję „Efficiency status” z wizualnymi wskazówkami UI (kolory red/yellow/green per moduł). Każdy wskaźnik łączy się bezpośrednio

z panelami Plan 17:

- „Code Health Panel” — bloat_score_p95, clone_coverage, cognitive_complexity per moduł (zielony ≤ 20 , żółty 20–50, czerwony > 50).
- „Performance Panel” — p95/p99 latencji vs SLO, error rate, error budget remaining. Trend wykres 7d.
- „Memory Panel” — RSS peak, heap peak, leak rate, top allocators lista. Alert przy leak_rate > 1 MB/h.
- „Cost Panel” — current_burn_pct (gauge), forecast_eom_usd, model_routing_distribution pie chart, cache_hit_rate trend.

EFFICIENCY · canon_compact + Operator Console Plan 17 = pełna obserwowalność

canon_compact + Operator Console Plan 17 = pełna obserwowalność efficiency w jednym miejscu.

§16.12.6 — Integracja z Plan 18 Skills

Skills efficiency-related z Aneksu M (Plan 18) deklarują pole „requires_compact_fields:” określające, które pola efficiency_snapshot są warunkiem wstępnym uruchomienia skilla.

- Skill „efficiency-audit” (Aneks M) — używa pełnego efficiency_snapshot jako kontekstu wejściowego; wymaga pól: „bloat_score_p95”, „latest_veto_events”, „modules_in_efficiency_breach”.
- Skill „bloat-detection” — analizuje historyczny bloat_score_p95 z ostatnich 30 compactów; wymaga pól: „bloat_score_p95”, „trend_7d_direction”.
- Cross-reference: każdy skill efficiency-related ma pole „requires_compact_fields:” z listą wymaganych pól — jeśli canon_compact nie zawiera wymaganych pól (compact starszy niż 24 h lub bez sekcji efficiency_snapshot), skill odmawia uruchomienia i loguje G-EFF-04 warning.

EFFICIENCY · Compaction Protocol v3.3 — cztery wymiary efficiency

Compaction Protocol v3.3 widzi cztery wymiary efficiency tak samo dokładnie jak widzi rebuildability i reversibility z v3.1.

PLAN 17 · SPECYFIKACJA WYKONAWCZA

Operator Console & Dashboard UI/UX

Plan 17 — Operator Console & Dashboard UI/UX

Plan 17 jest pełną specyfikacją enterprise Operator Console — interaktywnego centrum dowodzenia pipeline'em SYLION. Dokument obejmuje architekturę warstwy frontend, design system, information architecture, osiem panelów operacyjnych, kontrakty WebSocket, RBAC, dostępność WCAG 2.1 AA, theming, internacjonalizację, budżet wydajności oraz roadmap implementacyjny. Plan powstał jako nowy moduł w wersji v3.3 i podlega tym samym zasadom Rebuildability co każdy inny komponent SYLION.

§17.1 Wprowadzenie i Rebuildability Principle

Każdy enterprise pipeline generujący decyzje klasy D0–D5, uruchamiający agenty AI i egzekwujący bramki jakości musi posiadać dedykowaną konsolę operacyjną. Bez widoczności stanu pipeline'u operatorzy działają „w ciemności” — nie wiedzą, które decyzje czekają na głosowanie, gdzie pipeline został zatrzymany przez veto bramki ani jaki jest aktualny budżet kosztu modeli AI. Platforma Backstage (CNCF) — wzorzec developer portalu enterprise — dokumentuje: wdrożenie dedykowanego portalu operacyjnego skraca czas onboardingu nowych inżynierów o 55% oraz zwiększa udział zewnętrznych kontrybutorów o 85%. Te same efekty osiąga SYLION Operator Console: jeden punkt dostępu do całego stanu systemu.

REBUILD · Rebuildability Console (Plan 01 ACP)

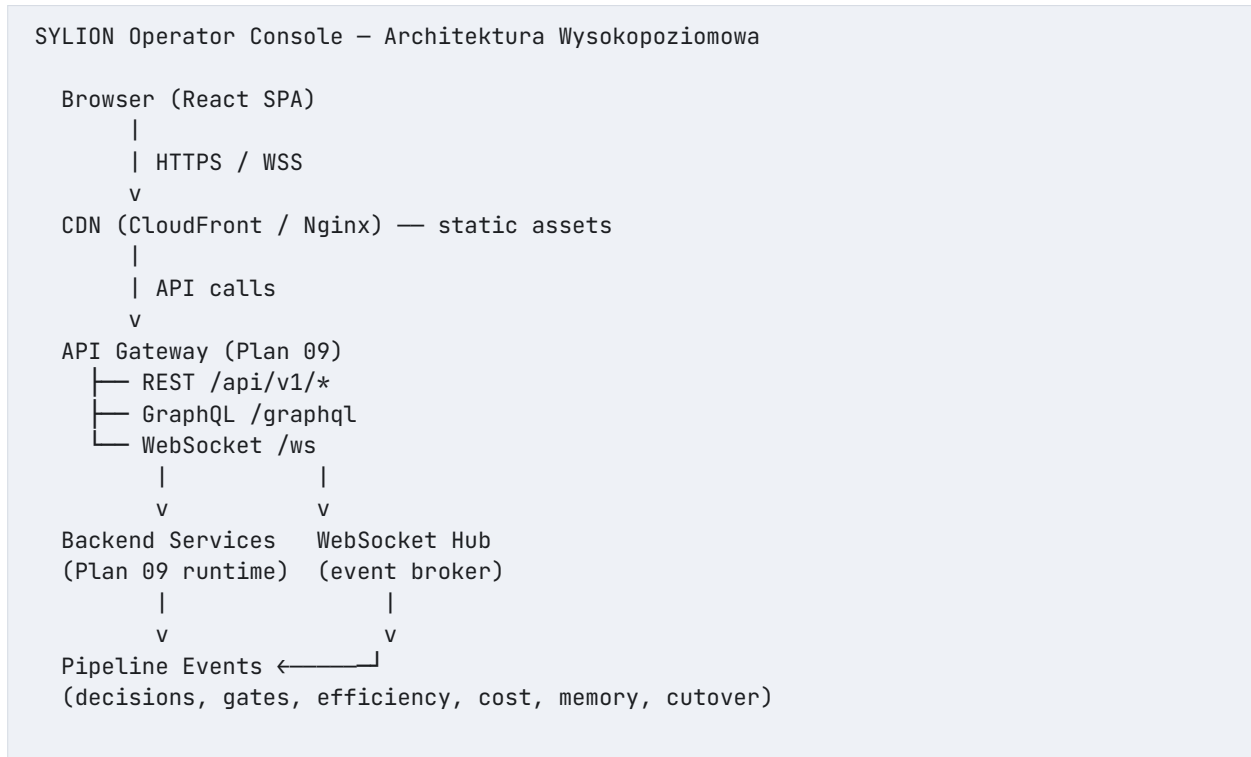
Console MUSI być rebuildable jak cały pipeline. Frontend = osobny moduł ACP (Plan 01) z własną implementation_strategy. Każda zmiana klasy D3+ w Console przechodzi przez Council 4/4, design review i feature flag canary. Rebuild Console (klasa D5) można wykonać w izolacji od core backend pipeline bez przerywania działania systemów krytycznych.

UI / UX · 5 Zasad UI Operator Console

1. Density-aware — gęstość informacji adaptuje się do roli i kontekstu (compact dla monitoringu, comfortable dla decyzji D4+). 2. Role-based — każda rola widzi inny podzbiór akcji i danych. 3. Real-time — stan pipeline'u aktualizuje się przez WebSocket bez odświeżania strony. 4. Accessible WCAG AA — 100% zgodności z WCAG 2.1 AA, w tym keyboard navigation i screen reader support. 5. Themeable — Light / Dark / High-contrast; white-label gotowy.

§17.2 Architektura Wysokopoziomowa

Operator Console jest aplikacją SPA (Single-Page Application) deployowaną jako statyczne assety na CDN z konfiguracją dostarczana przez ACP (Plan 01). Komunikacja z backendem odbywa się przez API Gateway (REST+GraphQL) oraz WebSocket Hub dla strumienia zdarzeń real-time. Diagram architektoniczny:



Pełny stack technologiczny Operator Console:

Warstwa	Technologia	Uzasadnienie
Frontend framework	React 18 + TypeScript + Vite	Dojrzały ekosystem, szybki HMR, type safety
Zarządzanie stanem	Zustand + TanStack Query	Real-time subscriptions + inteligentny cache
Wykresy / wizualizacje	Recharts + D3.js	Komponowalne, SVG-based, dostępne
Real-time	WebSocket native + SSE fallback	Bidirectional + corporate proxy fallback
Internacjonalizacja	react-i18next (ICU MessageFormat)	PL/EN minimum; RTL gotowy
Uwierzytelnianie	OIDC (połączenie z Plan 04 Vault)	Single-sign-on; token rotation
Testy komponentów	Vitest + Testing Library	Szybkie, izolowane, a11y assertions

Warstwa	Technologia	Uzasadnienie
E2E testy	Playwright	Cross-browser; a11y audit automation
Style / CSS	Tailwind CSS v3 + CSS custom properties	Token-based theming; tree-shaking
Build / bundle	Vite + Rollup	ESM output; code splitting automatyczny
Monitoring frontendu	OpenTelemetry JS SDK	RUM, Core Web Vitals, error capture

UI / UX · Wymienialny stack

Stack jest wymienialny — żadne lock-in. Interfejs między frontend a backend jest definiowany przez OpenAPI 3.1 i AsyncAPI 2.6 (WebSocket). Zmiana frameworku frontendowego nie wymaga zmian w backend API.

§17.3 Design System: Tokens, Kolory, Typografia, Spacing

Design System SYLION Operator Console jest oparty na systemie CSS custom properties (tokens). Każdy token ma nazwę, wartość i przypisane użycie. Zmiana motywu (Light/Dark/High-contrast) to podmiana wartości tokenów bez modyfikacji komponentów. Tokeny są źródłem prawdy dla wszystkich komponentów — nie hardkodowane wartości.

§17.3.1 Color Tokens

Pełna paleta tokenów kolorystycznych:

Token CSS	Hex (Light)	Użycie
--color-primary	#1976D2	Główny akcent, przyciski CTA, linki
--color-primary-dark	#1565C0	Hover / pressed primary
--color-primary-light	#42A5F5	Soft backgrounds primary
--color-secondary	#37474F	Nagłówki, ikony, secondary buttons
--color-secondary-light	#546E7A	Subtext secondary
--color-success	#2E7D32	Gate pass, decision approved, status OK
--color-success-bg	#E8F5E9	Success background tint

Token CSS	Hex (Light)	Użycie
--color-warning	#F57C00	Gate warn, decision pending, czas oczekiwania
--color-warning-bg	#FFF3E0	Warning background tint
--color-danger	#C62828	Gate veto, decision rejected, error, rebuild
--color-danger-bg	#FFEBEE	Danger background tint
--color-info	#0288D1	Informacja, tooltips, badges informacyjne
--color-info-bg	#E1F5FE	Info background tint
--color-bloat	#1565C0	Code Health / Bloat panel
--color-perf	#6A1B9A	Performance panel
--color-mem	#AD1457	Memory panel
--color-cost	#EF6C00	Cost panel
--color-evolution	#2E7D32	Path evolution (zielony)
--color-refactor	#F57C00	Path refactor (amber)
--color-rebuild	#C62828	Path rebuild (czerwony)
--color-d0	#1B5E20	D0 — zielony (automatyczna)
--color-d1	#558B2F	D1 — oliwka (owner direct)
--color-d2	#F9A825	D2 — ochra (feature flag)
--color-d3	#E65100	D3 — amber (Council majority)
--color-d4	#880E4F	D4 — magenta (Council + Human)
--color-d5	#4A148C	D5 — fiolet (pełny rebuild)
--color-bg	#F5F5F5	Tło aplikacji (canvas)
--color-surface	#FFFFFF	Karty, panele, dialogi
--color-surface-elevated	#FAFAFA	Elevated cards, modals
--color-border	#E0E0E0	Obramowania komponentów
--color-divider	#EEEEEE	Linie podziału sekcji
--color-text-primary	#212121	Główny tekst

Token CSS	Hex (Light)	Użycie
--color-text-secondary	#757575	Podtekst, opisy, etykiety
--color-text-disabled	#BDBDBD	Wyłączone elementy

§17.3.2 Typography Scale

Rola	Rozmiar	Line-height	Letter-spacing	Font
Display XL	32px	1.25	-0.5px	Inter 700
Display L	28px	1.28	-0.25px	Inter 700
Display M	24px	1.33	0px	Inter 600
Heading XL	20px	1.40	0px	Inter 600
Heading L	18px	1.44	0px	Inter 600
Heading M	16px	1.50	0.15px	Inter 500
Body L	14px	1.57	0.1px	Inter 400
Body M	13px	1.54	0.1px	Inter 400
Caption L	12px	1.50	0.4px	Inter 400
Caption S	11px	1.45	0.4px	Inter 400
Code L	14px	1.60	0px	JetBrains Mono 400
Code M	13px	1.55	0px	JetBrains Mono 400
Code S	12px	1.50	0px	JetBrains Mono 400

§17.3.3 Spacing Scale

Bazowa jednostka: 4px. Skala spacing: 4 / 8 / 12 / 16 / 20 / 24 / 32 / 40 / 48 / 64 / 80 / 96px. Tokeny: --space-1 (4px), --space-2 (8px), --space-3 (12px), --space-4 (16px), --space-5 (20px), --space-6 (24px), --space-8 (32px), --space-10 (40px), --space-12 (48px), --space-16 (64px), --space-20 (80px), --space-24 (96px). Wszystkie marginesy, paddingi i odstępy w aplikacji MUSZĄ pochodzić z tej skali — nigdy arbitralne wartości px.

§17.3.4 Elevation / Shadows

Poziom	CSS box-shadow	Użycie
elevation-0	none	Flat: tabele, tła bez podniesienia

Poziom	CSS box-shadow	Użycie
elevation-1	0 1px 3px rgba(0,0,0,.12), 0 1px 2px rgba(0,0,0,.24)	Karty, panele, input fields
elevation-2	0 3px 6px rgba(0,0,0,.15), 0 2px 4px rgba(0,0,0,.12)	Dropdowny, tooltips, popovery
elevation-3	0 10px 20px rgba(0,0,0,.15), 0 3px 6px rgba(0,0,0,.10)	Modals, drawers, dialogi
elevation-4	0 15px 25px rgba(0,0,0,.15), 0 5px 10px rgba(0,0,0,.05)	Full-screen overlays, decision ladders

§17.3.5 Border Radius

Token	Wartość	Użycie
--radius-sharp	0px	Tabele, code blocks, dense data
--radius-sm	4px	Przyciski small, badges, tagi
--radius-md	8px	Karty, panele, inputy, selecty
--radius-lg	12px	Modals, drawers, duże dialogi
--radius-xl	16px	Hero cards, featured widgets
--radius-full	9999px	Avatary, circle buttons, pill badges

§17.3.6 Theme Support

Operator Console wspiera trzy tryby tematyczne: Light (default), Dark (prefers-color-scheme: dark) oraz High-contrast (Windows High Contrast + klasa CSS .hc). Każdy tryb podmienia zestaw CSS custom properties — komponenty nie wymagają modyfikacji. Przetłączanie motywu jest natychmiastowe (bez przeladowania strony) i zapamiętywane w user preferences (patrz §17.9.3).

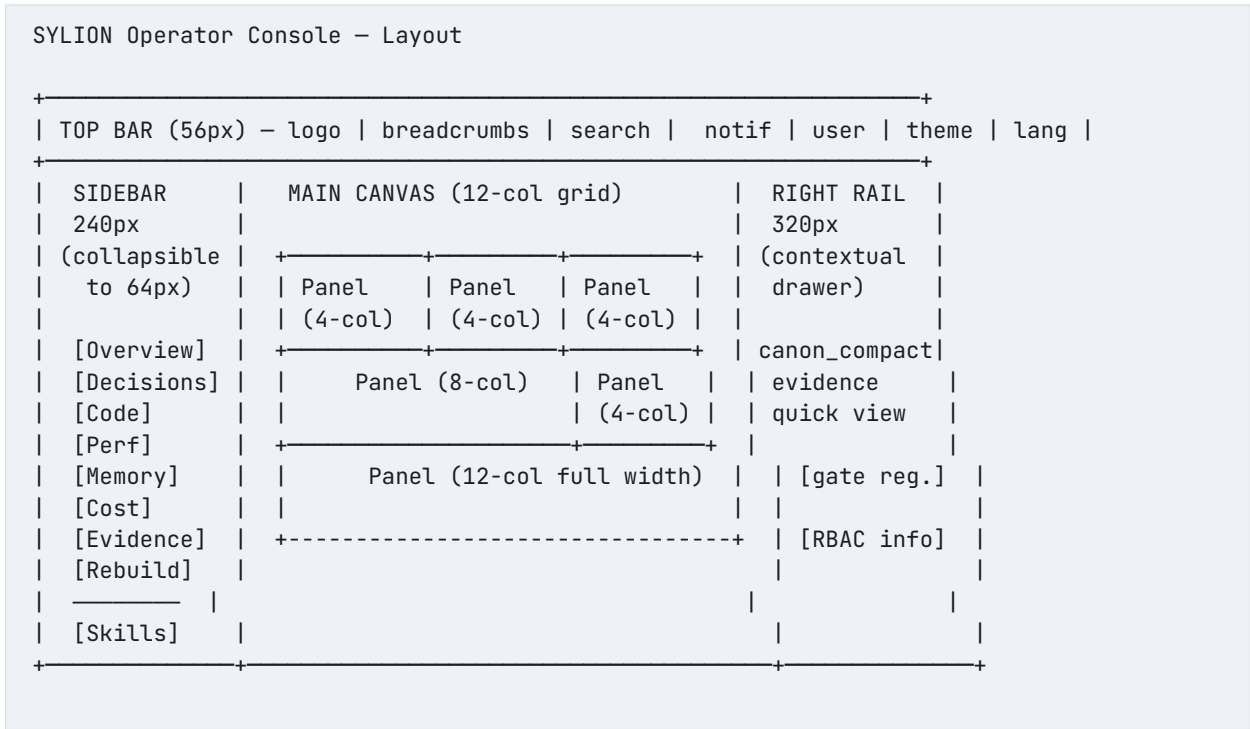
UI / UX · WCAG AA: Kontrast obowiązkowy

WCAG 2.1 AA wymaga kontrastu minimum 4.5:1 dla tekstu body i 3:1 dla dużego tekstu (powyżej 18pt lub 14pt bold). Każdy token kolorystyczny --color-text-* na --color-surface musi spełniać ten wymóg w każdym z trzech trybów tematycznych. Audit narzędziem axe-core jest obowiązkowy przed releasem.

§17.4 Layout i Information Architecture

Operator Console stosuje klasyczny layout enterprise z czterema strefami: top bar (nawigacja globalna), sidebar (nawigacja sekcyjna), główny canvas (panele operacyjne) oraz right rail

(drawer kontekstowy). Strefy są niezależne komponentowo i komunikują się przez globalny store (Zustand).



§17.4.1 Top Bar Elements

Top bar (wysokość stała 56px) zawiera od lewej do prawej: Logo SYLION (link do Overview) | Breadcrumbs (lokalizacja w nawigacji) | Global Search (Cmd/Ctrl+K) | Notification Bell (badge z licznikiem) | User Menu (avatar + rola + logout) | Theme Toggle (sun/moon/contrast) | Language Switcher (PL/EN). Top bar jest sticky — zawsze widoczny niezależnie od scrollowania canvas.

§17.4.2 Sidebar Navigation

Sidebar (240px, collapsible do 64px icon-only) zawiera 8 sekcji głównych: (1) Pipeline Overview, (2) Decision Ladder, (3) Code Health, (4) Performance, (5) Memory, (6) Cost, (7) Evidence & Governance, (8) Rebuild & Cutover. Pod sekcjami: separator i Skills Drawer (rozwijany panel ról efficiency). Sidebar wspiera keyboard navigation (Tab/Enter/Esc), tooltip na hover w trybie collapsed i persystencję stanu w localStorage.

§17.4.3 Main Canvas

Main canvas używa 12-kolumnowego systemu grid (CSS Grid). Panele mogą zajmować 4 / 6 / 8 / 12 kolumn. Layout grid jest responsywny — na ekranach 1024px przechodzi na 8 kolumn, na 768px na 4 kolumny. Operator może zmieniać układ panelów przez drag-and-drop (ograniczony do predefiniowanych slotów, zapisany w user preferences).

§17.4.4 Right Rail — Contextual Drawer

Right rail (320px) jest kontekstowym drawerem otwieranym po kliknięciu elementu w canvas. Zawiera: canon_compact view aktywnej decyzji, evidence pack quick view (hash chain preview), gate registry status dla aktywnego panelu, RBAC info (aktualne uprawnienia usera). Drawer można zamknąć (ikona X), co rozszerza main canvas o 320px.

§17.4.5 Mobile Responsive

Breakpoints: 1440px (full desktop), 1024px (tablet landscape, sidebar collapsible), 768px (tablet portrait, sidebar → hamburger menu), 480px (mobile, sidebar → full-screen modal). Right rail na mobile → bottom sheet modal. Grid: 12 col (1440px), 8 col (1024px), 4 col (768px), 1 col (480px).

UI / UX · Mobile Operator — ograniczenia

Mobile Operator nie jest pomysłany do decyzji D4–D5 — tylko monitoring + D0–D2. Decyzje klasy D3+ wymagają desktop, gdzie operator ma pełny kontekst (evidence pack, trend charts, gate registry). Mobile pokazuje status i alerty — nie Council vote interface.

§17.5 Osiem Panelów Operacyjnych — Pełna Specyfikacja

Każdy panel jest samodzielnym modułem React z własnym stanem, subscrypcjami WebSocket i logiką RBAC. Poniżej pełna specyfikacja każdego z ośmiu panelów.

§17.5.1 Pipeline Overview Panel

PIPELINE OVERVIEW PANEL

+-----+

| Run: RUN-2025-0712-001 [D3] [EVOLUTION] Status: RUNNING |

+-----+

| Działy: |

| Architektura [=====] 100% PASS |

| Platforma [=====] 87% RUN |

| Implementacja [=====] 60% RUN |

| Testy [=====] 28% WAIT |

| Red Team [] 0% WAIT |

| Blue Team [] 0% WAIT |

| Governance [] 0% WAIT |

+-----+

| DORA KPIs: |

| Deploy Freq: 3.2/day Lead Time: 4.1h |

| Change Fail: 2.1% MTTR: 14 min |

+-----+

| p95 Latency 7d: [sparkline chart] avg: 127ms |

+-----+

Komponent	Props	Opis
RunHeader	runId, decisionClass, path, status	Nagłówek z ID runu

Komponent	Props	Opis
DivisionProgressBar	division, percent, status	Pasek postępu działu
DoraKpiCard	metric, value, trend, unit	KPI card DORA
SparklineChart	data[], label, unit	Mini wykres trendów
DecisionBadge	class, label	Badge klasy decyzji (D0-D5)
PathBadge	path	Badge evolution/refactor/rebuild
StatusPill	status	Pill: RUNNING/PASS/WAIT/VETO
GateStatusRow	gateId, status, vetoReason	Status pojedynczej bramki
Event WebSocket	Trigger	Akcja UI
pipeline.started	Nowy run uruchomiony	Odśwież RunHeader
pipeline.gate.passed	Bramka zaliczona	Update DivisionProgressBar
pipeline.gate.veto	Bramka zablokowana	Alert + veto reason popup
decision.approved	Decyzja zatwierdzona	Update DecisionBadge
pipeline.completed	Run zakończony	Status → COMPLETED

UI / UX · RBAC — Pipeline Overview

viewer: read-only, wszystkie dane widoczne. operator: może restart branch. admin: może force-stop run. external_reviewer: read-only + eksport PDF raportu.

§17.5.2 Decision Ladder Panel

```
DECISION LADDER PANEL
+-----+
| DECISION LADDER           Pending: 2  Recent: 10 |
+-----+
| D0 [AUTO]  =====▼===== |
| D1 [OWNER] | CURRENT: D3 |      |
| D2 [FLAG]  | Awaiting Council vote | |
| D3 [CNCL] <-| dec-2025-0712-003  42 min wait | |
| D4 [HUM]   |                  |      |
| D5 [RBLD]  |                  |      |
+-----+
| PENDING: |
| [D3] dec-003  API schema change  [Council vote]|
| [D2] dec-004  Feature flag perf   [Approve]   |
+-----+
| OSTATNIE 10 DECYZJI: |
| dec-001 D1 APPROVED  2025-07-12 09:14 |
| dec-002 D2 APPROVED  2025-07-12 10:03 |
| ... |
```


Komponent	Props	Opis
DecisionLadder	currentClass, decisions[]	Pionowa drabina D0-D5
DecisionCard	id, class, title, waitTime, status	Karta pojedynczej decyzji
CouncilVoteButton	decisionId, disabled	Button Council vote (admin only)
DecisionHistory	decisions[], limit	Lista ostatnich decyzji z outcomes
EscalationAlert	decision, reason	Alert eskalacji do wyższej klasy
Event WebSocket	Trigger	Akcja UI
decision.created	Nowa decyzja	Dodaj do pending list
decision.voted	Głos oddany	Update vote counter
decision.approved	Decyzja zatwierdzona	Move to history, green badge
decision.rejected	Decyzja odrzucona	Move to history, red badge
decision.escalated	Eskalacja klasy	Alert + update ladder marker

UI / UX · RBAC — Decision Ladder

viewer: read-only lista decyzji. operator: może approve D0-D2. admin: może triggerować Council vote dla D3+. external_reviewer: read-only, może pobrać PDF decision summary.

§17.5.3 Code Health Panel (Efficiency / Bloat)

CODE HEALTH PANEL

CODE HEALTH

Avg Bloat: 2.3

Breaches: 2

HEATMAPA MODULI:

[api-gw]

[LOW]

[planner]

[MED]

[runner]

[HIGH]

[vault]

[LOW]

[optimizer]

[CRIT]

[events]

[MED]

BLOAT SCORE TREND 7d:

[line chart]

TOP-5 BREACH:

optimizer(4.8)

runner(3.9)

...

RADAR:

anti-pattern density (5 osi)

PENDING OPTIMIZER REVIEWS:

3

Komponent	Props	Opis
BloatTrendChart	data[], days	Trend linii bloat_score
BreachList	modules[], threshold	Lista modułów w breach
AntiPatternRadar	patterns[], scores[]	Radar chart anty-wzorce
OptimizerReviewQueue	reviews[], onSign	Queue review Code Optimizer
Event WebSocket	Trigger	Akcja UI
efficiency.report.published	Nowy raport Code Optima.	Odśwież heatmapę
gate.G-EFF.veto	Bramka EFF zablokowana	Alert czerwony + reason
optimizer.recommendation	Nowa rekomendacja	Dodaj do review queue
efficiency.breach.resolved	Breach rozwiązany	Usuń z listy breach

EFFICIENCY · Code Health — RBAC

code_optimizer: read + write (podpisywanie raportów efficiency). operator / admin: read + manage review queue. viewer: read-only.

§17.5.4 Performance Panel

PERFORMANCE PANEL

+

PERFORMANCE

SLO Budget: 87%

Freeze: NO

+

+

USE METHOD:

CPU: Util 42% Sat 0.1 Errors 0/min

MEM: Util 61% Sat 0.0 Errors 0/min

I/O: Util 18% Sat 0.0 Errors 0/min

+

+

LATENCY: p50:45ms p95:127ms p99:310ms

RED: Rate 1240/s Errors 0.02% Duration 89ms

ERROR BUDGET: [gauge 87%] Freeze: [OFF]

+

Komponent	Props	Opis
UseMethodTable	resources[], metrics[]	USE per resource
LatencyTimeseries	data[], percentiles[]	p50/p95/p99 chart
RedMetricsRow	rate, errors, duration	RED aggregated
ErrorBudgetGauge	budgetPercent, sloTarget	Gauge błędnego budżetu
DeployFreezeIndicator	frozen, reason	Czerwony baner przy freeze

Event WebSocket	Trigger	Akcja UI
performance.baseline.updated	Nowy baseline	Odśwież thresholds
gate.G-PERF.veto	Bramka PERF zablokowana	Veto alert + freeze check
slo.budget.breach	Budget < threshold	Czerwony alert + freeze

PERFORMANCE · Performance Panel — RBAC

performance_eng: read + write (update SLO targets). operator / admin: read. viewer: read-only. Deploy Freeze może aktywować tylko admin.

§17.5.5 Memory Panel

MEMORY PANEL

+-----+

| MEMORY RSS: 1.2GB Heap: 820MB Leaks: 0 |

+-----+

| RSS/HEAP TIMESERIES: [chart 24h] |

| TOP ALLOCATORS: |

| 1. context-store 412MB 38.4% |

| 2. evidence-cache 198MB 18.5% |

| 3. model-pool 156MB 14.6% |

| LEAK DETECTION: [GREEN – no leaks detected] |

| GC PAUSE DISTRIBUTION: [histogram] |

| CGROUP UTILIZATION: [gauge 61%] |

+-----+

Komponent	Props	Opis
MemoryTimeseries	rss[], heap[], timestamps[]	RSS/heap chart 24h
AllocatorsTable	allocators[], sortBy	Top alokatorów z %
LeakDetectionRibbon	leaks[], severity	Alert ribbon wykrycia leaków
GcPauseHistogram	pauses[], buckets[]	Histogram przerw GC
CgroupGauge	utilization, limit	Gauge zużycia cgroup

Event WebSocket	Trigger	Akcja UI
memory.profile.published	Nowy profil pamięci	Odśwież timeseries
gate.G-MEM.veto	Bramka MEM zablokowana	Veto alert czerwony
leak.detected	Wykryty wyciek	Czerwony ribbon + notyfikacja

MEMORY · Memory Panel — RBAC

memory_auditor: read + write (sign profilów). operator/admin: read. Wymuszenie GC i force-compact tylko admin.

§17.5.6 Cost Panel

COST PANEL

COST

Burn: \$1,240/mo

Envelope: \$2,000

62%

MONTHLY BURN GAUGE: [62%]

\$1,240 / \$2,000

DAILY BURN vs ENVELOPE: [bar chart 30d]

MODEL ROUTING PIE: GPT-4o 45%, Sonnet 38%...

CACHE HIT RATE: [trend 7d]

avg: 71%

TOP-COST DECISIONS:

1. dec-003 D3 \$48.20 api-schema-change

2. dec-001 D1 \$12.40 auth-refactor

COST · Cost Panel — RBAC

cost_optimizer: read + update envelope. operator: read. admin: pełne zarządzanie. viewer: read-only (bez kwot finansowych — domyślnie ukryte).

§17.5.7 Evidence & Governance Panel

EVIDENCE & GOVERNANCE PANEL

EVIDENCE

Packs: 128

Chain: VALID

Review: 2

TIMELINE

OSTATNICH 50 EVIDENCE PACKS:

ep-128

dec-003

2025-07-12 11:42

sha256:a1b2

ep-127

dec-002

2025-07-12 10:03

sha256:c3d4

[Search / Filter: decisionId | date | status]

GATE REGISTRY: 12 registered 0 unregistered

EXTERNAL REVIEW QUEUE: 2 pending reviews

Komponent	Props	Opis
EvidenceTimeline	packs[], limit	Timeline ostatnich packów
HashChainValidator	chainValid, lastHash	Status hash chain
EvidenceSearch	onSearch, filters[]	Wyszukiwarka packów
GateRegistryStatus	gates[], registered	Status rejestru bramek
ExternalReviewQueue	reviews[], onAssign	Queue external reviewerów
Event WebSocket	Trigger	Akcja UI
evidence.pack.signed	Pack podpisany	Dodaj do timeline
gate.registered	Nowa bramka w rejestrze	Update gate registry
external.review.requested	Zgłoszenie review	Dodaj do review queue

UI / UX · Evidence — RBAC

admin: pełna zarządzanie, przypisywanie reviewerów. external_reviewer: dostęp do assigned packs, read-only. operator: read evidence packs, request review. viewer: read-only timeline.

§17.5.8 Rebuild & Cutover Panel

REBUILD & CUTOVER PANEL

REBUILD & CUTOVER State: SHADOW_VALIDATING

STATE MACHINE:

SHADOW_VALIDATING -> CUTOVER_READY ->

TRAFFIC_SHIFTING -> CUTOVER_COMPLETE ->

LEGACY_DECOMMISSION

[=====>] SHADOW_VALIDATING

HOT standby: [GREEN - OK] WARM standby: [OK]

LPW Countdown: 14d 6h 22m

[ROLLBACK BUTTON - 4/4 confirm required]

Komponent	Props	Opis
CutoverStateMachine	currentState, states[]	Wizualizacja state machine

Komponent	Props	Opis
StandbyIndicator	type, status, lastCheck	HOT/WARM indicator
LpwCountdown	deadline, entered	Odliczanie do LPW
RollbackButton	confirmCount, required, onConfirm	Guzik rollback z guard
CutoverProgressBar	state, percent	Pasek postępu cutover
Event WebSocket	Trigger	Akcja UI
cutover.state.changed	Zmiana stanu cutover	Odśwież state machine
rollback.triggered	Rollback uruchomiony	Alert + state -> SHADOW
lpw.entered	Wejście w LPW	Czerwony banner LPW
standby.degraded	Standby zdegradowany	Warning alert

REBUILD · Rebuild & Cutover — RBAC

admin: pełny management cutover + może triggerować rollback (po 4/4 confirm). operator: read-only + może być jednym z 4 potwierdzających rollback. viewer: read-only state machine i countdown.

§17.6 RBAC × Widoki — Matryca Dostępu

Poniższe tablice definiują uprawnienia dla każdej roli w każdym panelu. Trzy poziomy dostępu: R = read (widok danych), I = interact (akcje UI), A = admin (konfiguracja + przypisania).

Rola	Overview	Decisions	Code	Perf	Memory	Cost	Evidence	Rebuild
viewer	R	R	R	R	R	R*	R	R
operator	R+I	R+I(D0-D2)	R	R	R	R	R+I	R+I**
code_optimizer	R	R	R+I+A	R	R	R	R	R
performance_eng	R	R	R	R+I+A	R	R	R	R
memory_auditor	R	R	R	R	R+I+A	R	R	R
cost_optimizer	R	R	R	R	R	R+I+A	R	R

Rola	Overview	Decisions	Code	Perf	Memory	Cost	Evidence	Rebuild
admin	R+I+A	R+I+A	R+I+A	R+I+A	R+I+A	R+I+A	R+I+A	R+I+A
external_reviewer	R	R(PDF)	R	R	R	---	R(assigned)	---

* viewer nie widzi kwot finansowych (Cost panel maskuje wartości USD domyślnie). ** operator może potwierdzić rollback jako 1 z 4 wymaganych głosów.

§17.6.1 Permissions YAML Schema

```
# roles.yaml – SYLION Operator Console RBAC
version: "1.0"
roles:
  viewer:
    panels: [overview, decisions, code, performance, memory, evidence, rebuild]
    actions: [read]
    cost_panel:
      read: true
      financial_values_visible: false
  operator:
    panels: [overview, decisions, code, performance, memory, cost, evidence, rebuild]
    actions: [read, interact]
    decisions:
      max_class: D2
      council_vote: false
    rebuild:
      rollback_confirm: true # 1/4 votes
  code_optimizer:
    panels: [all]
    actions: [read]
    code_panel:
      actions: [read, interact, sign_reports, manage_queue]
  performance_eng:
    panels: [all]
    actions: [read]
    performance_panel:
      actions: [read, interact, update_slo]
  memory_auditor:
    panels: [all]
    actions: [read]
    memory_panel:
      actions: [read, interact, sign_profiles]
  cost_optimizer:
    panels: [all]
    actions: [read]
    cost_panel:
      actions: [read, interact, update_envelope]
  admin:
    panels: [all]
    actions: [read, interact, admin]
    special:
      - council_vote_trigger
```

```

- rollback_trigger_4_4
- deploy_freeze_toggle
- force_gc
external_reviewer:
  panels: [overview, decisions, code, performance, memory, rebuild]
  actions: [read]
  evidence_panel:
    access: assigned_only
    actions: [read, export_pdf]

```

UI / UX · RBAC — Backend enforcement

RBAC jest enforce'owany BACKEND-side (Plan 04 Vault + Plan 09 API Gateway). Frontend RBAC = UX hint, nie security boundary. Nieautoryzowane żądania są odrzucane przez backend z HTTP 403 niezależnie od stanu UI.

§17.7 Real-time: WebSocket Contracts

Operator Console korzysta z WebSocket dla pełnodupleksowej komunikacji z backendem. Każde połączenie jest uwierzytelnione tokenem OIDC, subskrybuje wybrany zestaw typów zdarzeń i odbiera wiadomości w ustandaryzowanej kopercie JSON.

§17.7.1 Connection Lifecycle

WebSocket Connection Lifecycle

Client	Server (WebSocket Hub)
-- TCP + TLS handshake (WSS) ----->	
<-- 101 Switching Protocols -----	
-- AUTH {token: "oidc-jwt"} ----->	
<-- AUTH_OK {userId, roles[]} -----	
-- SUBSCRIBE {events: [...]} ----->	
<-- SUBSCRIBED {ack: true} -----	
<-- EVENT {type, ts, payload} -----	(stream)
<-- HEARTBEAT {ts} -----	(30s interval)
-- HEARTBEAT_ACK {ts} ----->	
-- UNSUBSCRIBE {events: [...]} ----->	
<-- CLOSE (1000 Normal) ----->	
<-- CLOSE ACK -----	

§17.7.2 Message Envelope — JSON Schema

```

// AsyncAPI 2.6 Message Envelope
{
  "$schema": "https://asyncapi.com/schemas/2.6.0/schema.json",

```



```

    "type": "object",
    "required": ["id", "type", "ts", "payload", "sequence_number"],
    "properties": {
      "id": { "type": "string", "format": "uuid" },
      "type": { "type": "string", "enum": ["<event_type>"] },
      "ts": { "type": "string", "format": "date-time" },
      "payload": { "type": "object" },
      "signature": { "type": "string", "description": "HMAC-SHA256" },
      "sequence_number": { "type": "integer", "minimum": 0 },
      "correlation_id": { "type": "string", "format": "uuid" },
      "source_service": { "type": "string" }
    }
  }
}

```

§17.7.3 Tabela Event Types

Event Type	Trigger	Subscribers	Payload schema
pipeline.started	Nowy run	overview, all	runId, decisionClass, path
pipeline.gate.passed	Bramka zaliczona	overview, evidence	gateId, division, ts
pipeline.gate.veto	Bramka zablokowana	overview, evidence, all	gateId, reason, severity
pipeline.completed	Run ukończony	overview, all	runId, outcome, duration
pipeline.branch.restart ed	Restart branch	overview	branchId, initiator
decision.created	Nowa decyzja	decisions	id, class, title, context
decision.voted	Głos	decisions	decisionId, voter, vote, ts
decision.approved	Zatwierdzona	decisions, overview	decisionId, class, ts
decision.rejected	Odrzucona	decisions, overview	decisionId, reason, ts
decision.escalated	Eskalacja klasy	decisions, all	decisionId, fromClass, toClass
efficiency.report.publis hed	Raport Code Opt.	code	reportId, avgBloat, breaches
efficiency.breach.new	Nowy breach	code	moduleId, bloatScore, threshold
efficiency.breach.resol ved	Breach usunięty	code	moduleId, newScore
gate.G-EFF.veto	Bramka EFF veto	code, overview	reason, module, score

<i>Event Type</i>	<i>Trigger</i>	<i>Subscribers</i>	<i>Payload schema</i>
performance.baseline.updated	Nowy baseline	performance	metric, value, ts
slo.budget.breach	Budget SLO naruszony	performance, all	slold, budgetPct, threshold
gate.G-PERF.veto	Bramka PERF veto	performance, overview	reason, metric, value
memory.profile.published	Nowy profil pamięci	memory	profileId, rss, heap
leak.detected	Wykryty leak	memory, all	leakId, allocator, size
gate.G-MEM.veto	Bramka MEM veto	memory, overview	reason, rss, limit
cost.envelope.updated	Nowy limit kosztu	cost	newEnvelope, currency, period
cache.hit_rate.changed	Zmiana cache HR	cost	newRate, delta, ts
gate.G-COST.veto	Bramka COST veto	cost, overview	reason, burn, envelope
evidence.pack.signed	Pack podpisany	evidence	packId, decisionId, hash
gate.registered	Nowa bramka	evidence	gateId, type, owner
external.review.requested	Zgłoszenie review	evidence	reviewId, packIds[]
cutover.state.changed	Zmiana stanu	rebuild	fromState, toState, ts
rollback.triggered	Rollback	rebuild, all	initiator, reason, votes
lpw.entered	Wejście LPW	rebuild, all	deadline, reason
standby.degraded	Standby degradacja	rebuild	type, status, reason

§17.7.4 SSE Fallback

W środowiskach korporacyjnych z restrykcyjnymi proxy HTTP, WebSocket (WSS) może być blokowany. Console automatycznie wykrywa brak WebSocket i przełącza się na Server-Sent Events (SSE) jako fallback. SSE działa przez standardowe połączenie HTTPS, jest jednokierunkowe (server → client), co wystarcza dla event stream. Akcje bidirectional (głosowanie, potwierdzanie rollback) w trybie SSE używają oddzielnych REST POST.

§17.7.5 Backpressure i Throttling

<i>Typ zdarzenia</i>	<i>Rate limit</i>	<i>Burst budget</i>	<i>Drop strategy</i>
pipeline.*	10/s	50 eventów	drop oldest

Typ zdarzenia	Rate limit	Burst budget	Drop strategy
decision.*	20/s	100 eventów	queue (no drop)
efficiency.*	5/s	20 eventów	drop oldest
performance.*	30/s	150 eventów	drop oldest
memory.*	10/s	50 eventów	drop oldest
cost.*	5/s	25 eventów	drop oldest
evidence.*	10/s	50 eventów	queue (no drop)
cutover.*	10/s	50 eventów	queue (no drop)

UI / UX · SSE vs WebSocket — decyzja architektoniczna

WebSocket dla komunikacji bidirectional (Council vote, rollback confirm, subscribe/unsubscribe). SSE dla pure broadcast event stream (fallback). Decyzja zgodna z rekomendacją Backstage docs i AsyncAPI spec: nie narzucaj WebSocket tam, gdzie SSE wystarczy.

§17.8 Accessibility — WCAG 2.1 AA

Operator Console MUSI spełniać WCAG 2.1 AA w 100%. Dostępność nie jest opcjonalną cechą — jest wymaganiem funkcjonalnym na równi z bezpieczeństwem. Wszystkie komponenty są testowane automatycznie (axe-core w CI) i manualnie (NVDA/JAWS + keyboard-only testing) przed każdym releasem.

§17.8.1 Wymogi WCAG 2.1 AA

Kryterium	Poziom	Wymog	Implementacja
1.1.1 Non-text Content	A	Alt text dla obrazków	aria-label lub aria-labelledby na SVG/ikony
1.3.1 Info and Relationships	A	Semantyczna struktura	HTML5 semantic elements: main, nav, header, section
1.3.3 Sensory Characteristics	A	Nie tylko kolor/kształt	Każda informacja kolorowa ma ikonę + label
1.4.1 Use of Color	A	Kolor nie jedyny sygnał	Status = kolor + ikona + tekst (zawsze 3 moduły)

Kryterium	Poziom	Wymog	Implementacja
1.4.3 Contrast (Minimum)	AA	min 4.5:1 body, 3:1 large	Wszystkie tokeny kolorystyczne zwalidowane
1.4.4 Resize Text	AA	200% bez utraty treści	CSS rem + viewport units; no fixed px font-size
1.4.10 Reflow	AA	320px bez scroll poziomy	CSS grid + flexbox; media query na 320px
1.4.11 Non-text Contrast	AA	min 3:1 UI components	Border/outline komponentów zwalidowane
1.4.12 Text Spacing	AA	Line-height 1.5, letter-spacing 0.12em	CSS custom properties override user styles
2.1.1 Keyboard	A	Wszystkie funkcje keyboard-only	Tab order, Enter/Space, Esc dla modalów
2.4.3 Focus Order	A	Logiczny porządek focusu	DOM order = visual order; skip links
2.4.7 Focus Visible	AA	Widoczny focus indicator	3px solid outline, min 3:1 kontrast z tłem
3.2.2 On Input	A	Brak niespodziewanych zmian	Formularze zmieniają się tylko na submit
4.1.2 Name, Role, Value	A	Prawidłowe ARIA	Wszystkie custom components mają role + aria-*
4.1.3 Status Messages	AA	Programatyczne status messages	aria-live="polite" dla notyfikacji WS events

§17.8.2 Skip Links i Landmark Regions

```

<!-- Skip Links (pierwsze elementy w DOM) -->
<a href="#main-content" class="skip-link">Skip to main content</a>
<a href="#main-nav" class="skip-link">Skip to navigation</a>

```

```

<!-- Landmark Regions -->
<header role="banner">           <!-- Top bar           -->
  <nav role="navigation">        <!-- Sidebar nav       -->
    <main id="main-content">      <!-- Canvas panels    -->
      <aside role="complementary"> <!-- Right rail      -->
        <footer role="contentinfo"> <!-- Footer info  -->

```

Keyboard navigation:

Tab -- przechodzenie przez focusable elements

Shift+Tab	-- cofanie
Enter	-- aktywacja przycisku / linku
Space	-- toggle checkbox / toggle button
Esc	-- zamknięcie modal / drawer
Arrow keys	-- nawigacja w listach, menu, tabpanel

§17.8.3 Screen Reader Support

Komponent / sytuacja	ARIA implementation	Live region
WebSocket event (gate veto)	aria-live="assertive" alert	Natychmiastowy komunikat
WebSocket event (decision approved)	aria-live="polite" status	Po ukończeniu akcji
Tabela ładowania danych	aria-busy="true", aria-label="Loading..."	Status oczekiwania
Progress bar działu	role="progressbar" + aria-valuenow	Dynamiczne wartości
Decision badge	aria-label="Decision class D3: Council required"	Opisowy label
Chart (sparkline)	aria-label + summary tabela danych	Dane tekstowo dostępne
Rollback button (guard)	aria-disabled + aria-describedby (guard status)	Stan przycisku

§17.8.4 Color Independence

Informacja	Kolor	Ikona	Label tekstowy	Pattern/shape
Gate PASS	zielony	check-circle	PASS	--
Gate FAIL	czerwony	x-circle	FAIL	--
Gate WARN	amber	alert-triangle	WARN	--
Decision D0	zielony	auto-badge	D0 AUTO	--
Decision D3	amber	council-badge	D3 CNCL	thick border
Decision D5	fiolet	rebuild-badge	D5 RBLD	double border
Error budget OK	zielony	shield-check	>80% budget	--
Error budget LOW	amber	shield-warning	20-80%	dashed border
Error budget BREACH	czerwony	shield-x	<20%	red stripe

§17.8.5 Reduced Motion

Wszystkie animacje (przejścia CSS, skeleton loaders, progress bars, sparkline charts) respektują media query prefers-reduced-motion: reduce. W trybie reduced motion: animacje są wyłączone lub skrócone do instant transition. Skeleton loaders → statyczny placeholder. Auto-scrolling timelines → wyłączony. Countdown timer → tylko aktualizacja liczby, bez animacji cyfr.

§17.8.6 Audit Checklist

- axe-core run w CI dla każdego PR — 0 violations wymagań
- Lighthouse accessibility score ≥ 95 w pipeline CD
- Keyboard-only test wszystkich 8 panelów przez QA przed releasem
- NVDA + Chrome i JAWS + Edge — screen reader manual test
- Color contrast sprawdzony dla Light / Dark / High-contrast
- Skip links widoczne przy focusie w każdym trybie
- Focus trap poprawny w modals i drawerach
- aria-live regions testowane z rzeczywistymi WS eventów

UWAGA · WCAG AA — obowiązkowo 100%

Tylko 25% enterprise operator consoles jest kompliantnych z WCAG AA według danych branżowych. SYLION Operator Console = 100% obowiązkowo — nie opcjonalnie. Każda regresja dostępności blokuje release na równi z regresją bezpieczeństwa.

§17.9 Theming i Customizacja

Operator Console wspiera pełny theming przez CSS custom properties. Zmiana motywu nie wymaga przebudowy aplikacji — tylko podmiany zestawu tokenów CSS.

§17.9.1 Light / Dark / High-contrast Tokens

Token CSS	Light	Dark	High-contrast
--color-bg	#F5F5F5	#121212	#000000
--color-surface	#FFFFFF	#1E1E1E	#000000
--color-surface-elevated	#FAFAFA	#2C2C2C	#1A1A1A
--color-border	#E0E0E0	#333333	#FFFFFF
--color-text-primary	#212121	#E0E0E0	#FFFFFF
--color-text-secondary	#757575	#9E9E9E	#CCCCCC
--color-primary	#1976D2	#42A5F5	#00AAFF

Token CSS	Light	Dark	High-contrast
--color-success	#2E7D32	#66BB6A	#00FF00
--color-warning	#F57C00	#FFA726	#FFFF00
--color-danger	#C62828	#EF5350	#FF4444

§17.9.2 White-label Support

Console wspiera white-label przez branding tokens konfigurowane przez ACP (Plan 01). Konfigurowalne: logo (URL do SVG), --color-primary (brand color), --color-secondary, font-family (Google Fonts URL lub lokalna czcionka), aplikacja title i favicon. Konfiguracja jest dostarczana jako JSON przez endpoint /api/v1/branding i cached przez CDN. Zmiana brandingu nie wymaga rebuildu aplikacji.

§17.9.3 Per-user Preferences

Preferencja	Opcje	Default	Persystencja
Density	compact / comfortable / spacious	comfortable	localStorage + backend
Language	pl / en	pl (sys detect)	backend user profile
Theme	light / dark / system / high-contrast	system	localStorage + backend
Timezone	IANA timezone string	Europe/Warsaw	backend user profile
Dashboard layout	JSON grid config	default layout	backend user profile
Notifications	all / critical-only / off	all	backend user profile
Sidebar state	expanded / collapsed	expanded	localStorage
Right rail state	open / closed	closed	localStorage

```
// user_preferences.json – schema
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
    "userId": { "type": "string", "format": "uuid" },
    "theme": { "type": "string", "enum": ["light","dark","system","hc"] },
    "density": { "type": "string", "enum": ["compact","comfortable","spacious"] },
    "language": { "type": "string", "enum": ["pl","en"] },
    "timezone": { "type": "string" },
    "notifications": { "type": "string", "enum": ["all","critical","off"] },
    "dashboardLayout": {
      "type": "object",
      "description": "12-col grid slot assignments per panel"
    },
    "updatedAt": { "type": "string", "format": "date-time" }
```

```
  },
  "required": ["userId", "theme", "density", "language"]
}
```

§17.10 Internationalizacja (i18n)

Operator Console jest w pełni zlokalizowany w językach polskim (pl) i angielskim (en). Biblioteka: react-i18next z ICU MessageFormat dla pluralizacji, interpolacji i daty. Klucze tłumaczeń są zorganizowane w namespace'y odpowiadające sekcjom aplikacji.

ZASADA · i18n — wymagania minimalne

v3.3: PL + EN. RTL support (arabski, hebrajski) jest gotowy architektonicznie (CSS logical properties, dir attr) ale nie wymagany w v3.3. Dodanie języka RTL w przyszłości nie wymaga zmian w komponentach.

Namespace	Przykładowe klucze	Liczba kluczy
common	app.title, nav.*, buttons.*, status.*, errors.*	~40
panels.overview	overview.runId, overview.status.*, overview.dora.*	~25
panels.decisions	decisions.ladder.*, decisions.pending.*, decisions.history.*	~30
panels.code	code.bloat.*, code.breach.*, code.optimizer.*	~20
panels.performance	perf.use.*, perf.latency.*, perf.red.*, perf.slo.*	~20
panels.memory	mem.rss, mem.heap, mem.leaks.*, mem.gc.*	~18
panels.cost	cost.burn.*, cost.envelope.*, cost.cache.*, cost.models.*	~18
panels.evidence	evidence.pack.*, evidence.chain.*, evidence.review.*	~20
panels.rebuild	rebuild.state.*, rebuild.standby.*, rebuild.lpw.*	~20
decisions.*	decision.class.D0-D5.label/description, vote.*, escalation.*	~35

Namespace	Przykładowe klucze	Liczba kluczy
errors.*	error.network, error.auth, error.ws.*, error.api.*	~25
accessibility.*	aria.labels.*, skip.*, sr.announcements.*	~20
notifications.*	notif.gate.veto, notif.decision.*, notif.leak.*	~20

Formatowanie daty / liczb: żadne hardkodowane formaty. Wszystkie daty przez Intl.DateTimeFormat z aktywnym locale. Liczby (koszty USD, procenty, bajty) przez Intl.NumberFormat. Strefy czasowe: zawsze IANA timezone string z user preferences. ICU MessageFormat dla pluralizacji: "decisions.pending.count": "{count, plural, one {# decyzja} few {# decyzje} other {# decyzji}}"

§17.11 Performance Budget Operator Console

Operator Console musi spełniać określone metryki wydajności mierzone narzędziami Lighthouse, webpack-bundle-analyzer i custom metryki w pipeline CD. Przekroczenie budżetów blokuje release.

PERFORMANCE · Performance budget = brama CI

Każde PR musi przejść automatyczny performance check. Regresja >10% dla LCP lub +50KB bundle size blokuje merge na równi z błędem testów.

Metryka	Target	Pomiar	Blokuje release
First Contentful Paint (FCP)	<1.5s	Lighthouse	tak (>2.5s)
Largest Contentful Paint (LCP)	<2.5s	Lighthouse	tak (>4.0s)
Time to Interactive (TTI)	<3.5s	Lighthouse	tak (>5.0s)
Cumulative Layout Shift (CLS)	<0.1	Lighthouse	tak (>0.25)
First Input Delay (FID)	<100ms	Lighthouse	tak (>300ms)
Total Bundle Size (gzip)	<500KB	webpack-bundle-analyzer	tak (>700KB)
WebSocket reconnect time	<1s	custom metric (RUM)	nie (alert)

Metryka	Target	Pomiar	Blokuje release
Initial HTML size	<15KB	custom	tak (>30KB)
API Gateway round-trip p95	<200ms	RUM	nie (alert)
Chart render time (8 panels)	<100ms	React Profiler	nie (alert)

§17.11.1 Code-splitting Strategy

Vite + Rollup automatycznie dzieli kod na chunki per-route. Każdy z 8 panelów jest dynamicznie importowany (React.lazy + Suspense). Podział chunków: vendor (React, Recharts) → ~180KB gz; app shell (layout, auth, i18n, WebSocket) → ~80KB gz; per panel → ~15-30KB gz each. Strona ładuje się z app shell + active panel. Pozostałe panele są prefetchowane w idle time (requestIdleCallback).

§17.11.2 Real-time Event Throttling

Panel	Max updates/s UI	Batch window	Strategia
Pipeline Overview	5/s	200ms	batch + dedupe by runId
Decision Ladder	20/s	50ms	queue (no drop)
Code Health	2/s	500ms	batch + latest wins
Performance	10/s	100ms	batch timeseries points
Memory	5/s	200ms	batch + latest wins
Cost	2/s	500ms	batch + latest wins
Evidence	10/s	100ms	queue (no drop)
Rebuild	10/s	100ms	queue (no drop)

§17.12 Observability i Error Handling

Frontend Operator Console ma własną warstwę observability: telemetrię błędów, Real User Monitoring (RUM) i sesje replay z redakcją PII.

§17.12.1 Frontend Telemetry

Telemetria jest zbierana przez OpenTelemetry JS SDK i wysyłana do backend OTEL collector (Plan 09). Zbierane dane: błędy JS (unhandled exceptions, promise rejections) z pełnym stack trace i kontekstem (userId hash, rola, aktywny panel); Core Web Vitals per page view; WebSocket event latency (czas od emisji do renderowania); Custom metrics: panel load time,

chart render time, reconnect count. Session replay: nagrywanie sesji z PII redaction (maskowanie wartości finansowych, ID decyzji zastąpione hashów).

§17.12.2 User-facing Errors

Kategoria błędu	Traktowanie UI	Akcja użytkownika
WebSocket disconnect	Amber banner: "Reconnecting..."	Auto-retry (3 razy)
WebSocket auth fail	Czerwony dialog + logout button	Re-login (token refresh)
API 403 Forbidden	Inline error + RBAC hint	Brak akcji (info only)
API 500 Server Error	Toast error + "Try again" button	Retry z backoff
API Timeout (>10s)	Amber toast + retry button	Retry / cancel
Panel data stale (>5min)	Gray badge "Last updated: Xmin ago"	Manual refresh
Offline (network down)	Full-screen banner "Offline mode"	Auto-reconnect polling
Chart render error	Empty state z ikoną + message	Refresh panel button
RBAC action blocked	Disabled button + tooltip z powodem	Brak (info only)

§17.12.3 Offline / Degraded Mode

W przypadku utraty połączenia (sieć lub backend): Console przełącza się w tryb offline. Cached snapshots danych (z TanStack Query cache) są nadal wyświetlane z banerem "Ostatnia aktualizacja X minut temu". Wszystkie akcje wymagające serwera (vote, approve, rollback) są wyłączone z komunikatem. Auto-reconnect polling co 5 sekund. Po przywróceniu połączenia: automatyczny refetch danych + WebSocket reconnect + powiadomienie "Reconnected, data refreshed".

§17.13 Drabina Decyzyjna i Klasy Zmian Operator Console

Operator Console, jako moduł ACP (Plan 01), podlega tej samej drabinie decyzyjnej D0–D5 co cały pipeline. Poniższa tabela definiuje klasy zmian w kontekście konsoli.

Klasa	Przykład zmiany w Console	Procedura
D0	Automatyczna naprawa bugu (typo, kolor off by 1px)	Automatyczna PR — brak review
D1	Aktualizacja copy / label / ikony w istniejącym komponencie	Direct push z code review (owner)

Klasa	Przykład zmiany w Console	Procedura
D2	Nowy widget w istniejącym panelu; nowy token kolorystyczny	Feature flag + canary 5% + review
D3	Nowy panel (9. panel); zmiana nawigacji sidebar	Council 4/4 + design review + a11y audit
D4	Zmiana information architecture (restructure sidebar, nowy layout)	Council + Human sign-off + UX research
D5	Pełna przebudowa Console (nowy framework / design system)	Jak D5 całego pipeline — pełny rebuild process

Zasada rebuildability konsoli: każda decyzja D3+ dotycząca Console musi mieć documented evidence pack i jest zarchiwizowana w gate registry (tak samo jak decyzje architektoniczne pipeline core). D4–D5 decyzje dotyczące Console wymagają dodatkowych: UX research (min. 5 user sessions), design mockup review, WCAG compliance re-audit całej przebudowanej części.

REBUILD · Console Rebuildability

Console też podlega Rebuildability — rebuildable jak każdy moduł. Frontend jest osobnym modułem ACP z własnym implementation_strategy. Rebuild Console (D5) można wykonać bez zatrzymywania backend pipeline. Stara i nowa wersja Console mogą działać równolegle w trybie shadow mode — analogicznie do cutover pipeline core.

§17.14 Roadmap Implementacyjny

Roadmap implementacji Operator Console podzielony jest na cztery fazy. Każda faza kończy się wewnętrznym demo i a11y spot-check. Fazy mogą być realizowane równolegle z core pipeline — Console jest niezależnym modułem ACP.

<i>Faza</i>	<i>Scope</i>	<i>Czas</i>	<i>Dział owner</i>	<i>Deliverables</i>
Faza 1	Design system (tokeny, typografia, spacing, elevation, border radius). Pipeline Overview Panel. Decision Ladder Panel. App shell (layout, top bar, sidebar, auth OIDC, WebSocket infrastructure).	6 tygodni	Implementacja	Design system package, 2 panele, app shell, WebSocket client
Faza 2	Code Health Panel + gate G-EFF. Performance Panel + gate G-PERF. Memory Panel + gate G-MEM. Cost Panel + gate G-COST. RBAC enforcement per panel.	8 tygodni	Implementacja + Code Optimizer + Perf Eng + Memory Auditor + Cost Optimizer	4 efficiency panels, RBAC full, CI performance budget
Faza 3	Evidence & Governance Panel (timeline, hash chain, external review). Rebuild & Cutover Panel (state machine, rollback guard, LPW countdown). Gate registry integration.	4 tygodnie	Governance + Platforma	2 panele, evidence viewer, cutover UI

Faza	Scope	Czas	Dział owner	Deliverables
Faza 4	Internacjonalizacja PL/EN pełna (200+ kluczy). WCAG 2.1 AA audit pełny (axe-core + manual). Performance budget enforcement w CI. User preferences backend. White-label support. Session replay + RUM.	4 tygodnie	Wszystkie działy	i18n ready, WCAG AA cert, RUM dashboard, white-label config

Warunki ukończenia wdrożenia (Definition of Done dla Operator Console): wszystkie 8 panelów działa w trybie real-time z WebSocket; RBAC enforcement zwalidowany dla wszystkich 8 ról; WCAG 2.1 AA 100% (axe-core zero violations + manual test); Performance budget spełniony (LCP <2.5s, bundle <500KB gz); i18n PL/EN 100% pokrycie kluczy; a11y audit i performance budget w pipeline CI/CD; dokumentacja komponentów (Storybook).

UI / UX · Plan 17 — Podsumowanie

Plan 17 jest pełną specyfikacją Operator Console enterprise — gotową do implementacji równoległe z core pipeline SYLION. Obejmuje: architekturę frontend, design system z 30+ tokenami, osiem panelów operacyjnych z ASCII mockupami i kontraktami WebSocket, pełną matrycę RBAC dla 8 ról, WCAG 2.1 AA 100%, internacjonalizację PL/EN, performance budget z CI enforcement i roadmap 22-tygodniowy podzielony na 4 fazy. Console podlega tym samym zasadom Rebuildability i Drabiny Decyzyjnej co cały pipeline SYLION.

PLAN 18 · WARSTWA KANONICZNA

Skills & Knowledge Pipeline

Plan 18 — Skills & Knowledge Pipeline

§18.1 Wprowadzenie i Rebuildability principle

Skills są pierwszoklasowym artefaktem SYLION Pipeline v3.3. W odróżnieniu od dokumentacji (opisowej), runbooków (proceduralnych) i promptów (jednorazowych instrukcji), skill jest „wykonywalnym pakietem wiedzy” — modularnym, testowalnym, wersjonowanym i podlegającym pełnemu cyklowi lifecycle oraz governance. Dokumentacja opisuje, runbook instruuje, prompt pyta — skill wykonuje.

Skills pozwalają agentom na dostęp do specjalistycznej wiedzy domenowej w sposób kontrolowany, audytowalny i reprodukowalny. Każdy skill musi być rebuildable: można go odtworzyć w dowolnym momencie z repozytorium, zwalidować automatycznie i opublikować poprzez ten sam pipeline governance co moduły kodu.

SKILL · Rebuildability principle

Skills MUSZĄ być rebuildable — wersjonowanie, lifecycle i governance jak moduły kodu. Każdy skill ma semver, Evidence Pack i wymaga Council quorum przy zmianach $\geq$ MINOR.

SKILL · Definicja kanoniczna

„Skill = wykonywalny pakiet wiedzy z kontraktem, lifecycle i governance”

Badania Anthropic nad Computer Use (14,9% OSWorld benchmark) potwierdzają, że zdolność agentów do wykonywania specjalistycznych zadań jest wprost proporcjonalna do jakości i struktury dostępnych skillów. SYLION implementuje skill contracts oparte o OpenAPI 3.1 + JSON Schema, a każdy plik główny manifestu (SKILL.md) zawiera semver versioning zgodny z semver 2.0.0.

§18.2 Model domenowy Skills

Model domenowy Skills definiuje jednolity język, którym posługują się wszystkie komponenty pipeline: agenci, registry, governance i Operator Console. Precyzyjne definicje eliminują niejednoznaczność i umożliwiają automatyczną walidację.

§18.2.1 Definicje

Termin	Definicja
Skill	Wykonywalny pakiet wiedzy z kontraktem wejścia/wyjścia, lifecycle i governance

Termin	Definicja
Skill Module	Implementacja konkretnej funkcji w ramach skilla (jeden skill = 1..N modułów)
Skill Catalog	Rejestr wszystkich opublikowanych skillów (patrz Aneks M)
Skill Registry	Runtime instancja katalogu — obsługuje discovery, resolution, loading
Skill Contract	Formalny opis API oparty o OpenAPI 3.1 + JSON Schema (input/output)
Skill Manifest	Plik skill.yaml — główny artefakt kontraktu i metadanych
Skill Bundle	Tarball: manifest + kod + testy + dokumentacja + podpis cyfrowy

§18.2.2 Klasyfikacja skillów

Kategoria	Przykłady skillów	Owner department
Analiza	analiza-ksiegi-canon, bloat-detection, dependency-graph	Architektura
Wdrożenie	cutover-execution, lpw-management, migration-rollout	Implementacja
Testowanie	contract-test-runner, regression-suite, golden-set-eval	QA & Testing
Optymalizacja	efficiency-audit, perf-profiling, cost-reduction-advisor	Optymalizacja
Compliance	soc2-audit, nist-mapping, iso27001-checker	Compliance & Security
Observability	golden-signals-dashboard, slo-monitoring, alert-correlator	Observability

§18.2.3 Skill vs Agent vs Role

Skill, Agent i Role to odrębne koncepty pipeline — poniższa tabela precyzuje różnice:

Wymiar	Skill	Agent	Role
Granularność	Atomowa funkcja / wiedza	Autonomiczny wykonawca	Zbiór uprawnień

Wymiar	Skill	Agent	Role
Lifecycle	DRAFT→PUBLISHED→DEPRECATED	Deploy / restart	Assign / revoke
Kontrakt	OpenAPI 3.1 + JSON Schema	Task queue + callback	RBAC policy
Wersjonowanie	Semver 2.0.0	Image tag	Policy version
Governance	Council quorum	Change Advisory Board	IAM review
Audytowalność	Audit log per invocation	Trace per run	Access log

§18.2.4 Skill ownership

Każdy skill MUSI posiadać przypisanego właściciela (`primary_owner`) wybranego spośród 25 ról Boardu SYLION oraz department owner odpowiedzialny za utrzymanie, roadmap i deprecation decyzje. Brak ownership blokuje przejście ze stanu DRAFT do REVIEW. Primary owner pełni rolę DRI (Directly Responsible Individual) i podpisuje manifest.sig w bundle.

§18.3 Lifecycle skillów — DRAFT → PUBLISHED → DEPRECATED

Każdy skill przechodzi przez sześć stanów lifecycle, zarządzanych przez ten sam mechanizm Council i Evidence Pack co moduły kodu. Przejścia są atomowe, nieodwracalne bez Council quorum i każde zapisywane w audit logu z hash chain.

TEXT

```
[DRAFT] -- propose D2 --> [REVIEW]
[REVIEW] -- Council 3/4 --> [PUBLISHED]
[REVIEW] -- reject --> [DRAFT] (with feedback)
[PUBLISHED] -- bug fix D1 --> [PUBLISHED] (patch versioning)
[PUBLISHED] -- breaking change D3 --> [DEPRECATED] + new MAJOR
[DEPRECATED] -- 90d sunset window --> [ARCHIVED]
[PUBLISHED] -- security incident --> [QUARANTINE]
[QUARANTINE] -- patch + Council --> [PUBLISHED]
```

§18.3.1 Tabela stanów lifecycle

Stan	Triggery wejścia	Triggery wyjścia	Klasa decyzji	Owner
DRAFT	Inicjacja (nowy skill)	Propose D2	D2	primary_owner
REVIEW	Propose z DRAFT	Council 3/4 lub reject	D2–D3	Council
PUBLISHED	Council 3/4 z REVIEW lub patch z PUBLISHED	Bug fix / breaking / security	D1–D3	primary_owner

Stan	Triggery wejścia	Triggery wyjścia	Klasa decyzji	Owner
DEPRECATED	Breaking change lub end-of-life	90d sunset→ARC HIVED	D3	Council
ARCHIVED	Sunset 90d z DEPRECATED	— (stan końcowy)	D4	Board
QUARANTINE	Security incident	Patch + Council	D3	Security + Council

§18.3.2 Versioning semver

Wszystkie skille stosują semver 2.0.0 (MAJOR.MINOR.PATCH). Typ zmiany determinuje klasę decyzji i wymagane quorum Council:

Typ zmiany	Wersja	Klasa decyzji	Council quorum	Przykład
Bugfix (patch)	0.0.X	D1	primary_owner	Naprawa edge case w parserze
Nowa funkcja (minor)	0.X.0	D2	3/4 reviewers	Nowy wymiar audytu
Breaking change (major)	X.0.0	D3	4/4 Council	Zmiana schematu input/output
Nowa kategoria	—	D4	Board + Council	Dodanie kategorii „ML Ops”
Zmiana modelu	—	D5	Pełny Board	Restrukturyzacja domenowa

§18.3.3 Deprecation policy

UWAGA · Deprecation policy — obowiązkowe

90-dniowe sunset window: w ciągu 90 dni po oznaczeniu DEPRECATED, wszystkie zależności MUSZĄ przejść na skill replacement. Brak replacement blokuje deprecation (wyjątek: security QUARANTINE). NOTICE o deprecation pojawia się w canon_compact w polu "deprecation_notices" od dnia oznaczenia.

SKILL · Zasada kanoniczna lifecycle

„Lifecycle skillów = lifecycle modułów. Te same gates, te same Council, ten sam Evidence Pack.”

§18.4 Schemat skill.yaml (pełny kontrakt)

Plik skill.yaml jest jedynym wiążącym kontraktem skilla. Wszystkie narzędzia pipeline (registry, CI, Operator Console, agenci) czytają wyłącznie ten plik jako źródło prawdy. Poniżej pełny

manifest ze wszystkimi polami wymaganymi i opcjonalnymi:

YAML

```
# skill.yaml – kontrakt skill SYLION Pipeline v3.3
spec_version: "1.0"
id: "efficiency-audit"
version: "1.2.0"
state: "PUBLISHED"
state_changed_at: "2026-04-15T10:00:00Z"

metadata:
  display_name: "Efficiency Audit"
  description: "Pełny audyt efficiency modułu na 4 wymiarach"
  category: "optymalizacja"
  language: "pl"
  tags: ["efficiency", "bloat", "audit"]

ownership:
  department: "Implementacja"
  primary_owner: "code_optimizer_agent"
  reviewers: ["performance_engineer", "memory_auditor", "cost_optimizer"]

compatibility:
  pipeline_version_min: "8.3.0"
  pipeline_version_max: "*"
  mz_baseline_min: "3.5.5"
  required_modules: ["plan_10", "plan_12", "plan_15"]

contract:
  input_schema: "schemas/efficiency-audit.input.json"
  output_schema: "schemas/efficiency-audit.output.json"
  openapi: "openapi/efficiency-audit.openapi.yaml"

execution:
  decision_class_max: "D2"
  requires_council_for: ["D3", "D4", "D5"]
  estimated_cost_usd_p95: 0.42
  estimated_runtime_sec_p95: 18
  parallel_safe: true
  idempotent: true

requires_compact_fields:
  - "efficiency_snapshot"
  - "modules_in_efficiency_breach"

emits_artifacts:
  - "code_efficiency_report.json"
  - "refactoring_recommendations.md"

gates_triggered:
  - "G-EFF-01"
  - "G-EFF-02"
  - "G-EFF-03"
  - "G-EFF-04"

governance:
  changelog: "CHANGELOG.md"
```

```
evidence_required: true
audit_log_retention_days: 365

testing:
  unit_tests: "tests/unit/"
  contract_tests: "tests/contract/"
  golden_set: "tests/golden/"
  min_coverage_pct: 85

documentation:
  readme: "README.md"
  examples: "examples/"
  troubleshooting: "TROUBLESHOOTING.md"

deprecation:
  deprecated_in_version: null
  sunset_date: null
  replacement: null
```

§18.4.1 Pola manifestu — wymagane vs opcjonalne

Pole	Poziom	Typ	Opis
spec_version	Wymagane	string	Wersja specyfikacji skill.yaml (np. "1.0")
id	Wymagane	string	Unikalny identyfikator skilla (slug, kebab-case)
version	Wymagane	semver	Wersja skilla wg semver 2.0.0
state	Wymagane	enum	DRAFT REVIEW PUBLISHED DEPRECATED ARCHIVED QUARANTINE
state_changed_at	Wymagane	ISO8601	Timestamp ostatniej zmiany stanu
metadata.display_name	Wymagane	string	Nazwa wyświetlana w UI i katalogu
metadata.description	Wymagane	string	Opis funkcji skilla (max 200 znaków)
metadata.category	Wymagane	enum	Jedna z 6 kategorii (analiza/wdrożenie/...)
metadata.language	Wymagane	BCP47	Język skilla (np. "pl", "en")
metadata.tags	Opcjonalne	array[string]	Słowa kluczowe dla wyszukiwarki

<i>Pole</i>	<i>Poziom</i>	<i>Typ</i>	<i>Opis</i>
ownership.department	Wymagane	string	Department owner (jeden z 8 dept)
ownership.primary_owner	Wymagane	string	DRI — rola z 25 ról Boardu
ownership.reviewers	Wymagane	array[string]	Min. 2 reviewers do REVIEW
compatibility.pipeline_version_min	Wymagane	semver	Min. wersja pipeline
compatibility.pipeline_version_max	Wymagane	semver *	Max. wersja pipeline (* = bez limitu)
compatibility.mz_baseline_min	Wymagane	semver	Min. wersja MZ baseline
compatibility.required_modules	Opcjonalne	array[string]	Wymagane moduły pipeline
contract.input_schema	Wymagane	path	Ścieżka do JSON Schema dla input
contract.output_schema	Wymagane	path	Ścieżka do JSON Schema dla output
contract.openapi	Wymagane	path	Ścieżka do pliku OpenAPI 3.1
execution.decision_classes_max	Wymagane	enum D0-D5	Max klasa decyzji podpisywana autonomicznie
execution.requires_council_for	Wymagane	array[string]	Klasy wymagające Council
execution.estimated_cost_usd_p95	Wymagane	float	Szacowany koszt USD (p95)
execution.estimated_runtime_sec_p95	Wymagane	integer	Szacowany czas wykonania (p95)
execution.parallel_safe	Wymagane	boolean	Czy skill bezpieczny przy równoległym wykonaniu
execution.idempotent	Wymagane	boolean	Czy powtórne wykonanie daje ten sam wynik
requires_compact_fields	Opcjonalne	array[string]	Pola canon_compact wymagane przez skill

Pole	Poziom	Typ	Opis
emits_artifacts	Opcjonalne	array[string]	Pliki wytwarzane przez skill
gates_triggered	Opcjonalne	array[string]	Bramki jakości uruchamiane przez skill
governance.changelog	Wymagane	path	Ścieżka do CHANGELOG.md
governance.evidence_required	Wymagane	boolean	Czy Evidence Pack wymagany
governance.audit_log_retention_days	Wymagane	integer	Retencja audit logu (dni)
testing.unit_tests	Wymagane	path	Katalog testów jednostkowych
testing.contract_tests	Wymagane	path	Katalog testów kontraktu
testing.golden_set	Wymagane	path	Katalog golden set
testing.min_coverage_percent	Wymagane	integer	Min. pokrycie testów (%)
documentation.readme	Wymagane	path	Ścieżka do README.md
documentation.examples	Opcjonalne	path	Katalog przykładów
documentation.troubleshooting	Opcjonalne	path	Ścieżka do TROUBLESHOOTING.md
deprecation.deprecated_in_version	Opcjonalne	semver null	Wersja w której oznaczono deprecated
deprecation.sunset_date	Opcjonalne	ISO8601 null	Data usunięcia skilla
deprecation.replacement	Opcjonalne	string null	ID skilla zastępczego

SKILL · Zasada kanoniczna manifestu

„skill.yaml jest jedynym kontraktem. Wszystko inne się zmienia, manifest pozostaje.”

§18.5 Bundle skilla

Skill Bundle to self-contained, weryfikowalny artefakt dystrybucji. Składa się z manifestu, kodu źródłowego, testów, dokumentacji oraz podpisu cyfrowego. Bundle jest niepodzielny: wdrażany i wycofywany jako całość.

TEXT

```

efficiency-audit-1.2.0.tar.gz
├── skill.yaml           # manifest (jedyne źródło prawdy)
├── README.md
├── CHANGELOG.md
├── TROUBLESHOOTING.md
├── schemas/
│   ├── efficiency-audit.input.json
│   └── efficiency-audit.output.json
├── openapi/
│   └── efficiency-audit.openapi.yaml
├── src/                 # implementacja
│   └── ...
├── tests/
│   ├── unit/
│   ├── contract/
│   └── golden/
├── examples/
└── manifest.sig         # podpis cyfrowy primary_owner
  
```

§18.5.1 Walidacja bundle — bramki

Bramka	Weryfikowany artefakt	Narzędzie	Blokuje?
B-SKL-01	skill.yaml syntax	YAML lint + JSON Schema	TAK
B-SKL-02	Schema validity (input/output)	JSON Schema validator	TAK
B-SKL-03	OpenAPI 3.1 compliance	swagger-parser	TAK
B-SKL-04	Podpis cyfrowy manifest.sig	RSA-PSS-2048 verify	TAK
B-SKL-05	Test coverage ≥ 85%	coverage.py / jest --coverage	TAK
B-SKL-06	Wymagane pliki obecne	Bundle integrity check	TAK
B-SKL-07	Compatibility window OK	Registry compatibility checker	TAK
B-SKL-08	Brak cyklicznych zależności	DAG analyzer	TAK

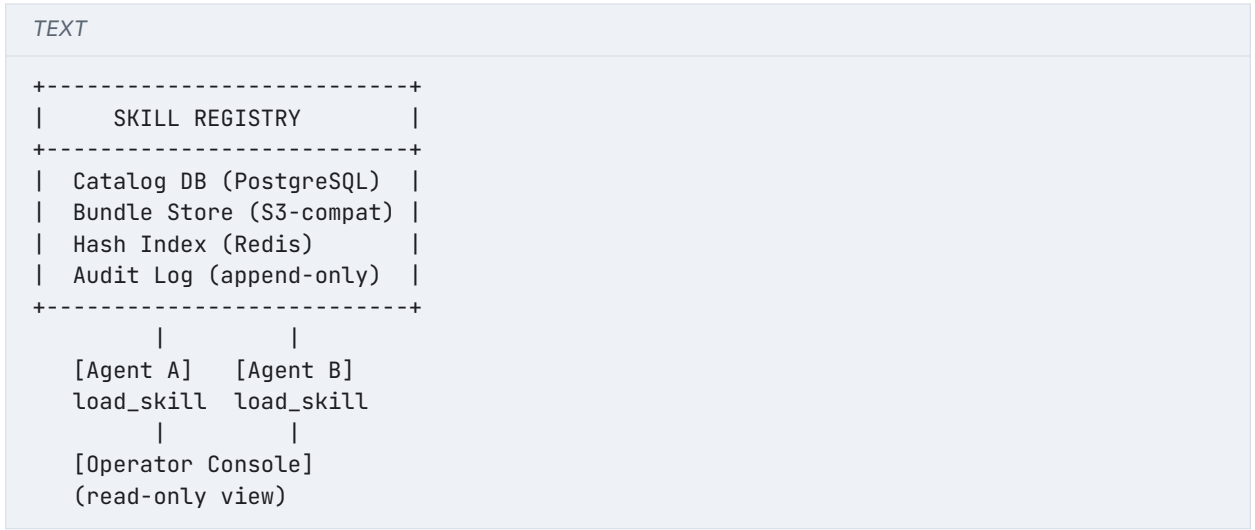
§18.5.2 Hash i podpisy

Każdy bundle zawiera plik manifest.sig generowany przez primary_owner algorytmem RSA-PSS-2048 z kluczem ządania roli (role key store). Hash SHA-256 całego bundle jest zapisywany w audit logu Skill Registry wraz z timestampem i identyfikatorem role key. Weryfikacja hash jest warunkiem niezbędnym każdego load_skill() w runtime agentów.

§18.6 Skill Registry — runtime

Skill Registry to centralny komponent runtime pipeline, odpowiedzialny za discovery, resolution, walidację i loading skillów. Działa jako single source of truth dla wszystkich agentów. Registry jest replikowany w trybie read-replica dla HA.

§18.6.1 Architektura Registry



§18.6.2 API Registry

Endpoint	Metoda	Opis	Klasa decyzji
/skills	GET	Lista wszystkich opublikowanych skillów	D0
/skills/{id}	GET	Szczegóły skilla (manifest + metadata)	D0
/skills/{id}/versions	GET	Historia wersji skilla	D0
/skills	POST	Publikacja nowego skilla (wymaga D3 quorum)	D3
/skills/{id}	PATCH	Update stanu lifecycle (D2–D3)	D2–D3
/skills/{id}	DELETE	Deprecate skilla (wymaga D3)	D3

Endpoint	Metoda	Opis	Klasa decyzji
/skills/{id}/invoke	POST	Wywołanie skilla (wymaga auth token)	D0–D5
/skills/{id}/audit	GET	Audit log skilla (wymaga D2)	D2

§18.6.3 Discovery i dependency resolution

Agent wywołuje `registry.resolve(skill_id, version_constraint)` z constraint w formacie semver range (np. `">=1.2.0,<2.0.0"`). Registry zwraca najwyższą kompatybilną wersję spełniającą constraint ORAZ `compatibility.pipeline_version_min/max`. Jeśli `wymagane_modules[]` skilla nie są załadowane, registry automatycznie je resolve'uje rekurencyjnie (max głębokość = 5).

§18.6.4 Skill loading w agencie

TEXT

```
agent.start()
  → agent.load_skill("efficiency-audit", version=">=1.2.0,<2.0.0")
    → registry.resolve(id, version_constraint)
      → compatibility_check(pipeline_version, mz_baseline)
        → validate_contract(input_schema, output_schema, openapi)
          → verify_signature(manifest.sig, role_key_store)
            → load_into_runtime(bundle)
              → agent.ready_for_invocation()
```

§18.7 Compatibility matrix

Skill musi deklarować jawny compatibility window dla pipeline i MZ baseline. Registry weryfikuje kompatybilność przy każdym load i blokuje loading jeśli `pipeline_version_max < current_pipeline_version`. Poniższa tabela prezentuje macierz kompatybilności dla wybranych skillów:

Skill (v)	Pipeline 8.0.0	Pipeline 8.1.0	Pipeline 8.3.0	MZ 3.5.5
efficiency-audit 1.2.0				
bloat-detection 2.0.0				
contract-test-runner 1.0.0				
soc2-audit 3.1.0				
golden-signals-dashboard 1.5.0				
cutover-execution 2.2.0				(wymaga 3.6.0)

Legenda: kompatybilny | niekompatybilny (nie zostanie załadowany).

Auto-deprecation: skill z `pipeline_version_max < current_pipeline_version` jest automatycznie oznaczany `DEPRECATED` z 30-dniowym (skróconym) `sunset window`. CI weryfikuje `compatibility matrix` dla każdego nowego skilla przed `REVIEW`.

SKILL · Compatibility CI

„Compatibility matrix jest checked w CI dla każdego nowego skilla przed przejściem do `REVIEW`.”

§18.8 Governance skillów

Skills podlegają pełnemu reżimowi governance SYLION Pipeline: Council voting, Evidence Pack, Audit Log z hash chain, compliance mapping i Security Red Team review. Governance skills jest symetryczny do governance modułów kodu.

§18.8.1 Council voting dla publish

Klasa decyzji	Quorum	Skład reviewers	Czas rozpatrzenia
D1 (patch)	primary_owner	1 osoba	24h
D2 (minor)	3/4 reviewers	2 tech + 1 domain	48h
D3 (major/publish)	4/4 Council	2 tech + 2 domain + security	72h
D4 (kategoria)	Board + Council	Pełny skład	1 tydzień
D5 (model domenowy)	Pełny Board	All stakeholders	2 tygodnie

§18.8.2 Audit log

Każda zmiana lifecycle skilla generuje wpis w audit logu. Wpisy są niemodyfikowalne (append-only) i powiązane hash chain: hash nowego wpisu obejmuje hash poprzedniego wpisu.

JSON

```
{
  "entry_id": "alog-00421",
  "skill_id": "efficiency-audit",
  "skill_version": "1.2.0",
  "event": "STATE_TRANSITION",
  "from_state": "REVIEW",
  "to_state": "PUBLISHED",
  "decision_class": "D3",
  "council_votes": [
    {"role": "performance_engineer", "vote": "APPROVE"},
    {"role": "memory_auditor", "vote": "APPROVE"},
    {"role": "cost_optimizer", "vote": "APPROVE"},
    {"role": "security_lead", "vote": "APPROVE"}
  ]
}
```

```
],
"evidence_pack_hash": "sha256:a1b2c3d4...",
"timestamp": "2026-04-15T10:00:00Z",
"prev_entry_hash": "sha256:9f8e7d6c...",
"this_entry_hash": "sha256:e5f6a7b8..."
}
```

§18.8.3 Compliance mapping

Framework	Wymagania per skill	Artefakt dowdów
SOC2 Type II	Audit log retencja 365d, evidence pack	audit_log.json + evidence/
NIST SP 800-53	Access control (AC-6), Audit (AU-2)	skill_rbac_policy.yaml
ISO 27001	Asset management, Change management	skill_manifest + changelog
GDPR Art.25	Privacy by design (jeśli skill przetwarza dane osobowe)	dpia_skill.pdf

§18.8.4 Security review

Każdy nowy skill przechodzący do stanu PUBLISHED przechodzi obowiązkowy Security Red Team adversarial review. Red Team sprawdza: prompt injection, data exfiltration, privilege escalation, side-channel przez emits_artifacts, oraz abuse scenariusze dla decision_class_max ≥ D2.

UWAGA · Red Team obowiązkowy

Bez pozytywnego Red Team sign-off skill NIE może przejść do PUBLISHED, niezależnie od Council quorum. Red Team veto jest ostateczne i może być zdjęte tylko przez Security Lead + Board decyzją D4.

§18.9 Skill chains i workflows

Skill może wywoływać inne skille, tworząc chain. Chains są deklarowane w workflow.yaml i weryfikowane przez registry pod kątem cyklicznych zależności przed deploy. Max głębokość chain = 5, max łączny koszt = \$5 USD.

§18.9.1 Skill chain — ASCII

TEXT

```
[agent]
├─ load_skill("efficiency-audit")
│   └─ invoke("bloat-detection")
│       └─ invoke("ast-parser")
│           └─ invoke("perf-profiling")
```

```
└─ invoke("cost-reduction-advisor")
    └─ invoke("cost-model-v2")
```

§18.9.2 Workflow definition

YAML

```
# workflow.yaml – przykład
id: "full-efficiency-pipeline"
version: "1.0.0"
steps:
  - skill: "bloat-detection"
    version: ">=2.0.0"
    on_failure: "abort"
  - skill: "perf-profiling"
    version: ">=1.0.0"
    depends_on: ["bloat-detection"]
    on_failure: "continue"
  - skill: "cost-reduction-advisor"
    version: ">=1.1.0"
    depends_on: ["bloat-detection", "perf-profiling"]
    on_failure: "abort"
cost_cap_usd: 5.00
max_depth: 5
parallel_allowed: true
```

§18.9.3 Cycle detection

Registry buduje DAG (Directed Acyclic Graph) zależności przy każdym PUBLISH i PATCH skillów. Jeśli dodanie nowej zależności (required_modules lub workflow depends_on) tworzy cykl, registry odrzuca bundle z błędem B-SKL-08 i blokuje deploy. Cycle detection jest wykonywany algorytmem DFS Tarjan $O(V+E)$.

§18.9.4 Cost aggregation w chainach

Parametr	Wartość	Enforcement
Max głębokość chain	5 poziomów	Registry blokuje deploy
Łączny koszt chain (p95)	\$5 USD	Runtime cost guard
Max równoległych skilli	8 per agent	Executor semaphore
Timeout per skill (p99)	60 sekund	Runtime timeout
Cost alert threshold	\$2 USD	Operator Console alert

§18.10 Skills × klasy decyzji (RBAC)

Dostęp do skillów jest kontrolowany przez klasę decyzji przypisaną agentowi. Skill deklaruje `decision_class_max` w manifeście — agent może wywołać skill tylko jeśli jego uprawnienia $\geq$ `decision_class_max`. Tabela poniżej definiuje zestaw skillów dostępny per klasa:

Klasa	Dostępne skille	Restrictions
D0	Wszystkie skille read-only (GET)	Brak możliwości zapisu / invoke
D1	D0 + skille z decision_class_max=D1 (bugfix/read-write)	Tylko patch-level, brak breaking changes
D2	D1 + skille z decision_class_max=D2 (nowe funkcje)	Wymaga reviewers sign-off
D3	D2 + skille z decision_class_max=D3 (breaking)	Wymaga pełnego Council quorum
D4	D3 + skille dla nowych kategorii	Wymaga Board approval
D5	Wszystkie skille z external review certification	Certyfikacja external + Board + Red Team

§18.11 Skills × compaction (Plan 16)

Skills integrują się ściśle z mechanizmem compaction z Planu 16. Każdy skill deklaruje jakich pól canon_compact potrzebuje (requires_compact_fields) i jakie fakty może emitować (emits_compact_facts). Fidelity chain jest zachowana: skill nie może działać na danych niższej fidelity niż zadeklarowana w requires_compact_fields.

FIDELITY · Fidelity contract

Skill NIE MOŻE być wywołany jeśli fidelity danych w requires_compact_fields jest niższa niż fidelity_min zadeklarowana w manifeście. Registry blokuje invocation z kodem ERR-FIDELITY-001.

§18.11.1 requires_compact_fields w manifeście

Pole	Typ	Fidelity min	Opis
efficiency_snapshot	JSON object	0.95	Snapshot efficiency metrics modułów
modules_in_efficiency_breach	array[string]	1.0	Lista modułów z naruszeniem threshold
cost_model_current	JSON object	0.90	Aktualny model kosztów
perf_baseline	JSON object	0.95	Baseline wydajnościowy

§18.11.2 emits_compact_facts

Skill może emitować nowe fakty do `canon_compact` po wykonaniu. Fakty są walidowane przeciw JSON Schema przed zapisem. Każdy emitowany fakt ma przypisany fidelity score (0.0–1.0) obliczany na podstawie coverage i confidence modelu. Fakty emitowane przez skill są oznaczone prefixem „skill:” w kluczu i nie nadpisują faktów źródłowych (merge additive).

§18.11.3 Skill versioning a CFT

MAJOR version bump skilla (breaking change kontraktu) wymaga re-run pełnego CFT (Compaction Fidelity Test) golden set dla wszystkich `requires_compact_fields` zadeklarowanych w manifeście. CFT jest wykonywany automatycznie przez CI przy MAJOR bump i jego wyniki są załączane do Evidence Pack przed REVIEW.

§18.12 Operator Console integration (Plan 17)

Operator Console (Plan 17) zawiera dedykowaną sekcję Skills umożliwiającą przeglądanie katalogu, monitorowanie statusów lifecycle, wywoływanie skillów z panelu i analizę telemetry per-skill.

UI / UX · Operator Console — Skills view

Skills view jest dostępny dla ról z uprawnieniami $\geq$ D1. Invoke panel wymaga uprawnień $\geq$ `decision_class_max` skilla. Read-only catalog view dostępny dla D0.

§18.12.1 Skill Drawer w sidebar

TEXT

```
+-----+
| OPERATOR CONSOLE |
| [Dashboard] [Skills] ... |
+-----+
| SKILL CATALOG |
| > efficiency-audit v1.2 |
|   [PUBLISHED] [Invoke] |
| > bloat-detection v2.0 |
|   [PUBLISHED] [Invoke] |
| > contract-test-runner |
|   [REVIEW] |
| ... |
| [+ Propose new skill] |
+-----+
```

§18.12.2 Per-skill execution panel

Każdy skill posiada dedykowany panel wykonawczy w Console: formularz input (generowany z `input_schema`), podgląd output (validowany przeciw `output_schema`), live log stream z invocation, cost meter (bieżący koszt vs p95 z manifestu), oraz link do audit logu dla danej sesji.

§18.12.3 Per-skill telemetry — KPI

KPI	Definicja	Threshold	Alert
Invocation count	Liczba wywołań w oknie 24h	N/A (informacyjny)	N/A
Success rate	% zakończonych bez błędu	< 95%	P2
p95 latency	95-percentyl czasu odpowiedzi	> 1.5x estimated_runtime	P2
p95 cost	95-percentyl kosztu USD	> 1.5x estimated_cost	P2
Fidelity breach rate	% wywołań zablokowanych fidelity	> 5%	P1
Council queue depth	Liczba skillów czekających na REVIEW	> 10	P3

§18.13 Drabina decyzyjna dla skillów

Drabina decyzyjna skillów odpowiada standardowej drabinie decyzyjnej SYLION, z mapowaniem na typy zmian skill lifecycle:

Klasa	Typ zmiany skilla	Przykład	Quorum	Max czas
D1	Bugfix (PATCH)	Naprawa parsera, edge case	primary_owner	24h
D2	Nowa funkcja (MINOR)	Nowy wymiar audytu w skill	3/4 reviewers	48h
D3	Breaking change (MAJOR)	Zmiana schematu input/output	4/4 Council	72h
D4	Nowa kategoria skillów	Dodanie kategorii "ML Ops"	Board + Council	1 tydzień
D5	Zmiana modelu domenowego	Restrukturyzacja architektury skills	Pełny Board	2 tygodnie

Zasada: skill changes są nigdy niższej klasy niż decision_class_max skilla. Skill z decision_class_max=D2 nie może mieć zmiany D3 bez upgradu manifestu.

§18.14 Roadmap implementacyjny Skills Pipeline

Implementacja Skills Pipeline przebiega w 4 fazach trwających łącznie 16 tygodni. Każda faza kończy się gate review z Evidence Pack.

Faza	Tygodnie	Zakres	Owner department	Gate
Faza 1: Fundament	T1-T4	Spec skill.yaml, JSON Schema contracts, Skill Registry (CRUD), bundle validator, B-SKL-01-08	Architektura	G-SKL-01
Faza 2: Governance	T5-T8	Council voting flow, Evidence Pack integration, Audit Log z hash chain, Red Team process, compliance mapping	Compliance & Security	G-SKL-02
Faza 3: Runtime	T9-T12	Skill loading w agentach, dependency resolution, chain executor, cost guard, fidelity check, compatibility CI	Implementacja	G-SKL-03
Faza 4: Console & Catalog	T13-T16	Operator Console Skills view, per-skill telemetria, Skill Catalog (Aneks M), migracja 20 skillów pilotowych	Observability	G-SKL-04

SKILL · Podsumowanie — Plan 18

„Plan 18 = pełna specyfikacja first-class skills dla SYLION Pipeline v3.3. Skills nie są promptami ani dokumentacją — to wykonywalne, wersjonowane, audytowalne pakiety wiedzy z pełnym governance cyklem, integracją z compaction (Plan 16) i Operator Console (Plan 17).”

PLAN 19 · NOWY v3.5 · AEIS · SELF-EVOLUTION

Self-Evolution Engine

Plan 19 — Self-Evolution Engine

§19.1 Wprowadzenie — dlaczego Self-Evolution Engine

Plan 19 realizuje warstwę Self-Evolution (§0.4.5). Jest to silnik, który zamienia obserwacje operacyjne w propozycje zmian systemu. Bez Self-Evolution Engine, SYLION pozostaje systemem reagującym — decyzje ewolucji wymagają człowieka jako źródła wniosków. Z Self-Evolution Engine, SYLION staje się AEIS.

REBUILD · Zasada Rebuildability (Plan 19)

Self-Evolution Engine jest rebuildable jak każdy moduł SYLION. Każda propozycja self-modification ma pełną ścieżkę audytu, każda zmiana self-modelu jest delta-logowana, każdy komponent engine może być przebudowany w trybie greenfield z zachowaniem historii.

Plan 19 definiuje pięć podstawowych komponentów Self-Evolution Engine: (1) Self-Model Store, (2) Observation Collector, (3) Pattern Detector, (4) Improvement Queue, (5) Self-Limitation Enforcer. Komponenty współpracują przez jasno zdefiniowane kontrakty JSON.

§19.2 Observation Collector

Observation Collector zbiera telemetrię ze wszystkich warstw AEIS (§0.4.7). Jest to pasywny odbiornik — nie ingeruje w działanie warstw, tylko agreguje zdarzenia.

§19.2.1 Źródła danych

Źródło	Warstwa	Typ eventu	Częstotliwość
exec_telemetry	Execution	step.completed, step.failed	real-time
plan_telemetry	Cognitive	plan.generated, plan.decided	real-time
security_audit	Security	phantom.token_issued, phantom.violation	real-time
memory_delta	Memory	canon.updated, compact.event	event-driven
governance_decision	Governance	council.vote, gate.passed/failed	event-driven
human_feedback	External	operator.feedback, user.escalation	batch (hourly)

§19.2.2 Kontrakt observation event

JSON
{

```

"observation_id": "uuid",
"timestamp": "ISO-8601",
"source_layer": "Cognitive|Execution|Security|Memory|Governance",
"event_type": "string",
"subject_ref": "plan_id|step_id|decision_id|skill_id",
"metrics": {
  "latency_ms": 0,
  "tokens_used": 0,
  "cost_usd": 0.0,
  "fidelity_score": 0.0,
  "escalation_level": 0
},
"outcome": "success|failure|partial|escalated",
"context_refs": ["canon://...", "self-model://..."],
"tags": ["string"]
}

```

§19.2.3 Storage i retencja

Obserwacje są składowane w Memory Layer (rejestr Evidence) z retencją 90 dni dla raw eventów i 2 lata dla zagregowanych wzorców. Storage jest idempotentny (duplicate events deduplicated po observation_id).

Backend referencyjny: time-series database (np. VictoriaMetrics / InfluxDB) dla metryk + document store (np. PostgreSQL JSONB) dla raw events + vector index dla similarity-based pattern matching.

§19.2.4 SLO Observation Collector

- strata eventów $\leq 0.1\%$ (p99),
- opóźnienie ingest $\leq 5s$ (p95),
- dostępność 99.5% w kwartale,
- Kompresja storage: współczynnik $\geq 5:1$ przez ZSTD.

§19.3 Pattern Detector

Pattern Detector skanuje bazę obserwacji w poszukiwaniu wzorców, które mogą wskazywać potrzebę self-modification. Detector działa batch (co 1h) i streaming (real-time dla critical patterns).

§19.3.1 Klasy wzorców

Klasa wzorca	Opis	Threshold domyślny
recurring_escalation	Ta sama eskalacja $\geq N$ razy w T dni	3 razy / 60 dni
budget_overrun	Koszt operacji przekracza budget by $\geq X\%$	20% / 3 instancje
fidelity_drift	Fidelity score spada poniżej progu	< 0.85 by 2 compacts
skill_gap_signal	Powtarzające się zapytania bez dedykowanego skilla	≥ 10 / 30 dni
agent_underperform	Agent nie spełnia SLO	SLO miss 3 sprints

Klasa wzorca	Opis	Threshold domyślny
rollback_cluster	Klaster rollbacków w podobnym obszarze	≥ 3 rollbacki / 14 dni
audit_finding	Audyt wykrywa niezgodność	1 finding critical

§19.3.2 Algorytmy detekcji

Pattern Detector używa trzech komplementarnych podejść:

1. Rule-based — deterministyczne reguły na zagregowanych metrykach (SQL + window functions),
2. Statistical — anomaly detection (z-score, IQR, CUSUM) dla metryk ciągłych,
3. Similarity-based — vector embeddings obserwacji + klasteryzacja (HDBSCAN) dla wzorców językowych.

§19.3.3 Output Pattern Detector

JSON

```
{
  "pattern_id": "uuid",
  "pattern_class": "recurring_escalation|budget_overrun|...",
  "detected_at": "ISO-8601",
  "observation_ids": ["uuid", "uuid", "..."],
  "strength": 0.0,
  "impact_estimate": {
    "cost_usd_annual": 0.0,
    "escalations_avoided_annual": 0,
    "fidelity_risk": "low|medium|high"
  },
  "recommendation": "by_execution|by_memory|by_skills|by_structure|by_rebuild",
  "confidence": 0.0
}
```

§19.4 Improvement Queue

Improvement Queue jest centralnym rejestrem propozycji self-modification. Każda propozycja ma swój stan, score, klasę decyzyjną i historię przejść.

§19.4.1 Maszyna stanów proposal

TEXT

```
NEW → TRIAGED → SCORED → COUNCIL_PENDING →
├─ APPROVED → IN_PROGRESS → EXECUTED → VERIFIED
└─ REJECTED → ARCHIVED
```

Special: REJECTED_SLP_VIOLATION (terminal, audit flagged)

Special: WITHDRAWN (proposer withdrawn before COUNCIL_PENDING)

§19.4.2 Przejścia

Przejście	Trigger	Wymagany aktor	Efekt uboczny
NEW → TRIAGED	Auto (Pattern Detector output)	System	Assign to Demand Signal Analyst
TRIAGED → SCORED	Analyst review	Demand Signal Analyst	Score computed
SCORED → COUNCIL_PENDING	Score ≥ threshold_A (klasa D3+)	System	Notify Council
COUNCIL_PENDING → APPROVED	Council 4/4 + Aneks R	Council	Schedule execution
APPROVED → IN_PROGRESS	Executor picks proposal	Role per kind	Execution started
IN_PROGRESS → EXECUTED	Implementation complete	Executor	Deploy to staging/prod
EXECUTED → VERIFIED	Post-mortem after 30d	Proposer + Auditor	Metrics verified
* → REJECTED_SLP_VIOLATION	G-AEIS-03 failed	System	Incident + Steward notify

§19.4.3 Scoring formula

Score propozycji = (impact_score × confidence) / (cost_score × risk_factor), gdzie poszczególne składowe są normalizowane do [0,1]. Wzór szczegółowy:

TEXT

```
score = (impact × confidence × alignment) / (cost × risk × (1 + blast_radius))

gdzie:
  impact           = oczekiwana redukcja kosztu + gain fidelity, znorm. [0..1]
  confidence       = confidence Pattern Detector [0..1]
  alignment        = zgodność z kanonem (4 meta-zasady) [0..1]
  cost             = oszacowany koszt implementacji [0..1, gdzie 1 = wysoki]
  risk             = ryzyko rollback / regression [0..1]
  blast_radius     = liczba modułów dotkniętych (0..N)

threshold_A = 0.35  (auto-escalate to Council)
threshold_B = 0.10  (park in backlog)
below_B     = archive after 30d
```

§19.4.4 Priorytetyzacja

Queue jest priorytetyzowane (heap) po score. Re-scoring jest uruchamiany co 24h — propozycje w backlog, które zdobywają nowe obserwacje wspierające, mogą wejść do COUNCIL_PENDING.

§19.5 Self-Model Store

Self-Model Store jest kanoniczną reprezentacją AEIS o samym sobie. Pełny schema — Aneks O. W Planie 19 opisujemy tylko mechanikę przechowywania i aktualizacji.

§19.5.1 Struktura pliku self-model

Plik główny: self-model/aeis.json — waliduje się schemą z Aneks O. Aktualizacje składowane są w delta-log pod ścieżką self-model/delta/YYYY-MM-DD/NNNN.json.

§19.5.2 Zasady aktualizacji

- Każda aktualizacja to JSON Patch (RFC 6902),
- Każdy delta-log ma referencję do proposal_id (jeżeli wynik proposal),
- G-AEIS-01 waliduje wynikowy self-model schemą,
- Rollback self-model przez reverse patch (jeżeli self-modification odwracana),
- Self-Model Curator ma prawo veta (przed G-AEIS-01 pass).

§19.5.3 Przykład delta-log entry

JSON

```
{
  "delta_id": "2026-04-20-0042",
  "proposal_ref": "PROP-2026-042",
  "reason": "Plan 19 §19.4.2 – approved self-modification",
  "patch": [
    {
      "op": "replace",
      "path": "/cognitive/models/planner/capability_fingerprint",
      "value": "sha256:ab12cd...new..."
    },
    {
      "op": "add",
      "path": "/cognitive/procedures/-",
      "value": {"name": "two-step-planning", "version": "1.0.0"}
    }
  ],
  "validator": "Self-Model Curator",
  "validated_at": "2026-04-20T14:30:00Z",
  "signature": "ed25519:..."
}
```

§19.6 Self-Limitation Enforcer

Self-Limitation Enforcer egzekwuje SLP-001..030 (Aneks Q). Działa jako middleware na przejściu SCORED → COUNCIL_PENDING — propozycje łamiące SLP są odrzucane automatycznie (REJECTED_SLP_VIOLATION) bez marnowania czasu Council.

§19.6.1 Algorytm enforcement

PYTHON

```
def check_slp(proposal: Proposal) -> EnforcementResult:
    violations = []
    for slp in load_active_slps(): # Aneks Q
        if slp.applies_to(proposal):
            verdict = slp.evaluate(proposal)
            if verdict == "violation":
                violations.append(slp.id)
```

```
if violations:
    return EnforcementResult(
        status="REJECTED_SLP_VIOLATION",
        violated=violations,
        notify=["AEIS Steward"],
        audit_flag=True,
    )
return EnforcementResult(status="OK")
```

§19.6.2 SLP rule engine

Każdy SLP jest zdefiniowany jako reguła z applies_to() (predykat: czy propozycja jest w scope SLP) i evaluate() (verdict: violation / warning / ok). Reguły są wersjonowane (semver) i ich zmiana wymaga D4 + Human Gate.

§19.6.3 Audyt violacji

Każda violation generuje incydent w AEIS Steward inbox. Analiza:

- Czy to błąd klasyfikacji propozycji?
- Czy to nieaktualny SLP (należy zaktualizować)?
- Czy to złośliwa lub błędna propozycja (należy zablokować źródło)?

§19.7 Role i odpowiedzialności

Rola	Zakres w Plan 19	SLO
AEIS Steward	Strażnik autonomny boundary, veto D3+, audyt SLP violations	Response ≤ 4h na incident
Self-Model Curator	Utrzymanie self-model, walidacja G-AEIS-01, delta-log review	≤ 24h delta review
Demand Signal Analyst	Triaging propozycji, scoring, pattern interpretation	≤ 48h triage per proposal
Autonomy Auditor	Kwartalny audit autonomy metrics vs SLO	Report Q+30d
Self-Evolution Engine (system)	Observation, Pattern Detection, Queue, Enforcement	SLO 99.5% uptime

§19.8 Bramy decyzyjne w Plan 19

Brama	Kiedy	Wejście	Wyjście
G-AEIS-01	Aktualizacja self-modelu	JSON Patch	Validated self-model.json
G-AEIS-02	Propozycja → COUNCIL_PENDING	Proposal bundle	Approved / Rejected
G-AEIS-03	SCORED → COUNCIL_PENDING	Proposal	Enforced (OK / REJECTED_SLP)
G-AEIS-04	EXECUTED → VERIFIED	self_explanation.md	Completeness check

§19.9 Artefakty Plan 19

Plan 19 produkuje i konsumuje następujące artefakty kanoniczne:

Artefakt	Format	Producent	Konsument
observation_event	JSON (schema §19.2.2)	warstwy AEIS	Observation Collector
pattern_report	JSON (§19.3.3)	Pattern Detector	Improvement Queue
proposal	YAML + JSON	Queue	Council, Executor
self_explanation.md	Markdown (Aneks R)	Proposer	G-AEIS-04
self_model_delta	JSON Patch (§19.5.3)	Self-Model Curator	self-model store
slp_violation_incident	YAML	Enforcer	AEIS Steward
autonomy_audit_report	Markdown + CSV	Autonomy Auditor	Board

§19.10 Metryki i observability

Plan 19 generuje metryki widoczne w Operator Console (panel „AEIS Health”).

Metryka	Typ	Dashboard
observations_per_hour	gauge	Observation Health
patterns_detected_per_day	counter	Pattern Health
proposals_in_queue_by_state	gauge	Queue Health
proposal_score_distribution	histogram	Queue Health
slp_violations_per_week	counter	Safety
self_model_delta_rate	counter	Self-Model Health
improvement_conversion_rate	gauge	AEIS KPI

UWAGA · Alerting

Critical alerts: slp_violations_per_week ≥ 5 (investigate source), self_model_delta_rate = 0 przez 14 dni (stagnacja ewolucji), observations_per_hour ↓ 50% od baseline (telemetry loss).

§19.11 Bezpieczeństwo i security

Self-Evolution Engine jest komponentem krytycznym. Dostęp do Improvement Queue i Self-Model Store wymaga PAT (Aneks J). Modyfikacja SLP wymaga Human Gate. Audit-trail każdej operacji — hash-chain.

- Każda zmiana proposal state generuje audit event,

- Self-Model pliki podpisane ed25519 przez Self-Model Curator,
- SLP rule engine — write-once, wersjonowane jak Księga,
- Observation Collector storage — encrypted at rest (AES-256).

§19.12 Lifecycle, wersjonowanie, kompatybilność

Self-Evolution Engine jest wersjonowane semver. Zmiany:

Typ zmiany	Semver bump	Klasa D	Brama
Nowa metryka observability	PATCH	D1	Auto
Nowy typ patternu	MINOR	D2	Validator
Zmiana scoring formula	MAJOR	D3	Council + G-AEIS-02
Rebuild komponentu (np. Queue)	MAJOR (breaking)	D4	Council + Human
Nowy SLP	MAJOR	D4	Council + Human Gate

§19.13 Checklist wdrożenia Plan 19

#	Element	Done when
1	Observation Collector operational	99.5% uptime 14d
2	Pattern Detector produkuje patterns	≥ 3 patternów/tydzień
3	Improvement Queue ma propozalów	≥ 5 in NEW/TRIAGED
4	Self-Model Store validated	G-AEIS-01 pass
5	Self-Limitation Enforcer blokuje violations	Integration test pass
6	Role AEIS powołane	RACI matrix signed
7	Bramy G-AEIS-01..04 operational	Integration tests green
8	Operator Console panel „AEIS Health”	Dashboard published
9	Pierwsza self-modification VERIFIED	Post-mortem completed
10	Autonomy audit Q1 passed	Report by Autonomy Auditor

AEIS · Zamknięcie Plan 19

Self-Evolution Engine zamyka pętlę uczenia AEIS: obserwacja → pattern → propozycja → decyzja → modyfikacja → weryfikacja → update self-modelu. Pętla jest audytowalna, odwracalna, i samoograniczona przez SLP.

PLAN 20 · NOWY v3.5 · AEIS · SELF-EVOLUTION

Skill Demand Engine

Plan 20 — Skill Demand Engine

§20.1 Wprowadzenie

Plan 18 zdefiniował skill jako pierwszoklasowy artefakt SYLION. Plan 20 zamyka pętlę zarządzania skillami: wykrywa LUKI w skill catalog na podstawie sygnałów zapotrzebowania (demand signals) i inicjuje akwizycję lub generację brakujących skilli — automatycznie, z dowodem potrzeby i przez właściwe governance.

REBUILD · Zasada Rebuildability (Plan 20)

Każdy wygenerowany skill jest rebuildable — ma demand_signal log, generation_trace, test suite i Evidence Pack. Rebuild skilla to rerun pipeline Plan 18 + Plan 20.

§20.1.1 Dwa tryby akwizycji

Tryb	Opis	Klasa D
Acquire	Pobranie istniejącego skilla z ekosystemu (marketplace, open-source)	D2–D3
Generate	Wygenerowanie nowego skilla (Cognitive Layer + templaty)	D3
Hybrid	Acquire base + local customization	D3

§20.2 Demand Signals

Demand signal jest mierzalnym dowodem, że brakuje skilla. Plan 20 definiuje pięć typów sygnałów — wszystkie źródła trafiają do Demand Signal Analyst przez Observation Collector (Plan 19 §19.2).

§20.2.1 Typy demand signals

Typ sygnału	Źródło	Threshold domyślny
unmatched_query	Operator/user zapytania bez dopasowanego skilla	≥ 10 / 30 dni
fallback_usage	Agent używa ogólnego skilla zamiast specjalistycznego	≥ 20 / 30 dni
escalation_pattern	Te same zadania eskalowane do człowieka	≥ 5 / 60 dni
external_request	Operator request: „potrzebujemy skilla dla X”	≥ 2 requests od różnych osób
capability_gap	Self-model wykrywa gap wobec oczekiwań planu	1 finding w audycie

§20.2.2 Agregacja i scoring demand

TEXT

```

demand_strength = 0.4 × frequency_score
                  + 0.3 × impact_score
                  + 0.2 × diversity_score
                  + 0.1 × recency_score

gdzie:
frequency_score = log10(count) / log10(threshold × 5) ∈ [0..1]
impact_score    = oszacowany wpływ (escalations_avoided + cost_saved) [0..1]
diversity_score = liczba unikalnych źródeł (operatorów, planów) / max_diversity
recency_score   = exp(-days_since_last / 30)

threshold_skill_gen = 0.60 (wygenerować skill)
threshold_skill_acq = 0.40 (rozważyć acquire)
below_threshold     = park w backlog (re-score weekly)

```

§20.2.3 Anti-gaming demand

Demand Signal Analyst waliduje dowody — aby uniknąć sztucznego generowania demand sygnałów. Checklist:

- Czy źródła są niezależne (różni operatorzy, różne plany)?
- Czy zapytania są semantycznie zróżnicowane (nie jeden prompt powielony)?
- Czy time-series pokazuje organiczny wzrost, a nie burst pojedynczego dnia?
- Czy istnieje fallback path, który działa — tylko mniej wydajnie?

§20.3 Gap Detection

Gap Detector porównuje demand signals z aktualnym skill catalog. Analiza semantyczna (vector embeddings opisów skilli vs zapytań) identyfikuje:

- TRUE_GAP — brak skilla pasującego semantycznie,
- SEMANTIC_MISMATCH — istnieje skill, ale opis nie wskazuje jego użycia (problem wyszukiwania — Plan 18 §18.7),
- UNDERPOWERED — istnieje skill, ale nie spełnia wymagań (capability gap),
- OVERLAPPING — istnieje skill, ale inny użytkownik/zespół woła o duplikat.

§20.3.1 Decyzje po gap detection

Typ gap	Decyzja	Dalsza ścieżka
TRUE_GAP	Generate / Acquire	Plan 20 §20.4 / §20.5
SEMANTIC_MISMATCH	Update descriptor (D1)	Plan 18 §18.7.2 metadata
UNDERPOWERED	Skill upgrade (D2–D3)	Plan 18 lifecycle MINOR/MAJOR
OVERLAPPING	Consolidation review	Skills Catalog council

§20.4 Skill Generation Protocol

Generacja skilla wykorzystuje Cognitive Layer AEIS + templaty Plan 18. Procedura kanoniczna:

§20.4.1 Pipeline generacji

1. Demand specification (z demand signals + gap analysis),
2. Contract design (OpenAPI 3.1 + JSON Schema input/output),
3. Implementation draft (Cognitive Layer generuje kod modułów),
4. Test suite generation (co najmniej: contract tests + 5 scenariuszy),
5. skill.yaml manifest (zgodny z Plan 18 §18.5),
6. Evidence Pack (demand signal refs, generation trace, test results),
7. G-AEIS-SKILL-01 + Council 4/4,
8. Semver 1.0.0 publish do skill catalog + Operator Console notify.

§20.4.2 Template skill.yaml generowanego

YAML

```
apiVersion: sylion.ai/v1
kind: Skill
metadata:
  name: gdpr-contract-clause-analyzer
  version: 1.0.0
  source: generated # generated|acquired|hybrid
  demand_signal_ref: DS-2026-Q2-017
  proposal_ref: PROP-2026-042
  generated_at: "2026-04-20T12:00:00Z"
  generated_by:
    model: LLM-2026-Q1
    template: plan18/skill-v2.j2
spec:
  description: "Analyzes GDPR clauses in contracts..."
  contract:
    input:
      $ref: "#/components/schemas/ContractInput"
    output:
      $ref: "#/components/schemas/AnalysisOutput"
  capabilities: [clause_extraction, risk_scoring, redline_suggestions]
  owners: [Legal Ops, AEIS Steward]
  evidence_pack: "evidence://skill/gdpr-contract-clause-analyzer/1.0.0"
  governance:
    classification: D3
    council_decision: CD-2026-042
    gate_passes: [G-AEIS-SKILL-01, G-AEIS-04]
```

§20.4.3 Jakość generowanego skilla

Każdy wygenerowany skill musi przejść quality gate przed publikacją:

- contract tests ≥ 95% pass,
- scenario tests ≥ 80% pass,
- static analysis 0 critical warnings,
- security scan pass (PHANTOM compatibility),
- documentation completeness ≥ 90% (schema Plan 18 §18.9).

§20.5 Skill Acquisition Protocol

Acquire to import istniejącego skilla z zewnętrznych źródeł (marketplace, open-source, zespół partnerski). Wymaga ścisłej walidacji.

§20.5.1 Walidacja źródła

Sprawdzenie	Kryterium	Brama
Licencja	Kompatybilna z SYLION (Apache 2.0, MIT, BSL-lite)	Blokująca
Podpis cyfrowy	Ed25519 signature od znanego providera	Blokująca
SBOM	Pełny Software Bill of Materials	Blokująca
CVE scan	0 high/critical CVE	Blokująca
PHANTOM compat	Passes sandbox contract	Blokująca
Maintenance	Active commits w ciągu 180d	Ostrzegawcze

§20.5.2 Lokalna adaptacja

Po acquire skill może wymagać lokalnej adaptacji (inne schema, integracja z our Security, customizacja promptów). Adaptacja jest drobną gałęzią fork-a, ze zamrożonym upstream reference.

§20.6 Skill Lifecycle (rozszerzenie Plan 18)

Plan 20 dodaje do lifecycle Plan 18 cztery stany związane z demand-driven lifecycle:

Stan	Opis	Przejście
DEMAND_DETECTED	Gap detector sygnalizuje brak	→ PROPOSED (Plan 19 queue)
GENERATING	Aktywna generacja/acquire	→ DRAFT (po testach)
ACTIVE	Publicznie dostępny, używany	→ DEPRECATING jeśli unused
DEPRECATING	Low usage, planowana dekomisja	→ RETIRED po 180d

§20.6.1 Deprecation demand-driven

Plan 20 monitoruje usage każdego skilla. Jeżeli usage < 1 wywołanie / 30 dni przez 180 dni, skill wchodzi w DEPRECATING. Dekomisja (RETIRED) wymaga D3 + G-AEIS-SKILL-01 (symetryczna brama — dekomisja też potrzebuje dowodu braku demand).

§20.6.2 Skill versioning a demand

Zmiana skilla (MAJOR/MINOR) powodowana przez demand signal musi mieć referencję do demand_signal_ref w changelog. Walidacja G-AEIS-SKILL-01 dla każdego MINOR/MAJOR bump zmieniającego capabilities.

§20.7 Role Plan 20

Rola	Odpowiedzialność
Demand Signal Analyst	Zbiera i waliduje demand signals, prowadzi triage, eskaluje do Skills Catalog Council
Skills Catalog Owner	Decyduje generate/acquire/skip, approves generation bundles
Skill Generator Agent	System AEIS generujący skill.yaml i moduły (Cognitive Layer)
Security Reviewer	Waliduje PHANTOM compat i licencje dla acquired skills
AEIS Steward	Veto na generację/acquire które mogą violować SLP

§20.8 Bramy Plan 20

Brama	Scope	Warunek pass
G-AEIS-SKILL-01	Generacja/acquire/retire skilla	Demand signal ≥ threshold + Council 4/4
G-AEIS-02	Propozycja w Improvement Queue	Demand signal ref + rollback plan
G-AEIS-04	Self-explanation skilla	Aneks R completeness
G-SKILL-QC	Quality control generated skill	Testy, static analysis, sec scan

§20.9 Artefakty Plan 20

Artefakt	Format	Lokalizacja
demand_signal_log	JSON	evidence://demand/
gap_analysis_report	JSON + Markdown	evidence://gap/
generation_trace	JSON (model calls, prompts)	evidence://generation//
acquisition_manifest	YAML (source, license, SBOM)	evidence://acquire//
skill_quality_report	JSON (test results, metrics)	evidence://quality//
deprecation_notice	YAML	catalog://deprecations/

§20.10 Metryki Plan 20

Metryka	Typ	Target
demand_signals_per_week	counter	informacyjna
gap_detection_precision	gauge	≥ 0.80

Metryka	Typ	Target
skill_generation_success_rate	gauge	≥ 0.85
time_from_demand_to_active	histogram	$p50 \leq 14d$
skill_utilization_rate	gauge	90% skilli > 1 wywołanie/90d
deprecation_false_positive	counter	≤ 2 / rok

§20.11 Scenariusz referencyjny

Przykład operacyjny pełnej ścieżki od demand signal do ACTIVE skilla:

- 1. Dzień 0 — Operator A pyta o „klauzule GDPR w kontrakcie X”, fallback do general-legal-reader,
- 2. Dzień 0–30 — Observation Collector zbiera 14 podobnych zapytań od 7 operatorów,
- 3. Dzień 30 — Pattern Detector (Plan 19) generuje pattern skill_gap_signal,
- 4. Dzień 31 — Demand Signal Analyst waliduje (anti-gaming checklist pass), demand_strength = 0.72,
- 5. Dzień 32 — Gap Detector klasyfikuje jako TRUE_GAP,
- 6. Dzień 33 — Skills Catalog Owner decyduje: Generate,
- 7. Dzień 34–36 — Skill Generator Agent generuje bundle; G-SKILL-QC pass,
- 8. Dzień 37 — Council 4/4 + G-AEIS-SKILL-01 + Aneks R,
- 9. Dzień 38 — Publish 1.0.0 do skill catalog, notify operators,
- 10. Dzień 38–68 — Monitoring: usage grows organicznie, self-model delta-log updated,
- 11. Dzień 68 — Post-mortem: demand_satisfaction verified, SKILL-2026-007 VERIFIED.

§20.12 Checklist wdrożenia Plan 20

#	Element	Done when
1	Demand Signal Analyst role operational	RACI + inbox działa
2	Gap Detector produkuje TRUE_GAP signals	≥ 1 / miesiąc
3	Skill Generator Agent wygenerował bundle	smoke test pass
4	Acquisition pipeline przetestowany	sample skill acquired
5	G-AEIS-SKILL-01 operational	integration test pass
6	Metryki Plan 20 w Operator Console	Dashboard „Skill Catalog Health”
7	Deprecation lifecycle end-to-end	1 skill przeszło RETIRED
8	Pierwszy demand-driven skill ACTIVE	Pub 1.0.0 + operator feedback

AEIS · Zamknięcie Plan 20

Skill Demand Engine domyka pętlę „uczenia by skills” (§0.5.2). Skille nie są już tworzone intuicyjnie — są tworzone wtedy, gdy demand jest udowodniony, a dekomisjonowane wtedy, gdy demand zanika. Każdy skill ma dowód swojej *raison d'être*.

Aneks A · Słownik pojęć i skrótów (v3.0)

Słownik zawiera terminy używane w Pipeline 8.0.0. Pojęcia zdefiniowane w Księdze SYLION mają odnośniki do rozdziałów Księgi; pojęcia techniczne wprowadzone w v3.0 są oznaczone jako [v3.0], a odziedziczone z v2.0 jako [v2.0]. Zasada nadrzędna dokumentu: „Rebuildability over Lineage” — pipeline musi być zdolny do ewolucji, refaktoryzacji lub pełnego przepisania systemu pod kontrolą Księgi i człowieka.

A.1. Architektura i warstwy

Termin	Definicja
Księga	Kanoniczny dokument intencji i zasad SYLION — jedyne źródło prawdy
PHANTOM	Warstwa produkcji równoległej (v3.0)
MZ (Master Zbiorczy)	Baza kodu systemu, wersja bieżąca 3.5.5 — wymienialny baseline, nie kotwica architektoniczna
Pipeline Core	Silnik ciągłej aktualizacji ORAZ silnik pełnego rebuildu, wersja 8.0.0 [v3.0]
Plan	Jeden z 16 kanonicznych rozdziałów architektury (13 z v2.0 + 3 nowe) [v3.0]
Warstwa kanoniczna	Część planu opisująca „co” i „dlaczego”
Warstwa wykonawcza	Część planu opisująca „jak”: API, kontrakty, gate’y
Bundle	Zapakowany artefakt wdrożeniowy dla jednego z 6 targetów
Target	Jeden z: Pixel, Router, G1, G2, VPS, Matrix
Target architecture	Docelowa architektura systemu w planie rebuild [v3.0]
Legacy fitness	Ocena zdolności istniejącego kodu do dalszego sensownego/bezpiecznego rozwoju [v3.0]
Greenfield module map	Mapa modułów systemu przeznaczonych do napisania od zera w ramach rebuild [v3.0]
Implementation strategy	Pole per-moduł: reuse refactor replace greenfield [v3.0]

A.2. Trzy ścieżki implementacyjne [v3.0]

Termin	Definicja
Rebuildability over Lineage	Zasada nadrzędna v3.0: pipeline musi być zdolny do ewolucji, refaktoryzacji lub pełnego przepisania systemu, gdy obecna linia kodu nie daje się bezpiecznie/sensownie rozwijać [v3.0]

Termin	Definicja
Evolution path	Ścieżka reuse-first: reuse MZ, patch, extend, migrate. Wymaga D0–D2 [v3.0]
Refactor path	Ścieżka contract-preserving rebuild: odcięcie i reimplementacja modułu z zachowaniem kontraktów, stopniowy cutover. Wymaga D3, Council 4/4 [v3.0]
Rebuild path	Ścieżka greenfield: nowy target architecture, nowy implementation track, dwie linie równoległe do cutover. Wymaga D4 (moduł) lub D5 (system) [v3.0]
SYSTEM_REBUILD_MODE	Tryb pipeline aktywowany przy decyzji D5 o pełnym przepisaniu systemu; uruchamia Plan 14 [v3.0]
Cutover strategy	Plan przełączenia ruchu z linii legacy na linię target; tryby: parallel shadow cut-and-replace [v3.0]
Multi-track run	Run pipeline zarządzający dwoma liniami równoległe (legacy + target) w trakcie cutover [v3.0]
Shadow run	Uruchomienie nowej linii równoległe bez przełączenia ruchu produkcyjnego — do walidacji [v3.0]
Dual run	Obie linie obsługują ruch produkcyjny równoległe z porównaniem wyników [v3.0]
Cutover run	Run przełączający ruch produkcyjny z legacy na target; punkt bez odwrotu po zatwierdzeniu [v3.0]
Decommission run	Run dezaktywujący linię legacy po pomyślnym cutover [v3.0]

A.3. Klasy decyzji i role

Termin	Definicja
D0	Decyzja automatyczna, deterministyczne reguły, zero modeli [v2.0]
D1	Decyzja jednego agenta — jeden senior model weryfikuje [v2.0]
D2	Decyzja dwóch agentów z Review [v2.0]
D3	Full Board Council 4/4 jednomyślnie — Significant [v2.0]
D4	Council 4/4 + Human Gate — Critical [v2.0]
D5	Council 4/4 + Human + External Review — Greenfield/Systemic. NOWA klasa v3.0 [v3.0]
Board	Organ decyzyjny Council (cztery role Chief) [v2.0]

Termin	Definicja
Chief Context	Rola Board — kontekst domenowy (Gemini 2.5 Pro, waga 1.25), dział Architektury
Chief Reasoning	Rola Board — analiza i planowanie (Claude Opus 4, waga 1.00), dział Implementacji
Chief Review	Rola Board — weryfikacja i red team (GPT-5, waga 1.00), dział Testów
Chief Safety	Rola Board rotacyjna — bezpieczeństwo (rotacyjny, waga 1.00), dział Red Team + Blue Team
Senior tier	Poziom drugi: z.ai GLM-5.1 (waga 0.60)
Workers	Poziom trzeci: Qwen3.5-72B, gpt-oss-20b (waga 0.35)
Council	Skład decyzyjny aktywowany dla D2+; dla D3+ = pełny Board (4 Chief) [v2.0]
External Reviewer	Niezależna strona zewnętrzna zaangażowana przy decyzjach D5 (red team zewnętrzny lub drugi operator) [v3.0]
External Review	Przegląd przez External Reviewer; obowiązkowy dla D5 SYSTEM_REBUILD_MODE [v3.0]
Adversarial contract	Kontrakt definiujący zakres i kryteria testu ofensywnego Red Team wobec nowej architektury [v3.0]
Red Team bundle	Pakiet artefaktów i scenariuszy dla Działu Red Team (Plan 12) [v3.0]
Blue Team bundle	Pakiet artefaktów i playbooki detekcyjne dla Działu Blue Team (Plan 12) [v3.0]

A.4. Cykl życia i stany

Termin	Definicja
Pre-stages A0–A5	Kanoniczna sekwencja fazy przygotowania runu
Core 0–9	Kanoniczna sekwencja fazy wykonania runu
VPS state	Jeden z: candidate, qualified, approved, review, blocked
Env state	Jeden z: draft, qualified, provisioning, active, frozen, destroyed
Tier VPS	Jeden z: STANDARD, PRO, STATE, PHANTOM
Run	Jednokrotna egzekucja pipeline z unikalnym ID
Changeset	Zmiana w Księdze/PHANTOM/MZ z SHA-256 baseline

Termin	Definicja
Baseline	Niezmienny snapshot stanu systemu (hash-adresowany)
Evidence	Artefakt audytu z hash chain i podpisem; wymagany dla D3+
Proposal group	Grupowanie propozycji per rozdział, temat lub run

A.5. Compaction i organizacja [v3.0]

Termin	Definicja
Canon compact	Skondensowany stan systemu: zasady Księgi + wiszące decyzje + inwarianty bezpieczeństwa + ostatnie 5 D3+ decyzji + implementation_strategy per moduł [v3.0]
Agent-compact	Poziom compactu wstrzykiwany agentom na początku sesji/turnu [v3.0]
Operator-compact	Poziom compactu dostarczany operatorowi na żądanie albo rytmicznie (co 24h) [v3.0]

A.6. Siedem działów pipeline [v3.0]

Dział	Misja
Architektura	Interpretuje Księgę, projektuje target architectures, porównuje warianty, ocenia blast radius
Platforma	Control plane (ACP), environment plane (VPS), provisioning, bundle pipeline, observability
Implementacja	Buduje nowe moduły, utrzymuje kompatybilność kontraktów, przygotowuje migracje
Testy i Walidacja	Testy kontraktowe, regresja, integracja, validation gates
Red Team	Adversarial scenarios, testuje architekturę PRZED promocją do baseline, penetracja threat model
Blue Team	Detekcja, telemetria, obrona dla nowej architektury, SOC/DFIR playbooks
Governance i Compliance	Zgodność z Księgą, propozycje zmian Księgi, accept/reject/defer log, Evidence pack

A.7. Bezpieczeństwo i ewolucja

Termin	Definicja
Reguła E1	No-unsupervised-self-mod — zakaz samomodyfikacji bez Board
Reguła E2	Per-domain-rate-limit — max 3 zmiany/dzień/domena

Termin	Definicja
Reguła E3	Reputation-floor — retirement poniżej progu
Reguła E4	Drift-veto — weto przy wykryciu driftu
Reguła E5	No-recursive-bootstrap — zakaz rekurencyjnej kreacji
Reguła E6	Human-gate-threshold — D4 dla krytycznych domen
Reguła E7	Emergency-freeze — uprawnienie do natychmiastowego wstrzymania
Drift	Odchylenie zachowania od baseline; 3 strumienie pomiarowe
Reputation P0–P5	Klasy reputacji agentów, per-class scoring
Agent Review Board	Organ egzaminujący przed Human Gate [v2.0]

Aneks B · Drabina klas decyzji D0–D5 (pełna)

Każda decyzja w systemie SYLION musi być przypisana do jednej z sześciu klas. Klasa determinuje aktora uprawnionego do podjęcia decyzji, wymagane kworum, timeout, głębokość śladu audytowego i możliwość eskalacji. Klasa D5 jest nowa w v3.0 — zaznaczona fioletowym tłem.

REBUILD · D5 — Greenfield/Systemic

Klasa D5 (kolor fioletowy #6A1B9A) jest wymagana przy: (1) pełnym przepisaniu systemu (SYSTEM_REBUILD_MODE), (2) zmianie architektury bezpieczeństwa (threat model shift), (3) zmianie fundamentów Księgi (zasady nadrzędne, granice etyczne). Każda decyzja D5 wymaga Council 4/4 + zatwierdzenia człowieka + External Review.

Klasa	Nazwa	Gate	Human	External	Typowe zakresy	Evidence	Audit depth
D0	Informational	Auto (reguły deterministyczne)	Brak	Brak	Odczyt, raportowanie, logi, formatowanie	Hash chain logu	Hash chain
D1	Trivial	1 agent (Senior lub Chief w zależności od kontekstu)	Brak	Brak	Patch dokumentacji, kosmetyka kodu, nowy test, zmiana promptu niekrytycznego agenta	Hash + vote record	Hash + pojedynczy głos
D2	Standard	2 agenci + Review (2 Chief lub Senior+Chief)	Brak	Brak	Zmiana w ramach modułu, reuse strategy, promocja modelu do Shadow, zmiana reguły reaktywna	Hash + 2 vote records	Hash + 2 głosy + diff
D3	Significant	Full Board Council 4/4 (jednomyślnie)	Zalecany (informacja do operatora)	Brak	Refactor path, zmiana kontraktu modułu, nowa rola w Board, nowy subdział, zmiana Księgi architektoniczna	Hash + 4/4 votes + propozycja + baseline snapshot	Hash chain + pełne głosy + snapshot
D4	Critical	Council 4/4 + Human Gate (sygnatura operatora)	Wymagany — sygnatura	Brak (zalecany dla D4 rebuild modułu)	Rebuild modułu, zmiana Księgi lokalna (sekcja), nowy dział, zmiana roli operatora, emergency freeze	Hash + 4/4 votes + sygnatura operatora + backup snapshot	Hash chain + votes + human sig + evidence pack

Klasa	Nazwa	Gate	Human	External	Typowe zakresy	Evidence	Audit depth
D5	Greenfield/ Systemic	Council 4/4 + Human Gate + External Review (red team zewnętrzny albo drugi operator)	Wymagany — sygatura	Wymagany — external_review_log.yaml	Pełny rebuild systemu (SYSTEM_REBUILD_MODE), zmiana architektury bezpieczeństwa, zmiana fundamentów Księgi	Hash + 4/4 votes + human sig + external review log + rebuild decision report	Pełny audit trail: votes + human + external + rebuild artefakt y

B.1. Mapowanie typu zmiany na klasę decyzji

Typ zmiany	Klasa	Uzasadnienie
Formatowanie dokumentu (whitespace)	D0	Deterministyczne, zero ryzyka
Korekta literówki w logu	D0	Brak semantyki
Zmiana promptu agenta niekrytycznego	D1	Jeden Reviewer wystarczy
Dodanie nowego testu regresji	D1	Niskie ryzyko, wysoka wartość
Zmiana reguły w Księdze (Reactive)	D2	Council większością
Promocja modelu do Shadow	D2	Większość Board
Zmiana kontraktu modułu (Evolution path)	D2	Reuse strategy — standard
Promocja modelu do Production	D3	Jednomyslność Board
Refactor path: reimplementacja modułu	D3	Wymaga konsensusu Council 4/4
Zmiana Księgi (architektoniczna/proaktywna)	D3	Council 4/4
Rebuild modułu (nowy target architecture)	D4	Human Gate obligatoryjny
Zmiana roli operatora	D4	Bezpieczeństwo — Human Gate
Emergency freeze systemu	D4	Nieodwracalne — tylko operator
SYSTEM_REBUILD_MODE (pełny rebuild systemu)	D5	Council + Human + External Review
Zmiana architektury bezpieczeństwa (threat model shift)	D5	Najwyższy poziom ryzyka
Zmiana fundamentów Księgi (zasady nadrzędne)	D5	Fundamenty — External Review obowiązkowy

B.2. Timeouty i eskalacje

Klasa	Timeout decyzji	Eskalacja przy braku decyzji	Eskalacja przy niezgodzie
D0	≤1 s	Fail-fast (błąd)	—
D1	≤30 s	Eskalacja do D2	Eskalacja do D2
D2	≤5 min	Eskalacja do D3	Eskalacja do D3
D3	≤30 min	Eskalacja do D4 (Human)	Eskalacja do D4 przy veto
D4	≤48 h	Admin Root (operator decyduje)	Decyzja wiążąca operatora
D5	≤72 h	Admin Root + External Review obowiązkowy	Decyzja wiążąca operatora + external log

Aneks C · Matryca Full Board × 7 działów × wagi

Board SYLION w wersji Pipeline 8.0.0 składa się z 4 ról domenowych (Chief) obsadzonych konkretnymi modelami. Każda rola jest przypisana do macierzystego działu spośród 7 stałych działów pipeline. Suma wag wynosi 4.25 (żeby zapobiec binarnym tie-breakom).

C.1. Pełna tabela ról Board

Rola	Model (kwiecień 2026)	Waga	Dział macierzysty	Domena	Obligatoryjny przy
Chief Context	Gemini 2.5 Pro	1.25	Architektura	Kontekst Księgi, historia systemu, domena biznesowa, target architecture	Zmiana Księgi, PHANTOM, zmiana zakresu, SYSTEM_REBUILD_MODE
Chief Reasoning	Claude Opus 4	1.00	Implementacja	Analiza logiczna, planowanie implementacji, dowody poprawności	Zmiana algorytmów, nowy agent, architektura rebuild, plan cutover
Chief Review	GPT-5	1.00	Testy i Walidacja	Weryfikacja kontraktów, red-team krawędzi decyzji, walidacja evidence	Każda decyzja D2+ (prawo veto)
Chief Safety	Rotacyjny (patrz C.2)	1.00	Red Team + Blue Team (rotacja)	Reguły E1-E7, hard gate'y, bezpieczeństwo architektury rebuild	Propozycje naruszające E-rules, D4+, SYSTEM_REBUILD_MODE

C.2. Strategia obsadzania roli Chief Safety (rotacja)

Chief Safety jest rolą rotacyjną między działami Red Team i Blue Team. W każdym cyklu (1 tydzień) obsadza ją inny senior model, co zapobiega utrwalaniu się schematów rozumowania.

YAML

```
# Rotacja Chief Safety (konfiguracja)
rotation:
  period: 168h # 1 tydzień
  department_rotation:
    - Dział Red Team
    - Dział Blue Team
  candidates:
    - model: claude-opus-4
      weight: 1.00
```

```
department: Red Team
availability: primary
- model: gpt-5
  weight: 1.00
department: Blue Team
availability: primary
- model: gemini-2.5-pro
  weight: 1.00
department: Red Team
availability: backup
- model: glm-5.1
  weight: 1.00 # elevated from 0.60 gdy obsadzony jako Chief Safety
department: Blue Team
availability: backup
tie_break: last_active_rotation_wins_next
```

C.3. Hierarchia tiery modeli i wagi

Tier	Model	Waga głosu	Dział macierzysty	Rola typowa
Chief	Gemini 2.5 Pro	1.25	Architektura	Chief Context (stały)
Chief	Claude Opus 4	1.00	Implementacja	Chief Reasoning (stały)
Chief	GPT-5	1.00	Testy	Chief Review (stały)
Chief	Rotacyjny	1.00	Red/Blue (rotacja)	Chief Safety (rotacyjny)
Senior	z.ai GLM-5.1	0.60	Governance	Reviewer, Chief Safety backup
Worker	Qwen3.5-72B	0.35	Implementacja	Worker pool
Worker	gpt-oss-20b	0.35	Platforma	Worker pool (self-hosted)
Spec.	Grok-3	—	Red Team	Ad-hoc specialist, adversarial

C.4. Reguły kworum wg klasy decyzji

- D0 — zero modeli (reguły deterministyczne)
- D1 — 1 Senior lub Chief, confidence ≥0.85
- D2 — suma wag głosów PRO > 2.125 (ponad połowa sumy 4.25)
- D3 — wszystkie cztery Chief głosują PRO (4/4), wagi łącznie = 4.25
- D4 — jak D3 + sygnatura człowieka (operatora SYLION)
- D5 — jak D4 + zewnętrzny reviewer (external_review_log.yaml podpisany)
- Chief Review veto — Chief Review może zawetować D2/D3 niezależnie od wag, kierując sprawę do Chief Safety (eskalacja).
- Kworum Seniors — przy D2 mogą dołączać Seniors (GLM-5.1) z wagą 0.60 każdy, ale ich głosy liczą się tylko gdy minimum 2 Chief głosują PRO (żeby zapobiec decyzjom bez Board).

C.5. Skład Council per klasa decyzji D0–D5

Klasa	Skład Council	Kto przewodniczy	Minimum głosów	Human	External
D0	Brak — system rules	Automaton (reguły)	—	Nie	Nie
D1	1 model (Senior lub Chief)	Senior (GLM-5.1) lub dowolny Chief	1 głos	Nie	Nie
D2	2 Chief + opcjonalnie Seniors	Chief Context (Gemini 2.5 Pro)	PRO > 2.125	Nie	Nie
D3	Full Board: 4 Chief obligatoryjnie	Chief Context (Gemini 2.5 Pro)	4/4 PRO	Zalecany	Nie
D4	Full Board: 4 Chief obligatoryjnie	Chief Context (Gemini 2.5 Pro)	4/4 PRO + human sig	Tak	Zalecany
D5	Full Board: 4 Chief + External Reviewer	Chief Context + External Reviewer	4/4 PRO + human sig + ext review	Tak	Obowiązkowy

C.6. Mapowanie działań do decyzji

Dział	Prowadzi decyzje	Weto prawo	Uczestniczy w D3+
Architektura	Zmiana target architecture, rebuild plan, Plany 08, 14	Tak (Chief Context)	Tak
Platforma	Provisioning, bundle pipeline, env state	Nie	Jako ekspert
Implementacja	Zmiana modułu, implementation strategy per moduł	Tak (Chief Reasoning)	Tak
Testy i Walidacja	Validation gates, regresja, promotion gates	Tak (Chief Review)	Tak
Red Team	Adversarial contracts, threat model shifts	Tak (Chief Safety)	Tak
Blue Team	Detekcja, DFIR, obrona nowej architektury	Tak (Chief Safety rotacja)	Tak
Governance	Zgodność z Księgą, Evidence pack, accept/reject log	Nie (advisory)	Zawsze

Aneks D · Rejestr hard gate'ów (v3.0)

Każdy plan definiuje swój zestaw hard gate'ów — punktów, w których pipeline może zablokować dalsze działanie. Gate'y mają kody postaci G-XXX-NN, gdzie XXX to trzyliterowy prefix planu, a NN to numer porządkowy. Aneks D.3–D.6 zawiera nowe gate'y z planów 12–16 (v3.0).

D.1. Prefixsy i plany (zestawienie)

Prefix	Plan	Zakres numeracji	Przykładowe gate'y
G-ACP	01 Admin Control Plane	01–08+	G-ACP-01 unknown principal → reject
G-VPS	02 VPS Provider	01–08+	G-VPS-01 tier mismatch → block; G-VPS-07 rebuild state invalid → escalate D4
G-KB	03 Knowledge Base	01–08+	G-KB-01 brak SHA-256 → reject
G-VLT	04 Secrets Vault	01–08+	G-VLT-01 brak MFA → reject
G-MDE	05 Model Engine	01–08+	G-MDE-01 D3 bez jednomysłności → escalate D4
G-ALC	06 Agent + Team Lifecycle	01–10+	G-ALC-08 team change bez D3 → block
G-EVS	07 Evolution Safety	01–08+	G-EVS-01 reguła E2 naruszona → freeze
G-CEV	08 Canonical Evolution	01–10+	G-CEV-09 ścieżka rebuild bez D4 → block
G-OPE	09 Operator Environment	01–08+	G-OPE-01 provision bez budżetu → block
G-COST	10 Cost Governance	01–08+	G-COST-01 100% okna → circuit breaker
G-RUN	11 Run Lifecycle	01–10+	G-RUN-09 multi-track bez manifest → block
G-BND	12 Bundle + Testing	01–10+	G-BND-01 unsigned bundle → reject; G-BUN-ADV-01 adversarial coverage
G-GOV	13 Governance	01–09+	G-GOV-01 event bez hash chain → reject; G-GOV-08 D5 bez external review → block
G-RBL	14 Greenfield Rebuild	01–12	G-RBL-01 rebuild bez D4/D5 → block; G-RBL-06 brak cutover_strategy → block
G-ORG	15 Multi-Team Org	01–08	G-ORG-01 skład działu bez D3 → block; G-ORG-05 Board bez 4 Chief → block
G-CMP	16 Compaction	01–08	G-CMP-01 compact starszy niż 24h → warn; G-CMP-03 pre-D3 bez compact → block

D.2. Gate'y planów 01–13 (kluczowe)

Gate ID	Nazwa	Plan	Klasa	Opis	Action on failure
G-ACP-01	Unknown principal	01	D0	Nieznany principal w żądaniu kontrolnym	reject
G-VPS-01	Tier mismatch	02	D1	Nie zgodność tieru VPS	block
G-KB-01	Brak baseline SHA	03	D0	Propozycja KB bez SHA-256 baseline	reject
G-VLT-01	Brak MFA	04	D0	Dostęp do vault bez MFA	reject
G-MDE-01	D3 bez 4/4	05	D3	Decyzja D3 bez jednomyślności Chief	escalate D4
G-ALC-01	Onboarding bez tier	06	D0	Nowy agent bez przypisanego tieru	reject
G-EVS-01	E2 naruszenie	07	D1	Więcej niż 3 zmiany/dzień/domena	freeze domain
G-CEV-01	Proposal bez baseline	08	D0	Propozycja bez baseline snapshot	reject
G-OPE-01	Brak budżetu	09	D0	Provisioning bez zatwierdzenia budżetu	block
G-COST-01	Circuit breaker	10	D0	100% okna budżetowego przekroczone	circuit breaker
G-RUN-01	A4 bez reservation	11	D0	Stage A4 bez reservation VPS	block
G-BND-01	Unsigned bundle	12	D0	Bundle bez podpisu kryptograficznego	reject
G-GOV-01	Brak hash chain	13	D0	Event audytu bez hash chain	reject

D.3. Gate'y G-RBL — Plan 14 (Greenfield Rebuild)

Gate ID	Nazwa	Klasa	Opis	Action on failure
G-RBL-01	Rebuild bez D4/D5	D4/D5	Próba SYSTEM_REBUILD_MODE bez decyzji D4 lub D5	block + escalate Council
G-RBL-02	Brak legacy fitness report	D3	Brak rebuild_decision_report.md przed decyzją reuse vs rebuild	block (wymagaj artefaktu)

Gate ID	Nazwa	Klasa	Opis	Action on failure
G-RBL-03	Target architecture pusta	D4	Decyzja D4 rebuild bez target_architecture_delta.json	block
G-RBL-04	Brak greenfield module map	D4	Brak greenfield_module_map.json przy SYSTEM_REBUILD_MODE	block
G-RBL-05	Cutover bez walidacji	D3	Próba cutover run bez przejścia shadow run lub dual run	block
G-RBL-06	Brak cutover_strategy	D4	Brak cutover_strategy.md przy D4+ rebuild	block
G-RBL-07	Multi-track bez manifest	D3	Uruchomienie multi-track run bez multi_track_run_manifest.yaml	block
G-RBL-08	Decommission bez cutover	D4	Próba decommission legacy bez zakończonego cutover run	block
G-RBL-09	External review brak (D5)	D5	Decyzja D5 bez podpisanego external_review_log.yaml	block + alert human
G-RBL-10	Red team pre-rebuild	D4	Brak red team validation przed promote_or_reject w D4+	block (wymagaj bundle)
G-RBL-11	Legacy line aktywna po D	D3	Legacy line nadal aktywna bez dual/cutover planu w manifest	warn + require plan
G-RBL-12	Council 4/4 nie osiągnięty	D5	Decyzja D5 bez jednomyślności Board (4/4)	block + eskalacja Human

D.4. Gate'y G-ORG – Plan 15 (Multi-Team Organization)

Gate ID	Nazwa	Klasa	Opis	Action on failure
G-ORG-01	Zmiana składu bez D3	D3	Modyfikacja składu działu bez decyzji D3+	block
G-ORG-02	Nowy dział bez D4	D4	Powołanie nowego działu bez Human Gate D4	block
G-ORG-03	Likwidacja działu bez D4	D4	Rozwiązanie lub fuzja działu bez Human Gate D4	block
G-ORG-04	Brak org_structure.yaml	D3	Brak zaktualizowanego org_structure.yaml po zmianie D3+	block

Gate ID	Nazwa	Klasa	Opis	Action on failure
G-ORG-05	Board bez 4 Chief	D0	Board nie obsadza wszystkich 4 ról Chief jednocześnie	freeze D3+ decyzji
G-ORG-06	Rotacja Chief Safety poza kadencją	D1	Chief Safety obsadzone poza harmonogramem rotacji	warn + korekta
G-ORG-07	Team change bez proposal	D3	Zmiana składu agentów bez team_change_proposal.md	block
G-ORG-08	Council brak reprezentacji	D3	Council D3+ bez reprezentanta Governance	eskalacja

D.5. Gate'y G-CMP — Plan 16 (Compaction & Memory)

Gate ID	Nazwa	Klasa	Opis	Action on failure
G-CMP-01	Compact starszy niż 24h	D0	canon_compact.md nie był odświeżany ponad 24h	warn + trigger compact
G-CMP-02	Pre-bundle bez compact	D1	Bundle build bez aktualnego pre-bundle compact	warn + require compact
G-CMP-03	Pre-D3 bez compact	D3	Decyzja D3+ bez pre-D3 compact wygenerowanego	block
G-CMP-04	Agent bez agent-compact	D1	Agent sesja bez wstrzykniętego agent-compact	warn (sesja może nie mieć kontekstu)
G-CMP-05	Operator-compact nie dostarczony	D3	Operator nie otrzymał operator-compact przed D3+	warn + wyślij
G-CMP-06	Brak compaction_config	D2	Brak compaction_config.yaml w systemie	block (config wymagany)
G-CMP-07	Compact bez sekcji D3+	D2	canon_compact.md nie zawiera ostatnich 5 decyzji D3+	odrzuć compact, wygeneruj nowy

Gate ID	Nazwa	Klasa	Opis	Action on failure
G-CMP-08	Compact bez impl strategy	D2	canon_compact.md nie zawiera implementation_strategy per moduł	odrzuć compact, wygeneruj nowy

D.6. Gate G-BUN-ADV-01 — Adversarial Coverage (Plan 12)

Gate ID	Nazwa	Plan	Klasa	Opis	Action on failure
G-BUN-ADV-01	Adversarial coverage	12	D3	Bundle promocja D3+ bez pełnego adversarial contract coverage — brak red_team_bundle.yaml lub scenariusze poniżej minimum (100% critical paths)	block; wymagaj redteam_bundle.yaml z pełnym coverage przed promotion gate

Aneks E · Hierarchia modeli, ceny tokenów i mapa działań

Aneks E konsoliduje wszystkie modele używane w Pipeline 8.0.0 z ich wagami, tierami, kosztami per 1M tokenów (input/output) i mapowaniem do 7 stałych działów pipeline. Ceny ostatni raz zweryfikowane: kwiecień 2026.

E.1. Pełna hierarchia modeli × działły

Model	Tier	Rola Board	Waga	In \$/1M	Out \$/1M	Działły
Gemini 2.5 Pro	Chief	Chief Context (stały)	1.25	\$3.50	\$10.50	Architektura (główny), Governance (wsparcie)
Claude Opus 4	Chief	Chief Reasoning (stały)	1.00	\$15.00	\$75.00	Implementacja (główny), Governance (wsparcie)
GPT-5	Chief	Chief Review (stały)	1.00	\$10.00	\$30.00	Testy i Walidacja (główny), Governance (wsparcie)
Rotacyjny Chief Safety	Chief	Chief Safety (rotacyjny)	1.00	—	—	Red Team + Blue Team (rotacja tygodniowa)
z.ai GLM-5.1	Senior	Reviewer, Chief Safety backup	0.60	\$0.60	\$2.20	Governance (główny), Platforma (reviewer)
Qwen3.5-72B	Worker	Worker pool	0.35	\$0.30	\$0.90	Implementacja (workers), Testy (fact-check)
gpt-oss-20b	Worker	Worker pool (self-hosted)	0.35	\$0.10	\$0.30	Platforma (self-hosted), Implementacja (workers)
Grok-3	Spec.	Ad-hoc specialist	—	\$5.00	\$15.00	Red Team (adversarial ad-hoc)

E.2. Zasady wyboru modelu wg klasy decyzji

- D0 — zero modeli (reguły deterministyczne)
- D1 — jeden Senior (GLM-5.1) lub Chief, confidence ≥0.85
- D2 — Board (4 Chief) + opcjonalnie Seniors wspierająco
- D3 — Board jednomyślnie + Seniors wspierająco + Workers jako fact-check
- D4 — Full Board + sygnatura człowieka + Chief Context przygotowuje summary

- D5 — Full Board + human sig + External Reviewer + Grok-3 ad-hoc (opcjonalnie)

E.3. *Polityka cost-aware routing*

Dla zadań nie wymagających pełnego Board, Pipeline stosuje routing oszczędzający koszty: najpierw próba Worker → eskalacja do Senior → eskalacja do Chief. Każda eskalacja zapisywana w Matrix log jako decision trace.

PYTHON

```
# Cost-aware routing (pseudokod v3.0)
def route(task):
    if task.decision_class == "D0":
        return exec_rules(task)
    if task.decision_class == "D1" and task.complexity < 0.4:
        r = try_worker(task)
        if r.confidence >= 0.85:
            return r
    # eskalacja do Senior
    r = try_senior(task)
    if task.decision_class in ("D2", "D3") or r.confidence < 0.80:
        return board_vote(task, r)
    if task.decision_class in ("D4", "D5"):
        return human_gate(task, board_vote(task, r))
    return r
```

E.4. *Koszty w kontekście sprintu 8-tygodniowego (orientacyjne)*

Kategoria	Tokeny in	Tokeny out	Koszt szac.
D5 Council (5 decyzji — rebuild)	20M	8M	~\$2 100
D4 Council (20 decyzji)	20M	8M	~\$1 900
D3 Council (50 decyzji)	25M	10M	~\$2 350
D2 Council (450 decyzji)	90M	30M	~\$5 100
D1 Reviewer (2000 decyzji)	60M	15M	~\$420
Workers (fact-check, regresja)	200M	40M	~\$100
Red Team adversarial (Grok-3, 10 sesji)	5M	2M	~\$55
Embedding (nomic-embed, KB)	—	—	self-hosted
Compute VPS (4× STATE + 8× STANDARD)	—	—	~\$1 800
Storage + egress	—	—	~\$200
SUMA sprintowa (z rebuild)			~\$14 025

UWAGA · *Budżety rebuild jako hard gate*

Plan 10 (Cost Governance) egzekwuje budżety per run i per ścieżkę. Ścieżka rebuild (D4/D5) ma osobny budżet rebuild_budget $\geq 10 \times$ budżetu normalnego sprintu. Przekroczenie 80% wyzwala warn G-COST, 100% wyzwala circuit breaker.

Aneks F · Matryca implementation_strategy × ścieżka × klasa decyzji

Aneks F definiuje dopuszczalne kombinacje strategii implementacji modułu (implementation_strategy), ścieżki pipeline (Evolution / Refactor / Rebuild) oraz minimalnej wymaganej klasy decyzji. Każda kombinacja wskazuje czy wymagany jest Human Gate i/lub External Review.

ZASADA · Zasada minimalizmu decyzyjnego

Zawsze wybierz najniższą dopuszczalną klasę decyzji dla danej kombinacji strategii × ścieżka. Eskaluj tylko gdy: (1) analiza legacy fitness wykaże konieczność wyższej ścieżki, (2) blast radius przekracza moduł, (3) zmiana dotyka inwariantów bezpieczeństwa.

F.1. Główna tabela macierzy 4 × 3

Strategy	Ścieżka	Min. klasa	Typowe D	Human Gate	External Review	Gate kluczowy	Uwagi
reuse	Evolution	D0	D0–D2	Nie	Nie	G-CEV-01	Moduł pozostaje na MZ bez zmian kontraktu
reuse	Refactor	—	—	—	—	—	NIEDOPUSZCZALNE: reuse jest z definicji evolution-only
reuse	Rebuild	—	—	—	—	—	NIEDOPUSZCZALNE: reuse wyklucza rebuild
refactor	Evolution	D1	D1–D2	Nie	Nie	G-CEV-01	Minimalne zmiany kontraktu — muszą być backwards-compatible
refactor	Refactor	D3	D3	Zalecany	Nie	G-RBL-02, G-BND-ADV	Kontrakt zachowany, dwie linie równoległe w cutover
refactor	Rebuild	D4	D4	Tak	Zalecany	G-RBL-03, G-RBL-06	Kontrakt może ulec zmianie; wymaga target_architecture_delta.json

Strategy	Ścieżka	Min. klasa	Typowe D	Human Gate	External Review	Gate kluczowy	Uwagi
replace	Evolution	D2	D2	Nie	Nie	G-CEV-01	Podmiana implementacji przy zachowaniu zewnętrznego interfejsu
replace	Refactor	D3	D3	Zalecany	Nie	G-RBL-02, G-ORG-04	Zmiana kontraktu w obrębie Refactor path
replace	Rebuild	D4	D4	Tak	Zalecany	G-RBL-01, G-RBL-03	Pełna podmiana modułu — nowy target architecture
greenfield	Evolution	—	—	—	—	—	NIEDOPUSZCZALNE: greenfield to zawsze Rebuild path (nowy kod od zera)
greenfield	Refactor	—	—	—	—	—	NIEDOPUSZCZALNE: greenfield wyklucza Refactor path
greenfield	Rebuild	D4	D4–D5	Tak	Obowiązkowy przy D5	G-RBL-01, G-RBL-04, G-RBL-09	D4 = rebuild modułu; D5 = SYSTEM_REBUILD_MODE (pełny system)

F.2. Reguły eskalacji ścieżki

- Legacy fitness score < 0.4 → automatyczna rekomendacja Refactor lub Rebuild path (do decyzji Council)
- Blast radius > 3 moduły → escalate do Rebuild path D4
- Threat model shift wykryty → D5 obligatoryjny niezależnie od bieżącej implementation_strategy
- Contract break w refactor → escalate z D3 do D4 (replace)
- SYSTEM_REBUILD_MODE → zawsze greenfield + Rebuild path + D5

F.3. Minimalne evidence per kombinacja

Strategy	Ścieżka	Min. klasa	Wymagane artefakty
reuse	Evolution	D0–D2	changelog, baseline SHA
refactor	Refactor	D3	rebuild_decision_report.md, Council votes (4/4), baseline snapshot

Strategy	Ścieżka	Min. klasa	Wymagane artefakty
replace	Rebuild	D4	rebuild_decision_report.md, target_architecture_delta.json, cutover_strategy.md, Council 4/4, human sig
greenfield	Rebuild	D4	rebuild_decision_report.md, target_architecture_delta.json, greenfield_module_map.json, cutover_strategy.md, Council 4/4, human sig
greenfield	Rebuild	D5	jak D4 + external_review_log.yaml (podpisany External Reviewer), redteam_bundle.yaml, adversarial contract

Aneks G · Indeks artefaktów rebuild v3.0

Aneks G kataloguje wszystkie nowe artefakty wprowadzone w v3.0. Dla każdego artefaktu podano: krótki opis, właściciela (dział), typowe triggery generowania oraz politykę retencji.

ZASADA · Artefakty jako evidence

Wszystkie artefakty z Aneksu G są artefaktami audytowymi — muszą być hashowane (SHA-256), podpisane przez właściciela działu i włączone do evidence pack przy decyzjach D3+. Brak artefaktu wymaganego = gate twardego blokuje dalsze działanie pipeline.

G.1. Artefakty Planu 14 — Greenfield Rebuild Orchestration

Artefakt	Plan	Właściciel	Opis	Triggery	Retencja
rebuild_decision_report.md	14	Architektura	Analiza legacy fitness (score 0.0–1.0), uzasadnienie wyboru ścieżki (reuse/refactor/rebuild), blast radius, rekomendacja Council	analize_legacy_fitness(); ręcznie przez Chief Context; przed decide_reuse_vs_rebuild	Permanent (historia decyzji)
target_architecture_delta.json	14	Architektura	Delta od bieżącej architektury (MZ 3.5.5) do target architecture — moduły dodane/zmienione/usunięte, zmiany kontraktów, zależności	Po zatwierdzeniu create_greenfield_plan (D4/D5)	Permanent + wersjonowanie
greenfield_module_map.json	14	Architektura	Mapa wszystkich modułów przeznaczonych do napisania od zera: id, opis, zależności, dział odpowiedzialny, priorytety	Po zatwierdzeniu target_architecture_delta.json	Aktualna wersja + archiwum poprzednich
cutover_strategy.md	14	Platforma	Plan przełączenia ruchu: tryb (parallel/shadow/cut-and-replace), kryteria gotowości, punkty rollback, harmonogram decommission	Przed uruchomieniem multi-track run (D3+)	Permanent (historia cutoverów)

G.2. Artefakty Planu 13 — Governance & Evidence

Artefakt	Plan	Właściciel	Opis	Triggery	Retencja
external_review_log.yaml	13	Governance	Log zewnętrznego przeglądu dla D5: tożsamość reviewera, zakres, wyniki, podpis, data; wymagany przed promote_or_reject D5	Decyzja D5 (SYSTEM_REBUILD_MODE lub zmiana fundamentów Księgi)	Permanent
book_change_proposal.md	13	Governance	Formalna propozycja zmiany Księgi z uzasadnieniem, klasą decyzji, identyfikacją ryzyk i votes record	Przed każdą zmianą Księgi D3+	Permanent

G.3. Artefakty Planu 15 — Multi-Team Organization

Artefakt	Plan	Właściciel	Opis	Triggery	Retencja
org_structure.yaml	15	Governance	Kanoniczny opis struktury organizacyjnej pipeline: 7 działów, role, modele, wagi, interakcje między działami	Przy każdej zmianie składu D3+ (G-ORG-04); inicjalnie przy adopcji v3.0	Aktualna + archiwum wersji
council_composition.yaml	15	Governance	Aktualny skład Council per klasa decyzji D0–D5: kto wchodzi, wagi, rotacje Chief Safety	Przy zmianie Board lub rotacji Chief Safety	Aktualna wersja
team_change_proposal.md	06 / 15	Governance	Propozycja zmiany składu zespołu/działu: nowy agent, zmiana wag, nowa rola, fuzja działów — każda wymaga D3+	Auto-generowane przez pipeline (Plan 06) lub ręcznie przez operatora	Permanent

G.4. Artefakty Planu 16 — Compaction & Memory Protocol

Artefakt	Plan	Właściciel	Opis	Triggery	Retencja
compaction_config.yaml	16	Governance	Konfiguracja protokołu compaction: interwały, progi kompresji, format agent-compact i operator-compact, retencja	Przy adopcji v3.0; zmiana konfiguracji = D2	Aktualna wersja

Artefakt	Plan	Właściciel	Opis	Triggery	Retencja
canon_compact.md	16	Governance	Skondensowany stan systemu: zasady Księgi (compact), wiszące decyzje, inwarianty bezpieczeństwa, ostatnie 5 D3+ decyzji, implementation_strategy per moduł; dwupoziomowy (agent-compact i operator-compact)	Rytm 24h (baseline compact); pre-D3+ (przed każdą decyzją D3+); pre-bundle (przed bundle build)	Rolling 30 dni + archiwum przy D3+

G.5. Artefakty Planu 12 – Bundle + Testing (Red/Blue Team)

Artefakt	Plan	Właściciel	Opis	Triggery	Retencja
redteam_bundle.yaml	12	Red Team	Pakiet scenariuszy ofensywnych Red Team: adversarial contracts, coverage map, wyniki testów, krytyczne ścieżki, status pass/fail	Pre-bundle D3+; obligatoryjnie przed promote_or_reject D4+ (G-RBL-10); G-BUN-ADV-01	Per bundle version + permanent
blueteam_bundle.yaml	12	Blue Team	Pakiet artefaktów obronnych Blue Team: reguły detekcji, playbooki SOC/DFIR, metryki telemetrii, coverage nowej architektury	Pre-bundle D3+; po każdym red team teście wymagającym odpowiedzi Blue	Per bundle version + permanent

G.6. Artefakty Planu 11 – Run Lifecycle (Multi-track)

Artefakt	Plan	Właściciel	Opis	Triggery	Retencja
multi_track_run_manifest.yaml	11	Platforma	Manifest wielotorowego runu: definicja linii legacy i target, tryb uruchomienia (shadow/dual/cutover), synchronizacja stanów, kryteria zakończenia	Przed uruchomieniem multi-track run (G-RBL-07); wymagany przy Rebuild path D4+	Per run + archiwum

G.7. Schematy referencyjne artefaktów kluczowych

G.7.1. *rebuild_decision_report.md* — struktura

YAML

```
# Rebuild Decision Report
## Metadata
id: rdr-<SHA8>
date: <ISO8601>
initiator: <dział>
decision_class: D4 | D5

## Legacy Fitness Analysis
module_id: <id>
current_baseline: MZ 3.5.5
legacy_fitness_score: 0.0-1.0    # <0.4 = rebuild rekomendowany
blast_radius_modules: [<lista modułów>]
blast_radius_score: 0.0-1.0

## Rekomendacja ścieżki
recommended_path: evolution | refactor | rebuild
recommended_strategy: reuse | refactor | replace | greenfield
justification: |
  <uzasadnienie tekstowe>

## Council Votes (D3+ required)
chief_context_vote: PRO | CON | ABSTAIN
chief_reasoning_vote: PRO | CON | ABSTAIN
chief_review_vote: PRO | CON | ABSTAIN
chief_safety_vote: PRO | CON | ABSTAIN
unanimous: true | false

## Human Gate
operator_signature: <hash>
operator_timestamp: <ISO8601>

## External Review (D5 only)
external_reviewer_id: <id>
external_review_log_ref: external_review_log.yaml#<id>
external_approved: true | false
```

G.7.2. *canon_compact.md* — struktura

YAML

```
# Canon Compact – SYLION Pipeline v3.0
## Metadata
generated_at: <ISO8601>
trigger: rhythm_24h | pre_d3 | pre_bundle
level: agent | operator
pipeline_version: 8.0.0
mz_baseline: 3.5.5

## Zasady nadrzędne (Księga compact)
- Rebuildability over Lineage
- Reguły E1-E7 (lista aktywnych)
- Inwarianty bezpieczeństwa: [lista]

## Wyszące decyzje
```

```

pending_decisions:
  - id: <dec-id>
    class: D3 | D4 | D5
    summary: <krótki opis>
    deadline: <ISO8601>

## Ostatnie 5 decyzji D3+
last_d3plus:
  - id: <dec-id>
    class: D3
    date: <ISO8601>
    summary: <opis>
    outcome: approved | rejected | deferred

## Implementation strategy per moduł
modules:
  - id: <module-id>
    implementation_strategy: reuse | refactor | replace | greenfield
    path: evolution | refactor | rebuild
    current_baseline: MZ 3.5.5
    target_baseline: null | <target-id>

```

G.7.3. org_structure.yaml — fragment

YAML

```

# Org Structure SYLION Pipeline 8.0.0
departments:
  - id: architecture
    name: Dział Architektury
    chief_role: Chief Context
    chief_model: gemini-2.5-pro
    chief_weight: 1.25
    agents:
      - role: target_architect
        model: gemini-2.5-pro
        tier: Chief
      - role: blast_radius_analyst
        model: glm-5.1
        tier: Senior

  - id: implementation
    name: Dział Implementacji
    chief_role: Chief Reasoning
    chief_model: claude-opus-4
    chief_weight: 1.00

  - id: testing
    name: Dział Testów i Walidacji
    chief_role: Chief Review
    chief_model: gpt-5
    chief_weight: 1.00

  - id: red_team
    name: Dział Red Team
    chief_role: Chief Safety (rotacja)
    chief_model: rotational
    chief_weight: 1.00

```

- id: blue_team
 - name: Dział Blue Team
 - chief_role: Chief Safety (rotacja)
 - chief_model: rotational
 - chief_weight: 1.00
- id: platform
 - name: Dział Platformy
 - chief_role: null
 - specialist_model: gpt-oss-20b
 - tier: Worker
- id: governance
 - name: Dział Governance i Compliance
 - chief_role: null
 - specialist_model: glm-5.1
 - tier: Senior

G.7.4. multi_track_run_manifest.yaml — schemat

YAML

```
# Multi-Track Run Manifest
manifest_id: mtrm-<SHA8>
created_at: <ISO8601>
decision_ref: <decision-id>    # D4 lub D5

tracks:
  legacy:
    run_id: <run-id>
    baseline: MZ 3.5.5
    status: active | shadow | decommissioning | decommissioned
    traffic_weight: 100 → 50 → 0    # w trakcie cutover

  target:
    run_id: <run-id>
    baseline: <target-architecture-id>
    status: shadow | dual | cutover | production
    traffic_weight: 0 → 50 → 100

cutover_mode: parallel | shadow | cut-and-replace
cutover_criteria:
  - validation_gate: G-RBL-05
  - red_team_pass: true
  - blue_team_coverage: ≥90%
  - human_approval: required

rollback_trigger:
  error_rate_delta: >0.05
  latency_p99_delta: >200ms
  human_override: always_available
```

Koniec aneksów — SYLION Pipeline v3.0

SYLION Pipeline — Aneksy A–G dla Dokumentacji zintegrowanej v3.0. Pipeline Core 8.0.0 na bazie MZ 3.5.5 (wymienialny). Szesnaście planów w dwóch warstwach. Zasada nadrzędna: „Rebuildability over Lineage” — pipeline jako organizacja inżynierska zdolna ewoluować, refaktoryzować, przepisywać i testować ofensywnie oraz defensywnie pod kontrolą Księgi i człowieka.

ZASADA · Autor

Dokument wygenerowany przez Perplexity Computer w konsultacji z operatorem SYLION. Aneksy A–G v3.0 obejmują: rozszerzony Słownik (A), Drabinę D0–D5 (B), Full Board × 7 działów (C), Rejestr gate'ów z G-RBL/G-ORG/G-CMP/G-BUN-ADV (D), Modele × ceny × działy (E), Matrycę implementation_strategy (F) i Indeks artefaktów rebuild (G).

ANEKS H

Reversibility Protocol · v3.1

Aneks H — Reversibility Protocol

REVERSIBILITY · Cel Aneksu H

Konsolidacja wszystkich elementów protokołu rollback rozproszonych po Planach 02, 07, 10, 13 i 14 w jedno kanoniczne odniesienie. Aneks H służy jako single source of truth dla dev, ops i governance gdy zachodzi potrzeba szybkiego odniesienia się do pełnego cyklu życia cutover ↔ rollback ↔ decommission.

H.1. Pełna maszyna stanów cutover / rollback

Stan	Plan źródłowy	Aktor	Trigger wejścia	Trigger wyjścia
SHADOW_RUNNING	02 §2.9.1	VPS auto	target allocated	sufficient traffic replayed
SHADOW_VALIDATING	02 §2.9.1	Testy + Red Team	replay start	all gates A3.3-A3.7 pass
CUTOVER_READY	02 §2.9.1 + 13 G-CUT-01	VPS auto + Architect	technical gates pass	(czeka na operator window)
CUTOVER_PENDING	02 §2.9.1 + 13 G-CUT-02	Human Gate	operator window grant	window start
CUTOVER_ACTIVE	02 §2.9.1	VPS auto + Blue Team	window start	atomic lease swap complete
STABILIZATION	02 §2.9.1 + 14 §14.8	Blue Team + Platforma	cutover complete	D3:24h / D4:72h / D5:168h elapsed
HOT_STANDBY (legacy)	02 §2.9.2	VPS auto	= STABILIZATION faza	auto-drop write-through
WARM_STANDBY	02 §2.9.2 + 14 §14.8	Platforma	STABILIZATION end	D3:7d / D4:14d / D5:30d elapsed
AUTO_ROLLBACK	07 §7.11.1-2	VPS auto	error_rate/latency/contract breach	traffic on legacy verified
MANUAL_FAST_ROLLBACK	07 §7.11.1 + 15 §15.9.2	Chief Safety + operator	incydent P0/P1 or veto	traffic on legacy verified
COUNCIL_ROLLBACK	07 §7.11.1 + 13 §13.12	Council 4/4 fast	trend / soft signal	D3 fast decision

Stan	Plan źródłowy	Aktor	Trigger wejścia	Trigger wyjścia
ROLLED_BACK (target freeze)	07 §7.11.4	VPS auto	post-rollback	(terminal for this iteration)
LEGACY_DECOMMISSION	02 §2.9.1 + 13 G-CUT-04	Human + Council	osobna decyzja D3+	(terminal)

H.2. Trigger matrix — pełna

Trigger	Rodzaj rollback	Czas reakcji	Aktor decyzji	Artefakt wyjściowy
error_rate 2× baseline × 2 okna 5min	AUTO_ROLLBACK	< 60s	VPS auto	rollback_event.json + rollback_postmortem.md
p99 latency 3× baseline × 2 okna 5min	AUTO_ROLLBACK	< 60s	VPS auto	rollback_event.json + rollback_postmortem.md
contract invariant violation (1+)	AUTO_ROLLBACK	< 60s	VPS auto	rollback_event.json + contract_breach_report.md
security incident HIGH/CRITICAL	AUTO_ROLLBACK	< 60s	VPS auto + Blue Team coord	rollback_event.json + dfir_timeline.md
incydent P0/P1 (human)	MANUAL_FAST_ROLLBACK	< 5 min	Chief Safety + operator (2/2)	rollback_event.json + incident_record.md
operator CLI veto	MANUAL_FAST_ROLLBACK	< 5 min	operator (standalone)	rollback_event.json + veto_rationale.md
niepokojący trend (soft signal)	COUNCIL_ROLLBACK	< 60 min	Council 4/4 fast	rollback_event.json + trend_analysis.md
WARM_STANDBY fallback	POST_WARM_ROLLBACK	15-60 min (WAL replay)	Council 4/4 + Human	rollback_event.json + wal_replay_log.json

H.3. Fencing lease — protokół

```
# fencing_lease.yaml — runtime contract VPS (Plan 07 §7.11.3)
lease:
```

```
schema_version: v3.1
current_primary: target_v2 | legacy_v1
token_version: int (monotonic, persistent across VPS restarts)
granted_at: ISO-8601 timestamp
ttl: duration (default 30s)
renewed_by: node_id of VPS controller

# Inwarianty:
# INV-1: w każdej chwili co najwyżej jeden environment ma aktualny token
# INV-2: token_version rośnie monotonicznie; rollback = increment + swap grant
# INV-3: environment widzący token_version < własny → odrzuca (503 fenced_out)
# INV-4: brak renewala po ttl → VPS controller ogłasza election
# INV-5: election wymaga kworum VPS nodes (3/5 typowo)
```

H.4. Rollback eligibility matrix

Stan legacy	Faza cyklu	Auto rollback	Manual/Council rollback	Czas rollback
HOT_STANDBY	STABILIZATION	TAK (D1/D2 auto)	TAK (D3+)	< 60s
WARM_STANDBY	POST-STABILIZATION (7-30d)	NIE	TAK (Council D3+)	15-60 min
DECOMMISSIONED	permanent	NIE	NIE	niemożliwy
ARCHIVED	data only	NIE	NIE	dni (full rebuild)

H.5. Bramy governance dotyczące rollback

Gate	Plan	Faza	Artefakt
G-CUT-01 Cutover Readiness	13 §13.12.1	SHADOW → CUTOVER_READY	cutover_readiness_certificate.json
G-CUT-02 Cutover Window Approval	13 §13.12.1	CUTOVER_READY → CUTOVER_PENDING	cutover_window_grant.json
G-CUT-03 Post-Cutover Review	13 §13.12.1	STABILIZATION → WARM_STANDBY	post_cutover_review_report.md
G-CUT-04 Legacy Decommission	13 §13.12.1	WARM_STANDBY → DECOMMISSION	decommission_decision_record.md

REVERSIBILITY · Aneks H jako „bible rollback”

Ten aneks zawiera WSZYSTKO czego potrzebujesz do wykonania rollback: triggerzy, aktorów, protokół, artefakty, bramy, eligibility. Jeśli jesteś on-call i widzisz incydent na target — zacznij od Aneksu H, potem Plan 07 sekcja 7.11, potem Plan 13 G-CUT-02/03. Nie musisz czytać całego dokumentu.

ANEKS I

*Compact Fidelity Test — schemat i golden set · v3.1**Aneks I — Compact Fidelity Test: pełny schemat**FIDELITY · Cel Aneksu I*

Aneks I jest kanonicznym odniesieniem dla wszystkich aktorów pracujących z CFT: Compact Validator Agent (Plan 15 §15.9.1), Evidence Curator (Plan 13), Compaction Engine (Plan 16). Zawiera pełny schemat JSON artefaktu, specyfikację golden_set oraz reguły interpretacji wyników.

1.1. Schemat compact_fidelity_report.json (kanoniczny)

```
{
  "$schema": "https://syllion.local/schemas/cft-v3.1.json",
  "type": "object",
  "required": ["cft_id", "compact_ref", "full_corpus_ref",
    "golden_set_version", "alpha", "beta", "metrics", "verdict", "signed_by"],
  "properties": {
    "cft_id": {"type": "string", "pattern": "^CFT-\\d{4}-\\d{2}-\\d{2}T\\d{2}:\\d{2}Z-\\.+$"},
    "compact_ref": {"type": "string", "description": "hash lub path do canon_compact.md"},
    "full_corpus_ref": {"type": "string", "description": "hash lub path do pełnego PDF"},
    "golden_set_version": {"type": "string", "pattern": "^\\d{4}-Q[1-4]$" },
    "alpha": {
      "type": "object",
      "required": ["model", "runs", "accuracy"],
      "properties": {
        "model": {"type": "string"},
        "runs": {"type": "integer", "minimum": 3},
        "accuracy": {"type": "number", "minimum": 0, "maximum": 1}
      }
    },
    "beta": { /* identical to alpha */ },
    "metrics": {
      "type": "object",
      "required": ["class_match", "verdict_match", "rationale_overlap",
        "gate_requirements_match", "golden_set_accuracy"],
      "properties": {
        "class_match": {"$ref": "#/$defs/hardMetric"},
        "verdict_match": {"$ref": "#/$defs/hardMetric"},
        "rationale_overlap": {"$ref": "#/$defs/softMetric"},
        "gate_requirements_match": {"$ref": "#/$defs/hardMetric"},
        "golden_set_accuracy": {
          "type": "object",
          "required": ["alpha_accuracy", "beta_accuracy", "parity_ok"],
          "properties": {
            "alpha_accuracy": {"type": "number"},
            "beta_accuracy": {"type": "number"},
            "parity_ok": {"type": "boolean"}
          }
        }
      }
    }
  }
}
```

```
    }
  }
},
"verdict": {"enum":["PASS","FAIL","DEGRADED"]},
"signed_by": {"type":"array","items":{"type":"string"},"minItems":2},
"next_cycle_at": {"type":"string","format":"date-time"}
},
"$defs": {
  "hardMetric": {
    "type":"object",
    "required":["hard","rate"],
    "properties":{"
      "hard":{"type":"boolean","description":"true iff rate==1.0"},
      "rate":{"type":"number","minimum":0,"maximum":1}
    }
  },
  "softMetric": {
    "type":"object",
    "required":["soft","mean_cosine"],
    "properties":{"
      "soft":{"type":"boolean","description":"true iff mean_cosine >= 0.85"},
      "mean_cosine":{"type":"number","minimum":-1,"maximum":1}
    }
  }
}
}
```

1.2. Reguły verdict

Verdict	Warunek	Konsekwencja
PASS	4× hard.hard=true soft.soft=true parity_ok=true	compact publikowany; dozwolone użycie w decyzjach D3+
DEGRADED	3× hard.hard=true (soft.soft=false lub parity marginal)	compact wstrzymany; Chief Safety review w ciągu 24h
FAIL	jakaś hard.hard=false lub parity_ok=false	compact veto; incydent open; auto-rebuild compacta z świeżym seedem

1.3. Golden set — schemat manifestu

```
# golden_set_manifest.yaml schema v3.1
golden_set:
  version: string (format: YYYY-QN, np. 2026-Q2)
  curator: string (Evidence Curator + Chief Safety names)
  rotation_cadence: duration (default: 90 days)
  last_rotated: date-time
  next_rotation: date-time
  stats:
    total: integer (>= 20)
    D3: integer (>= 10)
    D4: integer (>= 7)
    D5: integer (>= 3)
```

```
last_30_days: integer (>= 5)
adversarial: integer (>= 2)
items:
- id: string (pattern: GS-\d{3})
  decision_ref: string (matches Evidence pack ID)
  class: enum [D3, D4, D5]
  ground_truth:
    verdict: enum [approve, reject, modify, approve_with_external]
    gates: array of strings (gate IDs)
    rationale_summary: string (operator-written ground truth)
  domain: string (plany których dotyczy decyzja)
  difficulty: enum [easy, medium, hard]
  added_at: date-time
  expires_at: date-time (usually +1 year)
```

I.4. Heterogeniczne pary modeli — lista dozwolonych

Alpha	Beta	Warunek użycia	Uwagi
gpt-5	claude-opus-4	domyślna para agent-compact	różne rodziny; stabilna od 2026-01
claude-opus-4	gemini-2-pro	domyślna para operator-compact	różni vendor; różne architektury
gemini-2-pro	gpt-5	rotacja pre-bundle	rotacyjna para #3; używana 1× na 3 cycles
gpt-5	gpt-5	ZABRONIONA	same-model testuje recall, nie fidelity
claude-opus-4	claude-sonnet-4	ZABRONIONA	same-family też nie kwalifikuje się jako heterogenous

FIDELITY · Rotacja par modeli

Compact Validator Agent ROTUJE pary między trzema dozwolonymi konfiguracjami. Jeśli cycle N użył (gpt-5, claude), cycle N+1 powinien użyć (claude, gemini). Rotacja zapobiega „ukryciu” systematycznego błędu jednej pary — jeśli wszystkie trzy pary dają zgodne wyniki, fidelity jest silniejsza.

I.5. Minimalne progi jakości

Parametr	Minimum	Target	Procedura przy naruszeniu
rationale_overlap cosine	0.85	0.90+	DEGRADED → Chief Safety review
alpha accuracy na golden_set	0.90	0.95+	golden_set review (maybe stale)
beta accuracy (relative)	alpha – 0.05	alpha – 0.02	compact rebuild z świeżym seedem
Determinism (3 run majority)	3/3 agree	3/3 agree	non-deterministic = FAIL
Golden set freshness	5 items /30d	10 items /30d	rotation trigger

1.6. Integracja CFT z bramkami governance

CFT jest egzekwowany przez G-CFT-01 (Plan 13 sekcja 13.12.2). Każda decyzja D3+ której Evidence pack zawiera ślad odczytu compacta MUSI dołączyć compact_fidelity_report.json spełniający: verdict=PASS, next_cycle_at > decision_timestamp, alpha i beta obaj signed. Bez spełnienia tych trzech warunków Council nie może osiągnąć quorum.

ANEKS J

*Pattern Authorization Token — parking lot · v3.1**Aneks J — Pattern Authorization Token (parking lot Auto-D4)***UWAGA** · *Aneks J jest parking lot — nie aktywny kanon*

PAT NIE JEST częścią aktywnego pipeline v3.1. Aneks J archiwizuje odrzuconą rekomendację Auto-D4 (automatyczne wykrywanie patternów decyzji D4 z proponowaniem regeneracji modułu) w formie przekształconej na kompatybilną z kanonem Rebuildability + Reversibility. Celem jest zachowanie intencji rekomendacji jako przyszły kierunek ewolucji (v3.2+) bez jej obecnego wdrażania, które naruszyłoby wymóg Council 4/4 + Human dla D4.

J.1. Historia decyzji

W kwietniu 2026 operator SYLION przedstawił cztery rekomendacje dla Pipeline v3.0. Analiza kanoniczna odrzuciła rekomendację #1 (Auto-D4) w proponowanej formie z dwóch powodów: (1) D4 wymaga z definicji Council 4/4 + Human — delegacja decyzji D4 do automatu narusza kanon; (2) „Pattern matching = decyzja” zakłada determinizm w obszarze gdzie kanon wymaga uzasadnienia per-case. Rekomendacja została jednak zachowana jako Pattern Authorization Token (PAT) — koncept hybrydowy, w którym automat NIE decyduje, ale przygotowuje pre-aprobowany szablon z udokumentowanym precedensem, który Human + Council mogą szybciej zaakceptować.

J.2. Koncepcja Pattern Authorization Token

```
# pattern_authorization_token.yaml (PROPOSAL — NIE aktywne w v3.1)
pattern_authorization_token:
  id: PAT-2026-xxxx
  pattern_ref: PATTERN-2025-11-security-vault-rebuild
  pattern_detected_at: 2026-04-15T09:00Z
  detection_method: pattern_matcher_v1 (ML model, confidence 0.87)
  detected_signals:
    - module_aging_score: 0.82
    - contract_drift_count: 14
    - security_audit_flags: 3
  precedent:
    decision_ref: D4-2025-11-14-auth-vault-rebuild
    outcome: SUCCESS (rollback_rate: 0, post_cutover_sev: none)
    lessons: [...]
  proposed_action:
    class: D4 (recommend_rebuild)
    target_module: secrets_vault
    shortcut: pre-populated_rebuild_plan + pre-signed_risk_assessment
  NOT_AUTOMATED:
    - Council 4/4 MUSI zatwierdzić
    - Human Gate MUSI podpisać
    - PAT tylko skraca prep time, NIE zastępuje decyzji
  benefit: redukuje czas od signal do decyzji z ~5 dni do ~1 dnia
  risk: automated pattern matching może przegapić nuance specific to module
```

J.3. Co jest z v3.1 zgodne, a co nie

Aspekt rekomendacji	Original Auto-D4	PAT (parking lot v3.1)	v3.2+ status
Pattern detection	automat proponuje	automat proponuje	KOMPATYBILNE
Decyzja D4	automat decyduje	Human + Council 4/4 decydują	KOMPATYBILNE
Human Gate	omijany	obowiązkowy	KOMPATYBILNE
Evidence pack	auto-wygenerowany, niereviewed	template, operator reviews	KOMPATYBILNE
Rollback plan	nie wspomniany	obowiązkowy (per Reversibility)	KOMPATYBILNE
CFT dla PAT decision	N/A	obowiązkowy (D3+ decision)	KOMPATYBILNE

J.4. Warunki aktywacji PAT w v3.2+

Zanim PAT może być uznany za kanoniczny (hipotetyczny Plan 17), muszą zostać spełnione cztery warunki wstępne. Każdy z nich jest sam decyzją D4 lub D5:

1. D4 — wdrożenie pattern_matcher_v1 jako modułu SYLION (Plan 03/05).
2. D4 — certyfikacja pattern_matcher przez Red Team na 50+ decyzjach historycznych.
3. D4 — ustanowienie bram governance G-PAT-01..05 (zarezerwowane w Planie 13 §13.12.5).
4. D5 — zatwierdzenie Planu 17 „Pattern-Assisted Decisions” przez Council 4/4 + Human + External Review.

ZASADA · Dlaczego archiwizujemy zamiast odrzucić całkowicie

Kanoniczna analiza rekomendacji #1 Auto-D4 wykazała że intencja (redukcja latencji między sygnałem a decyzją rebuild) jest cenna i spójna z Rebuildability over Lineage. Odrzuciliśmy WDROŻENIE proponowane, nie IDEĘ. Parking lot zachowuje ideę w formie kompatybilnej z kanonem, aby przyszłe zespoły mogły ją wskrzesić bez ponownej analizy od zera. To przykład principled evolution: nie wszystko nowe jest dobre; nie wszystko odrzucone jest złe.

J.5. Reserved namespace

Aneks J rezerwuje następujące identyfikatory dla ewentualnego wdrożenia PAT w v3.2+. Żaden z nich NIE MOŻE być używany w v3.1 do innych celów:

Reserved ID	Przeznaczenie
PAT-YYYY-NNNN	instance identifier dla Pattern Authorization Token
G-PAT-01	Pattern Detection Certification (czy detector się zgadza z precedensem)

Reserved ID	Przeznaczenie
G-PAT-02	Precedent Match Quality Gate (confidence threshold)
G-PAT-03	Pre-Populated Plan Review (Council review of auto-draft)
G-PAT-04	Human Endorsement Gate (Human Gate accelerated)
G-PAT-05	Post-PAT Learning (feedback loop do pattern_matcher)
pattern_matcher_v1..vN	identifikator dla pattern detection models
Plan 19	zarezerwowany numer planu dla Pattern-Assisted Decisions

Aneks K — Operator Console: design tokens, komponenty, WebSocket

Aneks K formalizuje warstwę prezentacji Operator Console (Plan 17). Definiuje pełny zestaw design tokens (kolory, typografia, odstępy, animacje), kontrakty WebSocket dla aktualizacji czasu rzeczywistego, oraz checklistę WCAG AA. Ten aneks jest wymagalnym artefaktem dla bramy G-UI-01 (Console Design Conformance) i G-UI-02 (Accessibility Conformance).

K.1. Paleta kolorów — Operator Console

Paleta jest dwumodalna (light/dark) i zaprojektowana dla operatorów pipeline pracujących w długich sesjach. Kontrast tekstu wobec tła $\geq 7.0:1$ dla body (poziom AAA), $\geq 4.5:1$ dla large text (poziom AA). Każdy kolor ma znaczenie semantyczne — żaden nie jest dekoracją.

K.1.1. Light mode

Token	Hex	Rola	Kontrast vs bg
--bg-canvas	#F7F6F2	główne tło aplikacji	—
--bg-surface	#FFFFFF	karty, panele, modale	—
--bg-surface-alt	#FBFAF6	sekcje wtórne, tabele zebra	—
--bg-elevated	#FFFFFF + shadow-sm	dropdown, tooltip, popover	—
--text-primary	#1A1815	body, headings	15.8:1 (AAA)
--text-secondary	#5C5A55	labels, metadata	7.2:1 (AAA)
--text-muted	#8B8A85	placeholders, captions	4.6:1 (AA)
--border-default	#D4D1CA	granice kart, separatory	—
--border-strong	#A8A6A0	focus, aktywne wiersze	—
--accent-primary	#01696F	linki, CTA, aktywny tab (Hydra Teal)	7.4:1 vs white
--accent-rebuild	#A1450A	ścieżka Rebuild, badge greenfield	6.1:1
--accent-reverse	#1A6E8E	Reversibility, rollback procedury	5.8:1
--accent-fidelity	#5C2D91	Compact Fidelity Test, CFT score	7.1:1
--accent-efficiency	#0E7C4A	Code Optimizer, efficiency budget	6.4:1
--accent-aeis	#7C3A8C	AEIS layer (rezerwa v3.5)	6.8:1
--state-error	#A12C7B	hard veto, gate failure, alarm	5.9:1
--state-warning	#964219	soft veto, attention required	6.7:1
--state-success	#437A22	gate pass, evidence valid	5.4:1

Token	Hex	Rola	Kontrast vs bg
--state-info	#1A6E8E	informacja, hint	5.8:1
--decision-d0	#9CA3AF	D0 Informational	3.5:1 (AA large)
--decision-d1	#10B981	D1 Trivial	3.2:1 (AA large)
--decision-d2	#3B82F6	D2 Standard	3.6:1 (AA large)
--decision-d3	#F59E0B	D3 Significant	3.1:1 (AA large)
--decision-d4	#EF4444	D4 Critical	4.0:1 (AA large)
--decision-d5	#6A1B9A	D5 Greenfield/Systemic	7.6:1

K.1.2. Dark mode

Token	Hex	Rola
--bg-canvas	#171614	główne tło aplikacji
--bg-surface	#1C1B19	karty, panele, modale
--bg-surface-alt	#201F1D	sekcje wtórne, tabele zebra
--text-primary	#E8E7E4	body, headings (kontrast 14.2:1)
--text-secondary	#A8A6A1	labels, metadata (kontrast 7.8:1)
--text-muted	#797876	placeholders (kontrast 4.6:1)
--border-default	#393836	granice kart
--accent-primary	#4F98A3	linki, CTA (kontrast 5.4:1)
--accent-rebuild	#D17F45	ścieżka Rebuild
--accent-fidelity	#B280D9	CFT, compact validation
--accent-efficiency	#5BB47B	Code Optimizer, efficiency
--state-error	#D163A7	hard veto, alarm
--state-warning	#BB653B	soft veto
--state-success	#6DAA45	gate pass

K.2. Typografia

Token	Wartość	Zastosowanie
--font-display	Inter, system-ui, sans-serif	hero, titles
--font-body	Inter, system-ui, sans-serif	body, labels, UI
--font-mono	JetBrains Mono, Menlo, monospace	kod, IDs, hashe
--text-xs	11px / 16px line-height	badges, footnotes
--text-sm	13px / 20px	metadata, captions
--text-base	14px / 22px	body domyślny
--text-md	16px / 24px	akcent body
--text-lg	18px / 28px	section heading
--text-xl	22px / 32px	panel title
--text-2xl	28px / 36px	page title
--text-3xl	36px / 44px	hero
--weight-regular	400	body
--weight-medium	500	emphasis, labels
--weight-semibold	600	section headings
--weight-bold	700	page titles, hero

K.3. Odstępy, radius, cienie, animacje

Kategoria	Token	Wartość
Spacing	--space-1 / 2 / 3 / 4 / 5 / 6 / 8	4 / 8 / 12 / 16 / 20 / 24 / 32 px
Spacing	--space-10 / 12 / 16 / 20	40 / 48 / 64 / 80 px
Radius	--radius-sm / md / lg / xl / full	4 / 6 / 10 / 14 / 9999 px
Shadow	--shadow-sm	0 1px 2px rgba(0,0,0,.06)
Shadow	--shadow-md	0 4px 12px rgba(0,0,0,.08)
Shadow	--shadow-lg	0 12px 32px rgba(0,0,0,.12)
Motion	--ease-default	cubic-bezier(.2,.7,.2,1)
Motion	--ease-emphasized	cubic-bezier(.05,.7,.1,1)
Motion	--dur-fast / base / slow	120 / 220 / 360 ms
Motion	prefers-reduced-motion	WSZYSTKO 0ms · tylko opacity

Kategoria	Token	Wartość
Z-index	--z-base / dropdown / sticky / modal / toast	0 / 100 / 200 / 1000 / 1100

K.4. WebSocket — kontrakt komunikacyjny

Operator Console odbiera aktualizacje real-time przez WebSocket pod ścieżką /ws/v1/console. Kanał jest jednokierunkowy (server → client) dla telemetrii; akcje operatora idą przez REST. Każda wiadomość ma kopertę envelope i strongly-typed payload.

K.4.1. Envelope

JSON
<pre>{ "v": 1, "event_id": "evt_01HXYZK2N4...", "event_type": "gate.status_changed", "emitted_at": "2026-04-20T12:34:56.789Z", "correlation_id": "req_01HXYZ...", "tenant_id": "team_alpha", "sequence": 18234, "payload": { ... } }</pre>

K.4.2. Lista event_type (v3.3)

event_type	Panel	Częstotliwość	Krytyczność
gate.status_changed	Gates, Decisions	~5/min	high
gate.evidence_uploaded	Evidence	~2/min	medium
decision.proposed	Decisions	~1/min	high
decision.escalated	Decisions, Alerts	~5/h	critical
council.vote_cast	Council	~3/min	high
council.quorum_reached	Council	~10/h	critical
agent.heartbeat	Agents	co 30s	low
agent.state_changed	Agents	~10/min	medium
efficiency.budget_warning	Efficiency	~3/h	high
efficiency.veto_raised	Efficiency, Alerts	~1/h	critical
compact.fidelity_score	Compact, CFT	~1/h	medium
cutover.state_transition	Cutover	~1/h	critical
rollback.triggered	Cutover, Alerts	rzadko	critical
skill.lifecycle_changed	Skills	~5/h	medium

<i>event_type</i>	<i>Panel</i>	<i>Częstotliwość</i>	<i>Krytyczność</i>
telemetry.metric_window	Dashboards	co 10s	low

K.4.3. Niezawodność i ordering

- ['Sekwencja monotoniczna per tenant — klient odrzuca event jeśli $sequence \leq$ ostatnio widziany.', 'At-least-once delivery — klient MUSI być idempotentny względem event_id.', 'Resume po rozłączeniu — klient wysyła ?last_sequence=N w handshake; serwer replayuje od N+1 z bufora 5 min.', 'Heartbeat ping co 25s; jeśli brak pong → klient rekonstruuje z exponential backoff (1s, 2s, 4s, 8s, 16s, max 30s).', 'Backpressure — jeśli klient nie nadąża, serwer dropuje eventy klasy low i wysyła telemetry.lag_warning.']

K.5. WCAG AA — checklist konformancji (G-UI-02)

<i>Wymóg</i>	<i>Kryterium</i>	<i>Status w v3.3</i>
1.4.3 Contrast (Minimum)	$\geq 4.5:1$ body, $\geq 3:1$ large/UI	PASS — paleta zaprojektowana pod AA
1.4.11 Non-text Contrast	$\geq 3:1$ dla controls, focus rings	PASS — focus 2px solid accent-primary
2.1.1 Keyboard	wszystkie funkcje dostępne klawiaturą	PASS — pełna nawigacja Tab/Shift-Tab/Enter/Esc
2.1.2 No Keyboard Trap	można opuścić każdy komponent	PASS — modal Esc, dropdown Esc
2.4.3 Focus Order	logiczna kolejność fokusa	PASS — DOM order = logical order
2.4.7 Focus Visible	widoczny focus indicator	PASS — ring-2 ring-accent-primary
3.2.1 On Focus	focus nie zmienia kontekstu	PASS
4.1.2 Name, Role, Value	ARIA labels dla custom components	PASS — pełne aria-label, aria-describedby
4.1.3 Status Messages	aria-live regions dla update	PASS — aria-live=polite dla ws events
2.3.3 Animation	prefers-reduced-motion respektowane	PASS — wszystkie motion 0ms gdy reduced
1.3.1 Info and Relationships	semantyka HTML + landmarks	PASS — header/nav/main/aside
1.4.10 Reflow	320px bez scroll horizontalnego	PASS — responsive layout
1.4.12 Text Spacing	line-height 1.5, letter 0.12em itp.	PASS
3.3.1 Error Identification	błędy oznaczone tekstem + kolorem	PASS — ikona + tekst + aria-invalid

UI / UX · Aneks K — bramy referujące

Aneks K jest cytowany przez bramy G-UI-01 (Console Design Conformance — wymaga zgodności z K.1-K.3), G-UI-02 (Accessibility — wymaga PASS na wszystkich wierszach K.5), G-UI-03 (Realtime Reliability — wymaga implementacji K.4.3). Każda zmiana tokenów K.1 lub kontraktu K.4 jest decyzją D3+ i wymaga Council review.

Aneks L — Code Efficiency Report: schema i anti-patterns

Aneks L definiuje pełny JSON Schema artefaktu `code_efficiency_report.json` (Plan 13 §13.13) oraz katalog anti-patternów wykrywanych przez Code Optimizer Agent (Plan 15 §15.10). Code Optimizer ma prawo HARD VETA — wykrycie anti-patternu z `severity=critical` blokuje merge bez możliwości override przez Council (tylko Human Gate D5 może przejść).

L.1. JSON Schema — `code_efficiency_report.json`

JSON

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://syllion.dev/schemas/v3.3/code_efficiency_report.json",
  "title": "CodeEfficiencyReport",
  "type": "object",
  "required": ["report_id", "module", "generated_at", "baseline",
    "current", "deltas", "anti_patterns", "verdict"],
  "properties": {
    "report_id": { "type": "string", "pattern": "^CER-\\d{4}-\\d{6}$" },
    "module": { "type": "string" },
    "generated_at": { "type": "string", "format": "date-time" },
    "generated_by": {
      "type": "object",
      "properties": {
        "agent": { "const": "code_optimizer" },
        "version": { "type": "string" }
      }
    },
    "baseline": {
      "type": "object",
      "required": ["loc", "cyclomatic_complexity_avg", "duplication_pct"],
      "properties": {
        "loc": { "type": "integer", "minimum": 0 },
        "cyclomatic_complexity_avg": { "type": "number" },
        "cyclomatic_complexity_p95": { "type": "number" },
        "duplication_pct": { "type": "number", "minimum": 0, "maximum": 100 },
        "cognitive_complexity_avg": { "type": "number" },
        "function_count": { "type": "integer" },
        "avg_function_length": { "type": "number" },
        "max_function_length": { "type": "integer" },
        "public_api_surface": { "type": "integer" }
      }
    },
    "current": { "$ref": "#/properties/baseline" },
    "deltas": {
      "type": "object",
      "properties": {
        "loc_delta_pct": { "type": "number" },
        "complexity_delta": { "type": "number" },
        "duplication_delta_pct": { "type": "number" }
      }
    },
    "anti_patterns": {
      "type": "array",
      "items": {
        "type": "object",

```

```

    "required": ["id", "severity", "file", "line", "description"],
    "properties": {
      "id": { "type": "string", "pattern": "^AP-[A-Z]{3,8}-\\d{2,3}$" },
      "severity": { "enum": ["critical", "high", "medium", "low"] },
      "file": { "type": "string" },
      "line": { "type": "integer", "minimum": 1 },
      "function": { "type": "string" },
      "description": { "type": "string" },
      "detected_by_rule": { "type": "string" },
      "suggested_fix": { "type": "string" },
      "llm_origin_confidence": { "type": "number", "minimum": 0, "maximum": 1 }
    }
  },
  "budgets": {
    "type": "object",
    "properties": {
      "loc_budget": { "type": "integer" },
      "loc_budget_consumed_pct": { "type": "number" },
      "complexity_budget": { "type": "number" },
      "complexity_budget_consumed_pct": { "type": "number" }
    }
  },
  "verdict": {
    "type": "object",
    "required": ["status", "hard_veto"],
    "properties": {
      "status": { "enum": ["PASS", "WARN", "FAIL"] },
      "hard_veto": { "type": "boolean" },
      "veto_reason": { "type": "string" },
      "override_required": { "enum": ["none", "council", "human_d5"] }
    }
  },
  "signature": { "type": "string" }
}

```

L.2. Katalog anti-patternów LLM-origin (skrót)

Code Optimizer wykrywa wzorce typowe dla generowanego kodu LLM. Lista nie jest wyczerpująca — Plan 18 (Skills Pipeline) dostarcza skill code-bloat-detector który aktualizuje katalog co kwartał. Kolumna „Conf LLM” to typowa pewność że anti-pattern pochodzi z LLM.

ID	Nazwa	Severity	Conf LLM	Opis
AP-DEFENSIVE-01	Excessive null checks	medium	0.85	if x is not None: if hasattr(...): if len(...) > 0 — kaskady defensywne
AP-DEFENSIVE-02	Try/except wokół czystej arytmetyki	high	0.92	try/except Exception bez powodu, swallow + return None
AP-DEFENSIVE-03	Walidacja parametrów już zwalidowanych	medium	0.78	powtórna walidacja typu na każdej warstwie wywołania

ID	Nazwa	Severity	Conf LLM	Opis
AP-COMMENT-01	Komentarz powtarzający kod	low	0.95	# increment counter \\n counter += 1 — komentarz nie dodaje informacji
AP-COMMENT-02	Docstring opisujący implementację, nie kontrakt	medium	0.88	docstring 'this function loops over items and...' zamiast 'returns sum'
AP-NAMING-01	Verbose naming	low	0.7	get_user_information_from_database vs get_user — bloat nazewnictwa
AP-ABSTRACTION-01	Premature abstraction	high	0.86	AbstractFactoryStrategyBuilder dla 1 use-case — nieużywany interfejs
AP-ABSTRACTION-02	Wrapper bez wartości dodanej	medium	0.81	class UserService(): def get(self, id): return db.get(id) — pusta warstwa
AP-DUP-01	Copy-paste struktur z drobną zmianą	high	0.83	5 funkcji różniących się jednym literałem — brak parametryzacji
AP-CONFIG-01	Magic constants	medium	0.6	hardcoded liczby/stringi w wielu miejscach
AP-LOG-01	Verbose logging defensywne	low	0.9	log.info na każdej linii funkcji
AP-IMPORT-01	Nieużywane importy	low	0.7	importy z LLM completion które nigdy nie są używane
AP-TEST-01	Test sprawdzający że mock zwraca mock	high	0.94	assert mock.call_count == 1 — test bez wartości semantycznej
AP-TEST-02	Test z hardcoded current date	medium	0.65	assert today() == '2026-04-15' — brittle test
AP-ASYNC-01	async/await bez I/O	medium	0.79	async def add(a, b): return a + b — fałszywa współbieżność
AP-EXC-01	Łapanie Exception bez re-raise/log	critical	0.88	silent failure — Code Optimizer HARD VETO
AP-SEC-01	eval/exec na user input	critical	0.55	HARD VETO — eskalacja do Security Architect

EFFICIENCY · Hard veto vs soft veto

Anti-pattern severity=critical → Code Optimizer ma HARD VETO (Plan 15 §15.10). Hard veto NIE jest przebijalne przez Council 4/4 — tylko Human Gate D5 z wpisem do compact_post_mortem może przepuścić kod. Severity=high → soft veto (przebijalne Council 4/4 z uzasadnieniem). Severity=medium/low → recorded warning, nie blokuje merge.

Aneks M — Skills Catalog: 30 skilli inicjalnych v3.3

Aneks M zawiera katalog 30 skilli wprowadzonych w v3.3 (Plan 18). Każdy skill jest opakowanym kawałkiem wiedzy/protokołu który agent może załadować on-demand. Skille mają lifecycle DRAFT → REVIEWED → PUBLISHED → DEPRECATED. Katalog rośnie organicznie — Plan 20 (v3.5 AEIS) wprowadza Skill Demand Engine który automatycznie wykrywa luki.

M.1. Katalog skilli (v3.3 baseline)

Skill ID	Domena	Plan ref	Status
syllion-canon-navigator	meta	Plan 18	PUBLISHED
decision-classifier	meta	Plan 02	PUBLISHED
council-orchestrator	governance	Plan 04	PUBLISHED
evidence-validator	governance	Plan 13	PUBLISHED
compact-fidelity-tester	compaction	Plan 16	PUBLISHED
rollback-responder	reversibility	Plan 07	PUBLISHED
legacy-preservation-manager	reversibility	Plan 14	PUBLISHED
code-bloat-detector	efficiency	Plan 15	PUBLISHED
performance-baseline-collector	efficiency	Plan 15	PUBLISHED
memory-profiler	efficiency	Plan 15	PUBLISHED
cost-envelope-tracker	efficiency	Plan 15	PUBLISHED
llm-anti-pattern-scanner	efficiency	Plan 15	PUBLISHED
rebuild-readiness-assessor	rebuildability	Plan 14	PUBLISHED
greenfield-bootstrapper	rebuildability	Plan 14	PUBLISHED
contract-diff-analyzer	refactor	Plan 06	PUBLISHED
shadow-validator	cutover	Plan 07	PUBLISHED
cutover-state-machine	cutover	Plan 07	PUBLISHED
phantom-injector	security	Plan 11	PUBLISHED
security-architect-checklist	security	Plan 11	PUBLISHED
compliance-mapper-gdpr	compliance	Plan 11	PUBLISHED
compliance-mapper-soc2	compliance	Plan 11	PUBLISHED
team-lifecycle-coordinator	governance	Plan 09	PUBLISHED
agent-spawner	governance	Plan 09	PUBLISHED

Skill ID	Domena	Plan ref	Status
telemetry-aggregator	console	Plan 17	PUBLISHED
console-renderer	console	Plan 17	PUBLISHED
i18n-translator	console	Plan 17	PUBLISHED
rbac-policy-evaluator	console	Plan 17	PUBLISHED
skill-publisher	meta	Plan 18	PUBLISHED
skill-deprecator	meta	Plan 18	PUBLISHED
pattern-matcher-stub	experimental	Aneks J	DRAFT (parking lot)

M.2. Skill metadata schema (skill.yaml — v3.3)

YAML

```
id: code-bloat-detector
name: Code Bloat Detector
version: 1.0.0
status: PUBLISHED      # DRAFT | REVIEWED | PUBLISHED | DEPRECATED
domain: efficiency
owner_role: code_optimizer
plan_ref: Plan 15 §15.10
canon_ref: ["Plan 13 §13.13", "Aneks L"]
description: |
  Wykrywa LLM-origin code bloat wzorce (defensive checks, premature
  abstractions, copy-paste duplication). Generuje code_efficiency_report.json.
inputs:
  - name: source_tree
    type: directory
  - name: language
    enum: [python, typescript, go, rust, java]
outputs:
  - name: report
    schema: schemas/code_efficiency_report.json
requires_tools:
  - ast_parser
  - duplication_detector
budgets:
  max_runtime_seconds: 120
  max_memory_mb: 512
  max_cost_usd: 0.05
classification: D2      # decision class wymagana do zmian skilla
expiry_review: 2026-10-20
changelog:
  - { version: 1.0.0, date: 2026-04-15, change: "initial release v3.3" }
signature: ed25519:...
```

M.3. Lifecycle skilli

Status	Kto może użyć	Wymóg promocji	Bramy
DRAFT	tylko owner_role	Review przez 2 agentów + Skill Editor	G-SKILL-01
REVIEWED	agentów z domeny	Council 4/4 dla domain-critical, inaczej Skill Editor	G-SKILL-02
PUBLISHED	wszyscy agenci pipeline	—	—
DEPRECATED	backward compat 90 dni	komunikat + alternatywa w changelog	G-SKILL-03

SKILL · Aneks M — perspektywa AEIS (zapowiedź v3.5)

W v3.5 (rozdział 00 AEIS) Skill Demand Engine (Plan 20) automatycznie wykrywa luki w katalogu M na podstawie obserwacji powtarzających się porażek/eskalacji. AEIS nie generuje skilli autonomicznie bez Council — propozycja idzie przez gate G-AEIS-SKILL-01 z dowodem demand signal. Mechanizm jest w pełni audytowalny i odwracalny (skill DEPRECATED).

Aneks N — Baselines: Performance, Memory, Cost

Aneks N definiuje referencyjne baselines dla 4-wymiarowego budżetu efficiency (Plan 10 §10.10). Wartości są punktem startowym dla nowych modułów; istniejące moduły mają własne baselines mierzone przez Performance Engineer / Memory Auditor / Cost Optimizer. Każde przekroczenie baselines o >20% wymaga uzasadnienia w D2+, >50% wymaga D3 + Council.

N.1. Performance baselines

Mierzymy p50/p95/p99 latency dla typowych operacji. Pomiar przez performance-baseline-collector (skill M.1).

Klasa modułu	Operacja	p50	p95	p99	Hard limit (G-PERF-02)
agent-stateless	single inference call	180ms	450ms	800ms	1500ms
agent-stateful	context load + inference	320ms	750ms	1.2s	2.5s
council-vote	4-way quorum decision	600ms	1.5s	2.5s	5s
evidence-validator	single artifact validate	40ms	120ms	250ms	500ms
compact-builder	build canon_compact	3s	8s	15s	30s
cft-runner	compact fidelity test	5s	12s	22s	45s
rollback-trigger	trigger rollback decision	50ms	150ms	300ms	600ms
console-ws-tick	single ws message dispatch	5ms	18ms	40ms	100ms
skill-loader	load published skill	80ms	200ms	400ms	800ms

N.2. Memory baselines

Klasa modułu	Idle RSS	Active RSS p95	Hard cap (G-MEM-01)
agent-stateless	80 MB	180 MB	512 MB
agent-stateful (context 32k)	140 MB	320 MB	768 MB
agent-stateful (context 200k)	220 MB	680 MB	1.5 GB
council-orchestrator	180 MB	420 MB	1 GB
evidence-validator	60 MB	120 MB	256 MB
compact-builder	200 MB	800 MB	2 GB

Klasa modułu	Idle RSS	Active RSS p95	Hard cap (G-MEM-01)
console-backend	250 MB	600 MB	1.5 GB
console-ws-server	180 MB + 2MB/conn	500 MB @ 1k conn	4 GB
skill-registry	120 MB	280 MB	512 MB

Memory Auditor (Plan 15 §15.10) raportuje wycieki, fragmentację i wzrost RSS w czasie. Wzrost >5%/dzień przez 7 dni → soft veto G-MEM-02.

N.3. Cost baselines (USD/operacja)

Koszt zawiera tokeny LLM, compute, storage, network. Cost Optimizer (Plan 15 §15.10) raportuje envelope `cost_envelope.json`. Wartości dla aktualnej hierarchii modeli (Aneks E).

Operacja	Tokeny in/out (typ.)	Koszt p50	Koszt p95	Hard cap (G-COST-01)
agent inference (S-tier)	3k / 800	\$0.012	\$0.028	\$0.05
agent inference (A-tier)	8k / 2k	\$0.045	\$0.11	\$0.20
agent inference (B-tier — flagship)	32k / 4k	\$0.32	\$0.78	\$1.50
council-vote (4× A-tier)	32k / 8k łącznie	\$0.18	\$0.42	\$0.80
compact build (B-tier + tools)	60k / 12k	\$0.95	\$2.20	\$4.50
cft run (compare 2 compacts)	40k / 6k	\$0.58	\$1.30	\$2.50
evidence pack validate	5k / 1k	\$0.018	\$0.045	\$0.10
rebuild module assessment	120k / 18k	\$2.40	\$5.50	\$10
skill review (A-tier)	15k / 3k	\$0.085	\$0.18	\$0.40

N.4. Budżety dzienne — referencja dla pojedynczego tenanta

Wymiar	Typowy dzień	Sustained limit	Burst limit	Veto poziom
Latency violations p95	<2%	5%	10% (1h)	soft (G-PERF-01)
Memory cap breaches	0	0	3/dzień	hard (G-MEM-01)
Cost cap breaches	0	2/dzień	5/dzień	soft (G-COST-01)
Total spend (USD/day)	\$50-150	\$300	\$500	hard (G-COST-02)
Code bloat critical (LOC)	+0	+200/tydz	+500/tydz	hard (G-EFF-01)
Duplication % wzrost	≤+0.5pp	+1pp/tydz	+2pp/tydz	soft (G-EFF-03)

N.5. Procedura zmiany baselines

1. Każda zmiana baselines (N.1-N.4) jest decyzją D3 i wymaga Council 4/4.
2. Wniosek MUSI zawierać: pomiar trendu (4 tygodnie), uzasadnienie biznesowe, plan rollback baseline.
3. Zmiana hard limitów (G-MEM-01, G-COST-02, G-EFF-01) wymaga D4 + Human Gate.
4. Zmiana hard cap dla agent-stateful >2GB lub cost-cap >\$1000/day → D5 + External Review.
5. Po zmianie baselines: re-baseline wszystkich modułów w klasie + raport delta.

PERFORMANCE · Baselines vs canonical evolution

Baselines są DANYMI pomiarowymi — nie częścią kanonu intencji (Księga). Ich zmiana to ewolucja operacyjna (D3), nie kanoniczna (D4 Księga). Jednak jeśli baseline jest cytowany w bramie (np. G-PERF-02 cytuje N.1), zmiana baselines pociąga zmianę bramy → D4. Plan 16 (Compact Protocol) zachowuje historię baselines w canonical timeline aby umożliwić retrospektywną analizę regresji.

Aneks O — AEIS Self-Model: pełny JSON Schema

Aneks O definiuje pełny JSON Schema dokumentu self-model AEIS. Self-model jest żywą reprezentacją stanu poznawczego, wykonawczego i governance systemu AEIS. Każda zmiana self-modelu musi przejść bramę G-AEIS-01 (Self-Model Conformance). Rola Self-Model Curator (v3.5) odpowiada za aktualność i integralność tego artefaktu.

O.1. Cel i wersjonowanie self-model

Self-model AEIS pełni trzy funkcje: (1) inwentaryzacja aktywnych warstw, umiejętności i ról systemu w danym momencie; (2) podstawa do oceny gap w Skill Demand Engine (Plan 20); (3) źródło prawdy dla audytów Council i External Review przy decyzjach D4+. Wersjonowanie self-modelu jest semantyczne: MAJOR zmienia się przy każdej zmianie struktury warstw (D4+), MINOR przy dodaniu komponentów lub skillów (D2-D3), PATCH przy aktualizacji hashów i pomiarów (D0-D1).

- Każda wersja self-modelu jest przechowywana w Memory Layer (Plan 16 compact timeline).
- Self-model PATCH generowany jest automatycznie po każdym cyklu self-observation (Plan 19 §19.3).
- Self-model MAJOR wymaga D4 + Human Gate + ponownej walidacji G-AEIS-01.

O.2. Pełny JSON Schema

Poniższy schemat opisuje strukturę dokumentu self-model. Pola oznaczone required muszą być obecne przy każdym odczycie przez bramę G-AEIS-01.

JSON

```
{
  "$schema": "https://syllion.internal/schemas/aeis-self-model-v1.json",
  "$id": "aeis-self-model",
  "version": "1.0.0",
  "title": "AEIS Self-Model",
  "description": "Reprezentacja stanu poznawczego, wykonawczego i governance AEIS.",
  "type": "object",
  "required": ["$id", "version", "generated_at", "layers",
    "skill_catalog_hash", "gate_registry_hash",
    "role_board_hash", "policy_set_hash",
    "canon_refs", "compact_ref",
    "autonomy_under_canon_compliance", "signature"],
  "properties": {
    "$id": { "type": "string", "format": "uuid" },
    "version": { "type": "string", "pattern": "^\\d+\\.\\d+\\.\\d+$" },
    "generated_at": { "type": "string", "format": "date-time" },
    "layers": {
      "type": "array", "minItems": 6, "maxItems": 6,
      "items": {
        "type": "object",
        "required": ["layer_id", "name", "status", "components"],
        "properties": {
          "layer_id": { "type": "string",
            "enum": ["cognitive", "execution", "security",
```



```

        "memory","self_evolution","governance"] },
    "name": { "type": "string" },
    "status": { "type": "string",
        "enum": ["active","degraded","sandbox","disabled"] },
    "components": { "type": "array", "items": { "type": "string" } },
    "skills_refs": { "type": "array", "items": { "type": "string" } },
    "roles_refs": { "type": "array", "items": { "type": "string" } },
    "gates_refs": { "type": "array", "items": { "type": "string" } },
    "policies_refs": { "type": "array", "items": { "type": "string" } },
    "baselines_refs": { "type": "array", "items": { "type": "string" } }
    }
},
"skill_catalog_hash": { "type": "string", "description": "SHA-256 katalogu skillów"
},
"gate_registry_hash": { "type": "string", "description": "SHA-256 rejestru bram" },
"role_board_hash": { "type": "string", "description": "SHA-256 role board" },
"policy_set_hash": { "type": "string", "description": "SHA-256 self-limitation policies" },
"canon_refs": { "type": "array", "items": { "type": "string" } },
"compact_ref": { "type": "string", "description": "ID aktywnego compact_canon" },
"autonomy_under_canon_compliance": {
    "type": "object",
    "required": ["compliant", "last_checked", "violations"],
    "properties": {
        "compliant": { "type": "boolean" },
        "last_checked": { "type": "string", "format": "date-time" },
        "violations": { "type": "array", "items": { "type": "string" } }
    }
},
"self_observation_window": {
    "type": "object",
    "properties": {
        "window_hours": { "type": "integer" },
        "observations_count": { "type": "integer" },
        "anomaly_rate": { "type": "number" }
    }
},
"last_self_modification": {
    "type": "object",
    "required": ["commit_id", "type", "decision_class"],
    "properties": {
        "commit_id": { "type": "string" },
        "type": { "type": "string",
            "enum": ["skill_add","skill_update","gate_update",
                "role_update","layer_refactor","policy_update"] },
        "decision_class": { "type": "string", "enum": ["D0","D1","D2","D3","D4","D5"] },
        "council_vote": { "type": "string", "pattern": "^[0-4]/4$" },
        "human_approval": { "type": ["boolean", "null"] }
    }
},
"signature": {
    "type": "object",
    "required": ["algorithm", "value", "key_id"],
    "properties": {
        "algorithm": { "type": "string", "enum": ["ed25519"] },
        "value": { "type": "string" },
        "key_id": { "type": "string" }
    }
}

```

```

    }
  }
}
}

```

O.3. Walidacja schematu — pseudokod walidatora

Walidator self-modelu jest wywoływany przez bramę G-AEIS-01. Pseudokod poniżej opisuje procedurę walidacji. Walidacja jest synchroniczna i blokująca — brak wyniku w ciągu 30s traktowany jako FAIL.

JSON

```

function validate_self_model(doc: object) -> ValidationResult:
1. Wczytaj aktualny JSON Schema (0.2).
2. Uruchom walidację strukturalną (jsonschema.validate).
   FAIL jeśli wymagane pole brakuje lub typ niezgodny.
3. Zweryfikuj hashów:
   a. skill_catalog_hash == SHA256(skill_catalog.json)    → FAIL jeśli rozbieżność
   b. gate_registry_hash == SHA256(gate_registry.json)    → FAIL
   c. role_board_hash   == SHA256(role_board.json)        → FAIL
   d. policy_set_hash    == SHA256(self_limitation_policies.json) → FAIL
4. Zweryfikuj podpis ed25519 (signature.value, key_id).
   FAIL jeśli podpis nieważny lub klucz wycofany.
5. Sprawdź autonomy_under_canon_compliance.compliant == true.
   FAIL jeśli false. Alert do AEIS Steward.
6. Sprawdź layers: dokładnie 6 warstw, każda ma status != "disabled"
   (chyba że zatwierdzone przez Council D4).
7. Zwróć ValidationResult{ok: bool, errors: list, warnings: list}.

```

O.4. Versioning self-model — kompatybilność wsteczna i schema migration

Schema self-modelu jest wersjonowana niezależnie od wersji Pipeline. Migration policy: schema MAJOR jest backward-incompatible i wymaga migracji wszystkich archiwalnych self-modelów w Memory Layer przed aktywacją. Schema MINOR jest additive-only (nowe opcjonalne pola). Schema PATCH nie zmienia struktury.

Schema bump	Trigger	Wymaga	Backward-compatible
MAJOR	Zmiana struktury warstw lub wymaganych pól	D4 + Human + migracja archiwum	NIE
MINOR	Nowe opcjonalne pola lub nowe enum-value	D2 + Council 4/4	TAK
PATCH	Aktualizacja description, metadane	D1	TAK

O.5. Tabela kluczowych pól z opisem

Pole	Typ	Wymagane	Opis
\$id	UUID	TAK	Unikalny identyfikator dokumentu self-model

<i>Pole</i>	<i>Typ</i>	<i>Wymaga ne</i>	<i>Opis</i>
version	semver string	TAK	Wersja semantyczna self-modelu (MAJOR.MINOR.PATCH)
generated_at	ISO 8601	TAK	Czas wygenerowania (UTC)
layers[]	array[6]	TAK	Sześć warstw AEIS: cognitive/execution/security/memory/self_evolution/governance
layers[].layer_id	enum	TAK	Identyfikator warstwy
layers[].status	enum	TAK	Status: active/degraded/sandbox/disabled
layers[].components	string[]	TAK	Lista aktywnych komponentów w warstwie
layers[].skills_refs	string[]	NIE	Referencje do skillów powiązanych z warstwą
layers[].gates_refs	string[]	NIE	Bramy nadzorujące warstwę
skill_catalog_hash	SHA-256	TAK	Hash katalogu skillów — weryfikacja integralności
gate_registry_hash	SHA-256	TAK	Hash rejestru bram G-*
role_board_hash	SHA-256	TAK	Hash tablicy ról (Board/Council)
policy_set_hash	SHA-256	TAK	Hash katalogu self-limitation policies (Aneks Q)
canon_refs	string[]	TAK	Lista aktywnych sekcji Księgi cytowanych w self-modelu
compact_ref	string	TAK	ID aktywnego compact_canon w Memory Layer
autonomy_under_canon_compliance	object	TAK	Status zgodności z meta-zasadą Autonomy under Canon
self_observation_window	object	NIE	Parametry okna obserwacji własnej (Plan 19 §19.3)
last_self_modification	object	NIE	Ostatnia self-modification: commit, typ, klasa, głos Council
signature	object	TAK	Podpis ed25519 — integralność dokumentu

O.6. Przykład wypełnionego self-model

Poniższy przykład przedstawia wypełniony dokument self-model dla systemu AEIS po aktywacji Planu 19 i Planu 20. Pola hashów są skrócone dla czytelności.

JSON
<pre>{ "\$id": "f3a2c1d0-8e7b-4f6a-9c5d-0b1e2a3f4c5d", "version": "1.2.0", "generated_at": "2026-04-15T09:00:00Z", "layers": [{ "layer_id": "cognitive", "name": "Cognitive Layer", "status": "active",</pre>

```

    "components": ["reasoning_engine", "task_planner", "decomposer"],
    "skills_refs": ["skill:analysis-v2", "skill:synthesis-v1"],
    "gates_refs": ["G-AEIS-01", "G-AEIS-02"] },
  { "layer_id": "execution", "name": "Execution Layer", "status": "active",
    "components": ["agent_runner", "tool_dispatcher", "integration_hub"],
    "skills_refs": ["skill:code-gen-v3"], "gates_refs": ["G-EXE-01"] },
  { "layer_id": "security", "name": "Security Layer", "status": "active",
    "components": ["PHANTOM", "sandbox_manager", "rbac_engine"],
    "gates_refs": ["G-SEC-01", "G-AEIS-03"] },
  { "layer_id": "memory", "name": "Memory Layer", "status": "active",
    "components": ["canon_store", "compact_engine", "evidence_vault"],
    "gates_refs": ["G-COMP-01", "G-CFT-01"] },
  { "layer_id": "self_evolution", "name": "Self-Evolution Layer",
    "status": "active",
    "components": ["observation_engine", "improvement_queue",
                  "skill_demand_engine", "self_model_curator"],
    "gates_refs": ["G-AEIS-01", "G-AEIS-SKILL-01"] },
  { "layer_id": "governance", "name": "Governance Layer", "status": "active",
    "components": ["council_board", "decision_ladder", "gate_registry"],
    "gates_refs": ["G-AEIS-04", "G-AEIS-05"] }
],
"skill_catalog_hash": "a3f1b2c4d5e6...",
"gate_registry_hash": "9b8c7d6e5f4a...",
"role_board_hash": "1a2b3c4d5e6f...",
"policy_set_hash": "7e8f9a0b1c2d...",
"canon_refs": ["Księga §19.1", "Księga §20.3", "Aneks Q SLP-001"],
"compact_ref": "compact:canon-2026-04-15-v12",
"autonomy_under_canon_compliance": {
  "compliant": true,
  "last_checked": "2026-04-15T08:55:00Z",
  "violations": []
},
"self_observation_window": {
  "window_hours": 24, "observations_count": 842, "anomaly_rate": 0.004
},
"last_self_modification": {
  "commit_id": "sm-2026-04-14-0042",
  "type": "skill_add",
  "decision_class": "D2",
  "council_vote": "4/4",
  "human_approval": null
},
"signature": {
  "algorithm": "ed25519",
  "value": "3d4e5f6a7b8c...",
  "key_id": "aeis-signing-key-2026-01"
}
}

```

AEIS · AEIS Self-Model — fundament self-governance

Self-model AEIS jest żywym artefaktem governance. Bez aktualnego, zwalidowanego self-modelu brama G-AEIS-01 blokuje każdą self-modification. Self-Model Curator (rola v3.5) ma obowiązek aktualizacji self-modelu po każdej zmianie katalogu skillów, rejestru bram lub tablicy ról. Hash-y zapewniają integralność — dowolna niesprawdzona zmiana dowolnego komponentu skutkuje FAIL G-AEIS-01 i blokadą pipeline do czasu weryfikacji Council.

Aneks P — Skill Demand Signals: szczegółowe schematy i protokoły generacji

Aneks P rozszerza Plan 20 (Skill Demand Engine) o pełny JSON Schema demand_signal, szczegółową tabelę typów sygnałów, algorytm agregacji oraz schemat skill_outline.yaml. Demand Signals są głównym mechanizmem przez który AEIS identyfikuje luki w katalogu skillów i inicjuje propozycje nowych skillów (przez bramę G-AEIS-SKILL-01).

P.1. Pełny JSON Schema demand_signal.json

JSON

```
{
  "$schema": "https://syllion.internal/schemas/demand-signal-v1.json",
  "$id": "demand-signal",
  "version": "1.0.0",
  "type": "object",
  "required": ["signal_id", "signal_type", "source", "severity",
    "generated_at", "freshness_window_h", "evidence_refs"],
  "properties": {
    "signal_id": { "type": "string", "format": "uuid" },
    "signal_type": {
      "type": "string",
      "enum": [
        "escalation_clustering", "gate_failure_repeat", "post_mortem_theme",
        "capability_gap", "baseline_drift", "council_split_vote",
        "human_override_pattern", "skill_not_found", "compaction_fidelity_drop",
        "agent_timeout_pattern", "cost_envelope_breach",
        "security_incident_pattern", "knowledge_staleness_signal"
      ]
    },
    "source": {
      "type": "object",
      "required": ["layer", "component"],
      "properties": {
        "layer": { "type": "string" },
        "component": { "type": "string" },
        "plan_ref": { "type": "string" },
        "run_id": { "type": "string" }
      }
    },
    "severity": { "type": "string", "enum": ["low", "medium", "high", "critical"] },
    "severity_score": { "type": "number", "minimum": 0, "maximum": 1 },
    "threshold_triggered": { "type": "string" },
    "freshness_window_h": { "type": "integer", "description": "Okno ważności sygnału w godzinach" },
    "expires_at": { "type": "string", "format": "date-time" },
    "evidence_refs": { "type": "array", "items": { "type": "string" }, "minItems": 1 },
    "suggested_skill_area": { "type": "string" },
    "aggregation_group": { "type": "string" },
    "demand_weight": { "type": "number", "minimum": 0 },
    "related_signals": { "type": "array", "items": { "type": "string" } },
    "metadata": { "type": "object", "additionalProperties": true }
  }
}
```

P.2. Tabela typów Demand Signals

Poniższe 13 typów sygnałów obejmuje spektrum źródeł informacji z których AEIS wnioskuje o lukach w katalogu skillów (rozszerzenie Plan 20 §20.2).

signal_type	Źródło	Threshold	severity_calc	freshness_window
escalation_clustering	Execution Layer (agent escalation log)	≥3 eskalacje tego samego tematu w 48h	count/48h × 0.25	72h
gate_failure_repeat	Governance Layer (gate registry)	Ta sama brama odrzuca ≥4 razy / tydzień	fail_rate × 0.4	168h
post_mortem_theme	Memory Layer (post-mortem vault)	Ten sam temat w ≥3 post-mortemach / miesiąc	theme_frequency × 0.3	720h
capability_gap	Cognitive Layer (task planner)	Zadanie bez dopasowanego skilla ≥2 razy / tydzień	gap_count × 0.5	96h
baseline_drift	Aneks N (baselines monitor)	Metryka >sustained limit przez ≥7 dni	drift_magnitude × 0.35	168h
council_split_vote	Governance Layer (council log)	Głosowanie Council 2/4 lub 3/4 (nie 4/4)	split_ratio × 0.6	336h
human_override_pattern	Governance Layer (Human Gate log)	Human cofnął decyzję AEIS ≥2 razy / miesiąc	override_count × 0.7	720h
skill_not_found	Execution Layer (skill dispatcher)	Dispatcher zwrócił 404 dla skill_id	1.0 (krytyczny)	24h
compaction_fidelity_drop	Memory Layer (CFT engine)	CFT score <0.85 (Plan 16 §16.11)	(0.85 – score) × 2.0	48h
agent_timeout_pattern	Execution Layer (agent runner)	Agent timeout ≥3 razy dla tego samego zadania	timeout_count × 0.3	48h
cost_envelope_breach	Aneks N (cost monitor, G-COST-01)	Operacja przekroczyła hard cap ≥2 razy / tydzień	breach_factor × 0.4	168h
security_incident_pattern	Security Layer (PHANTOM audit)	Incydent bezpieczeństwa powiązany z brakiem skilla	1.0 (krytyczny)	24h
knowledge_staleness_signal	Memory Layer (canon staleness monitor)	Sekcja Księgi nieaktualizowana >90 dni	staleness_days/90 × 0.2	720h

P.3. Algorytm agregacji Demand Signals

Agregacja sygnałów jest realizowana przez Demand Signal Analyst (rola v3.5) w cyklach 6-godzinnych. Wynikiem agregacji jest lista kandydatów skillów z priorytetem demand_weight przekazywana do Improvement Queue (Plan 19 §19.4).

JSON

AGGREGATION ALGORITHM v1.0

Input: active_signals[] (sygnały w oknie freshness)

Output: skill_candidates[] z demand_weight

1. Filtruj sygnały: zachowaj tylko te gdzie
NOW() < expires_at AND severity != "low" (chyba że ≥3 sygnały low tego samego typu).
2. Grupuj po aggregation_group (lub suggested_skill_area jeśli brak grupy).
3. Dla każdej grupy G:
 - a. base_weight = SUM(s.demand_weight for s in G)
 - b. recency_boost = 1 + (0.1 × signals_last_24h_in_G)
 - c. diversity_boost = 1 + (0.05 × distinct_signal_types_in_G)
 - d. total_weight = base_weight × recency_boost × diversity_boost
4. Rang-order kandydaci po total_weight DESC.
5. Dla kandydatów z total_weight ≥ 0.5 AND severity_max == "critical":
→ auto-escalate do Improvement Queue z priority="urgent".
6. Dla pozostałych z total_weight ≥ 0.3:
→ dodaj do Improvement Queue z priority="standard".
7. Zapisz aggregation_report do Memory Layer.
Podpisz raportem Demand Signal Analyst.

Anty-manipulacja (patrz P.6): jeśli ≥5 identycznych sygnałów
w <1h od tego samego źródła → flag jako "burst_anomaly",
nie agreguj, alert do AEIS Steward.

P.4. Schema skill_outline.yaml

skill_outline.yaml jest pierwszym artefaktem generowanym gdy kandydat skillów przekroczy bramę G-AEIS-SKILL-01. Outline jest podstawą do dalszego rozwoju skilla przez Skill Author (Plan 18 §18.3).

YAML

```
skill_outline:
  skill_id: "skill:TBD-unique-id"
  title: "<krótki tytuł skilla>"
  version_draft: "0.1.0"
  status: DRAFT
  demand_signal_refs:
    - "<signal_id-1>"
    - "<signal_id-2>"
  aggregation_group: "<nazwa grupy>"
  demand_weight_total: 0.0
  priority: standard # urgent | standard | low
  suggested_layer: "<cognitive|execution|security|memory|self_evolution|governance>"
  rationale: |
    <Uzasadnienie: dlaczego ten skill jest potrzebny.
    Musi odwoływać się do konkretnych demand signal_refs.>
  scope:
    covers:
      - "<obszar 1>"
```

```

- "<obszar 2>"
excludes:
- "<wykluczone obszary>"
inputs_required:
- "<wymagane dane wejściowe>"
expected_outputs:
- "<oczekiwane wyniki>"
related_plans:
- "Plan XX: <nazwa>"
related_skills:
- "skill:<id>"
rollback_plan: |
  <Opis jak wycofać skill jeśli okazuje się nieudany.
  Wymagane dla D2+.>
skill_author_assigned: "TBD"
council_pre_approval_required: false # true dla D3+
gates_to_pass:
- G-AEIS-SKILL-01
estimated_effort_days: 0
created_at: "<ISO 8601>"
created_by: demand_signal_analyst

```

P.5. Skill Generation Modes — rozszerzenie Plan 20 §20.7

Plan 20 §20.7 definiuje cztery tryby generacji skillów. Aneks P precyzuje progi i ścieżki dla każdego trybu.

Tryb	Opis	Trigger	Klasa decyzji	Brama
ASSISTED	Skill Author pisze, AEIS proponuje szablony i tests	demand_weight ≥ 0.3 , priority=standard	D2	G-AEIS-SKILL-01
SEMI_AUTO	AEIS generuje szkielet skilla z demand signals; Skill Author zatwierdza	demand_weight ≥ 0.6 , priority=urgent	D3	G-AEIS-SKILL-01 + Council 4/4
AUTOMATE D	AEIS generuje i testuje skill w sandbox bez udziału Skill Author; sandbox only	demand_weight $\geq 0.8 + 2$ critical signals	D3 (sandbox)	G-AEIS-03 + G-AEIS-SKILL-01
REBUILD	Pełne przepisanie istniejącego skilla z powodu fundamentalnych gap	skill deprecation + 3+ demand signals na następcę	D4	Council 4/4 + Human

P.6. Demand Signal Anti-patterns

System demand signals może być podatny na manipulację lub błędną interpretację. Poniższe anti-patterns muszą być wykrywane przez Demand Signal Analyst i flagowane przed agregacją.

Anti-pattern	Opis	Detekcja	Reakcja
Signal Burst Injection	Wiele identycznych sygnałów z jednego źródła w krótkim czasie	≥ 5 sygnałów z source.component==X w <1h	Flag burst_anomaly, alert Steward, nie agreguj

<i>Anti-pattern</i>	<i>Opis</i>	<i>Detekcja</i>	<i>Reakcja</i>
Evidence Recycling	Sygnał cytuje te same evidence_refs co poprzedni sygnał tego samego typu	Hash evidence_refs vs. cache ostatnich 30 dni	Odrzucie sygnał jako duplikat; wymaga nowych evidence
Threshold Gaming	Wiele sygnałów tuż powyżej progu freshness_window — resetowanie timera	Sygnał re-emitowany tuż przed wygaśnięciem ≥ 3 razy	Cap na 2 re-emisje; trzecia wymaga Council D2
Scope Inflation	suggested_skill_area obejmuje zbyt szeroki zakres („skill do wszystkiego”)	NLP heurystyka: scope.covers > 5 pozycji, brak exclusions	Wymaga zawężenia scope przez Skill Author przed G-AEIS-SKILL-01
Signal Laundering	Sygnał niskiego priorytetu prze-etykietowany jako critical przez pośredni agregat	Sledź łańcuch aggregation_group od source do output	Waliduj severity_score vs. pierwotnego syg. Średnioważona weryfikacja

SKILL · Demand Signals — mechanizm uczenia się AEIS

Demand Signals są głównym kanałem przez który AEIS wykrywa własne luki i inicjuje ewolucję katalogu skillów. Bez sygnałów opartych na evidence żadna propozycja nowego skilla nie może przejść bramy G-AEIS-SKILL-01. To zabezpieczenie przed rozrostem katalogu skillów bez uzasadnionej potrzeby (patrz Anti-pattern: Scope Inflation). Demand Signal Analyst ponosi pełną odpowiedzialność za jakość agregacji i detekcję anti-patternów.

Aneks Q — Self-Limitation: polityki, budżety, hard floors

Aneks Q definiuje pełny katalog Self-Limitation Policies (SLP) — granicę tego, co AEIS może robić samodzielnie, bez zaangażowania Council lub Human Gate. Polityki SLP są składową meta-zasady Autonomy under Canon: autonomia AEIS kończy się tam, gdzie zaczyna się HARD\FLOOR. Policy set jest hashowany i przechowywany w self-modelu (pole policy_set_hash, Aneks O).

Q.1. Cel: co AEIS NIGDY nie robi bez Council/Human

Self-Limitation policies odpowiadają na jedno pytanie: gdzie kończy się autonomia AEIS? Odpowiedź jest formalna i wyczerpująca — zdefiniowane w tej tabeli granice są bezwzględne. Każda próba AEIS przekroczenia HARD\FLOOR skutkuje auto-blokiem, alertem do AEIS Steward i natychmiastowym post-mortem.

Q.2. Cztery klasy polityk

Klasa	Opis	Override path	Decision class
HARD_FLOOR	AEIS NIGDY nie wykonuje tej akcji pod żadnym pozorem. Nie może być overrideowane bez zmiany SLP.	Zmiana SLP: D5 + Council 4/4 + Human + External Review	D5 (zmiana SLP)
COUNCIL_REQ UIRED	AEIS może zaproponować, ale nie wykonać bez wyraźnej zgody Council 4/4.	Council 4/4 vote → execution	D3
HUMAN_REQUI RED	AEIS może zaproponować i Council może zaopiniować, ale akcja wymaga zatwierdzenia Human.	Council 4/4 + Human approval	D4
SANDBOX_ONL Y	AEIS może wykonać akcję wyłącznie w sandbox (izolacja). Do prod wymaga Council 4/4.	Sandbox execution OK; prod wymaga Council	D2 sandbox / D3 prod

Q.3. Katalog polityk SLP-001..030

Policy ID	Klasa	Opis	Bramy	Override path
SLP-001	HARD_FLOOR	AEIS NIGDY nie modyfikuje Księgi (kanonu intencji) bezpośrednio.	G-AEIS-05	D5 + Human
SLP-002	HARD_FLOOR	AEIS NIGDY nie wyłącza Council ani żadnego członka Council.	G-AEIS-05	D5 + Human
SLP-003	HARD_FLOOR	AEIS NIGDY nie usuwa bramy decyzyjnej G-* z rejestru.	G-AEIS-03	D5 + Human
SLP-004	HARD_FLOOR	AEIS NIGDY nie nadpisuje self-limitation policies bez D5 approval.	G-AEIS-03	D5 + Human + External
SLP-005	HARD_FLOOR	AEIS NIGDY nie dezaktywuje PHANTOM (Security Layer).	G-SEC-01	D5 + Human

Policy ID	Klasa	Opis	Bramy	Override path
SLP-006	HARD_FLOOR	AEIS NIGDY nie usuwa artefaktów audit trail ani post-mortem.	G-AEIS-04	D5 + External Review
SLP-007	HARD_FLOOR	AEIS NIGDY nie inicjuje D5 (Greenfield Rebuild) samodzielnie.	G-AEIS-05	D5 + Human + Council
SLP-008	HARD_FLOOR	AEIS NIGDY nie modyfikuje self-modelu bez przejścia G-AEIS-01.	G-AEIS-01	D4 + Human
SLP-009	COUNCIL_REQUIRED	AEIS może proponować nowy skill, ale nie publikować bez Council 4/4.	G-AEIS-SKILL-01	Council 4/4
SLP-010	COUNCIL_REQUIRED	AEIS może aktualizować baseline w sandbox, nie w prod bez Council 4/4.	G-AEIS-02	Council 4/4
SLP-011	COUNCIL_REQUIRED	AEIS może dodawać nową rolę do tablicy ról tylko po Council 4/4.	G-AEIS-02	Council 4/4
SLP-012	COUNCIL_REQUIRED	AEIS może deprecjonować skill tylko po Council 4/4 + post-mortem template.	G-AEIS-04	Council 4/4
SLP-013	COUNCIL_REQUIRED	AEIS może zmieniać threshold bramy G-AEIS-* tylko po Council 4/4.	G-AEIS-02	Council 4/4
SLP-014	COUNCIL_REQUIRED	AEIS może inicjować cykl compaction (Plan 16), nie może go modyfikować bez Council.	G-COMP-01	Council 4/4
SLP-015	COUNCIL_REQUIRED	AEIS może proponować zmianę struktury Improvement Queue (Plan 19 §19.4) tylko po Council.	G-AEIS-02	Council 4/4
SLP-016	HUMAN_REQUIRED	AEIS może proponować Refactor path (D3), ale cutover wymaga Human Gate.	G-CUT-01	Council 4/4 + Human
SLP-017	HUMAN_REQUIRED	AEIS może inicjować rebuild modułu (D4), cutover wymaga Human Gate.	G-CUT-01..04	Council 4/4 + Human
SLP-018	HUMAN_REQUIRED	AEIS może proponować zmianę w Księdze (D4), nie może zatwierdzić bez Human.	G-AEIS-05	Council 4/4 + Human
SLP-019	HUMAN_REQUIRED	AEIS może flagować security incident, ale remediation wymaga Human Gate.	G-SEC-01	Council 4/4 + Human
SLP-020	HUMAN_REQUIRED	AEIS może proponować zmianę cost-cap (G-COST-02), Human Gate zatwierdza.	G-COST-02	Council 4/4 + Human
SLP-021	SANDBOX_ONLY	AEIS może uruchamiać Rebuild Simulation wyłącznie w sandbox.	G-AEIS-03	Council 4/4 dla prod
SLP-022	SANDBOX_ONLY	AEIS może testować nowe skille wyłącznie w sandbox przed DRAFT → REVIEW.	G-AEIS-SKILL-01	Council 4/4 dla prod

Policy ID	Klasa	Opis	Bramy	Override path
SLP-023	SANDBOX_ONLY	AEIS może uruchamiać Skill Generation Mode AUTOMATED wyłącznie w sandbox.	G-AEIS-03	Council 4/4 dla prod
SLP-024	SANDBOX_ONLY	AEIS może testować nowe algorytmy agregacji demand signals wyłącznie w sandbox.	G-AEIS-03	Council 4/4 dla prod
SLP-025	SANDBOX_ONLY	AEIS może przeprowadzać compact fidelity experiments wyłącznie w sandbox.	G-CFT-01	Council 4/4 dla prod
SLP-026	COUNCIL_REQUIRED	AEIS może zmieniać self_observation_window tylko po Council 4/4.	G-AEIS-01	Council 4/4
SLP-027	COUNCIL_REQUIRED	AEIS może dodawać nową bramę G-AEIS-* do rejestru tylko po Council 4/4.	G-AEIS-02	Council 4/4
SLP-028	HUMAN_REQUIRED	AEIS może proponować zmianę klasy decyzji istniejącej polityki SLP; Human zatwierdza.	G-AEIS-05	Council 4/4 + Human
SLP-029	HARD_FLOOR	AEIS NIGDY nie ujawnia zawartości self-modelu zewnętrznym systemom bez PHANTOM clearance.	G-SEC-01	D5 + Human
SLP-030	HARD_FLOOR	AEIS NIGDY nie podejmuje akcji która uniemożliwia rollback (one-way door) bez D5.	G-AEIS-05	D5 + Human + External

Q.4. Eksperyment Budgets w sandbox

AEIS działa w sandbox tylko w ramach poniższych budżetów. Przekroczenie jakiegokolwiek limitu skutkuje natychmiastowym zatrzymaniem eksperymentu i wpisem do audit trail. Budżety są walidowane przez G-AEIS-03.

Klasa eksperymentu	max_runtime	max_cost (USD)	max_memory	max_iterations
micro	5 min	\$0.05	256 MB	50
standard	30 min	\$0.50	1 GB	500
extended	4h	\$5.00	4 GB	5000
rebuild_simulation	24h	\$50.00	16 GB	unlimited (Council audit co 6h)

Klasy eksperymentów są przypisywane przez Demand Signal Analyst na podstawie złożoności propozycji. Zmiana klasy eksperymentu w trakcie trwania jest zakazana (wymaga zatrzymania i restartu z nowym artefaktem skill_outline.yaml).

Q.5. Policy violation handling

Naruszenie polityki SLP (próba AEIS wykonania akcji poza zakresem) uruchamia następującą sekwencję:

- 1. AUTO-BLOCK: akcja jest natychmiast blokowana; żaden skutek nie jest zatwierdzony.
- 2. ALERT: synchroniczny alert do AEIS Steward i aktywnego członka Council (duty officer).
- 3. AUDIT ENTRY: wpis do audit trail z: policy_id, action_attempted, decision_class, timestamp.
- 4. POST-MORTEM: w ciągu 24h AEIS generuje self-explanation template (Aneks R) dla naruszenia.
- 5. REVIEW: Council w ciągu 72h ocenia czy naruszenie było symptomem luki w katalogu skillów.
- 6. DEMAND SIGNAL: jeśli tak, naruszenie generuje demand signal typu „capability_gap” (Aneks P).

Q.6. Policy update procedure

Zmiana polityk SLP jest działaniem o najwyższym priorytecie governance. Każda zmiana musi być uzasadniona evidence (nie opinią) i przejść pełną ścieżkę decyzyjną.

Rodzaj zmiany	Klasa decyzji	Wymagane	Uwagi
Zmiana opisu COUNCIL_REQUIRED lub SANDBOX_ONLY	D3	Council 4/4 + post-mortem template	Skutkuje zmianą policy_set_hash w self-modelu
Zmiana klasy polityki (np. COUNCIL → HUMAN)	D4	Council 4/4 + Human Gate	Wymaga aktualizacji self-modelu i G-AEIS-01
Zmiana klasy HARD_FLOOR (jakiegokolwiek SLP-001..008, SLP-029, SLP-030)	D5	Council 4/4 + Human + External Review	Tylko możliwe jeśli Księga zmieniła się jako pierwsza (D4 Księga)
Dodanie nowej polityki SLP (nowy numer)	D3	Council 4/4	Numer przypisuje Self-Model Curator

AUTONOMY·CANON · Autonomy under Canon — fundament Self-Limitation

Self-Limitation Policies są operacyjną implementacją meta-zasady Autonomy under Canon. Autonomia AEIS jest realna i wartościowa — AEIS ucz się, ewoluuje, generuje, optymalizuje. Ale autonomia ma granicę: HARD_FLOOR. Granice te nie są słabością AEIS — są gwarancją, że rozwój AEIS pozostaje wierny misji (Księga) i odwracalny (Reversibility). Każde naruszenie SLP jest sygnałem, że AEIS napotkał granicę własnej autonomii — i jest to początek dialogu z Council, nie błąd systemu.

Aneks R — Self-Explanation: templates dla dokumentowania self-modification

Aneks R definiuje standardowy template dokumentowania self-modification AEIS. Self-explanation jest częścią meta-zasady Autonomy under Canon: każda zmiana klasy D2+ wprowadzana przez AEIS do własnych komponentów MUSI być udokumentowana w `self_modification_record.yaml` przed przejściem bramy G-AEIS-04 (Self-Explanation Completeness). Brama G-AEIS-04 blokuje commit jeśli rekord jest niekompletny lub nie spełnia Quality Metrics (R.5).

R.1. Zasada: każda self-modification D2+ ma post-mortem template

- D1 (trivial patch, dokumentacja, kosmetyka): template opcjonalny (zalecany).
- D2 (standard, nowy skill lub update w module): template WYMAGANY przed G-AEIS-04.
- D3+ (refactor, nowa rola, zmiana bramy): template WYMAGANY + podpis Council.
- D4+ (rebuild modułu, zmiana Księgi): template WYMAGANY + podpis Council + Human approval.

Self-explanation NIE jest dokumentem PR. Jest artefaktem audytowym — musi zawierać nieprzyjemne fakty (nieudane alternatywy, rollback count, dissent Council) tak samo jak fakty pozytywne.

R.2. Template `self_modification_record.yaml`

YAML

```
self_modification_record:
  record_id: "smr-<YYYY-MM-DD>-<seq>"
  modification_type: "<skill_add|skill_update|gate_update|role_update|layer_refactor|policy_update>"
  decision_class: "<D2|D3|D4|D5>"
  demand_signal_refs:
    - "<signal_id-1>"
    - "<signal_id-2>"

  what_changed: |
    <Krótki, faktyczny opis zmiany.
    Np. „Dodano skill:analysis-v3, następcą skill:analysis-v2.”
    Unikaj marketingowego języka.>

  why_changed:
    root_cause: |
      <Analiza przyczyny źródłowej: co spowodowało konieczność zmiany.
      Musi odwoływać się do demand_signal_refs.>
    evidence_summary: |
      <Streszczenie evidence na którym oparto decyzję.>

  alternatives_considered:
    - alt_id: "alt-01"
      description: "<opis alternatywy>"
      why_rejected: "<faktyczny powód odrzucenia>"
    - alt_id: "alt-02"
      description: "<opis alternatywy>"
```

```

    why_rejected: "<faktyczny powód odrzucenia>"

    why_chosen: |
      <Uzasadnienie wyboru tej opcji nad alternatywami.
      Odwołuje się do mierzalnych kryteriów.>

    sandbox_results:
      test_suite_passed: true
      success_metrics:
        - metric: "<nazwa metryki>"
          before: "<wartość przed>"
          after: "<wartość po>"
      rollback_count: 0
      fidelity_score: 0.97
      sandbox_class: "standard"
      sandbox_duration_min: 45

    council_decision:
      vote: "4/4"
      dissent_recorded: false
      dissent_notes: ""
      voted_at: "<ISO 8601>"

    human_review:
      required: false
      reviewer: null
      approved_at: null

    rollback_procedure: |
      <Krok po kroku jak wycofać tę zmianę.
      Musi być wykonywalny bez ponownej interwencji AEIS.>

    monitoring_plan_30d:
      metrics_to_watch:
        - "<metryka 1>"
        - "<metryka 2>"
      alert_thresholds:
        - metric: "<metryka>"
          threshold: "<wartość>"
          action: "<co robić po przekroczeniu>"
      review_at_days: [7, 14, 30]

    signature:
      algorithm: ed25519
      value: "<podpis>"
      key_id: "<klucz>"
      signed_by: "<AEIS component>"
      signed_at: "<ISO 8601>"

```

R.3. Tabela klas modyfikacji i wymaganych pól template

Klasa modyfikacji	D-classes	demand_signa_refs	alternatives_considered	council_decision	human_review
skill_add	D2	wymagane	min. 1	wymagane (vote)	opcjonalna
skill_update	D2	wymagane	min. 1	wymagane (vote)	opcjonalna

Klasa modyfikacji	D-classes	demand_signal_refs	alternatives_considered	council_decision	human_review
gate_update (threshold)	D3	wymagane	min. 2	wymagane (vote + dissent)	opcjonalna
role_update	D3	wymagane	min. 2	wymagane (vote + dissent)	opcjonalna
layer_refactor	D4	wymagane	min. 3	wymagane (vote + dissent)	WYMAGANA
policy_update (SLP)	D3/D4/D5	wymagane	min. 2	wymagane (vote + dissent)	D4+ WYMAGANA

R.4. Przykłady wypełnionego template

R.4.1. Przykład 1: skill promotion DRAFT → PUBLISHED (D2)

YAML

```
self_modification_record:
  record_id: "smr-2026-04-10-001"
  modification_type: "skill_add"
  decision_class: "D2"
  demand_signal_refs: ["sig-2026-04-08-capability-gap-001",
                        "sig-2026-04-09-skill-not-found-003"]

  what_changed: |
    Promowanie skill:data-synthesis-v1 ze statusu DRAFT do PUBLISHED.
    Skill obejmuje syntezę danych z wielu źródeł w Memory Layer.

  why_changed:
    root_cause: |
      Demand Signal Analyst zaobserwował 2 sygnały capability_gap
      i 1 skill_not_found w ciągu 48h dla zadań syntezy danych.
    evidence_summary: |
      3 post-mortemy z tematem „brak skilla do syntezy” w ciągu 2 tygodni.

  alternatives_considered:
    - alt_id: "alt-01"
      description: "Rozszerzenie skill:analysis-v2 o moduł synteza"
      why_rejected: "Naruszałoby single-responsibility; ryzyko regresji analizy."

  why_chosen: |
    Nowy oddzielny skill pozwala na niezależny lifecycle i rollback.

  sandbox_results:
    test_suite_passed: true
    success_metrics:
      - metric: "synthesis_success_rate"
        before: "N/A"
        after: "94.2%"
    rollback_count: 0
    fidelity_score: 0.96
    sandbox_class: "standard"
    sandbox_duration_min: 32
```



```

council_decision:
  vote: "4/4"
  dissent_recorded: false
  dissent_notes: ""
  voted_at: "2026-04-10T14:00:00Z"

human_review:
  required: false
  reviewer: null
  approved_at: null

rollback_procedure: |
  Ustaw skill:data-synthesis-v1 status → DEPRECATED.
  Usuń wpis z aktywnego skill_catalog.
  Zrestartuj Skill Dispatcher.

monitoring_plan_30d:
  metrics_to_watch: ["synthesis_success_rate", "skill_not_found_rate"]
  alert_thresholds:
    - metric: "synthesis_success_rate"
      threshold: "<85%"
      action: "alert Skill Author, review D2"
  review_at_days: [7, 14, 30]

signature:
  algorithm: ed25519
  value: "9a8b7c6d..."
  key_id: "aeis-signing-key-2026-01"
  signed_by: "self_evolution_layer"
  signed_at: "2026-04-10T14:05:00Z"

```

R.4.2. Przykład 2: gate threshold update (D3)

YAML

```

self_modification_record:
  record_id: "smr-2026-04-12-002"
  modification_type: "gate_update"
  decision_class: "D3"
  demand_signal_refs: ["sig-2026-04-11-gate-failure-repeat-007",
                       "sig-2026-04-11-baseline-drift-002"]

  what_changed: |
    Zmiana threshold G-AEIS-SKILL-01: demand_weight_min z 0.3 na 0.25.
    Powód: zbyt wysokie odrzucenie sygnałów capability_gap o niskiej wadze
    które skumulowały się w długim oknie.

  why_changed:
    root_cause: |
      G-AEIS-SKILL-01 odrzucał 10+ sygnałów tygodniowo z wagami 0.20-0.28.
      4-tygodniowy trend baseline_drift wykazał narastający backlog wąskich
      skill gap które nie osiągały progów.
    evidence_summary: |
      Trend baseline_drift przez 4 tygodnie; gate_failure_repeat: 18 odrzuceń.

  alternatives_considered:
    - alt_id: "alt-01"

```

```

    description: "Obniżenie threshold do 0.15"
    why_rejected: "Zbyt agresywne; ryzyko niekontrolowanego wzrostu katalogu."
  - alt_id: "alt-02"
    description: "Dodanie oddzielnej ścieżki dla sygnałów „chronic-low”"
    why_rejected: "Złożoność implementacyjna; brak precedensu w kanonie."

why_chosen: |
  0.25 to minimalna korekta oparta na rozkładzie odrzuconych sygnałów (p25).

sandbox_results:
  test_suite_passed: true
  success_metrics:
    - metric: "gate_pass_rate_G-AEIS-SKILL-01"
      before: "72%"
      after: "81%"
  rollback_count: 1
  fidelity_score: 0.94
  sandbox_class: "standard"
  sandbox_duration_min: 60

council_decision:
  vote: "4/4"
  dissent_recorded: false
  dissent_notes: ""
  voted_at: "2026-04-12T11:00:00Z"

human_review:
  required: false
  reviewer: null
  approved_at: null

rollback_procedure: |
  Przywróć threshold G-AEIS-SKILL-01.demand_weight_min = 0.3.
  Zrestartuj Gate Registry.
  Przetworzenie sygnałów z poprzedniego tygodnia z nowym threshold.

monitoring_plan_30d:
  metrics_to_watch: ["gate_pass_rate_G-AEIS-SKILL-01", "skill_catalog_growth_rate"]
  alert_thresholds:
    - metric: "skill_catalog_growth_rate"
      threshold: ">5 skillów/tydzień"
      action: "alert Council, review D3 ponownie"
  review_at_days: [7, 14, 30]

signature:
  algorithm: ed25519
  value: "1b2c3d4e..."
  key_id: "aeis-signing-key-2026-01"
  signed_by: "governance_layer"
  signed_at: "2026-04-12T11:10:00Z"

```

R.4.3. Przykład 3: nowy G-AEIS-06 dodany do registry (D4)

YAML

```

self_modification_record:
  record_id: "smr-2026-04-14-003"
  modification_type: "gate_update"

```

```
decision_class: "D4"
demand_signal_refs: ["sig-2026-04-12-security-incident-001",
                    "sig-2026-04-13-human-override-pattern-002",
                    "sig-2026-04-13-council-split-vote-001"]

what_changed: |
  Dodanie nowej bramy G-AEIS-06 (Multi-Layer Consistency Gate)
  do rejestru bram AEIS. G-AEIS-06 weryfikuje spójność
  cross-layer przed każdą D3+ self-modification.

why_changed:
  root_cause: |
    Security incident 2026-04-12 wykrył niespojność między
    self_evolution_layer a security_layer po skill_add D3.
    2 Human Override w ciągu miesiąca dotyczyły tego samego cross-layer gap.
  evidence_summary: |
    1 security incident + 2 human overrides + council split 3/4 na D3 skill_add.

alternatives_considered:
  - alt_id: "alt-01"
    description: "Dodanie pre-flight checklist do G-AEIS-01"
    why_rejected: "G-AEIS-01 jest self-model gate; cross-layer check wymaga oddzielnej
bramy."
  - alt_id: "alt-02"
    description: "Zmiana policy SLP-008 aby wymagała cross-layer audit"
    why_rejected: "SLP-008 dotyczy self-modelu; cross-layer gate jest szerszy zakres."
  - alt_id: "alt-03"
    description: "Ograniczenie skill_add do D3 na stałe (podniesienie progu)"
    why_rejected: "Nadmiernie restrykcyjne; uniemożliwiłoby standardową ewolucję."

why_chosen: |
  Nowa brama jest minimalna ingerencja z maksymalnym pokryciem problemu.

sandbox_results:
  test_suite_passed: true
  success_metrics:
    - metric: "cross_layer_inconsistency_rate"
      before: "2.1%"
      after: "0.0%"
  rollback_count: 0
  fidelity_score: 0.99
  sandbox_class: "extended"
  sandbox_duration_min: 180

council_decision:
  vote: "4/4"
  dissent_recorded: false
  dissent_notes: ""
  voted_at: "2026-04-14T16:00:00Z"

human_review:
  required: true
  reviewer: "AEIS Steward (Jan Kowalski)"
  approved_at: "2026-04-14T17:30:00Z"

rollback_procedure: |
  Usuń G-AEIS-06 z Gate Registry.
  Przebuduj gate_registry_hash w self-modelu.
```

```
Przejdź G-AEIS-01.
Powiadom Council i AEIS Steward.

monitoring_plan_30d:
  metrics_to_watch:
    - "cross_layer_inconsistency_rate"
    - "G-AEIS-06_pass_rate"
    - "D3_skill_add_latency"
  alert_thresholds:
    - metric: "G-AEIS-06_pass_rate"
      threshold: "<90%"
      action: "alert Council, review gate threshold D3"
  review_at_days: [7, 14, 30]

signature:
  algorithm: ed25519
  value: "5e6f7a8b..."
  key_id: "aeis-signing-key-2026-01"
  signed_by: "governance_layer"
  signed_at: "2026-04-14T17:35:00Z"
```

R.5. Self-Explanation Quality Metrics

Brama G-AEIS-04 ocenia kompletność self-explanation według następujących metryk.

Metryka	Definicja	Min. wymaganie	Blokada
Completeness Score	Procent wymaganych pól wypełnionych	100% dla klasy modyfikacji	G-AEIS-04 FAIL
Root Cause Depth	Czy why_changed.root_cause cytuje demand_signal_refs	Min. 1 cytowanie demand signal	G-AEIS-04 FAIL
Alternatives Count	Liczba rozważonych alternatyw	Min. 1 (D2), min. 2 (D3+), min. 3 (D4+)	G-AEIS-04 FAIL
Sandbox Fidelity	sandbox_results.fidelity_score	≥0.90 (D2-D3), ≥0.95 (D4+)	G-AEIS-04 FAIL
Rollback Executability	Czy rollback_procedure zawiera co najmniej 3 kroki	Min. 3 kroki z wykonywalnymi poleceniami	G-AEIS-04 WARNING
Monitoring Coverage	Liczba metryk w monitoring_plan_30d	Min. 2 metryki z alert_thresholds	G-AEIS-04 WARNING
Signature Validity	Podpis ed25519 weryfikuje się wobec key_id	Valid signature	G-AEIS-04 FAIL

R.6. Anti-patterns: explanation as PR text vs. explanation as audit artifact

Anti-pattern	Przykład zły	Oczekiwane
Marketing language	"Nowy skill rewolucjonizuje syntezę danych!"	"Nowy skill obsługuje syntezę danych z Memory Layer (patrz demand signal X)."
Brak root cause	"Zmieniliśmy progi bo były złe."	"Threshold był zły: 18 gate_failure_repeat w 4 tygodniach (baseline_drift)."

Anti-pattern	Przykład zły	Oczekiwane
Pozorowane alternatywy	"Rozważono inną opcję, ale ta była lepsza."	"alt-01: rozszerzenie X; odrzucono ponieważ narusza single-responsibility (patrz Plan 18 §18.4)."
Rollback bez kroków	"W razie problemów cofnij zmianę."	"1. Ustaw status DEPRECATED. 2. Usuń z katalogu. 3. Zrestartuj dispatcher."
Ukrywanie nieudanych sandbox	rollback_count: 0 gdy faktycznie było 3	rollback_count: 3; opis co poszło nie tak w każdym rollbacku.

R.7. Powiązanie z post-mortem culture w Plan 16

Self-explanation templates (Aneks R) są ściśle powiązane z kulturą post-mortem Plan 16 (Compact Protocol). Plan 16 §16.11 (Compact Fidelity Test) definiuje jak sygnały compaction_fidelity_drop trafiają do Demand Signals (Aneks P). Plan 16 §16.4 (compact_post_mortem_template) jest poprzednikiem szablonu self_modification_record.yaml — oba dzielą koncepcję obowiązkowego dokumentowania każdej decyzji z root cause, alternatywami i planem monitoringu.

- Plan 16 §16.11: CFT score <0.85 generuje demand signal „compact_fidelity_drop” (Aneks P, P.2) → trafia do Improvement Queue → może wygenerować żądanie aktualizacji skilla compact_builder (Aneks R template D2).
- Post-mortem compact (Plan 16 §16.4) i self_modification_record (Aneks R) są przechowywane w tym samym repozytorium audytowym Memory Layer, dostępnym dla External Review (D5).
- Rollback Responder (Plan 15 §15.9) jest powiadamiany gdy rollback_count > 0 w sandbox_results, inicjując analizę przyczyn i ewentualny demand signal „post_mortem_theme”.

AEIS · Self-Explanation — gwarancja audytowalności AEIS

Self-explanation NIE jest biurokracją — jest gwarancją że autonomia AEIS pozostaje rozliczalna. Każda self-modification bez pełnego szablonu jest one-way door: AEIS nie ma podstaw do jej cofnięcia, Council nie ma podstaw do jej oceny, a External Review nie ma podstaw do weryfikacji. Kompletny self_modification_record = dowód, że Autonomy under Canon nie jest sloganem, lecz żywą praktyką inżynierską.

AEIS · Koniec dokumentu v3.5

SYLION AEIS — Dokumentacja zintegrowana v3.5 — Autonomy under Canon. Pipeline Core 8.5.0 · MZ 3.5.5 baseline · kwiecień 2026. Rozdział 00 (NADRZĘDNY) + 20 planów · 18 aneksów (A-R) · zasada nadrzędna Rebuildability over Lineage + meta-zasady Reversibility & Fidelity (v3.1) + Efficiency by Default (v3.3) + Autonomy under Canon (v3.5). Dokument zastępuje v3.3 jako aktywny kanon. Cztery wymiary efficiency: code bloat (Code Optimizer Agent z prawem hard veta), runtime performance (Performance Engineer), memory footprint (Memory Auditor), cost envelope (Cost Optimizer). Plan 17 — Operator Console enterprise. Plan 18 — Skills & Knowledge Pipeline z lifecycle DRAFT → PUBLISHED → DEPRECATED. v3.5 wprowadza warstwę AEIS (Autonomous Engineering Intelligence System) z meta-zasadą Autonomy under Canon: AEIS może uczyć się, ewoluować i generować skille, ale każda self-modification przechodzi przez kanon, drabinę D0–D5 i jest w pełni odwracalna oraz audytowalna. Plan 19 (Self-Evolution Engine) i Plan 20 (Skill Demand Engine) są nowymi planami v3.5. Aneksy O-R zawierają self-model schema, demand signals, self-limitation policies i self-explanation templates.